

IvorySQL

Version v4.4, 2025-05-26

欢迎	1
发行说明	2
关于IvorySQL	4
IvorySQL入门	7
快速开始	7
日常监控	9
日常维护	48
IvorySQL高级	71
安装指南	71
集群搭建	76
开发者指南	81
运维管理指南	159
迁移指南	194
IvorySQL生态	211
PostGIS	211
pgvector	213
IvorySQL架构设计	217
查询处理	217
兼容框架	219
Oracle兼容功能列表	222
1、框架设计	222
2、GUC框架	222
3、大小写转换	225
4、双模式设计	232
5、兼容Oracle like	233
6、兼容Oracle匿名块	234
7、兼容Oracle函数与存储过程	235
8、内置数据类型与内置函数	237
9、新增Oracle兼容模式的端口与IP	257
10、XML函数	257
11、兼容Oracle sequence	262
12、包	263
13、不可见列	267
社区贡献指南	269
工具参考	290
FAQ	397

欢迎

IvorySQL 是唯一一款基于 PostgreSQL 研发的兼容 Oracle 的数据库。

开始

从 [Github](#) 下载源码开始。

发布

请前往 [IvorySQL](#) 发布页面。

关于 IvorySQL

IvorySQL 项目是瀚高软件提出的一个开源项目，旨在将 Oracle 兼容性功能添加到流行的 PostgreSQL 数据库中。

IvorySQL 开源并且可以免费使用，如果您有任何建议请联系 support@ivorysql.org

文档下载

[IvorySQL v4.4 pdf 文档](#)

发行说明

版本概览

[发行日期：2025年03月10日]

IvorySQL 4.4，基于PostgreSQL 17.4，并修复了多个问题。有关更新的完整列表，请访问我们的 [文档网站](#)。

增强功能及问题修复

- PostgreSQL 17.3

1. 增强 PQescapeString 及相关函数对无效编码输入字符串的防护能力。
2. 恢复连接请求中数据库名和用户名的自动截断功能。
3. 将并行工作进程排除在连接权限检查及连接数限制之外。
4. 从 LWLock 等待事件名称中移除 "Lock" 后缀。
5. 修复窗口聚合函数中可能重用陈旧结果的问题。
6. 防止在关系表截断操作期间触发检查点（Checkpoint）。

更多细节, 请参阅 [PostgreSQL发布说明](#).

- PostgreSQL 17.4

1. 改进 libpq 引用函数（quoting functions）的行为。
2. 修复 pg_createsubscriber 中的轻微内存泄漏问题。
3. 修复 meson 构建系统，使其能正确检测系统头文件 bsd_auth.h 的可用性。

更多细节, 请参阅 [PostgreSQL发布说明](#).

- IvorySQL 4.4

1. ARM64 全平台打包支持：特性 [#729](#)

为 ARM 架构提供多平台介质包，支持国内外主流操作系统，包括 Red Hat、Debian、麒麟、UOS、凝思等。

2. X86 全平台打包支持：特性 [#735](#)

为 X86 架构提供多平台介质包，支持国内外主流操作系统，包括 Red Hat、Debian、麒麟、UOS、凝思等。

3. 新增内核架构设计文档框架：特性 [#739](#)

新增内核架构设计系列文档，首篇聚焦 Oracle 兼容性。该系列深入解析各模块的框架、设计与实现原理，帮助用户深度理解 Oracle 兼容框架。

4. Oracle 兼容框架之双解析器架构原理分析：特性 [#740](#)

详细介绍 Oracle 兼容框架中双解析器架构的设计与实现原理。

5. Oracle 兼容框架之 initdb 过程原理分析：特性 [#741](#)

详细介绍 Oracle 兼容框架中 initdb 过程的设计与实现原理。

6. IvorySQL 官网新增聊天室功能：特性 [#133](#)

在官网集成聊天室功能，实现与 IvorySQL 相关的智能交互与问答交流。

源代码

IvorySQL主要包含2个代码仓库:

- IvorySQL数据库源码: <https://github.com/IvorySQL/IvorySQL>
- IvorySQL官方网站: <https://github.com/IvorySQL/Ivory-www>

贡献人员

以下个人（按姓氏排序）作为补丁作者、提交者、审查者、测试者或问题报告者为此版本做出了贡献。

- Alex Guo
- Cary Huang
- Denis Lussier
- Flyingbeecd
- Grant Zhou
- Imran Zaheer
- 高雪玉
- 冯明冉
- 冯若航
- 矫顺田
- 梁翔宇
- 吕新杰
- 牛世继
- 潘振浩
- 王康
- 王丽
- 王守波
- 严少安
- 赵法威
- 邹仁利

关于IvorySQL

IvorySQL简介

概述

IvorySQL是一款以PostgreSQL为基础进行开发，并且兼容Oracle的开源数据库。

IvorySQL社区始终承诺与PostgreSQL数据库保持100%兼容，并且可以直接替换最新版本的PostgreSQL。

IvorySQL增加了一个名为 **ivorysql.compatible_mode** 的GUC参数用以控制IvorySQL的兼容模式，该参数有 **oracle** 和 **pg** 两种值。在初始化数据目录的时候，通过指定 **-m** 参数来指定数据目录的兼容模式，**-m pg** 则数据目录为PostgreSQL模式，该模式下 **ivorysql.compatible_mode** 参数将会失效，**-m oracle** 或者不指定 **-m** 参数则数据目录为兼容Oracle模式，该模式下 **ivorysql.compatible_mode** 参数初始值为 **oracle** 并且不支持部分PostgreSQL的语法，通过 **set ivorysql.compatible_mode to pg** 就可以使得数据库100%支持PostgreSQL的语法及功能。

IvorySQL的亮点之一是PL/iSQL过程语言，它支持Oracle的PL/SQL语法。同时，IvorySQL通过增加与内核绑定的插件 **ivorysql_ora** 来实现兼容Oracle的功能，目前实现的功能包括内置函数、数据类型、系统视图、merge以及GUC参数的增加，未来将会继续以绑定内核的插件的形式来实现新的兼容功能。

IvorySQL项目是在Apache 2.0许可证下发布的，社区鼓励且欢迎所有类型的贡献和参与。

产品目标和范围

我们致力于遵守 [开源方式](#) 的原则，我们坚信建立一个健康和包容的社区。我们坚持认为，好的想法可以来自任何地方。虽然IvorySQL目前的版本主要关注Oracle兼容性功能，但未来的路线图和功能集将由社区以开源方式确定。

核心特性

IvorySQL基于PostgreSQL数据库开发，与Oracle数据库兼容，具有强大的兼容性。适用于PostgreSQL数据库和Oracle数据库场景。

竞争优势

- 核心开源：IvorySQL的核心代码包括兼容功能全部在开源协议下公开，没有厂商的限制。并应用于瀚高股份数据库公司实例，且拥有一个活跃的开发者社区。
- 兼容Oracle：可以将Oracle数据库迁移到IvorySQL。
- 可定制化：只需下载代码，并按照你的想法自定义。
- 简单易用：对系统管理员来说，IvorySQL大幅降低了管理和维护的代价。对开发者来说，IvorySQL提供了简单的接口、极简的解决方案和与第三方工具的无缝集成。对数据分析专家来说，IvorySQL提供了便捷的数据访问能力。
- 瀚高支持：由领先的PostgreSQL数据库提供商瀚高股份提供支持。

技术生态

IvorySQL基于PostgreSQL，具有完整的SQL、坚如磐石的可靠性和庞大的生态系统。

核心应用场景

Ivory数据库的主要应用场景：

- 企业数据库

如 ERP、交易系统、财务系统涉及资金、客户等信息，数据不能丢失且业务逻辑复杂，选择 IvorySQL 作为数据底层存储，一是可以帮助您在数据一致性前提下提供高可用性，二是可以用简单的编程实现复杂的业务逻辑。

- 含 LBS 的应用

大型游戏、O2O 等应用需要支持世界地图、附近的商家，两个点的距离等能力，PostGIS 增加了对地理对象的支持，允许您以 SQL 运行位置查询，而不需要复杂的编码，帮助您更轻松理顺逻辑，更便捷的实现 LBS，提高用户粘性。

- 数据仓库和大数据

IvorySQL 更多数据类型和强大的计算能力，能够帮助您更简单搭建数据库仓库或大数据分析平台，为企业运营加分。

- 建站或 App

IvorySQL 良好的性能和强大的功能，可以有效的提高网站性能，降低开发难度。

- 数据库迁移

如果需要将Oracle数据库迁移到PostgreSQL数据库，可以直接使用IvorySQL数据库进行迁移。

主要、基本功能

IvorySQL是一个功能强大的开源对象关系数据库管理系统(ORDBMS)。用于安全地存储数据，支持最佳做法，并允许在处理请求时检索它们。除此之外，还兼容了Oracle的语法，适用于使用Oracle的场景。

与Oracle的兼容性

- [ivorysql框架设计](#)
- [GUC框架](#)
- [大小写转换](#)
- [双模式设计](#)
- [兼容Oracle like](#)
- [兼容Oracle匿名块](#)
- [兼容Oracle函数与存储过程](#)
- [内置数据类型与内置函数](#)
- [新增Oracle兼容模式的端口与IP](#)
- [XML函数](#)
- [兼容Oracle sequence](#)

- 包
- 不可见列

IvorySQL入门

快速开始

环境要求

- 硬件要求

配置参数	最低配置	推荐配置
CPU	4核	16核
内存	4GB	64GB
存储	800MB，机械硬盘	5GB以上，SSD或NvMe
网络	千兆网络	万兆网络

- 软件要求

IvorySQL数据库目前支持的操作系统包括但不限于CentOS 8.X、CentOS Stream 9以及Ubuntu系统。

快速安装

快速开始示例所使用的操作系统为CentOS Stream 9。

从yum源安装IvorySQL数据库

- 安装前准备

安装前请先创建一个用户，并赋予其root权限，安装和使用均以该用户执行。这里以ivorysql用户为例。
[如何创建sudo用户](#)

- 下载安装

创建或编辑IvorySQL yum源配置文件/etc/yum.repos.d/ivorysql.repo

```
vim /etc/yum.repos.d/ivorysql.repo
[ivorysql4]
name=IvorySQL Server 4 $releasever - $basearch
baseurl=https://yum.highgo.com/dists/ivorysql-rpms/4/redhat/rhel-$releasever-$basearch
enabled=1
gpgcheck=0
```

保存退出后，安装IvorySQL4

```
$ sudo dnf install -y IvorySQL-4.4
```

正确安装后，数据库将被安装在/opt/IvorySQL-4.4/路径下的IvorySQL-version(如:IvorySQL-4.4)文件夹内

执行以下命令为ivorysql用户赋权：

```
$ sudo chown -R ivorysql:ivorysql /opt/IvorySQL-4.4
```

- 配置环境变量

将以下配置写入 ~/.bash_profile 文件并使用 source 命令该文件使环境变量生效：

```
PATH=/opt/IvorySQL-4.4/bin:$PATH
export PATH
PGDATA=/opt/IvorySQL-4.4/data
export PGDATA
```

```
$ source ~/.bash_profile
```

- 数据库初始化

```
$ initdb -D /opt/IvorySQL-4.4/data
```

其中 -D 参数用来指定数据库的数据目录。更多参数使用方法，请使用 `initdb --help` 命令获取。

- 启动数据库服务

```
$ pg_ctl -D /opt/IvorySQL-4.4/data -l ivory.log start
```

其中 -D 参数用来指定数据库的数据目录，如果 [\[v4-4::3::配置环境变量\]](#) 配置了 PGDATA，则该参数可以省略。-l 参数用来指定日志目录。更多参数使用方法，请使用 `pg_ctl --help` 命令获取。

查看确认数据库启动成功：

```
$ ps -ef | grep postgres
ivorysql 3214      1  0 20:35 ?        00:00:00 /opt/IvorySQL-4.4/bin/postgres -D
/opt/IvorySQL-4.4/data
ivorysql  3215    3214  0 20:35 ?        00:00:00 postgres: checkpointer
ivorysql  3216    3214  0 20:35 ?        00:00:00 postgres: background writer
ivorysql  3218    3214  0 20:35 ?        00:00:00 postgres: walwriter
ivorysql  3219    3214  0 20:35 ?        00:00:00 postgres: autovacuum launcher
ivorysql  3220    3214  0 20:35 ?        00:00:00 postgres: logical replication launcher
ivorysql  3238   1551  0 20:35 pts/0    00:00:00 grep --color=auto postgres
```

docker方式运行

- 从 Docker Hub 上获取 IvorySQL 镜像

```
$ docker pull ivorysql/ivorysql:4.4-ubi8
```

- 运行IvorySQL

```
$ docker run --name ivorysql -p 5434:5432 -e IVORYSQL_PASSWORD=your_password -d  
ivorysql/ivorysql:4.4-ubi8
```

- 查看IvorySQL容器运行是否成功

```
$ docker ps | grep ivorysql
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
6faa2d0ed705	ivorysql:4.4-ubi8	"docker-entrypoint.s..."	50 seconds ago	Up 49 seconds
	5866/tcp, 0.0.0.0:5434->5432/tcp	ivorysql		

数据库连接

psql连接数据库：

```
$ psql -d <database>  
psql (17.4)  
Type "help" for help.  
  
ivorysql=#
```

其中-

d参数用来指定想要连接到的数据库名称。IvorySQL默认使用ivorysql数据库，但较低版本的IvorySQL首次使用时需用户先连接postgres数据库，然后自己创建ivorysql数据库。较高版本的IvorySQL则已为用户创建好ivorysql数据库，可以直接连接。

更多参数使用方法，请使用psql --help命令获取。



Docker运行IvorySQL时，需要添加额外参数，如 psql -d ivorysql -U ivorysql -h 127.0.0.1 -p 5434

现在可以开始使用IvorySQL啦！就是这么简单！

想要获得更多安装方式，请参考[安装指南](#)

日常监控

监控数据活动

在大部分 Unix

平台上，IvorySQL会修改由`ps`报告的命令标题，这样个体服务器进程可以被标识。一个显示样例

```
$ ps auxww | grep ^postgres
postgres 15551 0.0 0.1 57536 7132 pts/0 S 18:02 0:00 postgres -i
postgres 15554 0.0 0.0 57536 1184 ? Ss 18:02 0:00 postgres: background
writer
postgres 15555 0.0 0.0 57536 916 ? Ss 18:02 0:00 postgres:
checkpointer
postgres 15556 0.0 0.0 57536 916 ? Ss 18:02 0:00 postgres: walwriter
postgres 15557 0.0 0.0 58504 2244 ? Ss 18:02 0:00 postgres: autovacuum
launcher
postgres 15558 0.0 0.0 17512 1068 ? Ss 18:02 0:00 postgres: stats
collector
postgres 15582 0.0 0.0 58772 3080 ? Ss 18:04 0:00 postgres: joe runbug
127.0.0.1 idle
postgres 15606 0.0 0.0 58772 3052 ? Ss 18:07 0:00 postgres: tgl
regression [local] SELECT waiting
postgres 15610 0.0 0.0 58772 3056 ? Ss 18:07 0:00 postgres: tgl
regression [local] idle in transaction
```

（`ps`的调用方式随不同的平台而变，但是显示的细节都差不多。这个例子来自于一个最近的 Linux 系统）。列在这里的第一个进程是主服务器进程。为它显示的命令参数是当它被启动时使用的那些。接下来的五个进程是由主进程自动启动的后台工作者进程（如果你已经设置系统为不启动统计收集器，“统计收集器”进程将不会出现；同样“自动清理发动”进程也可以被禁用）。剩余的每一个进程都是一个处理一个客户端连接的服务器进程。每个这种进程都会把它的命令行显示设置为这种形式

postgres: user database host activity

在该客户端连接的生命期中，用户、数据库以及（客户端）主机项保持不变，但是活动指示器会改变。活动可以是`闲置`（即等待一个客户端命令）、**在事务中闲置**（在一个`BEGIN`块里等待客户端）或者一个命令类型名，例如`SELECT`。还有，如果服务器进程正在等待一个其它会话持有的锁，`等待中`会被追加到上述信息中。在上面的例子中，我们可以推断：进程 15606 正在等待进程 15610 完成其事务并且因此释放一些锁（进程 15610 必定是阻塞者，因为没有其他活动会话。在更复杂的情况中，可能需要查看pg_locks系统视图来决定谁阻塞了谁）。

如果配置了cluster_name，则集簇的名字 也将会显示在`ps`的输出中：

```
$ psql -c 'SHOW cluster_name'
cluster_name
-----
server1
(1 row)

$ ps aux|grep server1
postgres 27093 0.0 0.0 30096 2752 ? Ss 11:34 0:00 postgres: server1:
```


background writer

...

如果你已经关闭了update_process_title，那么活动指示器将不会被更新，进程标题仅在新进程被启动的时候设置一次。在某些平台上这样做可以为每个命令节省可观的开销，但在其它平台上却不明显。

提示

Solaris需要特别的处理。你必需使用`/usr/ucb/ps`而不是`/bin/ps`。
你还必需使用两个`w`标志，而不是一个。另外，你对`postgres`命令的最初调用必须用一个比服务器进程提供的短的`ps`状态显示。如果你没有满足全部三个要求，每个服务器进程的`ps`输出将是原始的`postgres`命令行。command line。

统计收集器

IvorySQL的统计收集器是一个支持收集和报告服务器活动信息的子系统。
目前，这个收集器可以对表和索引的访问计数，计数可以按磁盘块和个体行来进行。它还跟踪每个表中的总行数、每个表的清理和分析动作的信息。它也统计调用用户定义函数的次数以及在每次调用中花费的总时间。

IvorySQL也支持报告有关系统正在干什么的动态信息，例如当前正在被其他服务器进程执行的命令以及系统中存在哪些其他连接。
这个功能是独立于收集器进程存在的。

统计收集配置

因为统计收集给查询执行增加了一些负荷，系统可以被配置为收集或不收集信息。这由配置参数控制，它们通常在`postgresql.conf`中设置。

参数track_activities允许监控当前被任意服务器进程执行的命令。

参数track_counts控制是否收集关于表和索引访问的统计信息。

参数track_functions启用对用户定义函数使用的跟踪。

参数track_io_timing启用对块读写次数的监控。

通常这些参数被设置在`postgresql.conf`中，这样它们会应用于所有服务器进程，但是可以在单个会话中使用SET命令打开或关闭它们（为了阻止普通用户对管理员隐藏他们的活动，只有超级用户被允许使用`SET`来改变这些参数）。

统计收集器通过临时文件将收集到的信息传送给其他IvorySQL进程。这些文件被存储在名字由stats_temp_directory参数指定的目录中，默认是`pg\_stat\_tmp`。为了得到更好的性能，`stats\_temp\_directory`可以被指向一个基于RAM的文件系统来降低物理I/O需求。当服务器被干净地关闭时，一份统计数据的永久拷贝被存储在`pg\_stat`子目录中，这样在服务器重启后统计信息能被保持。当在服务器启动时执行恢复时（例如立即关闭、服务器崩溃以及时间点恢复之后），所有统计计数器会被重置。

查看统计信息

表1中列出了一些预定义视图可以用来显示系统的当前状态。表2中列出了另一些视图可以显示统计收集的结果。你也可以使用底层统计函数来建立自定义的视图。

在使用统计信息监控收集到的数据时，你必须了解这些信息并非是实时更新的。每个独立的服务器进程只在进入闲置状态之前才向收集器传送新的统计计数；因此正在进行的查询或事务并不影响显示出来的总数。同样，收集器本身也最多每`PGSTAT\_STAT\_INTERVAL`毫秒（缺省为500ms，除非在编译服务器的时候修改过）发送一次新的报告。因此显示的信息总是落后于实际活动。但是由`track\_activities`收集的当前查询信息总是最新的。

另一个重点是当一个服务器进程被要求显示任何这些统计信息时，它首先取得收集器进程最近发出的报告并且接着为所有统计视图和函数使用这个快照，直到它的当前事务的结尾。因此只要你继续当前事务，统计数据将会一直显示静态信息。相似地，当任何关于所有会话的当前查询的信息在一个事务中第一次被请求时，这样的信息将被收集。并且在整个事务期间将显示相同的信息。这是一种特性而非缺陷，因为它允许你在该统计信息上执行多个查询并且关联结果而不用担心那些数字会在你不知情的情况下改变。但是如果你希望用每个查询都看到新结果，要确保在任何事务块之外做那些查询。或者，你可以调用`pg\_stat\_clear\_snapshot`(), 那将丢弃当前事务的统计快照（如果有）。下一次对统计性信息的使用将导致获取一个新的快照。

一个事务也可以在视图`pg\_stat\_xact\_all\_tables`、**pg_stat_xact_sys_tables**、`pg\_stat\_xact\_user\_tables`和`pg\_stat\_xact\_user\_functions`中看到它自己的统计信息（还没有被传送给收集器）。这些数字并不像上面所述的那样行动，相反它们在事务期间持续被更新。

表 1中显示的动态统计视图其中的一些信息是有安全限制的。

普通用户只能看到关于他们自己的会话的所有信息（属于他们是成员的角色会话）。

在关于其他会话的行中，许多列将为空。

但是，请注意，一个会话的存在和它的一般属性，例如会话用户和数据库，对所有用户都是可见的。

超级用户和内置角色`pg\_read\_all\_stats`的成员可以看到所有会话的所有信息。

表1.动态统计视图

视图名称	描述
pg_stat_activity	每个服务器进程一行，显示与那个进程的当前活动相关的信息，例如状态和当前查询。
pg_stat_replication	每一个 WAL 发送进程一行，显示有关到该发送进程连接的后备服务器的复制的统计信息。
pg_stat_wal_receiver	只有一行，显示来自 WAL 接收器所连接服务器的有关该接收器的统计信息。
pg_stat_subscription	每个订阅至少一行，显示有关该订阅的工作者的信息。
pg_stat_ssl	每个连接（常规的或者复制）一行，显示在这个连接上使用的SSL的信息。
pg_stat_gssapi	每个连接（常规和复制）有一行，显示关于GSSAPI验证和加密的信息。
pg_stat_progress_analyze	每个运行`ANALYZE`的后端(包括自动清理工作者进程)的行，显示当前进度。
pg_stat_progress_create_index	每个后台运行`CREATE INDEX`或`REINDEX`的后端都有一行，显示当前的进度。
pg_stat_progress_vacuum	每个运行着`VACUUM`的后端（包括autovacuum工作者进程）一行，显示当前的进度。
pg_stat_progress_cluster	每个运行着`CLUSTER`或`VACUUM FULL`的后端一行，显示当前进度。
pg_stat_progress_basebackup	每一个WAL发送者进程的行显示一个基础备份，显示当前进度。

表2.已收集统计信息的视图

视图名称	描述
pg_stat_archiver	只有一行，显示有关 WAL 归档进程活动的统计信息。
pg_stat_bgwriter	只有一行，显示有关后台写进程的活动的统计信息。
pg_stat_database	每个数据库一行，显示数据库范围的统计信息。

pg_stat_database_conflicts	每个数据库一行，显示数据库范围的统计信息，这些信息的内容是关于由于与后备服务器的恢复过程发生冲突而被取消的查询。
pg_stat_all_tables	当前数据库中每个表一行，显示有关访问指定表的统计信息。
pg_stat_sys_tables	和`pg_stat_all_tables`一样，但只显示系统表。
pg_stat_user_tables	和`pg_stat_all_tables`一样，但只显示用户表。
pg_stat_xact_all_tables	和`pg_stat_all_tables`相似，但计数动作只在当前事务内发生，用于生存和死亡行数量的列以及清理和分析动作在此视图中不出现。
pg_stat_xact_sys_tables	和`pg_stat_xact_all_tables`一样，但只显示系统表。
pg_stat_xact_user_tables	和`pg_stat_xact_all_tables`一样，但只显示用户表。
pg_stat_all_indexes	当前数据库中的每个索引一行，显示：表OID、索引OID、模式名、表名、索引名、使用了该索引的索引扫描总数、索引扫描返回的索引记录数、使用该索引的简单索引扫描抓取的活表(livetable)中数据行数。 当前数据库中的每个索引一行，显示与访问指定索引有关的统计信息。
pg_stat_sys_indexes	和`pg_stat_all_indexes`一样，但只显示系统表上的索引。
pg_stat_user_indexes	和`pg_stat_all_indexes`一样，但只显示用户表上的索引。
pg_statio_all_tables	当前数据库中每个表一行(包括TOAST表)，显示：表OID、模式名、表名、从该表中读取的磁盘块总数、缓冲区命中次数、该表上所有索引的磁盘块读取总数、该表上所有索引的缓冲区命中总数、在该表的辅助TOAST表(如果存在)上的磁盘块读取总数、在该表的辅助TOAST表(如果存在)上的缓冲区命中总数、TOAST表的索引的磁盘块读取总数、TOAST表的索引的缓冲区命中总数。 当前数据库中的每个表一行，显示有关在指定表上I/O的统计信息。
pg_statio_sys_tables	和`pg_statio_all_tables`一样，但只显示系统表。
pg_statio_user_tables	和`pg_statio_all_tables`一样，但只显示用户表。
pg_statio_all_indexes	当前数据库中每个索引一行，显示：表OID、索引OID、模式名、表名、索引名、该索引的磁盘块读取总数、该索引的缓冲区命中总数。 当前数据库中的每个索引一行，显示与指定索引上的I/O有关的统计信息。
pg_statio_sys_indexes	和`pg_statio_all_indexes`一样，但只显示系统表上的索引。
pg_statio_user_indexes	和`pg_statio_all_indexes`一样，但只显示用户表上的索引。
pg_statio_all_sequences	当前数据库中每个序列对象一行，显示：序列OID、模式名、序列名、序列的磁盘读取总数、序列的缓冲区命中总数。 当前数据库中的每个序列一行，显示与指定序列上的I/O有关的统计信息。

pg_statio_sys_sequences	和`pg_statio_all_sequences`一样，但只显示系统序列（目前没有定义系统序列，因此这个视图总是为空）。
pg_statio_user_sequences	和`pg_statio_all_sequences`一样，但只显示用户序列。
pg_stat_user_functions	对于所有跟踪功能，函数的OID，模式，名称，数量 通话总时间，和自我的时间。自我时间是在函数本身所花费的时间量，总时间包括它调用函数所花费的时间。时间值以毫秒为单位。每一个被跟踪的函数一行，显示与执行该函数有关的统计信息。
pg_stat_xact_user_functions	和`pg_stat_user_functions`相似，但是只统计在当前事务期间的调用
pg_stat_slru	每个SLRU一行, 显示操作的统计信息。

针对每个索引的统计信息对于判断哪个索引正被使用以及它们的效果特别有用。

`pg\_statio\_`系列视图主要用于判断缓冲区的效果。当实际磁盘读取数远小于缓冲区命中时，这个缓冲能满足大部分读请求而无需进行内核调用。但是，这些统计信息并没有给出所有的事情：由于IvorySQL处理磁盘I/O的方式，不在IvorySQL缓冲区中的数据库仍然驻留在内核的I/O缓存中，并且因此可以被再次读取而不需要物理磁盘读取。我们建议希望了解IvorySQL I/O行为更多细节的用户将IvorySQL统计收集器和操作系统中允许观察内核处理I/O的工具一起使用。

pg_stat_activity

`pg\_stat\_activity`视图每个服务器进程将有一行，显示与该进程当前活动相关的信息。

表3.pg_stat_activity 视图

列	类型	描述
datid	oid	这个后端连接到的数据库的OID
datname	name	这个后端连接到的数据库的名称
pid	integer	这个后端的进程 ID
leader_pid	integer	并行组组长的进程ID，如果该进程是并行查询工作者。如果该进程是一个并行组的组长或不参与并行查询，则为`NULL`。
usesysid	oid	登录到这个后端的用户的 OID
username	name	登录到这个后端的用户的 OID
application_name	text	连接到这个后端的应用的名称
client_addr	inet	连接到这个后端的客户端的 IP 地址。如果这个字段为空，它表示客户端通过服务器机器上的一个 Unix 套接字连接或者这是一个内部进程，如自动清理。
client_hostname	text	已连接的客户端的主机名，由 client_addr 的反向 DNS 查找报告。这个字段将只对 IP 连接非空，并且只有log_hostname被启用时才会非空。

<code>client_port</code>	<code>integer</code>	客户端用于与此后端通信的TCP端口号，如果使用Unix套接字，则为`-1`。如果该字段为空，它表示这是一个内部服务器进程。
<code>backend_start</code>	<code>timestamp with time zone</code>	这个进程被启动的时间。对客户端后端来说，这就是客户端连接到服务器的时间。
<code>xact_start</code>	<code>timestamp with time zone</code>	这个进程的当前事务被启动的时间，如果没有活动事务则为空。如果当前查询是它的第一个事务，这一列等于 <code>query_start</code> 列。
<code>query_start</code>	<code>timestamp with time zone</code>	当前活动查询被开始的时间，如果 <code>state</code> 不是 <code>active</code> ，则为上一个查询开始的时间
<code>state_change</code>	<code>timestamp with time zone</code>	<code>state</code> 上一次被改变的时间
<code>wait_event_type</code>	<code>text</code>	后端等待的事件类型，如果有的话；否则NULL。
<code>wait_event</code>	<code>text</code>	如果后端当前正在等待，则等待事件名称，否则为NULL。
<code>state</code>	<code>text</code>	这个后端的当前总体状态。可能的值为： <code>active</code> : 后端正在执行一个查询。 <code>idle</code> : 后端正在等待一个新的客户端命令。 <code>idle in transaction</code> : 后端在一个事务中，但是当前没有正在执行一个查询。 <code>idle in transaction (aborted)</code> : 这个状态与 <code>idle in transaction</code> 相似，除了在该事务中的一个语句导致了一个错误。 <code>fastpath function call</code> : 后端正在执行一个 fast-path 函数。 <code>disabled</code> : 如果在这个后端中 <code>track_activities</code> 被禁用，则报告这个状态。
<code>backend_xid</code>	<code>xid</code>	这个后端的顶层事务标识符，如果存在。
<code>backend_xmin</code>	<code>xid</code>	当前后端的 <code>xmin</code> 范围。
<code>query</code>	<code>text</code>	这个后端最近查询的文本。如果 <code>state</code> 为 <code>active</code> ，这个字段显示当前正在执行的查询。 在所有其他状态下，它显示上一个被执行的查询。默认情况下，查询文本会被截断至1024个字节，这个值可以通过参数 <code>track_activity_query_size</code> 更改。

backend_type	text	当前后端的类型。可能的类型为 autovacuum launcher, autovacuum worker, logical replication launcher, logical replication worker, parallel worker, background writer, client backend, checkpoint, startup, walreceiver, walsender and walwriter . 除此以外，由扩展注册的后台Worker可能有额外的类型。
---------------------	-------------	--

wait_event`和`state`列是独立的。如果一个后端处于`active`状态，它可能是也可能不是某个事件上的`waiting`。如果状态是`active`并且`wait_event`为非空，它意味着一个查询正在被执行，但是它被阻塞在系统中某处。

表4.等待事件类型

等待事件类型	描述
Activity	服务器进程空闲。此事件类型表示在其主处理循环中等待活动的进程。 `wait_event`将识别特定的等待点。
BufferPin	服务器进程正在等待对数据缓冲的独占访问。如果另一个进程持有一个打开的游标，该游标最后一次从相关缓冲区读取数据，则缓冲区销等待可能是漫长的。
Client	服务器进程正在等待连接到用户应用程序的套接字上的活动。 因此，服务器预计发生一些独立于其内部进程的事情。 `wait_event`将识别特定的等待点。
Extension	服务器进程正在等待扩展模块定义的某个条件。
I/O	服务器进程正在等待一个I/O操作完成。 `wait_event`将识别特定的等待点。
IPC	服务器进程正在等待与另一个服务器进程进行交互。 `wait_event`将识别特定的等待点。
Lock	服务器进程正在等待一个重量级锁。重量级锁，也称为锁管理器锁或简单锁，主要保护表等SQL可见对象。 然而，它们也用于确保某些内部操作的互斥，例如关系扩展。 `wait_event`将识别等待的锁的类型。
LWLock	服务器进程正在等待一个轻量级锁。大多数这样的锁保护共享内存中的特定数据结构。 `wait_event`将包含标识轻量级锁用途的名称。 (有些锁有特定的名称；其他锁是一组锁的一部分，每个锁具有类似的目的。)
Timeout	服务器进程正在等待超时过期。 `wait_event`将识别特定的等待点。

表5.`Activity`类型的等待事件

Activity 等待事件	描述
ArchiverMain	在归档进程的主循环中等待。
AutoVacuumMain	在自动清理启动过程的主循环中等待。
BgWriterHibernate	在后台写进程中等待，休眠状态。
BgWriterMain	在后台写进程主循环中等待。

CheckpointMain	在校验指针进程的主循环中等待。
LogicalApplyMain	在逻辑复制应用进程的主循环中等待。
LogicalLauncherMain	在逻辑复制启动器进程的主循环中等待。
PgStatMain	在统计收集器进程的主循环中等待。
RecoveryWalStream	流恢复期间，在启动进程主循环等待WAL到达。
SysLoggerMain	在syslogger进程的主循环中等待。
WalReceiverMain	在WAL接收器进程的主循环中等待。
WalSenderMain	在WAL发送者进程的主循环中等待。
WalWriterMain	在WAL写入进程的主循环中等待。

表6. `BufferPin` 类型的等待事件

BufferPin 等待事件	描述
BufferPin	等待获得缓冲区上的独占锁。

表7. `Client` 类型的等待事件

Client 等待事件	描述
ClientRead	等待从客户端读取数据。
ClientWrite	等待写入数据到客户端。
GSSOpenServer	在建立GSSAPI会话时等待从客户端读取数据。
LibPQWalReceiverConnect	在WAL接收器等待与远程服务器建立连接。
LibPQWalReceiverReceive	在WAL接收器中等待从远程服务器接收数据。
SSLOpenServer	在尝试连接时等待SSL。
WalReceiverWaitStart	等待启动进程发送用于流复制的初始数据。
WalSenderWaitForWAL	在WAL发送器进程中等待WAL被刷新。
WalSenderWriteData	在WAL发送器进程中处理WAL接收器的回复时，等待任何活动。

表8. `Extension` 类型的等待事件

Extension 等待事件	描述
Extension	在扩展中等待。

表9. `IO` 类型的等待事件

IO 等待事件	描述
BufFileRead	等待从缓冲文件中读取。
BufFileWrite	等待对缓冲文件的写入。
ControlFileRead	等待读取 `pg_control` 文件。
ControlFileSync	等待 `pg_control` 文件到达持久存储。
ControlFileSyncUpdate	等待更新 `pg_control` 文件以达到持久存储。
ControlFileWrite	等待写入 `pg_control` 文件。
ControlFileWriteUpdate	等待写入更新 `pg_control` 文件。
CopyFileRead	在文件复制操作期间等待读取。
CopyFileWrite	在文件拷贝操作期间等待写入。

DSMFillZeroWrite	等待用零填充动态共享内存备份(backing)文件。
DataFileExtend	等待关系数据文件被扩展。
DataFileFlush	等待关系数据文件达到持久存储。
DataFileImmediateSync	等待关系数据文件到持久存储的立即同步。
DataFilePrefetch	等待关系数据文件的异步预取。
DataFileRead	等待对关系数据文件的读取。
DataFileSync	等待对关系数据文件的更改达到持久存储。
DataFileTruncate	等待关系数据文件被截断。
DataFileWrite	等待对关系数据文件的写入。
LockFileAddToDataDirRead	在向数据目录锁文件中添加一行时等待读取。
LockFileAddToDataDirSync	等待数据到达持久存储，同时向数据目录锁文件添加一行。
LockFileAddToDataDirWrite	在向数据目录锁文件中添加一行时等待写操作。
LockFileCreateRead	创建数据目录锁文件时等待读取。
LockFileCreateSync	在创建数据目录锁文件时等待数据到达持久存储。
LockFileCreateWrite	在创建数据目录锁文件时等待写操作。
LockFileReCheckDataDirRead	在重新检查数据目录锁文件期间等待读取。
LogicalRewriteCheckpointSync	等待逻辑重写映射到检查点到达持久存储。
LogicalRewriteMappingSync	在逻辑重写期间等待映射数据到达持久存储
LogicalRewriteMappingWrite	在逻辑重写期间等待映射数据的写入。
LogicalRewriteSync	等待逻辑重写映射到达持久存储。
LogicalRewriteTruncate	等待在逻辑重写期间截断映射数据。
LogicalRewriteWrite	等待逻辑重写映射的写入。
RelationMapRead	等待关系映射文件的读取。
RelationMapSync	等待关系映射文件到达持久存储。
RelationMapWrite	等待对关系映射文件的写入。
ReorderBufferRead	在重新排序缓冲区管理期间等待读取。
ReorderBufferWrite	在重新排序缓冲区管理期间等待写操作。
ReorderLogicalMappingRead	在重新排序缓冲区管理期间等待读取逻辑映射。
ReplicationSlotRead	等待从复制槽位控制文件读取。
ReplicationSlotRestoreSync	等待复制槽控制文件到达持久存储，同时将其恢复到内存中。
ReplicationSlotSync	等待复制槽控制文件到达持久存储。
ReplicationSlotWrite	等待对复制槽控制文件的写入。
SLRUFlushSync	在检查点或数据库关闭期间等待SLRU数据到达持久存储。
SLRURead	等待读取SLRU页面。
SLRUSync	在写页面后等待SLRU数据到达持久存储。
SLRUWrite	等待SLRU页面的写入。
SnapbuildRead	等待读取序列化的历史目录快照。
SnapbuildSync	等待序列化历史目录快照到达持久存储。
SnapbuildWrite	等待串行历史目录快照的写入。

TimelineHistoryFileSync	等待通过流复制接收的时间线历史文件到达持久存储。
TimelineHistoryFileWrite	等待通过流复制接收的时间线历史文件的写入。
TimelineHistoryRead	等待读取时间线历史文件。
TimelineHistorySync	等待新创建的时间线历史文件到达持久存储。
TimelineHistoryWrite	等待写入新创建的时间线历史文件。
TwophaseFileRead	等待读取两阶段状态文件。
TwophaseFileSync	等待两阶段状态文件到达持久存储。
TwophaseFileWrite	等待对两阶段状态文件的写入。
WALBootstrapSync	在引导过程中等待WAL达到持久存储。
WALBootstrapWrite	在引导过程中等待WAL页面的写入。
WALCopyRead	通过复制一个已有WAL段来创建一个新的WAL段时等待读取。
WALCopySync	等待通过复制一个已有WAL段到持久存储来创建一个新的WAL段。
WALCopyWrite	通过复制一个已有WAL段来创建一个新的WAL段时等待写入。
WALInitSync	等待一个新初始化的WAL文件到持久存储。
WALInitWrite	在初始化一个新的WAL文件时等待写入。
WALRead	等待WAL文件的读取。
WALSenderTimelineHistoryRead	在walsender时间线命令期间等待从时间线历史文件读取。
WALSync	等待WAL文件到达持久存储。
WALSyncMethodAssign	等待数据到达持久存储，同时分配一个新的WAL同步方法。
WALWrite	等待写入WAL文件。

表10. `IPC` 类型的等待事件

IPC 等待事件	描述
BackupWaitWalArchive	等待备份所需的WAL文件成功存档。
BgWorkerShutdown	等待后台工作者关闭。
BgWorkerStartup	等待后台工作者启动。
BtreePage	正等待继续并行B-树扫描所需的页号变得可用。
CheckpointDone	等待检查点完成。
CheckpointStart	等待检查点开始。
ExecuteGather	在执行`Gather`计划节点时，等待子进程的活动。
HashBatchAllocate	等待一个选定的并行哈希参与者分配哈希表。
HashBatchElect	等待选择一个并行哈希参与者来分配哈希表。
HashBatchLoad	等待其他并行哈希参与者完成哈希表的加载。
HashBuildAllocate	等待一个选定的并行哈希参与者分配初始哈希表。
HashBuildElect	等待选择一个并行哈希参与者来分配初始哈希表。
HashBuildHashInner	等待其他并行哈希参与者完成内部关系的散列。
HashBuildHashOuter	等待其他Parallel 哈希参与者完成对外部关系的分区。

HashGrowBatchesAllocate	等待选定的并行哈希参与者分配更多批处理。
HashGrowBatchesDecide	等待选择一个并行哈希参与者来决定未来的批处理增长。
HashGrowBatchesElect	等待选择一个Parallel 哈希参与者来分配更多批处理。
HashGrowBatchesFinish	等待一个选定的并行哈希参与者决定未来的批量增长。
HashGrowBatchesRepartition	等待一个选定的并行哈希参与者决定未来的批处理增长。
HashGrowBucketsAllocate	等待选定的并行哈希参与者完成更多bucket的分配。
HashGrowBucketsElect	等待选择一个并行哈希参与者来分配更多的buckets。
HashGrowBucketsReinsert	等待其他Parallel 哈希参与者完成将元组插入到新buckets中。
LogicalSyncData	等待逻辑复制远程服务器发送用于初始表同步的数据。
LogicalSyncStateChange	等待逻辑复制远程服务器更改状态。
MessageQueueInternal	等待另一个进程附加到共享消息队列。
MessageQueuePutMessage	等待将协议消息写入共享消息队列。
MessageQueueReceive	等待从共享消息队列接收字节。
MessageQueueSend	等待将字节发送到共享消息队列。
ParallelBitmapScan	等待并行位图扫描被初始化。
ParallelCreateIndexScan	等待并行`CREATE INDEX`工作者完成堆扫描。
ParallelFinish	等待并行工作人员完成计算。
ProcArrayGroupUpdate	等待组领导在并行操作结束时清除事务ID。
ProcSignalBarrier	等待屏障事件被所有后端处理。
Promote	等待备用系统提升。
RecoveryConflictSnapshot	等待vacuum清理的恢复冲突解决。
RecoveryConflictTablespace	等待恢复冲突解决删除表空间。
RecoveryPause	等待恢复继续进行。
ReplicationOriginDrop	等待复制源变为非活动状态，以便可以删除它。
ReplicationSlotDrop	等待复制槽变为非活动状态，以便可以删除它。
SafeSnapshot	等待获取`READ ONLY DEFERRABLE`事务的有效快照。
SyncRep	在同步复制期间等待远程服务器的确认。
XactGroupUpdate	等待分组组长在并行操作结束时更新事务状态。

表11. `Lock` 类型的等待事件

Lock 等待事件	描述
advisory	等待获得一个建议用户锁。
extend	等待扩展一个关系。
frozenid	等待升级 <code>pg_database.datfrozenxid</code> 和 <code>pg_database.datminmxid</code> 。
object	等待获取非关系数据库对象上的锁。
page	等待获取一个关系页面上的锁。

relation	等待获得一个关系的锁。
spectoken	等待获取推测的插入锁。
transactionid	等待事务完成。
tuple	等待获取元组上的锁。
user lock	等待获取用户锁。
virtualxid	等待获取虚拟事务ID锁。

表12. `LWLock` 类型的等待事件

LWLock 等待事件	描述
AddinShmemInit	等待管理共享内存中的扩展空间分配。
AutoFile	等待更新 `postgresql.auto.conf` 文件。
Autovacuum	等待读取或更新自动清理工作者的当前状态。
AutovacuumSchedule	等待确保选择为自动清理的表仍然需要清理。
BackgroundWorker	等待读取或更新后台工作者状态。
BtreeVacuum	等待读取或更新b-树索引的清理相关信息。
BufferContent	等待访问内存中的数据页。
BufferIO	等待数据页上的I/O。
BufferMapping	等待将数据块与缓冲池中的缓冲区关联。
Checkpoint	等待开始一个检查点。
CheckpointInterComm	等待管理fsync请求。
CommitTs	等待读取或更新事务提交时间戳的最后一个值集。
CommitTsBuffer	在提交时间戳SLRU缓冲区上等待I/O。
CommitTsSLRU	等待访问提交时间戳SLRU缓存。
ControlFile	等待读取或更新 `pg_control` 文件或创建一个新的WAL文件。
DynamicSharedMemoryControl	等待读取或更新动态共享内存分配信息。
LockFastPath	等待读取或更新进程的快速路径锁信息。
LockManager	等待读取或更新关于 “heavyweight” 锁。
LogicalRepWorker	等待读取或更新逻辑复制工作者的状态。
MultixactGen	等待读取或更新共享的multixact状态。
MultixactMemberBuffer	在multixact成员SLRU缓冲区上等待I/O。
MultixactMemberSLRU	等待访问multixact成员SLRU缓存。
MultixactOffsetBuffer	在multixact 偏移 SLRU缓冲区上等待I/O。
MultixactOffsetSLRU	等待访问multixact 偏移 SLRU缓存。
MultixactTruncation	等待读取或截断multixact信息。
NotifyBuffer	在 `NOTIFY` 消息 SLRU缓冲区上等待I/O。
NotifyQueue	等待读取或更新 `NOTIFY` 消息。
NotifyQueueTail	等待 `NOTIFY` 消息存储上的更新限制。
NotifySLRU	等待访问 `NOTIFY` 消息SLRU缓存。
OidGen	等待分配一个新的OID。
OldSnapshotTimeMap	等待读取或更新旧的快照控制信息。
ParallelAppend	在并行附加计划执行期间等待选择下一个子计划。

ParallelHashJoin	在并行哈希连接计划执行期间等待同步工作器。
ParallelQueryDSA	等待并行查询动态共享内存分配。
PerSessionDSA	等待并行查询动态共享内存分配。
PerSessionRecordType	等待访问有关复合类型的并行查询信息。
PerSessionRecordTypmod	等待访问有关标识匿名记录类型的类型修饰符的并行查询信息。
PerXactPredicateList	在并行查询期间等待访问当前可序列化事务持有的谓词锁列表。
PredicateLockManager	等待访问可序列化事务使用的谓词锁信息。
ProcArray	等待访问每个进程共享的数据结构(通常情况, 是获取快照或报告会话的事务ID)。
RelationMapping	等待读取或更新`pg_filenode.map`文件(用于跟踪某些系统目录的文件节点分配)。
RelCacheInit	等待读取或更新`pg_internal.init`关系缓存初始化文件。
ReplicationOrigin	等待创建、删除或使用复制源。
ReplicationOriginState	等待读取或更新一个复制源的进度。
ReplicationSlotAllocation	等待分配或释放复制槽。
ReplicationSlotControl	等待读取或更新复制槽状态。
ReplicationSlotIO	在复制槽位上等待I/O。
SerialBuffer	在可串行事务冲突的SLRU缓冲区上等待I/O。
SerializableFinishedList	等待访问已完成的可序列化事务列表。
SerializablePredicateList	等待访问可序列化事务持有的谓词锁列表。
SerializableXactHash	等待读取或更新关于可序列化事务的信息。
SerialSLRU	等待访问可序列化事务冲突SLRU缓存。
SharedTidBitmap	在并行位图索引扫描期间等待访问共享的TID位图。
SharedTupleStore	在并行查询期间等待访问共享元组存储。
ShmemIndex	等待在共享内存中找到或分配空间。
SInvalRead	等待从共享目录失效队列中检索消息。
SInvalWrite	等待向共享编目失效队列添加消息。
SubtransBuffer	在子事务SLRU缓冲区上等待I/O。
SubtransSLRU	等待访问子事务SLRU缓存。
SyncRep	等待读取或更新有关同步复制状态的信息。
SyncScan	等待选择同步表扫描的起始位置。
TablespaceCreate	等待创建或删除表空间。
TwoPhaseState	等待读取或更新已准备事务的状态。
WALBufMapping	等待在WAL缓冲区中替换一个页面。
WALInsert	等待将WAL数据插入内存缓冲区。
WALWrite	等待WAL缓冲区写入磁盘。
WrapLimitsVacuum	等待更新事务id和multixact消费的限制。
XactBuffer	在事务状态的SLRU缓冲区上等待I/O。
XactSLRU	等待访问事务状态的SLRU缓存。
XactTruncation	等待执行`pg_xact_status`或更新它可用的最早的事务ID。

XidGen	等待分配新的事务ID。
--------	-------------

表13. `Timeout` 类型的等待事件

Timeout 等待事件	描述
BaseBackupThrottle	当有限流活动时在基础备份期间等待。
PgSleep	由于调用 `pg_sleep` 或同类函数而等待。
RecoveryApplyDelay	由于延迟设置，在恢复期间等待应用WAL。
RecoveryRetrieveRetryInterval	当WAL数据无法从任何来源(pg_wal ，存档或流)获得时，在恢复期间等待。
VacuumDelay	在一个基于代价的清理延迟点。

下面的例子展示了如何查看等待事件：

```
SELECT pid, wait_event_type, wait_event FROM pg_stat_activity WHERE wait_event is NOT
NULL;
 pid | wait_event_type | wait_event
-----+-----+-----
 2540 | Lock            | relation
 6644 | LWLock          | ProcArray
(2 rows)
```

pg_stat_replication

`pg\_stat\_replication` 视图将在每个WAL发送方进程中包含一行，显示关于复制到发送方连接的备用服务器的统计信息。只有直接连接的备用设备被列出;没有关于下游备用服务器的信息。

表14.pg_stat_replication 视图

列类型描述
pid `integer` 一个 WAL 发送进程的进程 ID
usesysid `oid` 登录到这个 WAL 发送进程的用户的 OID
username `name` 登录到这个 WAL 发送进程的用户的名称
application_name `text` 连接到这个 WAL 发送进程的应用的名称
client_addr `inet` 连接到这个 WAL 发送进程的客户端的 IP 地址。 如果这个域为空，它表示该客户端通过服务器机器上的一个Unix 套接字连接。
client_hostname `text` 连接上的客户端的主机名，由一次对 `client_addr` 的逆向 DNS 查找报告。 这个域将只对 IP 连接非空，并且只有在 log_hostname被启用时非空。
client_port `integer` 客户端用来与这个 WAL 发送进程通讯的 TCP 端口号，如果使用 Unix 套接字则为 -1`
backend_start `timestamp with time zone` 这个进程开始的时间，即客户端是何时连接到这个WAL 发送进程的。
backend_xmin `xid` 由hot_standby_feedback报告的这个后备机的 `xmin` 水平线。
state `text` 当前的 WAL 发送进程状态。可能的值是：`startup`：这个WAL发送器正在启动。 catchup ：这个WAL发送者连接的备用服务器正在赶上主服务器。 streaming ：在其连接的备用服务器赶上主服务器之后，这个WAL发送方正在流化变化。 backup ：这个WAL发送器正在发送一个备份。 stopping ：这个WAL发送器正在停止。
sent_lsn `pg_lsn` 在这个连接上发送的最后一个预写式日志的位置

write_lsn	<code>pg_lsn</code> 被这个后备服务器写入到磁盘的最后一个预写式日志的位置
flush_lsn	<code>pg_lsn</code> 被这个后备服务器刷入到磁盘的最后一个预写式日志的位置
replay_lsn	<code>pg_lsn</code> 被重放到这个后备服务器上的数据库中的最后一个预写式日志的位置
write_lag	<code>interval</code> 从本地刷新近期的WAL与接收到此备用服务器已写入WAL的通知(但尚未刷新或应用它)之间的时间经过。 如果将此服务器配置为同步备用服务器，则可以使用此参数来衡量在提交时 <code>synchronous_commit</code> 级别 <code>remote_write</code> 所导致的延迟。
flush_lag	<code>interval</code> 在本地刷写近期的WAL与接收到后备服务器已经写入并且刷写它（但还没有应用）的通知之间流逝的时间。 如果这台服务器被配置为一个同步后备，这可以用来计量在提交时 <code>synchronous_commit</code> 的级别 <code>on</code> 所导致的延迟。
replay_lag	<code>interval</code> 在本地刷写近期的WAL与接收到后备服务器已经写入它、刷写它并且应用它的通知之间流逝的时间。 如果这台服务器被配置为一个同步后备，这可以用来计量在提交时 <code>synchronous_commit</code> 的级别 <code>remote_apply</code> 所导致的延迟。
sync_priority	<code>integer</code> 在基于优先的同步复制中，这台后备服务器被选为同步后备的优先级。在基于规定数量的同步复制中，这个值没有效果。
sync_state text	这一台后备服务器的同步状态。可能的值是： <code>async</code> ：这台后备服务器是异步的。 <code>potential</code> ：这台后备服务器现在是异步的，但可能在当前的同步后备失效时变成同步的。 <code>sync</code> ：这台后备服务器是同步的。 <code>quorum</code> ：这台后备服务器被当做规定数量后备服务器的候选。
reply_time	<code>带时区的时间戳</code> 从备用服务器收到的最后一条回复信息的发送时间

`pg_stat_replication` 视图中报告的滞后时间近期的WAL被写入、刷写并且重放以及发送器知道这一切所花的时间的度量。如果远程服务器被配置为一台同步后备，这些时间表示由每一种同步提交级别所带来（或者是可能带来）的提交延迟。对于一台异步后备，`replay_lag` 列是最近的事务变得对查询可见的延迟时间的近似值。如果后备服务器已经完全追上了发送服务器并且没有WAL活动，在短时间内将继续显示最近测到的滞后时间，再然后就会显示为NULL。

对于物理复制会自动测量滞后时间。逻辑解码插件可能会选择性地发出跟踪消息，如果它们没有这样做，跟踪机制将把滞后显示为NULL。

注意

报告的滞后时间并非按照当前的重放速率该后备还有多久才能追上发送服务器的预测。在新的WAL被生成期间，这样一种系统将显示类似的时间，但是当发送器变为闲置时会显示不同的值。特别是当后备服务器完全追上时，`pg_stat_replication` 显示的是写入、刷写及重放最近报告的WAL位置所花的时间而不是一些用户可能预期的零。这种做法与为近期的写事务测量同步提交和事务可见性延迟的目的的一致。为了降低用户预期一种不同的滞后模型带来的混淆，在一个完全重放完的闲置系统上，`lag` 列会在一段比较短的时间后回复成NULL。监控系统应该选择将这种情况表示为缺失数据、零或者继续显示最近的已知值。

pg_stat_wal_receiver

`pg_stat_wal_receiver` 事务只包含一行，它显示了从WAL接收器所连接的服务器得到的有关该接收器的统计信息。

表15.pg_stat_wal_receiver 视图

列类型描述
pid <code>integer</code> WAL接收器进程的进程ID

status	`text` WAL接收进程的活动状态
receive_start_lsn	`pg_lsn` WAL接收器启动时使用的第一个写前日志位置
receive_start_tli	`integer` WAL接收器启动时使用的第一个时间线数字
written_lsn	`pg_lsn` 已经接收并写入磁盘的最后一个预写式日志位置，但没有刷入。这不能用于数据完整性检查。
flushed_lsn	`pg_lsn` 已经接收并刷入到磁盘的最后一个预写式日志位置，该字段的初始值是启动WAL接收器时使用的第一个日志位置
received_tli	`integer` 接收并刷入到磁盘的最后一个预写式日志位置的时间线数字，该字段的初始值为启动WAL接收器时使用的第一个日志位置的时间线数字
last_msg_send_time	`timestamp with time zone` 从源头WAL发送器收到的最后一条信息的发送时间
last_msg_receipt_time	`timestamp with time zone` 从源头WAL发送器收到的最后一条信息的接收时间
latest_end_lsn	`pg_lsn` 向源头WAL发送器报告的最后的预写式日志位置
latest_end_time	`timestamp with time zone` 向源头WAL发送方报告的最后一次写前日志位置的时间
slot_name	`text` 这个WAL接收器使用的复制槽的名称
sender_host	text` 这个WAL接收器连接到的IvorySQL实例的主机。 这可以是主机名、IP地址，或者目录路径，如果连接是通过Unix套接字进行的。（路径的情况可以区分，因为它总是以/开头的绝对路径。）
sender_port	`integer` 这个WAL接收器连接的IvorySQL实例的端口号。
conninfo	`text` 这个WAL接收器使用的连接字符串，对安全敏感的字段进行了模糊处理。

pg_stat_subscription

每一个订阅的主工作者都在`pg\_stat\_subscription`视图中有一行（如果工作者没有运行则PID为空），处理被订阅表的初始数据拷贝操作的工作者还会有额外的行。

表16.pg_stat_subscription 视图

列类型描述
subid `oid` 订阅的OID
subname `name` 订阅的名称
pid `integer` 订阅工作者进程的进程ID
relid `oid` 工作器正在同步的关系的OID;Null用于主应用工作器
received_lsn `pg_lsn` 接收到的最后一个预写式日志位置，该字段的初始值为0
last_msg_send_time `timestamp with time zone` 从WAL发送器收到的最后一条信息的发送时间
last_msg_receipt_time `timestamp with time zone` 从WAL发送器收到的最后一条信息的接收时间
latest_end_lsn `pg_lsn` 向WAL发送器报告的最后预写式日志位置
latest_end_time `timestamp with time zone` 向WAL发送器报告的最后一次预写式日志位置的时间

pg_stat_ssl

`pg\_stat\_ssl`视图将为每一个后端或者WAL发送进程包含一行，用来显示这个连接上的SSL使用情况。可以把它与`pg\_stat\_activity`或者`pg\_stat\_replication`通过`pid`列连接来得到更多有关该连接的细节。

表17.pg_stat_ssl 视图

pid	`integer` 后端或WAL发送器进程ID
ssl	`boolean` 如果在此连接上使用SSL，则为真

version	`text` 使用SSL的版本，如果此连接上没有使用SSL则为NULL
cipher	`text` 正在使用的SSL密码的名称，如果此连接上没有使用SSL则为NULL
bits	`integer` 使用的加密算法中的位数，如果此连接上没有使用SSL则为NULL
compression	`boolean` 如果使用SSL压缩则为真，否则为假，如果此连接未使用SSL则为NULL
client_dn	text` 区别名称(DN, Distinguished Name)字段与使用的客户端证书，如果没有提供客户端证书或在此连接上没有使用SSL，则为NULL。如果DN字段长于`NAMEDATALEN(标准构建中为64个字符)，则该字段将被截断。
client_serial	`numeric` 客户端证书的序列号，如果没有提供客户端证书或在此连接上没有使用SSL，则为NULL。证书序列号和证书颁发者的组合唯一标识一个证书(除非颁发者错误地重用序列号)。
issuer_dn	`text` 客户端证书颁发者的区别名称(DN, Distinguished Name)，如果没有提供客户端证书或在此连接上没有使用SSL，则为NULL。该字段像`client_dn`一样被截断。

pg_stat_gssapi

`pg\_stat\_gssapi` 视图将包含每一个后端一个行，显示该连接上的GSSAPI使用情况。它可以加入到`pg\_stat\_activity`或`pg\_stat\_replication`上的`pid`列，获取更多关于连接的详细信息。

表18.pg_stat_gssapi 视图

列类型描述
pid `integer` 后端进程ID
gss_authenticated `boolean` 如果此连接使用了GSSAPI身份验证，则为True
principal text `用于验证此连接的主体，如果未使用GSSAPI对此连接进行身份验证，则为NULL。如果主体长度超过`NAMEDATALEN(标准构建中为64个字符)，则该字段被截断。`
encrypted `boolean` 如果在此连接上使用了GSSAPI加密，则为真

pg_stat_archiver

`pg\_stat\_archiver` 视图总是有一行，其中包含关于集群的存档进程的数据。

表19.pg_stat_archiver 视图

列类型描述
archived_count `bigint` 已成功存档的WAL文件数
last_archived_wal `text` 最后一个成功存档的WAL文件的名称
last_archived_time `timestamp with time zone` 最后一次成功存档操作的时间
failed_count `bigint` 记录WAL文件归档失败次数
last_failed_wal `text` 最后一次失败的存档操作的WAL文件的名称
last_failed_time `timestamp with time zone` 上次存档操作失败的时间
stats_reset `timestamp with time zone` 这些统计数据最后一次重置的时间

pg_stat_bgwriter

`pg\_stat\_bgwriter` 视图始终只有一行，其中包含集群的全局数据。

表20.pg_stat_bgwriter 视图

列类型描述
checkpoints_timed `bigint` 已执行的预定检查点数

checkpoints_req	`bigint` 请求已执行的检查点数
checkpoint_write_time	`double precision` 检查点处理中将文件写入磁盘的部分所花费的总时间，以毫秒为单位
checkpoint_sync_time	`double precision` 检查点处理中将文件同步到磁盘的部分所花费的总时间，以毫秒为单位
buffers_checkpoint	`bigint` 检查点期间写入的缓冲区数
buffers_clean	`bigint` 后台写入器写入的缓冲区数
maxwritten_clean	`bigint` 后台写入器因为写入太多缓冲区而停止清理扫描的次数
buffers_backend	`bigint` 后端直接写入的缓冲区数
buffers_backend_fsync	`bigint` 后端必须执行自己的`fsync`调用的次数(通常后台写入器处理这些，即使后端执行自己的写入)
buffers_alloc	`bigint` 分配的缓冲区数
stats_reset	`timestamp with time zone` 这些统计数据最后一次重置的时间

pg_stat_database

`pg\_stat\_database` 视图将包含一行用于集群中的每个数据库，加一行用于共享对象，显示数据库范围的统计信息。

表21. **pg_stat_database** 视图

列类型描述	
datid	`oid` 该数据库的OID，属于共享关系的对象为0
datname	name 这个数据库的名称，或者共享对象为`NULL`。
numbackends	integer 当前连接到此数据库的后端数，对于共享对象则为`NULL`。这是该视图中唯一返回反映当前状态的值的列；所有其他列返回自上次重置以来累积的值。
xact_commit	`bigint` 此数据库中已提交的事务数
xact_rollback	`bigint` 该数据库中已回滚的事务数
blks_read	`bigint` 在该数据库中读取的磁盘块数
blks_hit	`bigint` 在缓存中发现磁盘块的次数，因此读取不是必需的(这只包括在IvorySQL缓存中，而不是在操作系统的文件系统缓存中)
tup_returned	`bigint` 这个数据库中查询返回的行数
tup_fetched	`bigint` 这个数据库中查询获取的行数
tup_inserted	`bigint` 查询在该数据库中插入的行数
tup_updated	`bigint` 这个数据库中查询更新的行数
tup_deleted	`bigint` 这个数据库中被查询删除的行数
conflicts	`bigint` 由于与此数据库中的恢复冲突而取消的查询数。(冲突只发生在备用服务器上)
temp_files	`bigint` 这个数据库中查询创建的临时文件的数量。所有临时文件都将被计数，而不顾及临时文件为什么被创建(例如，排序或散列)，也不考虑log_temp_files设置。
temp_bytes	`bigint` 这个数据库中的查询写入临时文件的数据总量。所有临时文件都将被计数，而不考虑临时文件为什么被创建，也不考虑log_temp_files设置。
deadlocks	`bigint` 在此数据库中检测到的死锁数
checksum_failures	`bigint` 在此数据库(或共享对象)中检测到的数据页校验码失败数，如果没有启用数据校验码则为NULL。

checksum_last_failure `timestamp with time zone` 在此数据库(或共享对象)中检测到最后一个数据页校验码失败的时间，如果没有启用数据校验码则为NULL。
blk_read_time `double precision` 在这个数据库中通过后端读取数据文件块所花费的时间，以毫秒为单位(如果启用了track_io_timing，否则为零)
blk_write_time `double precision` 在这个数据库中通过后端写数据文件块所花费的时间，以毫秒为单位(如果启用了track_io_timing，否则为零)
stats_reset `timestamp with time zone` 这些统计数据最后一次重置的时间

pg_stat_database_conflicts

`pg\_stat\_database\_conflicts` 视图为每一个数据库包含一行，用来显示数据库范围内由于与后备服务器上的恢复过程冲突而被取消的查询的统计信息。这个视图将只包含后备服务器上的信息，因为冲突不会发生在主服务器上。

表22.pg_stat_database_conflicts 视图

datid `oid` 数据库的OID
datname `name` 数据库的名称
confl_tablespace `bigint` 这个数据库中由于删除表空间而取消的查询的数量
confl_lock `bigint` 此数据库中由于锁定超时而被取消的查询数
confl_snapshot `bigint` 此数据库中由于旧快照而取消的查询数
confl_bufferpin `bigint` 此数据库中由于固定缓冲区而被取消的查询数
confl_deadlock `bigint` 此数据库中由于死锁而被取消的查询数

pg_stat_all_tables

`pg\_stat\_all\_tables` 视图将为当前数据库中的每一个表（包括 TOAST 表）包含一行，该行显示与对该表的访问相关的统计信息。
`pg\_stat\_user\_tables` 和 `pg\_stat\_sys\_tables` 视图包含相同的信息，但是被过滤得分别只显示用户和系统表。

表23.pg_stat_all_tables 视图

列类型描述
relid `oid` 表的OID
schemaname `name` 该表所在的模式的名称
relname `name` 这个表的名称
seq_scan `bigint` 在此表上启动的顺序扫描数
seq_tup_read `bigint` 连续扫描获取的实时行数
idx_scan `bigint` 对这个表发起的索引扫描数
idx_tup_fetch `bigint` 索引扫描获取的实时行数
n_tup_ins `bigint` 插入的行数
n_tup_upd `bigint` 更新的行数(包括HOT更新的行)
n_tup_del `bigint` 删除的行数
n_tup_hot_upd `bigint` HOT更新的行数(即，不需要单独的索引更新)
n_live_tup `bigint` 活的行的估计数量
n_dead_tup `bigint` 僵死行的估计数量

<code>n_mod_since_analyze</code>	`bigint` 自上次分析此表以来修改的行的估计数量
<code>n_ins_since_vacuum</code>	`bigint` 自上次清空此表以来插入的行的估计数量
<code>last_vacuum_timestamp with time zone</code>	最后一次手动清理这个表(不包括`VACUUM FULL`)
<code>last_autovacuum</code>	`timestamp with time zone` 这个表最后一次被自动清理守护进程清理的时间
<code>last_analyze</code>	`timestamp with time zone` 上一次手动分析这个表
<code>last_autoanalyze</code>	`timestamp with time zone` 自动清理守护进程最后一次分析这个表
<code>vacuum_count</code>	`bigint` 这个表被手动清理的次数(`VACUUM FULL`不计数)
<code>autovacuum_count</code>	`bigint` 这个表被autovacuum守护进程清理的次数
<code>analyze_count</code>	`bigint` 手动分析这个表的次数
<code>autoanalyze_count</code>	`bigint` 这个表被autovacuum守护进程分析的次数

pg_stat_all_indexes

`pg\_stat\_all\_indexes` 视图将为当前数据库中的每个索引包含一行，该行显示关于对该索引访问的统计信息。`pg\_stat\_user\_indexes` 和 `pg\_stat\_sys\_indexes` 视图包含相同的信息，但是被过滤得只分别显示用户和系统索引。

表24.pg_stat_all_indexes 视图

列类型描述
relid `oid` 对于此索引的表的OID
indexrelid `oid` 这个索引的OID
schemaname `name` 这个索引所在的模式名称
relname `name` 这个索引的表的名称
indexrelname `name` 这个索引的名称
idx_scan `bigint` 在这个索引上开启的索引扫描的数量
idx_tup_read `bigint` 扫描此索引返回的索引项数
idx_tup_fetch `bigint` 使用此索引进行简单索引扫描获取的活动表行数

索引可以被简单索引扫描、“位图”索引扫描以及优化器使用。在一次位图扫描中，多个索引的输出可以通过 AND 或 OR

规则组合，因此当使用一次位图扫描时难以将取得的个体堆行与特定的索引关联起来。因此，一次位图扫描会增加它使用的索引的`pg\_stat\_all\_indexes.idx\_tup\_read`计数，并且为每个表增加`pg\_stat\_all\_tables.idx\_tup\_fetch`计数，但是它不影响`pg\_stat\_all\_indexes.idx\_tup\_fetch`。如果所提供的常量值不在优化器统计信息记录的范围之内，优化器也会访问索引来检查，因为优化器统计信息可能已经“不新鲜”了。

注意

即使不用位图扫描，`idx\_tup\_read` 和 `idx\_tup\_fetch` 计数也可能不同，因为 `idx\_tup\_read` 统计从该索引取得的索引项而 `idx\_tup\_fetch` 统计从表取得的活着的行。如果使用该索引取得了任何死亡行或还未提交的行，或者如果通过一次只用索引扫描的方式避免了任何堆获取，后者将较小。

pg_statio_all_tables

`pg\_statio\_all\_tables` 视图将为当前数据库中的每个表（包括 TOAST 表）包含一行，该行显示指定表上有关 I/O 的统计信息。`pg\_statio\_user\_tables` 和 `pg\_statio\_sys\_tables` 视图包含相同的信息，但是被过滤得分别只显示用户表和系统表。

表25.pg_statio_all_tables 视图

列类型描述
relid `oid` 表的OID
schemaname `name` 该表所在的模式名
relname `name` 这个表的名称
heap_blks_read `bigint` 从该表中读取的磁盘块的数量
heap_blks_hit `bigint` 该表中的缓冲区命中数
idx_blks_read `bigint` 从这个表上所有索引读取的磁盘块数
idx_blks_hit `bigint` 这个表上所有索引中的缓冲区命中数
toast_blks_read `bigint` 从这个表的TOAST表中读取的磁盘块的数量(如果有的话)
toast_blks_hit `bigint` 这个表的TOAST表中的缓冲区命中数(如果有的话)
tidx_blks_read `bigint` 从这个表的TOAST表索引中读取的磁盘块的数量(如果有的话)
tidx_blks_hit `bigint` 这个表的TOAST表索引中的缓冲区命中数(如果有的话)

pg_statio_all_indexes

`pg\_statio\_all\_indexes` 视图将为当前数据库中的每个索引包含一行，该行显示指定索引上有关 I/O 的统计信息。

`pg\_statio\_user\_indexes` 和 `pg\_statio\_sys\_indexes` 视图包含相同的信息，但是被过滤得分别只显示用户索引和系统索引。

表26.pg_statio_all_indexes 视图

列类型描述
relid `oid` 对这个索引的表的OID
indexrelid `oid` 这个索引的OID
schemaname `name` 索引所在的模式名称
relname `name` 此索引的表的名称
indexrelname `name` 这个索引的名称
idx_blks_read `bigint` 从此索引中读取的磁盘块的数量
idx_blks_hit `bigint` 此索引中的缓冲区命中数

pg_statio_all_sequences

`pg\_statio\_all\_sequences` 视图将为当前数据库中的每个序列包含一行，该行显示在指定序列上有关 I/O 的统计信息。

表27.pg_statio_all_sequences 视图

列类型描述
relid `oid` 序列的OID
schemaname `name` 此序列所在的模式的名称
relname `name` 此序列的名称
blks_read `bigint` 从这个序列中读取的磁盘块的数量
blks_hit `bigint` 在此序列中的缓冲区命中数

pg_stat_user_functions

`pg\_stat\_user\_functions` 视图将为每一个被追踪的函数包含一行，该行显示有关该函数执行的统计信息。
track_functions参数控制到底哪些函数被跟踪。

表28.pg_stat_user_functions 视图

列类型描述
funcid `oid` 函数的OID
schemaname `name` 这个函数所在的模式的名称
funcname `name` 这个函数的名称
calls `bigint` 这个函数已经被调用的次数
total_time `double precision` 在这个函数以及它所调用的其他函数中花费的总时间，以毫秒计
self_time `double precision` 在这个函数本身花费的总时间，不包括被它调用的其他函数，以毫秒计

pg_stat_slru

IvorySQL通过*SLRU*(simple least-recently-used，简单的最近-最少-使用)缓存访问某些磁盘上的信息。
`pg\_stat\_slru` 视图将为每个被跟踪的SLRU缓存包含一行，显示关于访问缓存页面的统计信息。

表29.pg_stat_slru 视图

列类型描述
name `text` SLRU的名称
blks_zeroed `bigint` 初始化期间被置零的块数
blks_hit `bigint` 已经在SLRU中的磁盘块被发现的次数，因此不需要读取(这只包括SLRU中的命中，而不是操作系统的文件系统缓存)
blks_read `bigint` 为这个SLRU读取的磁盘块数
blks_written `bigint` 为这个SLRU写入的磁盘块数
blks_exists `bigint` 为这个SLRU检查是否存在的块数
flushes `bigint` 此SLRU的脏数据刷新数
truncates `bigint` 这个SLRU的截断数
stats_reset `timestamp with time zone` 这些统计数据最后一次重置的时间

Statistics Functions

其他查看统计信息的方法是直接使用查询，这些查询使用上述标准视图用到的底层统计信息访问函数。
如要了解如函数名等细节，可参考标准视图的定义（例如，在psql中你可以发出`d+ pg\_stat\_activity`）。
针对每一个数据库统计信息的访问函数把一个数据库 OID
作为参数来标识要报告哪个数据库。而针对每个表和每个索引的函数要求表或索引 OID。
针对每个函数统计信息的函数用一个函数
OID。注意只有在当前数据库中的表、索引和函数才能被这些函数看到。

更多统计集合的函数列在 表 30 中。

表30.Additional Statistics Functions

函数描述
pg_backend_pid () → `integer` 返回附加到当前会话的服务器进程的进程ID。
pg_stat_get_activity (integer) → setof record `使用指定的进程ID返回有关后端信息的记录，如果指定了`NULL`，则返回系统中每个活动后端的一条记录。返回的字段是`pg_stat_activity` 视图中字段的子集。
pg_stat_get_snapshot_timestamp () → `timestamp with time zone` 返回当前统计快照的时间戳。
pg_stat_clear_snapshot () → `void` 丢弃当前的统计快照。

pg_stat_reset () → `void` 将当前数据库的所有统计计数器重置为零。默认情况下该函数仅限于超级用户，但是其他用户可以被授予EXECUTE来运行此函数。
pg_stat_reset_shared (text) → `void` 根据参数的不同，将一些集群范围的统计计数器重置为零。参数可以是`bgwriter`来重置`pg_stat_bgwriter`视图中显示的所有计数器，或者`archiver`来重置`pg_stat_archiver`视图中显示的所有计数器。默认情况下该函数仅限于超级用户，但是其他用户可以被授予EXECUTE来运行此函数。
pg_stat_reset_single_table_counters (oid) → `void` 将当前数据库中单个表或索引的统计信息重置为零。默认情况下该函数仅限于超级用户，但是其他用户可以被授予EXECUTE来运行此函数。
pg_stat_reset_single_function_counters (oid) → `void` 将当前数据库中单个函数的统计信息重置为零。默认情况下该函数仅限于超级用户，但是其他用户可以被授予EXECUTE来运行此函数。
pg_stat_reset_slru (text) → void` 将单个SLRU缓存或集群中所有SLRU的统计信息重置为零。如果该参数为NULL，则所有SLRU缓存的`pg_stat_slru`视图中显示的计数器将被重置。参数可以是`CommitTs`、`MultiXactMember`、`MultiXactOffset`、`Notify`、`Serial`、`Subtrans`、或`Xact`中的一个，以便只重置该条目的计数器。如果参数是`other` (或实际上，任何无法识别的名称)，那么所有其他SLRU缓存的计数器，如扩展定义的缓存，将被重置。默认情况下该函数仅限于超级用户，但是其他用户可以被授予EXECUTE来运行此函数。

`pg\_stat\_get\_activity`是`pg\_stat\_activity`视图的底层函数，它返回一个行集合，其中包含有关每个后端进程所有可用的信息。有时只获得该信息的一个子集可能会更方便。在那些情况中，可以使用一组更老的针对每个后端的统计访问函数，这些显示在 表 31中。这些访问函数使用一个后端 ID 号，范围从 1 到当前活动后端数目。函数`pg\_stat\_get\_backend\_idset`提供了一种方便的方法为每个活动后端产生一行来调用这些函数。例如，要显示PID以及所有后端当前的查询：

```
SELECT pg_stat_get_backend_pid(s.backendid) AS pid,
       pg_stat_get_backend_activity(s.backendid) AS query
FROM (SELECT pg_stat_get_backend_idset() AS backendid) AS s;
```

表31.Per-Backend Statistics Functions

函数描述
pg_stat_get_backend_idset () → `setof integer` 返回当前活动后端ID号的集合(从1到活动后端数)。
pg_stat_get_backend_activity (integer) → `text` 返回此后端最近查询的文本。
pg_stat_get_backend_activity_start (integer) → `timestamp with time zone` 返回后端最近一次查询开始的时间。
pg_stat_get_backend_client_addr (integer) → `inet` 返回连接到此后端的客户端的IP地址。
pg_stat_get_backend_client_port (integer) → `integer` 返回客户端用于通信的TCP端口号。
pg_stat_get_backend_dbid (integer) → `oid` 返回此后端连接的数据库的OID。
pg_stat_get_backend_pid (integer) → `integer` 返回此后端进程ID。
pg_stat_get_backend_start (integer) → `timestamp with time zone` 返回该进程开始的时间。
pg_stat_get_backend_userid (integer) → `oid` 返回登录到此后端的用户的OID。
pg_stat_get_backend_wait_event_type (integer) → `text` 如果后端当前正在等待，返回等待事件类型名称，否则返回NULL。
pg_stat_get_backend_wait_event (integer) → `text` 如果后端当前正在等待，则返回等待事件名称，否则为NULL。
pg_stat_get_backend_xact_start (integer) → `timestamp with time zone` 返回后端当前事务开始的时间。

查看锁

监控数据库活动的另外一个有用的工具是`pg\_locks`系统表。这样就允许数据库管理员查看在锁管理器里面未解决的锁的信息。例如，这个功能可以被用于：

- 查看当前所有未解决的锁、在一个特定数据库中的关系上所有的锁、在一个特定关系上所有的锁，或者由一个特定IvorySQL会话持有的所有的锁。
- 判断当前数据库中带有最多未授予锁的关系（它很可能是数据库客户端的竞争源）。
- 判断锁竞争给数据库总体性能带来的影响，以及锁竞争随着整个数据库流量的变化范围。

Progress Reporting

IvorySQL具有在命令执行过程中报告某些命令进度的能力。

目前，支持进度报告的命令只有`ANALYZE`、**CLUSTER**、**CREATE INDEX**、**VACUUM**，和 **BASE_BACKUP**例如 **pg_basebackup**发出的进行基础备份的复制命令。未来可能还会扩展。

ANALYZE Progress Reporting

每当`ANALYZE`运行时，`pg\_stat\_progress\_analyze`视图将包含当前运行该命令的每个后端的一行。下面的表描述了将要报告的信息，并提供了关于如何解释它们的信息。

表32.**pg_stat_progress_analyze** 视图

列类型描述
pid `integer` 后端的进程ID。
datid `oid` 后端连接到的数据库的OID。
datname `name` 后端连接到的数据库的名称。
relid `oid` 被分析的表的OID。
phase `text` 当前处理阶段。参见 表 33。
sample_blks_total `bigint` 将被采样的堆块的总数。
sample_blks_scanned `bigint` 扫描的堆块数量。
ext_stats_total `bigint` 扩展统计信息的数量。
ext_stats_computed `bigint` 已经计算的扩展统计的数量. 此计数器仅在 `computing extended statistics` 阶段增进。
child_tables_total `bigint` 子表的数量。
child_tables_done `bigint` 扫描的子表数。此计数器只有在 `acquiring inherited sample rows` 阶段才会增进。
current_child_table_relid `oid` 当前正在扫描的子表的OID。此字段仅在 `acquiring inherited sample rows` 时有效。

表33.ANALYZE phases

阶段	描述
initializing	命令正在准备开始扫描堆。这个阶段预计会非常短暂。
acquiring sample rows	该命令当前正在扫描`relid`给出的表以获得示例行。
acquiring inherited sample rows	该命令当前正在扫描子表以获得示例行。列`child_tables_total`、 child_tables_done 、和`current_child_table_relid`包含此阶段的进度信息。
computing statistics	该命令从表扫描期间获得的样例行计算统计信息。

computing extended statistics	该命令从表扫描期间获得的样例行计算扩展统计信息。
finalizing analyze	该命令在更新`pg_class`。当此阶段完成时， ANALYZE 将结束。

注意

当在分区表上运行`ANALYZE`时，它的所有分区也会被递归分析，如在ANALYZE中曾提到过。在这种情况下，首先报告父表的`ANALYZE`进度，收集它的继承统计信息，然后是每个分区的(继承统计信息)。

CREATE INDEX Progress Reporting

每当运行`CREATE INDEX`或`REINDEX`时，`pg\_stat\_progress\_create\_index`视图将包含当前正在创建索引的每个后端的一行。下面的表描述了将要报告的信息，并提供了关于如何解释它的信息。

表34.pg_stat_progress_create_index 视图

列类型描述
pid `integer` 后端的进程ID。
datid `oid` 后端连接到的数据库的OID。
datname `name` 后端连接到的数据库的名称。
relid `oid` 正在创建索引的表的OID。
index_relid `oid` 正在创建或重建索引的OID。在非并发`CREATE INDEX`的时候，此为0。
command text 在运行的命令：`CREATE INDEX`,`CREATE INDEX CONCURRENTLY`,`REINDEX`,或`REINDEX CONCURRENTLY`。
phase `text` 索引创建的当前处理阶段。参见 表 35。
lockers_total `bigint` 在适用的情况下，需要等待的储物柜总数
lockers_done `bigint` 已经等待的储物柜数量。
current_locker_pid `bigint` 目前正在等待的储物柜的进程ID。
blocks_total `bigint` 本阶段要处理的区块总数。
blocks_done `bigint` 当前阶段已经处理的区块数量。
tuples_total `bigint` 当前阶段要处理的元组总数。
tuples_done `bigint` 在当前阶段已经处理的元组数量。
partitions_total `bigint` 在分区表上创建索引时，该列被设置为要在其上创建索引的分区总数。
partitions_done `bigint` 当在分区表上创建索引时，该列被设置为在其上完成索引的分区数。

表35.CREATE INDEX 的阶段

阶段	描述
初始化	`CREATE INDEX`或`REINDEX`正在准备创建索引。这个阶段预计会非常短暂。
构建前等待读写器	CREATE INDEX CONCURRENTLY`或`REINDEX CONCURRENTLY`正在等待有可能看到表的写锁的事务完成。</b> 当不在并发模式时，这个阶段会被跳过。`lockers_total`、`lockers_done`和`current_locker_pid`列包含了这个阶段的进度信息。

新建索引	索引是由访问方法专用代码建立的。 在这一阶段，支持进度报告的访问方法填写自己的进度数据，子阶段在这一栏中表示。 通常情况下， <code>blocks_total`</code> 和 <code>blocks_done`</code> 将包含进度数据，也可能包含 <code>tuples_total`</code> 和 <code>tuples_done`</code> 。
在验证前等待读写器	<code>CREATE INDEX CONCURRENTLY`</code> 或 <code>REINDEX CONCURRENTLY`</code> 正在等待有可能写入表的事务完成写锁的事务。当不在并发模式时，这个阶段会被跳过。 <code>lockers_total`</code> 、 <code>lockers_done`</code> 和 <code>current_locker_pid`</code> 列包含了这个阶段的进度信息。
索引验证：扫描索引	<code>CREATE INDEX CONCURRENTLY`</code> 正在扫描索引，搜索需要验证的图元组。如果不是在并发模式下，这个阶段会被跳过。列 <code>blocks_total`</code> （设置为索引的总大小）和 <code>blocks_done`</code> 包含了这个阶段的进度信息。
指数验证：排序元组	<code>CREATE INDEX CONCURRENTLY`</code> 正在对索引扫描阶段的输出进行排序。
索引验证：扫描表	<code>CREATE INDEX CONCURRENTLY`</code> 正在扫描表，以验证前两个阶段收集的索引图元。当不在并发模式时，这个阶段被跳过。 <code>blocks_total`</code> 列（设置为表的总大小）和 <code>blocks_done`</code> 列包含这个阶段的进度信息。
等待旧照	<code>CREATE INDEX CONCURRENTLY`</code> 或 <code>REINDEX CONCURRENTLY`</code> 正在等待可能看到表的事务释放快照。当不处于并发模式时，这个阶段会被跳过。 <code>lockers_total`</code> 、 <code>lockers_done`</code> 和 <code>current_locker_pid`</code> 列包含了这个阶段的进度信息。
标记 dead之前等待readers	<code>REINDEX CONCURRENTLY`</code> 等待表上有读锁的事务完成后，再将旧索引标记为死索引。当不在并发模式时，这个阶段被跳过。 <code>lockers_total`</code> 、 <code>lockers_done`</code> 和 <code>current_locker_pid`</code> 列包含了这个阶段的进度信息。
在 dropping之前等待readers	<code>REINDEX CONCURRENTLY`</code> 等待表上有读锁的事务完成后，再丢弃旧索引。当不在并发模式时，这个阶段被跳过。列 <code>lockers_total`</code> 、 <code>lockers_done`</code> 和 <code>current_locker_pid`</code> 包含了这个阶段的进度信息。

VACUUM进度报告

只要`VACUUM``正在运行，每一个当前正在清理的后端（包括autovacuum工作者进程）在`pg_stat_progress_vacuum``视图中都会有一行。下面的表描述了将被报告的信息并且提供了如何解释它们的信息。`VACUUM FULL``命令的进度是通过`pg_stat_progress_cluster``报告的，因为`VACUUM FULL``和`CLUSTER``都是重写表，而普通的`VACUUM``只是原地修改表。

表36.pg_stat_progress_vacuum 视图

列类型描述
<code>pid`</code> integer` 后端的进程ID。
<code>datid`</code> oid` 这个后端连接的数据库的OID。
<code>datname`</code> name` 这个后端连接的数据库的名称。
<code>relid`</code> oid` 被vacuum的表的OID。
<code>phase`</code> text` vacuum的当前处理阶段。

heap_blks_total
`bigint` 该表中堆块的总数。这个数字在扫描开始时报告，之后增加的块将不会（并且不需要）被这个`VACUUM`访问。
heap_blks_scanned
`bigint` 被扫描的堆块数量。由于visibility map被用来优化扫描，一些块将被跳过而不做检查，被跳过的块会被包括在这个总数中，因此当清理完成时这个数字最终将会等于`heap_blks_total`。仅当处于`扫描堆`阶段时这个计数器才会前进。
heap_blks_vacuumed
`bigint` 被清理的堆块数量。除非表没有索引，这个计数器仅在处于`清理堆`阶段时才会前进。不包含死亡元组的块会被跳过，因此这个计数器可能有时会向前跳跃一个比较大的增量。
index_vacuum_count
`bigint` 已完成的索引清理周期数。
max_dead_tuples
`bigint` 在需要执行一个索引清理周期之前我们可以存储的死亡元组数，取决于maintenance_work_mem。
num_dead_tuples
`bigint` 从上一个索引清理周期以来收集的死亡元组数。

表37.VACUUM的阶段

阶段	描述
初始化	`VACUUM`正在准备开始扫描堆。这个阶段应该很短。
扫描堆	`VACUUM`正在扫描堆。如果需要，它将会对每个页面进行修建以及碎片整理，并且可能会执行冻结动作。`heap_blks_scanned`列可以用来监控扫描的进度。
清理索引	`VACUUM`当前正在清理索引。如果一个表拥有索引，那么每次清理时这个阶段会在堆扫描完成后至少发生一次。如果maintenance_work_mem不足以存放找到的死亡元组，则每次清理时会多次清理索引。
清理堆	VACUUM`当前正在清理堆。清理堆与扫描堆不是同一个概念，清理堆发生在每一次清理索引的实例之后。如果`heap_blks_scanned`小于`heap_blks_total`，系统将在这个阶段完成之后回去扫描堆；否则，系统将在这个阶段完成后开始清理索引。
清除索引	`VACUUM`当前正在清除索引。这个阶段发生在堆被完全扫描并且对堆和索引的所有清理都已经完成以后。
截断堆	`VACUUM`正在截断堆，以便把关系尾部的空页面还给操作系统。这个阶段发生在清除完索引之后。
执行最后的清除	`VACUUM`在执行最终的清除。在这个阶段中，`VACUUM`将清理空闲空间映射、更新`pg_class`中的统计信息并且将统计信息报告给统计收集器。当这个阶段完成时，`VACUUM`也就结束了。

CLUSTER进度报告

每当`CLUSTER`或`VACUUM

FULL`运行时，`pg_stat_progress_cluster`视图将包含当前正在运行的每一个后台的记录。下面的表格描述了将被报告的信息，并提供了关于如何解释这些信息的信息。

表38.pg_stat_progress_cluster 视图

列类型描述
pid `integer` 后台的进程ID。

datid `oid` 该后端连接的数据库的OID。
datname `name` 与此后端连接的数据库的名称。
relid `oid` 被集群的表的OID。
command text `正在运行的命令。`CLUSTER`或`VACUUM FULL。
phase `text` 当前处理阶段。
cluster_index_relid `oid` 如果正在使用索引对表进行扫描，这就是正在使用的索引的OID；否则为0。
heap_tuples_scanned `bigint` 扫描的堆元组数。这个计数器只有在阶段为`seq scanning heap,index scanning heap`或`writing new heap`时才会增进。
heap_tuples_written `bigint` 写入的堆元组的数量。这个计数器只有在阶段为`seq scanning heap,index scanning heap`或`writing new heap`时才会前进。
heap_blks_total `bigint` 表中的堆块总数。这个数字是在`seq scanning heap`的开始时报告的。
heap_blks_scanned `bigint` 扫描的堆块数量。这个计数器只有在阶段为`seq scanning heap`时才会增进。
index_rebuild_count `bigint` 重建的索引数。该计数器仅在`重建索引`阶段时才会增进。

表39.CLUSTER 和 VACUUM FULL 阶段

阶段	描述
初始化	该命令准备开始扫描堆栈。这个阶段预计会非常短暂。
seq扫描堆	该命令目前采用顺序扫描的方式对表进行扫描。
索引扫描堆	`CLUSTER`目前正在使用索引扫描表。
元组排序	`CLUSTER`目前正在对元组进行排序。
新写入堆	`CLUSTER`目前正在编写新的堆。
交换关系文件	目前，该命令正在将新建的文件调换到位。
重建索引	该命令目前正在重建一个索引。
清理	该命令正在执行最后的清理工作。当此阶段完成后，`CLUSTER`或`VACUUM FULL`将结束。

基础备份进度报告

每当像pg_basebackup这样的应用程序进行基本备份时，`pg\_stat\_progress\_basebackup`视图将包含当前运行`BASE\_BACKUP`复制命令和流备份的每个WAL发送进程的一行。下面的表描述了将要报告的信息，并提供了关于如何解释它的信息。

表40.pg_stat_progress_basebackup 视图

列类型描述
pid `integer` WAL发送方进程ID。
phase `text` 目前的处理阶段。
backup_total `bigint` 将被流传输的数据总量。这是在`streaming database files`阶段开始时的估计和报告。注意，这只是一个近似值，因为在`streaming database files`阶段，数据库可能会改变，而WAL日志可能会在稍后的备份中包含。一旦流数据量超过了估计的总大小，该值始终与`backup_streamed`相同。如果在pg_basebackup中禁用估算(也就是说，指定了`--no-estimate-size`选项)，这为`NULL`。
backup_streamed `bigint` 数据流的总量。这个计数器只在`streaming database files`阶段或`transferring wal files`时增进。
tablespaces_total `bigint` 要流传输的表空间总数。

tablespaces_streamed `bigint` 流输送的表空间数。此计数器仅在 `streaming database files` 阶段增进。

表41.基础备份阶段

阶段	描述
initializing	WAL发送器进程正在准备开始备份。这个阶段预计会非常短暂。
waiting for checkpoint to finish	WAL发送器进程目前正在执行 <code>pg_start_backup</code> 以准备进行基础备份，并等待启动备份检查点完成。
estimating backup size	WAL发送程序目前正在估计将作为基础备份流传输的数据库文件的总量。
streaming database files	WAL发送器当前正在流数据库文件作为基础备份。
waiting for wal archiving to finish	WAL发送方进程目前正在执行 <code>pg_stop_backup</code> 以完成备份，并等待基础备份所需的所有WAL文件成功存档。如果在 <code>pg_basebackup</code> 中指定了 <code>--wal-method=none</code> 或 <code>--wal-method=stream</code> ，则备份将在此阶段完成后结束。
transferring wal files	WAL发送器进程正在传输备份过程中产生的所有WAL日志。如果 <code>pg_basebackup</code> 中指定了 <code>--wal-method=fetch</code> ，则该阶段发生在 <code>waiting for wal archiving to finish</code> 阶段之后。当此阶段完成时备份将结束。

动态追踪

IvorySQL提供了功能来支持数据库服务器的动态追踪。这样就允许在代码中的特定点上调用外部工具来追踪执行过程。

一些探针或追踪点已经被插入在源代码中。这些探针的目的是被数据库开发者和管理员使用。默认情况下，探针不被编译到IvorySQL中；用户需要显式地告诉配置脚本使得探针可用。

目前，DTrace已被支持，它在 Solaris、macOS、FreeBSD、NetBSD 和 Oracle Linux 上可用。Linux 的SystemTap项目提供了一种可用的 DTrace 等价物。支持其他动态追踪工具在理论上可以通过改变 `src/include/utils/probes.h` 中的宏定义实现。

动态追踪的编译

默认情况下，探针是不可用的，因此你将需要显式地告诉配置脚本让探针在IvorySQL中可用。要包括 DTrace 支持，在配置时指定 `--enable-dtrace`。

内建探针

如表 42所示，源代码中提供了一些标准探针。表 43显式了在探针中使用的类型。当然，可以增加更多探针来增强IvorySQL的可观测性。

表42.内建 DTrace 探针

名称	参数	描述
transaction-start	(LocalTransactionId)	在一个新事务开始时触发的探针。 arg0 是事务 ID。
transaction-commit	(LocalTransactionId)	在一个事务成功完成时触发的探针。 arg0 是事务 ID。
transaction-abort	(LocalTransactionId)	当一个事务失败完成时触发的探针。 arg0 是事务 ID。

query-start	(const char *)	当一个查询的处理被开始时触发的探针。arg0 是查询字符串。
query-done	(const char *)	当一个查询的处理完成时触发的探针。arg0 是查询字符串。
query-parse-start	(const char *)	当一个查询的解析被开始时触发的探针。arg0 是查询字符串。
query-parse-done	(const char *)	当一个查询的解析完成时触发的探针。arg0 是查询字符串。
query-rewrite-start	(const char *)	当一个查询的重写被开始时触发的探针。arg0 是查询字符串。
query-rewrite-done	(const char *)	当一个查询的重写完成时触发的探针。arg0 是查询字符串。
query-plan-start	()	当一个查询的规划被开始时触发的探针。
query-plan-done	()	当一个查询的规划完成时触发的探针。
query-execute-start	()	当一个查询的执行被开始时触发的探针。
query-execute-done	()	当一个查询的执行完成时触发的探针。
statement-status	(const char *)	任何时候当服务器进程更新它的`pg_stat_activity`.`status`时触发的探针。arg0 是新的状态字符串。
checkpoint-start	(int)	当一个检查点被开始时触发的探针。 arg0 传递位标志来区分不同的检查点类型，例如关闭 (shutdown)、立即 (immediate) 或强制 (force)。
checkpoint-done	(int, int, int, int, int)	当一个检查点完成时触发的探针（检查点处理过程中序列中列出的下一个触发的探针）。arg0 是要写的缓冲区数量。arg1 是缓冲区的总数。arg2、arg3 和 arg4 分别包含了增加、删除和循环回收的 WAL 文件的数量。
clog-checkpoint-start	(bool)	当一个检查点的 CLOG 部分被开始时触发的探针。arg0 为真表示正常检查点，为假表示关闭检查点。
clog-checkpoint-done	(bool)	当一个检查点的 CLOG 部分完成时触发的探针。arg0 的含义与`clog-checkpoint-start`中相同。
subtrans-checkpoint-start	(bool)	当一个检查点的 SUBTRANS 部分被开始时触发的探针。arg0 为真表示正常检查点，为假表示关闭检查点。
subtrans-checkpoint-done	(bool)	当一个检查点的 SUBTRANS 部分完成时触发的探针。arg0 的含义与`subtrans-checkpoint-start`中相同。

<code>multixact-checkpoint-start</code>	<code>(bool)</code>	当一个检查点的 MultiXact 部分被开始时触发的探针。arg0 为真表示正常检查点，为假表示关闭检查点。
<code>multixact-checkpoint-done</code>	<code>(bool)</code>	当一个检查点的 MultiXact 部分完成时触发的探针。arg0 的含义与 <code>`multixact-checkpoint-start`</code> 中相同。
<code>buffer-checkpoint-start</code>	<code>(int)</code>	当一个检查点的写缓冲区部分被开始时触发的探针。arg0 传递位标志来区分不同的检查点类型，例如关闭（shutdown）、立即（immediate）或强制（force）。
<code>buffer-sync-start</code>	<code>(int, int)</code>	当我们在检查点期间开始写脏缓冲区时（在标识哪些缓冲区必须被写之后）触发的探针。arg0 是缓冲区总数，arg1 是当前为脏并且需要被写的缓冲区数量。
<code>buffer-sync-written</code>	<code>(int)</code>	在检查点期间当每个缓冲区被写完之后触发的探针。arg0 是缓冲区的 ID。
<code>buffer-sync-done</code>	<code>(int, int, int)</code>	当所有脏缓冲区被写之后触发的探针。arg0 是缓冲区总数。arg1 是检查点进程实际写的缓冲区数量。arg2 是期望写的数目（ <code>`buffer-sync-start`</code> 的 arg1）；arg1 和 arg2 的任何不同反映在该检查点期间有其他进程刷写了缓冲区。
<code>buffer-checkpoint-sync-start</code>	<code>()</code>	在脏缓冲区被写入到内核之后并且在开始发出 fsync 请求之前触发的探针。
<code>buffer-checkpoint-done</code>	<code>()</code>	当同步缓冲区到磁盘完成时触发的探针。
<code>twophase-checkpoint-start</code>	<code>()</code>	当一个检查点的两阶段部分被开始时触发的探针。
<code>twophase-checkpoint-done</code>	<code>()</code>	当一个检查点的两阶段部分完成时触发的探针。
<code>buffer-read-start</code>	<code>(ForkNumber, BlockNumber, Oid, Oid, Oid, int, bool)</code>	当一次缓冲区读被开始时触发的探针。arg0 和 arg1 包含该页的分叉号和块号（如果这是一次关系扩展请求，arg1 为 -1）。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。对于一个本地缓冲区，arg5 是创建临时关系的后端的 ID；对于一个共享缓冲区，arg5 是 InvalidBackendId (-1)。arg6 为真表示一次关系扩展请求，为假表示正常读。

buffer-read-done	(ForkNumber, BlockNumber, Oid, Oid, Oid, int, bool, bool)	当一次缓冲区读完成时触发的探针。 arg0 和 arg1 包含该页的分叉号和块号（如果这是一次关系扩展请求，arg1 现在包含新增加块的块号）。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。对于一个本地缓冲区，arg5 是创建临时关系的后端的 ID；对于一个共享缓冲区，arg5 是 InvalidBackendId (-1)。arg6 为真表示一次关系扩展请求，为假表示正常读。arg7 为真表示在池中找到该缓冲区，为假表示没有找到。
buffer-flush-start	(ForkNumber, BlockNumber, Oid, Oid, Oid)	在发出对一个共享缓冲区的任意写请求之前触发的探针。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。
buffer-flush-done	(ForkNumber, BlockNumber, Oid, Oid, Oid)	当一个写请求完成时触发的探针（注意这只反映传递数据给内核的时间，它通常并没有实际地被写入到磁盘）。参数和 `buffer-flush-start` 的相同。
buffer-write-dirty-start	(ForkNumber, BlockNumber, Oid, Oid, Oid)	当一个服务器进程开始写一个脏缓冲区时触发的探针（如果这经常发生，表示 shared_buffers 太小，或需要调整后台写入器的控制参数）。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。
buffer-write-dirty-done	(ForkNumber, BlockNumber, Oid, Oid, Oid)	当一次脏缓冲区写完成时触发的探针。参数与 `buffer-write-dirty-start` 相同。
wal-buffer-write-dirty-start	()	当一个服务器进程因为没有可用 WAL 缓冲区空间开始写一个脏 WAL 缓冲区时触发的探针（如果这经常发生，表示 wal_buffers 太小）。
wal-buffer-write-dirty-done	()	当一次脏 WAL 缓冲区完成时触发的探针。
wal-insert	(unsigned char, unsigned char)	当一个 WAL 记录被插入时触发的探针。arg0 是该记录的资源管理者（rmid）。arg1 包含 info 标志。
wal-switch	()	当请求一次 WAL 段切换时触发的探针。

smgr-md-read-start	(ForkNumber, BlockNumber, Oid, Oid, Oid, int)	当开始从一个关系读取一块时触发的探针。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。对一个本地缓冲区，arg5 是创建临时关系的后端的 ID；对于一个共享缓冲区，arg5 是 `InvalidBackendId` (-1)。
smgr-md-read-done	(ForkNumber, BlockNumber, Oid, Oid, Oid, int, int, int)	当一次块读取完成时触发的探针。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。对一个本地缓冲区，arg5 是创建临时关系的后端的 ID；对于一个共享缓冲区，arg5 是 `InvalidBackendId` (-1)。arg6 是实际读取的字节数，而 arg7 是请求读取的字节数（如果两者不同就意味着麻烦）。
smgr-md-write-start	(ForkNumber, BlockNumber, Oid, Oid, Oid, int)	当开始向一个关系中写入一个块时触发的探针。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 用以识别该关系。对一个本地缓冲区，arg5 是创建临时关系的后端的 ID；对于一个共享缓冲区，arg5 是 `InvalidBackendId` (-1)。
smgr-md-write-done	(ForkNumber, BlockNumber, Oid, Oid, Oid, int, int, int)	当一个块写操作完成时触发的探针。arg0 和 arg1 包含该页的分叉号和块号。arg2、arg3 和 arg4 包含表空间、数据库和关系 OID 来标识该关系。对于一个本地缓冲区，arg5 是创建临时关系的后端 ID；对于一个共享缓冲区，arg5 是 `InvalidBackendId` (-1)。arg6 是实际写的字节数，而 arg7 是要求写的字节数（如果这两者不同，则意味着麻烦）。
sort-start	(int, bool, int, int, bool, int)	当一次排序操作开始时触发的探针。arg0 指示是堆排序、索引排序或数据排序。arg1 为真表示唯一值强制。arg2 是键列的数目。arg3 是允许使用的工作内存数（以千字节计）。如果要求随机访问排序结果，那么 arg4 为真。arg5 为 `0` 时表示串行，为 `1` 时表示并行工作者，为 `2` 时表示并行领袖。

sort-done	(bool, long)	当一次排序完成时触发的探针。arg0 为真表示外排序，为假表示内排序。arg1 是用于一次外排序的磁盘块的数目，或用于一次内排序的以千字节计的内存。
lwlock-acquire	(char *, LWLockMode)	当成功获得一个 LWLock 时触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。arg1 所请求的锁模式，是排他或共享。
lwlock-release	(char *)	当一个 LWLock 被释放时（但是注意还没有唤醒任何一个被释放的等待者）触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。
lwlock-wait-start	(char *, LWLockMode)	当一个 LWLock 不是当即可用并且一个服务器进程因此开始等待该锁变为可用时触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。arg1 请求的锁模式，是排他或共享。
lwlock-wait-done	(char *, LWLockMode)	当一个进程从对一个 LWLock 的等待中被释放时（它实际还没有得到该锁）触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。arg1 所请求的锁模式，是排他或共享。
lwlock-condacquire	(char *, LWLockMode)	当调用者指定无需等待而成功获得一个 LWLock 时触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。arg1 所请求的锁模式，是排他或共享。
lwlock-condacquire-fail	(char *, LWLockMode)	当调用者指定无需等待而没有成功获得一个 LWLock 时触发的探针。arg0 是该 LWLock 所在的切片 (Tranche)。arg1 所请求的锁模式，是排他或共享。
lock-wait-start	(unsigned int, unsigned int, unsigned int, unsigned int, LOCKMODE)	当一个重量级锁 (lmgr 锁) 的请求由于锁不可用开始等待时触发的探针。arg0 到 arg3 是标识被锁定对象的标签域。arg4 指示被锁对象的类型。arg5 表示被请求的锁类型。
lock-wait-done	(unsigned int, unsigned int, unsigned int, unsigned int, LOCKMODE)	当一个重量级锁 (lmgr 锁) 的请求结束等待时（即已经得到锁）触发的探针。参数与 `lock-wait-start` 一样。
deadlock-found	()	当死锁检测器发现死锁时触发的探针。

表43.定义用在探针参数中的类型

类型	定义
LocalTransactionId	unsigned int

LWLockMode	int
LOCKMODE	int
BlockNumber	unsigned int
Oid	unsigned int
ForkNumber	int
bool	unsigned char

使用探针

下面的例子展示了一个分析系统中事务计数的 DTrace 脚本，可以用来代替一次性能测试之前和之后的 `pg\_stat\_database` 快照：

```
#!/usr/sbin/dtrace -qs

postgresql$1:::transaction-start
{
    @start["Start"] = count();
    self->ts = timestamp;
}

postgresql$1:::transaction-abort
{
    @abort["Abort"] = count();
}

postgresql$1:::transaction-commit
/self->ts/
{
    @commit["Commit"] = count();
    @time["Total time (ns)"] = sum(timestamp - self->ts);
    self->ts=0;
}
```

当被执行时，该例子 D 脚本给出这样的输出：

```
# ./txn_count.d `pgrep -n postgres` or ./txn_count.d <PID>
^C

Start                                71
Commit                              70
Total time (ns)                     2312105013
```

注意

SystemTap 为追踪脚本使用一个不同于 DTrace 的标记，但是底层的探针是兼容的。值得注意的是，在这样写的时候，SystemTap 脚本必须使用双下划线代替连字符来引用探针名。在未来的 SystemTap 发行中这很可能被修复。你应该记住，DTrace 脚本需要细心地编写和调试，否则被收集的追踪信息可能会毫无意义。在大部分发现问题的情况中，它就是发生问题的部件，而不是底层系统。当讨论使用动态追踪发现的信息时，一定要附上使用的脚本以便其也被检查和讨论。

定义新探针

开发者可以在代码中任意位置定义新的探针，当然这要重新编译之后才能生效。下面是插入新探针的步骤：

1. 决定探针名称以及探针可用的数据
2. 把该探针定义加入到 `src/backend/utils/probes.d`
3. 如果 `pg\_trace.h` 还不存在于包含该探针点的模块中，包括它，并且在源代码中期望的位置插入 `TRACE\_POSTGRESQL` 探针宏
4. 重新编译并验证新探针是可用的

例子：. 这里是一个如何增加一个探针来用事务 ID 追踪所有新事务的例子。

1. 决定探针将被命名为 `transaction-start` 并且需要一个 `LocalTransactionId` 类型的参数
2. 将该探针定义加入到 `src/backend/utils/probes.d`：

```
...  
  
probe transaction__start(LocalTransactionId);  
...
```

注意探针名字中双下划线的使用。在一个使用探针的 DTrace 脚本中，双下划线需要被替换为一个连字符，因此，对用户而言 `transaction-start` 是文档名。

3. 在编译时，**transaction__start** 被转换成一个宏调用 **TRACE_POSTGRESQL_TRANSACTION_START**（注意这里是单下划线），可以通过包括头文件 `pg\_trace.h` 获得。将宏调用加入到源代码中的合适位置。在这种情况下，看起来类似：

```
...  
  
TRACE_POSTGRESQL_TRANSACTION_START(vxid.localTransactionId);  
...
```

4. 在重新编译和运行新的二进制文件之后，通过运行下面的 DTrace 命令来检查新增的探针是否可用。你应该看到类似下面的输出：

```
...  
  
# dtrace -ln transaction-start  
   ID    PROVIDER      MODULE      FUNCTION NAME  
18705 postgresql49878 postgres  StartTransactionCommand transaction-start
```

```
18755 postgresql49877      postgres      StartTransactionCommand transaction-start
18805 postgresql49876      postgres      StartTransactionCommand transaction-start
18855 postgresql49875      postgres      StartTransactionCommand transaction-start
18986 postgresql49873      postgres      StartTransactionCommand transaction-start
...
```

向C代码中添加追踪宏时，有一些事情需要注意：

- 需要小心的是，为探针参数指定的数据类型要匹配宏中使用的变量的数据类型，否则会发生编译错误。
- 在大多数平台上，如果用`--enable-dtrace`编译了IvorySQL，无论何时当控制经过一个追踪宏时，都会评估该宏的参数，即使没有进行追踪也会这样做。通常不需要担心你是否只在报告一些局部变量的值。但是要注意不要将开销大的函数调用放入参数中。如果你需要这样做，考虑通过检查追踪是否真的被启用来保护该宏：

```
...
if (TRACE_POSTGRESQL_TRANSACTION_START_ENABLED())
    TRACE_POSTGRESQL_TRANSACTION_START(some_function(...));
...
```

每个追踪宏有一个对应的`ENABLED`宏。

监控磁盘的使用

判断磁盘用量

每个表都有一个主要的堆磁盘文件，大多数数据都存储在其中。如果一个表有着可能会很宽（尺寸大）的列，则另外还有一个TOAST文件与这个表相关联，它用于存储因为太宽而不能存储在主表里面的值。如果有这个附属文件，那么TOAST表上会有一个可用的索引。当然，同时还可能有索引和基表关联。每个表和索引都存放在单独的磁盘文件里 — 如果文件超过 1G 字节，甚至可能多于一个文件。

你可以以三种方式监视磁盘空间：使用oid2name模块或者人工观察系统目录。SQL函数是最容易使用的方法，同时也是我们通常推荐的方法。本节剩余的部分将展示如何通过观察系统目录来监视磁盘空间。

在一个最近清理过或者分析过的数据库上使用psql，你可以发出查询来查看任意表的磁盘用量：

```
SELECT pg_relation_filepath(oid), relpages FROM pg_class WHERE relname = 'customer';

pg_relation_filepath | relpages
-----+-----
base/16384/16806      |        60
(1 row)
```

每个页通常都是 8K 字节（记住，**relpages`只会由`VACUUM`、`ANALYZE`和少数几个 DDL 命令如`CREATE INDEX`所更新**）。如果你想直接检查表的磁盘文件，那么文件路径名应该有用。

要显示TOAST表使用的空间，我们可以使用一个类似下面这样的查询：

```
SELECT relname, relpages
FROM pg_class,
```

```

(SELECT reltoastrelid
 FROM pg_class
 WHERE relname = 'customer') AS ss
WHERE oid = ss.reltoastrelid OR
      oid = (SELECT indexrelid
              FROM pg_index
              WHERE indrelid = ss.reltoastrelid)
ORDER BY relname;

```

relname	relpages
pg_toast_16806	0
pg_toast_16806_index	1

你也可以很容易地显示索引的尺寸：

```

SELECT c2.relname, c2.relpages
FROM pg_class c, pg_class c2, pg_index i
WHERE c.relname = 'customer' AND
      c.oid = i.indrelid AND
      c2.oid = i.indexrelid
ORDER BY c2.relname;

```

relname	relpages
customer_id_index	26

我们很容易用下面的信息找出最大的表和索引：

```

SELECT relname, relpages
FROM pg_class
ORDER BY relpages DESC;

```

relname	relpages
bigtable	3290
customer	3144

磁盘满失败

一个数据库管理员最重要的磁盘监控任务就是确保磁盘不会写满。一个写满了的数据磁盘可能不会导致数据的崩溃，但它肯定会让系统变得不可用。如果保存 WAL 文件的磁盘变满，会发生数据库服务器致命错误并且可能发生关闭。

如果你不能通过删除一些其他的東西来释放一些磁盘空间，那么你可以通过使用表空间把一些数据库文件移

动到其他文件系统上去。

提示

有些文件系统在快满的时候性能会急剧恶化，因此不要等到磁盘完全满的时候才采取行动。

如果你的系统支持每用户的磁盘份额，那么数据库将自然地受制于用户所处的服务器给他的份额限制。超过份额的负面影响和完全用光磁盘是完全一样的。

日常维护

日常清理

IvorySQL数据库要求周期性的清理维护。对于很多安装，让自动清理守护进程来执行清理已经足够，你可能需要调整其中描述的自动清理参数来获得最佳结果。某些数据库管理员会希望使用手动管理的`VACUUM`命令来对后台进程的活动进行补充或者替换，这通常使用cron或任务计划程序脚本来执行。要正确地设置手动管理的清理。

清理的基础知识

IvorySQL的VACUUM命令出于几个原因必须定期处理每一个表：

1. 恢复或重用被已更新或已删除行所占用的磁盘空间。
2. 更新被IvorySQL查询规划器使用的数据统计信息。
3. 更新可见性映射，它可以加速只用索引的扫描。
4. 保护老旧数据不会由于事务ID回卷或多事务ID回卷而丢失。

正如后续小节中解释的，每一个原因都将指示以不同的频率和范围执行`VACUUM`操作。

有两种`VACUUM`的变体：标准`VACUUM`和`VACUUM FULL`。**VACUUM FULL`可以收回更多磁盘空间但是运行起来更慢。另外，标准形式的`VACUUM`可以和生产数据库操作并行运行（`SELECT`、`INSERT`、`UPDATE`和`DELETE`等命令将继续正常工作，但在清理期间你无法使用`ALTER TABLE`等命令来更新表的定义）。`VACUUM FULL`要求在其工作的表上得到一个`ACCESS EXCLUSIVE`锁，因此无法和对此表的其他使用并行。因此，通常管理员应该努力使用标准`VACUUM`并且避免`VACUUM FULL`。**

`VACUUM`会产生大量I/O流量，这将导致其他活动会话性能变差。可以调整一些配置参数来后台清理活动造成的性能冲击。

恢复磁盘空间

在IvorySQL中，一次行的`UPDATE`或`DELETE`不会立即移除该行的旧版本。这种方法对于从多版本并发控制获益是必需的：当旧版本仍可能对其他事务可见时，它不能被删除。但是最后，任何事务都不会再对一个过时的或者被删除的行版本感兴趣。它所占用的空间必须被回收来用于新行，这样可避免磁盘空间需求的无限制增长。这通过运行`VACUUM`完成。

`VACUUM`的标准形式移除表和索引中的死亡行版本并将该空间标记为可在未来重用。不过，它将不会把该空间交还给操作系统，除非在特殊的情况中表尾部的一个或多个页面变成完全空闲并且能够很容易地得到一个排他表锁。相反，`VACUUM FULL`通过把死亡空间之外的内容写成一个完整的新版本表文件来主动紧缩表。这将最小化表的尺寸，但是要花费较长的时间。它也需要额外的磁盘空间用于表的新副本，直到操作完成。

例行清理的一般目标是多做标准的`VACUUM`来避免需要`VACUUM FULL`。自动清理守护进程尝试这样工作，并且实际上永远不会发出`VACUUM FULL`。在这种方法中，其思想不是让表保持它们的最小尺寸，而是保持磁盘空间使用的稳定状态：每个表占用的空间等于其最小尺寸外加清理之间将使用的空间量。尽管`VACUUM FULL`可被用来把一个表收缩回它的最小尺寸并将该磁盘空间交还给操作系统，但是如果该表将在未来再次增长这样就没什么意义。因此，对于维护频繁被更新的表，适度运行标准`VACUUM`运行比少量运行`VACU

UM FULL`要更好。

一些管理员更喜欢自己计划清理，例如在晚上负载低时做所有的工作。根据一个固定日程来做清理的难点在于，如果一个表有一次预期之外的更新活动尖峰，它可能膨胀得真正需要`VACUUM FULL`来回收空间。使用自动清理守护进程可以减轻这个问题，因为守护进程会根据更新活动动态规划清理操作。除非你的负载是完全可以预估的，完全禁用守护进程是不理智的。一种可能的折中方案是设置守护进程的参数，这样它将只对异常的大量更新活动做出反应，因而保证事情不会失控，而在负载正常时采用有计划的`VACUUM`来做批量工作。

对于那些不使用自动清理的用户，一种典型的方法是计划一个数据库范围的`VACUUM`，该操作每天在低使用量时段执行一次，并根据需要辅以在重度更新表上的更频繁的清理（一些有着极高更新率的安装会每几分钟清理一次它们的最繁忙的表）。如果你在一个集群中有多个数据库，别忘记`VACUUM`每一个，你会用得上vacuumdb程序。

提示

当一个表因为大量更新或删除活动而包含大量死亡行版本时，纯粹的`VACUUM`可能不能令人满意。如果你有这样一张表并且你需要回收它占用的过量磁盘空间，你将需要使用`VACUUM FULL`，或者CLUSTER，或者ALTER TABLE的表重写变体之一。这些命令重写该表的一整个新拷贝并且为它构建新索引。所有这些选项都要求`ACCESS EXCLUSIVE`锁。注意它们也临时使用大约等于该表尺寸的额外磁盘空间，因为直到新表和索引完成之前旧表和索引都不能被释放。

提示

如果你有一个表，它的整个内容会被周期性删除，考虑用TRUNCATE而不是先用`DELETE`再用`VACUUM`。`TRUNCATE`会立刻移除该表的整个内容，而不需要一次后续的`VACUUM`或`VACUUM FULL`来回收现在未被使用的磁盘空间。其缺点是违背严格的MVCC语义。

更新规划器统计信息

IvorySQL查询规划器依赖于有关表内容的统计信息来为查询产生好的计划。这些统计信息由ANALYZE命令收集，它除了直接被调用之外还可以作为`VACUUM`的一个可选步骤被调用。拥有适度准确的统计信息很重要，否则差的计划可能降低数据库性能。

自动清理守护进程如果被启用，当一个表的内容被改变得足够多时，它将自动发出`ANALYZE`命令。不过，管理员可能更喜欢依靠手动的`ANALYZE`操作，特别是如果知道一个表上的更新活动将不会影响“感兴趣的”列的统计信息时。守护进程严格地按照一个被插入或更新行数的函数来计划`ANALYZE`，它不知道那是否将导致有意义的统计信息改变。

正如用于空间恢复的清理一样，频繁更新统计信息对重度更新的表更加有用。但即使对于一个重度更新的表，如果该数据的统计分布没有很大改变，也没有必要更新统计信息。一个简单的经验法则是考虑表中列的最大和最小值改变了多少。例如，一个包含行被更新时间的`timestamp`列将在行被增加和更新时有一直增加的最大值；这样一列将可能需要更频繁的统计更新，而一个包含一个网站上被访问页面URL的列则不需要。URL列可以经常被更改，但是其值的统计分布的变化相对很慢。

可以在指定表上运行`ANALYZE`甚至在表的指定列上运行，因此如果你的应用需要，可以更加频繁地更新某些统计。但实际上，通常只分析整个数据库是最好的，因为它是一种很快的操作。`ANALYZE`对一个表的行使用一种统计的随机采样，而不是读取每一个单一行。

提示

尽管对每列的`ANALYZE`频度调整可能不是非常富有成效，你可能会发现值得为每列调整被`ANALYZE`收集统计信息的详细程度。经常在`WHERE`中被用到的列以及数据分布非常不规则的列可能需要比其他列更细粒度的数据直方图。见`ALTER TABLE SET STATISTICS`，或者使用default_statistics_target配置参数改变数据库范围的默认值。

提示

自动清理守护进程不会为外部表发出`ANALYZE`命令，因为无法确定一个合适的频度。如果你的查询需要外部表的统计信息来正确地进行规划，比较好的方式是按照一个合适的时间表在那些表上手工运行`ANALYZE`命令。

提示

autovacuum守护进程不会对分区表发出ANALYZE命令。继承性父表只有在父表本身发生变化时才会被分析—
对于表的变化不会触发对父表的自动分析。如果你的查询需要对父表进行统计以进行正确的规划，那么有必要定期对这些表运行手动ANALYZE以保持统计的最新性。

更新可见性映射

清理机制为每一个表维护着一个可见性映射，它被用来跟踪哪些页面只包含对所有活动事务（以及所有未来的事务，直到该页面被再次修改）可见的元组。这样做有两个目的。第一，清理本身可以在下一次运行时跳过这样的页面，因为其中没有什么需要被清除。

第二，这允许IvorySQL回答一些只用索引的查询，而不需要引用底层表。因为IvorySQL的索引不包含元组的可见性信息，一次普通的索引扫描会为每一个匹配的索引项获取堆元组，用来检查它是否能被当前事务所见。另一方面，一次*只用索引的扫描*会首先检查可见性映射。如果它了解到在该页面上的所有元组都是可见的，堆获取就可以被跳过。这对大数据集很有用，因为可见性映射可以防止磁盘访问。可见性映射比堆小很多，因此即使堆非常大，可见性映射也可以很容易地被缓存起来。

防止事务ID回卷失败

IvorySQL的MVCC事务语义依赖于能够比较事务ID（XID）数字：如果一个行版本的插入XID大于当前事务的XID，它就是“属于未来的”并且不应该对当前事务可见。但是因为事务ID的尺寸有限（32位），一个长时间（超过40亿个事务）运行的集群会遭受到*事务ID回卷*问题：XID计数器回卷到0，并且本来属于过去的事务突然间就变成了属于未来—
这意味着它们的输出变成不可见。简而言之，灾难性的数据丢失（实际上数据仍然在那里，但是如果你不能得到它也无济于事）。为了避免发生这种情况，有必要至少每20亿个事务就清理每个数据库中的每个表。

周期性的清理能够解决该问题的原因是，**VACUUM`会把行标记为冻结，这表示它们是被一个在足够远的过去提交的事务所插入，这样从MVCC的角度来看，效果就是该插入事务对所有当前和未来事务来说当然都是可见的。IvorySQL保留了一个特殊的XID（`FrozenTransactionId`），这个XID并不遵循普通XID的比较规则并且总是被认为比任何普通XID要老。普通XID使用模-232算法来比较。这意味着对于每一个普通XID都有20亿个XID“更老”并且有20亿个“更新”，另一种解释的方法是普通XID空间是没有端点的环。因此，一旦一个行版本创建时被分配了一个特定的普通XID，该行版本将成为接下来20亿个事务的“过去”（与我们谈论的具体哪个普通XID无关）。如果在20亿个事务之后该行版本仍然存在，它将突然变得好像在未来。要阻止这一切发生，被冻结行版本会被看成其插入XID为`FrozenTransactionId`，这样它们对所有普通事务来说都是“在过去”，而不管回卷问题。并且这样的行版本将一直有效直到被删除，不管它有多旧。**

vacuum_freeze_min_age控制在其行版本被冻结前一个XID值应该有多老。如果被冻结的行将很快会被再次修改，增加这个设置可以避免不必要的工作。但是减少这个设置会增加在表必须再次被清理之前能够流逝的事务数。

`VACUUM`通常会跳过不含有任何死亡行版本的页面，但是不会跳过那些含有带旧XID值的行版本的页面。要保证所有旧的行版本都已经被冻结，需要对整个表做一次扫描。vacuum_freeze_table_age控制`VACUUM`什么时候这样做：如果该表经过`vacuum\_freeze\_table\_age`减去`vacuum\_freeze\_min\_age`个事务还没有被完全扫描过，则会强制一次全表清扫。将这个参数设置为0将强制`VACUUM`总是扫描所有页面而实际上忽略可见性映射。

一个表能保持不被清理的最长时间是20

亿个事务减去`VACUUM`上次扫描全表时的`vacuum\_freeze\_min\_age`值。如果它超过该时间没有被清理，可能会导致数据丢失。要保证这不会发生，将在任何包含比`autovacuum\_freeze\_max\_age`配置参数所指定的年龄更老的XID的未冻结行的表上调用自动清理（即使自动清理被禁用也会发生）。

这意味着如果一个表没有被清理，大约每`autovacuum\_freeze\_max\_age`减去`vacuum\_freeze\_min\_age`事务就会在该表上调用一次自动清理。对那些为了空间回收目的而被正常清理的表，这是无关紧要的。然而，对静态表（包括接收插入但没有更新或删除的表）就没有为空间回收而清理的需要，因此尝试在非常大的静态表上强制自动清理的间隔最大化会非常有用。显然我们可以通过增加`autovacuum\_freeze\_max\_age`或减少`vacuum\_freeze\_min\_age`来实现此目的。

`vacuum_freeze_table_age`的实际最大值是 0.95 *

`autovacuum_freeze_max_age`，高于它的设置将被上限到最大值。一个高于`autovacuum\_freeze\_max\_age`的值没有意义，因为不管怎样在那个点上都会触发一次防回卷自动清理，并且0.95的乘数为在防回卷自动清理发生之前运行一次手动`VACUUM`留出了一些空间。作为一种经验法则，`vacuum\_freeze\_table\_age`应当被设置成一个低于`autovacuum\_freeze\_max\_age`的值，留出一个足够的空间让一次被正常调度的`VACUUM`或一次被正常删除和更新活动触发的自动清理可以在这个窗口中被运行。将它设置得太接近可能导致防回卷自动清理，即使该表最近因为回收空间的目的被清理过，而较低的值将导致更频繁的全表扫描。

增加`autovacuum\_freeze\_max\_age`（以及和它一起的`vacuum\_freeze\_table\_age`）的唯一不足是数据库集群的`pg\_xact`和`pg\_commit\_ts`子目录将占据更多空间，因为它必须存储所有向后`autovacuum\_freeze\_max\_age`范围内的所有事务的提交状态和（如果启用了`track\_commit\_timestamp`）时间戳。提交状态为每个事务使用两个二进制位，因此如果`autovacuum\_freeze\_max\_age`被设置为它的最大允许值20亿，`pg\_xact`将会增长到大约0.5吉字节，`pg\_commit\_ts`大约20GB。如果这对于你的总数据库尺寸是微小的，我们推荐设置`autovacuum\_freeze\_max\_age`为它的最大允许值。否则，基于你想要允许`pg\_xact`和`pg\_commit\_ts`使用的存储空间大小来设置它（默认情况下2亿个事务大约等于`pg\_xact`的50MB存储空间，`pg\_commit\_ts`的2GB的存储空间）。

减小`vacuum\_freeze\_min\_age`的一个不足之处是它可能导致`VACUUM`做无用的工作：如果该行在被替换成`FrozenXID`之后很快就被修改（导致该行获得一个新的XID），那么冻结一个行版本就是浪费时间。因此该设置应该足够大，这样直到行不再可能被修改之前，它们都不会被冻结。

为了跟踪一个数据库中最老的未冻结XID的年龄，`VACUUM`在系统表`pg\_class`和`pg\_database`中存储XID的统计信息。特别地，一个表的`pg\_class`行的`relfrozenxid`列包含被该表的上一次全表`VACUUM`所用的冻结截止XID。该表中所有被有比这个截断XID老的普通XID的事务插入的行都确保被冻结。相似地，一个数据库的`pg\_database`行的`datfrozenxid`列是出现在该数据库中的未冻结XID的下界——它只是数据库中每一个表的`relfrozenxid`值的最小值。一种检查这些信息的方便方法是执行这样的查询：

```
SELECT c.oid::regclass as table_name,
       greatest(age(c.relfrozenxid),age(t.relfrozenxid)) as age
FROM pg_class c
LEFT JOIN pg_class t ON c.reltoastrelid = t.oid
WHERE c.relkind IN ('r', 'm');

SELECT datname, age(datfrozenxid) FROM pg_database;
```

`age`列度量从该截断XID到当前事务XID的事务数。

`VACUUM`通常只扫描从上次清理后备修改过的页面，但是只有当全表被扫描时`relfrozenxid`才能被推进。当`relfrozenxid`比`vacuum\_freeze\_table\_age`个事务还老时、当`VACUUM`的`FREEZE`选项被使用时或当所有页面正好要求清理来移除死亡行版本时，全表将被扫描。当`VACUUM`扫描全表时，在它被完成后，`age(relfrozenxid)`应该比被使用的`vacuum\_freeze\_min\_age`设置略大（比在`VACUUM`开始后开始的事务数多）。如果在`autovacuum\_freeze\_max\_age`被达到之前没有全表扫描`VACUUM`在该表上被发出，将很快为该表强制一次自动清理。

如果出于某种原因自动清理无法从一个表中清除旧的 XID，当数据库的最旧 XID 和回卷点之间达到 4 千万个事务时，系统将开始发出这样的警告消息：

```
WARNING: database "mydb" must be vacuumed within 39985967 transactions
HINT: To avoid a database shutdown, execute a database-wide VACUUM in that database.
```

（如该示意所建议的，一次手动的 `VACUUM` 应该会修复该问题；但是注意该次 `VACUUM` 必须由一个超级用户来执行，否则它将无法处理系统目录并且因而不能推进数据库的 `datfrozenxid`）。如果这些警告被忽略，一旦距离回卷点只剩下 3 百万个事务时，该系统将会关闭并且拒绝开始任何新的事务：

```
ERROR: database is not accepting commands to avoid wraparound data loss in database
"mydb"
HINT: Stop the postmaster and vacuum that database in single-user mode.
```

这 3

百万个事务的安全余量是为了让管理员能通过手动执行所要求的 `VACUUM` 命令进行恢复而不丢失数据。但是，由于一旦系统进入到安全关闭模式，它将不会执行命令。做这个操作的唯一方法是停止服务器并且以单一用户启动服务器来执行 `VACUUM`。单一用户模式中不会强制该关闭模式。

Multixact ID 被用来支持被多个事务锁定的行。由于在一个元组头部只有有限的空间可以用来存储锁信息，所以只要有多于一个事务并发地锁住一个行，锁信息将使用一个“多个事务 ID”（或简称多事务 ID）来编码。任何特定多事务 ID 中包括的事务 ID 的信息被独立地存储在 `pg_multixact` 子目录中，并且只有多事务 ID 出现在元组头部的 `xmax` 域中。和多事务 ID 类似，多事务 ID 也是用一个 32 位计数器实现，并且也采用了相似的存储，这些都要求仔细的年龄管理、存储清除和回卷处理。在每个多事务中都有一个独立的存储区域保存成员列表，它也使用一个 32 位计数器并且也应被管理。

在一次 `VACUUM` 表扫描（部分或者全部）期间，任何比 `vacuum_multixact_freeze_min_age` 要老的多事务 ID 会被替换为一个不同的值，该值可以是零值、一个单一事务 ID 或者一个更新的多事务 ID。对于每一个表，`pg_class.relmixmapid` 存储了在该表任意元组中仍然存在的最老可能多事务 ID。如果这个值比 `vacuum_multixact_freeze_table_age` 老，将强制一次全表扫描。可以在 `pg_class.relmixmapid` 上使用 `mxid_age()` 来找到它的年龄。

全表 `VACUUM` 扫描（不管是什么导致它们）将为表推进该值。最后，当所有数据库中的所有表被扫描并且它们的最老多事务值被推进，较老的多事务的磁盘存储可以被移除。

作为一种安全设备，对任何多事务年龄超过 `autovacuum_multixact_freeze_max_age` 的表，都将发生一次全表清理扫描。当多事务成员占用的存储超过 2GB 时，从那些具有最老多事务年龄的表开始，全表清理扫描也将逐步在所有表上进行。即使自动清理被在名义上被禁用，也会发生这两种主动扫描。

自动清理后台进程

IvorySQL 有一个可选的但是被高度推荐的特性 `autovacuum`，它的目的是自动执行 `VACUUM` 和 `ANALYZE` 命令。当它被启用时，自动清理会检查被大量插入、更新或删除元组的表。这些检查会利用统计信息收集功能，因此除非 `track_counts` 被设置为 `true`，自动清理不能被使用。在默认配置下，自动清理是被启用的并且相关配置参数已被正确配置。

“自动清理后台进程”实际上由多个进程组成。有一个称为自动清理启动器*的常驻后台进程，它负责为所有数据库启动*自动清理工作者*进程。启动器将把工作散布在一段时间上，它每隔 `autovacuum_naptime` 秒尝试在每个数据库中启动一个工作者（因此，如果安装中有 N 个数据库，则每 `autovacuum_naptime/N` 秒将启动一个新的工作者）。在同一时间只允许最多 `autovacuum_max_workers` 个工作者进程运行。如果有超过 `autovacuum_max_workers` 个数据库需要被处理，下一个数据库将在第一个工作者结束后马上被处理。每一个工作者进程将检查其数据库中的每一个表并且在需要时执行 `VACUUM` 和/或 `ANALYZE`。可以设置 `log_autovacuum_min_duration` 来监控自动清理工作者的活动。

如果在一小段时间内多个大型表都变得可以被清理，所有的自动清理工作者可能都会被占用来在一段长的时间内清理这些表。这将会造成其他的表 and 数据库无法被清理，直到一个工作者变得可用。对于一个数据库中的工作者数量并没有限制，但是工作者确实会试图避免重复已经被其他工作者完成的工作。注意运行着的工作者的数量不会被计入max_connections或superuser_reserved_connections限制。

`relfrozenxid` 值比 `autovacuum_freeze_max_age` 事务年龄更大的表总是会被清理（这页表示这些表的冻结最大年龄被通过表的存储参数修改过，参见后文）。否则，如果从上次 `VACUUM` 以来失效的元组数超过“清理阈值”，表也会被清理。清理阈值定义为：

清理阈值 = 清理基本阈值 + 清理缩放系数 * 元组数

其中清理基本阈值为 `autovacuum_vacuum_threshold`，
清理缩放系数为 `autovacuum_vacuum_scale_factor`，元组数为 `pg_class.rel tuples`。

如果自上次清理以来插入的元组数量超过了定义的插入阈值，表也会被清理，该阈值定义为：

清理插入阈值 = 清理基础插入阈值 + 清理插入缩放系数 * 元组数

清理插入基础阈值为 `autovacuum_vacuum_insert_threshold`，清理插入缩放系数为 `autovacuum_vacuum_insert_scale_factor`。这样的清理可以允许部分的表被标识为 `*all visible*`，并且也可以允许元组被冻结，可以减小后续清理的工作需要。

对于可以接收 `INSERT` 操作但是不能或几乎不能 `UPDATE` / `DELETE` 操作的表，可能会从降低表的 `autovacuum_freeze_min_age` 中受益，因为这可能允许元组在早期清理中被冻结。废弃元组的数量和插入元组的数量可从统计收集器中获得；它是一个半精确的计数，由每个 `UPDATE`、`DELETE` 和 `INSERT` 操作进行更新。（它只是半精确的，因为一些信息可能会在重负载情况下丢失。）

如果表的 `relfrozenxid` 值大于 `vacuum_freeze_table_age` 事务老的，执行一个主动的清理来冻结旧的元组，并推进 `relfrozenxid`；否则，只有上次清理以后修改过的页面被扫描。

对于分析，也使用了一个相似的阈值：

分析阈值 = 分析基本阈值 + 分析缩放系数 * 元组数

该阈值将与自从上次 `ANALYZE` 以来被插入、更新或删除的元组数进行比较。

临时表不能被自动清理访问。因此，临时表的清理和分析操作必须通过会话期间的SQL命令来执行。

默认的阈值和缩放系数都取自于 `postgresql.conf`，但是可以为每一个表重写它们(和许多其他自动清理控制参数)，详情参见Storage Parameters。

如果一个设置已经通过一个表的存储参数修改，那么在处理该表时使用该值，否则使用全局设置。

当多个工作者运行时，在所有运行着的工作者之间自动清理代价延迟参数是“平衡的”，这样不管实际运行的工作者数量是多少，对于系统的总体 I/O 影响总是相同的。不过，任何正在处理已经设置了每表 `autovacuum_vacuum_cost_delay` 或 `autovacuum_vacuum_cost_limit` 存储参数的表的工作者不会被考虑在均衡算法中。

`autovacuum` 工作进程通常不会阻止其他命令。如果某个进程尝试获取与 `autovacuum` 持有的 `SHARE UPDATE`

`EXCLUSIVE` 锁冲突的锁，则锁获取将中断该 `autovacuum`。有关冲突的锁定模式，但是，如果 `autovacuum` 正在运行以防止事务ID回卷（即在 `pg_stat_activity` 视图中的 `autovacuum` 查询名以 `(to prevent wraparound)` 结尾），则 `autovacuum` 不会被自动中断。

警告

定期运行需要获取与 `SHARE UPDATE`

EXCLUSIVE`锁冲突的锁的命令（例如ANALYZE）可能会让autovacuum始终无法完成。

日常重建索引

在某些情况下值得周期性地使用REINDEX命令或一系列独立重构步骤来重建索引。

已经完全变成空的B树索引页面被收回重用。但是，还是有一种低效的空间利用的可能性：如果一个页面上除少量索引键之外的全部键被删除，该页面仍然被分配。因此，在这种每个范围中大部分但不是全部键最终被删除的使用模式中，可以看到空间的使用是很差的。对于这样的使用模式，推荐使用定期重索引。

对于非B树索引可能的膨胀还没有很好地定量分析。在使用非B树索引时定期监控索引的物理尺寸是个好主意。

还有，对于B树索引，一个新建立的索引比更新了多次的索引访问起来要略快，因为在新建立的索引上，逻辑上相邻的页面通常物理上也相邻（这样的考虑目前并不适用于非B树索引）。仅仅为了提高访问速度也值得定期重索引。

REINDEX在所有情况下都可以安全和容易地使用。默认情况下，此命令需要一个`ACCESS EXCLUSIVE`锁，因此通常最好使用`CONCURRENTLY`选项执行它，该选项仅需要获取`SHARE UPDATE EXCLUSIVE`锁。

日志文件维护

把数据库服务器的日志输出保存在一个地方是个好主意，而不是仅仅通过`/dev/null`丢弃它们。在进行问题诊断的时候，日志输出是非常宝贵的。不过，日志输出可能很庞大（特别是在比较高的调试级别上），因此你不会希望无休止地保存它们。你需要轮转日志文件，这样在一段合理的时间后会开始新的日志文件并且移除旧的。

如果你简单地把`postgres`的stderr定向到一个文件中，你会得到日志输出，但是截断该日志文件的唯一方法是停止并重起服务器。这样做对于开发环境中使用的IvorySQL可能是可接受的，但是你肯定不想在生产环境上这么干。

一个更好的办法是把服务器的stderr输出发送到某种日志轮转程序里。我们有一个内建的日志轮转程序，你可以通过在`postgresql.conf`里设置配置参数`logging\_collector`为`true`的办法启用它。你也可以使用这种方法把日志数据捕捉成机器可读的CSV（逗号分隔值）格式。

另外，如果你已经使用的其他服务器软件中有一个外部日志轮转程序，你可能更喜欢使用它。比如，包含在Apache发布里的rotatelog工具就可以用于IvorySQL。要做到这一点，方法之一是把服务器的stderr用管道重定向到要用的程序。如果你用`pg\_ctl`启动服务器，那么stderr已经重定向到stdout，因此你只需要一个管道命令，比如：

```
pg_ctl start | rotatelog /var/log/pgsql_log 86400
```

您可以通过设置logrotate来收集由IvorySQL内置日志收集器生成的日志文件来组合这些方法。在这种情况下，日志收集器定义日志文件的名称和位置，而logrotate则定期归档这些文件。启动日志轮转时，logrotate必须确保应用程序将进一步的输出发送到新文件。这通常是通过`postrotate`脚本完成的，该脚本向应用程序发送`SIGHUP`信号，使其重新打开日志文件。在IvorySQL中，您可以使用`logrotate`选项运行`pg\_ctl`。服务器收到此命令后，服务器将切换到新的日志文件或重新打开现有文件，具体取决于日志记录配置。

注意

定期运行需要获取与`SHARE UPDATE EXCLUSIVE`锁冲突的锁的命令（例如ANALYZE）可能会让autovacuum始终无法完成使用静态日志文件名时，如果达到最大打开文件数限制或发生文件表溢出，则服务器可能无法重新打开日志文件。在这种情况下，日志消息将发送到旧的日志文件，直到成功进行日志轮转为止。如果将logrotate配置为压缩日志文件并将其删除，则服务器可能会丢失此时间范围内记录的消息。

为避免此问题，可以将日志收集器配置为动态分配日志文件名，并使用`prerotate`脚本忽略打开的日志文件。

另外一种生产级的管理日志输出的方法就是要把它们发送给syslog，让syslog处理文件轮转。要利用这个工具，我们需要设置`postgresql.conf`里的`log\_destination`配置参数设置为`syslog`（记录`syslog`日志）。然后在你想强迫syslog守护进程开始写入一个新日志文件的时候，你就可以发送一个`SIGHUP`信号给它。如果你想自动进行日志轮转，可以配置logrotate程序处理来自syslog的日志文件。

不过，在很多系统上，syslog不是非常可靠，特别是在面对大量日志消息的情况下；它可能在你最需要那些消息的时候截断或者丢弃它们。另外，在Linux，syslog会把每个消息刷写到磁盘上，这将导致很差的性能（你可以在syslog配置文件里面的文件名开头使用一个“-”来禁用这种行为）。

请注意上面描述的所有解决方案关注的是在可配置的间隔上开始一个新的日志文件，但它们并没有处理对旧的、不再需要的日志文件的删除。你可能还需要设置一个批处理任务来定期地删除旧日志文件。另一种可能的的方法是配置日志轮转程序，让它循环地覆盖旧的日志文件。

pgBadger

是一个外部项目，它可以进行日志文件的深度分析。check_postgres可在重要消息出现在日志文件中时向Nagios提供警告，也可以探测很多其他的特殊情况。

高可用、负载均衡和复制

不同方案的比较

共享磁盘故障转移

共享磁盘故障转移避免了只使用一份数据库拷贝带来的同步开销。它使用一个由多个服务器共享的单一磁盘阵列。如果主数据库服务器失效，后备服务器则可以挂载并启动数据库，就好像它从一次数据库崩溃中恢复过来了。这是一种快速的故障转移，并且不存在数据丢失。

共享硬件功能在网络存储设备中很常见。也可以使用一个网络文件系统，但是要注意的是该文件系统应具有完全的POSIX行为。这种方法的一个重大限制是如果共享磁盘阵列失效或损坏，主要和后备服务器都会变得无法工作。另一个问题是在主要服务器运行时，后备服务器永远不能访问共享存储。

文件系统（块设备）复制

共享硬件功能的一种修改版本是文件系统复制，在其中对一个文件系统的所有改变会被镜像到位于另一台计算机上的一个文件系统。唯一的限制是该镜像过程必须能保证后备服务器有一份该文件系统的一致性的拷贝——特别是对后备服务器的写入必须按照主控机上相同的顺序进行。DRBD是用于Linux的一种流行的文件系统复制方案。

预写式日志传送

温备和热备服务器能够通过读取一个预写式日志（WAL）记录的流来保持为当前状态。如果主服务器失效，后备服务器拥有主服务器的几乎所有数据，并且能够快速地被变成新的主数据库服务器。这可以是同步的或异步的，并且只能用于整个数据库服务器。

可以使用基于文件的日志传送、流复制或两者的组合来实现一个后备服务器。

逻辑复制

逻辑复制允许数据库服务器发送数据更新流给另一台服务器。IvorySQL逻辑复制从WAL构建出逻辑数据更新流。逻辑复制允许您逐个表复制数据更改。此外，发布数据更新的服务器可以同时订阅其他服务器的更改，从而允许数据在多个方向流动。第三方扩展也能提供类似的功能。

基于触发器的主-备复制

基于触发器的复制通常会将修改数据的查询发送到指定的主服务器。它在逐个表的基础上工作，主服务器（通常）将数据更改异步发送到备用服务器。

主服务器运行时，备用服务器可以响应查询，并执行本地数据修改或写入操作。这种形式的复制通常用于减轻大数据分析型平台或者数据仓库查询负荷。

Slony-

l是这种复制类型的一个例子。它使用表粒度，并且支持多个后备服务器。因为它会异步更新后备服务器（批量），在故障转移时可能会有数据丢失。

基于SQL的复制中间件

通过基于SQL的复制中间件，一个程序拦截每一个 SQL

查询并把它发送给一个或所有服务器。每一个服务器独立地操作。读写查询必须被发送给所有服务器，这样每一个服务器都能接收到任何修改。但只读查询可以被只发送给一个服务器，这样允许读负载在服务器之间分布。

如果查询未经修改发送，则函数的`random()`随机值和`CURRENT\_TIMESTAMP`函数的当前时间和序列值可能因不同服务器而异。因为每个服务器独立运行，并且它发送 SQL 查询而没有真正的更改数据。如果这是不可接受的，那么中间件或应用程序必须从单一服务器源确定此类值，并将结果用于写入查询。还必须注意确保所有服务器在提交或中止事务时都是相同的。这将涉及使用两阶段提交PREPARE TRANSACTION和COMMIT PREPARED。Pgpool-II和Continuent Tungsten就是这种复制的例子。

异步多主控机复制

对于不会被定期连接或通讯链路较慢的服务器，如笔记本或远程服务器，保持服务器间的数据一致是一个挑战。通过使用异步的多主控机复制，每一个服务器独立工作并且定期与其他服务器通信来确定冲突的事务。这些冲突可以由用户或冲突解决规则来解决。Bucardo 是这种复制类型的一个例子。

同步多主控机复制

在同步多主控机复制中，每一个服务器能够接受写请求，并且在每一个事务提交之前，被修改的数据会被从原始服务器传送给每一个其他服务器。繁重的写活动可能导致过多的锁定和提交延迟，进而导致很差的性能。读请求可以被发送给任意服务器。某些实现使用共享磁盘来减少通信负荷。同步多主控机复制主要对于读负载最好，尽管它的大优点是任意服务器都能接受写请求——没有必要在主服务器和后备服务器之间划分负载，并且因为数据修改被从一个服务器发送到另一个服务器，不会有非确定函数（如`random()`）的问题。

IvorySQL不提供这种复制类型，尽管在应用代码或中间件中可以使用 IvorySQL 的两阶段提交PREPARE TRANSACTION和COMMIT PREPARED来实现这种复制。

下表总结了上述多种方案的能力。

特性	共享磁盘	文件系统复制	预写式日志传送	逻辑复制	基于触发器的复制	SQL复制中间件	异步多主控机复制	同步多主控机复制
常用的示例	NAS	DRBD	内建流复制	内建逻辑复制, pglogical	Londiste, Slony	pgpool-II	Bucardo	
通信方法	共享磁盘	磁盘块	WAL	逻辑解码	表行	SQL	表行	表行和行锁
不要求特殊硬件		•	•	•	•	•	•	•
允许多个主控机服务器				•		•	•	•
无主服务器负载	•		•	•		•		
不等待多个服务器	•		with sync off	with sync off	•		•	
主控机失效将永不丢失数据	•	•	with sync on	with sync on		•		•

复制体接受只读查询			with hot	•	•	•	•	•
每个表粒度				•	•		•	•
不需要冲突解决	•	•	•		•	•		•

有一些方案不适合上述的类别：

- 数据分区

数据分区将表分开成数据集。每个集合只能被一个服务器修改。例如，数据可以根据办公室划分，如伦敦和巴黎，每一个办公室有一个服务器。如果查询有必要组合伦敦和巴黎的数据，一个应用可以查询两个服务器，或者可以使用主/备复制来在每一台服务器上保持其他办公室数据的一个只读拷贝。

- 多服务器并行查询执行

上述的很多方案允许多个服务器来处理多个查询，但是没有一个允许一个单一查询使用多个服务器来更快完成。这种方案允许多个服务器在一个单一查询上并发工作。这通常通过把数据在服务器之间划分并且让每一个服务器执行该查询中属于它的部分，然后将结果返回给一个中心服务器，由它整合结果并发回给用户。这也可以使用PL/Proxy工具集来实现这种方案。

日志传送后备服务器

规划

创建主服务器和后备服务器通常是明智的，因此它们可以尽可能相似，至少从数据库服务器的角度来看是这样。特别地，与表空间相关的路径名将被未经修改地传递，因此如果该特性被使用，主、备服务器必须对表空间具有完全相同的挂载路径。记住如果CREATE TABLESPACE在主服务器上被执行，在命令被执行前，它所需要的任何新挂载点必须在主服务器和所有后备服务器上先创建好。硬件不需要完全相同，但是经验显示，在应用和系统的生命期内维护两个相同的系统比维护两个不相似的系统更容易。在任何情况下硬件架构必须相同 — 从一个 32 位系统传送到一个 64 位系统将不会工作。

通常，不能在两个运行着不同主版本IvorySQL的服务器之间传送日志。IvorySQL全球开发组的策略是不在次版本升级中改变磁盘格式，因此在主服务器和后备服务器上运行不同次版本将会成功地工作。不过，在这方面并没有提供正式的支持，因此我们建议让主备服务器上运行的版本尽可能相同。当升级到一个新的次版本时，最安全的策略是先升级后备服务器 — 一个新的次版本发行更可能兼容从前一个次版本读取 WAL 文件。

后备服务器操作

服务器启动时，数据目录中存在 **standby.signal** 文件，服务器进入standby模式。

在后备模式中，服务器持续地应用从主控服务器接收到的 WAL。后备服务器可以从一个 WAL 归档restore_command或者通过一个 TCP 连接直接从主机（流复制）读取 WAL。后备服务器将也尝试恢复在后备集簇的`pg\_wal`目录中找到的 WAL。那通常在一次数据库重启后发生，那时后备机将在重启之前重播从主机流过来的 WAL，但是你也可以在任何时候手动拷贝文件到`pg\_wal`让它们被重播。

在启动时，后备机通过恢复归档位置所有可用的 WAL 来开始，这称为`restore\_command`。一旦它到达那里可用的 WAL 的末尾并且`restore\_command`失败，它会尝试恢复`pg\_wal`目录中可用的任何

WAL。如果那也失败并且流复制已被配置，后备机会尝试连接到主服务器并且从在归档或`pg\_wal`中找到的最后一个可用记录开始流式传送

WAL。如果那失败并且没有配置流复制，或者该连接后来断开，后备机会返回到步骤 1

并且尝试再次从归档里的文件恢复。这种尝试归档、`pg\_wal`和流复制的循环会一直重复知道服务器停止或者一个触发器文件触发了故障转移。

当`pg\_ctl promote`被运行，

pg_promote()被调用，或一个触发器文件被找到（**promote_trigger_file**），后备模式会退出并且服务器会切换到普通操作。在故障转移之前，在归档或`pg\_wal`中立即可用的任何 WAL 将被恢复，但不会尝试连接到主控机。

为后备服务器准备主控机

在主服务器上设置连续归档到一个后备服务器可访问的归档目录。即使主服务器垮掉该归档位置也应当是后备服务器可访问的，即它应当位于后备服务器本身或者另一个可信赖的服务器，而不是位于主控服务器上。

如果你想要使用流复制，在主服务器上设置认证来允许来自后备服务器的复制连接。即创建一个角色并且在`pg\_hba.conf`中提供一个或多个数据库域被设置为`replication`的项。还要保证在主服务器的配置文件中`max\_wal\_senders`被设置为足够大的值。如果要使用复制槽，请确保`max\_replication\_slots`也被设置得足够高。

建立一个后备服务器

要建立后备服务器，恢复从主服务器取得的基础备份。在后备服务器的集簇数据目录中创建一个文件standby.signal。将restore_command设置为一个从 WAL 归档中复制文件的简单命令。

如果你计划为了高可用性目的建立多个后备服务器，确认`recovery\_target\_timeline`被设置为`latest`（默认）来使得该后备服务器遵循发生在故障转移到另一个后备服务器之后发生的时间线改变。

注意

restore_command应该立即返回，如果必要该服务器将再次尝试该命令。

如果你想要使用流复制，在primary_conninfo中填入一个 libpq 连接字符串，其中包括主机名（或 IP 地址）和连接到主服务器所需的任何附加细节。如果主服务器需要一个口令用于认证，口令也应该被指定primary_conninfo中。

如果你正在为高性能目的建立后备服务器，像主服务器一样建立 WAL 归档、连接和认证，因为在故障转移后该后备服务器将作为一个主服务器工作。

如果你正在使用一个 WAL

归档，可以使用archive_cleanup_command参数来移除后备服务器不再需要的文件，这样可以最小化 WAL 归档的尺寸。pg_archivecleanup工具被特别设计为在典型单一后备配置下与`archive\_cleanup\_command`共同使用，见pg_archivecleanup。不过要注意，如果你正在为备份目的使用归档，有一些文件即使后备服务器不再需要你也需要保留它们，它们被用来从至少最后一个基础备份恢复。

配置的一个简单例子是：

```
primary_conninfo = 'host=192.168.1.50 port=5432 user=foo password=foopass options='-c wal_sender_timeout=5000''
restore_command = 'cp /path/to/archive/%f %p'
archive_cleanup_command = 'pg_archivecleanup /path/to/archive %r'
```

你可以有任意数量的后备服务器，但是如果你使用流复制，确保你在主服务器上将`max\_wal\_senders`设置得足够高，这样可以允许它们能同时连接。

流复制允许一台后备服务器比使用基于文件的日志传送更能保持为最新的状态。后备服务器连接到主服务器，主服务器则在 WAL 记录产生时即将它们以流式传送给后备服务器而不必等到 WAL 文件被填充。

默认情况下流复制是异步的，在这种情况下主服务器上提交一个事务与该变化在后备服务器上变得可见之间存在短暂的延迟。不过这种延迟比基于文件的日志传送方式中要小得多，在后备服务器的能力足以跟得上负载的前提下延迟通常低于一秒。在流复制中，不需要 `archive\_timeout` 来缩减数据丢失窗口。

如果你使用的流复制没有基于文件的连续归档，该服务器可能在后备机收到 WAL 段之前回收这些旧的 WAL 段。如果发生这种情况，后备机将需要重新从一个新的基础备份初始化。通过设置 `wal\_keep\_size` 为一个足够高的值来确保旧的 WAL 段不会被太早重用或者为后备机配置一个复制槽，可以避免发生这种情况。如果设置了一个后备机可以访问的 WAL 归档，就不需要这些解决方案，因为该归档可以为后备机保留足够的段，后备机总是可以使用该归档来追赶主机。

要使用流复制，建立一个基于文件的日志传送后备服务器。将一个基于文件日志传送后备服务器转变成流复制后备服务器的步骤是把 `recovery.conf` 文件中的设置以指向主服务器。设置主服务器上的 `listen\_addresses` 和认证选项（见 `pg\_hba.conf`），这样后备服务器可以连接到主服务器上的伪数据库 `replication`。

在支持 keepalive

套接字选项的系统上，设置 `tcp\_keepalives\_idle`、`tcp\_keepalives\_interval` 和 `tcp\_keepalives\_count` 有助于主服务器迅速地注意到一个断开的连接。

设置来自后备服务器的并发连接的最大数目（详见 `max\_wal\_senders`）。

当后备服务器被启动并且 `primary\_conninfo` 被正确设置，后备服务器将在重放完归档中所有可用的 WAL 文件之后连接到主服务器。如果连接被成功建立，你将在后备服务器中看到一个 **walreceiver**，并且在主服务器中有一个相应的 **walsender** 进程。

认证

设置好用于复制的访问权限非常重要，这样只有受信的用户可以读取 WAL 流，因为很容易从 WAL 流中抽取需要特权才能访问的信息。

后备服务器必须作为一个具有 `REPLICATION` 特权的账户或一个超级用户来向主服务器认证。

推荐为复制创建一个专用的具有 `REPLICATION` 和 `LOGIN` 特权的用户账户。

虽然 `REPLICATION` 特权给出了非常高的权限，但它不允许用户修改主系统上的任何数据，而 `SUPERUSER` 特权则可以。

复制的客户端认证由一个在 ***database*** 域中指定 `replication` 的 `pg\_hba.conf` 记录控制。例如，如果后备服务器运行在主机 IP

192.168.1.100 并且用于复制的账户名为 **foo**，管理员可以在主服务器上向 `pg\_hba.conf` 文件增加下列行：

```
# 允许来自 192.168.1.100 的用户 "foo" 在提供了正确的口令时作为一个
# 复制后备机连接到主控机。
#
# TYPE  DATABASE  USER  ADDRESS          METHOD
host    replication  foo    192.168.1.100/32 md5
```

主服务器的主机名和端口号、连接用户名和口令在 `primary\_conninfo` 中指定。在后备服务器上还可以在 `~/.pgpass` 文件中设置口令（在 ***database*** 域中指定 `replication`）。例如，如果主服务器运行在主机 IP **192.168.1.50**、端口 **5432** 上，并且口令为 **foopass**，管理员可以在后备服务器的 `postgresql.conf` 文件中增加下列行：

```
# 后备机要连接到的主控机运行在主机 192.168.1.50 上，
# 端口号是 5432，连接所用的用户名是 "foo"，口令是 "foopass"。
```

```
primary_conninfo = 'host=192.168.1.50 port=5432 user=foo password=foopass'
```

监控

流复制的一个重要健康指标是在主服务器上产生但还没有在后备服务器上应用的 WAL 记录数。你可以通过比较主服务器上的当前 WAL 写位置和后备服务器接收到的最后一个 WAL 位置来计算这个滞后量。这些位置分别可以用主服务器上的`pg\_current\_wal\_lsn`和后备服务器上的`pg\_last\_wal\_receive\_lsn`来检索。后备服务器的最后 WAL 接收位置也被显示在 WAL 接收者进程的进程状态中，即使用`ps`命令显示的状态。

你可以通过`pg\_stat\_replication`视图检索 WAL 发送者进程的列表。
`pg\_current\_wal\_lsn`与`sent\_lsn`域之间的巨大差异表示主服务器承受着巨大的负载，而`sent\_lsn`和后备服务器上`pg\_last\_wal\_receive\_lsn`之间的差异可能表示网络延迟或者后备服务器正承受着巨大的负载。

在一台热后备上，WAL接收者进程的状态可以通过`pg\_stat\_wal\_receiver`视图检索到。
`pg\_last\_wal\_replay\_lsn`和该视图的`flushed\_lsn`的差别表示WAL的接收速度大于它被重放的速度。

复制槽

复制槽提供了一种自动化的方法来确保主控机在所有的后备机收到 WAL 段之前不会移除它们，并且主控机也不会移除可能导致恢复冲突的行，即使后备机断开也是如此。

作为复制槽的替代，也可以使用`wal\_keep\_size`阻止移除旧的 WAL 段，或者使用`archive\_command`把段保存在一个归档中。不过，这些方法常常会导致保留的 WAL 段比需要的更多，而复制槽只保留已知所需要的段。
另一方面，复制槽可以保留很多的WAL段以至于它们填满了分配给`pg\_wal`的空间；
`max\_slot\_wal\_keep\_size`限制复制槽所保留的WAL文件的大小。

类似地，`hot\_standby\_feedback`和`vacuum\_defer\_cleanup\_age`保护了相关行不被`vacuum`移除，但是前者在后备机断开期间无法提供保护，而后者则需要被设置为一个很高的值已提供足够的保护。复制槽克服了这些缺点。

查询和操纵复制槽

每个复制槽都有一个名字，名字可以包含小写字母、数字和下划线字符。

已有的复制槽和它们的状态可以在`pg\_replication\_slots`视图中看到。

槽可以通过流复制协议 或者 SQL 函数创建并且移除。

配置实例

你可以这样创建一个复制槽：

```
postgres=# SELECT * FROM pg_create_physical_replication_slot('node_a_slot');
 slot_name | lsn
-----+-----
 node_a_slot |

postgres=# SELECT slot_name, slot_type, active FROM pg_replication_slots;
 slot_name | slot_type | active
-----+-----+-----
 node_a_slot | physical  | f
(1 row)
```

要配置后备机使用这个槽，在后备机中应该配置`primary\_slot\_name`。这里是一个简单的例子：

```
primary_conninfo = 'host=192.168.1.50 port=5432 user=foo password=foopass'
primary_slot_name = 'node_a_slot'
```

级联复制

级联复制特性允许一台后备服务器接收复制连接并且把 WAL 记录流式传送给其他后备服务器，就像一个转发器一样。这可以被用来减小对于主控机的直接连接数并且使得站点间的带宽开销最小化。

一台同时扮演着接收者和发送者角色的后备服务器被称为一台级联后备服务器。“更直接”（通过更少的级联后备服务器）连接到主控机的后备服务器被称为上游服务器，而那些离得更远的后备服务器被称为下游服务器。级联复制并没有对下游服务器的数量或布置设定限制。

一台级联后备服务器不仅仅发送从主控机接收到的 WAL 记录，还要发送那些从归档中恢复的记录。因此即使某些上游连接中的复制连接被中断，只要还有新的 WAL 记录可用，下游的流复制都会继续下去。

级联复制目前是异步的。同步复制设置当前对级联复制无影响。

不管在什么样的级联布置中，热备反馈都会向上游传播。

如果一台上游后备服务器被提升为新的主控机，且下游服务器的`recovery\_target\_timeline`被设置成`latest`（默认），下游服务器将继续从新的主控机得到流。

要使用级联复制，要建立级联后备服务器让它能够接受复制连接（即设置`max\_wal\_senders`和`hot\_standby`，并且配置基于主机的认证）。你还将需要设置下游后备服务器中的`primary\_conninfo`指向级联后备服务器。

同步复制

PostgreSQL流复制默认是异步的。如果主服务器崩溃，则某些已被提交的事务可能还没有被复制到后备服务器，这会导致数据丢失。数据的丢失量与故障转移时的复制延迟成比例。

同步复制能够保证一个事务的所有修改都能被传送到一台或者多台同步后备服务器。这扩大了由一次事务提交所提供的标准持久化级别。在计算机科学理论中这种保护级别被称为 2-safe 复制。而当`synchronous\_commit`被设置为`remote\_write`时，则是 group-1-safe（group-safe 和 1-safe）。

在请求同步复制时，一个写事务的每次提交将一直等待，直到收到一个确认表明该提交在主服务器和后备服务器上都被写入到磁盘上的预写式日志中。数据会被丢失的唯一可能性是主服务器和后备服务器在同一时间都崩溃。这可以提供更高级别的持久性，尽管只有系统管理员要关系两台服务器的放置和管理。等待确认提高了用户对于修改不会丢失的信心，但是同时也不必要地增加了对请求事务的响应时间。最小等待时间是在主服务器和后备服务器之间的来回时间。

只读事务和事务回滚不需要等待后备服务器的回复。子事务提交也不需要等待后备服务器的响应，只有顶层提交才需要等待。长时间运行的动作（如数据载入或索引构建）不会等待最后的提交消息。所有两阶段提交动作要求提交等待，包括预备和提交。

同步后备可以是物理复制后备或者是逻辑复制订阅者。它还可以是任何其他物理或者逻辑WAL复制流的消费者，它懂得如何发送恰当的反馈消息。除内建的物理和逻辑复制系统之外，还包括`pg\_receivewal`和`pg\_recvlogical`之类的特殊程序，以及一些第三方复制系统和定制程序。同步复制支持的细节请查看相应的文档。

基本配置

一旦流复制已经被配置，配置同步复制就只需要一个额外的配置步骤：`synchronous\_standby\_names`必须被设置为一个非空值。**`synchronous_commit`也必须被设置为`on`**，但由于这是默认值，通常不需要改变。这样的配置将导致每一次提交都等待确认消息，以保证后备服务器已经将提交记录写入到持久化存储中。`synchron

`onous_commit``可以由个体用户设置，因此它可以在配置文件中配置、可以为特定用户或数据库配置或者由应用动态配置，这样可以在一种每事务基础上控制持久性保证。

在一个提交记录已经在主服务器上被写入到磁盘后，WAL 记录接着被发送到后备服务器。每次一批新的 WAL 数据被写入到磁盘后，后备服务器会发送回复消息，除非在后备服务器上 `wal_receiver_status_interval`` 被设置为零。如果 `synchronous_commit`` 被设置为 `remote_apply``，当提交记录被重放时后备服务器会发送回应消息，这会让该事务变得可见。如果根据主服务器的 `synchronous_standby_names`` 设置选中该后备服务器作为一个同步后备，将会根据来自该后备服务器和其他同步后备的回应消息来决定何时释放正在等待确认提交记录被收到的事务。这些参数允许管理员指定哪些后备服务器应该是同步后备。注意同步复制的配置主要在主机上。命名的后备服务器必须直接连接到主机，主机对使用级联复制的下游后备服务器一无所知。

将 `synchronous_commit`` 设置为 `remote_write`` 将导致每次提交都等待后备服务器已经接收提交记录并将它写出到其自身所在的操作系统的确认，但并非等待数据都被刷出到后备服务器上的磁盘。这种设置提供了比 `on`` 要弱一点的持久性保障：在一次操作系统崩溃事件中后备服务器可能丢失数据，尽管它不是一次 Ivory SQL 崩溃。不过，在实际中它是一种有用的设置，因为它可以减少事务的响应时间。只有当主服务器和后备服务器都崩溃并且主服务器的数据库同时被损坏的情况下，数据丢失才会发生。

把 `synchronous_commit`` 设置为 `remote_apply`` 将导致每一次提交都会等待，直到当前的同步后备服务器报告说它们已经重放了该事务，这样就会使该事务对用户查询可见。在简单的情况下，这为带有因果一致性的负载均衡留出了余地。

如果请求一次快速关闭，用户将停止等待。不过，在使用异步复制时，在所有未解决的 WAL 记录被传输到当前连接的后备服务器之前，服务器将不会完全关闭。

多个同步后备

同步复制支持一个或者更多个同步后备服务器，事务将会等待，直到所有同步后备服务器都确认收到了它们的数据为止。事务必须等待其回复的同步后备的数量由 `synchronous_standby_names`` 指定。这个参数还指定一个后备服务器名称及方法（**FIRST`** 和 **ANY`**）的列表来从列出的后备中选取同步后备。

方法 `FIRST`` 指定一种基于优先的同步复制并且让事务提交等待，直到它们的 WAL 记录被复制到基于优先级选中的所要求数量的同步后备上为止。在列表中出现较早的后备被给予较高的优先级，并且将被考虑为同步后备。其他在这个列表中位置靠后的后备服务器表示可能的同步后备。如果任何当前的同步后备由于任何原因断开连接，它将立刻被下一个最高优先级的后备所替代。

基于优先的多同步后备的 `synchronous_standby_names`` 示例为：

```
synchronous_standby_names = 'FIRST 2 (s1, s2, s3)'
```

在这个例子中，如果有四个后备服务器 `s1``、**`s2``**、`s3`` 和 `s4`` 在运行，两个后备服务器 `s1`` 和 `s2`` 将被选中为同步后备，因为它们出现在后备服务器名称列表的前部。`s3`` 是一个潜在的同步后备，当 `s1`` 或 `s2`` 中的任何一个失效，它就会取而代之。`s4`` 则是一个异步后备因为它的名字不在列表中。

方法 `ANY`` 指定一种基于规定数量的同步复制并且让事务提交等待，直到它们的 WAL 记录至少被复制到列表中所要求数量的同步后备上为止。

`synchronous_standby_names`` 的基于规定数量的多同步后备的例子：

```
synchronous_standby_names = 'ANY 2 (s1, s2, s3)'
```

在这个例子中，如果有四台后备服务器 `s1``、**`s2``**、`s3`` 以及 `s4`` 正在运行，事务提交将会等待来自至少其中任意两台后备服务器的回复。`s4`` 是一台异步后备，因为它的名字不在该列表中。

后备服务器的同步状态可以使用 `pg_stat_replication`` 视图查看。

同步复制通常要求仔细地规划和放置后备服务器来保证应用能令人满意地工作。等待并不利用系统资源，但是事务锁会持续保持直到传输被确认。其结果是，不小心使用同步复制将由于响应时间增加以及较高的争用率而降低数据库应用的性能。

IvorySQL允许应用开发者通过复制来指定所要求的持久性级别。这可以为整个系统指定，不过它也能够为特定的用户或连接指定，甚至还可以为单个事务指定。

例如，一个应用的载荷的组成可能是这样：10%的改变是重要的客户详情，而90%的改变是不太重要的数据，即使它们丢失业务也比较容易容忍（例如用户间的聊天消息）。

通过在应用级别（在主服务器上）指定的同步复制选项，我们可以为大部分重要的改变提供同步复制，并且不会拖慢整体的载荷。应用级别选项是使高性能应用享受同步复制的一种重要和实用的工具。

你应该认为网络带宽必须比 WAL 数据的产生率高。

高可用性规划

当 `synchronous_commit` 被设置为 `on`、`remote_apply` 或者 `remote_write` 时，`synchronous_standby_names` 指定产生的事务提交要等待其回应的同步后备的数量和名称。如果任一同步后备崩溃，这类事务提交可能无法完成。

高可用的最佳方案是确保有所要求数量的同步后备。这可以通过使用 `synchronous_standby_names` 指定多个潜在后备服务器来实现。

在基于优先的同步复制中，出现在该列表前部的后备服务器将被用作同步后备。后面的后备服务器将在当前同步后备服务器失效时取而代之。

在基于规定数量的同步复制中，所有出现在该列表中的后备服务器都将被用作同步后备的候选。即使其中的一个失效，其他后备仍将继续担任候选同步后备的角色。

当一台后备服务器第一次附加到主服务器时，它将处于一种还没有正确同步的状态。这被描述为 `追赶` 模式。一旦后备服务器和主服务器之间的迟滞第一次变成零，我们就来到了实时的 `流式` 状态。在后备服务器被创建之后的很长一段时间内可能都是追赶模式。如果后备服务器被关闭，则追赶周期将被增加，增加量由后备服务器被关闭的时间长度决定。只有当后备服务器到达 `流式` 状态后，它才能成为一台同步后备。这种状态可以使用 `pg_stat_replication` 视图查看。

如果在提交正在等待确认时主服务器重启，那些正在等待的事务将在主数据库恢复时被标记为完全提交。没有办法确认所有后备服务器已经收到了在主服务器崩溃时所有还未处理的 WAL 数据。某些事务可能不会在后备服务器上显示为已提交，即使它们在主服务器上显示为已提交。我们提供的保证是：在 WAL 数据已经被所有后备服务器安全地收到之前，应用将不会收到一个事务成功提交的显式确认。

如果实在无法保持所要求数量的同步后备，那么应该减少 `synchronous_standby_names` 中指定的事务提交应该等待其回应的同步后备的数量（或者禁用），并且在主服务器上重载配置文件。

如果主服务器与剩下的后备服务器是隔离的，你应当故障转移到那些其他剩余后备服务器中的最佳候选者上。

如果在事务等待时你需要重建一台后备服务器，确保命令 `pg_start_backup()` 和 `pg_stop_backup()` 被运行在一个 `synchronous_commit = 'off'` 的会话中，否则那些请求将永远等待后备服务器出现。

在后备机上连续归档

当在一个后备机上使用连续归档时，有两种不同的情景：WAL 归档在主服务器和后备机之间共享，或者后备机有自己的 WAL 归档。当后备机拥有其自身的 WAL 归档时，将 `archive_mode` 设置为 **always**，后备机将在收到每个 WAL 段时调用归档命令，不管它是从归档恢复还是使用流复制恢复。共享归档可以类似地处理，但是 `archive_command` 必须测试要被归档的文件是否已经存在，以及现有的文件是否有相同的内容。这要求 `archive_command` 中有更多处理，因为它必须当心不要覆盖具有不同内容的已有文件，但是如果完全相同的文件被归档两次时应返回成功。并且如果两个服务器尝试同时归档同一个文件，所有这些都必须

在没有竞争情况的前提下完成。

如果`archive\_mode`被设置为`on`，归档器在恢复或者后备模式中无法启用。

如果后备服务器被提升，它将在被提升后开始归档，但是它将不会归档任何不是它自身产生的WAL或时间线历史文件。要在归档中得到完整的一系列WAL文件，你必须确保所有WAL在到达后备机之前都被归档。

对于基于文件的日志传输来说天然就是这样，因为后备机只能恢复在归档中找到的文件，而启用了流复制时则不是这样。当一台服务器不在恢复模式中时，在`on`和`always`模式之间没有差别。

故障转移

如果主服务器失效，则后备服务器应该开始故障转移过程。

如果后备服务器失效，则不会有故障转移发生。如果后备服务器可以被重启（即使晚一点），由于可重启恢复的优势，那么恢复处理也能被立即重启。如果后备服务器不能被重启，则一个全新的后备服务器实例应该被创建。

如果主服务器失效并且后备服务器成为了新的主服务器，那么接下来旧的主服务器重启后，你必须有一种机制来通知旧的主服务器不再成为主服务器。有些时候这被称为STONITH（Shoot The Other Node In The Head，关闭其他节点），这对于避免出现两个系统都认为它们是主服务器的情况非常必要，那种情况将导致混乱并且最终导致数据丢失。

很多故障转移系统仅使用两个系统，主系统和后备系统，它们由某种心跳机制连接来持续验证两者之间的连接性和主系统的可用性。也可能会使用第三个系统（称为目击者服务器）来防止某些不当故障转移的情况，但是除非非常小心地建立它并且经过了严格地测试，额外的复杂度可能会使该工作得不偿失。

IvorySQL并不提供在主服务器上标识失败并且通知后备数据库服务器所需的系统软件。现在已有很多这样的工具并且很好地与成功的故障转移所需的操作系统功能整合在一起，例如IP地址迁移。

一旦发生到后备服务器的故障转移，就只有单一的一台服务器在操作。这被称为一种退化状态。之前的后备服务器现在是主服务器，但之前的主服务器处于关闭并且可能一直保持关闭。要回到正常的操作，一个后备服务器必须被重建，要么在之前的主系统起来时使用它重建，要么使用第三台（可能是全新的）服务器来重建。在大型集群上，pg_rewind功能可以被用来加速这个过程。一旦完成，主服务器和后备服务器可以被认为是互换了角色。某些人选择使用第三台服务器来为新的主服务器提供备份，直到新的后备服务器被重建，不过显然这会使得系统配置和操作处理更复杂。

因此，从主服务器切换到后备服务器可以很快，但是要求一些时间来重新准备故障转移集群。从主服务器到后备服务器的常规切换是有用的，因为它允许每个系统有常规的关闭时间来进行维护。这也可以作为一种对故障转移机制的测试，以保证在你需要它时它真地能够工作。我们推荐写一些管理过程来做这些事情。

要触发一台日志传送后备服务器的故障转移，运行`pg\_ctl promote`，调用

pg_promote()，或者创建一个触发器文件，其文件名和路径由`promote\_trigger\_file`设置指定。

如果你正在规划使用`pg_ctl

promote`或调用`pg_promote()`以进行故障转移，`promote_trigger_file`就不是必要的。

如果你正在建立只用于从主服务器分流只读查询而不是高可用性目的的只读服务器，你不需要提升它。

热备

术语热备用来描述处于归档恢复或后备模式中的服务器连接到服务器并运行只读查询的能力。这有助于复制目的以及以高精度恢复一个备份到一个期望的状态。术语热备也指服务器从恢复转移到正常操作而用户能继续运行查询并且保持其连接打开的能力。

在热备模式中运行查询与正常查询操作相似，尽管如下所述存在一些用法和管理上的区别。

用户概览

当hot_standby参数在一台后备服务器上被设置为真时，一旦恢复将系统带到一个一致的状态它将开始接受连接。所有这些连接都被限制为只读，甚至临时表都不能被写入。

后备服务器上的数据需要一些时间从主服务器到达后备服务器，因此在主服务器和后备服务器之间将有一段可以度量的延迟。近乎同时在主服务器和后备服务器上运行相同的查询可能因此而返回不同的结果。我们说后备服务器上的数据与主服务器是*最终一致*的。一旦一个事务的提交记录在后备服务器上被重播，那个事

务所作的修改将对后备服务器上所有新取得的快照可见。快照可以在每个查询或每个事务的开始时取得，这取决于当前的事务隔离级别。

在热备期间开始的事务可能发出下列命令：

- 查询访问: **SELECT**、**COPY TO**
- 游标命令: **DECLARE**、**FETCH**、**CLOSE**
- 设置: **SHOW**、**SET**、**RESET**
- 事务管理命令:
 - **BEGIN**、**END**、**ABORT**、**START TRANSACTION**
 - **SAVEPOINT**、**RELEASE**、**ROLLBACK TO SAVEPOINT**
 - ``EXCEPTION`` 块或其他内部子事务
- **LOCK TABLE**，不过只在下列模式之一中明确发出: **ACCESS SHARE**、**ROW SHARE** 或 **ROW EXCLUSIVE**.
- 计划和资源: **PREPARE**、**EXECUTE**、**DEALLOCATE**、**DISCARD**
- 插件和扩展: **LOAD**
- **UNLISTEN**

在热备期间开始的事务将不会被分配一个事务 ID 并且不能被写入到系统的预写式日志。因此，下列动作将产生错误消息：

- 数据操纵语言 (DML) : **INSERT**、**UPDATE**、**DELETE**、**COPY FROM**、**TRUNCATE**。注意不允许在恢复期间导致一个触发器被执行的动作。这个限制甚至被应用到临时表，因为不分配事务 ID 表行就不能被读或写，而当前不可能在一个热备环境中分配事务 ID。
- 数据定义语言 (DDL) : **CREATE**、**DROP**、**ALTER**、**COMMENT**。这个限制甚至被应用到临时表，因为执行这些操作会要求更新系统目录表。
- **SELECT ... FOR SHARE | UPDATE**，因为不更新底层数据文件就无法取得行锁。
- ``SELECT`` 语句上的能产生 DML 命令的规则。
- 显式请求一个高于 ``ROW EXCLUSIVE MODE`` 的模式的 ``LOCK``。
- 默认短形式的 ``LOCK``，因为它请求 ``ACCESS EXCLUSIVE MODE``。
- 显式设置非只读状态的事务管理命令：
 - **BEGIN READ WRITE**, **START TRANSACTION READ WRITE**
 - **SET TRANSACTION READ WRITE**, **SET SESSION CHARACTERISTICS AS TRANSACTION READ WRITE**
 - **SET transaction_read_only = off**
- 两阶段提交命令: **PREPARE TRANSACTION**、**COMMIT PREPARED**、**ROLLBACK PREPARED**，因为即使只读事务也需要在准备阶段（两阶段提交中的第一个阶段）写 WAL。
- 序列更新: **nextval()**、**setval()**
- **LISTEN**, **NOTIFY**

在正常操作中，“只读”事务被允许使用 ``LISTEN`` 和 ``NOTIFY``，因此热备会话在比普通只读会话更紧一点的限制下操作。这些限制中的某些可能会在一个未来的发行中被放松。

在热备期间，参数 `transaction_read_only`` 总是为真并且不可以被改变。但是只要不尝试修改数据库，热备期间的连接工作起来更像其他数据库连接。如果发生故障转移或切换，该数据库将切换到正常处理模式。当服务器改变模式时会话将保持连接。一旦热备结束，它将可以发起读写事务（即使是一个在热备期间启动的会话）。

用户可以通过 ``SHOW in_hot_standby`` 来检查 hot standby 会话是否是活跃的 (在服务器版本 14 之前该参数 `in_hot_standby`` 不存在。对于更早版本的服务器，可行的替代方法是 **SHOW transaction_read_only**。) 此外，还有一些函数允许用户访问有关备用服务器的信息。它们允许您编写程序来识别数据库当前的状态。用于监控恢复进度，或者您可以编写复杂的程序将数据库恢复到特定状态。

主服务器和后备服务器在多方面都松散地连接在一起。主服务器上的动作将在后备服务器上产生效果。结果是在它们之间有潜在的副作用或冲突。最容易理解的冲突是性能：如果在主服务器上发生一次大数据量的载入，那么着将在后备服务器上产生一个相似的 WAL 记录流，因而后备服务器查询可能要竞争系统资源（例如 I/O）。

随着热备发生的还可能还有其他类型的冲突。对于可能需要被取消的查询和（某些情况中）解决它们的已断开会话来说，这些冲突是硬冲突。用户可以用几种方式来处理这种冲突。冲突情况包括：

- 在主服务器上取得了访问排他锁（包括显式 `LOCK`` 命令和多种 DDL 动作）与后备查询中的表访问冲突。
- 在主服务器上删除一个表空间与使用该表空间存储临时工作文件的后备查询冲突。
- 在主服务器上删除一个数据库与在后备服务器上连接到该数据库的会话冲突。
- 从 WAL 清除记录的应用与快照仍能“看见”任意要被移除的行的后备事务冲突。
- 从 WAL 清除记录的应用与在后备服务器上访问该目标页的查询冲突，不管要被移除的数据是否为可见。

在主服务器上，这些情况仅仅会导致等待；并且用户可以选择取消这些冲突动作中间的一个。但是，在后备服务器上则没有选择：已被 WAL 记录的动作已经在主服务器上发生，那么后备服务器不能在应用它时失败。此外，允许 WAL 应用无限等待是非常不可取的，因为后备服务器的状态将变得逐渐远远落后于主服务器的状态。因此，提供了一种机制来强制性地取消与要被应用的 WAL 记录冲突的后备查询。

该问题情形的一个例子是主服务器上的一位管理员在一个表上运行 `DROP TABLE``，而该表正在后备服务器上被查询。如果 `DROP TABLE`` 被应用在后备服务器上，很明显该后备查询不能继续。如果这种情况在主服务器上发生，`DROP TABLE`` 将等待直到其他查询结束。但是当 `DROP TABLE`` 被运行在主服务器上，主服务器没有关于运行在后备服务器上查询的信息，因此它将不会等待任何这样的后备查询。WAL 改变记录在后备查询还在运行时来到后备服务器上，导致一个冲突。后备服务器必须要么延迟 WAL 记录的应用（还有它们之后的任何事情），要么取消冲突查询这样 `DROP TABLE`` 可以被应用。

当一个冲突查询很短时，我们通常期望能延迟 WAL 应用一小会儿让它完成；但是在 WAL 应用中的一段长的延迟通常是不受欢迎的。因此取消机制有参数，`max_standby_archive_delay` 和 `max_standby_streaming_delay`，它们定义了 WAL 应用中的最大允许延迟。当应用任何新收到的 WAL 数据花费了超过相关延迟设置值时，冲突查询将被取消。设立两个参数是为了对从一个归档读取 WAL 数据（即来自一个基础备份的初始恢复或者“追赶”一个已经落后很远的后备服务器）和通过流复制读取 WAL 数据的两种情况指定不同的延迟值。

在一台后备服务器上这主要是为了该可用性而存在，最好把延迟参数设置得比较短，这样服务器不会由于后备查询导致的延迟落后主服务器太远。但是，如果该后备服务器是位了执行长时间运行的查询，则一个较高甚至无限的延迟值更好。但是记住一个长时间运行的查询延迟了 WAL 记录的应用，它可能导致后备服务器上的其他会话无法看到主服务器上最近的改变。

一旦 `max_standby_archive_delay`` 或 `max_standby_streaming_delay`` 指定的延迟被超越，冲突查询将被取消。这通常仅导致一个取消错误，尽管在重放一个 `DROP DATABASE`` 的情况下整个冲突会话都将被中断。另外，如果冲突发生在一个被空闲事务持有的锁上，该冲突会话会被中断（这种行为可能在未来被改变）。

被取消的查询可能会立即被重试（当然是在开始一个新的事务后）。因为查询取消依赖于 WAL 记录被重放的本质，如果一个被取消的查询被再次执行，它可能会很好地成功完成。

记住延迟参数是从 WAL 数据被后备服务器收到后流逝的时间。因此，留给后备服务器上任何一个查询的宽限期从不会超过延迟参数，并且如果后备服务器已经由于等待之前的查询完成而落后或者因为过重的更新负载而无法跟上主服务器，宽限期可能会更少。

在后备查询和 WAL 重播之间发生冲突的最常见原因是“过早清除”。正常地，PostgreSQL 允许在没有事务需要看到旧行版本时对它们进行清除，这样可以保证根据 MVCC 规则的正确的数据可见性。不过，这个规则只能被应用于执行在主机上的事务。因此有可能主机上的清除会移除对一个后备服务器事务还可见的行版本。

有经验的用户应当注意行版本清除和行版本冻结都可能与后备查询冲突。即便在一个没有被更新或被删除行的表上运行一次手工`VACUUM FREEZE`也可能导致冲突。

用户应当清楚，主服务器上被正常和重度更新的表将快速地导致后备服务器上长时间运行的查询被取消。在这样的情况下，`max\_standby\_archive\_delay`或`max\_standby\_streaming\_delay`的有限制设置可以被视作`statement\_timeout`设置。

如果发现后备查询取消的数量不可接受，还是有补救的可能。第一种选项是设置参数

hot_standby_feedback，它阻止`VACUUM`

移除最近死亡的元组并且因此清除冲突不会产生。如果你这样做，你应当

注意这将使主服务器上的死亡元组清除被延迟，这可能会导致不希望发生

的表膨胀。不过，清除的情况不会比在主服务器上直接运行后备查询时更糟，

并且你仍然能够享受将执行分流到后备服务器的好处。如果后备服务器频繁地连接和

断开，你可能想要做些调整来处理无法提供`hot\_standby\_feedback`

反馈的时期。例如，考虑增加`max\_standby\_archive\_delay`，这样在断开连接的期间查询就不会快速地被WAL 归档文件中的冲突取消。你也应该考虑

增加`max\_standby\_streaming\_delay`来避免重新连接后新到达的流WAL 项导致的快速取消。

另一个选项是增加主服务器上的`vacuum\_defer\_cleanup\_age`，这样死亡行不会像平常那么快地被清理。这将允许在后备服务器上的查询能在被取消前有更多时间执行，并且不需要设置一个很高的`max\_standby\_streaming\_delay`。但是，这种方法很难保证任何指定的执行时间窗口，因为`vacuum\_defer\_cleanup\_age`是用主服务器上被执行的事务数来衡量的。

查询取消的数量和原因可以使用后备服务器上的`pg\_stat\_database\_conflicts`系统视图查看。`pg\_stat\_database`系统视图也包含汇总信息。

当WAL 重放由于冲突而需要比`deadlock\_timeout`更长时间时，用户可以控制是否打印日志消息。由参数`log\_recovery\_conflict\_waits`控制。

管理员概览

如果`hot\_standby`在`postgresql.conf`中被设置为`on`并且存在一个`standby.signal`文件，服务器将运行在热备模式。但是，可能需要一些时间来允许热备连接，因为在服务器完成足够的恢复来为查询提供一个一致的状态之前，它将不会接受连接。在此期间，尝试连接的客户端将被一个错误消息拒绝。要确认服务器已经可连接，要么循环地从应用尝试连接，要么在服务器日志中查找这些消息：

```
LOG:  entering standby mode
```

```
... then some time later ...
```

```
LOG:  consistent recovery state reached
```

```
LOG:  database system is ready to accept read only connections
```

在主服务器上，一旦创建一个检查点，一致性信息就被记录下来。当读取在特定时段（当在主服务器上`wal\_level`没有被设置为`replica`或者`logical`的期间）产生的WAL 时无法启用热备。在同时存在这些条件时，到达一个一致状态也会被延迟：

- 一个写事务有超过 64 个子事务
- 生存时间非常长的写事务

如果你正在运行基于文件的日志传送（“温备”），你可能需要等到下一个WAL 文件到达，这可能和主服务器上的`archive\_timeout`设置一样长。

设置几个参数可确定用于跟踪事务ID、锁和预备事务的共享内存大小。备用服务器上的设置必须大于或等于主服务器上的设置，以确保在恢复过程中不会耗尽共享内存。例如，如果主数据库正在执行预备事务，而备用数据库没有获取共享内存来跟踪预备事务，则备用数据库将无法继续恢复，直到配置更改。受影响的参数是：

- `max_connections`
- `max_prepared_transactions`
- `max_locks_per_transaction`
- `max_wal_senders`
- `max_worker_processes`

确保这不是问题的可靠方法是使备用数据库上的这些参数的值等于或大于主数据库上的值。因此，如果您想增加这些值，您应该先更改备用服务器上的设置，然后再更改主服务器上的设置。相反，如果要减小这些值，则应先更改主服务器上的设置，然后再更改备用服务器上的设置。请记住，当一个备用数据库被提升时，它会成为后续备用数据库所需参数设置的新基准。因此，最好在所有备用服务器上保持这些设置相同，这样在切换/故障转移期间就不会出现问题。

WAL 跟踪主节点上这些参数的变化。如果热备处理一个 WAL，表明主节点当前值大于备用数据库上的值，它将记录一个警告并中止恢复。例如：

WARNING: 由于参数设置不足，无法进行热备

详细信息：`max_connections = 80` 的设置低于主服务器上的设置，其值为 100。

LOG: 恢复已暂停

详细信息：如果恢复未暂停，服务器将关闭。

提示：您可以在进行必要的配置更改后重新启动服务器。

此时，您应该更改备库设置并重新启动实例以继续恢复。如果备库不是热备，不兼容的参数更改将立即将其关闭而不会暂停。因为这样继续开机没有意义。

管理员为 `max_standby_archive_delay` 和 `max_standby_streaming_delay` 选择适当的设置很重要。最好的选择取决于业务的优先级。例如如果服务器主要的任务是作为高可用服务器，那么您将想要低延迟设置，甚至是零（尽管它是一个非常激进的设置）。如果后备服务器的任务是作为一个用于决策支持查询的额外服务器，那么将其最大延迟值设置为很多小时甚至 -1（表示无限等待）可能都是可以接受的。

在主服务器上写出的事务状态 "hint bits" 是不被 WAL 记录的，因此后备服务器上的数据将可能重新写出该提示。这样，即使所有用户都是只读的，后备服务器仍将执行磁盘写操作；但数据值本身并没有发生改变。用户将仍写出大的排序临时文件并且重新生成 `relcache` 信息文件，这样在热备模式中数据库没有哪个部分是真正只读的。还要注意使用 `dblink` 模块写到远程数据库以及其他使用 PL 函数位于数据库之外的操作仍将可用，即使该事务是本地只读的。

在恢复模式期间，下列类型的管理命令是不被接受的：

- 数据定义语言（DDL）：e.g., `CREATE INDEX`
- 特权和所有权：`GRANT`, `REVOKE`,
- 维护命令：`ANALYZE`, `VACUUM`, `CLUSTER`, `REINDEX`

注意这些命令中的某些实际上在主服务器上的“只读”模式事务期间是被允许的。

结果是，你无法创建只存在于后备服务器上的额外索引以及统计信息。如果需要这些管理命令，它们应该在主服务器上被执行，并且最后那些改变将被传播到后备服务器。

``pg_cancel_backend()`` 和 ``pg_terminate_backend()`` 将在用户后端上工作，而不是执行恢复的 Startup 进程。``pg_stat_activity`` 不会为 Startup 进程显示一个项，也不会把恢复事务显示为活动。结果是在恢复期间 ``pg_prepared_xacts`` 总是为空。如果你希望解决不能确定的预备事务，查看主服务器上的 ``pg_prepared_xacts`` 并且发出命令来解决那里的事务或者在恢复结束后来解决它们。

和平常一样，`pg_locks`` 将显示被后端持有的锁。``pg_locks`` 也会显示一个由 **Startup 进程管理的虚拟事务**，它拥有被恢复重播的事务所持有的所有 `AccessExclusiveLocks`。注意 Startup 进程不请求锁来做数据库更改，并且因此对于 Startup 进程除 `AccessExclusiveLocks`` 之外的锁不显示在 `pg_locks`` 中，它们仅被假定存在。

Nagios的插件check_pgsql将可以工作，因为它检查的简单信息是存在的。check_postgres监控脚本也将能工作，尽管某些被报告的值可能给出不同或者混乱的结果。例如，上一次清理时间将不会被维护，因为在后备服务器上不会发生清理。在主服务器上运行的清理仍会把它们的改变发送给后备服务器。

WAL 文件控制命令在恢复期间将不会工作，如`pg\_start\_backup`、`pg\_switch\_wal`等。

可动态载入的模块可以工作，包括`pg\_stat\_statements`。

咨询锁在恢复期间工作正常，包括死锁检测。注意咨询锁从来都不会被 WAL 记录，因此在主服务器或后备服务器上一个咨询锁不可能与 WAL 重播发生冲突。也不可能会在主服务器上获得一个咨询锁并且在后备服务器上开始一个相似的咨询锁。咨询锁只与它们被取得的那个服务器相关。

基于触发器的复制系统（如Slony、Londiste和Bucardo）将根本不会运行在后备服务器上，然而只要改变不被发送到要被应用的后备服务器，它们将在主服务器上运行得很好。WAL 重播不是基于触发器的，因此你不能用后备服务器接替任何需要额外数据库写操作或依赖触发器使用的系统。

新的 OID 不能被分配，然而某些UUID生成器仍然能工作，只要它们不依赖于向数据库写新的状态。

当前，在只读事务期间不允许创建临时表，因此在某些情况中现有的脚本将不会正确运行。这个限制可能会在稍后的发行中被放松。这既是一个 SQL 标准符合问题也是一个技术问题。

只有在表空间为空时`DROP TABLESPACE`才能成功。某些后备服务器用户可能正在通过他们的`temp\_tablespaces`参数使用该表空间。如果在该表空间中有临时文件，所有活动查询将被取消来保证临时文件被移除，这样该表空间可以被移除并且 WAL 重播可以继续。

在主服务器上运行`DROP DATABASE`或`ALTER DATABASE ... SET TABLESPACE`将产生一个 WAL 项，它将导致所有连接到后备服务器上那个数据库的用户被强制地断开连接。这个动作会立即发生，不管`max\_standby\_streaming\_delay`的设置是什么。注意`ALTER DATABASE ... RENAME`不会断开用户，这在大部分情况中不会有提示，然而如果它依赖某种基于数据库名的方法，在某些情况中会导致程序混乱。

在普通（非恢复）模式中，如果你为具有登录能力的角色发出`DROP USER`或`DROP ROLE`，而该用户仍然连接着，则对已连接用户不会发生任何事情 - 他们保持连接。但是用户不能重新连接。这种行为也适用于恢复，因此在主服务器上的一次`DROP USER`不会使后备服务器上的用户断开。

在恢复期间统计收集器是活动的。所有扫描、读、阻塞、索引使用等将在后备服务器上被正常的记录。被重播的动作将不会重复它们在主服务器上的效果，因此重播一个插入将不会导致pg_stat_user_tables的Inserts列上的递增。在恢复的开始stats文件会被删除，因此来自主服务器和后备服务器的stats将不同；这被认为是一种特性而不是缺陷。

在恢复期间自动清理不是活动的。它将在恢复末尾正常启动。

检查点进程和后台写入进程在恢复期间是活动状态的。检查点进程将执行重新启动点（与主服务器上的检查点相似），后台写入进程将执行正常的块清理活动。这可以包括存储在后备服务器上的提示位信息的更新。在恢复期间，`CHECKPOINT`命令会被接受，然而它会执行一个重启点而不是一个新的检查点。

热备参数参考

在主服务器上，可以使用参数wal_level和vacuum_defer_cleanup_age。在主服务器上设置max_standby_archive_delay和max_standby_streaming_delay不会产生效果。

在主服务器上，可以使用参数hot_standby、max_standby_archive_delay和max_standby_streaming_delay。只要服务器保持在后备模式vacuum_defer_cleanup_age就没有效果，然而当后备服务器变成主服务器时它将变得相关。

热备有一些限制。这些限制很可能在未来的发行中被修复：

- 在能够取得快照之前，需要正在运行的事务的完整知识。使用大量子事务（目前指超过 64 个）的事务将延迟只读连接的启动，直到最长的运行着的写事务完成。如果发生这种情况，说明消息将被发送到服务器日志。
- 主服务器上的每一个检查点将产生用于后备查询的可用启动点。如果后备服务器在主机处于关闭状态时被关闭，就没有办法在主服务器启动之前重新进入热后备，因此它在 WAL 日志中产生一个进一步启动点。这种情况在它可能发生的大部分常见情况中不是一个问题。通常，如果主服务器被关闭并且不再可用，这可能是由于某种严重错误要求后备服务器被转变成为一个新的主服务器来操作。并且在主服务器被故意关闭的情况下，协调保证后备服务器平滑地过渡为新的主服务器也是一种标准过程。
- 在恢复尾声，由预备事务持有的 `AccessExclusiveLocks` 将要求两倍的正常锁表项。如果你计划运行大量并发的通常要求 `AccessExclusiveLocks` 的预备事务，或者你计划运行一个需要很多 `AccessExclusiveLocks` 的大型事务，我们建议你为 `max\_locks\_per\_transaction` 选择一个更大的值，也许是主服务器上该参数值的两倍。如果你的 `max\_prepared\_transactions` 设置为 0，你根本不需要考虑这个问题。
- 可序列化事务隔离级别目前在热备中不可用。尝试在热备模式中将一个事务设置为可序列化隔离级别将产生一个错误。

IvorySQL高级

安装指南

概述

IvorySQL安装方式包括以下5种：

- [yum源安装](#)
- [docker安装](#)
- [rpm安装](#)
- [源码安装](#)
- [deb安装](#)

本章将详细介绍每种方式的安装、运行及卸载过程。想要更快获得IvorySQL，请参阅[快速开始](#)。

同样，安装前请先创建一个用户，并赋予其root权限，安装、使用和卸载均以该用户执行。这里以ivorysql用户为例。

yum源安装

创建或编辑IvorySQL yum源配置文件/etc/yum.repos.d/ivorysql.repo

```
vim /etc/yum.repos.d/ivorysql.repo
[ivorysql4]
name=IvorySQL Server 4 $releasever - $basearch
baseurl=https://yum.highgo.com/dists/ivorysql-rpms/4/redhat/rhel-$releasever-$basearch
enabled=1
gpgcheck=0
```

保存退出后，安装IvorySQL4

```
$ sudo dnf install -y IvorySQL-4.4
```

- 查看安装结果

```
dnf search IvorySQL
```

查看结果说明如下：

序号	包名	描述
1	ivorysql4.x86_64	IvorySQL客户端程序和库文件
2	ivorysql4-contrib.x86_64	随IvorySQL发布的已贡献的源代码和二进制文件
3	ivorysql4-devel.x86_64	IvorySQL开发头文件和库
4	ivorysql4-docs.x86_64	IvorySQL的额外文档
5	ivorysql4-libs.x86_64	所有IvorySQL客户端所需的共享库

6	ivorysql4-llvmjit.x86_64	对IvorySQL的即时编译支持
7	ivorysql4-plperl.x86_64	用于IvorySQL的过程语言Perl
8	ivorysql4-plpython3.x86_64	用于IvorySQL的过程语言Python3
9	ivorysql4-pltcl.x86_64	用于IvorySQL的过程语言Tcl
10	ivorysql4-server.x86_64	创建和运行IvorySQL服务器所需的程序
11	ivorysql4-test.x86_64	随IvorySQL发布的测试套件
12	ivorysql-release.noarch	瀚高基础软件股份有限公司的Yum源配置RPM包

docker安装

- 从Docker Hub上获取IvorySQL镜像

```
$ docker pull ivorysql/ivorysql:4.4-ubi8
```

- 运行IvorySQL

```
$ docker run --name ivorysql -p 5434:5432 -e IVORYSQL_PASSWORD=your_password -d
ivorysql/ivorysql:4.4-ubi8
```

-e参数说明

参数名	是否必填	描述
IVORYSQL_USER	否	数据库用户，默认 ivorysql
IVORYSQL_PASSWORD	是	数据库用户密码
IVORYSQL_DB	否	数据库名称，默认 ivorysql
POSTGRES_HOST_AUTH_METHOD	否	修改主机身份验证方式，参考值：md5
POSTGRES_INITDB_ARGS	否	为initdb是添加额外参数，参考值：“--data-checksums”
PGDATA	否	将数据目录定义到其他路径文件夹下（例如子目录），默认 /var/lib/ivorysql/data
POSTGRES_INITDB_WALDIR	否	定义IvorySQL transaction文件夹路径，默认在数据目录（PGDATA）的子目录中。



1. 不推荐将POSTGRES_HOST_AUTH_METHOD参数填写为trust，这样将会使IVORYSQL_PASSWORD设置失效。
2. 如果POSTGRES_HOST_AUTH_METHOD参数设置为scram-sha-256，同时需将POSTGRES_INITDB_ARGS设置为—auth-host=scram-sha-256，才会使数据库正确进行初始化。

rpm安装

- 安装依赖

```
$ sudo dnf install -y lz4 libicu libxslt python3
```

- 获取rpm包

```
$ sudo wget  
https://github.com/IvorySQL/IvorySQL/releases/download/IvorySQL_4.4/IvorySQL-4.4-  
a50789d-20250304.x86_64.rpm
```

- 安装rpm包

使用以下命令安装所有rpm包：

```
$ sudo yum --disablerepo=* localinstall *.rpm
```

数据库将被安装在/opt/IvorySQL-4.4/路径下。

源码安装

- 安装依赖

```
$ sudo dnf install -y bison readline-devel zlib-devel openssl-devel  
$ sudo dnf groupinstall -y 'Development Tools'
```

- 获取IvorySQL源代码

```
$ git clone https://github.com/IvorySQL/IvorySQL.git  
$ cd IvorySQL  
$ git checkout -b IVORY_REL_4_STABLE origin/IVORY_REL_4_STABLE
```

- 配置

在IvorySQL目录下，执行以下命令进行配置，请使用—prefix指定安装目录：

```
$ ./configure --prefix=/usr/local/ivorysql/ivorysql-4
```

- 编译

执行以下命令进行编译：

```
$ make
```



编译完毕，安装之前可先执行make check或make all-check-world测试刚刚编译的结果。

- 安装

执行以下命令安装，数据库将被安装在上述由—prefix指定的路径下：


```
$ sudo make install
```

deb安装

- 安装依赖

```
$ sudo apt -y install pkg-config libreadline-dev libicu-dev libldap2-dev uuid-dev tcl-dev libperl-dev python3-dev bison flex openssl libssl-dev libpam-dev libxml2-dev libxslt-dev liboss-puuid-dev libsasl2-dev gettext
```

- 获取deb包

```
$ sudo wget https://github.com/IvorySQL/IvorySQL/releases/download/IvorySQL_4.4/IvorySQL-4.4-a50789d-20250304.amd64.deb
```

- 安装deb包

```
$ sudo dpkg -i IvorySQL-4.4-a50789d-20250304.amd64.deb
```

数据库将被安装在/opt/IvorySQL-4.4/路径下。

启动数据库

参考[yum源安装](#)、[rpm安装](#)、[源码安装](#)、[deb安装](#)的用户，需要手动启动数据库。

- 赋权

执行以下命令为安装用户赋权，示例用户为ivorysql，安装目录为/opt/IvorySQL-4.4/：

```
$ sudo chown -R ivorysql:ivorysql /opt/IvorySQL-4.4/
```

- 配置环境变量

将以下配置写入用户的~/.bash_profile文件并使用source命令该文件使环境变量生效：

```
PATH=/opt/IvorySQL-4.4/bin:$PATH
export PATH
PGDATA=/opt/IvorySQL-4.4/data
export PGDATA
```

```
$ source ~/.bash_profile
```

- 数据库初始化

```
$ mkdir /opt/IvorySQL-4.4/data
$ initdb -D /opt/IvorySQL-4.4/data
```

其中-D参数用来指定数据库的数据目录。更多参数使用方法，请使用initdb --help命令获取。

- 启动数据库服务

```
$ pg_ctl -D /opt/IvorySQL-4.4/data -l ivory.log start
```

其中-D参数用来指定数据库的数据目录，如果[v4-4::6::配置环境变量]配置了PGDATA，则该参数可以省略。-l参数用来指定日志目录。更多参数使用方法，请使用pg_ctl --help命令获取。

查看确认数据库启动成功：

```
$ ps -ef | grep postgres
ivorysql 130427      1  0 02:45 ?           00:00:00 /opt/IvorySQL-4.4/bin/postgres -D
/opt/IvorySQL-4.4/data
ivorysql 130428 130427  0 02:45 ?           00:00:00 postgres: checkpointer
ivorysql 130429 130427  0 02:45 ?           00:00:00 postgres: background writer
ivorysql 130431 130427  0 02:45 ?           00:00:00 postgres: walwriter
ivorysql 130432 130427  0 02:45 ?           00:00:00 postgres: autovacuum launcher
ivorysql 130433 130427  0 02:45 ?           00:00:00 postgres: logical replication
launcher
ivorysql 130445 130274  0 02:45 pts/1    00:00:00 grep --color=auto postgres
```

数据库连接

psql连接数据库：

```
$ psql -d <database>
psql (17.4)
Type "help" for help.

ivorysql=#
```

其中-

d参数用来指定想要连接到的数据库名称。IvorySQL默认使用ivorysql数据库，但较低版本的IvorySQL首次使用时需用户先连接postgres数据库，然后自己创建ivorysql数据库。较高版本的IvorySQL则已为用户创建好ivorysql数据库，可以直接连接。

更多参数使用方法，请使用psql --help命令获取。



Docker运行IvorySQL时，需要添加额外参数，参考：psql -d ivorysql -U ivorysql -h

卸载IvorySQL



使用任何一种方法卸载前请先停止数据库服务并做好数据备份。

yum源安装的卸载

执行以下命令依次卸载：

```
$ sudo dnf remove -y IvorySQL-4.4
$ sudo rpm -e ivorysql-release-4.2-1.noarch
```

docker安装的卸载

执行以下命令，使IvorySQL容器停止运行，并删除IvorySQL容器和镜像：

```
$ docker stop ivorysql
$ docker rm ivorysql
$ docker rmi ivorysql/ivorysql:4.4-ubi8
```

rpm安装的卸载

执行以下命令卸载并清理文件夹：

```
$ sudo yum remove --disablerepo=* ivorysql4\*
$ sudo rm -rf /opt/IvorySQL-4.4
```

源码安装的卸载

执行以下命令卸载数据库并清理文件夹：

```
$ sudo make uninstall
$ make clean
$ sudo rm -rf /opt/IvorySQL-4.4
```

deb安装的卸载

执行以下命令卸载数据库并清理文件夹：

```
$ sudo dpkg -P IvorySQL-4.4
$ sudo rm -rf /opt/IvorySQL-4.4
```

集群搭建

这里以最简单的一主一备（异步备）集群为例，介绍IvorySQL数据库集群的搭建方法。

主节点

安装并启动数据库

yum源快速安装数据库，请参考[快速安装](#)。

想要获取更多安装方式，请参考[安装指南](#)。



主节点数据库需要安装并启动

关闭防火墙

集群中各节点均需关闭防火墙才能正常通信。

```
$ sudo systemctl stop firewalld
```

配置文件

为了从主节点向备节点搭建流复制，主节点需要对data目录下的postgresql.conf和pg_hba.conf两个文件进行配置。

- postgresql.conf

将以下配置追加到postgresql.conf文件末尾：

```
listen_addresses = '*'
max_connections = 100
wal_level = replica
max_wal_senders = 5
hot_standby = on
```

- pg_hba.conf

将以下配置追加到pg_hba.conf文件末尾：

```
host all all 0.0.0.0/0 trust
host replication all 0.0.0.0/0 trust
```



示例中的pg_hba的配置，仅做为demo用来测试，这种配置会导致数据库密码失效，请根据环境实际情况进行配置

重启主节点数据库服务

```
$ pg_ctl restart
```

备节点

安装数据库

Yum源快速安装数据库，请参考[快速安装](#)。

想要获取更多安装方式，请参考[安装指南](#)。



备节点数据库只需要安装，不需要启动

关闭防火墙

集群中各节点均需关闭防火墙才能正常通信。

```
$ sudo systemctl stop firewalld
```

搭建流复制

在备节点上执行以下命令，创建一个主节点的基础备份，即搭建流复制：

```
$ sudo pg_basebackup -F p -P -X fetch -R -h <primary_ip> -p <primary_port> -U ivorysql  
-D /usr/local/ivorysql/ivorysql-4/data
```

- -h为主节点ip；
- -p为主节点数据库端口号，默认为5432；
- -U为数据库用户；
- -D为要创建的备节点数据库数据目录。

其他参数使用方法，请使用pg_basebackup --help命令获取。

配置环境变量

将以下配置写入~/.bash_profile文件：

```
PATH=/usr/local/ivorysql/ivorysql-4/bin:$PATH  
export PATH  
PGDATA=/usr/local/ivorysql/ivorysql-4/data  
export PGDATA
```

source该文件使环境变量生效：

```
$ source ~/.bash_profile
```

启动备节点数据库服务

```
$ pg_ctl -D /usr/local/ivorysql/ivorysql-4/data start
```

集群的使用

查看集群状态

在主节点上执行以下命令可以看到walsender：

```
$ ps -ef |grep postgres
...
ivorysql 11176  8067  0 21:54 ?          00:00:00 postgres: walsender ivorysql
192.168.31.102(53416) streaming 0/7000060...
```

而备节点则可看到walreceiver:

```
$ ps -ef | grep postgres
...
ivorysql  6567  6139  0 21:54 ?                00:00:00 postgres: walreceiver streaming
0/7000060
...
```

在主节点上psql连接数据库，并查看集群状态：

[illegible]

这里192.168.31.102为备节点的ip，`async`表示数据同步方式为异步流复制。=== 使用集群
集群中所有的写操作均在主节点执行，读操作则主备节点都可以执行。主节点的数据通过流复制同步到备节点。主节点写操作的结果在任何一个备节点都能够查询到。
例如，在主节点创建一个新的数据库`test`，并在主节点进行查询：

```
$ psql -d ivorysql
psql (17.2)
Type "help" for help.
```

```
ivorysql=# create database test;
```

```
CREATE DATABASE
```

```
ivorysql=# \l
```

				List of databases		
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU
Locale	ICU Rules	Access privileges				
-----+-----+-----+-----+-----+-----+-----						
ivorysql	ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
template0	ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
	=c/ivorysql		+			
template1	ivorysql=CTc/ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
	=c/ivorysql		+			
test	ivorysql=CTc/ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
(4 rows)						

在备节点查询：

```
$ psql -d ivorysql
```

```
psql (17.2)
```

```
Type "help" for help.
```

```
ivorysql=# \l
```

				List of databases		
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU
Locale	ICU Rules	Access privileges				
-----+-----+-----+-----+-----+-----+-----						
ivorysql	ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
template0	ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
	=c/ivorysql		+			
template1	ivorysql=CTc/ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	
	=c/ivorysql		+			
test	ivorysql=CTc/ivorysql	UTF8	libc	en_US.UTF-8	en_US.UTF-8	

开发者指南

概览

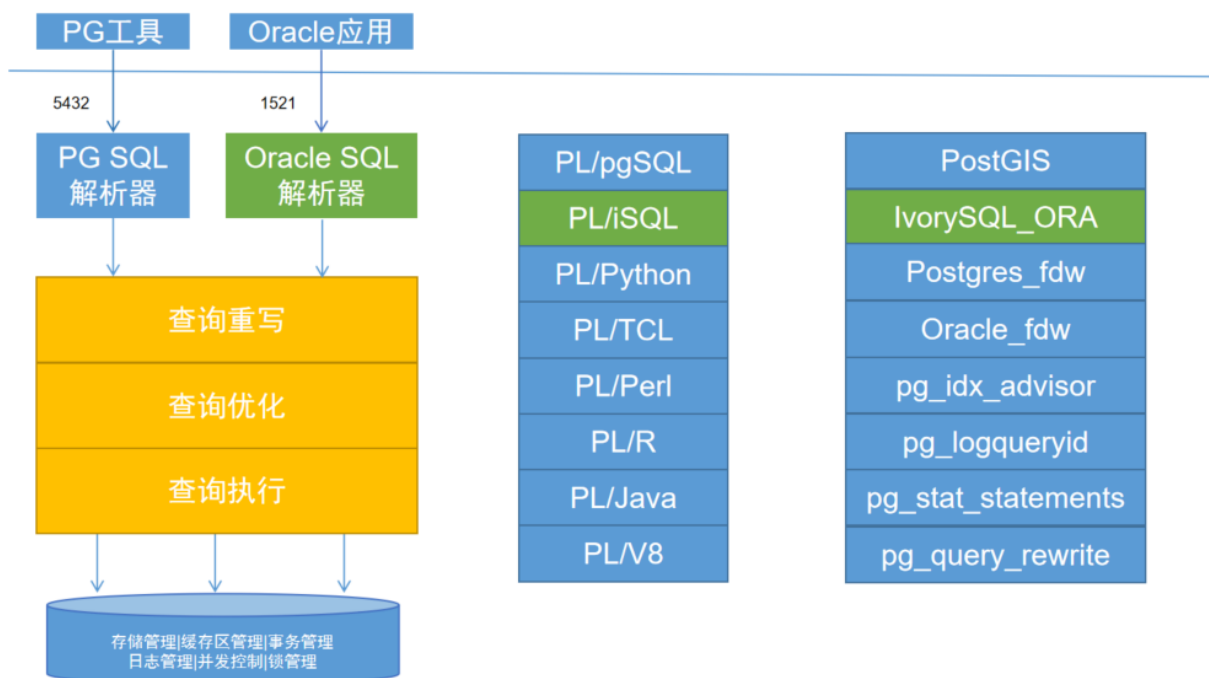
IvorySQL在开源PostgreSQL数据库的基础上提供独特的附加功能。

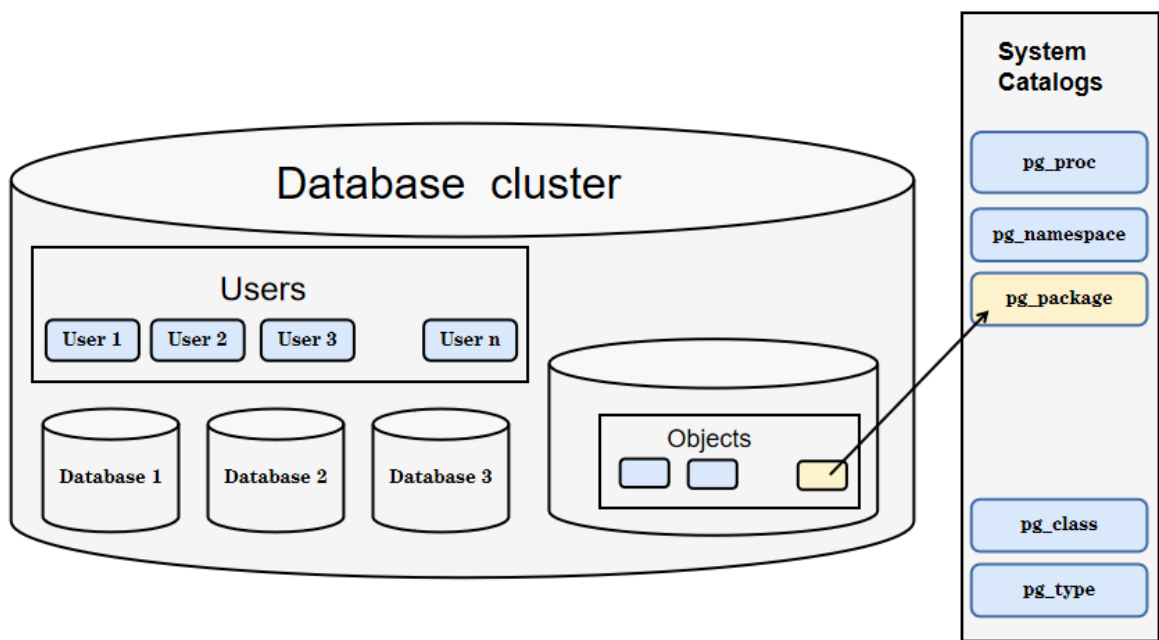
IvorySQL致力于通过创新和建立在开源数据库解决方案之上为其终端用户提供价值。我们的目标是中小型企业提供一个具有高性能、可扩展性、可靠性和易于使用的解决方案。

IvorySQL提供的扩展功能将使用户能够建立高性能和可扩展的PostgreSQL数据库集群，具有更好的数据库兼容性和管理。这简化了从其他DBMS迁移到PostgreSQL的过程，增强了数据库管理经验。

架构概述

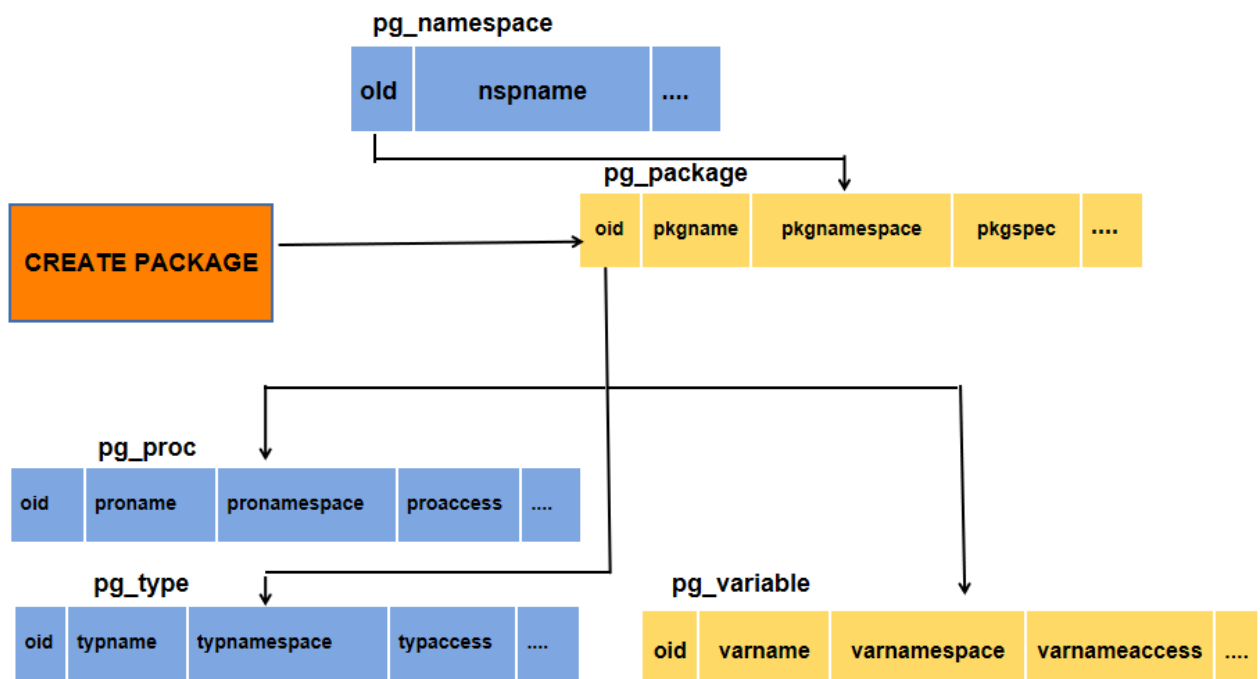
在对原有的PostgreSQL改动最小的前提下，实现对Oracle的兼容。我们需要实现双parser、双端口、模式PL/pgSQL实现PL/iSQL的框架。实现流程图如下：





系统表的变化

下图描述了对PostgreSQL现有系统表的变化以及所做的补充。



数据库建模（第一章创建库+第二章创建表）

创建一个数据库

看看你能否访问数据库服务器的第一个例子就是试着创建一个数据库。一台运行着的IvorySQL服务器可以管理许多数据库。通常我们会为每个项目和每个用户单独使用一个数据库。

你的站点管理员可能已经为你创建了可以使用的数据库。如果这样你就可以省略这一步，并且跳到下一节。

要创建一个新的数据库，在我们这个例子里叫`mydb`，你可以使用下面的命令：

```
$ createdb mydb
```

如果不产生任何响应则表示该步骤成功，你可以跳过本节的剩余部分。

如果你看到类似下面这样的信息：

```
createdb: command not found
```

那么就是IvorySQL没有安装好。或者是根本没安装，或者是你的shell搜索路径没有设置正确。尝试用绝对路径调用该命令试试：

```
$ /usr/local/pgsql/bin/createdb mydb
```

在你的站点上这个路径可能不一样。和你的站点管理员联系或者看看安装指导获取正确的位置。

另外一种响应可能是这样：

```
createdb: error: connection to server on socket "/tmp/.s.PGSQL.5432" failed: No such
file or directory
        Is the server running locally and accepting connections on that socket?
```

这意味着该服务器没有启动，或者在`createdb`期望去连接它的时候没有在监听。同样，你也要查看安装指导或者咨询管理员。

另外一个响应可能是这样：

```
createdb: error: connection to server on socket "/tmp/.s.PGSQL.5432" failed: FATAL:
role "joe" does not exist
```

在这里提到了你自己的登录名。如果管理员没有为你创建IvorySQL用户帐号，就会发生这些现象。（IvorySQL用户帐号和操作系统用户帐号是不同的。）如果你是管理员，参阅 [第 22 章](#) 获取创建用户帐号的帮助。你需要变成安装IvorySQL的操作系统用户的身份（通常是 **postgres**）才能创建第一个用户帐号。也有可能是赋予你的IvorySQL用户名和你的操作系统用户名不同；这种情况下，你需要使用`-U`选项或者使用`PGUSER`环境变量指定你的IvorySQL用户名。

如果你有个数据库用户帐号，但是没有创建数据库所需要的权限，那么你会看到下面的信息：

```
createdb: error: database creation failed: ERROR:  permission denied to create
database
```

并非所有用户都被许可创建新数据库。如果IvorySQL拒绝为你创建数据库，那么你需要让站点管理员赋予你创建数据库的权限。出现这种情况时请咨询你的站点管理员。如果你自己安装了IvorySQL，那么你应该以你启动数据库服务器的用户身份登录然后参考手册完成权限的赋予工作。<http://www.postgresql.org/docs/17/tutorial-createdb.html#ftn.id-1.4.3.4.10.4>[[1]]

你还可以用其它名字创建数据库。IvorySQL允许你在一个站点上创建任意数量的数据库。数据库名必须是以字母开头并且小于63个字符长。一个方便的做法是创建和你当前用户名同名的数据库。许多工具假设该数据库名为缺省数据库名，所以这样可以节省你的敲键。要创建这样的数据库，只需要键入：

```
$ createdb
```

如果你再也不想使用你的数据库了，那么你可以删除它。比如，如果你是数据库`mydb`的所有人（创建人），那么你就可以用下面的命令删除它：

```
$ dropdb mydb
```

（对于这条命令而言，数据库名不是缺省的用户名，因此你就必须声明它）。这个动作将在物理上把所有与该数据库相关的文件都删除并且不可取消，因此做这中操作之前一定要考虑清楚。

更多关于 **createdb** 和 **dropdb** 的信息可以分别在 [createdb](#)和 [dropdb](#)中找到。

创建一个新表

你可以通过指定表的名字和所有列的名字及其类型来创建表：

```
CREATE TABLE weather (  
    city          varchar(80),  
    temp_lo       int,          -- 最低温度  
    temp_hi       int,          -- 最高温度  
    prcp          real,         -- 湿度  
    date          date  
);
```

你可以在 **psql** 输入这些命令以及换行符。**psql** 可以识别该命令直到分号才结束。

你可以在SQL命令中自由使用空白（即空格、制表符和换行符）。这就意味着你可以用和上面不同的对齐方式键入命令，或者将命令全部放在一行中。两个划线（“--”）引入注释。任何跟在它后面直到行尾的东西都会被忽略。SQL是对关键字和标识符大小写不敏感的语言，只有在标识符用双引号包围时才能保留它们的大小写（上例没有这么做）。

varchar(80) 指定了一个可以存储最长80个字符的任意字符串的数据类型。**int** 是普通的整数类型。**real** 是一种用于存储单精度浮点数的类型。**date** 类型应该可以自解释（没错，类型为`date`的列名字也是`date`。这么做可能比较方便或者容易让人混淆——你自己选择）。

IvorySQL支持标准的SQL类型`int`、**smallint**、**real**、**double precision**、**char(N)**、**varchar(N)**、**date**、**time**、**timestamp**和**interval**，还支持其他的通用功能的类型和丰富的几何类型。IvorySQL中可以定制任意数量的用户定义数据类型。因而类型名并不是语法关键字，除了SQL标准要求支持的特例外。

第二个例子将保存城市和它们相关的地理位置：

```
CREATE TABLE cities (  
    name          varchar(80),  
    location      point  
);
```

类型`point`就是一种IvorySQL特有数据类型的例子。

最后，我们还要提到如果你不再需要某个表，或者你想以不同的形式重建它，那么你可以用下面的命令删除它：

```
DROP TABLE tablename;
```

写入数据（SQL写入） 参考第 6 章 数据操纵

当一个表被创建后，它不包含任何数据。在数据库发挥作用之前，首先要做的是插入数据。一次插入一行数据。你也可以在一个命令中插入多行，但不能插入不完整的行。即使只知道其中一些列的值，也必须创建完整的行。

要创建一个新行，使用 `INSERT` 命令。这条命令要求提供表的名字和其中列的值。例如，考虑 第 5 章中的产品表：

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric  
);
```

一个插入一行的命令将是：

```
INSERT INTO products VALUES (1, 'Cheese', 9.99);
```

数据的值是按照这些列在表中出现的顺序列出的，并且用逗号分隔。通常，数据的值是文字（常量），但也允许使用标量表达式。

上面的语法的缺点是你必须知道表中列的顺序。要避免这个问题，你也可以显式地列出列。例如，下面的两条命令都有和上文那条命令一样的效果：

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);  
INSERT INTO products (name, price, product_no) VALUES ('Cheese', 9.99, 1);
```

许多用户认为明确列出列的名字是个好习惯。

如果你没有获得所有列的值，那么你可以省略其中的一些。在这种情况下，这些列将被填充为它们的缺省值。例如：

```
INSERT INTO products (product_no, name) VALUES (1, 'Cheese');  
INSERT INTO products VALUES (1, 'Cheese');
```

第二种形式是IvorySQL的一个扩展。它从使用给出的值从左开始填充列，有多少个给出的列值就填充多少个列，其他列的将使用缺省值。

为了保持清晰，你也可以显式地要求缺省值，用于单个的列或者用于整个行：

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', DEFAULT);  
INSERT INTO products DEFAULT VALUES;
```

你可以在一个命令中插入多行：

```
INSERT INTO products (product_no, name, price) VALUES
(1, 'Cheese', 9.99),
(2, 'Bread', 1.99),
(3, 'Milk', 2.99);
```

也可以插入查询的结果（可能没有行、一行或多行）：

```
INSERT INTO products (product_no, name, price)
SELECT product_no, name, price FROM new_products
WHERE release_date = 'today';
```

这提供了用于计算要插入的行的SQL查询机制（第7章）的全部功能。

提示

在一次性插入大量数据时，考虑使用 `COPY` 命令。它不如 `INSERT` 命令那么灵活，但是更高效。

查询数据 参考 第七章查询的组合查询 第十五章 并行查询

组合查询

两个查询的结果可以用集合操作并、交、差进行组合。语法是：

```
query1 UNION [ALL] query2
query1 INTERSECT [ALL] query2
query1 EXCEPT [ALL] query2
```

`query1` 和 `query2` 都是可以使用以上所有特性的查询。集合操作也可以嵌套和级连，例如：

```
query1 UNION query2 UNION query3
```

实际执行的是：

```
(query1 UNION query2) UNION query3
```

`UNION` 有效地把 `query2` 的结果附加到 `query1` 的结果上（不过我们不能保证这就是这些行实际被返回的顺序）。此外，它将删除结果中所有重复的行，就像 `DISTINCT` 做的那样，除非你使用了 `UNION ALL`。

`INTERSECT` 返回那些同时存在于 `query1` 和 `query2` 的结果中的行，除非声明了 `INTERSECT ALL`，否则所有重复行都被消除。

`EXCEPT` 返回所有在 `query1` 的结果中但是不在 `query2` 的结果中的行（有时侯这叫做两个查询的*差）。同样的，除非声明了 `EXCEPT ALL`，否则所有重复行都被消除。

为了计算两个查询的并、交、差，这两个查询必须是“并操作兼容的”，也就意味着它们都返回同样数量的列，并且对应的列有兼容的数据类型，如第10.5节中描述的那样。

并行查询

并行查询如何工作

当优化器判断对于某一个特定的查询，并行查询是最快的执行策略时，优化器将创建一个查询计划。该计划包括一个 *Gather* 或者 *Gather Merge* 节点。下面是一个简单的例子：

```
EXPLAIN SELECT * FROM pgbench_accounts WHERE filler LIKE '%x%';
               QUERY PLAN
-----
Gather  (cost=1000.00..217018.43 rows=1 width=97)
  Workers Planned: 2
    -> Parallel Seq Scan on pgbench_accounts  (cost=0.00..216018.33 rows=1 width=97)
        Filter: (filler ~ '%x%'::text)
(4 rows)
```

在所有的情形下，`Gather` 或 *Gather Merge* 节点都只有一个子计划，它是将被并行执行的计划的一部分。如果 `Gather` 或 *Gather Merge* 节点位于计划树的最顶层，那么整个查询将并行执行。如果它位于计划树的其他位置，那么只有查询中在它之下的那一部分会并行执行。在上面的例子中，查询只访问了一个表，因此除 `Gather` 节点本身之外只有一个计划节点。因为该计划节点是 `Gather` 节点的孩子节点，所以它会并行执行。

使用 [EXPLAIN](#) 命令，

你能看到规划器选择的工作者数量。当查询执行期间到达 `Gather` 节点时，实现用户会话的进程将会请求和规划器选中的工作者数量一样多的 [后台工作者进程](#)。规划器将考虑使用的后台工作者的数量被限制为最多 [max_parallel_workers_per_gather](#) 个。任何时候能够存在的后台工作者进程的总数由 [max_worker_processes](#) 和 [max_parallel_workers](#) 限制。因此，一个并行查询可能会使用比规划中少的工作者来运行，甚至有可能根本不使用工作者。最优的计划可能取决于可用的工作者的数量，因此这可能会导致不好的查询性能。如果这种情况经常发生，那么就应当考虑一下提高 `max\_worker\_processes` 和 `max\_parallel\_workers` 的值，这样更多的工作者可以同时运行；或者降低 `max\_parallel\_workers\_per\_gather`，这样规划器会要求少一些的工作者。

为一个给定并行查询成功启动的后台工作者进程都将会执行计划的并行部分。这些工作者的领导者也将执行该计划，不过它还有一个额外的任务：它还必须读取所有由工作者产生的元组。当整个计划的并行部分只产生了少量元组时，领导者通常将表现为一个额外的加速查询执行的工作者。反过来，当计划的并行部分产生大量的元组时，领导者将几乎全用来读取由工作者产生的元组并且执行 `Gather` 或 `Gather Merge` 节点上层计划节点所要求的任何进一步处理。在这些情况下，领导者所作的执行并行部分的工作将会很少。

当计划的并行部分的顶层节点是 `Gather Merge` 而不是 `Gather` 时，它表示每个执行计划并行部分的进程会产生有序的元组，并且领导者执行一种保持顺序的合并。相反，`Gather` 会以任何方便的顺序从工作者读取元组，这会破坏可能已经存在的排序顺序。

何时会用到并行查询？

有几种设置会导致查询规划器在任何情况下都不生成并行查询计划。为了让并行查询计划能够被生成，必须配置好下列设置。

- [max_parallel_workers_per_gather](#) 必须被设置为大于零的值。这是一种特殊情况，更加普遍的原则是所用的工作者数量不能超过 `max\_parallel\_workers\_per\_gather` 所配置的数量。

此外，系统一定不能运行在单用户模式下。因为在单用户模式下，整个数据库系统运行在单个进程中，没有后台工作者进程可用。

如果下面的任一条件为真，即便对一个给定查询通常可以产生并行查询计划，规划器都不会为它产生并行查询计划：

- 查询要写任何数据或者锁定任何数据库行。如果一个查询在顶层或者CTE中包含了数据修改操作，那么不会为该查询产生并行计划。一种例外是，**CREATE TABLE ... AS**、``SELECT INTO`` 以及 ``CREATE MATERIALIZED VIEW`` 这些创建新表并填充它的命令可以使用并行计划。
- 查询可能在执行过程中被暂停。只要在系统认为可能发生部分或者增量式执行，就不会产生并行计划。例如：用 `DECLARE CURSOR` 创建的游标将永远不会使用并行计划。类似地，一个 ``FOR x IN query LOOP .. END LOOP`` 形式的 PL/pgSQL 循环也永远不会使用并行计划，因为当并行查询进行时，并行查询系统无法验证循环中的代码执行起来是安全的。
- 使用了任何被标记为 ``PARALLEL UNSAFE`` 的函数的查询。大多数系统定义的函数都被标记为 ``PARALLEL SAFE``，但是用户定义的函数默认被标记为 ``PARALLEL UNSAFE``。参见 第 15.4 节中的讨论。
- 该查询运行在另一个已经存在的并行查询内部。例如，如果一个被并行查询调用的函数自己发出一个 SQL 查询，那么该查询将不会使用并行计划。这是当前实现的一个限制，但是或许不值得移除这个限制，因为它会导致单个查询使用大量的进程。

即使对于一个特定的查询已经产生了并行查询计划，在一些情况下执行时也不会并行执行该计划。如果发生这种情况，那么领导者将会自己执行该计划在 ``Gather`` 节点之下的部分，就好像 ``Gather`` 节点不存在一样。上述情况将在满足下面的任一条件时发生：

- 因为后台工作者进程的总数不能超过 `max_worker_processes`，导致不能得到后台工作者进程。
- 由于为并行查询目的启动的后台工作者数量不能超过 `max_parallel_workers` 这一限制而不能得到后台工作者。
- 客户端发送了一个执行消息，并且消息中要求取元组的数量不为零。执行消息可见 [扩展查询协议](#) 中的讨论。因为 `libpq` 当前没有提供方法来发送这种消息，所以这种情况只可能发生在不依赖 `libpq` 的客户端中。如果这种情况经常发生，那在它可能发生的会话中设置 `max_parallel_workers_per_gather` 为零是一个很好的主意，这样可以避免产生连续运行时次优的查询计划。

并行计划

因为每个工作者只执行完成计划的并行部分，所以不可能简单地产生一个普通查询计划并使用多个工作者运行它。每个工作者都会产生输出结果集的一个完全，因而查询并不会比普通查询运行得更快甚至还会产生不正确的结果。相反，计划的并行部分一定被查询优化器在内部当作一个*部分计划*，即它必须被构建出来，这样每一个执行该计划的进程将以无重复地方式产生输出行的一个子集，即保证每一个所需要的输出行正好只被一个合作进程生成。通常，这意味着该查询的驱动表上的扫描必须是一种可并行的扫描。

并行扫描

当前支持下列可并行的表扫描。

- 在一个*并行顺序扫描*中，表块将在合作进程之间被划分。一次会分发一个块，这样对表的访问还是保持顺序方式。
- 在一个*并行位图堆扫描*中，一个进程被选为领导者。这个进程执行对一个或者多个索引的扫描并且构建出一个位图指示需要访问哪些表块。这些表块接着会在合作进程之间划分（和并行顺序扫描中一样）。换句话说，堆扫描以并行方式进行但底层的索引扫描不是并行。
- 在一个*并行索引扫描*或者*并行只用索引的扫描*中，合作进程轮流从索引读取数据。当前，并行索引扫描仅有 B-树索引支持。每一个进程将认领一个索引块并且扫描和返回该索引块引用的所有元组，其他进程可以同时地从一个不同的索引块返回元组。并行 B-树扫描的结果会以每个工作者进程内的顺序返回。

其他扫描类型（例如非 B-树索引的扫描）可能会在未来支持并行扫描。

并行连接

正如在非并行计划中那样，驱动表可能被使用嵌套循环、哈希连接或者归并连接连接到一个或者多个其他表。连接的内侧可以是任何类型的被规划器支持的非并行计划，假设它能够安全地在并行工作者中运行。根据连接类型，内侧还可以是一种并行计划。

- 在一个*嵌套循环连接*中，内侧总是非并行的。尽管它会被完全执行，如果内侧是一个索引扫描也会很高效，因为外侧元组以及在索引中查找值的循环会被划分到多个合作进程。
- 在一个*归并连接*中，内侧总是一个非并行计划并且因此会被完全执行。这可能是不太高效的，特别是在排序必须被执行时，因为在每一个合作进程中工作数据和结果数据是重复的。
- 在一个哈希连接（没有“并行”前缀）中，每个合作进程都会完全执行内侧以构建哈希表的相同。如果哈希表很大或者该计划开销很大，这种方式就很低效。在一个并行哈希连接*中，内侧是一个*并行哈希*，它把构建共享哈希表的工作划分到多个合作进程。

并行聚集

IvorySQL通过按两个阶段进行聚集来支持并行聚集。首先，每个参与到查询并行部分的进程执行一个聚集步骤，为该进程注意到的每个分组产生一个部分结果。这在计划中反映为一个`Partial Aggregate`节点。然后，部分结果通过`Gather`或者`Gather Merge`被传输到领导者。最后，领导者对来自所有工作者的结果进行重新聚集得到最终的结果。这在计划中反映为一个`Finalize Aggregate`节点。

因为`Finalize

Aggregate`节点运行在领导者进程上，如果查询产生的分组数相对于其输入行数来说比较大，则查询规划器不会喜欢它。例如，在最坏的情况下，`Finalize Aggregate`节点看到的分组数可能与所有工作者进程在`Partial Aggregate`阶段看到的输入行数一样多。对于这类情况，使用并行聚集显然得不到性能收益。查询规划器会在规划过程中考虑这一点并且不太会在这种情况下选择并行聚集。

并行聚集并非在所有情况下都被支持。每一个聚集都必须是对并行安全的并且必须有一个组合函数。如果该聚集有一个类型为`internal`的转移状态，它必须有序列化和反序列化函数。更多细节请参考 [CREATE AGGREGATE](#)。如果任何聚集函数调用包含`DISTINCT`或`ORDER BY`子句，则不支持并行聚集。对于有序聚集或者当查询涉及`GROUPING SETS`时，也不支持并行聚集。只有在查询中涉及的所有连接也是该计划并行部分的组成部分时，才能使用并行聚集。

并行Append

只要当IvorySQL需要从多个源中整合行到一个单一结果集时，它会使用`Append`或`MergeAppend`计划节点。在实现`UNION ALL`或扫描分区表时常常会发生这种情况。就像这些节点可以被用在任何其他计划中一样，它们可以被用在并行计划中。不过，在并行计划中，规划器使用的是`Parallel Append`节点。

当一个`Append`节点被用在并行计划中时，每个进程将按照子计划出现的顺序执行子计划，这样所有的参与进程会合作执行第一个子计划直到它被完成，然后同时移动到第二个计划。而在使用`Parallel Append`时，执行器将把它的子计划尽可能均匀地散布在参与进程中，这样多个子计划会被同时执行。这避免了竞争，也避免了子计划在那些不执行它的进程中产生启动代价。

此外，和常规的`Append`节点不同（在并行计划中使用时仅有部分子计划），**Parallel Append`节点既可以有部分子计划也可以有非部分子计划。非部分子计划将仅被单个进程扫描，因为扫描它们不止一次会产生重复的结果。因此涉及到追加多个结果集的计划即使在没有有效的部分计划可用时，也能实现粗粒度的并行。例如，考虑一个针对分区表的查询，它只能通过使用一个不支持并行扫描的索引来实现。规划器可能会选择常规`Index Scan`计划的`Parallel Append`。每个索引扫描必须被单一的进程执行完，但不同的扫描可以由不同的进程同时执行。**

[enable_parallel_append](#)可以被用来禁用这种特性。

并行计划小贴士

如果我们想要一个查询能产生并行计划但事实上又没有产生，可以尝试减小 [parallel_setup_cost](#) 或者 [parallel_tuple_cost](#)。当然，这个计划可能比规划器优先产生的顺序计划还要慢，但也不总是如此。如果将这些设置为很小的值（例如把它们设置为零）也不能得到并行计划，那就可能是有某种原因导致查询规划器无法为你的查询产生并行计划。可能的原因可见 [第 15.2 节](#) 和 [第 15.4 节](#)。

在执行一个并行计划时，可以用`EXPLAIN (ANALYZE,VERBOSE)`来显示每个计划节点在每个工作者上的统计信息。这些信息有助于确定是否所有的工作被均匀地分发到所有计划节点以及从总体上理解计划的性能特点。

事务（参考Sql命令）

ABORT — 中止当前事务

大纲

```
ABORT [ WORK | TRANSACTION ] [ AND [ NO ] CHAIN ]
```

描述

ABORT`回滚当前事务并且导致由该事务所作的所有更新被丢弃。这个命令的行为与标准SQL命令`ROLLBACK的行为一样，并且只是为了历史原因存在。

参数

- **WORK TRANSACTION**

可选关键词。它们没有效果。

- **AND CHAIN**

如果规定了`AND CHAIN`，新事务立即启动，具有与刚刚完成的事务相同的事务特征(参见<http://www.postgresql.org/docs/17/sql-set-transaction.html>[`SET TRANSACTION`])。否则，不会启动新事务。

注解

使用 **COMMIT** 成功地终止一个事务。

在一个事务块之外发出`ABORT`会发出一个警告消息并且不会产生效果。

例子

中止所有更改：

```
ABORT;
```

兼容性

这个命令是一个因为历史原因而存在的IvorySQL扩展。`ROLLBACK`是等效的标准SQL命令。

BEGIN — 开始一个事务块

大纲

```
BEGIN [ WORK | TRANSACTION ] [ transaction_mode [, ...] ]
```

其中 `transaction_mode` 是以下之一：

```
ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ | READ COMMITTED | READ
```

```
UNCOMMITTED }  
  READ WRITE | READ ONLY  
  [ NOT ] DEFERRABLE
```

描述

BEGIN 开始一个事务块，也就是说所有 **BEGIN** 命令之后的所有语句将在一个事务中执行，直到给出一个显式的 **COMMIT** 或者 **ROLLBACK**。默认情况下（没有 **BEGIN**），IvorySQL 在“自动提交”模式中执行事务，也就是说每个语句都在自己的事务中执行并且在语句结束时隐式地执行一次提交（如果执行成功，否则会完成一次回滚）。

在一个事务块内的语句会执行得更快，因为事务的开始/提交也要求可观的 CPU 和磁盘活动。在进行多个相关更改时，在一个事务内执行多个语句也有助于保证一致性：在所有相关更新还没有完成之前，其他会话将不能看到中间状态。

如果指定了隔离级别、读/写模式或者延迟模式，新事务也会有那些特性，就像执行了 **SET TRANSACTION** 一样。

参数

- **WORK` `TRANSACTION**

可选的关键词。它们没有效果。

这个语句其他参数的含义请参考 [SET TRANSACTION](#)。

注解

START TRANSACTION 具有和 **`BEGIN`** 相同的功能。

使用 **COMMIT** 或者 **ROLLBACK** 来终止一个事务块。

在已经在一个事务块中时发出 **`BEGIN`** 将惹出一个警告消息。事务状态不会被影响。要在一个事务块中嵌套事务，可以使用保存点（见 [SAVEPOINT](#)）。

由于向后兼容的原因，连续的 **transaction_modes** 之间的逗号可以被省略。

示例

开始一个事务块：

```
BEGIN;
```

兼容性

BEGIN 是一种 IvorySQL 语言扩展。它等效于 SQL 标准的命令 **START TRANSACTION**，它的参考页包含额外的兼容性信息。

DEFERRABLE transaction_mode 是一种 IvorySQL 语言扩展。

附带地，**`BEGIN`** 关键词被用于嵌入式 SQL 中的一种不同目的。在移植数据库应用时，我们建议小心对待事务语义。

COMMIT — 提交当前事务

大纲

`COMMIT` 提交当前事务。所有由该事务所作的更改会变得对他人可见并且被保证在崩溃发生时仍能持久。

参数

WORK TRANSACTION

可选的关键词。它们没有效果。

AND CHAIN

如果指定了 **AND CHAIN**，则立即启动与刚刚完成的事务具有相同事务特征（参见 [SET TRANSACTION](#)）的新事务。否则，没有新事务被启动。

注解

使用 [ROLLBACK](#)中止一个事务。

当不在一个事务内时发出 **COMMIT** 不会产生危害，但是它会产生一个警告消息。当 `COMMIT AND CHAIN` 不在事务内时是一个错误。

示例

要提交当前事务并且让所有更改持久化：

```
COMMIT;
```

兼容性

命令 `COMMIT` 符合SQL标准。表单 `COMMIT TRANSACTION` 为IvorySQL扩展。

COMMIT PREPARED — 提交一个早前为两阶段提交预备的事务

大纲

```
COMMIT PREPARED transaction_id
```

描述

`COMMIT PREPARED` 提交一个处于预备状态的事务。

参数

transaction_id

要被提交的事务的事务标识符。

注解

要提交一个预备的事务，你必须是原先执行该事务的同一用户或者超级用户。但是不需要处于执行该事务的同一会话中。

这个命令不能在一个事务块中执行。该预备事务将被立刻提交。

pg_prepared_xacts系统视图中列出了所有当前可用的预备事务。

例子

提交由事务标识符`foobar`标识的事务：

```
COMMIT PREPARED 'foobar';
```

兼容性

COMMIT PREPARED 是一种IvorySQL扩展。其意图是用于外部事务管理系统，其中有些已经被标准涵盖（例如X/Open XA），但是那些系统的SQL方面未被标准化。

END - 提交当前事务

大纲

```
END [ WORK | TRANSACTION ] [ AND [ NO ] CHAIN ]
```

描述

END

提交当前事务。所有该事务做的更改使得对他人可见并且被保证发生崩溃时仍然是持久的。这个命令是一种IvorySQL扩展，它等效于 **COMMIT**。

参数

WORK TRANSACTION

可选关键词，它们没有效果。

AND CHAIN

如果规定了`AND CHAIN`，则立即启动与刚完成事务具有相同事务特征(参见 [SET TRANSACTION](#))的新事务。否则，没有新事务被启动。

注解

使用 **ROLLBACK**可以中止一个事务。

当不在一个事务中时发出`END`没有危害，但是会产生一个警告消息。

示例

要提交当前事务并且让所有更改持久化：

```
END;
```

兼容性

END 是一种IvorySQL扩展，它提供和 **COMMIT**等效的功能，后者在SQL标准中指定。

PREPARE TRANSACTION — 为两阶段提交准备当前事务

大纲

```
PREPARE TRANSACTION transaction_id
```

描述

``PREPARE`

`TRANSACTION`` 为两阶段提交准备当前事务。在这个命令之后，该事务不再与当前会话关联。相反，它的状态被完全存储在磁盘上，并且有很高的可能性它会被提交成功（即使在请求提交前发生数据库崩溃）。

一旦被准备好，事务稍后就可以分别用 **COMMIT PREPARED** 或者 **ROLLBACK PREPARED** 提交或者回滚。可以从任何会话而不仅仅是执行原始事务的会话中发出这些命令。

从发出命令的会话的角度来看，``PREPARE`

`TRANSACTION`` 不像 ``ROLLBACK`` 命令：在执行它之后，就没有活跃的当前事务，并且该预备事务的效果也不再可见（如果该事务被提交，效果将重新变得可见）。

如果由于任何原因 ``PREPARE TRANSACTION`` 命令失败，它会变成一个 ``ROLLBACK``：当前事务会被取消。

参数

transaction_id

一个任意的标识符，``COMMIT PREPARED`` 或者 ``ROLLBACK`

`PREPARED`` 以后将用这个标识符来标识这个事务。该标识符必须写成一个字符串，并且长度必须小于200字节。它也不能与任何当前已经准备好的事务的标识符相同。

注解

PREPARE

TRANSACTION` 并不是设计为在应用或者交互式会话中使用。它的目的是允许一个外部事务管理器在多个数据库或者其他事务性来源之间执行原子的全局事务。除非你在编写一个事务管理器，否则你可能不会用到 **PREPARE TRANSACTION**。

这个命令必须在一个事务块中使用。事务块用 **BEGIN** 开始。

当前在已经执行过任何涉及到临时表或者会话的临时命名空间、创建带 ``WITH`

`HOLD`` 的游标或者执行 ``LISTEN``、``UNLISTEN`` 或

``NOTIFY`` 的事务中，不允许 ``PREPARE`` 该事务。这些特性与当前会话绑定得太紧密，所以对一个要被准备的事务来说没有什么用处。

如果用 ``SET``（不带 ``LOCAL`` 选项）修改过事务的任何运行时参数，这些效果会持续到 ``PREPARE`

`TRANSACTION`` 之后，并且将不会被后续的任何 ``COMMIT PREPARED`` 或 ``ROLLBACK`

`PREPARED`` 所影响。因此，在这一方面 ``PREPARE`

`TRANSACTION`` 的行为更像 ``COMMIT`` 而不是 ``ROLLBACK``。

所有当前可用的准备好事务被列在 **pg_prepared_xacts** 系统视图中。

小心

让一个事务处于准备好状态太久是不明智的。这将会干扰 ``VACUUM`` 回收存储的能力，并且在极限情况下可能导致数据库关闭以阻止事务ID回卷（见 [第 25.1.5 节](#)）。

还要记住，该事务会继续持有它已经持有的锁。该特性的设计用法是，只要一个外部事务管理器已经验证其他数据库也准备好了要提交，一个准备好的事务将被正常地提交或者回滚。

如果没有建立一个外部事务管理器来跟踪准备好的事务并且确保它们被迅速地结束，最好禁用准备好事务特性（设置

[max_prepared_transactions](#) 为零）。这将防止意外地创建准备好事务，不然该事务有可能被忘记并且最终导致问题。

例子

为两阶段提交准备当前事务，使用 ``foobar`` 作为事务标识符：

```
PREPARE TRANSACTION 'foobar';
```

兼容性

``PREPARE`

`TRANSACTION`` 是一种IvorySQL扩展。其意图是用于外部事务管理系统，其中有些已经被标准涵盖（例如X/Open XA），但是那些系统的SQL方面未被标准化。

ROLLBACK — 中止当前事务

大纲

```
ROLLBACK [ WORK | TRANSACTION ] [ AND [ NO ] CHAIN ]
```

描述

``ROLLBACK`` 回滚当前事务并且导致该事务所作的所有更新都被抛弃。

参数

WORK TRANSACTION

可选关键词，没有效果。

AND CHAIN

如果指定了``AND CHAIN``，则立即启动与刚刚完成事务具有相同事务特征（参见 [SET TRANSACTION](#)）的新事务。否则，不会启动任何新事务。

注解

使用 **COMMIT** 可成功地终止一个事务。

在一个事务块之外发出``ROLLBACK`` 会发出一个警告并且不会有效果。事务块之外的``ROLLBACK AND CHAIN`` 是一个错误。

示例

要中止所有更改：

```
ROLLBACK;
```

兼容性

命令``ROLLBACK`` 符合SQL标准。窗体``ROLLBACK TRANSACTION`` 是一个IvorySQL扩展。

ROLLBACK PREPARED — 取消一个之前为两阶段提交准备好的事务

大纲

```
ROLLBACK PREPARED transaction_id
```

描述

``ROLLBACK PREPARED`` 回滚一个处于准备好状态的事务。

参数

transaction_id

要被回滚的事务的事务标识符。

注解

要回滚一个准备好的事务，你必须是原先执行该事务的同一个用户或者是一个超级用户。但是你必须处在执行该事务的同一个会话中。

这个命令不能在一个事务块内被执行。准备好的事务会被立刻回滚。

pg_prepared_xacts系统视图中列出了当前可用的所有准备好的事务。

例子

用事务标识符`foobar`回滚对应的事务：

```
ROLLBACK PREPARED 'foobar';
```

兼容性

`ROLLBACK PREPARED`是一种IvorySQL扩展。其意图是用于外部事务管理系统，其中有些已经被标准涵盖（例如X/Open XA），但是那些系统的SQL方面未被标准化。

SAVEPOINT — 在当前事务中定义一个新的保存点

大纲

```
SAVEPOINT savepoint_name
```

描述

`SAVEPOINT`在当前事务中建立一个新保存点。

保存点是事务内的一种特殊标记，它允许所有在它被建立之后执行的命令被回滚，把该事务的状态恢复到它处于保存点时的样子。

参数

savepoint_name

给新保存点的名字。

注解

使用 **ROLLBACK TO**回滚到一个保存点。使用 **RELEASE SAVEPOINT**销毁一个保存点，但保持在它被建立之后执行的命令的效果。

保存点只能在一个事务块内建立。可以在一个事务内定义多个保存点。

示例

要建立一个保存点并且后来撤销在它建立之后执行的所有命令的效果：

```
BEGIN;  
  INSERT INTO table1 VALUES (1);  
  SAVEPOINT my_savepoint;  
  INSERT INTO table1 VALUES (2);  
  ROLLBACK TO SAVEPOINT my_savepoint;
```

```
INSERT INTO table1 VALUES (3);
COMMIT;
```

上面的事务将插入值 1 和 3，但不会插入 2。

要建立并且稍后销毁一个保存点：

```
BEGIN;
  INSERT INTO table1 VALUES (3);
  SAVEPOINT my_savepoint;
  INSERT INTO table1 VALUES (4);
  RELEASE SAVEPOINT my_savepoint;
COMMIT;
```

上面的事务将插入 3 和 4。

兼容性

当建立另一个同名保存点时，SQL 要求之前的那个保存点自动被销毁。在 IvorySQL 中，旧的保存点会被保留，不过在进行回滚或释放时只能使用最近的那一个（用 `RELEASE SAVEPOINT` 释放较新的保存点将会导致较旧的保存点再次变得可以被 `ROLLBACK TO SAVEPOINT` 和 `RELEASE SAVEPOINT` 访问）。在其他方面，`SAVEPOINT` 完全符合 SQL。

SET CONSTRAINTS — 为当前事务设置约束检查时机

大纲

```
SET CONSTRAINTS { ALL | name [, ...] } { DEFERRED | IMMEDIATE }
```

描述

`SET

CONSTRAINTS` 设置当前事务内约束检查的行为。`IMMEDIATE` 约束在每个语句结束时被检查。`DEFERRED` 约束直到事务提交时才被检查。每个约束都有自己的 `IMMEDIATE` 或 `DEFERRED` 模式。

在创建时，一个约束会被给定三种特性之一：**DEFERRABLE INITIALLY DEFERRED**、**DEFERRABLE INITIALLY IMMEDIATE** 或者 **NOT DEFERRABLE**。第三类总是 `IMMEDIATE` 并且不会受到 `SET CONSTRAINTS` 命令的影响。前两类在每个事务开始时都处于指定的模式，但是它们的行为可以在一个事务内用 `SET CONSTRAINTS` 更改。

带有一个约束名称列表的 `SET

CONSTRAINTS` 只更改那些约束（都必须是可延迟的）的模式。每一个约束名称都可以是模式限定的。如果没有指定模式名称，则当前的模式搜索路径将被用来寻找第一个匹配的名称。`SET CONSTRAINTS ALL` 更改所有可延迟约束的模式。

当 `SET

CONSTRAINTS` 把一个约束的模式从 `DEFERRED` 改成 `IMMEDIATE` 时，新模式会有追溯效果：任何还没有解决的数据修改（本来会在事务结束时被检查）会转而在 `SET CONSTRAINTS` 命令的执行期间被检查。如果任何这种约束被违背，**SET CONSTRAINTS 将会失败（并且不会改变该约束模式）**。这样，`SET CONSTRAINTS` 可以被用来在一个事务中的特定点强制进行约束检查。

当前，只有 `UNIQUE`、**PRIMARY KEY**、

REFERENCES（外键）以及 `EXCLUDE` 约束受到这个设置的影响。`NOT NULL` 以及 `CHECK` 约束总是在一行被插入或修改时立即检查（不是在语句结束时）。没有被声明为 `DEFE

RRABLE`的唯一和排除约束也会被立刻检查。

被声明为“约束触发器”的触发器的引发也受到这个设置的控制 — 它们会在相关约束被检查的同时被引发。

注解

因为IvorySQL并不要求约束名称在模式内唯一（但是在表内要求唯一），可能有多于一个约束匹配指定的约束名称。在这种情况下`SET CONSTRAINTS`将会在所有的匹配上操作。对于一个非模式限定的名称，一旦在搜索路径中的某个模式中发现一个或者多个匹配，路径中后面的模式将不会被搜索。

这个命令只修改当前事务内约束的行为。在事务块外部发出这个命令会产生一个警告并且也不会有任何效果。

兼容性

这个命令符合SQL标准中定义的行为，但有一点限制：在IvorySQL中，它不会应用在`NOT NULL`和`CHECK`约束上。还有，IvorySQL会立刻检查非可延迟的唯一约束，而不是按照标准建议的在语句结束时检查。

SET TRANSACTION — 设置当前事务的特性

大纲

```
SET TRANSACTION transaction_mode [, ...]
SET TRANSACTION SNAPSHOT snapshot_id
SET SESSION CHARACTERISTICS AS TRANSACTION transaction_mode [, ...]
```

其中transaction_mode是下列之一：

```
ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ | READ COMMITTED | READ
UNCOMMITTED }
READ WRITE | READ ONLY
[ NOT ] DEFERRABLE
```

描述

`SET TRANSACTION`命令设置当前会话的特性。`SET SESSION CHARACTERISTICS`设置一个会话后续事务的默认事务特性。在个体事务中可以用`SET TRANSACTION`覆盖这些默认值。

可用的事务特性是事务隔离级别、事务访问模式（读/写或只读）以及可延迟模式。此外，可以选择一个快照，不过只能用于当前事务而不能作为会话默认值。

一个事务的隔离级别决定当其他事务并行运行时该事务能看见什么数据：

READ COMMITTED

一个语句只能看到在它开始前提交的行。这是默认值。

REPEATABLE READ

当前事务的所有语句只能看到这个事务中执行的第一个查询或者数据修改语句之前提交的行。

SERIALIZABLE

当前事务的所有语句只能看到这个事务中执行的第一个查询或者数据修改语句之前提交的行。如果并发的可序列化事务间的读写模式可能导致一种那些事务串行（一次一个）执行时不可能出现的情况，其中之一将会被回滚并且得到一个`serialization\_failure`错误。

SQL标准定义了一种额外的级别：**READ UNCOMMITTED**。在IvorySQL中`READ UNCOMMITTED`被视作`READ COMMITTED`。

一个事务执行了第一个查询或者数据修改语句（**SELECT**、**INSERT**、**DELETE**、**UPDATE**、**FETCH**或**COPY**）之后就无法更改事务隔离级别。更多有关事务隔离级别和并发控制的信息可见 第 13 章。

事务的访问模式决定该事务是否为读/写或者只读。读/写是默认值。当一个事务为只读时，如果SQL命令`INSERT`、**UPDATE**、**DELETE**和**COPY FROM**要写的表不是一个临时表，则它们不被允许。不允许**CREATE**、**ALTER**以及**DROP**命令。不允许**COMMENT**、**GRANT**、**REVOKE**、**TRUNCATE**。如果`EXPLAIN ANALYZE`和`EXECUTE`要执行的命令是上述命令之一，则它们也不被允许。这是一种高层的只读概念，它不能阻止所有对磁盘的写入。

只有事务也是`SERIALIZABLE`以及`READ ONLY`时，`DEFERRABLE`事务属性才会有效。当一个事务的所有这三个属性都被选择时，该事务在第一次获取其快照时可能会阻塞，在那之后它运行时就不会有`SERIALIZABLE`事务的开销并且不会有任何牺牲或者被一次序列化失败取消的风险。这种模式很适合于长时间运行的报表或者备份。

SET TRANSACTION

SNAPSHOT命令允许新的事务使用与一个现有事务相同的*快照*运行。已经存在的事务必须已经把它快照用`pg\_export\_snapshot`函数（见 第 9.27.5 节）导出。该函数会返回一个快照标识符，**SET TRANSACTION SNAPSHOT**需要被给定一个快照标识符来指定要导入的快照。在这个命令中该标识符必须被写成一个字符串，例如`000003A1-1`。**SET TRANSACTION SNAPSHOT**只能在一个事务的开始执行，并且要在该事务的第一个查询或者数据修改语句（`SELECT`、**INSERT**、**DELETE**、**UPDATE**、**FETCH**或**COPY**）之前执行。此外，该事务必须已经被设置为`SERIALIZABLE`或者`REPEATABLE READ`隔离级别（否则，该快照将被立刻抛弃，因为`READ COMMITTED`模式会为每一个命令取一个新快照）。如果导入事务使用了`SERIALIZABLE`隔离级别，那么导入快照的事务也必须使用该隔离级别。还有，一个非只读可序列化事务不能导入来自只读事务的快照。

注解

如果执行`SET TRANSACTION`之前没有`START TRANSACTION`或者`BEGIN`，它会发出一个警告并且不会有任何效果。

可以通过在`BEGIN`或者`START TRANSACTION`中指定想要的`transaction\_modes`来省掉**SET TRANSACTION**。但是在`SET TRANSACTION SNAPSHOT`中该选项不可用。

会话默认的事务模式也可以通过配置参数

[default_transaction_isolation](#)、<http://www.postgresql.org/docs/17/runtime-config-client.html#GUC-DEFAULT-TRANSACTION-READ-ONLY>[[default_transaction_read_only](#)]和

[default_transaction_deferrable](#)来设置或检查（实际上`SET SESSION CHARACTERISTICS`只是用`SET`设置这些变量的等效体）。这意味着可以通过配置文件、**ALTER DATABASE**等方式设置默认值。详见 第 20 章。

当前事务的模式可以类似的通过配置参数 [transaction_isolation](#)、[transaction_read_only](#)、和 [transaction_deferrable](#)来设置或检查。设置这其中一个参数的作用与相应的`SET TRANSACTION`选项相同，在它何时可以完成方面，也有相同的限制。但是，这些参数不能在配置文件中设置，或者从活动SQL以外的任何来源来设置。

示例

要用一个已经存在的事务的同一快照开始一个新事务，首先要从该现有事务导出快照。这将会返回快照标识符，例如：

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT pg_export_snapshot();
pg_export_snapshot
-----
00000003-0000001B-1
```

(1 row)

然后在一个新开始的事务的开头把该快照标识符用在一个`SET TRANSACTION SNAPSHOT`命令中：

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
SET TRANSACTION SNAPSHOT '00000003-0000001B-1';
```

兼容性

SQL标准中定义了这些命令，不过`DEFERRABLE`事务模式和`SET TRANSACTION SNAPSHOT`形式除外，这两者是IvorySQL扩展。

SERIALIZABLE`是标准中默认的事务隔离级别。在IvorySQL中默认值是普通的`READ COMMITTED`，但是你可以按上述的方式更改。

在SQL标准中，可以用这些命令设置一个其他的事务特性：诊断区域的尺寸。这个概念与嵌入式SQL有关，并且因此没有在IvorySQL服务器中实现。

SQL标准要求连续的 **transaction_modes** 之间有逗号，但是出于历史原因IvorySQL允许省略逗号。

START TRANSACTION — 开始一个事务块

大纲

```
START TRANSACTION [ transaction_mode [, ...] ]
```

其中 transaction_mode 是下列之一：

```
ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ | READ COMMITTED | READ  
UNCOMMITTED }  
READ WRITE | READ ONLY  
[ NOT ] DEFERRABLE
```

描述

这个命令开始一个新的事务块。如果指定了隔离级别、读写模式或者可延迟模式，新的事务将会具有这些特性，就像执行了 **SET TRANSACTION** 一样。这和 **BEGIN** 命令一样。

参数

这些参数对于这个语句的含义可参考 [SET TRANSACTION](#)。

兼容性

在标准中，没有必要发出`START TRANSACTION`来开始一个事务块：任何SQL命令会隐式地开始一个块。IvorySQL的行为可以被视作在每个命令之后隐式地发出一个没有跟随在`START TRANSACTION`（或者`BEGIN`）之后的`COMMIT`并且因此通常被称作“自动提交”。为了方便，其他关系型数据库系统也可能会提供自动提交特性。

DEFERRABLE transaction_mode 是一种IvorySQL语言扩展。

SQL标准要求连续的 **transaction_modes** 之间有逗号，但是由于历史原因IvorySQL允许省略逗号。

Sql参考（第 4 章 SQL语法）

词法结构

SQL输入由一个命令序列组成。一个命令由一个记号的序列构成，并由一个分号（“;”）终结。输入流的末端也会标志一个命令的结束。具体哪些记号是合法的与具体命令的语法有关。

一个记号可以是一个关键词、一个标识符、一个带引号的标识符、一个 literal（或常量）或者一个特殊字符符号。记号通常以空白（空格、制表符、新行）来分隔，但在无歧义时并不强制要求如此（唯一的例子是一个特殊字符紧挨着其他记号）。

例如，下面是一个（语法上）合法的SQL输入：

```
SELECT * FROM MY_TABLE;  
UPDATE MY_TABLE SET A = 5;  
INSERT INTO MY_TABLE VALUES (3, 'hi there');
```

这是一个由三个命令组成的序列，每一行一个命令（尽管这不是必须地，在同一行中可以有超过一个命令，而且命令还可以被跨行分割）。

另外，注释也可以出现在SQL输入中。它们不是记号，它们和空白完全一样。

根据标识命令、操作符、参数的记号不同，SQL的语法不很一致。最前面的一些记号通常是命令名，因此在上面的例子中我们通常会说一个“SELECT”、一个“UPDATE”和一个“INSERT”命令。但是例如`UPDATE`命令总是要求一个`SET`记号出现在一个特定位置，而`INSERT`则要求一个`VALUES`来完成命令。每个命令的精确语法规则在 [第 VI 部分](#) 中介绍。

标识符和关键词

上例中的`SELECT`、`UPDATE`或`VALUES`记号是关键词的例子，即SQL语言中具有特定意义的词。记号`MY\_TABLE`和`A`则是标识符的例子。它们标识表、列或者其他数据库对象的名字，取决于使用它们的命令。因此它们有时也被简称为“名字”。关键词和标识符具有相同的词法结构，这意味着我们无法在没有语言知识的前提下区分一个标识符和关键词。一个关键词的完整列表可以在 [附录 C](#) 中找到。

SQL标识符和关键词必须以一个字母（**a-z**，也可以是带变音符的字母和非拉丁字母）或一个下划线（**_**）开始。后续字符可以是字母、下划线（**_**）、数字（**0-9**）或美元符号（**\$**）。注意根据SQL标准的字母规定，美元符号是不允许出现在标识符中的，因此它们的使用可能会降低应用的可移植性。SQL标准不会定义包含数字或者以下划线开头或结尾的关键词，因此这种形式的标识符不会与未来可能的标准扩展冲突。

系统中一个标识符的长度不能超过`NAMEDATALEN` - 1字节，在命令中可以写超过此长度的标识符，但是它们会被截断。默认情况下，`NAMEDATALEN`的值为64，因此标识符的长度上限为63字节。如果这个限制有问题，可以在`src/include/pg\_config\_manual.h`中修改`NAMEDATALEN`常量。

关键词和不被引号修饰的标识符是大小写不敏感的。因此：

```
UPDATE MY_TABLE SET A = 5;
```

可以等价地写成：

```
uPDaTE my_Table SeT a = 5;
```

一个常见的习惯是将关键词写成大写，而名称写成小写，例如：

```
UPDATE my_table SET a = 5;
```

这里还有第二种形式的标识符：受限标识符*或*被引号修饰的标识符。它是由双引号 (") 包围的一个任意字符序列。一个受限标识符总是一个标识符而不会是一个关键字。因此`"select"`可以用于引用一个名为“select”的列或者表，而一个没有引号修饰的`select`则会被当作一个关键词，从而在本应使用表或列名的地方引起解析错误。在上例中使用受限标识符的例子如下：

```
UPDATE "my_table" SET "a" = 5;
```

受限标识符可以包含任何字符，除了代码为0的字符（如果要包含一个双引号，则写两个双引号）。这使得可以构建原本不被允许的表或列的名称，例如包含空格或花号的名字。但是长度限制依然有效。

引用标识符也使其区分大小写，而未引用的名称总是折叠成小写。例如，标识符`FOO`、`foo`和`"foo"`在IvorySQL中被认为是相同的，但是`"Foo"`和`"F00"`与这三个不同，并且彼此不同。（在IvorySQL中，将不带引号的名称折叠为小写与SQL标准不兼容，SQL标准规定不带引号的名称应折叠为大写。因此，根据标准，`foo`应等同于`"F00"`而不是`"foo"`。如果您想编写可移植应用程序，建议您始终引用某个特定的名称，或者永远不要引用它。）

一种受限标识符的变体允许包括转义的用代码点标识的Unicode字符。这种变体以`U&`（大写或小写U跟上一个花号）开始，后面紧跟双引号修饰的名称，两者之间没有任何空白，如`U&"foo"`（注意这里与操作符`&`似乎有一些混淆，但是在`&`操作符周围使用空白避免了这个问题）。在引号内，Unicode字符可以以转义的形式指定：反斜线接上4位16进制代码点号码或者反斜线和加号接上6位16进制代码点号码。例如，标识符`"data"`可以写成：

```
U&"d\0061t\+000061"
```

下面的例子用斯拉夫语字母写出了俄语单词“slon”（大象）：

```
U&"\0441\043B\043E\043D"
```

如果希望使用其他转义字符来代替反斜线，可以在字符串后使用`UESCAPE`子句，例如：

```
U&"d!0061t!+000061" UESCAPE '!'
```

转义字符可以是除了16进制位、加号、单引号、双引号、空白字符之外的任意单个字符。请注意，转义字符在`UESCAPE`之后用单引号而不是双引号书写。

为了在标识符中包括转义字符本身，将其写两次即可。

4位或6位转义形式都可以被用来定义UTF-16代理对来组成代码点大于U+FFFF的字符，尽管6位形式的存在使得这种做法变得不必要（代理对并不被直接存储，而是绑定成一个单独的代码点）。

如果服务器编码不是UTF-

8，则由其中一个转义序列标识的Unicode代码点转换为实际的服务器编码；如果不可能，则报告错误。

常量

在IvorySQL中有三种

隐式类型常量：字符串、位串和数字。常量也可以被指定显式类型，这可以使得它被更精确地展示以及更有效地处理。这些选择将会在后续小节中讨论。

字符串常量

在SQL中，一个字符串常量是一个由单引号（'）包围的任意字符序列，例如 'This is a string'。为了在一个字符串中包括一个单引号，可以写两个相连的单引号，例如 'Dianne's horse'。注意这和一个双引号（"）不同。

两个只由空白及至少一个新行分隔的字符串常量会被连接在一起，并且将作为一个写在一起的字符串常量来对待。例如：

```
SELECT 'foo'
'bar';
```

等同于：

```
SELECT 'foobar';
```

但是：

```
SELECT 'foo'      'bar';
```

则不是合法的语法（这种有些奇怪的行为是SQL指定的，IvorySQL遵循了该标准）。

C风格转义的字符串常量

IvorySQL也接受“转义”字符串常量，这也是SQL标准的一个扩展。一个转义字符串常量可以通过在开单引号前面写一个字母'E'（大写或小写形式）来指定，例如'E' foo'（当一个转义字符串常量跨行时，只在第一个开引号之前写'E'）。在一个转义字符串内部，一个反斜线字符（\）会开始一个C风格的反斜线转义序列，在其中反斜线和后续字符的组合表示一个特殊的字节值（如表4.1中所示）。

表 4.1. 反斜线转义序列

反斜线转义序列	解释
\b	退格
\f	换页
\n	换行
\r	回车
\t	制表符
\o, *\oo*, *\ooo* (o = 0-7)	八进制字节值
\x*h, \x`hh` (h* = 0-9, A-F)	十六进制字节值
\u*`xxxx, \U`xxxxxxxx` (x* = 0-9, A-F)	16 或 32-位十六进制 Unicode 字符值

跟随在一个反斜线后面的任何其他字符被当做其字面意思。因此，要包括一个反斜线字符，请写两个反斜线（\\）。在一个转义字符串中包括一个单引号除了普通方法'`之外，还可以写成'\`。

你要负责保证你创建的字节序列由服务器字符集编码中合法的字符组成，特别是在使用八进制或十六进制转义时。一个有用的替代方法是使用Unicode转义或替代的Unicode转义语法，如第4.1.2.3节中所述；然后服务器将检查字符转换是否可行。

小心

如果配置参数

`standard_conforming_strings`为`off`，那么IvorySQL对常规字符串常量和转义字符串常量中的反斜线转义都识别。不过，在IvorySQL中该参数的默认值为`on`，意味着只在转义字符串常量中识别反斜线转义。这种行为更兼容标准，但是可能打断依赖于历史行为（反斜线转义总是会被识别）的应用。作为一种变通，你可以设置该参数为`off`，但是最好迁移到符合新的行为。如果你需要使用一个反斜线转义来表示一个特殊字符，为该字符串常量写上一个`E`。在`standard\_conforming\_strings`之外，配置参数 `escape_string_warning`和 `backslash_quote`也决定了如何对待字符串常量中的反斜线。代码零的字符不能出现在一个字符串常量中。

带有 Unicode 转义的字符串常量

IvorySQL也支持另一种类型的字符串转义语法，它允许用代码点指定任意Unicode字符。一个Unicode转义字符串常量开始于`U&`（大写或小写形式的字母U，后跟花号），后面紧跟着开引号，之间没有任何空白，例如`U&'foo'`（注意这产生了与操作符`&`的混淆。在操作符周围使用空白来避免这个问题）。在引号内，Unicode字符可以通过写一个后跟4位十六进制代码点编号或者一个前面有加号的6位十六进制代码点编号的反斜线来指定。例如，字符串`data`可以被写为

```
U&'d\0061t\+000061'
```

下面的例子用斯拉夫字母写出了俄语的单词“slon”（大象）：

```
U&'\0441\043B\043E\043D'
```

如果想要一个不是反斜线的转义字符，可以在字符串之后使用`UESCAPE`子句来指定，例如：

```
U&'d!0061t!+000061' UESCAPE '!'
```

转义字符可以是出一个十六进制位、加号、单引号、双引号或空白字符之外的任何单一字符。

要在一个字符串中包括一个表示其字面意思的转义字符，把它写两次。

4位或6位转义形式可用于指定UTF-

16代理项对，以组成代码点大于U+FFFF的字符，尽管从技术上讲，6位形式的可用性使得这是不必要的（代理项对不是直接存储的，而是合并到单个代码点中。）

如果服务器编码不是UTF-

8，则由这些转义序列之一标识的Unicode代码点将转换为实际的服务器编码；如果不可能，则会报告错误。

此外，字符串常量的Unicode转义语法仅在配置参数

`standard_conforming_strings`开启时才有效。这是因为否则这种语法可能会混淆解析SQL语句的客户端，可能导致SQL注入和类似的安全问题。如果该参数设置为off，则此语法将被拒绝并显示错误消息。

美元引用的字符串常量

虽然用于指定字符串常量的标准语法通常都很方便，但是当字符串中包含了很多单引号或反斜线时很难理解它，因为每一个都需要被双写。要在这种情形下允许可读性更好的查询，IvorySQL提供了另一种被称为“美元引用”的方式来书写字符串常量。一个美元引用的字符串常量由一个美元符号（\$）、一个可选的另个或更多字符的“标签”、另一个美元符号、一个构成字符串内容的任意字符序列、一个美元符号、开始这个美元引用的相同标签和一个美元符号组成。例如，这里有两种不同的方法使用美元引用指定字符串“Dianne’s horse”：

```
$$Dianne's horse$$
$SomeTag$Dianne's horse$SomeTag$
```


注意在美元引用字符串中，单引号可以在不被转义的情况下使用。事实上，在一个美元引用字符串中不需要对字符进行转义：字符串内容总是按其字面意思写出。反斜线不是特殊的，并且美元符号也不是特殊的，除非它们是匹配开标签的一个序列的一部分。

可以通过在每一个嵌套级别上选择不同的标签来嵌套美元引用字符串常量。这最常被用在编写函数定义上。例如：

```
$function$
BEGIN
    RETURN ($1 ~ $q$[\t\r\n\v\\]$q$);
END;
$function$
```

这里，序列`\$q\$[\t\r\n\v\\]\$q\$表示一个美元引用的文字串`[\t\r\n\v\\]`，当该函数体被IvorySQL执行时它将被识别。但是因为该序列不匹配外层的美元引用的定界符`\$function\$`，它只是一些在外层字符串所关注的常量中的字符而已。

一个美元引用字符串的标签（如果有）遵循一个未被引用标识符的相同规则，除了它不能包含一个美元符号之外。标签是大小写敏感的，因此`\$tag\$string content\$tag\$是正确的，但是\$TAG\$string content\$tag\$`不正确。

一个跟着一个关键词或标识符的美元引用字符串必须用空白与之分隔开，否则美元引用定界符可能会被作为前面标识符的一部分。

美元引用不是SQL标准的一部分，但是在书写复杂字符串文字方面，它常常是一种比兼容标准的单引号语法更方便的方法。当要表示的字符串常量位于其他常量中时它特别有用，这种情况常常在过程函数定义中出现。如果用单引号语法，上一个例子中的每个反斜线将必须被写成四个反斜线，这在解析原始字符串常量时会被缩减到两个反斜线，并且接着在函数执行期间重新解析内层字符串常量时变成一个。

位串常量

位串常量看起来像常规字符串常量在开引号之前（中间无空白）加了一个`B`（大写或小写形式），例如`B'1001'`。位串常量中允许的字符只有`0`和`1`。

作为一种选择，位串常量可以用十六进制记号法指定，使用一个前导`X`（大写或小写形式），例如`X'1FF'`。这种记号法等价于一个用四个二进制位取代每个十六进制位的位串常量。

两种形式的位串常量可以以常规字符串常量相同的方式跨行继续。美元引用不能被用在位串常量中。

数字常量

在这些一般形式中可以接受数字常量：

```
digits
digits.[digits][e[+-]digits]
[digits].digits[e[+-]digits]
digitse[+-]digits
```

其中 **digits** 是一个或多个十进制数字（0到9）。如果使用了小数点，在小数点前面或后面必须至少有一个数字。如果存在一个指数标记（**e**），在其后必须跟着至少一个数字。在该常量中不能嵌入任何空白或其他字符。注意任何前导的加号或减号并不实际被考虑为常量的一部分，它是一个应用到该常量的操作符。

这些是合法数字常量的例子：


```
42
3.5
4.
.001
5e2
1.925e-3
```

如果一个不包含小数点和指数的数字常量的值适合类型`integer`（32位），它首先被假定为类型`integer`。否则如果它的值适合类型`bigint`（64位），它被假定为类型`bigint`。再否则它会被取做类型`numeric`。包含小数点和/或指数的常量总是首先被假定为类型`numeric`。

一个数字常量初始指派的数据类型只是类型转换算法的一个开始点。在大部分情况中，常量将被根据上下文自动被强制到最合适的类型。必要时，你可以通过造型它来强制一个数字值被解释为一种指定数据类型。例如，你可以这样强制一个数字值被当做类型`real`（**float4**）：

```
REAL '1.23' -- string style
1.23::REAL  -- IvorySQL (historical) style
```

这些实际上只是接下来要讨论的一般造型记号的特例。

其他类型的常量

一种任意类型的一个常量可以使用下列记号中的任意一种输入：

```
type 'string'
'string'::type
CAST ( 'string' AS type )
```

字符串常量的文本被传递到名为 **type** 的类型的输入转换例程中。其结果是指定类型的一个常量。如果对该常量的类型没有歧义（例如，当它被直接指派给一个表列时），显式类型造型可以被忽略，在那种情况下它会被自动强制。

字符串常量可以使用常规SQL记号或美元引用书写。

也可以使用一个类似函数的语法来指定一个类型强制：

```
typename ( 'string' )
```

但是并非所有类型名都可以用在这种方法中，详见 [第 4.2.9 节](#)。

如 [第 4.2.9 节](#) 中讨论的，`::`

、`CAST()` 以及函数调用语法也可以被用来指定任意表达式的运行时类型转换。要避免语法歧义，`type 'string'` 语法只能被用来指定简单文字常量的类型。`type 'string'` 语法上的另一个限制是它无法对数组类型工作，指定一个数组常量的类型可使用 `::`` 或 `CAST()`。

`CAST()` 语法符合SQL。``type`

`'string'` 语法是该标准的一般化：SQL指定这种语法只用于一些数据类型，但是IvorySQL允许它用于所有类型。带有`::``的语法是IvorySQL的历史用法，就像函数调用语法一样。

操作符

一个操作符名是最多`NAMEDATALEN`-1（默认为63）的一个字符序列，其中的字符来自下面的列表：

```
\+ - * / < > = ~ ! @ # % ^ & | ` ?
```

不过，在操作符名上有一些限制：

- `--`和`/*``不能在一个操作符名的任何地方出现，因为它们将被作为一段注释的开始。
- 一个多字符操作符名不能以``+或`-``结尾，除非该名称也至少包含这些字符中的一个：

```
~ ! @ # % ^ & | ` ?
```

例如，`@`-``是一个被允许的操作符名，但`*`-``

不是。这些限制允许IvorySQL解析SQL兼容的查询而不需要在记号之间有空格。

当使用非SQL标准的操作符名时，你通常需要用空格分隔相邻的操作符来避免歧义。例如，如果你定义了一个名为`@`的前缀操作符`，你不能写``X*@Y``，你必须写``X`
@Y``来确保IvorySQL把它读作两个操作符名而不是一个。

特殊字符

一些不是数字字母的字符有一种不同于作为操作符的特殊含义。这些字符的详细用法可以在描述相应语法元素的地方找到。这一节只是为了告知它们的存在以及总结这些字符的目的。

- 跟随在一个美元符号（`$`）后面的数字被用来表示在一个函数定义或一个预备语句中的位置参数。在其他上下文中该美元符号可以作为一个标识符或者一个美元引用字符串常量的一部分。
- 圆括号（`()`）具有它们通常的含义，用来分组表达式并且强制优先。在某些情况中，圆括号被要求作为一个特定SQL命令的固定语法的一部分。
- 方括号（`[]`）被用来选择一个数组中的元素。更多关于数组的信息见 [第 8.15 节](#)。
- 逗号（`,`）被用在某些语法结构中来分割一个列表的元素。
- 分号（`;`）结束一个SQL命令。它不能出现在一个命令中间的任何位置，除了在一个字符串常量中或者一个被引用的标识符中。
- 冒号（`:`）被用来从数组中选择“切片”（见 [第 8.15 节](#)）。在某些SQL的“方言”（例如嵌入式SQL）中，冒号被用来作为变量名的前缀。
- 星号（`*`）被用在某些上下文中标记一个表的所有域或者组合值。当它被用作一个聚集函数的参数时，它还有一种特殊的含义，即该聚集不要求任何显式参数。
- 句点（`.`）被用在数字常量中，并且被用来分割模式、表和列名。

注释

一段注释是以双横杠开始并且延伸到行结尾的一个字符序列，例如：

```
-- This is a standard SQL comment
```

另外，也可以使用C风格注释块：

```
/* multiline comment
```

```
* with nesting: /* nested block comment */
*/
```

这里该注释开始于 `/` 并且延伸到匹配出现的 `/`。这些注释块可按照SQL标准中指定的方式嵌套，但和C中不同。这样我们可以注释掉一大段可能包含注释块的代码。

在进一步的语法分析前，注释会被从输入流中被移除并且实际被替换为空白。

操作符优先级

表 4.2显示了IvorySQL中操作符的优先级和结合性。大部分操作符具有相同的优先并且是左结合的。操作符的优先级和结合性被硬写在解析器中。如果您希望以不同于优先级规则所暗示的方式解析具有多个运算符的表达式，请添加括号。

表 4.2. 操作符优先级（从高到低）

操作符/元素	结合性	描述
<code>.</code>	左	表/列名分隔符
<code>::</code>	左	IvorySQL-风格的类型转换
<code>[]</code>	左	数组元素选择
<code>+ -</code>	右	一元加、一元减
<code>^</code>	左	指数
<code>* / %</code>	左	乘、除、模
<code>+ -</code>	左	加、减
(任意其他操作符)	左	所有其他本地以及用户定义的操作符
<code>BETWEEN IN LIKE ILIKE SIMILAR</code>		范围包含、集合成员关系、字符串匹配
<code><> = <=> => <></code>		比较操作符
<code>IS ISNULL NOTNULL</code>		<code>IS TRUE</code> 、 <code>IS FALSE</code> 、 <code>IS NULL</code> 、 <code>`IS DISTINCT FROM`</code> 等
<code>NOT</code>	右	逻辑否定
<code>AND</code>	左	逻辑合取
<code>OR</code>	左	逻辑析取

注意该操作符有限规则也适用于与上述内建操作符具有相同名称的用户定义的操作符。例如，如果你为某种自定义数据类型定义了一个 `“”` 操作符，它将具有和内建的 `“”` 操作符相同的优先级，不管你的操作符要做什么。

当一个模式限定的操作符名被用在 ``OPERATOR`` 语法中时，如下面的例子：

```
SELECT 3 OPERATOR(pg_catalog.+) 4;
```

``OPERATOR`` 结构被用来为“任意其他操作符”获得 表 4.2中默认的优先级。不管出现在 ``OPERATOR()`` 中的是哪个指定操作符，这都是真的。

注意

版本9.5之前的PostgreSQL使用的操作符优先级规则略有不同。特别是，`<=>`、`>=>`和

`<>` 习惯于被当作普通操作符，`IS` 测试习惯于具有较高的优先级。并且在一些认为 `NOT` 比 `BETWEEN` 优先级高的情况下，`NOT BETWEEN` 和相关的结构的行为不一致。为了更好地兼容SQL标准并且减少对逻辑上等价的结构不一致的处理，这些规则也得到了修改。在大部分情况下，这些变化不会导致行为上的变化，或者可能会产生 “no such operator” 错误，但可以通过增加圆括号解决。不过在一些极端情况中，查询可能在没有被报告解析错误的情况下发生行为的改变。

值表达式

值表达式被用于各种各样的环境中，例如在 `SELECT` 命令的目标列表中、作为 `INSERT` 或 `UPDATE` 中的新列值或者若干命令中的搜索条件。为了区别于一个表表达式（是一个表）的结果，一个值表达式的结果有时候被称为一个 标量。值表达式因此也被称为 标量表达式（或者甚至简称为表达式）。表达式语法允许使用算数、逻辑、集合和其他操作从原始部分计算值。

一个值表达式是下列之一：

- 一个常量或文字值
- 一个列引用
- 在一个函数定义体或预备语句中的一个位置参数引用
- 一个下标表达式
- 一个域选择表达式
- 一个操作符调用
- 一个函数调用
- 一个聚集表达式
- 一个窗口函数调用
- 一个类型转换
- 一个排序规则表达式
- 一个标量子查询
- 一个数组构造器
- 一个行构造器
- 另一个在圆括号（用来分组子表达式以及重载优先级）中的值表达式

在这个列表之外，还有一些结构可以被分类为一个表达式，但是它们不遵循任何一般语法规则。这些通常具有一个函数或操作符的语义并且在 第 9 章中的合适位置解释。一个例子是 `IS NULL` 子句。

我们已经在 第 4.1.2 节中讨论过常量。下面的小节会讨论剩下的选项。

列引用

一个列可以以下面的形式被引用：

```
correlation.columnname
```

correlation

是一个表（有可能以一个模式名限定）的名字，或者是在 `FROM` 子句中为一个表定义的别名。如果列名在当前索引所使用的表中都是唯一的，关联名称和分隔用的句点可以被忽略（另见 第 7 章）。

位置参数

一个位置参数引用被用来指示一个由SQL语句外部提供的值。参数被用于SQL函数定义和预备查询中。某些客户端库还支持独立于SQL命令字符串来指定数据值，在这种情况下参数被用来引用那些线外数据值。一个

参数引用的形式是：

```
$number
```

例如，考虑一个函数`dept`的定义：

```
CREATE FUNCTION dept(text) RETURNS dept
AS $$ SELECT * FROM dept WHERE name = $1 $$
LANGUAGE SQL;
```

这里 **\$1** 引用函数被调用时第一个函数参数的值。

下标

如果一个表达式得到了一个数组类型的值，那么可以抽取出该数组值的一个特定元素：

```
expression[subscript]
```

或者抽取出多个相邻元素（一个“数组切片”）：

```
expression[lower_subscript:upper_subscript]
```

（这里，方括号 `[ ]` 表示其字面意思）。每一个 **下标** 自身是一个表达式，它将四舍五入到最接近的整数值。

通常，数组 **表达式**

必须被加上括号，但是当要被加下标的表达式只是一个列引用或位置参数时，括号可以被忽略。还有，当原始数组是多维时，多个下标可以被连接起来。例如：

```
mytable.arraycolumn[4]
mytable.two_d_column[17][34]
$1[10:42]
(arrayfunction(a,b))[42]
```

最后一个例子中的圆括号是必需的。详见 [第 8.15 节](#)。

域选择

如果一个表达式得到一个组合类型（行类型）的值，那么可以抽取该行的指定域：

```
expression.fieldname
```

通常行 **表达式**

必须被加上括号，但是当该表达式是仅从一个表引用或位置参数选择时，圆括号可以被忽略。例如：

```
mytable.mycolumn
$1.somecolumn
```

```
(rowfunction(a,b)).col3
```

(因此，一个被限定的列引用实际上只是域选择语法的一种特例)。一种重要的特例是从一个组合类型的表中抽取一个域：

```
(compositecol).somefield  
(mytable.compositecol).somefield
```

这里需要圆括号来显示 **compositecol** 是一个列名而不是一个表名，在第二种情况中则是显示 **mytable** 是一个表名而不是一个模式名。

你可以通过书写 **.*** 来请求一个组合值的所有域：

```
(compositecol).*
```

这种记法的行为根据上下文会有不同，详见 [第 8.16.5 节](#)。

操作符调用

对于一次操作符调用，有两种可能的语法：

```
expression operator expression (二元中缀操作符)
```

```
operator expression (一元前缀操作符)
```

其中 **operator** 记号遵循 [第 4.1.3 节](#)

的语法规则，或者是关键词 ``AND``、``OR`` 和 ``NOT`` 之一，或者是一个如下形式的受限定操作符名：

```
OPERATOR(schema.operatorname)
```

哪个特定操作符存在以及它们是一元的还是二元的取决于由系统或用户定义的那些操作符。 [第 9 章](#) 描述了内建操作符。

函数调用

一个函数调用的语法是一个函数的名称（可能受限于一个模式名）后面跟上封闭于圆括号中的参数列表：

```
function_name ([expression [, expression ... ]])
```

例如，下面会计算 2 的平方根：

```
sqrt(2)
```

当在一个某些用户不信任其他用户的数据库中发出查询时，在编写函数调用时应遵守 [第 10.3 节](#) 中的安全防范措施。

内建函数的列表在 [第 9 章](#) 中。其他函数可以由用户增加。

参数可以有选择地被附加名称。详见 [第 4.3 节](#)。

注意

一个采用单一组合类型参数的函数可以被有选择地称为域选择语法，并且反过来域选择可以被写成函数的风格。也就是说，记号 ``col(table)`` 和 ``table.col`` 是可以互换的。这种行为是非 SQL 标准的但是在 IvorySQL 中被提供，因为它允许函数的使用来模拟“计算域”。

聚集表达式

一个聚集表达式

表示在由一个查询选择的行上应用一个聚集函数。一个聚集函数将多个输入减少到一个单一输出值，例如对输入的求和或平均。一个聚集表达式的语法是下列之一：

```
aggregate_name (expression [ , ... ] [ order_by_clause ] ) [ FILTER ( WHERE  
filter_clause ) ]  
aggregate_name (ALL expression [ , ... ] [ order_by_clause ] ) [ FILTER ( WHERE  
filter_clause ) ]  
aggregate_name (DISTINCT expression [ , ... ] [ order_by_clause ] ) [ FILTER ( WHERE  
filter_clause ) ]  
aggregate_name ( * ) [ FILTER ( WHERE filter_clause ) ]  
aggregate_name ( [ expression [ , ... ] ] ) WITHIN GROUP ( order_by_clause ) [ FILTER  
( WHERE filter_clause ) ]
```

这里 **aggregate_name** 是一个之前定义的聚集（可能带有一个模式名限定），并且 **expression** 是任意自身不包含聚集表达式的值表达式或一个窗口函数调用。可选的 **order_by_clause** 和 **filter_clause** 描述如下。

第一种形式的聚集表达式为每一个输入行调用一次聚集。第二种形式和第一种相同，因为 **ALL** 是默认选项。第三种形式为输入行中表达式的每一个可区分值（或者对于多个表达式是值的可区分集合）调用一次聚集。第四种形式为每一个输入行调用一次聚集，因为没有特定的输入值被指定，它通常只对于 **count()** 聚集函数有用。最后一种形式被用于 ***有序集** 聚集函数，其描述如下。

大部分聚集函数忽略空输入，这样其中一个或多个表达式得到空值的行将被丢弃。除非另有说明，对于所有内建聚集都是这样。

例如，**count(*)** 得到输入行的总数。**count(f1)** 得到输入行中 **f1** 为非空的数量，因为 **count** 忽略空值。而 **count(distinct f1)** 得到 **f1** 的非空可区分值的数量。

一般地，交给聚集函数的输入行是未排序的。在很多情况中这没有关系，例如不管接收到什么样的输入，**min** 总是产生相同的结果。但是，某些聚集函数（例如 **array_agg** 和 **string_agg**）依据输入行的排序产生结果。当使用这类聚集时，可选的 **order_by_clause** 可以被用来指定想要的顺序。**order_by_clause** 与查询级别的 **ORDER BY** 子句（如 [第 7.5 节](#) 所述）具有相同的语法，除非它的表达式总是仅有表达式并且不能是输出列名称或编号。例如：

```
SELECT array_agg(a ORDER BY b DESC) FROM table;
```

在处理多参数聚集函数时，注意 **ORDER BY** 出现在所有聚集参数之后。例如，要这样写：

```
SELECT string_agg(a, ',' ORDER BY a) FROM table;
```

而不能这样写：

```
SELECT string_agg(a ORDER BY a, ',') FROM table; -- 不正确
```

后者在语法上是合法的，但是它表示用两个`ORDER BY`键来调用一个单一参数聚集函数（第二个是无用的，因为它是一个常量）。

如果在 **order_by_clause** 之外指定了 **DISTINCT**，那么所有的 **ORDER BY** 表达式必须匹配聚集的常规参数。也就是说，你不能在 **DISTINCT** 列表没有包括的表达式上排序。

注意

在一个聚集函数中指定 **DISTINCT** 以及 **ORDER BY** 的能力是一种IvorySQL扩展。按照到目前为止的描述，如果一般目的和统计性聚集中排序是可选的，在要为其排序输入行时可以在该聚集的常规参数列表中放置 **ORDER BY**。有一个聚集函数的子集叫做有序集聚集，它要求一个 **order_by_clause**，通常是因为该聚集的计算只对其输入行的特定顺序有意义。有序集聚集的典型例子包括排名和百分位计算。按照上文的最后一种语法，对于一个有序集聚集，**order_by_clause** 被写在 **WITHIN GROUP (...)** 中。**order_by_clause** 中的表达式会像普通聚集参数一样对每一个输入行计算一次，按照每个 **order_by_clause** 的要求排序并且交给该聚集函数作为输入参数（这和非 **WITHIN GROUP order_by_clause** 的情况不同，在其中表达式的结果不会被作为聚集函数的参数）。如果有在 **WITHIN GROUP** 之前的参数表达式，会把它们称为直接参数以便与列在 **order_by_clause** 中的聚集参数相区分。与普通聚集参数不同，针对每次聚集调用只会计算一次直接参数，而不是为每一个输入行计算一次。这意味着只有那些变量被 **GROUP BY** 分组时，它们才能包含这些变量。这个限制同样适用于根本不在一个聚集表达式内部是直接参数。直接参数通常被用于百分数之类的东西，它们只有作为每次聚集计算用一次的单一值才有意义。直接参数列表可以为空，在这种情况下，写成 **()** 而不是 **(*)**（实际上IvorySQL接受两种拼写，但是只有第一种符合SQL标准）。

有序集聚集的调用例子：

```
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY income) FROM households;
percentile_cont
-----
50489
```

这会从表 **households** 的 **income** 列得到第 50 个百分位或者中位的值。这里`0.5`是一个直接参数，对于百分位部分是一个在不同行之间变化的值的情况它没有意义。

如果指定了 **FILTER**，那么只有对 **filter_clause** 计算为真的输入行会被交给该聚集函数，其他行会被丢弃。例如：

```
SELECT
    count(*) AS unfiltered,
    count(*) FILTER (WHERE i < 5) AS filtered
FROM generate_series(1,10) AS s(i);
unfiltered | filtered
-----+-----
10 | 4
(1 row)
```


预定义的聚集函数在 第 9.21 节 中描述。其他聚集函数可以由用户增加。

一个聚集表达式只能出现在 **SELECT** 命令的结果列表或是 **HAVING** 子句中。在其他子句（如 **WHERE**）中禁止使用它，因为那些子句的计算在逻辑上是在聚集的结果被形成之前。

当一个聚集表达式出现在一个子查询中（见 第 4.2.11 节 和 第 9.23 节），聚集通常在该子查询的行上被计算。但是如果该聚集的参数（以及 **filter_clause**，如果有）只包含外层变量则会产生一个异常：该聚集则属于最近的那个外层，并且会在那个查询的行上被计算。该聚集表达式从整体上则是对其所出现于的子查询的一种外层引用，并且在那个子查询的任意一次计算中都作为一个常量。只出现在结果列表或 **HAVING** 子句的限制适用于该聚集所属的查询层次。

窗口函数调用

一个*窗口函数调用*表示在一个查询选择的行的某个部分上应用一个聚集类的函数。和非窗口聚集函数调用不同，这不会被约束为将被选择的行分组为一个单一的输出行 — 在查询输出中每一个行仍保持独立。不过，窗口函数能够根据窗口函数调用的分组声明（**PARTITION BY** 列表）访问属于当前行所在分组中的所有行。一个窗口函数调用的语法是下列之一：

```
function_name ([expression [, expression ... ]]) [ FILTER ( WHERE filter_clause ) ]  
OVER window_name  
function_name ([expression [, expression ... ]]) [ FILTER ( WHERE filter_clause ) ]  
OVER ( window_definition )  
function_name ( * ) [ FILTER ( WHERE filter_clause ) ] OVER window_name  
function_name ( * ) [ FILTER ( WHERE filter_clause ) ] OVER ( window_definition )
```

其中 **window_definition** 的语法是

```
[ existing_window_name ]  
[ PARTITION BY expression [, ...] ]  
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS { FIRST | LAST } ] [, ...] ]  
[ frame_clause ]
```

可选的 **frame_clause** 是下列之一

```
{ RANGE | ROWS | GROUPS } frame_start [ frame_exclusion ]  
{ RANGE | ROWS | GROUPS } BETWEEN frame_start AND frame_end [ frame_exclusion ]
```

其中 **frame_start** 和 **frame_end** 可以是下面形式中的一种

```
UNBOUNDED PRECEDING  
offset PRECEDING  
CURRENT ROW  
offset FOLLOWING  
UNBOUNDED FOLLOWING
```

而 **frame_exclusion** 可以是下列之一

EXCLUDE CURRENT ROW
EXCLUDE GROUP
EXCLUDE TIES
EXCLUDE NO OTHERS

这里，**expression** 表示任何自身不含有窗口函数调用的值表达式。

window_name 是对定义在查询的 **WINDOW** 子句中的一个命名窗口声明的引用。还可以使用在 **WINDOW** 子句中定义命名窗口的相同语法在圆括号内给定一个完整的 **window_definition**，详见 [SELECT](#) 参考页。值得指出的是，**OVER wname** 并不严格地等价于 **OVER (wname ...)**，后者表示复制并修改窗口定义，并且在被引用窗口声明包括一个帧子句时会被拒绝。

PARTITION BY 选项将查询的行分组成为 分区，窗口函数会独立地处理它们。**PARTITION BY** 工作起来类似于一个查询级别的 **GROUP BY** 子句，不过它的表达式总是只是表达式并且不能是输出列的名称或编号。如果没有 **PARTITION BY**，该查询产生的所有行被当作一个单一分区来处理。**ORDER BY** 选项决定被窗口函数处理的一个分区中的行的顺序。它工作起来类似于一个查询级别的 **ORDER BY** 子句，但是同样不能使用输出列的名称或编号。如果没有 **ORDER BY**，行将被以未指定的顺序被处理。

frame_clause 指定构成 窗口帧 的行集合，它是当前分区的一个子集，窗口函数将作用在该帧而不是整个分区。帧中的行集合会随着哪一行是当前行而变化。在 **RANGE**、**ROWS** 或者 **GROUPS** 模式中指定帧，在每一种情况下，帧的范围都是从 **frame_start** 到 **frame_end**。如果 **frame_end** 被省略，则末尾默认为 **CURRENT ROW**。

UNBOUNDED PRECEDING 的一个 **frame_start** 表示该帧开始于分区的第一行，类似地 **UNBOUNDED FOLLOWING** 的一个 **frame_end** 表示该帧结束于分区的最后一行。

在 **RANGE** 或 **GROUPS** 模式中，**CURRENT ROW** 的一个 **frame_start** 表示帧开始于当前行的第一个 平级行（被窗口的 **ORDER BY** 子句排序为与当前行等效的行），而 **CURRENT ROW** 的一个 **frame_end** 表示帧结束于当前行的最后一个平级行。在 **ROWS** 模式中，**CURRENT ROW** 就表示当前行。

在 **offset PRECEDING** 以及 **offset FOLLOWING** 帧选项中，**offset** 必须是一个不包含任何变量、聚集函数或者窗口函数的表达式。**offset** 的含义取决于帧模式：

- 在 **ROWS** 模式中，**offset** 必须得到一个非空、非负的整数，并且该选项表示帧开始于当前行之前或者之后指定数量的行。
- 在 **GROUPS** 模式中，**offset** 也必须得到一个非空、非负的整数，并且该选项表示帧开始于当前行的平级组之前或者之后指定数量的*平级组*，这里平级组是在 **ORDER BY** 顺序中等效的行集合（要使用 **GROUPS** 模式，在窗口定义中就必须有一个 **ORDER BY** 子句）。
- 在 **RANGE** 模式中，这些选项要求 **ORDER BY** 子句正好指定一列。**offset** 指定当前行中那一列的值与它在该帧中前面或后面的行中的列值的最大差值。**offset** 表达式的数据类型会随着排序列的数据类型而变化。对于数字的排序列，它通常是与排序列相同的类型，但对于日期时间排序列它是一个 **interval**。例如，如果排序列是类型 **date** 或者 **timestamp**，我们可以写 **RANGE BETWEEN '1 day' PRECEDING AND '10 days' FOLLOWING**。**offset** 仍然要求是非空且非负，不过“非负”的含义取决于它的数据类型。

在任何一种情况下，到帧末尾的距离都受限于是到分区末尾的距离，因此对于离分区末尾比较近的行来说，帧可能会包含比较少的行。

注意在 **ROWS** 以及 **GROUPS** 模式中，**0 PRECEDING** 和 **0 FOLLOWING** 与 **CURRENT ROW** 等效。通常在 **RANGE** 模式中，这个结论也成立（只要有一种合适的、与数据类型相关的“零”的含义）。

frame_exclusion

选项允许当前行周围的行被排除在帧之外，即便根据帧的开始和结束选项应该把它们包括在帧中。**EXCLUDE CURRENT ROW** 会把当前行排除在帧之外。**EXCLUDE GROUP** 会把当前行以及它在顺序上的平级行都排除在帧之外。**EXCLUDE TIES** 把当前行的任何平级行都从帧中排除，但不排除当前行本身。**EXCLUDE NO OTHERS**

只是明确地指定不排除当前行或其平级行的这种默认行为。

默认的帧选项是 **RANGE UNBOUNDED PRECEDING**，它和 **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** 相同。如果使用 **ORDER BY**，这会把该帧设置为从分区开始一直到当前行的最后一个 **ORDER BY** 平级行的所有行。如果不使用 **ORDER BY**，就意味着分区中所有的行都被包括在窗口帧中，因为所有行都成为了当前行的平级行。

限制是 **frame_start** 不能是 **UNBOUNDED FOLLOWING**、**frame_end** 不能是 **UNBOUNDED PRECEDING**，并且在上述 **frame_start** 和 **frame_end** 选项的列表中 **frame_end** 选择不能早于 **frame_start** 选择出现 — 例如不允许 **RANGE BETWEEN CURRENT ROW AND offset PRECEDING**，但允许 **ROWS BETWEEN 7 PRECEDING AND 8 PRECEDING**，虽然它不会选择任何行。

如果指定了 **FILTER**，那么只有对 **filter_clause** 计算为真的输入行会被交给该窗口函数，其他行会被丢弃。只有是聚集的窗口函数才接受 **FILTER**。

内建的窗口函数在 [表 9.60](#)

中介绍。用户可以加入其他窗口函数。此外，任何内建的或者用户定义的通用聚集或者统计性聚集都可以被用作窗口函数（有序集和假想集聚集当前不能被用作窗口函数）。

使用 ***** 的语法被用来把参数较少的聚集函数当作窗口函数调用，例如 **count(*) OVER (PARTITION BY x ORDER BY y)**。星号 (*****) 通常不被用于窗口相关的函数。窗口相关的函数不允许在函数参数列表中使用 **DISTINCT** 或 **ORDER BY**。

只有在 **SELECT** 列表和查询的 **ORDER BY** 子句中才允许窗口函数调用。

更多关于窗口函数的信息可以在 [第 3.5 节](#)、[第 9.22 节](#) 以及 [第 7.2.5 节](#) 中找到。

类型转换

一个类型造型指定从一种数据类型到另一种数据类型的转换。IvorySQL 接受两种等价的类型造型语法：

```
CAST ( expression AS type )
expression::type
```

CAST 语法遵从 SQL，而用 **::** 的语法是 IvorySQL 的历史用法。

当一个造型被应用到一种未知类型的值表达式上时，它表示一种运行时类型转换。只有已经定义了一种合适的类型转换操作时，该造型才会成功。注意这和常量的造型（如 [第 4.1.2.7 节](#) 中所示）使用不同。应用于一个未修饰串文字的造型表示一种类型到一个文字常量值的初始赋值，并且因此它将对任意类型都成功（如果该串文字的内容对于该数据类型的输入语法是可接受的）。

如果一个值表达式必须产生的类型没有歧义（例如当它被指派给一个表列），通常可以省略显式类型造型，在这种情况下系统会自动应用一个类型造型。但是，只有对在系统目录中被标记为 “OK to apply implicitly” 的造型才会执行自动造型。其他造型必须使用显式造型语法调用。这种限制是为了防止出人意料的转换被无声无息地应用。

还可以用像函数的语法来指定一次类型造型：

```
typename ( expression )
```

不过，这只对那些名字也作为函数名可用的类型有效。例如，**double precision** 不能以这种方式使用，但是等效的 **float8** 可以。还有，如果名称 **interval**、**time** 和 **timestamp** 被用双引号引用，那么由于语法冲突的原因，它们只能以这种风格使用。因此，函数风格的造型语法的使用会导致不一致性并且应该尽可能被避免。

注意

函数风格的语法事实上只是一次函数调用。当两种标准造型语法之一被用来做一次运行时转换时，它将在内部调用一个已注册的函数来执行该转换。简而言之，这些转换函数具有和它们的输出类型相同的名字，并且因此“函数风格的语法”无非是对底层转换函数的一次直接调用。显然，一个可移植的应用不应当依赖于它。详见 [CREATE CAST](#)。

排序规则表达式

COLLATE 子句会重载一个表达式的排序规则。它被追加到它适用的表达式：

```
expr COLLATE collation
```

这里 **collation** 可能是一个受模式限定的标识符。**COLLATE** 子句比操作符绑得更紧，必要时可以使用圆括号。

如果没有显式指定排序规则，数据库系统会从表达式所涉及的列中得到一个排序规则，如果该表达式没有涉及列，则会默认采用数据库的默认排序规则。

COLLATE 子句的两种常见使用是重载 **ORDER BY** 子句中的排序顺序，例如：

```
SELECT a, b, c FROM tbl WHERE ... ORDER BY a COLLATE "C";
```

以及重载具有区域敏感结果的函数或操作符调用的排序规则，例如：

```
SELECT * FROM tbl WHERE a > 'foo' COLLATE "C";
```

注意在后一种情况中，**COLLATE** 子句被附加到我们希望影响的操作符的一个输入参数上。**COLLATE** 子句被附加到该操作符或函数调用的哪个参数上无关紧要，因为被操作符或函数应用的排序规则是考虑所有参数得来的，并且一个显式的 **COLLATE** 子句将重载所有其他参数的排序规则（不过，附加非匹配 **COLLATE** 子句到多于一个参数是一种错误。详见 [第 24.2 节](#)）。因此，这会给出和前一个例子相同的结果：

```
SELECT * FROM tbl WHERE a COLLATE "C" > 'foo';
```

但是这是一个错误：

```
SELECT * FROM tbl WHERE (a > 'foo') COLLATE "C";
```

因为它尝试把一个排序规则应用到 **>** 操作符的结果，而它的数据类型是非可排序数据类型 **boolean**。

标量子查询

一个标量子查询是一种圆括号内的普通 **SELECT** 查询，它刚好返回一行一列（关于书写查询可见 [第 7 章](#)）。`SELECT` 查询被执行并且该单一返回值被使用在周围的值表达式中。将一个返回超过一行或一列的查询作为一个标量子查询使用是一种错误（但是如果是一次特定执行期间该子查询没有返回行则不是错误，该标量结果被当做空）。该子查询可以从周围的查询中引用变量，这些变量在该子查询的任何一次计算中都将作为常量。对于其他涉及子查询的表达式还可见 [第 9.23 节](#)。

例如，下列语句会寻找每个州中最大的城市人口：

```
SELECT name, (SELECT max(pop) FROM cities WHERE cities.state = states.name)
FROM states;
```

数组构造器

一个数组构造器是一个能构建一个数组值并且将值用于它的成员元素的表达式。一个简单的数组构造器由关键词 **ARRAY**、一个左方括号 `[`、一个用于数组元素值的表达式列表（用逗号分隔）以及最后的一个右方括号 `]` 组成。例如：

```
SELECT ARRAY[1,2,3+4];
      array
-----
{1,2,7}
(1 row)
```

默认情况下，数组元素类型是成员表达式的公共类型，使用和 **UNION** 或 **CASE** 结构（见 [第 10.5 节](#)）相同的规则决定。你可以通过显式将数组构造器造型为想要的类型来重载，例如：

```
SELECT ARRAY[1,2,22.7]::integer[];
      array
-----
{1,2,23}
(1 row)
```

这和把每一个表达式单独地造型为数组元素类型的效果相同。关于造型的更多信息请见 [第 4.2.9 节](#)。

多维数组值可以通过嵌套数组构造器来构建。在内层的构造器中，关键词 **ARRAY** 可以被忽略。例如，这些语句产生相同的结果：

```
SELECT ARRAY[ARRAY[1,2], ARRAY[3,4]];
      array
-----
{{1,2},{3,4}}
(1 row)

SELECT ARRAY[[1,2],[3,4]];
      array
-----
{{1,2},{3,4}}
(1 row)
```

因为多维数组必须是矩形的，处于同一层次的内层构造器必须产生相同维度的子数组。任何被应用于外层 **ARRAY** 构造器的造型会自动传播到所有的内层构造器。

多维数组构造器元素可以是任何得到一个正确种类数组的任何东西，而不仅仅是一个子-**ARRAY** 结构。例如：

```
CREATE TABLE arr(f1 int[], f2 int[]);

INSERT INTO arr VALUES (ARRAY[[1,2],[3,4]], ARRAY[[5,6],[7,8]]);

SELECT ARRAY[f1, f2, '{{9,10},{11,12}}'::int[]] FROM arr;
          array
-----
{{1,2},{3,4}},{5,6},{7,8}},{9,10},{11,12}}
(1 row)
```

你可以构造一个空数组，但是因为无法得到一个无类型的数组，你必须显式地把你的空数组造型成想要的类型。例如：

```
SELECT ARRAY[]::integer[];
          array
-----
{}
(1 row)
```

也可以从一个子查询的结果构建一个数组。在这种形式中，数组构造器被写为关键词 **ARRAY** 后跟着一个加了圆括号（不是方括号）的子查询。例如：

```
SELECT ARRAY(SELECT oid FROM pg_proc WHERE proname LIKE 'bytea%');
          array
-----
{2011,1954,1948,1952,1951,1244,1950,2005,1949,1953,2006,31,2412}
(1 row)

SELECT ARRAY(SELECT ARRAY[i, i*2] FROM generate_series(1,5) AS a(i));
          array
-----
{{1,2},{2,4},{3,6},{4,8},{5,10}}
(1 row)
```

子查询必须返回一个单一列。如果子查询的输出列是非数组类型，结果的一维数组将为该子查询结果中的每一行有一个元素，并且有一个与子查询的输出列匹配的元素类型。如果子查询的输出列是一种数组类型，结果将是同类型的一个数组，但是要高一个维度。在这种情况下，该子查询的所有行必须产生同样维度的数组，否则结果就不会是矩形形式。

用 **ARRAY** 构建的一个数组值的下标总是从一开始。更多关于数组的信息，请见 [第 8.15 节](#)。

行构造器

一个行构造器是能够构建一个行值（也称作一个组合类型）并用值作为其成员域的表达式。一个行构造器由关键词 **ROW**、一个左圆括号、用于行的域值的零个或多个表达式（用逗号分隔）以及最后的一个右圆括号组成。例如

:

```
SELECT ROW(1,2.5,'this is a test');
```

当在列表中有超过一个表达式时，关键词 **ROW** 是可选的。

一个行构造器可以包括语法 **rowvalue .***，它将被扩展为该行值的元素的一个列表，就像在一个顶层 **SELECT** 列表（见 第 8.16.5 节）中使用 **.*** 时发生的事情一样。例如，如果表 **t** 有列 **f1** 和 **f2**，那么这些是相同的：

```
SELECT ROW(t.*, 42) FROM t;  
SELECT ROW(t.f1, t.f2, 42) FROM t;
```

注意

在PostgreSQL 8.2 以前，**.*** 语法不会在行构造器中被扩展，这样写 **ROW(t.*, 42)** 会创建一个有两个域的行，其第一个域是另一个行值。新的行为通常更有用。如果你需要嵌套行值的旧行为，写内层行值时不要用 **.***，例如 **ROW(t, 42)**。

默认情况下，由一个 **ROW** 表达式创建的值是一种匿名记录类型。如果必要，它可以被造型为一种命名的组合类型 — 或者是一个表的行类型，或者是一种用 **CREATE TYPE AS** 创建的组合类型。为了避免歧义，可能需要一个显式造型。例如：

```
CREATE TABLE mytable(f1 int, f2 float, f3 text);  
  
CREATE FUNCTION getf1(mytable) RETURNS int AS 'SELECT $1.f1' LANGUAGE SQL;  
  
-- 不需要造型因为只有一个 getf1() 存在  
SELECT getf1(ROW(1,2.5,'this is a test'));  
getf1  
-----  
1  
(1 row)  
  
CREATE TYPE myrowtype AS (f1 int, f2 text, f3 numeric);  
  
CREATE FUNCTION getf1(myrowtype) RETURNS int AS 'SELECT $1.f1' LANGUAGE SQL;  
  
-- 现在我们需要一个造型来指示要调用哪个函数：  
SELECT getf1(ROW(1,2.5,'this is a test'));  
ERROR: function getf1(record) is not unique  
  
SELECT getf1(ROW(1,2.5,'this is a test')::mytable);  
getf1  
-----
```

```

      1
(1 row)

SELECT getf1(CAST(ROW(11,'this is a test',2.5) AS myrowtype));
getf1
-----
      11
(1 row)

```

行构造器可以被用来构建存储在一个组合类型表列中的组合值，或者被传递给一个接受组合参数的函数。还有，可以比较两个行值，或者用 **IS NULL** 或 **IS NOT NULL** 测试一个行，例如：

```

SELECT ROW(1,2.5,'this is a test') = ROW(1, 3, 'not the same');

SELECT ROW(table.*) IS NULL FROM table; -- detect all-null rows

```

详见 第 9.24 节。如 第 9.23 节 中所讨论的，行构造器也可以被用来与子查询相连接。

表达式计算规则

子表达式的计算顺序没有被定义。特别地，一个操作符或函数的输入不必按照从左至右或其他任何固定顺序进行计算。

此外，如果一个表达式的结果可以通过只计算其一部分来决定，那么其他子表达式可能完全不需要被计算。例如，如果我们写：

```
SELECT true OR somefunc();
```

那么 **somefunc()** 将（可能）完全不被调用。如果我们写成下面这样也是一样：

```
SELECT somefunc() OR true;
```

注意这和一些编程语言中布尔操作符从左至右的“短路”不同。

因此，在复杂表达式中使用带有副作用的函数是不明智的。在 **WHERE** 和 **HAVING** 子句中依赖副作用或计算顺序尤其危险，因为在建立一个执行计划时这些子句会被广泛地重新处理。这些子句中布尔表达式（**AND** / **OR** / **NOT** 的组合）可能会以布尔代数定律所允许的任何方式被重组。

当有必要强制计算顺序时，可以使用一个 **CASE** 结构（见 第 9.18 节）。例如，在一个 **WHERE** 子句中使用下面的方法尝试避免除零是不可靠的：

```
SELECT ... WHERE x > 0 AND y/x > 1.5;
```

但是这是安全的：

```
SELECT ... WHERE CASE WHEN x > 0 THEN y/x > 1.5 ELSE false END;
```

一个以这种风格使用的 **CASE**

结构将使得优化尝试失败，因此只有必要时才这样做（在这个特别的例子中，最好通过写 `y > 1.5*x` 来回避这个问题）。

不过，**CASE** 不是这类问题的万灵药。上述技术的一个限制是，它无法阻止常量表达式的提早计算。如 [第 38.7 节](#) 中所述，当查询被规划而不是被执行时，被标记成 **IMMUTABLE** 的函数和操作符可以被计算。因此

```
SELECT CASE WHEN x > 0 THEN x ELSE 1/0 END FROM tab;
```

很可能会导致一次除零失败，因为规划器尝试简化常量表达式。即便是表中的每一行都有 `x > 0`（这样运行时永远不会进入到 **ELSE** 分支）也是这样。

虽然这个特别的例子可能看起来愚蠢，没有明显涉及常量的情况可能会发生在函数内执行的查询中，因为函数参数的值和本地变量可以作为常量被插入到查询中用于规划目的。例如，在 PL/pgSQL 函数中，使用一个 **IF-THEN-ELSE** 语句来保护一种有风险的计算比把它嵌在一个 **CASE** 表达式中要安全得多。

另一个同类型的限制是，一个 **CASE** 无法阻止其所包含的聚集表达式的计算，因为在考虑 **SELECT** 列表或 **HAVING** 子句中的其他表达式之前，会先计算聚集表达式。例如，下面的查询会导致一个除零错误，虽然看起来好像已经这种情况加以了保护：

```
SELECT CASE WHEN min(employees) > 0
            THEN avg(expenses / employees)
            END
FROM departments;
```

min() 和 **avg()** 聚集会在所有输入行上并行地计算，因此如果任何行有 **employees** 等于零，在有机会测试 **min()** 的结果之前，就会发生除零错误。取而代之的是，可以使用一个 **WHERE** 或 **FILTER** 子句来首先阻止有问题的输入行到达一个聚集函数。

调用函数

IvorySQL 允许带有命名参数的函数被使用 位置 或 命名 记号法调用。命名记号法对于有大量参数的函数特别有用，因为它让参数和实际参数之间的关联更明显和可靠。在位置记号法中，书写一个函数调用时，其参数值要按照它们在函数声明中被定义的顺序书写。在命名记号法中，参数根据名称匹配函数参数，并且可以以任何顺序书写。对于每一种记法，还要考虑函数参数类型的效果，这些在 [第 10.3 节](#) 有介绍。

在任意一种记号法中，在函数声明中给出了默认值的参数根本不需要在调用中写出。但是这在命名记号法中特别有用，因为任何参数的组合都可以被忽略。而在位置记号法中参数只能从右往左忽略。

IvorySQL 也支持 混合 记号法，它组合了位置和命名记号法。在这种情况下，位置参数被首先写出并且命名参数出现在其后。

下列例子将展示所有三种记号法的用法：

```
CREATE FUNCTION concat_lower_or_upper(a text, b text, uppercase boolean DEFAULT false)
RETURNS text
AS
$$
SELECT CASE
    WHEN $3 THEN UPPER($1 || ' ' || $2)
    ELSE LOWER($1 || ' ' || $2)
END;
```

```
$$  
LANGUAGE SQL IMMUTABLE STRICT;
```

函数 `concat_lower_or_upper` 有两个强制参数，`a` 和 `b`。此外，有一个可选的参数 `uppercase`，其默认值为 `false`。`a` 和 `b` 输入将被串接，并且根据 `uppercase` 参数被强制为大写或小写形式。这个函数的剩余细节对这里并不重要（详见 第 38 章）。

使用位置记号

在 IvorySQL 中，位置记号法是给函数传递参数的传统机制。一个例子：

```
SELECT concat_lower_or_upper('Hello', 'World', true);  
concat_lower_or_upper  
-----  
HELLO WORLD  
(1 row)
```

所有参数被按照顺序指定。结果是大写形式，因为 `uppercase` 被指定为 `true`。另一个例子：

```
SELECT concat_lower_or_upper('Hello', 'World');  
concat_lower_or_upper  
-----  
hello world  
(1 row)
```

这里，`uppercase` 参数被忽略，因此它接收它的默认值 `false`，并导致小写形式的输出。在位置记号法中，参数可以按照从右往左被忽略并且因此而得到默认值。

使用命名记号

在命名记号法中，每一个参数名都用 `=>` 指定来把它与参数表达式分隔开。例如：

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World');  
concat_lower_or_upper  
-----  
hello world  
(1 row)
```

再次，参数 `uppercase` 被忽略，因此它被隐式地设置为 `false`。使用命名记号法的一个优点是参数可以用任何顺序指定，例如：

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World', uppercase => true);  
concat_lower_or_upper  
-----  
HELLO WORLD  
(1 row)
```

```
SELECT concat_lower_or_upper(a => 'Hello', uppercase => true, b => 'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

为了向后兼容性，基于 "!=" 的旧语法仍被支持：

```
SELECT concat_lower_or_upper(a := 'Hello', uppercase := true, b := 'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

使用混合记号

混合记号法组合了位置和命名记号法。不过，正如已经提到过的，命名参数不能超越位置参数。例如：

```
SELECT concat_lower_or_upper('Hello', 'World', uppercase => true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

在上述查询中，参数 **a** 和 **b** 被以位置指定，而 **uppercase** 通过名字指定。在这个例子中，这只增加了一点文档。在一个具有大量带默认值参数的复杂函数中，命名的或混合的记号法可以节省大量的书写并且减少出错的机会。

注意

命名的和混合的调用记号法当前不能在调用聚集函数时使用（但是当聚集函数被用作窗口函数时它们可以被使用）。

Oracle兼容功能

配置参数

参数设置采用与原生 PostgreSQL 相同的方法。所有参数名称都不区分大小写。每个参数都采用以下五种类型之一的值：布尔值、字符串、整数、浮点数或枚举 (enum)。

ivorysql.compatible_mode (enum)

此参数控制数据库服务器的行为。如果它设置为 `pg`，表示它是原生安装，服务器将作为原生 PG 安装。如果它设置为 `oracle`，那么查询的输出和系统行为整体会发生变化，因为它会更像 Oracle。

当设置为 **oracle** 时，此参数会隐式地将同样名字的 Schema 添加到 **search_path**。以便可以定位 Oracle 兼容对象。

该参数可以通过 `postgresql.conf` 配置文件设置，对整个数据库生效。或者可以通过客户端使用 `set` 命令在会话上进行设置。

`nls_length_semantics` (enum)

此参数控制如何根据长度语义创建 `CHAR` 和 `VARCHAR2` 列。默认设置为 `NONE`，这不会产生任何影响。`BYTE` 可以设置为以字节长度语义存储数据。`CHAR` 使用字符长度语义。

标识：

- [Compatibility Parameters](#)

更改表

语法

```
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
action;

action:
    ADD ( add_coldef [, ... ] )
    | MODIFY ( modify_coldef [, ... ] )
    | DROP [ COLUMN ] ( column_name [, ... ] )

add_coldef:
    column_name data_type

modify_coldef:
    column_name data_type alter_using

alter_using:
    USING expression
```

参数

`name` 表名.

`column_name` 列名.

`data_type` 列类型.

`expression` 值表达式.

ADD keyword 增加表的列, 可以增加一个列或多个列.

MODIFY keyword 修改表的列, 可以修改一个列或多个列.

DROP keyword 删除表的列, 可以删除一个列或多个列.

USING keyword 修改列的值.

ADD:

```
create table tb_test1(id int, flg char(10));
```

```
alter table tb_test1 add (name varchar);
```

```
alter table tb_test1 add (adress varchar, num int, flg1 char);
```

```
\d tb_test1
```

Table "public.tb_test1"				
Column	Type	Collation	Nullable	Default
id	integer			
flg	character(10)			
name	character varying			
adress	character varying			
num	integer			
flg1	character(1)			

MODIFY:

```
create table tb_test2(id int, flg char(10), num varchar);
```

```
insert into tb_test2 values('1', 2, '3');
```

```
alter table tb_test2 modify(id char);
```

```
\d tb_test2
```

Table "public.tb_test2"				
Column	Type	Collation	Nullable	Default
id	character(1)			
flg	character(10)			
num	character varying			

DROP:

```
create table tb_test3(id int, flg1 char(10), flg2 char(11), flg3 char(12), flg4  
char(13),
```

```
flg5 char(14), flg6 char(15));
```

```
alter table tb_test3 drop column(id);
```

```
\d tb_test3
```

Table "public.tb_test3"

Column	Type	Collation	Nullable	Default
flg1	character(10)			
flg2	character(11)			
flg3	character(12)			
flg4	character(13)			
flg5	character(14)			
flg6	character(15)			

删除表

语法

```
[ WITH [ RECURSIVE ] with_query [, ...] ]
DELETE [ FROM ] [ ONLY ] table_name [ * ] [ [ AS ] alias ]
    [ USING using_list ]
    [ WHERE condition | WHERE CURRENT OF cursor_name ]
    [ RETURNING * | output_expression [ [ AS ] output_name ] [, ...] ]
```

参数

table_name 表名.

alias 表别名.

using_list 一个表表达式的列表，它允许在WHERE条件中出现 来自其他表的列.

condition 一个返回boolean类型值的表达式.

cursor_name 要在WHERE CURRENT OF情况中使用的游标的名称.

output_expression 在每一行被删除后，会被DELETE计算并且返回的表达式.

output_name 被返回列的名称.

使用

```
create table tb_test4(id int, flg char(10));

insert into tb_test4 values(1, '2'), (3, '4'), (5, '6');

delete from tb_test4 where id = 1;

delete tb_test4 where id = 3;

table tb_test4;
 id |   flg
```

-----+-----

5 | 6
(1 row)

更新表

语法

```
[ WITH [ RECURSIVE ] with_query [, ...] ]  
UPDATE [ ONLY ] table_name [ * ] [ [ AS ] alias ]  
    SET { [ table_name | alias ] column_name = { expression | DEFAULT }  
    | ( [ table_name | alias ] column_name [, ...] ) = [ ROW ] ( { expression | DEFAULT  
} [, ...] )  
    | ( [ table_name | alias ] column_name [, ...] ) = ( sub-SELECT )  
        } [, ...]  
    [ FROM from_list ]  
    [ WHERE condition | WHERE CURRENT OF cursor_name ]  
    [ RETURNING * | output_expression [ [ AS ] output_name ] [, ...] ]
```

参数

table_name 表名.

alias 表别名.

column_name 列名.

expression 值表达式.

sub-SELECT select子句.

from_list 表表达式.

condition 一个返回boolean类型值的表达式.

cursor_name 要在WHERE CURRENT OF情况中使用的游标 的名称.

output_expression 在每一行被删除后, 会被DELETE计算并且返回的表达式.

output_name 被返回列的名称.

示例

```
create table tb_test5(id int, flg char(10));  
  
insert into tb_test5 values(1, '2'), (3, '4'), (5, '6');  
  
update tb_test5 a set a.id = 33 where a.id = 3;  
  
table tb_test5;
```

Id	flg
1	2
5	6
33	4

(3 rows)

GROUP BY

示例

```
set compatible_mode to oracle;

create table students(student_id varchar(20) primary key ,
student_name varchar(40),
student_pid int);

select student_id,student_name from students group by student_id;
ERROR: column "students.student_name" must appear in the GROUP BY clause or be used
in an aggregate function
```

UNION

示例

```
select null union select null union select 1.2;
?column?
-----
1.2

(2 rows)
```

Minus Operator

语法

```
select_statement MINUS [ ALL | DISTINCT ] select_statement;
```

参数

select_statement 任何没有ORDER BY、LIMIT、FOR NO KEY UPDATE、FOR UPDATE、FOR SHARE和FOR KEY SHARE子句的 SELECT语句.

ALL keyword 包含重复行结果.

DISTINCT keyword 显示的消除重复行.

示例

```
select * from generate_series(1, 3) g(i) MINUS select * from generate_series(1, 3)
g(i) where i = 1;
 i
---
 2
 3
(2 rows)
```

转义字符

概述

使用 q\ 转义特殊字符。q\ 转义字符通常在! [] {} () \<> 和其他转义字符之后使用, 您也可以使用 \, 也可以使用字母, 数字, \=, +, -, *, \&, \\$, \% , #, 等, 不允许使用空格。

示例

```
select q''' is goog ''';
      ?column?
-----
' is goog
(1 row)
```

序列

语法

```
SELECT [ database {schema} | schema ] sequence {nextval | currval};
```

参数

sequence 序列名.

示例

```
create sequence sq;

select sq.nextval;
      nextval
-----
         1
(1 row)

select sq.currval;
      nextval
```

```
-----  
1  
(1 row)
```

兼容时间和日期函数

from_tz

目的

将给定的不带时区的时间戳转换为指定的带时区的时间戳，如果指定时区或者时间戳为NULL，则返回NULL。

参数描述

参数	描述
day	不带时区的时间戳
tz	指定的时区

例子

```
select from_tz('2021-11-08 09:12:39','Asia/Shanghai') from dual;  
from_tz
```

```
-----  
2021-11-08 09:12:39 Asia/Shanghai  
(1 row)
```

```
select from_tz('2021-11-08 09:12:39','SAST') from dual;  
from_tz
```

```
-----  
2021-11-08 09:12:39 SAST
```

```
select from_tz(NULL,'SAST') from dual;  
from_tz
```

```
-----  
  
(1 row)
```

```
select from_tz('2021-11-08 09:12:31',NULL) from dual;  
from_tz
```

```
-----  
  
(1 row)
```

systimestamp

目的

获取当前数据库系统的时间戳。

例子

```
select systimestamp();
       systimestamp
-----
2021-12-02 14:38:59.879642+08
(1 row)

select systimestamp;
       statement_timestamp
-----
2021-12-02 14:39:33.262828+08
```

sys_extract_utc

目的

将给定的带时区的时间戳转换为不带时区的UTC时间。

参数描述

参数	描述
day	需要转化带时区的时间戳

例子

```
select sys_extract_utc('2018-03-28 11:30:00.00 +09:00'::timestampz) from dual;
       sys_extract_utc
-----
2018-03-28 02:30:00
(1 row)

select sys_extract_utc(NULL) from dual;
       sys_extract_utc
-----
(1 row)
```

目的

获取当前会话的时区。

例子

```
select sessiontimezone() from dual;
sessiontimezone
```

```
-----
```

```
PRC
(1 row)
```

```
set timezone to UTC;
```

```
select sessiontimezone();
sessiontimezone
```

```
-----
```

```
UTC
(1 row)
```

next_day

目的

next_day 返回由格式名相同的第一个工作日的日期，该日期晚于当前日期。
无论日期的数据类型如何，返回类型始终为 DATE。
返回值具有与参数日期相同的小时、分钟和秒部分。

参数描述

参数	描述
value	开始时间戳
weekday	星期几，可以是 "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday" 或者 0,1,2,3,4,5,6,0代表星期天

例子

```
select next_day(to_timestamp('2020-02-29 14:40:50', 'YYYY-MM-DD HH24:MI:SS'),
'Tuesday') from dual;
```

```
next_day
```

```
-----
```

```
2020-03-03 14:40:50
(1 row)
```

```
select next_day('2020-07-01 19:43:51 +8'::timestamp, 1) from dual;
      next_day
-----
2020-07-05 19:43:51
(1 row)
```

```
select next_day(oracle.date '2020-09-15 12:13:29', 6) from dual;
      next_day
-----
2020-09-18 12:13:29
(1 row)
```

last_day

目的

last_day返回档期日期所在月份的最后一天。

参数描述

参数	描述
value	指定的时间戳

例子

```
select last_day(timestamp '2020-05-17 13:27:19') from dual;
      last_day
-----
2020-05-31 13:27:19
(1 row)

select last_day('2020-11-29 19:20:40 +08'::timestamp) from dual;
      last_day
-----
2020-11-30 19:20:40
(1 row)

select last_day('-0004-2-1 13:27:19'::oracle.date) from dual;
      last_day
-----
-0004-02-29 13:27:19
(1 row)
```

add_months

目的

add_months 返回日期加上整数月份。 **date** 参数可以是日期时间值或任何可以隐式转换为 **DATE** 的值。 **整数**参数可以是整数或任何可以隐式转换为整数的值。

参数描述

参数	描述
day	oracle.date类型，需要被改变的时间戳
value	一个整形数据，需要增加的月数

例子

```
select add_months(date '2020-02-15',7) from dual;
      add_months
-----
2020-09-15 00:00:00
(1 row)

select add_months(timestamp '2018-12-15 19:12:09',12) from dual;
      add_months
-----
2019-12-15 19:12:09
(1 row)
```

sysdate

目的

sysdate 返回数据库服务器的操作系统时间。

例子

```
select sysdate;
      statement_sysdate
-----
2021-12-09 16:20:34
(1 row)

select sysdate();
      sysdate
-----
2021-12-09 16:21:39
```

(1 row)

new_time

目的

转换第一个时区的时间到第二个时区的时间。时区包括了 "ast", "adt", "bst", "bdt", "cst", "cdt", "est", "edt", "gmt", "hst", "hdt", "mst", "mdt", "nst", "pst", "pdt", "yst", "ydt".

参数描述

参数	描述
day	需要被转换的时间戳
tz1	时间戳的时区
tz2	目标时区

例子

```
select new_time(timestamp '2020-12-12 17:45:18', 'AST', 'ADT') from dual;  
      new_time
```

```
-----  
2020-12-12 18:45:18  
(1 row)
```

```
select new_time(timestamp '2020-12-12 17:45:18', 'BST', 'BDT') from dual;  
      new_time
```

```
-----  
2020-12-12 18:45:18  
(1 row)
```

```
select new_time(timestamp '2020-12-12 17:45:18', 'CST', 'CDT') from dual;  
      new_time
```

```
-----  
2020-12-12 18:45:18  
(1 row)
```

trunc

目的

trunc函数返回一个日期，按照指定格式被截断，fmt包括 "Y", "YY", "YYY", "YYYY", "YEAR", "SYYYY", "SYEAR", "I", "IY", "IYY", "IYYY", "Q", "WW", "Iw", "W", "DAY", "DY", "D", "MONTH", "MONn", "MM", "RM", "CC", "SCC", "DDD", "DD", "J", "HH", "HH12", "HH24", "MI".

参数描述

参数	描述
value	指定的日期 (oracle.date, timestamp, timestamptz)
fmt	指定的格式, 如果被省略, 默认为 "DDD"

例子

```
select trunc(timestamp '2020-07-28 19:16:12', 'Q');
      trunc
-----
2020-07-01 00:00:00
(1 row)

select trunc(timestamptz '2020-09-27 18:30:21 + 08', 'MONTH');
      trunc
-----
2020-09-01 00:00:00+08
(1 row)
```

round

目的

round函数返回一个日期,按照指定的格式四舍五入, fmt 包括了 "Y", "YY", "YYY", "YYYY", "YEAR", "SYYYY", "SYEAR", "I", "IY", "IYY", "IYYY", "Q", "WW", "Iw", "W", "DAY", "DY", "D", "MONTH", "MONn", "MM", "RM", "CC", "SCC", "DDD", "DD", "J", "HH", "HH12", "HH24", "MI".

参数描述

参数	描述
value	被转换的日期 (oracle.date, timestamp, timestamptz)
fmt	指定的格式, 如果被省略, 默认为 "DDD"

例子

```
select round(timestamp '2050-06-12 16:40:55', 'IYYY');
      round
-----
2050-01-03 00:00:00
(1 row)
```


兼容转换和比较以及与NULL相关的函数

TO_CHAR

目的

TO_CHAR(str,[fmt]) 根据给定的格式将输入参数转换为 TEXT 数据类型的值。如果省略 fmt，则数据将转换为系统默认格式的 TEXT 值。如果 str 为 null，则该函数返回 null。

参数

str 输入参数（任意类型）。

fmt 输入格式参数,详见格式fmt。

示例

```
select to_char('3 2:20:05');
      to_char
-----
3 days 02:20:05
(1 row)

select to_char('4.00'::numeric);
      to_char
-----
4
(1 row)

select to_char(NULL);
      to_char
-----

(1 row)

select to_char(123,'xx');
      to_char
-----
7b
(1 row)
```

TO_NUMBER

目的

TO_NUMBER(str,[fmt1]) 根据给定的格式将输入参数 str 转换为 NUMERIC 数据类型的值。如果省略 fmt1，则数据将转换为系统默认格式的 NUMERIC 值。如果 str 是 NUMERIC，则该函数返回 str。如果 str 计算结果为 null，则该函数返回 null。如果它不能转换为 NUMERIC 数据类型，则该函数返回错误。

参数

str 输入参数包括以下数据类型（double precision, numeric, text, integer等，但必须隐式转换为numeric）。

fmt1 输入格式参数，详见格式fmt1。

示例

```
select to_number(1210.73::numeric, 9999.99::numeric);
 to_number
-----
    1210.73
(1 row)

select to_number(NULL);
 to_number
-----

(1 row)

select to_number('123'::text);
 to_number
-----
        123
(1 row)
```

TO_DATE

目的

TO_DATE(str,[fmt]) 根据给定的格式将输入参数 str 转换为日期数据类型的值。如果省略 fmt，则数据将转换为系统默认格式的日期值。如果 str 为 null，则该函数返回 null。如果 fmt 是 J，对于 Julian，则 char 必须是整数。如果无法转换为 DATE，则该函数返回错误。

参数

str 输入参数（integer, text, 可以隐式转换为上述类型，符合日期格式的字符串）。

fmt 输入格式参数，详见格式fmt。

示例

```
select to_date('50-11-28 ', 'RR-MM-dd ');
 to_date
-----
1950-11-28 00:00:00
(1 row)
```

```

select to_date(2454336, 'J');
       to_date
-----
2007-08-23 00:00:00
(1 row)

select to_date('2019/11/22', 'yyyy-mm-dd');
       to_date
-----
2019-11-22 00:00:00
(1 row)

select to_date('20-11-28 10:14:22', 'YY-MM-dd hh24:mi:ss');
       to_date
-----
2020-11-28 10:14:22
(1 row)

select to_date('2019/11/22');
       to_date
-----
2019-11-22 00:00:00
(1 row)

select to_date('2019/11/27 10:14:22');
       to_date
-----
2019-11-27 10:14:22
(1 row)

select to_date('2020', 'RR');
       to_date
-----
2020-01-01 00:00:00
(1 row)

select to_date(NULL);
       to_date
-----

(1 row)

```

```
select to_date('-4712-07-23 14:31:23', 'syyyy-mm-dd hh24:mi:ss');
       to_date
-----
-4712-07-23 14:31:23
(1 row)
```

TO_TIMESTAMP

目的

TO_TIMESTAMP(str,[fmt]) 根据给定的格式将输入参数 str 转换为不带时区的时间戳。如果省略 fmt，则数据将转换为系统默认格式中不带时区值的时间戳。如果 str 为 null，则该函数返回 null。如果无法转换为不带时区的时间戳，则该函数返回错误。

参数

str 输入参数（double precision,text，可以隐式转换为上述类型）。

fmt 输入格式参数，详见格式fmt。

示例

```
select to_timestamp(1212121212.55::numeric);
       to_timestamp
-----
2008-05-30 12:20:12.55
(1 row)

select to_timestamp('2020/03/03 10:13:18 +5:00', 'YYYY/MM/DD HH:MI:SS TZh:TzM');
       to_timestamp
-----
2020-03-03 13:13:18
(1 row)

select to_timestamp(NULL,NULL);
       to_timestamp
-----
(1 row)
```

TO_YMINTERVAL

目的

TO_YMINTERVAL(str) 将输入参数 str 时间间隔转换为年到月范围内的时间间隔。只处理年月，其他部分省略。如果输入参数为NULL，函数返回NULL，如果输入参数格式错误，函数返回错误。

参数

str 输入参数 (text, 可以隐式转换为文本类型, 必须是时间间隔格式。兼容 SQL 标准的 SQL 间隔格式, ISO 持续时间格式与 ISO 8601:2004 标准兼容)。

示例

```
select to_ymininterval('P1Y-2M2D');
to_ymininterval
-----
10 mons
(1 row)

select to_ymininterval('P1Y2M2D');
to_ymininterval
-----
1 year 2 mons
(1 row)

select to_ymininterval('-P1Y2M2D');
to_ymininterval
-----
-1 years -2 mons
(1 row)

select to_ymininterval('-P1Y2M2D');
to_ymininterval
-----
-1 years -2 mons
(1 row)

select to_ymininterval('-01-02');
to_ymininterval
-----
-1 years -2 mons
(1 row)
```

TO_DSINTERVAL

目的

TO_DSINTERVAL(str) 将输入参数 str 的时间间隔转换为天到秒范围内的时间间隔。

输入参数包括: 日、时、分、秒和微秒。

如果输入参数为 NULL, 函数返回 NULL, 如果输入参数包含年月或格式错误, 函数返回错误。

参数

str 输入参数 (text, 可以隐式转换为文本类型, 必须是时间间隔格式。兼容 SQL 标准的 SQL 间隔格式, ISO 持续时间格式与 ISO 8601:2004 标准兼容)。

示例

```
select to_dsinterval('100 00 :02 :00');
```

```
to_dsinterval
```

```
-----  
100 days 00:02:00  
(1 row)
```

```
select to_dsinterval('-100 00:02:00');
```

```
to_dsinterval
```

```
-----  
-100 days -00:02:00  
(1 row)
```

```
select to_dsinterval(NULL);
```

```
to_dsinterval
```

```
-----  
  
(1 row)
```

```
select to_dsinterval('-P100D');
```

```
to_dsinterval
```

```
-----  
-100 days  
(1 row)
```

```
select to_dsinterval('-P100DT20H');
```

```
to_dsinterval
```

```
-----  
-100 days -20:00:00  
(1 row)
```

```
select to_dsinterval('-P100DT20S');
```

```
to_dsinterval
```

```
-----  
-100 days -00:00:20  
(1 row)
```

TO_TIMESTAMP_TZ

目的

TO_TIMESTAMP_TZ(str,[fmt]) 根据给定的格式将输入参数 str 转换为带时区的时间戳。如果省略 fmt，则数据将转换为具有系统默认格式带时区值的时间戳。如果 str 为 null，则该函数返回 null。如果无法转换为带时区的时间戳，则该函数返回错误。

参数

str 输入参数（text，可以隐式转换为文本类型）。

fmt 输入格式参数，详见格式fmt。

示例

```
select to_timestamp_tz('2019','yyyy');
      to_timestamp_tz
-----
2019-01-01 00:00:00+08
(1 row)

select to_timestamp_tz('2019-11','yyyy-mm');
      to_timestamp_tz
-----
2019-11-01 00:00:00+08
(1 row)

select to_timestamp_tz('2003/12/13 10:13:18 +7:00');
      to_timestamp_tz
-----
2003-12-13 11:13:18+08
(1 row)

select to_timestamp_tz('2019/12/13 10:13:18 +5:00', 'YYYY/MM/DD HH:MI:SS TZH:TZM');
      to_timestamp_tz
-----
2019-12-13 13:13:18+08
(1 row)

select to_timestamp_tz(NULL);
      to_timestamp_tz
-----

(1 row)
```

GREATEST

目的

GREATEST(expr1,expr2,...) 获取一个或多个表达式的输入列表中的最大值。如果任何 expr 的计算结果为 NULL，则该函数返回 NULL。

参数

expr1` 输入参数（任意类型）。
`expr2` 输入参数（任意类型）。
`...`

示例

```
select greatest('a','b','A','B');
greatest
-----
b
(1 row)

select greatest(' ','.', '/', ';', '!', '@', '?');
greatest
-----
@
(1 row)

select greatest('瀚','高','数','据','库');
greatest
-----
高
(1 row)

SELECT greatest('HARRY', 'HARRIOT', 'HARRA');
greatest
-----
HARRY
(1 row)

SELECT greatest('HARRY', 'HARRIOT', NULL);
greatest
-----

(1 row)
```



```

SELECT greatest(1.1, 2.22, 3.33);
greatest
-----
      3.33
(1 row)

SELECT greatest('A', 6, 7, 5000, 'E', 'F','G') A;
a
---
G
(1 row)

```

LEAST

目的

LEAST(expr1,expr2,...) 获取一个或多个表达式的输入列表中的最小值。 如果任何 expr 的计算结果为 NULL， 则该函数返回 NULL。

参数

```

expr1` 输入参数（任意类型）。
`expr2` 输入参数（任意类型）。
...

```

示例

```

SELECT least(1,' 2', '3' );
least
-----
      1
(1 row)

SELECT least(NULL, NULL, NULL);
least
-----

(1 row)

SELECT least('A', 6, 7, 5000, 'E', 'F','G') A;
a
-----
5000
(1 row)

```

```

select least(1,3,5,10);
least
-----
1
(1 row)

select least('a','A','b','B');
least
-----
A
(1 row)

select least(',', '!', '/', ';', '!', '@');
least
-----
!
(1 row)

select least('瀚','高','据','库');
least
-----
库
(1 row)

SELECT least('HARRY', 'HARRIOT', NULL);
least
-----

(1 row)

```

fmt（日期/时间格式的模板模式）

模式	描述
HH	一天中的小时（01-12）
HH12	一天中的小时（01-12）
HH24	一天中的小时（00-23）
MI	分钟（00-59） minute（00-59）
SS	秒（00-59）
MS	毫秒（000-999）
US	微秒（000000-999999）
SSSS	午夜后的秒（0-86399）
AM, am, PM or pm	正午指示器（不带句号）

A.M., a.m., P.M. or p.m.	正午指示器（带句号）
Y,YYY	带逗号的年（4 位或者更多位）
YYYY	年（4 位或者更多位）
YYY	年的后三位
YY	年的后两位
Y	年的最后一位
IYYY	ISO 8601 周编号方式的年（4 位或更多位）
IYY	ISO 8601 周编号方式的年的最后 3 位
IY	ISO 8601 周编号方式的年的最后 2 位
I	ISO 8601 周编号方式的年的最后一位
BC, bc, AD或者ad	纪元指示器（不带句号）
B.C., b.c., A.D.或者a.d.	纪元指示器（带句号）
MONTH	全大写形式的月名（空格补齐到 9 字符）
Month	全首字母大写形式的月名（空格补齐到 9 字符）
month	全小写形式的月名（空格补齐到 9 字符）
MON	简写的大写形式的月名（英文 3 字符，本地化长度可变）
Mon	简写的首字母大写形式的月名（英文 3 字符，本地化长度可变）
mon	简写的小写形式的月名（英文 3 字符，本地化长度可变）
MM	月编号（01-12）
DAY	全大写形式的日名（空格补齐到 9 字符）
Day	全首字母大写形式的日名（空格补齐到 9 字符）
day	全小写形式的日名（空格补齐到 9 字符）
DY	简写的大写形式的日名（英语 3 字符，本地化长度可变）
Dy	简写的首字母大写形式的日名（英语 3 字符，本地化长度可变）
dy	简写的小写形式的日名（英语 3 字符，本地化长度可变）
DDD	一年中的日（001-366）
IDDD	ISO 8601 周编号方式的年中的日（001-371，年的第 1 日时第一个 ISO 周的周一）
DD	月中的日（01-31）
D	周中的日，周日（1）到周六（7）
ID	周中的 ISO 8601 日，周一（1）到周日（7）
W	月中的周（1-5）（第一周从该月的第一天开始）
WW	年中的周数（1-53）（第一周从该年的第一天开始）
IW	ISO 8601 周编号方式的年中的周数（01 - 53；新的一年的第一个周四在第一周）
CC	世纪（2 位数）（21 世纪开始于 2001-01-01）
J	儒略日（从午夜 UTC 的公元前 4714 年 11 月 24 日开始的整数日数）
Q	季度（to_date和to_timestamp会忽略）

RM	大写形式的罗马计数法的月（I-XII；I 是一月）
rm	小写形式的罗马计数法的月（i-xii；i 是一月）
TZ	大写形式的时区名称
tz	小写形式的时区名称
OF	时区偏移量

fmt1（数字格式的模板模式）

模式	描述
9	带有指定位数的值
0	带前导零的值
. (period)	小数点
, (comma)	分组（千）分隔符
PR	尖括号内的负值
S	带符号的数字（使用区域）
L	货币符号（使用区域）
D	小数点（使用区域）
G	分组分隔符（使用区域）
MI	在指定位置的负号（如果数字 < 0）
PL	在指定位置的正号（如果数字 > 0）
SG	在指定位置的正/负号
RN	罗马数字（输入在 1 和 3999 之间）
TH or th	序数后缀
V	移动指定位数（参阅注解）
EEEE	科学记数的指数

NLS_LENGTH_SEMANTICS参数

概述

NLS_LENGTH_SEMANTICS 使您能够使用字节或字符长度语义创建 CHAR 和 VARCHAR2 列。现有列不受影响。在这种情况下，默认语义是 BYTE。

语法

```
SET NLS_LENGTH_SEMANTICS TO [NONE | BYTE | CHAR];
```

取值范围说明

BYTE:数据以字节长度来存储。
CHAR:数据以字符长度来存储。
NONE:数据使用原生PostgreSQL存储方式。

用例

--测试 “CHAR”

```
create table test(a varchar2(5));
CREATE TABLE

SET NLS_LENGTH_SEMANTICS TO CHAR;
SET

SHOW NLS_LENGTH_SEMANTICS;
  nls_length_semantics
-----
char
(1 row)

insert into test values ('李老师您好');
INSERT 0 1
```

--测试 “BYTE”

```
SET NLS_LENGTH_SEMANTICS TO BYTE;
SET

SHOW NLS_LENGTH_SEMANTICS;
  nls_length_semantics
-----
byte
(1 row)

insert into test values ('李老师您好');
2021-12-14 15:28:11.906 HKT [6774] ERROR:  value too long for type varchar2(5 byte)
2021-12-14 15:28:11.906 HKT [6774] STATEMENT:  insert into test values ('李老师您好');
ERROR:  value too long for type varchar2(5 byte)
```

VARCHAR2(size)类型

概述

具有最大长度大小字节或字符的可变长度字符串。 您必须为 VARCHAR2 指定大小。 最小大小为 1 个字节或 1 个字符。

语法

VARCHAR2(size)

```

create table test(a varchar2(5));
CREATE TABLE

SET NLS_LENGTH_SEMANTICS TO CHAR;
SET

SHOW NLS_LENGTH_SEMANTICS;
  nls_length_semantics
-----
char
(1 row)

insert into test values ('李老师您好');
INSERT 0 1

```

PL/iSQL

PL/iSQL 是 IvorySQL 的过程语言，用于为 IvorySQL 编写自定义函数、过程和包。PL/iSQL 派生自 PostgreSQL 的 PL/pgsql，并增加了一些功能，但在语法上 PL/iSQL 更接近 Oracle 的 PL/SQL。本文档描述了 PL/iSQL 程序的基本结构和构造。

PL/iSQL 程序的结构

iSQL 是一种程序化的块结构语言，支持四种不同的 程序类型，即 PACKAGES、PROCEDURES、FUNCTIONS 和 TRIGGERS。iSQL 对每种类型的受支持程序使用相同的块结构。一个块最多由三个部分组成：声明部分，可执行文件，和异常部分。而声明和异常部分是可选的。

```

[DECLARE
  declarations]
BEGIN
  statements
[ EXCEPTION
  WHEN <exception_condition> THEN
    statements]
END;

```

一个块至少可以由一个可执行部分组成 在 BEGIN 和 END 关键字中包含一个或多个 iSQL 语句。

```

CREATE OR REPLACE FUNCTION null_func() RETURN VOID AS
BEGIN
  NULL;
END;
/

```

所有关键字都不区分大小写。标识符被隐式转换为小写，除非双引号，就像它们在普通 SQL 命令中一样。声明部分可用于声明变量和游标，并取决于使用块的上下文，声明部分可以以关键字 DECLARE 开头。

```
CREATE OR REPLACE FUNCTION null_func() RETURN VOID AS
DECLARE
    quantity integer := 30;
    c_row pg_class%ROWTYPE;
    r_cursor refcursor;
    CURSOR c1 RETURN pg_proc%ROWTYPE;
BEGIN
    NULL;
end;
/
```

可选的异常部分也可以包含在 BEGIN - END 块中。异常部分以关键字 EXCEPTION 开始，一直持续到它出现的块的末尾。如果块内的语句抛出异常，程序控制转到异常部分，根据异常和异常部分的内容，可能会或不会处理抛出的异常。

```
CREATE OR REPLACE FUNCTION reraise_test() RETURN void AS
BEGIN

    BEGIN
        RAISE syntax_error;
    EXCEPTION
        WHEN syntax_error THEN

            BEGIN
                raise notice 'exception % thrown in inner block, reraising', sqlerrm;
                RAISE;
            EXCEPTION
                WHEN OTHERS THEN
                    raise notice 'RIGHT - exception % caught in inner block', sqlerrm;
            END;
        END;
    EXCEPTION
        WHEN OTHERS THEN
            raise notice 'WRONG - exception % caught in outer block', sqlerrm;
END;
/
```

注意

与 PL/pgSQL 类似，PL/iSQL 使用 BEGIN/END 对语句进行分组，

并且不要将它们与用于事务控制的同名 SQL 命令混淆。PL/iSQL 的 BEGIN/END 仅用于分组；他们不开始或结束事务

psql 对 PL/iSQL 程序的支持

要从 psql 客户端创建 PL/iSQL 程序，您可以使用类似于 PL/pgSQL 的 \$\$ 语法

```
CREATE FUNCTION func() RETURNS void as
$$
..
end$$ language plisql;
```

或者，您可以使用不带 \$\$ 的 Oracle 兼容语法的引用和语言规范，并使用 /（正斜杠）结束程序定义。
/（正斜杠）必须在换行符上

```
CREATE FUNCTION func() RETURN void AS
...
END;
/
```

PL/iSQL 程序语法

PROCEDURES

```
CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter_list)]
is
[DECLARE]
    -- variable declaration
BEGIN
    -- stored procedure body
END;
/
```

FUNCTIONS

```
CREATE [OR REPLACE] FUNCTION function_name ([parameter_list])
RETURN return_type AS
[DECLARE]
    -- variable declaration
BEGIN
    -- function body
    return statement
END;
/
```



```
CREATE [ OR REPLACE ] PACKAGE [schema.] *package_name* [invoker_rights_clause] [IS | AS]
```

```
    item_list[, item_list ...]
```

```
END [*package_name*];
```

```
invoker_rights_clause:
```

```
    AUTHID [CURRENT_USER | DEFINER]
```

```
item_list:
```

```
[
```

```
    function_declaration      |
```

```
    procedure_declaration    |
```

```
    type_definition          |
```

```
    cursor_declaration       |
```

```
    item_declaration
```

```
]
```

```
function_declaration:
```

```
    FUNCTION function_name [(parameter_declaration[, ...])] RETURN datatype;
```

```
procedure_declaration:
```

```
    PROCEDURE procedure_name [(parameter_declaration[, ...])]
```

```
type_definition:
```

```
    record_type_definition    |
```

```
    ref_cursor_type_definition
```

```
cursor_declaration:
```

```
    CURSOR name [(cur_param_decl[, ...])] RETURN rowtype;
```

```
item_declaration:
```

```
    cursor_declaration        |
```

```
    cursor_variable_declaration |
```

```
    record_variable_declaration |
```

```
    variable_declaration      |
```

```
record_type_definition:
```

```

TYPE record_type IS RECORD ( variable_declaration [, variable_declaration]... ) ;

ref_cursor_type_definition:
    TYPE type IS REF CURSOR [ RETURN type%ROWTYPE ];

cursor_variable_declaration:
    curvar curtype;

record_variable_declaration:
    recvar { record_type | rowtype_attribute | record_type%TYPE };

variable_declaration:
    varname datatype [ [ NOT NULL ] := expr ]

parameter_declaration:
    parameter_name [IN] datatype [[:= | DEFAULT] expr]

```

PACKAGE BODY

```

CREATE [ OR REPLACE ] PACKAGE BODY [schema.] package_name [IS | AS]
    [item_list[, item_list ...]] |
    item_list_2 [, item_list_2 ...]
    [initialize_section]
END [package_name];

initialize_section:
    BEGIN statement[, ...]

item_list:
[
    function_declaration      |
    procedure_declaration     |
    type_definition           |
    cursor_declaration        |
    item_declaration
]

item_list_2:
[
    function_declaration
    function_definition

```

```

    procedure_declaration
    procedure_definition
    cursor_definition
]

function_definition:
    FUNCTION function_name [(parameter_declaration[, ...])] RETURN datatype [IS | AS]
    [declare_section] body;

procedure_definition:
    PROCEDURE procedure_name [(parameter_declaration[, ...])] [IS | AS]
    [declare_section] body;

cursor_definition:
    CURSOR name [(cur_param_decl[, ...])] RETURN rowtype IS select_statement;

body:
    BEGIN statement[, ...] END [name];

statement:
    [<<LABEL>>] pl_statments[, ...];

```

层级查询

语法

```

{
    CONNECT BY [ NOCYCLE ] [PRIOR] condition [AND [PRIOR] condition]... [ START WITH
condition ]
    | START WITH condition CONNECT BY [ NOCYCLE ] [PRIOR] condition [AND [PRIOR]
condition]...
}

```

CONNECT BY 查询语法以 CONNECT BY 关键字开头，这些关键字定义了父行和子行之间的分层相互依赖关系。必须通过在 CONNECT BY 子句的条件部分指定 PRIOR 关键字来进一步限定结果。

PRIOR 关键字是一元运算符，它将前一行与当前行联系起来。此关键字可以用在相等条件的左边或右边。

START WITH 该子句指定从哪一行开始层次结构。

NOCYCLE 无操作语句。目前只有语法支持。该子句表示即使存在循环也返回数据。

附加列

LEVEL 返回层次结构中当前行的级别，从根节点的 1 开始，之后每级别递增 1。

CONNECT_BY_ROOT expr 返回层次结构中当前行的父列。

SYS_CONNECT_BY_PATH(col, chr) 它是一个返回从根到当前节点的列值的函数，由字符“chr”分隔。

限制

目前此功能有以下限制：

- 附加列可用于大多数表达式，如函数调用、CASE 语句和通用表达式，但有一些不受支持的列，如 ROW、TYPECAST、COLLATE、GROUPING 子句等
- 两个或多个列相同的情况下，可能需要输出列名，例如

```
> SELECT CONNECT_BY_ROOT col AS "col1", CONNECT_BY_ROOT col AS "col2" ....
```

- 不支持间接运算符或 “**”
- 不支持循环检测（Loop detection）

全局唯一索引

创建全局唯一索引

语法

```
CREATE UNIQUE INDEX [IF NOT EXISTS] name ON table_name [USING method] (columns) GLOBAL
```

示例

```
CREATE UNIQUE INDEX myglobalindex on mytable(bid) GLOBAL;
```

全局唯一性保证

在创建全局唯一索引期间，系统会对所有现有分区执行索引扫描，如果发现来自其他分区的重复项而不是当前分区，则会引发错误。例如：

命令

```
create table gidxpart (a int, b int, c text) partition by range (a);
create table gidxpart1 partition of gidxpart for values from (0) to (100000);
create table gidxpart2 partition of gidxpart for values from (100000) to (199999);
insert into gidxpart (a, b, c) values (42, 572814, 'inserted first on gidxpart1');
insert into gidxpart (a, b, c) values (150000, 572814, 'inserted second on
gidxpart2');
create unique index on gidxpart (b) global;
```

输出

```
ERROR: could not create unique index "gidxpart1_b_idx"
DETAIL: Key (b)=(572814) is duplicated.
```

插入和更新

插入和更新的全局唯一性保证

在全局唯一索引创建过程中，系统会对所有现有分区执行索引扫描，如果在其他分区而不是当前分区中发现重复项，则会引发错误。

示例

命令

```
create table gidx_part (a int, b int, c text) partition by range (a);
create table gidxpart (a int, b int, c text) partition by range (a);
create table gidxpart1 partition of gidxpart for values from (0) to (10);
create table gidxpart2 partition of gidxpart for values from (10) to (100);
create unique index gidx_u on gidxpart using btree(b) global;

insert into gidxpart values (1, 1, 'first');
insert into gidxpart values (11, 11, 'eleventh');
insert into gidxpart values (2, 11, 'duplicated (b)=(11) on other partition');
```

输出

```
ERROR:  duplicate key value violates unique constraint "gidxpart2_b_idx"
DETAIL:  Key (b)=(11) already exists.
```

附加和分离

附加语句的全球唯一性保证

将新表附加到具有全局唯一索引的分区表时，系统将对所有现有分区进行重复检查。如果在现有分区中发现与附加表中的元组匹配的重复项，则会引发错误并且附加失败。

附加需要所有现有分区上的共享锁（sharedlock）。如果其中一个分区正在进行并发 INSERT，则附加将等待它先完成。这可以在未来的版本中改进

示例

运行命令

```
create table gidxpart (a int, b int, c text) partition by range (a);
create table gidxpart1 partition of gidxpart for values from (0) to (100000);
insert into gidxpart (a, b, c) values (42, 572814, 'inserted first on gidxpart1');
create unique index on gidxpart (b) global;
create table gidxpart2 (a int, b int, c text);
insert into gidxpart2 (a, b, c) values (150000, 572814, 'dup inserted on gidxpart2');

alter table gidxpart attach partition gidxpart2 for values from (100000) to (199999);
```

```
ERROR:  could not create unique index "gidxpart1_b_idx"
DETAIL:  Key (b)=(572814) is duplicated.
```

运维管理指南

由于IvorySQL基于PostgreSQL开发而成，运维人员在阅读理解本节内容时，建议同时参阅 [手册](#)。

升级IvorySQL版本

升级方案概述

IvorySQL版本号由主要版本和次要版本组成。例如，IvorySQL 3.2 中的3是主要版本，2是次要版本。

发布次要版本是不会改变内存的存储格式，因此总是和相同的主要版本兼容。例如，IvorySQL 3.2 和Ivory SQL 3.0 以及后续的 Ivory SQL 3.x 兼容。对于这些兼容版本的升级非常简单，只要关闭数据库服务，安装替换二进制的可执行文件，重新启动服务即可。

接下来，我们主要讨论IvorySQL的跨版本升级问题，例如，从 IvorySQL 2.3 升级到 IvorySQL 3.4 。主要版本的升级可能会修改内部数据的存储格式，因此需要执行额外的操作。常用的跨版本升级方法和适用场景如下：

升级方法	适用场景	停机时间
通过pg_dumpall升级数据	中小型数据库，例如小于100GB支持跨平台数据迁移	取决于数据库的大小
pg_upgrade工具	大中型数据库，例如大于100GB本机就地升级	几分钟
逻辑复制	大中型数据库，例如大于100GB跨平台支持	几秒钟

注意事项：
新的主版本通常会引入一些用户可见的不兼容性，因此可能需要应用程序编程上的改变。所有用户可见的更改都被列在 [说明](#)中，请特别注意标有“Migration”的小节。尽管你可能从一个主版本升级到另一个，而不用升级中间版本，你应该阅读全部中间版本的主要发行说明。

通过pg_dumpall升级数据

传统的跨版本升级方法利用pg_dump/pg_dumpall逻辑备份到处数据库，然后在新版本中通过pg_restore进行还原。导出旧版本数据库时推荐使用新版本的pg_dump/pg_dumpall工具，可以利用最新的并行导出和还原功能，同时可以减少数据库膨胀问题。

逻辑备份与还原非常简单但速度比较慢，停机时间取决于数据库的大小，因此适合中小型数据库的升级。

下面介绍这种升级方法的具体操作，假如当前IvorySQL软件的安装目录位于/usr/local/pgsql，同时数据目录位于/usr/local/pgsql/data，我们在同一台服务器上进行升级。

1. 执行逻辑备份之前停止应用程序，确保没有数据更新，因为备份开始后的更新不会被导出。如有必要，可以修改/usr/local/pgsql/data/pg_hba.conf文件禁止其他人访问数据库。然后备份数据库：

```
pg_dumpall > outputfile
```

如果已经安装了新版本的IvorySQL，可以使用新版本的pg_dumpall命令备份旧版本数据库。

2.停止旧版本的后台服务：

```
pg_ctl stop
```

或者通过其他方式停止后台服务。

3.如果安装目录没有包含特定版本标识，可以将目录改名，必要时可以再修改回来。可以使用类似以下的命令重命名目录：

```
mv /usr/local/pgsql /usr/local/pgsql.old
```

4.安装新版本IvorySQL软件，假如安装目录仍然是/usr/local/pgsql。

5.初始化一个新的数据库集群，需要使用数据库专用用户（通常是postgres；如果升级版本，应该已经存在该用户）执行操作：

```
/usr/local/pgsql/bin/initdb -D /usr/local/pgsql/data
```

6.将旧版本配置文件pg_hba.conf 和 postgresql.conf 等中的改动在对应的新配置文件中再次进行修改。

7.使用数据库专用用户启动新版本的后台服务：

```
/usr/local/pgsql/bin/postgres -D /usr/local/pgsql/data
```

8.最后利用新版本的psql命令还原数据：

```
/usr/local/pgsql/bin/psql -d postgres -f outputfile
```

为了减少停机时间，可以将新版本的IvorySQL安装到另一个目录，同时使用不同的端口启动服务。然后同时执行数据库的导出和导入：

```
pg_dumpall -p 5432 | psql -d postgres -p 5433
```

执行以上操作时，新旧版本的后台服务同时运行，新版本使用5433端口，旧版本使用5432端口。

利用pg_upgrade 工具进行升级

pg_upgrade 工具可以支持IvorySQL跨版本的就地升级。升级可以在数分钟内被执行，特别是使用—link模式时。它要求和上面的pg_dumpall相似的步骤，例如启动/停止 服务器、运行initdb。pg_upgrade [文档](#)概述了所需的步骤。

通过复制升级数据

我们也可以使用IvorySQL的已更新版本逻辑复制来创建一个后备服务器，逻辑复制支持在不同主版本的Ivory SQL之间的复制。后备服务器可以在同一台计算机或者不同的计算机上。一旦它和主服务器（运行旧版本的IvorySQL）同步好，你可以切换主机并且将后备服务器作为主机，然后关闭旧的数据库实例。这样一种切换

使得一次升级的停机时间只有数秒。

这种升级方法可以用内置的逻辑复制工具和外部的逻辑复制系统如pglogical, Slony, Londiste, 和Bucardo。

管理IvorySQL版本

IvorySQL基于PostgreSQL开发，版本更新频率与PostgreSQL版本更新频率保持一致，每年更新一个大版本，每季度更新一个小版本。IvorySQL目前发布的版本有1.0到4.2，分别基于PostgreSQL 14.0到17.2进行开发，最新版本为IvorySQL 4.2，基于PostgreSQL 17.2进行开发。IvorySQL的所有版本全部都做到了向下兼容。相关版本特性可以查看 [官网](#)。

管理IvorySQL数据库访问

IvorySQL使用 角色

的概念管理数据库访问权限。一个角色可以看成是一个数据库用户或者是数据库用户组，这取决于角色被怎样设置。角色可以拥有数据库对象（例如，表和函数）并且能够把那些对象上的权限赋予给其他角色来控制谁能访问哪些对象。此外，还可以把一个角色中的 成员资格授予给另一个角色，这样允许成员角色使用被赋予给另一个角色的权限。

角色的概念把“用户”和“组”的概念都包括在内。

数据库角色在概念上已经完全与操作系统用户独立开来。事实上可能维护一个对应关系会比较方便，但是这并非必需。数据库角色在一个数据库集簇安装范围内是全局的（而不是独立数据库内）。要创建一个角色，可使用CREATE ROLE SQL 命令：

```
CREATE ROLE name;
```

name遵循 SQL

标识符的规则：或是未经装饰没有特殊字符，或是用双引号包围（实际上，你将总是给该命令要加上额外选项，例如LOGIN。更多细节可见下文）。要移除一个已有的角色，使用相似的DROP ROLE命令：

```
DROP ROLE name;
```

为了方便，createuser和dropuser程序被提供作为这些 SQL 命令的包装器，它们可以从 shell 命令行调用：

```
createuser name
dropuser name
```

要决定现有角色的集合，检查pg_roles系统目录，例如：

```
SELECT rolname FROM pg_roles;
```

psql程序的\du元命令也可以用来列出现有角色。

为了引导数据库系统，一个刚刚被初始化好的系统总是包含一个预定义角色。这个角色总是一个“superuser”，并且默认情况下（除非在运行initdb时修改）它的名字和初始化数据库集簇的操作系统用户相同。习惯上，这个角色将被命名为postgres。为了创建更多角色，你首先必须以初始角色的身份连接。

每一个到数据库服务器的连接都是使用某个特定角色名建立的，并且这个角色决定发起连接的命令的初始访问权限。要使用一个特定数据库连接的角色名由客户端指示，该客户端以一种应用相关的风格发起连接请求。例如，psql程序使用-

U命令行选项来指定要以哪个角色连接。很多应用假定该名字默认是当前操作系统用户（包括createuser和ps

ql) 。因此在角色和操作系统用户之间维护一个名字对应关系通常是很方便的。

一个给定客户端连接能够用来连接的数据库角色的集合由该客户端的认证设置决定，因此，一个客户端不止限于以匹配其操作系统用户的角色连接，就像一个人的登录名不需要匹配她的真实名字一样。因为角色身份决定一个已连接客户端可用的权限集合，在设置一个多用户环境时要小心地配置权限。

一个数据库角色可以有一些属性，它们定义角色的权限并且与客户端认证系统交互。

把用户分组在一起便于管理权限常常很方便：那样，权限可以被授予一整个组或从一整个组回收。在Ivory SQL中通过创建一个表示组的角色来实现，并且然后将在该组角色中的 成员关系 授予给单独的用户角色。

由于角色可以拥有数据库对象并且能持有访问其他对象的特权，删除一个角色常常并非一次DROP ROLE就能解决。

任何被该用户所拥有的对象必须首先被删除或者转移给其他拥有者，并且任何已被授予给该角色的权限必须被收回。

更多有关数据库访问管理的细节，可以参阅 [手册](#)。

定义数据对象

IvorySQL基于PostgreSQL，具有完整的SQL，其定义数据对象可以参考 [手册](#)。在此基础之上，IvorySQL为兼容Oracle，还做了一些Oracle专有数据对象的兼容。

VARCHAR2

概述

具有最大长度大小字节或字符的可变长度字符串。您必须为 VARCHAR2 指定大小。最小大小为 1 个字节或 1 个字符。

语法

```
VARCHAR2(size)
```

用例

```
create table test(a varchar2(5));
CREATE TABLE

SET NLS_LENGTH_SEMANTICS TO CHAR;
SET

SHOW NLS_LENGTH_SEMANTICS;
  nls_length_semantics
-----
char
(1 row)

insert into test values ('李老师您好');
INSERT 0 1
```

查询数据

IvorySQL基于PostgreSQL开发，具有完全的SQL，查询数据具体的操作可以参考 [手册](#)。

使用外部数据

IvorySQL实现了部分的SQL/MED规定，允许我们使用普通SQL查询来访问位于IvorySQL之外的数据。这种数据被称为外部数据（注意这种用法不要和外键混淆，后者是数据库中的一种约束）。

外部数据可以在一个外部数据包装器的帮助下被访问。一个外部数据包装器是一个库，它可以与一个外部数据源通讯，并隐藏连接到数据源和从它获取数据的细节。在contrib模块中有一些外部数据包装器，参见 [文档](#)。其他类型的外部数据包装器可以在第三方产品中找到。如果这些现有的外部数据包装器都不能满足你的需要，可以自己编写一个，参见 [手册](#)。

要访问外部数据，我们需要建立一个外部服务器对象，它根据它所支持的外部数据包装器所使用的一组选项定义了如何连接到一个特定的外部数据源。接着我们需要创建一个或多个外部表，它们定义了外部数据的结构。一个外部表可以在查询中像一个普通表一样地使用，但是在IvorySQL服务器中外部表没有存储数据。不管使用什么外部数据包装器，IvorySQL会要求外部数据包装器从外部数据源获取数据，或者在更新命令的情况下传送数据到外部数据源。

访问远程数据可能需要在外部数据源的授权。这些信息通过一个用户映射提供，它基于当前的IvorySQL角色提供了附加的数据例如用户名和密码。

备份与恢复

由于包含着有价值的数据，IvorySQL数据库应当被定期地备份。虽然过程相当简单，但清晰地理解其底层技术和假设是非常重要的。

有三种不同的基本方法来备份 IvorySQL 数据：

- SQL转储
- 文件系统级备份
- 连续归档

SQL转储

SQL

转储方法的思想是创建一个由SQL命令组成的文件，当把这个文件回馈给服务器时，服务器将利用其中的SQL命令重建与转储时状态一样的数据库。IvorySQL为此提供了工具pg_dump。这个工具的基本用法是：

```
pg_dump dbname > dumpfile
```

正如你所见，pg_dump把结果输出到标准输出。我们后面将看到这样做有什么用处。尽管上述命令会创建一个文本文件，pg_dump可以用其他格式创建文件以支持并行和细粒度的对象恢复控制。

pg_dump是一个普通的IvorySQL客户端应用（尽管是个相当聪明的东西）。这就意味着你可以在任何可以访问该数据库的远端主机上进行备份工作。但是请记住pg_dump不会以任何特殊权限运行。具体说来，就是它必须要有你想备份的表的读权限，因此为了备份整个数据库你几乎总是必须以一个数据库超级用户来运行它（如果你没有足够的特权来备份整个数据库，你仍然可以使用诸如-n schema 或-t table选项来备份该数据库中你能够访问的部分）。

要声明pg_dump连接哪个数据库服务器，使用命令行选项-h host和 -p port。

默认主机是本地主机或你的HOST环境变量指定的主机。

类似地，默认端口是环境变量PORT或（如果PORT不存在）内建的默认值。

（服务器通常有相同的默认值，所以还算方便。）

pg_dump默认使用与当前操作系统用户名同名的数据库用户名进行连接。要使用其他名字，要么声明-

U选项，要么设置环境变量PGUSER。请注意pg_dump的连接也要通过客户认证机制。

pg_dump对于其他备份方法的一个重要优势是，pg_dump的输出可以很容易地在新版本的IvorySQL中载入，而文件级备份和连续归档都是极度的服务器版本限定的。pg_dump也是唯一可以将一个数据库传送到一个不同机器架构上的方法，例如从一个32位服务器到一个64位服务器。

由pg_dump创建的备份在内部是一致的，也就是说，转储表现了pg_dump开始运行时刻的数据库快照，且在pg_dump运行过程中发生的更新将不会被转储。pg_dump工作的时候并不阻塞其他的对数据库的操作。（但是会阻塞那些需要排它锁的操作，比如大部分形式的ALTER TABLE）

从转储中恢复

pg_dump生成的文本文件可以由psql程序读取。从转储中恢复的常用命令是：

```
psql dbname < dumpfile
```

其中dumpfile就是pg_dump命令的输出文件。这条命令不会创建数据库dbname，你必须在执行psql前自己从template0创建（例如，用命令createdb -T template0 dbname）。psql支持类似pg_dump的选项用以指定要连接的数据库服务器和要使用的用户名。参阅psql的手册获取更多信息。非文本文件转储可以使用pg_restore工具来恢复。

在开始恢复之前，转储库中对象的拥有者以及在其上被授予了权限的用户必须已经存在。如果它们不存在，那么恢复过程将无法将对象创建成具有原来的所属关系以及权限（有时候这就是你所需要的，但通常不是）。

默认情况下，psql脚本在遇到一个SQL错误后会继续执行。你也许希望在遇到一个SQL错误后让psql退出，那么可以设置ON_ERROR_STOP变量来运行psql，这将使psql在遇到SQL错误后退出并返回状态3：

```
psql --set ON_ERROR_STOP=on dbname < infile
```

不管怎样，你将只能得到一个部分恢复的数据库。作为另一种选择，你可以指定让整个恢复作为一个单独的事务运行，这样恢复要么完全完成要么完全回滚。这种模式可以通过向psql传递-1或—single-transaction命令行选项来指定。在使用这种模式时，注意即使是很小的一个错误也会导致运行了数小时的恢复被回滚。但是，这仍然比在一个部分恢复后手工清理复杂的数据库要更好。

pg_dump和psql读写管道的能力使得直接从一个服务器转储一个数据库到另一个服务器成为可能，例如：

```
pg_dump -h host1 dbname | psql -h host2 dbname
```

重要：pg_dump产生的转储是相对于template0。这意味着在template1中加入的任何语言、过程等都会被pg_dump转储。结果是，如果在恢复时使用的是一个自定义的template1，你必须从template0创建一个空的数据库，正如上面的例子所示。

一旦完成恢复，在每个数据库上运行ANALYZE是明智的举动，这样优化器就有有用的统计数据了。

使用pg_dumpall

pg_dump每次只转储一个数据库，而且它不会转储关于角色或表空间（因为它们是集群范围的）的信息。为了支持方便地转储一个数据库集群的全部内容，提供了pg_dumpall程序。pg_dumpall备份一个给定集群中的每一个数据库，并且也保留了集群范围的数据，如角色和表空间定义。该命令的基本用法是：

```
pg_dumpall > dumpfile
```

转储的结果可以使用psql恢复：

```
psql -f dumpfile postgres
```

（实际上，你可以指定恢复到任何已有数据库名，但是如果你正在将转储载入到一个空集群中则通常要用（postgres）。在恢复一个pg_dumpall转储时常常需要具有数据库超级用户访问权限，因为它需要恢复角色和表空间信息。如果你在使用表空间，请确保转储中的表空间路径适合于新的安装。

pg_dumpall工作时发出命令重新创建角色、表空间和空数据库，接着为每一个数据库pg_dump。这意味着每个数据库自身是一致的，但是不同数据库的快照并不同步。

集群范围的数据可以使用pg_dumpall的—globals-only选项来单独转储。如果在单个数据库上运行pg_dump命令，上述做法对于完全备份整个集群是必需的。

处理大型数据库

在一些具有最大文件尺寸限制的操作系统上创建大型的pg_dump输出文件可能会出现问题。幸运地是，pg_dump可以写出到标准输出，因此你可以使用标准Unix工具来处理这种潜在的问题。有几种可能的方法：

使用压缩转储。你可以使用你喜欢的压缩程序，例如gzip：

```
pg_dump dbname | gzip > filename.gz
```

恢复：

```
gunzip -c filename.gz | psql dbname
```

或者：

```
cat filename.gz | gunzip | psql dbname
```

使用split。split命令允许你将输出分割成较小的文件以便能够适应底层文件系统的尺寸要求。例如，让每一块的大小为2G字节：

```
pg_dump dbname | split -b 2G - filename
```

恢复：

```
cat filename* | psql dbname
```

如果使用GNU split，可能会把它和gzip一起使用：

```
pg_dump dbname | split -b 2G --filter='gzip > $FILE.gz'
```

它可以使用zcat恢复。

使用pg_dump的自定义转储格式。如果IvorySQL所在的系统上安装了zlib压缩库，自定义转储格式将在写出数据到输出文件时对其压缩。这将产生和使用gzip时差不多大小的转储文件，但是这种方式的一个优势是其中的表可以被有选择地恢复。下面的命令使用自定义

义转储格式来转储一个数据库：

```
pg_dump -Fc dbname > filename
```

自定义格式的转储不是psql的脚本，只能通过pg_restore恢复，例如：

```
pg_restore -d dbname filename
```

更多细节可以参阅 [手册](#)。

对于非常大型的数据库，你可能需要将split配合其他两种方法之一进行使用。

使用pg_dump的并行转储特性。

为了加快转储一个大型数据库的速度，你可以使用pg_dump的并行模式。它将同时转储多个表。你可以使用-j参数控制并行度。并行转储只支持“目录”归档格式。

```
pg_dump -j num -F d -f out.dir dbname
```

你可以使用pg_restore

-j来以并行方式恢复一个转储。它只能适合于“自定义”归档或者“目录”归档，但不管归档是否由pg_dump -j创建。

文件系统级别备份

另外一种备份策略是直接复制

IvorySQL用于存储数据库中数据的文件，你可以采用任何你喜欢的方式进行文件系统备份，例如：

```
tar -cf backup.tar /usr/local/pgsql/data
```

但是这种方法有两个限制，使得这种方法不实用，或者说至少比pg_dump方法差：

1. 为了得到一个可用的备份，数据库服务器必须被关闭。例如阻止所有连接的半路措施是不起作用的（部分原因是tar和类似工具无法得到文件系统状态的一个原子的快照，还有服务器内部缓冲的原因）。不用说，在恢复数据之前你也需要关闭服务器。
2. 如果你已经深入地了解了数据库的文件系统布局的细节，你可能会有兴趣尝试通过相应的文件或目录来备份或恢复特定的表或数据库。这种方法也不会起作用，因为包含在这些文件中的信息只有配合提交日志文件（pg_xact/*）才有用，提交日志文件包含了所有事务的提交状态。一个表文件只有和这些信息一起才有用。当然也不可能只恢复一个表及相关的pg_xact数据，因为这会导致数据库集群中所有其他表变得无用。因此文件系统备份值适合于完整地备份或恢复整个数据库集群。

另一种文件系统备份方法是创建一个数据目录的“一致快照”，如果文件系统支持此功能（并且你相信它的实现正确）。典型的过程是创建一个包含数据库的卷的“冻结快照”，然后从该快照复制整个数据目录（如上，不能是部分复制）到备份设备，最后释放冻结快照。即使在数据库服务器运行时，这种方式也有效。但是，以这种方式创建的备份保存的文件看起来就像数据库没有被正确关闭时的状态。因此，当你从备份数据上启动数据库服务器时，它会认为上一次的服务器实例崩溃了并尝试重放WAL日志。这不是问题，只是需要注意（当然WAL文件必须要包括在备份中）。你可以在拍摄快照之前执行一次CHECKPOINT以便节省恢复时间。

如果你的数据库跨越多个文件系统，可能没有任何方式可以对所有卷获得完全同步的冻结快照。例如，如果你的数据文件和WAL日志放置在不同的磁盘上，或者表空间在不同的文件系统中，可能没有办法使用快照备份，因为快照必须是同步的。在这些情况下，一定要仔细阅读你的文件系统文档以了解其对一致快照技术的支持。

如果没有可能获得同步快照，一种选择是将数据库服务器关闭足够长的时间以建立所有的冻结快照。另一种选择是执行一次连续归档基础备份，因为这种备份对于备份期间发生的文件系统改变是免疫的。这要求在备份过程中允许连续归档，恢复时使用连续归档恢复。

还有一种选择是使用rsync来执行一次文件系统备份。其做法是先在数据库服务器运行时执行rsync，然后关闭数据库服务器足够长时间来做一次rsync --checksum（--checksum是必需的，因为rsync的文件修改时间粒度只能精确到秒）。第二次rsync会比第一次快，因为它只需要传送相对很少的数据，由于服务器是停止的，所以最终结果将是一致的。这种方法允许在最小停机时间内执行一次文件系统备份。

注意一个文件系统备份通常会比一个SQL转储体积更大（例如pg_dump不需要转储索引的内容，而是转储用于重建索引的命令）。但是，做一次文件系统备份可能更快。

连续归档和时间点恢复（PITR）

在任何时间，IvorySQL在数据集群目录的pg_wal/子目录下都保持有一个预写式日志（WAL）。这个日志存在的目的是为了保证崩溃后的安全：如果系统崩溃，可以“重放”从最后一次检查点以来的日志项来恢复数据库的一致性。该日志的存在也使得第三种备份数据库的策略变得可能：我们可以把一个文件系统级别的备份和WAL文件的备份结合起来。当需要恢复时，我们先恢复文件系统备份，然后从备份的WAL文件中重放来把系统带到一个当前状态。这种方法比之前的方法管理起来要更复杂，但是有其显著的优点：

- 我们不需要一个完美的一致性的文件系统备份作为开始点。备份中的任何内部不一致性将通过日志重放（这和崩溃恢复期间发生的并无显著不同）来修正。因此我们不需要文件系统快照功能，只需要tar或一个类似的归档工具。
- 由于我们可以结合一个无穷长的WAL文件序列用于重放，可以通过简单地归档WAL文件来达到连续备份。这对于大型数据库特别有用，因为在其中不方便频繁地进行完全备份。
- 并不需要一直重放WAL项一直到最后。我们可以在任何点停止重放，并得到一个数据库在当时的一致快照。这样，该技术支持时间点恢复：
在得到你的基础备份以后，可以将数据库恢复到它在其后任何时间的状态。
- 如果我们连续地将一系列WAL文件输送给另一台已经载入了相同基础备份文件的机器，我们就得到了一个热后备系统：在任何时间点我们都能提出第二台机器，它差不多是数据库的当前副本。

注意: pg_dump 和 pg_dumpall 不会产生文件系统级别的备份，并且不能用于连续归档方案。这类转换是逻辑的并且不包含足够的信息用于WAL重放。

就简单的文件系统备份技术来说，这种方法只能支持整个数据库集群的恢复，却无法支持其中一个子集的恢复。另外，它需要大量的归档存储：一个基础备份的体积可能很庞大，并且一个繁忙的系统将会产生大量需要被归档的WAL流量。尽管如此，在很多需要高可靠性的情况下，它是首选的备份技术。

要使用连续归档（也被很多数据库厂商称为“在线备份”）成功地恢复，你需要一个从基础备份时间开始的连续的归档WAL文件序列。为了开始，在你建立第一个基础备份之前，你应该建立并测试用于归档WAL文件的过程。对应地，我们首先讨论归档WAL文件的机制。关于如何建立归档和备份的方式以及操作过程中的要点，请参阅 [手册](#)。

装卸数据

copy 在 IvorySQL 表和标准文件之间移动数据。COPY TO 把一个表的内容复制到一个文件，而COPY FROM 则从一个文件复制数据到一个表（把数据追加到表中原有数据）。COPY TO 也能复制一个SELECT查询的结果。

如果指定了一个列列表，COPY TO 将只把指定列的数据复制到文件。对于COPY FROM，文件中的每个字段将按顺序插入到指定列中。COPY FROM 命令的列列表中并没有指定的表列则会采纳其默认值。

带一个文件名的COPY指示IvorySQL服务器直接从一个文件读取或者写入到一个文件。该文件必须是IvorySQL用户（运行服务器的用户ID）可访问的并且应该以服务器的视角来指定其名称。当指定了PROGRAM时，服务器执行给定的命令并且从该程序的标准输出读取或者写入到该程序的标准输入。该程序必须以服务器的视角指定，并且必须是IvorySQL用户可执行的。在指定STDIN或者STDOUT时，数据会通过客户端和服务器之间的连接传输。

运行COPY的每个后端将在pg_stat_progress_copy视图中报告其进度。

大纲

```
COPY table_name [ ( column_name [, ...] ) ]  
  FROM { 'filename' | PROGRAM 'command' | STDIN }  
  [ [ WITH ] ( option [, ...] ) ]  
  [ WHERE condition ]  
  
COPY { table_name [ ( column_name [, ...] ) ] | ( query ) }  
  TO { 'filename' | PROGRAM 'command' | STDOUT }  
  [ [ WITH ] ( option [, ...] ) ]
```

其中 option 可以是下列之一：

```
FORMAT format_name  
FREEZE [ boolean ]  
DELIMITER 'delimiter_character'  
NULL 'null_string'  
HEADER [ boolean ]  
QUOTE 'quote_character'  
ESCAPE 'escape_character'  
FORCE_QUOTE { ( column_name [, ...] ) | * }  
FORCE_NOT_NULL ( column_name [, ...] )  
FORCE_NULL ( column_name [, ...] )  
ENCODING 'encoding_name'
```

详细参数设置，请参阅 [手册](#)。

输出

在成功完成时，一个COPY命令会返回一个形为

COPY count

count 是被复制的行数。注意：如果命令不是COPY ... TO STDOUT或者等效的psql命令\copy ... to stdout, psql将只打印这个命令标签。这是为了防止弄混命令标签和刚刚打印的数据。

注解

COPY TO只能用于普通表，而不能用于视图，并且不能从子表或子分区复制行。例如，COPY table TO 复制与SELECT * FROM ONLY table 相同的行。语法COPY (SELECT * FROM table) TO ... 可用于转储一个继承层次结构、分区表或视图中的所有行。

COPY FROM可以被用于普通表、外部表、分区表或者具有INSTEAD OF INSERT触发器的视图。

你必须拥有被COPY TO读取的表上的选择特权， 以及被COPY FROM插入的表上的插入特权。

拥有在命令中列出的列上的特权就足够了。

如果对表启用了行级安全性，相关的SELECT策略将应用于COPY table TO语句。当前，有行级安全性的表不支持COPY FROM。不过可以使用等效的INSERT语句。

COPY命令中提到的文件会被服务器（而不是客户端应用）直接读取或写入。因此它们必须位于数据库服务器（不是客户端）的机器上或者是数据库服务器可以访问的。它们必须是 IvorySQL 用户（运行服务器的用户 ID）可访问的并且是可读或者可写的。类似地，用PROGRAM指定的命令也会由服务器（不是客户端应用）直接执行，它也必须是在 IvorySQL 用户可以执行的。只允许数据库超级用户或者授予了角色pg_read_server_files、pg_write_server_files及pg_execute_server_program之一的用户COPY一个文件或者命令，因为它允许读取或者写入服务器有特权访问的任何文件或者运行服务器有特权访问的程序。

不要把COPY和 psql指令 \copy 弄混。 \copy会调用 COPY FROM STDIN或者COPY TO STDOUT，然后读取/存储一个 psql客户端可访问的文件中的数据。因此，在使用 \copy时，文件的可访问性和访问权利取决于客户端而不是服务器。

我们推荐在COPY中使用的文件名总是指定为一个绝对路径。在COPY TO的情况下服务器会强制这一点，但是对于 COPY FROM你可以选择一个用相对路径指定的文件中读取。该路径将根据服务器进程（而不是客户端）的工作目录（通常是集簇的数据目录）解释。

用PROGRAM执行一个命令可能会受到操作系统的访问控制机制（如 SELinux）的限制。

COPY FROM将调用目标表上的任何触发器和检查约束。但是它不会调用规则。

对于标识列，COPY FROM命令将总是写上输入数据中提供的列值，这和INSERT的选项OVERRIDING SYSTEM VALUE的行为一样。

COPY输入和输出受到 DateStyle的影响。为了确保到其他可能使用非默认DateStyle设置的 IvorySQL 安装的可移植性，在使用 COPY TO前应该把 DateStyle设置为ISO。避免转储把 IntervalStyle设置为sql_standard的数据也是一个好主意，因为负的区间值可能会被具有不同IntervalStyle设置的服务器解释错误。

即使数据会被服务器直接从一个文件读取或者写入一个文件而不通过客户端，输入数据也会被根据ENCODING选项或者当前客户端编码解释，并且输出数据会被根据ENCODING或者当前客户端编码进行编码。

COPY会在第一个错误处停止操作。这在 COPY TO的情况下不会导致问题，但是在COPY FROM中目标表将已经收到了一些行。这些行将不会变得可见或者可访问，但是它们仍然占据磁盘空间。如果在一次大型的复制操作中出现错误，这可能浪费相当可观的磁盘空间。你可能希望调用VACUUM来恢复被浪费的空间。

FORCE_NULL和FORCE_NOT_NULL可以被同时用在同一列上。这会导致把已被引用的空值串转换为空值并且把未引用的空值串转换为空串。

文件格式

文本格式

在使用text格式时，读取或写入的是一个文本文件，其中每一行就是表中的一行。一行中的列被定界字符分隔。列值本身是由输出函数产生的或者是可被输入函数接受的属于每个属性数据类型的字符串。在为空值的列的位置使用指定的空值串。如果输入文件的任何行包含比预期更多或者更少的列，COPY FROM将会抛出一个错误。

数据的结束可以表示为一个只包含反斜线-点号 (\.) 的单一行。从一个文件读取时，数据结束标记并不是必要的，因为文件结束符就已经足够用了。只有使用3.0客户端协议之前的客户端应用复制数据时才需要它。

反斜线字符 (\) 可以被用在COPY数据中来引用被用作行或者列界定符的字符。特别地，如果下列字符作为一个列值的一部分出现，它们必须被前置一个反斜线：反斜线本身、新行、回车以及当前的定界符字符。

COPY TO会不加任何反斜线返回指定的空值串。相反，COPY FROM会在移除反斜线之前把输入与空值串相匹配。因此，一个空值串（例如\N）不会与实际的数据值\N（它会被表示为\\N）搞混。

COPY FROM识别下列特殊的反斜线序列：

序列	表示
\b	退格 (ASCII 8)
\f	换页 (ASCII 12)
\n	新行 (ASCII 10)
\r	回车 (ASCII 13)
\t	制表 (ASCII 9)
\v	纵向制表 (ASCII 11)
\digits	反斜线后跟一到三个八进制数字表示该数字代码对应的字节
\xdigits	反斜线加x后跟一到三个十六进制数字表示该数字代码对应的字节

当前，COPY TO不会发出一个八进制或十六进制位反斜线序列，但是它确实把上面列出的其他序列用于那些控制字符。

任何上述表格中没有提到的其他反斜线字符将被当作表示其本身。不过，要注意增加不必要的反斜线，因为那可能意外地产生一个匹配数据结束标记 (\.) 或者空值串（默认是\N）的字符串。这些字符串将在完成任何其他反斜线处理之前被识别。

强烈建议产生COPY数据的应用把数据新行和回车分别转换为\n和\r序列。当前可以把一个数据回车表示为一个反斜线和回车，把一个数据新行表示为一个反斜线和新行。不过，未来的发行可能不会接受这些表示。如果在不同的机器之间（例如从 Unix 到 Windows）传输COPY文件，它们也很容易受到破坏。

所有反斜杠序列都在编码转换后进行解释。用八进制和十六进制数字反斜杠序列指定的字节必须在数据库编码中形成有效字符。

COPY TO将用一个 Unix 风格的新行 (“\n”) 终止每一行。运行在 Microsoft Windows 上的服务器则会输出回车/新行 (“\r\n”)，不过只对COPY到一个服务器文件这样做。为了做到跨平台一致，COPY TO STDOUT总是发送 “\n” 而不管服务器平台是什么。COPY FROM能够处理以新行、回车或者回车/新行结尾的行。为了减少由作为数据的未加反斜线的新行或者回车带来的风险，如果输出中的行结束并不完全相似，COPY FROM将会抱怨。

CSV格式

这种格式选项被用于导入和导出很多其他程序（例如电子表格）使用的逗号分隔值 (CSV) 文件格式。不同于 IvorySQL 标准文本格式使用的转义规则，它产生并且识别一般的 CSV 转义机制。

每个记录中的值用DELIMITER字符分隔。如果值包含 定界符字符、QUOTE字符、NULL字符串、一个回车或者换行字符，那么整个值会被加上QUOTE字符作为前缀或者后缀，并且在该值内QUOTE字符或者 ESCAPE字符的任何一次出现之前放上转义字符。在输出指定列中非NULL值时，还可以使用 FORCE_QUOTE来强制加上引用。

CSV格式没有标准方式来区分NULL值和空字符串。IvorySQL的COPY用引用来处理这种区分工作。NULL被按照NULL参数字符串输出并且不会被引用，而匹配NULL参数字符串的非NULL值会被加上引用。例如，使用默认设置时，NULL被写作一个未被引用的空字符串，而一个空字符串数据值会被写成带双引号 ("")。

值的读取遵循类似的规则。你可以用FORCE_NOT_NULL来防止 对指定列的NULL输入比较。你还可以使用FORCE_NULL把带引用的空值字符串数据值转换成NULL。

因为反斜线在CSV格式中不是一种特殊字符，数据结束标记\\也可以作为一个数据值出现。为了避免任何解释误会，在一行上作为孤项出现的\\数据值输出时会自动被引用，并且输入时如果被引用，则不会被解释为数据结束标记。如果正在载入一个由另一个应用创建的文件并且其中具有一个未被引用的列且可能具有\\值，你可能需要在输入文件中引用该值。

注意

CSV格式中，所有字符都是有意义的。一个被空白或者其他非DELIMITER字符围绕的引用值将包括那些字符。在导入来自用空白填充CSV行到固定长度的系统的数据时，这可能会导致错误。如果出现这种情况，在导入数据到IvorySQL之前，你可能需要预处理该CSV文件以移除拖尾的空白。

注意

CSV 格式将识别并且产生带有包含嵌入的回车和换行的引用值的 CSV 文件。因此文件并不限于文本格式文件的每个表行一行的形式。

注意

很多程序会产生奇怪的甚至偶尔是不合常理的 CSV 文件，因此该文件格式更像是一种习惯而不是标准。因此你可能会碰到一些无法使用这种机制导入的文件，并且COPY也可能产生其他程序无法处理的文件。

二进制格式

binary格式选项导致所有数据被以二进制格式 而不是文本格式存储/读取。它比文本和CSV格式要快一些，但是二进制格式文件在不同的机器架构和 IvorySQL 版本之间的可移植性要差些。还有，二进制格式与数据格式非常相关。例如不能从一个smallint列中输出二进制数据并且把它读入到一个 integer列中，虽然这样做在文本格式中是可行的。

binary文件格式由一个文件头、零个或者更多个包含行数据的元组以及一个文件尾构成。头部和数据都以网络字节序表示。

文件头

文件头由 15 字节的固定域构成，后面跟着一个变长的头部扩展区。

固定域有：

签名

11-字节的序列PGCOPY\\n\\377\\r\\n\\0 — 注意
零字节是签名的一个必要的部分（该签名是为了能容易地发现文件被 无法正确处理 8 位字符编码的传输所破坏。这个签名将被行尾翻译过滤器、删除零字节、删除高位或者奇偶修改等改变）。

标志域

32-位整数位掩码，用以表示该文件格式的重要方面。位被编号为 从 0（LSB）到 31（MSB）。注意这个域以网络字节序存放（最高有效位在前），所有该文件格式 中使用的整数域都是这样。16-31 位被保留用来表示严重的文件格式 问题， 读取者如果在这个范围内发现预期之外的被设置位，它应该中止。0-15 位被保留用来表示向后兼容的格式问题，读取者应该简单地略过这个范围内任何预期之外的被设置位。当前只定义了一个标志 位，其他位必须为零：

位 16

如果为 1，表示数据中包含 OID；如果为 0，则不包含。IvorySQL 不再支持 Oid 系统列，但是格式仍然包含该指示符。

头部扩展区长度

32-为整数，表示头部剩余部分的以字节计的长度，不包括其本身。当前，这个长度为零，并且其后就紧跟着第一个元组。未来对该格式的更改可能会允许在头部中表示额外的数据。如果读取者不知道要对头部扩展区数据做什么，可以安静地跳过它。

头部扩展区域被预期包含一个能自我解释的块的序列。该标志域并不想告诉读取者扩展数据是什么。详细的头部扩展内容的设计留给后来的发行去做。

这种设计允许向后兼容的头部增加（增加头部扩展块或者设置低位标志位）以及非向后兼容的更改（设置高位标志位来表示这类更改并且在需要时向扩展区域 中增加支持数据）。

元组

每一个元组由一个表示元组中域数量的 16 位整数计数开始（当前，一个表中的所有元组都应该具有相同的计数，但是这可能不会总是为真）。然后是元组 中的每一个域，它是一个 32 位的长度字，后面则跟随着这么多个字节的域数据（长度字不包括其本身，并且可以是零）。作为一种特殊情况，-1 表示一个 NULL 域值。在 NULL 情况下，后面不会跟随值字节。

在域之间没有对齐填充或者任何其他额外的数据。

当前，一个二进制格式文件中的所有数据值都被假设为二进制格式（格式代码一）。可以预见未来的扩展可能会增加一个允许独立指定各列的格式代码的头部域。

要为实际的元组数据决定合适的二进制格式，你应该参考 IvorySQL 源码，特别是用于各列数据类型的 *send 和 *recv 函数（通常可以在源码的 src/backend/utils/adt/ 目录中找到 这些函数）。

如果文件中包含 OID，OID 域会紧跟在域计数字之后。它是一个普通域，不过它没有被包含在域计数中。注意 IvorySQL 当前版本不支持 oid 系统列。

文件尾

文件位由一个包含 -1 的 16 位整数字组成。这很容易与一个 元组的域计数字区分开。

如果一个域计数字不是 -1 也不是期望的列数，读取者应该报告错误。这提供了一种针对某种数据不同步的额外检查。

示例

下面的例子使用竖线 (|) 作为域定界符把一个表复制到客户端：

```
COPY country TO STDOUT (DELIMITER '|');
```

从一个文件中复制数据到 country 表中：

```
COPY country TO STDOUT (DELIMITER '|');
```

只把名称以 'A' 开头的国家复制到一个文件中：

```
COPY (SELECT * FROM country WHERE country_name LIKE 'A%') TO  
'/usr1/proj/bray/sql/a_list_countries.copy';
```

要复制到一个压缩文件中，你可以用管道把输出导到一个外部压缩程序：

```
COPY country TO PROGRAM 'gzip > /usr1/proj/bray/sql/country_data.gz';
```

这里是一个适合于从STDIN复制到表中的数据：

```
AF      AFGHANISTAN
AL      ALBANIA
DZ      ALGERIA
ZM      ZAMBIA
ZW      ZIMBABWE
```

注意每一行上的空白实际是一个制表符。

下面是用二进制格式输出的相同数据。该数据是用 Unix 工具 `od -c` 过滤后显示的。该表具有三列，第一列类型是 `char(2)`，第二列类型是 `text`，第三列类型是 `integer`。所有行在第三列都是空值。

```
0000000 P G C O P Y \n 377 \r \n \0 \0 \0 \0 \0
0000020 \0 \0 \0 \0 003 \0 \0 \0 002 A F \0 \0 \0 013 A
0000040 F G H A N I S T A N 377 377 377 377 \0 003
0000060 \0 \0 \0 002 A L \0 \0 \0 007 A L B A N I
0000100 A 377 377 377 377 \0 003 \0 \0 \0 002 D Z \0 \0 \0
0000120 007 A L G E R I A 377 377 377 377 \0 003 \0 \0
0000140 \0 002 Z M \0 \0 \0 006 Z A M B I A 377 377
0000160 377 377 \0 003 \0 \0 \0 002 Z W \0 \0 \0 \b Z I
0000200 M B A B W E 377 377 377 377 377 377
```

剩余的详细信息可以参阅 [手册](#)。

性能管理

查询性能可能受多种因素影响。其中一些因素可以由用户控制，而其他的则属于系统下层设计的基本原理。

使用EXPLAIN

IvorySQL 为每个收到查询产生一个查询计划。

选择正确的计划来匹配查询结构和数据的属性对于好的性能来说绝对是最关键的，因此系统包含了一个复杂的规划器来尝试选择好的计划。你可以使用 `EXPLAIN` 命令察看规划器为任何查询生成的查询计划。

阅读查询计划是一门艺术，它要求一些经验来掌握，但是本节只试图覆盖一些基础。

这些例子使用 `EXPLAIN` 的默认 “text” 输出格式，这种格式紧凑并且便于阅读。如果你想把 `EXPLAIN` 的输出交给一个程序做进一步分析，你应该使用它的某种机器可读的输出格式（XML、JSON 或 YAML）。

EXPLAIN基础

查询计划的结构是一个计划结点的树。最底层的结点是扫描结点：它们从表中返回未经处理的行。

不同的表访问模式有不同的扫描结点类型：顺序扫描、索引扫描、位图索引扫描。

也还有不是表的行来源，例如 `VALUES` 子句和 `FROM` 中返回集合的函数，它们有自己的结点类型。如果查询需要连接、聚集、排序、或者在未经处理的行上的其它操作，那么就会在扫描结点之上有其它额外的结点来执

行这些操作。并且，做这些操作通常都有多种方法，因此在这些位置也有可能出现不同的结点类型。EXPLAIN给计划树中每个结点都输出一行，显示基本的结点类型和计划器为该计划结点的执行所做的开销估计。第一行（最上层的结点）是对该计划的总执行开销的估计；计划器试图最小化的就是这个数字。

这里是一个简单的例子，只是用来显示输出看起来是什么样的：

```
EXPLAIN SELECT * FROM tenk1;
```

QUERY PLAN

```
-----  
Seq Scan on tenk1 (cost=0.00..458.00 rows=10000 width=244)
```

由于这个查询没有WHERE子句，它必须扫描表中的所有行，因此计划器只能选择使用一个简单的顺序扫描计划。被包含在圆括号中的数字是（从左至右）：

- 估计的启动开销。在输出阶段可以开始之前消耗的时间，例如在一个排序节点里执行排序的时间。
- 估计的总开销。这个估计值基于的假设是计划结点会被运行到完成，即所有可行的行都被检索。不过实际上一个结点的父节点可能很快停止读取所有可行的行（见下面的LIMIT例子）。
- 这个计划结点输出行数的估计值。同样，也假定该结点能运行到完成。
- 预计这个计划结点输出的行平均宽度（以字节计算）。

开销是用规划器的开销参数所决定的捏造单位来衡量的。传统上以取磁盘页面为单位来度量开销；也就是seq_page_cost将被按照习惯设为1.0，其它开销参数将相对于它来设置。本节的例子都假定这些参数使用默认值。

有一点很重要：一个上层结点的开销包括它的所有子结点的开销。还有一点也很重要：这个开销只反映规划器关心的东西。特别是这个开销没有考虑结果行传递给客户端所花费的时间，这个时间可能是实际花费时间中的一个重要因素；但是它被规划器忽略了，因为它无法通过修改计划来改变（我们相信，每个正确的计划都将输出同样的行集）。

行数值有一些小技巧，因为它不是计划结点处理或扫描过的行数，而是该结点发出的行数。这通常比被扫描的行数少一些，因为有些被扫描的行会被应用于此结点上的任意WHERE子句条件过滤掉。理想中顶层的行估计会接近于查询实际返回、更新、删除的行数。

回到我们的例子：

```
EXPLAIN SELECT * FROM tenk1;
```

QUERY PLAN

```
-----  
Seq Scan on tenk1 (cost=0.00..458.00 rows=10000 width=244)
```

这些数字的产生非常直接。如果你执行：

```
SELECT relpages, reltuples FROM pg_class WHERE relname = 'tenk1';
```

你会发现tenk1有358个磁盘页面和10000行。开销被计算为

（页面读取数*seq_page_cost）+（扫描的行数*cpu_tuple_cost）。默认情况下，seq_page_cost是1.0，cpu_tuple_cost是0.01，因此估计的开销是（358 * 1.0）+（10000 * 0.01）= 458。

现在让我们修改查询并增加一个WHERE条件：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 7000;
```

QUERY PLAN

```
-----  
Seq Scan on tenk1 (cost=0.00..483.00 rows=7001 width=244)  
  Filter: (unique1 < 7000)
```

请注意EXPLAIN输出显示WHERE子句被当做一个“过滤器”条件附加到顺序扫描计划结点。这意味着该计划结点为它扫描的每一行检查该条件，并且只输出通过该条件的行。因为WHERE子句的存在，估计的输出行数降低了。不过，扫描仍将必须访问所有 10000 行，因此开销没有被降低；实际上开销还有所上升（准确来说，上升了 10000 * cpu_operator_cost）以反映检查WHERE条件所花费的额外 CPU 时间。

这条查询实际选择的行数是 7000，但是估计的rows只是个近似值。如果你尝试重复这个试验，那么你很可能得到略有不同的估计。此外，这个估计会在每次ANALYZE命令之后改变，因为ANALYZE生成的统计数据是从该表中随机采样计算的。

现在，让我们把条件变得更严格：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100;
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on tenk1 (cost=5.07..229.20 rows=101 width=244)  
  Recheck Cond: (unique1 < 100)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)  
        Index Cond: (unique1 < 100)
```

这里，规划器决定使用一个两步的计划：子计划结点访问一个索引来找出匹配索引条件的行的位置，然后上层计划结点实际地从表中取出那些行。独立地抓取行比顺序地读取它们的开销高很多，但是不是所有的表页面都被访问，这么做实际上仍然比一次顺序扫描开销要少（使用两层计划的原因是因为上层规划结点把索引标识出来的行位置在读取之前按照物理位置排序，这样可以最小化单独抓取的开销。结点名称里面提到的“位图”是执行该排序的机制）。

现在让我们给WHERE子句增加另一个条件：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND stringu1 = 'xxx';
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on tenk1 (cost=5.04..229.43 rows=1 width=244)  
  Recheck Cond: (unique1 < 100)  
  Filter: (stringu1 = 'xxx'::name)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)
```

Index Cond: (unique1 < 100)

新增的条件stringu1 = 'xxx' 减少了估计的输出行计数，但是没有减少开销，因为我们仍然需要访问相同的行集合。请注意，stringu1子句不能被应用为一个索引条件，因为这个索引只是在unique1列上。它被用来过滤从索引中检索出的行。因此开销实际上略微增加了一些以反映这个额外的检查。

在某些情况下规划器将更倾向于一个“simple”索引扫描计划：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 = 42;
```

QUERY PLAN

```
-----  
Index Scan using tenk1_unique1 on tenk1 (cost=0.29..8.30 rows=1 width=244)  
  Index Cond: (unique1 = 42)
```

在这类计划中，表行被按照索引顺序取得，这使得读取它们开销更高，但是其中有一些是对行位置排序的额外开销。你很多时候将在只取得一个单一行的查询中看到这种计划类型。它也经常被用于拥有匹配索引顺序的ORDER BY子句的查询中，因为那样就不需要额外的排序步骤来满足ORDER BY。在此示例中，添加 ORDER BY unique1将使用相同的计划，因为索引已经隐式提供了请求的排序。

规划器可以通过多种方式实现ORDER BY子句。上面的例子表明，这样的排序子句可以隐式实现。规划器还可以添加一个明确的sort步骤：

```
EXPLAIN SELECT * FROM tenk1 ORDER BY unique1;
```

QUERY PLAN

```
-----  
Sort (cost=1109.39..1134.39 rows=10000 width=244)  
  Sort Key: unique1  
  -> Seq Scan on tenk1 (cost=0.00..445.00 rows=10000 width=244)
```

如果计划的一部分保证对所需排序键的前缀进行排序，那么计划器可能会决定使用incremental sort步骤：

```
EXPLAIN SELECT * FROM tenk1 ORDER BY four, ten LIMIT 100;
```

QUERY PLAN

```
-----  
-----  
Limit (cost=521.06..538.05 rows=100 width=244)  
  -> Incremental Sort (cost=521.06..2220.95 rows=10000 width=244)  
      Sort Key: four, ten  
      Presorted Key: four  
      -> Index Scan using index_tenk1_on_four on tenk1 (cost=0.29..1510.08  
rows=10000 width=244)
```

与常规排序相比，增量排序允许在对整个结果集进行排序之前返回元组，这尤其可以使用LIMIT查询进行优化。

它还可以减少内存使用和将排序溢出到磁盘的可能性，但其代价是将结果集拆分为多个排序批次的开销增加。

如果在WHERE引用的多个行上有独立的索引，规划器可能会选择使用这些索引的一个 AND 或 OR 组合：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000;
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on tenk1 (cost=25.08..60.21 rows=10 width=244)  
  Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))  
    -> BitmapAnd (cost=25.08..25.08 rows=10 width=0)  
      -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)  
          Index Cond: (unique1 < 100)  
      -> Bitmap Index Scan on tenk1_unique2 (cost=0.00..19.78 rows=999 width=0)  
          Index Cond: (unique2 > 9000)
```

但是这要求访问两个索引，所以与只使用一个索引并把其他条件作为过滤器相比，它不一定能胜出。如果你变动涉及到的范围，你将看到计划也会相应改变。

下面是一个例子，它展示了LIMIT的效果：

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 LIMIT 2;
```

QUERY PLAN

```
-----  
Limit (cost=0.29..14.48 rows=2 width=244)  
  -> Index Scan using tenk1_unique2 on tenk1 (cost=0.29..71.27 rows=10 width=244)  
      Index Cond: (unique2 > 9000)  
      Filter: (unique1 < 100)
```

这是和上面相同的查询，但是我们增加了一个LIMIT这样不是所有的行都需要被检索，并且规划器改变了它的决定。注意索引扫描结点的总开销和行计数显示出好像它会被运行到完成。但是，限制结点在检索到这些行的五分之一后就会停止，因此它的总开销只是索引扫描结点的五分之一，并且这是查询的实际估计开销。之所以用这个计划而不是在之前的计划上增加一个限制结点是因为限制无法避免在位图扫描上花费启动开销，因此总开销会是超过那种方法（25个单位）的某个值。

让我们尝试连接两个表，使用我们已经讨论过的列：

```
EXPLAIN SELECT *  
FROM tenk1 t1, tenk2 t2  
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----  
Nested Loop (cost=4.65..118.62 rows=10 width=488)
```



```

-> Bitmap Heap Scan on tenk1 t1 (cost=4.36..39.47 rows=10 width=244)
    Recheck Cond: (unique1 < 10)
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)
        Index Cond: (unique1 < 10)
-> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..7.91 rows=1 width=244)
    Index Cond: (unique2 = t1.unique2)

```

在这个计划中，我们有一个嵌套循环连接结点，它有两个表扫描作为输入或子结点。该结点的摘要行的缩进反映了计划树的结构。连接的第一个（或“outer”）子结点是一个与前面见到的相似的位图扫描。它的开销和行计数与我们从SELECT ... WHERE unique1 < 10得到的相同，因为我们将WHERE子句unique1 < 10用在了那个结点上。t1.unique2 = t2.unique2子句现在还不相关，因此它不影响 outer 扫描的行计数。嵌套循环连接结点将为从 outer 子结点得到的每一行运行它的第二个（或“inner”）子结点。当前 outer 行的列值可以被插入 inner 扫描。这里，来自 outer 行的t1.unique2值是可用的，所以我们得到的计划和开销与前面见到的简单SELECT ... WHERE t2.unique2 = constant情况相似（估计的开销实际上比前面看到的略低，是因为在t2上的重复索引扫描会利用到高速缓存）。循环结点的开销则被以 outer 扫描的开销为基础设置，外加对每一个 outer 行都要进行一次 inner 扫描（10 * 7.87），再加上用于连接处理一点 CPU 时间。

在这个例子里，连接的输出行计数等于两个扫描的行计数的乘积，但通常并不是所有的情况中都如此，因为可能有同时提及两个表的额外WHERE子句，并且因此它只能被应用于连接点，而不能影响任何一个输入扫描。这里是一个例子：

```

EXPLAIN SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t2.unique2 < 10 AND t1.hundred < t2.hundred;

```

QUERY PLAN

```

-----
Nested Loop (cost=4.65..49.46 rows=33 width=488)
  Join Filter: (t1.hundred < t2.hundred)
  -> Bitmap Heap Scan on tenk1 t1 (cost=4.36..39.47 rows=10 width=244)
      Recheck Cond: (unique1 < 10)
      -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)
          Index Cond: (unique1 < 10)
  -> Materialize (cost=0.29..8.51 rows=10 width=244)
      -> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..8.46 rows=10
width=244)
          Index Cond: (unique2 < 10)

```

条件t1.hundred < t2.hundred不能在tenk2_unique2索引中被测试，因此它被应用在连接结点。这缩减了连接结点的估计输出行计数，但是没有改变任何输入扫描。

注意这里规划器选择了“物化”连接的 inner 关系，方法是在它的上方放了一个物化计划结点。这意味着t2索引扫描将只被做一次，即使嵌套循环连接结点需要读取其数据十次（每个来自 outer 关系的行都要读一次）。物化结点在读取数据时将它保存在内存中，然后在每一次后续执行时从内存返回数据。

在处理外连接时，你可能会看到连接计划结点同时附加有“连接过滤器”和普通“过滤器”条件。连接过滤器条件来自于外连接的ON子句，因此一个无法通过连接过滤器条件的行也能够作为一个空值扩展的行被发出。但是一个普通过滤器条件被应用在外连接条件之后并且因此无条件移除行。在一个内连接中这两种过滤器类型没有语义区别。

如果我们把查询的选择度改变一点，我们可能得到一个非常不同的连接计划：

```
EXPLAIN SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
Hash Join (cost=230.47..713.98 rows=101 width=488)
  Hash Cond: (t2.unique2 = t1.unique2)
    -> Seq Scan on tenk2 t2 (cost=0.00..445.00 rows=10000 width=244)
    -> Hash (cost=229.20..229.20 rows=101 width=244)
        -> Bitmap Heap Scan on tenk1 t1 (cost=5.07..229.20 rows=101 width=244)
            Recheck Cond: (unique1 < 100)
            -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101
width=0)
                Index Cond: (unique1 < 100)
```

这里规划器选择了使用一个哈希连接，在其中一个表的行被放入一个内存哈希表，在这之后其他表被扫描并且为每一行查找哈希表来寻找匹配。同样要注意缩进是如何反映计划结构的：tenk1上的位图扫描是哈希结点的输入，哈希结点会构造哈希表。然后哈希表会返回给哈希连接结点，哈希连接结点将从它的 outer 子计划读取行，并为每一个行搜索哈希表。

另一种可能的连接类型是一个归并连接，如下所示：

```
EXPLAIN SELECT *
FROM tenk1 t1, onek t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
Merge Join (cost=198.11..268.19 rows=10 width=488)
  Merge Cond: (t1.unique2 = t2.unique2)
    -> Index Scan using tenk1_unique2 on tenk1 t1 (cost=0.29..656.28 rows=101
width=244)
        Filter: (unique1 < 100)
    -> Sort (cost=197.83..200.33 rows=1000 width=244)
        Sort Key: t2.unique2
        -> Seq Scan on onek t2 (cost=0.00..148.00 rows=1000 width=244)
```

归并连接要求它的输入数据被按照连接键排序。在这个计划中，tenk1数据被使用一个索引扫描排序，以便能

够按照正确的顺序来访问行。但是对于onek则更倾向于一个顺序扫描和排序，因为在那个表中有更多行需要被访问（对于很多行的排序，顺序扫描加排序常常比一个索引扫描好，因为索引扫描需要非顺序的磁盘访问）。

一种查看变体计划的方法是强制规划器丢弃它认为开销最低的任何策略，这可以使用启用/禁用标志实现例如，如果我们并不认同在前面的例子中顺序扫描加排序是处理表onek的最佳方法，我们可以尝试：

```
SET enable_sort = off;
```

```
EXPLAIN SELECT *
FROM tenk1 t1, onek t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----
Merge Join (cost=0.56..292.65 rows=10 width=488)
  Merge Cond: (t1.unique2 = t2.unique2)
    -> Index Scan using tenk1_unique2 on tenk1 t1 (cost=0.29..656.28 rows=101
width=244)
      Filter: (unique1 < 100)
    -> Index Scan using onek_unique2 on onek t2 (cost=0.28..224.79 rows=1000
width=244)
```

这显示规划器认为用索引扫描来排序onek的开销要比用顺序扫描加排序的方式高大约12%。当然，下一个问题是是否真的是这样。我们可以通过使用EXPLAIN ANALYZE来仔细研究一下，如下文所述。

EXPLAIN ANALYZE

可以通过使用EXPLAIN的ANALYZE选项来检查规划器估计值的准确性。通过使用这个选项，EXPLAIN会实际执行该查询，然后显示真实的行计数和在每个计划结点中累计的真实运行时间，还会有一个普通EXPLAIN显示的估计值。例如，我们可能得到这样一个结果：

```
EXPLAIN ANALYZE SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----
Nested Loop (cost=4.65..118.62 rows=10 width=488) (actual time=0.128..0.377 rows=10
loops=1)
  -> Bitmap Heap Scan on tenk1 t1 (cost=4.36..39.47 rows=10 width=244) (actual
time=0.057..0.121 rows=10 loops=1)
    Recheck Cond: (unique1 < 10)
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)
(actual time=0.024..0.024 rows=10 loops=1)
```

```

                Index Cond: (unique1 < 10)
-> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..7.91 rows=1 width=244)
(actual time=0.021..0.022 rows=1 loops=10)
                Index Cond: (unique2 = t1.unique2)
Planning time: 0.181 ms
Execution time: 0.501 ms

```

注意“actual time”值是以毫秒计的真实时间，而cost估计值被以捏造的单位表示，因此它们不大可能匹配上。在这里面要查看的最重要的一点是估计的行计数是否合理地接近实际值。在这个例子中，估计值都是完全正确的，但是在实际中非常少见。

在某些查询计划中，可以多次执行一个子计划结点。例如，inner 索引扫描可能会因为上层嵌套循环计划中的每一个 outer 行而被执行一次。在这种情况下，loops值报告了执行该结点的总次数，并且 actual time 和行数值是这些执行的平均值。这是为了让这些数字能够与开销估计被显示的方式有可比性。将这些值乘上 loops值可以得到在该结点中实际消耗的总时间。在上面的例子中，我们在执行tenk2的索引扫描上花费了总共 0.220 毫秒。

在某些情况中，EXPLAIN ANALYZE会显示计划结点执行时间和行计数之外的额外执行统计信息。例如，排序和哈希结点提供额外的信息：

```

EXPLAIN ANALYZE SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2 ORDER BY t1.fivethous;

                                QUERY PLAN

-----
Sort (cost=717.34..717.59 rows=101 width=488) (actual time=7.761..7.774 rows=100
loops=1)
  Sort Key: t1.fivethous
  Sort Method: quicksort  Memory: 77kB
-> Hash Join (cost=230.47..713.98 rows=101 width=488) (actual time=0.711..7.427
rows=100 loops=1)
    Hash Cond: (t2.unique2 = t1.unique2)
    -> Seq Scan on tenk2 t2 (cost=0.00..445.00 rows=10000 width=244) (actual
time=0.007..2.583 rows=10000 loops=1)
    -> Hash (cost=229.20..229.20 rows=101 width=244) (actual time=0.659..0.659
rows=100 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 28kB
      -> Bitmap Heap Scan on tenk1 t1 (cost=5.07..229.20 rows=101
width=244) (actual time=0.080..0.526 rows=100 loops=1)
        Recheck Cond: (unique1 < 100)
        -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101
width=0) (actual time=0.049..0.049 rows=100 loops=1)

```

Index Cond: (unique1 < 100)

Planning time: 0.194 ms

Execution time: 8.008 ms

排序结点显示使用的排序方法（尤其是，排序是在内存中还是磁盘上进行）和需要的内存或磁盘空间量。哈希结点显示了哈希桶的数量和批数，以及被哈希表所使用的内存量的峰值（如果批数超过一，也将会涉及到磁盘空间使用，但是并没有被显示）。

另一种类型的额外信息是被一个过滤器条件移除的行数：

```
EXPLAIN ANALYZE SELECT * FROM tenk1 WHERE ten < 7;
```

QUERY PLAN

```
-----  
-----  
Seq Scan on tenk1 (cost=0.00..483.00 rows=7000 width=244) (actual time=0.016..5.107  
rows=7000 loops=1)  
  Filter: (ten < 7)  
    Rows Removed by Filter: 3000  
Planning time: 0.083 ms  
Execution time: 5.905 ms
```

这些值对于被应用在连接结点上的过滤器条件特别有价值。只有在至少有一个被扫描行或者在连接结点中一个可能的连接对被过滤器条件拒绝时，“Rows Removed”行才会出现。

一个与过滤器条件相似的情况出现在“有损”索引扫描中。例如，考虑这个查询，它搜索包含一个指定点的多边形：

```
EXPLAIN ANALYZE SELECT * FROM polygon_tbl WHERE f1 @> polygon '(0.5,2.0)';
```

QUERY PLAN

```
-----  
-----  
Seq Scan on polygon_tbl (cost=0.00..1.05 rows=1 width=32) (actual time=0.044..0.044  
rows=0 loops=1)  
  Filter: (f1 @> '((0.5,2))'::polygon)  
    Rows Removed by Filter: 4  
Planning time: 0.040 ms  
Execution time: 0.083 ms
```

规划器认为（非常正确）这个采样表太小不值得劳烦一次索引扫描，因此我们得到了一个普通的顺序扫描，其中的所有行都被过滤器条件拒绝。但是如果我们强制使得一次索引扫描可以被使用，我们看到：

```
SET enable_seqscan TO off;
```

```
EXPLAIN ANALYZE SELECT * FROM polygon_tbl WHERE f1 @> polygon '(0.5,2.0)';
```

QUERY PLAN

```
-----  
-----  
Index Scan using gpolygonind on polygon_tbl (cost=0.13..8.15 rows=1 width=32)  
(actual time=0.062..0.062 rows=0 loops=1)  
  Index Cond: (f1 @> '((0.5,2))'::polygon)  
  Rows Removed by Index Recheck: 1  
Planning time: 0.034 ms  
Execution time: 0.144 ms
```

这里我们可以看到索引返回一个候选行，然后它会被索引条件的重新检查拒绝。这是因为一个 GiST 索引对于多边形包含测试是

“有损的”：它确实返回覆盖目标的多边形的行，然后我们必须在那些行上做精确的包含性测试。

EXPLAIN有一个BUFFERS选项可以和ANALYZE一起使用来得到更多运行时统计信息：

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000;
```

QUERY PLAN

```
-----  
-----  
Bitmap Heap Scan on tenk1 (cost=25.08..60.21 rows=10 width=244) (actual  
time=0.323..0.342 rows=10 loops=1)  
  Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))  
  Buffers: shared hit=15  
  -> BitmapAnd (cost=25.08..25.08 rows=10 width=0) (actual time=0.309..0.309 rows=0  
loops=1)  
    Buffers: shared hit=7  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)  
(actual time=0.043..0.043 rows=100 loops=1)  
      Index Cond: (unique1 < 100)  
      Buffers: shared hit=2  
      -> Bitmap Index Scan on tenk1_unique2 (cost=0.00..19.78 rows=999 width=0)  
(actual time=0.227..0.227 rows=999 loops=1)  
        Index Cond: (unique2 > 9000)  
        Buffers: shared hit=5  
Planning time: 0.088 ms  
Execution time: 0.423 ms
```

BUFFERS提供的数字帮助我们标识查询的哪些部分是对 I/O 最敏感的。

记住因为EXPLAIN

ANALYZE实际运行查询，任何副作用都将照常发生，即使查询可能输出的任何结果被丢弃来支持打印EXPLAIN数据。如果你想要分析一个数据修改查询而不想改变你的表，你可以在分析完后回滚命令，例如：

```
BEGIN;
```

```
EXPLAIN ANALYZE UPDATE tenk1 SET hundred = hundred + 1 WHERE unique1 < 100;
```

QUERY PLAN

```
-----  
-----  
Update on tenk1 (cost=5.07..229.46 rows=101 width=250) (actual time=14.628..14.628  
rows=0 loops=1)  
  -> Bitmap Heap Scan on tenk1 (cost=5.07..229.46 rows=101 width=250) (actual  
time=0.101..0.439 rows=100 loops=1)  
    Recheck Cond: (unique1 < 100)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)  
(actual time=0.043..0.043 rows=100 loops=1)  
      Index Cond: (unique1 < 100)  
Planning time: 0.079 ms  
Execution time: 14.727 ms
```

```
ROLLBACK;
```

正如在这个例子中所看到的，当查询是一个INSERT、UPDATE或DELETE命令时，应用表更改的实际工作由顶层插入、更新或删除计划结点完成。这个结点之下的计划结点执行定位旧行以及/或者计算新数据的工作。因此在上图，我们看到我们已经见过的位图表扫描，它的输出被交给一个更新结点，更新结点会存储被更新过的行。还有一点值得注意的是，尽管数据修改结点可能要可观的运行时间（这里，它消耗最大份额的时间），规划器当前并没有对开销估计增加任何东西来说明这些工作。这是因为这些工作对每一个正确的查询计划都得做，所以它不影响计划的选择。

当一个UPDATE或者DELETE命令影响继承层次时，输出可能像这样：

```
EXPLAIN UPDATE parent SET f2 = f2 + 1 WHERE f1 = 101;
```

QUERY PLAN

```
-----  
Update on parent (cost=0.00..24.53 rows=4 width=14)  
  Update on parent  
  Update on child1  
  Update on child2  
  Update on child3  
  -> Seq Scan on parent (cost=0.00..0.00 rows=1 width=14)  
      Filter: (f1 = 101)  
  -> Index Scan using child1_f1_key on child1 (cost=0.15..8.17 rows=1 width=14)  
      Index Cond: (f1 = 101)  
  -> Index Scan using child2_f1_key on child2 (cost=0.15..8.17 rows=1 width=14)  
      Index Cond: (f1 = 101)  
  -> Index Scan using child3_f1_key on child3 (cost=0.15..8.17 rows=1 width=14)
```

在这个例子中，更新节点需要考虑三个子表以及最初提到的父表。因此有四个输入的扫描子计划，每一个对应于一个表。为清楚起见，在更新节点上标注了将被更新的相关目标表，显示的顺序与相应的子计划相同（这些标注是从 PostgreSQL 9.5 开始新增的，在以前的版本中读者必须通过观察子计划才能知道这些目标表）。

EXPLAIN ANALYZE 显示的 Planning time 是从一个已解析的查询生成查询计划并进行优化所花费的时间，其中不包括解析和重写。

EXPLAIN ANALYZE 显示的 Execution

time 包括执行器的启动和关闭时间，以及运行被触发的任何触发器的时间，但是它不包括解析、重写或规划的时间。如果有花在执行 BEFORE 执行器的时间，它将被包括在相关的插入、更新或删除结点的时间内；但是用来执行 AFTER

触发器的时间没有被计算，因为 AFTER 触发器是在整个计划完成后被触发的。在每个触发器（BEFORE 或 AFTER）也被独立地显示。注意延迟约束触发器将不会被执行，直到事务结束，并且因此根本不会被 EXPLAIN ANALYZE 考虑。

警告

在两种有效的方法中 EXPLAIN

ANALYZE 所度量的运行时间可能偏离同一个查询的正常执行。首先，由于不会有输出行被递交给客户端，网络传输开销和 I/O 转换开销没有被包括在内。其次，由 EXPLAIN ANALYZE 所增加的度量开销可能会很可观，特别是在操作系统调用 gettimeofday() 很慢的机器上。你可以使用 pg_test_timing 工具来度量在你的系统上的计时开销。

EXPLAIN 结果不应该被外推到与你实际测试的非常不同的情况。例如，一个很小的表上的结果不能被假定成适合大型表。规划器的开销估计不是线性的，并且因此它可能为一个更大或更小的表选择一个不同的计划。一个极端例子是，在一个只占据一个磁盘页面的表上，你将几乎总是得到一个顺序扫描计划，而不管索引是否可用。规划器认识到它在任何情况下都将采用一次磁盘页面读取来处理该表，因此用额外的页面读取去查看一个索引是没有价值的（我们已经在前面的 polygon_tbl 例子中见过）。

在一些情况中，实际的值和估计的值不会匹配得很好，但是这并非错误。一种这样的情况发生在计划结点的执行被 LIMIT 或类似的效果很快停止。例如，在我们之前用过的 LIMIT 查询中：

```
EXPLAIN ANALYZE SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 LIMIT 2;
```

QUERY PLAN

```
-----
Limit (cost=0.29..14.71 rows=2 width=244) (actual time=0.177..0.249 rows=2 loops=1)
  -> Index Scan using tenk1_unique2 on tenk1 (cost=0.29..72.42 rows=10 width=244)
      (actual time=0.174..0.244 rows=2 loops=1)
        Index Cond: (unique2 > 9000)
        Filter: (unique1 < 100)
        Rows Removed by Filter: 287
Planning time: 0.096 ms
Execution time: 0.336 ms
```

索引扫描结点的估计开销和行计数被显示成好像它会运行到完成。但是实际上限制结点在得到两个行之后就停止请求行，因此实际的行计数只有 2 并且运行时间远低于开销估计所建议的时间。这并非估计错误，这仅仅一种估计值和实际值显示方式上的不同。

归并连接也有类似的现象。如果一个归并连接用尽了一个输入并且其中的最后一个键值小于另一个输入中的下一个键值，它将停止读取另一个输入。在这种情况下，不会有更多的匹配并且因此不需要扫描第二个输入的剩余部分。这会导致不读取一个子结点的所有内容，其结果就像在LIMIT中所提到的。另外，如果 outer（第一个）子结点包含带有重复键值的行，inner（第二个）子结点会被倒退并且被重新扫描来找能匹配那个键值的行。EXPLAIN ANALYZE会统计相同 inner 行的重复发出，就好像它们是真实的额外行。当有很多 outer 重复时，对 inner 子计划结点所报告的实际行计数会显著地大于实际在 inner 关系中的行数。

由于实现的限制，BitmapAnd 和 BitmapOr 结点总是报告它们的实际行计数为零。

通常，EXPLAIN将显示规划器生成的每个计划节点。

但是，在某些情况下，执行器可以不执行某些节点，因为根据规划时不可用的参数值能确定这些节点无法产生任何行。（当前，这仅会在扫描分区表的Append或MergeAppend节点的子节点中发生。）发生这种情况时，将从EXPLAIN输出中省略这些计划节点，并显示Subplans Removed: N的标识。

规划器使用的统计信息

单列统计信息

如我们在上一节所见，查询规划器需要估计一个查询要检索的行数，这样才能对查询计划做出好的选择。本节对系统用于这些估计的统计信息进行一个快速的介绍。

统计信息的一个部分就是每个表和索引中的项的总数，以及每个表和索引占用的磁盘块数。这些信息保存在pg_class表的reltuples和relpages列中。我们可以用类似下面的查询查看这些信息：

```
SELECT relname, relkind, reltuples, relpages
FROM pg_class
WHERE relname LIKE 'tenk1%';
```

relname	relkind	reltuples	relpages
tenk1	r	10000	358
tenk1_hundred	i	10000	30
tenk1_thous_tenthous	i	10000	30
tenk1_unique1	i	10000	30
tenk1_unique2	i	10000	30
(5 rows)			

这里我们可以看到tenk1包含 10000 行，它的索引也有这么多行，但是索引远比表小得多（不奇怪）。

出于效率考虑，reltuples和relpages不是实时更新的

，因此它们通常包含有些过时的值。它们被VACUUM、ANALYZE和几个 DDL 命令（例如CREATE INDEX）更新。一个不扫描全表的VACUUM或ANALYZE操作（常见情况）将以它扫描的部分为基础增量更新reltuples计数，这就导致了一个近似值。在任何情况中，规划器将缩放它在pg_class中找到的值来匹配当前的物理表尺寸，这样得到一个较紧的近似。

大多数查询只是检索表中行的一部分，因为它们有限制要被检查的行的WHERE子句。

因此规划器需要估算WHERE子句的选择度，即符合WHERE子句中每个条件的行的比例。

用于这个任务的信息存储在pg_statistic系统目录中。在pg_statistic中的项由ANALYZE和VACUUM ANALYZE命令更新，并且总是近似值（即使刚刚更新完）。

除了直接查看pg_statistic之外，

手工检查统计信息的时候最好查看它的视图pg_stats。pg_stats被设计为更容易阅读。

而且，pg_stats是所有人都可以读取的，而pg_statistic只能由超级用户读取（这样可以避免非授权用户从统计信息中获取一些其他人的表的内容的信息。pg_stats视图被限制为只显示当前用户可读的表）。例如，我

们可以：

```
SELECT attname, inherited, n_distinct,
       array_to_string(most_common_vals, E'\n') as most_common_vals
FROM pg_stats
WHERE tablename = 'road';
```

attname	inherited	n_distinct	most_common_vals
name	f	-0.363388	I- 580 Ramp+
			I- 880 Ramp+
			Sp Railroad +
			I- 580 +
			I- 680 Ramp
name	t	-0.284859	I- 880 Ramp+
			I- 580 Ramp+
			I- 680 Ramp+
			I- 580 +
			State Hwy 13 Ramp
(2 rows)			

注意，这两行显示的是相同的列，一个对应开始于road表（inherited=t）的完全继承层次，另一个只包括road表本身（inherited=f）。

ANALYZE在pg_statistic中存储的信息量（特别是每个列的most_common_vals中的最大项数和histogram_bounds数组）可以用ALTER TABLE SET STATISTICS命令为每一列设置，或者通过设置配置变量default_statistics_target进行全局设置。目前的默认限制是 100 个项。提升该限制可能会让规划器做出更准确的估计（特别是对那些有不规则数据分布的列），其代价是在pg_statistic中消耗了更多空间，并且需要略微多一些的时间来计算估计数值。相比之下，比较低的限制可能更适合那些数据分布比较简单的列。

更多规划器对统计信息的使用可以参阅 [手册](#)。

扩展统计信息

常常可以看到由于查询子句中用到的多个列相互关联而运行着糟糕的执行计划的慢查询。规划器通常会假设多个条件是彼此独立的，这种假设在列值相互关联的情况下是不成立的。由于常规的统计信息天然的针对个体列的性质，它们无法捕捉到跨列关联的知识。不过，IvorySQL有能力计算多元统计信息，它能捕捉这类信息。

由于可能的列组合数非常巨大，所以不可能自动计算多元统计信息。可以创建扩展统计信息对象（更常被称为统计信息对象）来指示服务器获得跨感兴趣列集合的统计信息。

统计信息对象可以使用CREATE STATISTICS命令创建。这样一个对象的创建仅仅是创建了一个目录项来表示对统计信息有兴趣。实际的数据收集是由ANALYZE（或者是一个手工命令，或者是后台的自动分析）执行的。收集到的值可以在pg_statistic_ext_data目录中看到。

ANALYZE基于它用来计算常规单列统计信息的表行样本来计算扩展统计信息。由于样本的尺寸会随着表或者表列的统计信息目标（如前一节所述）增大而增加，更大的统计信息目标通常将会导致更准确的扩展统计信息，同时也会导致更多花在计算扩展统计信息之上的时间。

下面的小节介绍当前支持的扩展统计信息类型。

函数依赖

最简单的一类扩展统计信息跟踪函数依赖，这是在数据库范式定义中使用的概念。如果列a的值的知识足以决定列b的值，即不会有两个行具有相同的a值但是有不同的b值，我们就说列b函数依赖于列a。在一个完全规范化的数据库中，函数依赖应该仅存在于主键和超键上。不过，实际上很多数据集会因为各种原因无法被完全规范化，常见的例子是为了性能而有意地反规范化。即使在一个完全规范化的数据库中，也会有某些列之间的部分关联，这些可以表达成部分函数依赖。

函数依赖的存在直接影响了特定查询中估计的准确性。如果一个查询包含独立列和依赖列上的条件，依赖列上的条件不会进一步降低结果的尺寸。但是如果没有函数依赖的知识，查询规划器将假定条件是独立的，导致对结果尺寸的低估。

要告知规划器有关函数依赖的信息，ANALYZE可以收集跨列依赖的测度。评估所有列组之间的依赖程度可能会昂贵到不可实现，因此数据收集被限制为针对那些在一个统计信息对象中一起出现的列组（用dependencies选项定义）。建议只对强相关的列组创建dependencies统计信息，以避免ANALYZE以及后期查询规划中不必要的开销。

这里是一个收集函数依赖统计信息的例子：

```
CREATE STATISTICS stts (dependencies) ON city, zip FROM zipcodes;

ANALYZE zipcodes;

SELECT stxname, stxkeys, stxddependencies
FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid)
WHERE stxname = 'stts';
```

stxname	stxkeys	stxddependencies
stts	1 5	{"1 => 5": 1.000000, "5 => 1": 0.423130}

(1 row)

这里可以看到列1（邮编）完全决定列5（城市），因此系数为1.0，而城市仅决定42%的邮编，意味着有很多城市（58%）有多个邮编。

在为涉及函数依赖列的查询计算选择度时，规划器会使用依赖系数来调整针对条件的选择度估计，这样就不会产生低估。

函数依赖的限制

当前只有在考虑简单等值条件（将列与常量值比较）和具有常量值的IN子句时，函数依赖才适用。不会使用它们来改进比较两个列或者比较列和表达式的等值条件的估计，也不会用它们来改进范围子句、LIKE或者任何其他类型的条件。

在用函数依赖估计时，规划器假定在涉及的列上的条件是兼容的并且因此是冗余的。如果它们是不兼容的，正确的估计将是零行，但是那种可能性不会被考虑。例如，给定一个这样的查询

```
SELECT * FROM zipcodes WHERE city = 'San Francisco' AND zip = '94105';
```

规划器将会忽视city子句，因为它不改变选择度，这是正确的。不过，即便真地只有零行满足下面的查询，规划器也会做出同样的假设

```
SELECT * FROM zipcodes WHERE city = 'San Francisco' AND zip = '90210';
```

不过，函数依赖统计信息无法提供足够的信息来排除这种情况。

在很多实际情况中，这种假设通常是能满足的。例如，在应用程序中可能有一个GUI仅允许选择兼容的城市和邮编值用在查询中。但是如果不是这样，函数依赖可能就不是一个可行的选项。

多元可区分值计数

单列统计信息存储每一列中可区分值的数量。在组合多个列（例如GROUP BY a, b）时，如果规划器只有单列统计数据，则对可区分值数量的估计常常会错误，导致选择不好的计划。

为了改进这种估计，ANALYZE可以为列组收集可区分值统计信息。和以前一样，为每一种可能的列组合做这件事情是不切实际的，因此只会为一起出现在一个统计信息对象（用ndistinct选项定义）中的列组收集数据。将会为列组中列出的列的每一种可能的组合都收集数据。

继续之前的例子，ZIP代码表中的可区分值计数可能像这样：

```
CREATE STATISTICS stts2 (ndistinct) ON city, state, zip FROM zipcodes;
```

```
ANALYZE zipcodes;
```

```
SELECT stxkeys AS k, stxdndistinct AS nd
FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid)
WHERE stxname = 'stts2';
-[ RECORD 1 ]-----
k | 1 2 5
nd | {"1, 2": 33178, "1, 5": 33178, "2, 5": 27435, "1, 2, 5": 33178}
(1 row)
```

这表示有三种列组合有33178个可区分值：ZIP代码和州、ZIP代码和城市、ZIP代码+城市+州（事实上对于表中给定的一个唯一的ZIP代码，它们本来就应该是相等的）。另一方面，城市和州的组合只有27435个可区分值。

建议只对实际用于分组的列组合以及分组数错误估计导致了糟糕计划的列组合创建ndistinct统计信息对象。否则，ANALYZE循环只会被浪费。

多元MCV列表

为每列存储的另一种统计信息是频繁值列表。

这样可以对单个列进行非常准确的估计，但是对于在多个列上具有条件的查询，可能会导致严重的错误估计。

为了改善这种估计，ANALYZE可以收集列组合上的MCV列表。与功能依赖和n-distinct系数类似，对每种可能的列分组进行此操作都是不切实际的。

在这种情况下，甚至更是如此，因为MCV列表（与功能依赖性和n-distinct系数不同）存储了公共列值。因此，仅收集在使用mcv选项定义的统计对象中同时出现的那些列组的数据。

继续前面的示例，邮政编码表的MCV列表可能类似于以下内容（与更简单的统计信息不同，它需要一个函数来检查MCV内容）：

```
CREATE STATISTICS stts3 (mcv) ON city, state FROM zipcodes;
```

```
ANALYZE zipcodes;
```

```
SELECT m.* FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid),  
       pg_mcv_list_items(stxdmccv) m WHERE stxname = 'stts3';
```

index	values	nulls	frequency	base_frequency
0	{Washington, DC}	{f,f}	0.003467	2.7e-05
1	{Apo, AE}	{f,f}	0.003067	1.9e-05
2	{Houston, TX}	{f,f}	0.002167	0.000133
3	{El Paso, TX}	{f,f}	0.002	0.000113
4	{New York, NY}	{f,f}	0.001967	0.000114
5	{Atlanta, GA}	{f,f}	0.001633	3.3e-05
6	{Sacramento, CA}	{f,f}	0.001433	7.8e-05
7	{Miami, FL}	{f,f}	0.0014	6e-05
8	{Dallas, TX}	{f,f}	0.001367	8.8e-05
9	{Chicago, IL}	{f,f}	0.001333	5.1e-05
...				
(99 rows)				

这表明城市和最常见组合是华盛顿特区，实际频率（在样本中）约为0.35%。
组合的基本频率（根据简单的每列频率计算）仅为0.0027%，导致两个数量级的低估。

建议仅在实际上在条件中一起使用的列的组合上创建MCV统计对象，对于这些组合，错误估计组数会导致糟糕的执行计划。否则，只会浪费ANALYZE和规划时间。

用显示JOIN子句控制规划器

我们可以在一定程度上用显式JOIN语法控制查询规划器。要明白为什么需要它，我们首先需要一些背景知识。

在一个简单的连接查询中，例如：

```
SELECT * FROM a, b, c WHERE a.id = b.id AND b.ref = c.id;
```

规划器可以自由地按照任何顺序连接给定的表。例如，它可以生成一个使用WHERE条件a.id = b.id连接A到B的查询计划，然后用另外一个WHERE条件把C连接到这个连接表。或者它可以先连接B和C然后再连接A得到同样的结果。或者也可以连接A到C然后把结果与B连接。

不过这么做效率不好，因为必须生成完整的A和C

的迪卡尔积，而在WHERE子句中并没有可用条件来优化该连接（IvorySQL执行器中的所有连接都发生在两个输入表之间，所以它必须以这些形式之一建立结果）。

重要的一点是这些不同的连接可能性给出在语义等效的结果，但在执行开销上却可能有巨大的差别。

因此，规划器会对它们进行探索并尝试找出最高效的查询计划。

当一个查询只涉及两个或三个表时，那么不需要考虑很多连接顺序。但是可能的连接顺序数随着表数目的增加成指数增长。

当超过十个左右的表以后，实际上根本不可能对所有可能性做一次穷举搜索，甚至对六七个表都需要相当长的时间进行规划。

当有太多的输入表时，IvorySQL规划器将从穷举搜索切换为一种遗传概率搜索，它只需要考虑有限数量的可

能性（切换的阈值用geqo_threshold运行时参数设置）。遗传搜索用时更少，但是并不一定会找到最好的计划。

当查询涉及外连接时，规划器比处理普通（内）连接时拥有更小的自由度。例如，考虑：

```
SELECT * FROM a LEFT JOIN (b JOIN c ON (b.ref = c.id)) ON (a.id = b.id);
```

尽管这个查询的约束表面上和前一个非常相似，但它们的语义却不同，因为如果 A 里有任何一行不能匹配 B 和 C 的连接表中的行，它也必须被输出。因此这里规划器对连接顺序没有什么选择：它必须先连接 B 到 C，然后把 A 连接到该结果上。

相应地，这个查询比前面一个花在规划上的时间更少。在其它情况下，规划器就有可能确定多种连接顺序都是安全的。例如，给定：

```
SELECT * FROM a LEFT JOIN b ON (a.bid = b.id) LEFT JOIN c ON (a.cid = c.id);
```

将 A 首先连接到 B 或 C 都是有效的。当前，只有 FULL JOIN 完全约束连接顺序。大多数涉及 LEFT JOIN 或 RIGHT JOIN 的实际情况都在某种程度上可以被重新排列。

显式连接语法（INNER JOIN、CROSS JOIN 或无修饰的 JOIN）在语义上和 FROM 中列出输入关系是一样的，因此它不约束连接顺序。

即使大多数类型的 JOIN 并不完全约束连接顺序，但仍然可以指示 IvorySQL 查询规划器将所有 JOIN 子句当作有连接顺序约束来对待。例如，这里的三个查询在逻辑上是等效的：

```
SELECT * FROM a, b, c WHERE a.id = b.id AND b.ref = c.id;  
SELECT * FROM a CROSS JOIN b CROSS JOIN c WHERE a.id = b.id AND b.ref = c.id;  
SELECT * FROM a JOIN (b JOIN c ON (b.ref = c.id)) ON (a.id = b.id);
```

但如果我们告诉规划器遵循 JOIN 的顺序，那么第二个和第三个还是要比第一个花在规划上的时间少。这个效果对于只有三个表的连接而言是微不足道的，但对于数目众多的表，可能就是救命稻草了。

要强制规划器遵循显式 JOIN 的连接顺序，我们可以把运行时参数 join_collapse_limit 设置为 1（其它可能值在下文讨论）。

你不必为了缩短搜索时间来完全约束连接顺序，因为可以在一个普通 FROM 列表里使用 JOIN 操作符。例如，考虑：

```
SELECT * FROM a CROSS JOIN b, c, d, e WHERE ...;
```

如果设置 join_collapse_limit = 1，那么这就强迫规划器先把 A 连接到 B，然后再连接到其它的表上，但并不约束它的选择。在这个例子中，可能的连接顺序的数目减少了 5 倍。

按照这种方法约束规划器的搜索是一个有用的技巧，不管是对减少规划时间还是对引导规划器生成好的查询计划。如果规划器按照默认选择了一个糟糕的连接顺序，你可以通过 JOIN 语法强迫它选择一个更好的顺序——假设你知道一个更好的顺序。我们推荐进行实验。

一个非常相近的影响规划时间的问题是把子查询压缩到它们的父查询中。例如，考虑：

```
SELECT *  
FROM x, y,  
      (SELECT * FROM a, b, c WHERE something) AS ss
```

```
WHERE somethingelse;
```

这种情况可能在使用包含连接的视图时出现；该视图的SELECT规则将被插入到引用视图的地方，得到与上文非常相似的查询。通常，规划器会尝试把子查询压缩到父查询里，得到：

```
SELECT * FROM x, y, a, b, c WHERE something AND somethingelse;
```

这样通常会生成一个比独立的子查询更好些的计划（例如，outer的WHERE条件可能先把X连接到A上，这样就消除了A中的许多行，因此避免了形成子查询的全部逻辑输出）。但是同时，我们增加了规划的时间；在这里，我们用五路连接问题替代了两个独立的三路连接问题。这样的差别是巨大的，因为可能的计划数的是按照指数增长的。如果有超过from_collapse_limit个FROM项将会导致父查询，规划器将尝试通过停止提升子查询来避免卡在巨大的连接搜索问题中。你可以通过调高或调低这个运行时参数在规划时间和计划的质量之间取得平衡。

from_collapse_limit和join_collapse_limit的命名相似，因为它们做的几乎是同一件事：一个控制规划器何时把子查询“平面化”，另外一个控制何时把显式连接平面化。通常，你要么把join_collapse_limit设置成和from_collapse_limit一样（这样显式连接和子查询的行为类似），要么把join_collapse_limit设置为1（如果你想用显式连接控制连接顺序）。但是你可以把它们设置成不同的值，这样你就可以细粒度地调节规划时间和运行时间之间的平衡。

填充一个数据库

第一次填充数据库时可能需要插入大量的数据。本节包含一些如何让这个处理尽可能高效的建议。

禁用自动提交

在使用多个INSERT时，关闭自动提交并且只在最后做一次提交（在普通SQL中，这意味着在开始发出BEGIN并且在结束时发出COMMIT。某些客户端库可能背着你就做了这些，在这种情况下你需要确定在你需要做这些时该库确实帮你做了）。如果你允许每一个插入都被独立地提交，IvorySQL要为每一个被增加的行做很多工作。在一个事务中做所有插入的一个额外好处是：如果一个行的插入失败则所有之前插入的行都会被回滚，这样你不会被卡在部分载入的数据中。

使用COPY

使用COPY在一条命令中装载所有记录，而不是一系列INSERT命令。COPY命令是为装载大量行而优化过的；它没INSERT那么灵活，但是在大量数据装载时导致的负荷也更少。因为COPY是单条命令，因此使用这种方法填充表时无须关闭自动提交。

如果你不能使用COPY，那么使用PREPARE来创建一个预备INSERT语句也有所帮助，然后根据需要EXECUTE多次。这样就避免了重复分析和规划INSERT的负荷。不同接口以不同的方式提供该功能，可参阅接口文档中的“预备语句”。

请注意，在载入大量行时，使用COPY几乎总是比使用INSERT快，即使使用了PREPARE并且把多个插入被成批地放入一个单一事务。

同样的事务中，COPY比更早的CREATE TABLE或TRUNCATE命令更快。在这种情况下，不需要写WAL，因为在一个错误的情况下，包含新载入数据的文件不管怎样都将被移除。不过，只有当wal_level设置为minimal（此时所有的命令必须写WAL）时才会应用这种考虑。

移除索引

如果你正在载入一个新创建的表，最快的方法是创建该表，用COPY批量载入该表的数据，然后创建表需要的任何索引。在已存在数据的表上创建索引要比在每一行被载入时增量地更新它更快。

如果你正在对现有表增加大量的数据，删除索引、载入表然后重新创建索引可能是最好的方案。当然，在缺少索引的期间，其它数据库用户的数据库性能将会下降。我们在删除唯一索引之前还需要仔细考虑清楚，因为唯一约束提供的错误检查在缺少索引的时候会丢失。

移除外键约束

和索引一样，“成批地”检查外键约束比一行行检查效率更高。因此，先删除外键约束、载入数据然后重建约束会很有用。同样，载入数据和约束缺失期间错误检查的丢失之间也存在平衡。

更重要的是，当你在已有外键约束的情况下向表中载入数据时，每个新行需要一个在服务器的待处理触发器事件（因为是一个触发器的触发会检查行的外键约束）列表的条目。载入数百万行会导致触发器事件队列溢出可用内存，造成不能接受的交换或者甚至是命令的彻底失败。因此在载入大量数据时，可能需要（而不仅仅是期望）删除并重新应用外键。如果临时移除约束不可接受，那唯一的其他办法可能就是就是将载入操作分解成更小的事务。

增加 maintenance_work_mem

在载入大量数据时，临时增大maintenance_work_mem配置变量可以改进性能。这个参数也可以帮助加速CREATE INDEX命令和ALTER TABLE ADD FOREIGN KEY命令。它不会对COPY本身起很大作用，所以这个建议只有在你使用上面的一个或两个技巧时才有用。

增加max_wal_size

临时增大max_wal_size配置变量也可以让大量数据载入更快。这是因为向IvorySQL中载入大量的数据将导致检查点的发生比平常（由checkpoint_timeout配置变量指定）更频繁。无论何时发生一个检查点时，所有脏页都必须被刷写到磁盘上。通过在批量数据载入时临时增加max_wal_size，所需的检查点数目可以被缩减。

禁用WAL归档和流复制

当使用 WAL

归档或流复制向一个安装中载入大量数据时，在录入结束后执行一次新的基础备份比处理大量的增量 WAL 数据更快。为了防止载入时记录增量

WAL，通过将wal_level设置为minimal、将archive_mode设置为off以及将max_wal_senders设置为零来禁用归档和流复制。但需要注意的是，修改这些设置需要重启服务。

除了避免归档器或 WAL 发送者处理 WAL

数据的时间之外，这样做将实际上使某些命令更快，因为如果wal_level是minimal并且当前子事务（或顶级事务）创建或截断了它们更改的表或索引，则它们根本不编写 WAL。（通过在最后执行一个fsync而不是写 WAL，它们能以更小地代价保证崩溃安全）。

事后运行ANALYZE

不管什么时候你显著地改变了表中的数据分布后，我们都强烈推荐运行ANALYZE。这包括向表中批量载入大量数据。运行ANALYZE（或者VACUUM ANALYZE）保证规划器有表的最新统计信息。如果没有统计数据或者统计数据过时，那么规划器在查询规划时可能做出很差劲决定，导致在任意表上的性能低下。需要注意的是，如果启用了 autovacuum 守护进程，它可能会自动运行ANALYZE。

关于pg_dump的一些笔记

pg_dump生成的转储脚本自动应用上面的若干个（但不是全部）技巧。

要尽可能快地载入pg_dump转储，你需要手工做一些额外的事情（请注意，这些要点适用于恢复一个转储，而不是创建它的时候。同样的要点也适用于使用psql载入一个文本转储或用pg_restore从一个pg_dump归档文件载入）。

默认情况下，pg_dump使用COPY，并且当它在生成一个完整的模式和数据转储时，它会很小心地先装载数据，然后创建索引和外键。因此在这种情况下，一些指导方针是被自动处理的。你需要做的是：

- 为maintenance_work_mem和max_wal_size设置适当的（即比正常值大的）值。
- 如果使用 WAL 归档或流复制，在转储时考虑禁用它们。在载入转储之前，可通过将archive_mode设置为off、将wal_le

vel设置为minimal以及将max_wal_senders设置为零（在录入dump前）来实现禁用。之后，将它们设回正确的值并执行一次新的基础备份。

- 采用pg_dump和pg_restore的并行转储和恢复模式进行实验并且找出要使用的最佳并发任务数量。通过使用-j选项的并行转储和恢复应该能为你带来比串行模式高得多的性能。
- 考虑是否应该在一个单一事务中恢复整个转储。要这样做，将-l或—single-transaction命令行选项传递给psql或pg_restore。
当使用这种模式时，即使是一个很小的错误也会回滚整个恢复，可能会丢弃已经处理了很多个小时的工作。根据数据间的相关性，可能手动清理更好。如果你使用一个单一事务并且关闭了WAL归档，COPY命令将运行得最快。
- 如果在数据库服务器上有多个CPU可用，可以考虑使用pg_restore的—jobs选项。这允许并行数据载入和索引创建。
- 之后运行ANALYZE。

一个只涉及数据的转储仍将使用COPY，但是它不会删除或重建索引，并且它通常不会触碰外键。因此当载入一个只有数据的转储时，如果你希望使用那些技术，你需要负责删除并重建索引和外键。在载入数据时增加max_wal_size仍然有用，但是不要去增加maintenance_work_mem；不如说在以后手工重建索引和外键时你已经做了这些。并且不要忘记在完成后执行ANALYZE。

非持久设置

持久性是数据库的一个保证已提交事务的记录的特性（即使是发生服务器崩溃或断电）。然而，持久性会明显增加数据库的负荷，因此如果你的站点不需要这个保证，IvorySQL可以被配置成运行更快。在这种情况下，你可以调整下列配置来提高性能。除了下面列出的，在数据库软件崩溃的情况下也能保证持久性。当这些设置被使用时，只有突然的操作系统停止会产生数据丢失或损坏的风险。

- 将数据库集簇的数据目录放在一个内存支持的文件系统上（即RAM磁盘）。这消除了所有的数据库磁盘I/O,但将数据存储限制到可用的内存量（可能有交换区）。
- 关闭fsync；不需要将数据刷入磁盘。
- 关闭synchronous_commit；可能不需要在每次提交时强制把WAL写入磁盘。这种设置可能会在数据库崩溃时带来事务丢失的风险（但是没有数据破坏）。
- 关闭full_page_writes；不需要警惕部分页面写入。
- 增加max_wal_size和checkpoint_timeout；这会降低检查点的频率，但会增加/pg_wal的存储要求。
- 创建不做日志的表来避免WAL写入，不过这会让表在崩溃时不安全。

迁移指南

迁移概述

数据库迁移是指将数据从一个数据库转移到另一个数据库的过程，两端的数据库可能是PostgreSQL, IvorySQL, MySQL, Oracle, SQL Server等。迁移过程是一个具有挑战性的复杂过程，需要对数据库的原理以及各自的特性了如指掌。如果应用已经部署到生产环境并处于正常运行状态，为了保持业务不中断运行，并且不发生数据丢失，数据库迁移后需进行平滑的应用迁移。

迁移后数据库和系统应符合以下要求：

- 迁移后的数据库系统应完全承载原数据库系统的数据。避免迁移过程中数据丢失导致新的数据库系统数据不完整。
- 迁移后的数据库系统应完全适配原有数据库的功能。避免迁移后因数据类型或语法、函数的不支持，且无替代方案，导致整个业务系统的无法运行或抛错。
- 迁移后的数据库应适配整个业务系统的上下游，稳定可靠的保障整个业务系统的运行。
- 迁移后的数据库各方面综合性能不能弱于原数据库，为整个业务系统提供性能保证。

迁移工具Ora2Pg

Ora2Pg 是一个免费的工具，用于将 Oracle 数据库迁移到 IvorySQL 兼容的模式。它连接您的 Oracle 数据库，自动扫描并提取它的结构或数据，然后生成可以装载到 IvorySQL 数据库的 SQL 脚本。Ora2Pg 可以从逆向工程 Oracle 数据库到大型企业数据库迁移，或者简单地将一些 Oracle 数据复制到 IvorySQL 数据库中。它非常容易使用，并且不需要任何 Oracle 数据库知识，而不需要提供连接到Oracle数据库所需的参数。

Ora2Pg 由一个 Perl 脚本（ora2pg）以及一个 Perl 模块（[Ora2Pg.pm](#)）组成，唯一需要做的事情就是修改它的配置文件 ora2pg.conf，设置连接 Oracle 数据库的 DSN 和一个可选的 SCHEMA 名称。完成之后，只需要设置导出的类型：TABLE（包括约束和索引）、VIEW、MVVIEW、TABLESPACE、SEQUENCE、INDEXES、TRIGGER、GRANT、FUNCTION、PROCEDURE、PACKAGE、PARTITION、TYPE、INSERT 或 COPY、FDW、QUERY、KETTLER 以及 SYNONYM。

默认情况下，Ora2Pg 导出一个 SQL 文件，可以通过 IvorySQL 客户端工具 psql 执行导出的SQL文件。当进行数据迁移时，可以在配置文件中设置一个目标数据库的 DSN，直接将数据从Oracle导入到 IvorySQL 数据库中。

对象	ora2pg是否支持
view	是
trigger	是，某些情况下需要手工修改脚本
sequence	是
function	是
procedure	是，某些情况下需要手工修改脚本
type	是，某些情况下需要手工修改脚本
materialized view	是，某些情况下需要手工修改脚本

迁移Oracle数据库至IvorySQL

环境准备

linux环境	Oracle版本	IvorySQL版本
Centos Stream 9	19.0.0.0	4.2

依赖环境安装

安装Perl

```
[root@localhost /]# dnf install -y perl perl-ExtUtils-CBuilder perl-ExtUtils-MakeMaker
[root@localhost /]# perl -v

This is perl 5, version 16, subversion 3 (v5.16.3) built for x86_64-linux-thread-multi
(with 44 registered patches, see perl -V for more detail)

Copyright 1987-2012, Larry Wall
```

Perl may be copied only under the terms of either the Artistic License or the GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on this system using "man perl" or "perldoc perl". If you have access to the Internet, point your browser at <http://www.perl.org/>, the Perl Home Page.

安装DBI模块

DBI, Database Independent Interface, 是 Perl 语言连接数据库的接口

下载地址 <https://mirrors.sjtug.sjtu.edu.cn/cpan/modules/by-module/Plack/TIMB/DBI-1.643.tar.gz>

```
# tar zxvf DBI-1.643.tar.gz
# cd DBI-1.643/
# perl Makefile.PL
# make && make install
```

安装DBD-Oracle

下载地址: <https://sourceforge.net/projects/ora2pg/>

设置环境变量; 加载环境变量; 因为必须定义ORACLE_HOME环境变量; 本例在ivorysql用户下配置环境变量

```
export ORACLE_HOME=/opt/oracle/product/19c/dbhome_1
# tar -zxvf DBD-Oracle-1.76.tar.gz # source /home/postgres/.bashrc
# cd DBD-Oracle-1.76
# perl Makefile.PL
# make && make install
```

安装Ora2pg

下载地址: <https://sourceforge.net/projects/ora2pg/>

```
# tar -xjf ora2pg-24.0.tar.bz2
# cd ora2pg-24.0
# perl Makefile.PL
# make && make install
```

默认安装在/usr/local/bin/目录下

检查软件环境:

```
# vi check.pl
#!/usr/bin/perl
```

```

use strict;
use ExtUtils::Installed;
my $inst= ExtUtils::Installed->new();
my @modules = $inst->modules();
foreach(@modules)
{
    my $ver = $inst->version($_) || "???";
    printf("%-12s -- %s\n", $_, $ver);
}
exit;

```

```

# perl check.pl
DBD::Oracle  -- 1.76
DBD::Pg      -- 3.8.0
DBI          -- 1.643
Ora2Pg       -- 24.0
Perl         -- 5.16.3
设置环境变量

```

export PERL5LIB=<your_install_dir> #export PERL5LIB=/usr/local/bin/

=== 源端准备工作

更新oracle统计信息 提高性能

```

BEGIN DBMS_STATS.GATHER_SCHEMA_STATS('SH'); DBMS_STATS.GATHER_SCHEMA_STATS('SCOTT');
DBMS_STATS.GATHER_SCHEMA_STATS('HR'); DBMS_STATS.GATHER_DATABASE_STATS;
DBMS_STATS.GATHER_DICTIONARY_STATS; END;/

```

查询源端对象的类型

```

```bash
SYS@PROD1>set pagesize 200
SYS@PROD1>select distinct OBJECT_TYPE from dba_objects where OWNER in
('SH','SCOTT','HR') ;
OBJECT_TYPE

INDEX PARTITION
TABLE PARTITION
SEQUENCE
PROCEDURE

```

```

LOB X
TRIGGER
DIMENSION X
MATERIALIZED VIEW
TABLE
INDEX
VIEW
11 rows selected.

```

ora2pg导出表结构

配置ora2pg.conf:

默认情况下，Ora2Pg会查找/etc/ora2pg/ora2pg.conf配置文件，如果文件存在，您只需执行：/usr/local/bin/ora2pg

```

cat /etc/ora2pg/ora2pg.conf.dist | grep -v ^# |grep -v ^$ >ora2pg.conf
vi ora2pg.conf
cat ora2pg.conf
ORACLE_HOME /opt/oracle/product/19c/dbhome_1
ORACLE_DSN dbi:Oracle:host=localhost;sid=ORCLCDB;port=1521
ORACLE_USER system
ORACLE_PWD oracle
SCHEMA SH
EXPORT_SCHEMA 1 # 将用户导入到PostgreSQL数据库中
DISABLE_UNLOGGED 1 #避免将NOLOGGING属性设为UNLOGGED
SKIP fkeys ukeys checks #跳过外键 唯一 和检查约束
TYPE
TABLE,VIEW,GRANT,SEQUENCE,TABLESPACE,PROCEDURE,TRIGGER,FUNCTION,PACKAGE,PARTITION,TYPE
,MVIEW,QUERY,DBLINK,SYNONYM,DIRECTORY,TEST,TEST_VIEW
NLS_LANG AMERICAN_AMERICA.UTF8
OUTPUT sh.sql

```

1. 只能同时执行一种类型的导出，因此TYPE指令必须是唯一的。如果您有多个，则只会在文件中找到最后一个。但我测试就可以同时导出多个类型的。
2. 请注意，您可以通过向TYPE指令提供以逗号分隔的导出类型列表来链接多个导出，但在这种情况下，您不能将COPY或INSERT与其他导出类型一起使用。
3. 某些导出类型不能或不应该直接加载到 IvorySQL 数据库中，仍然需要很少的手动编辑。GRANT，TABLESPACE，TRIGGER，FUNCTION，PROCEDURE，TYPE，QUERY和PACKAGE导出类型就是这种情况，特别是如果您有PLSQL代码或Oracle特定SQL。
4. 对于TABLESPACE，您必须确保系统上存在文件路径，对于SYNONYM，您可以确保对象的所有者和模式对应于新的PostgreSQL数据库设计。
5. 建议导出表结构时，一个类型一个类型的操作，避免其它错误相互影响。

设置Oracle数据库DSN后，您可以执行ora2pg以查看它是否有效：

```
ora2pg -t SHOW_VERSION -c ora2pg.conf
```

```
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0
```

#### 迁移成本评估

估算从Oracle到PostgreSQL的迁移过程的成本并不容易。为了获得对此迁移成本的良好评估，Ora2Pg将检查所有数据库对象，所有函数和存储过程，以检测是否仍有一些对象和PL / SQL代码无法由Ora2Pg自动转换。

Ora2Pg具有内容分析模式，该模式检查Oracle数据库以生成有关Oracle数据库包含的内容和无法导出的内容的文本报告。

```
ora2pg -t SHOW_REPORT --estimate_cost -c ora2pg.conf
[=====>] 11/11 tables (100.0%) end of scanning.
[=====>] 11/11 objects types (100.0%) end of objects auditing.

Ora2Pg v24.0 - Database Migration Report

Version Oracle Database 19c Enterprise Edition Release 19.0.0.0.0
Schema SH
Size 287.25 MB

Object Number Invalid Estimated cost Comments Details

DATABASE LINK 0 0 0 Database links will be exported as SQL/MED
IvorySQL's Foreign Data Wrapper (FDW) extensions using oracle_fdw.
DIMENSION 5 0 0
GLOBAL TEMPORARY TABLE 0 0 0 Global temporary table are not
supported by PostgreSQL and will not be exported. You will have to rewrite some
application code to match the PostgreSQL temporary table behavior.
INDEX 20 0 3.4 14 index(es) are concerned by the export, others are
automatically generated and will do so on PostgreSQL. Bitmap will be exported as
btree_gin index(es) and hash index(es) will be exported as b-tree index(es) if any.
Domain index are exported as b-tree but commented to be edited to mainly use FTS.
Cluster, bitmap join and IOT indexes will not be exported at all. Reverse indexes are
not exported too, you may use a trigram-based index (see pg_trgm) or a reverse()
function based index and search. Use 'varchar_pattern_ops', 'text_pattern_ops' or
'bpchar_pattern_ops' operators in your indexes to improve search with the LIKE
operator respectively into varchar, text or char columns. 11 bitmap index(es). 1
domain index(es). 2 b-tree index(es).

INDEX PARTITION 196 0 0 Only local indexes partition are exported,
they are build on the column used for the partitioning.
```

JOB      0          0          0          Job are not exported. You may set external cron job with them.

MATERIALIZED VIEW      2          0          6          All materialized view will be exported as snapshot materialized views, they are only updated when fully refreshed.

SYNONYM 0          0          0          SYNONYMs will be exported as views. SYNONYMs do not exists with PostgreSQL but a common workaround is to use views or set the PostgreSQL search\_path in your session to access object outside the current schema.

TABLE    11          0          1.1      1 external table(s) will be exported as standard table. See EXTERNAL\_T0\_FDW configuration directive to export as file\_fdw foreign tables or use COPY in your code if you just want to load data from external files. Total number of rows: 1063384. Top 10 of tables sorted by number of rows:. sales has 918843 rows. costs has 82112 rows. customers has 55500 rows. supplementary\_demographics has 4500 rows. times has 1826 rows. promotions has 503 rows. products has 72 rows. countries has 23 rows. channels has 5 rows. sales\_transactions\_ext has 0 rows. Top 10 of largest tables:.

TABLE PARTITION 56          0          5.6      Partitions are exported using table inheritance and check constraint. Hash and Key partitions are not supported by PostgreSQL and will not be exported. 56 RANGE partitions..

VIEW    1          0          1          Views are fully supported but can use specific functions.

-----  
Total    291          0          17.10    17.10 cost migration units means approximatively 1 man-day(s). The migration unit was set to 5 minute(s)  
-----

Migration level : A-1  
-----

Migration levels:

A - Migration that might be run automatically

B - Migration with code rewrite and a human-days cost up to 5 days

C - Migration with code rewrite and a human-days cost above 5 days

Technical levels:

1 = trivial: no stored functions and no triggers

2 = easy: no stored functions but with triggers, no manual rewriting

3 = simple: stored functions and/or triggers, no manual rewriting

4 = manual: no stored functions but with triggers or views with code rewriting

5 = difficult: stored functions and/or triggers with code rewriting

-----

导出SH表构

```
ora2pg -c ora2pg.conf
[=====>] 11/11 tables (100.0%) end of scanning.

[=====>] 12/12 tables (100.0%) end of table export.

[=====>] 1/1 views (100.0%) end of output.

[=====>] 0/0 sequences (100.0%) end of output.

[=====>] 0/0 procedures (100.0%) end of procedures export.

[=====>] 0/0 triggers (100.0%) end of output.

[=====>] 0/0 functions (100.0%) end of functions export.

[=====>] 0/0 packages (100.0%) end of output.

[=====>] 56/56 partitions (100.0%) end of output.

[=====>] 0/0 types (100.0%) end of output.

[=====>] 2/2 materialized views (100.0%) end of output.
[=====>] 0/0 dblink (100.0%) end of output.

[=====>] 0/0 synonyms (100.0%) end of output.

[=====>] 2/2 directory (100.0%) end of output.

Fixing function calls in output files....
```



导出SH用户数据

配置ora2pg.conf的TYPE为COPY或INSERT

```
cp ora2pg.conf sh_data.conf

vi sh_data.conf

ORACLE_HOME /opt/oracle/product/19c/dbhome_1

ORACLE_DSN dbi:Oracle:host=localhost;sid=ORCLCDB;port=1521

ORACLE_USER system

ORACLE_PWD oracle

SCHEMA SH

EXPORT_SCHEMA 1

DISABLE_UNLOGGED 1

SKIP fkeys ukeys checks

TYPE COPY

NLS_LANG AMERICAN_AMERICA.UTF8

OUTPUT sh_data.sql
```

导出数据

```
ora2pg -c sh_data.conf

[=====>] 11/11 tables (100.0%) end of scanning.

[=====>] 5/5 rows (100.0%) Table CHANNELS (5 recs/sec)

[>] 5/1063384 total rows (0.0%) - (0 sec., avg: 5
recs/sec).

[>] 0/82112 rows (0.0%) Table COSTS_1995 (0 recs/sec)

[>] 5/1063384 total rows (0.0%) - (0 sec., avg: 5
```

recs/sec).

[> ] 0/82112 rows (0.0%) Table COSTS\_H1\_1997 (0 recs/sec)

[> ] 5/1063384 total rows (0.0%) - (0 sec., avg: 5  
recs/sec).

[> ] 0/82112 rows (0.0%) Table COSTS\_1996 (0 recs/sec)

[> ] 5/1063384 total rows (0.0%) - (0 sec., avg: 5  
recs/sec).

.....

[=====>] 4500/4500 rows (100.0%) Table SUPPLEMENTARY\_DEMOGRAPHICS  
(4500 recs/sec)

[=====>] 1061558/1063384 total rows (99.8%) - (45 sec., avg: 23590  
recs/sec).

[=====>] 1826/1826 rows (100.0%) Table TIMES (1826 recs/sec)

[=====>] 1063384/1063384 total rows (100.0%) - (45 sec., avg: 23630  
recs/sec).

[=====>] 1063384/1063384 rows (100.0%) on total estimated data (45  
sec., avg: 23630 recs/sec)

Fixing function calls in output files...

查看导出的文件:

```
[root@test01 ora2pg]# ls -lrt *.sql
```

```
-rw-r--r-- 1 root root 15716 Jul 2 21:21 TABLE_sh.sql
```

```
-rw-r--r-- 1 root root 858 Jul 2 21:21 VIEW_sh.sql
```

```
-rw-r--r-- 1 root root 2026 Jul 2 21:21 TABLESPACE_sh.sql
```

```
-rw-r--r-- 1 root root 345 Jul 2 21:21 SEQUENCE_sh.sql
```

```
-rw-r--r-- 1 root root 2382 Jul 2 21:21 GRANT_sh.sql
```

```

-rw-r--r-- 1 root root 344 Jul 2 21:21 TRIGGER_sh.sql
-rw-r--r-- 1 root root 346 Jul 2 21:21 PROCEDURE_sh.sql
-rw-r--r-- 1 root root 344 Jul 2 21:21 PACKAGE_sh.sql
-rw-r--r-- 1 root root 345 Jul 2 21:21 FUNCTION_sh.sql
-rw-r--r-- 1 root root 6771 Jul 2 21:21 PARTITION_sh.sql
-rw-r--r-- 1 root root 341 Jul 2 21:21 TYPE_sh.sql
-rw-r--r-- 1 root root 342 Jul 2 21:21 QUERY_sh.sql
-rw-r--r-- 1 root root 950 Jul 2 21:21 MVIEW_sh.sql
-rw-r--r-- 1 root root 344 Jul 2 21:21 SYNONYM_sh.sql
-rw-r--r-- 1 root root 926 Jul 2 21:21 DIRECTORY_sh.sql
-rw-r--r-- 1 root root 343 Jul 2 21:21 DBLINK_sh.sql
-rw-r--r-- 1 root root 55281235 Jul 2 17:11 sh_data.sql

```

以同样的方法分别导出HR，SCOTT用户数据。

在IvorySQL环境中创建orcl库

创建ORCL 数据库

```

su - ivorysql

Last login: Tue Jul 2 20:04:30 CST 2019 on pts/3

$ createdb orcl

$ psql

psql (17.2)

Type "help" for help.

```

```
ivorysql=# \l
```

```
 List of databases
 Name | Owner | Encoding | Collate | Ctype | ICU Locale | Locale Provider |
Access privileges
-----+-----+-----+-----+-----+-----+-----+
ivorysql | ivorysql | SQL_ASCII | C | C | | libc |
orcl | ivorysql | SQL_ASCII | C | C | | libc |
postgres | ivorysql | SQL_ASCII | C | C | | libc |
template0 | ivorysql | SQL_ASCII | C | C | | libc |
=c/ivorysql +
| | | | | | | |
ivorysql=CTc/ivorysql
template1 | ivorysql | SQL_ASCII | C | C | | libc |
=c/ivorysql +
| | | | | | | |
ivorysql=CTc/ivorysql
```

(5 rows)

```
ivorysql=#
```

创建SH,HR,SCOTT 用户:

```
$ psql orcl
```

```
psql (17.2)
```

```
Type "help" for help.
```

```
orcl=#
```

```
orcl=# create user sh with password 'sh';
```

```
CREATE ROLE
```

## 迁移门户

### 导入表结构

由于有物化视图，在TABLE\_sh.sql里包含了物化视图的索引，会创建失败。需先创建表，在创建物化视图，最后创建索引。

取消物化视图索引，后面单独创建：

```

CREATE INDEX fw_psc_s_mv_chan_bix ON fweek_pscat_sales_mv (channel_id);

CREATE INDEX fw_psc_s_mv_promo_bix ON fweek_pscat_sales_mv (promo_id);

CREATE INDEX fw_psc_s_mv_subcat_bix ON fweek_pscat_sales_mv (prod_subcategory);

CREATE INDEX fw_psc_s_mv_wd_bix ON fweek_pscat_sales_mv (week_ending_day);

CREATE TEXT SEARCH CONFIGURATION en (COPY = pg_catalog.english);
ALTER TEXT SEARCH CONFIGURATION en ALTER MAPPING FOR hword, hword_part, word WITH
unaccent, english_stem;

```

```

psql orcl -f tab.sql

ALTER TABLE PARTITION sh.sales OWNER TO sh;
COMMENT
COMMENT
COMMENT
COMMENT
COMMENT
COMMENT
COMMENT
COMMENT
ALTER TABLE
ALTER TABLE
ALTER TABLE
.....

```

给对象授权

```

cat psql orcl -f GRANT_sh.sql
CREATE USER SH WITH PASSWORD 'change_my_secret' LOGIN;
ALTER TABLE sh.fweek_pscat_sales_mv OWNER TO sh;
GRANT ALL ON sh.fweek_pscat_sales_mv TO sh;

```

导入物化视图结构

物化视图需要相关查询权限,所以导入权限,注意这里要跟上用户

```

$ psql orcl sh -f MVIEW_sh.sql
SELECT 0
SELECT 0
CREATE INDEX

```

```
CREATE INDEX
CREATE INDEX
CREATE INDEX
```

#### 导入视图

```
$ psql orcl -f VIEW_sh.sql
SET
SET
SET
CREATE VIEW
```

#### 导入分区表

```
$ psql orcl -f PARTITION_sh.sql
SET
SET
SET
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
.....
```

#### 导入数据

```
$ psql orcl -f sh_data.sql
SET
COPY 0
SET
COPY 0
SET
COPY 0
SET
COPY 0
SET
COPY 0
SET
COPY 0
SET
```

```
COPY 0
SET
COPY 4500
SET
COPY 1826
COMMIT
```

## 数据验证

源库,目标端抽取部份对象对比:

```
SYS@PROD1>select count(*) from sh.products;
COUNT(*)

 72
```

```
orcl=# select count(*) from sh.products;
count

 72
(1 row)
```

```
SYS@PROD1>select count(*) from sh.channels;

COUNT(*)

 5
```

```
orcl=# select count(*) from sh.channels;
count

 5

(1 row)
```

```
SYS@PROD1>select count(*) from sh.customers ;
```

```
COUNT(*)
```

```

```

```
55500
```

```
orcl=# select count(*) from sh.customers ;
```

```
count
```

```

```

```
55500
```

```
(1 row)
```

## 生成迁移模板

使用时，两个选项—project\_base和—

init\_project向Ora2Pg表明他必须创建一个项目模板，其中包含工作树，配置文件和从Oracle数据库导出所有对象的脚本。生成通用配置文件。

1.创建脚本export\_schema.sh以自动执行所有导出。2.创建脚本import\_all.sh以自动执行所有导入。例：

```
mkdir -p /ora2pg/migration
```

```
ora2pg --project_base /ora2pg/migration/ --init_project test_project
```

```
Creating project test_project.
```

```
/ora2pg/migration//test_project/
```

```
schema/
```

```
 dblinks/
```

```
 directories/
```

```
 functions/
```

```
 grants/
```

```
 mviews/
```

```
 packages/
```

```
 partitions/
```

```
 procedures/
```

```
 sequences/
```

```
 synonyms/
```

```
 tables/
```

```
 tablespaces/
```

```
 triggers/
```

```
 types/
```

```
 views/
```

```
sources/
```

```
 functions/
```

```
 mviews/
```

```
 packages/
```

```
 partitions/
```

```
 procedures/
```



```
triggers/
types/
views/
data/
config/
reports/
```

Generating generic configuration file

Creating script export\_schema.sh to automate all exports.

Creating script import\_all.sh to automate all imports.

# IvorySQL生态

## PostGIS

### 概述

IvorySQL原生100%兼容PostgreSQL,因此, PostGIS可以完美适配IvorySQL。

### 安装

根据开发环境,用户可从 [PostGIS安装](#) 页面选择适合自己的方式进行安装PostGIS安装。

### 源码安装

除PostGIS社区提供的安装方式以外, IvorySQL社区也提供了源码安装方式, 源码安装环境为 CentOS Stream 9(x86\_64)。



请确保环境中已安装了IvorySQL3.0及以上版本

- 安装依赖

```
dnf install -y gcc gcc-c++ libtiff libtiff-devel.x86_64 libcurl-devel.x86_64 libtool
libxml2-devel redhat-rpm-config clang llvm geos311 automake protobuf-c-devel
```

- 安装SQLITE

```
$ wget https://www.sqlite.org/2022/sqlite-autoconf-3400000.tar.gz
$ tar -xvf sqlite-autoconf-3400000.tar.gz
$ cd sqlite-autoconf-3400000
$ sed -n '1i\#define SQLITE_ENABLE_COLUMN_METADATA 1' sqlite3.c
$./configure --prefix=/usr/local/sqlite
$ make && make install
$ rm usr/bin/sqlite3 && ln -s /usr/local/sqlite/bin/sqlite3 /usr/bin/sqlite3
$ sqlite3 -version
$ export PKG_CONFIG_PATH=/usr/local/sqlite/lib/pkgconfig:$PKG_CONFIG_PATH
```

- 安装PROJ

```
$ wget https://download.osgeo.org/proj/proj-8.2.1.tar.gz
$ tar -xvf proj-8.2.1.tar.gz
$ cd proj-8.2.1
$./configure --prefix=/usr/local/proj-8.2.1
$ make && make install
```

- 安装GDAL

```
$ wget https://github.com/OSGeo/gdal/releases/download/v3.4.3/gdal-3.4.3.tar.gz
```

```
$ tar -xvf gdal-3.4.3.tar.gz
$ cd gdal-3.4.3
$ sh autogen.sh
$./configure --prefix=/usr/local/gdal-3.4.3 --with-proj=/usr/local/proj-8.2.1
$ make && make install
```

- 安装 GEOS

```
$ wget https://download.osgeo.org/geos/geos-3.9.2.tar.bz2
$ tar -xvf geos-3.9.2.tar.bz2
$ cd geos-3.9.2
$./configure --prefix=/usr/local/geos-3.9.2
$ make && make install
```

- 安装 Protobuf

```
$ wget https://plug-neomirror.rcac.purdue.edu/adelie/source/archive/protobuf-3.20.1/protobuf-3.20.1.tar.gz
$ tar -xvf protobuf-3.20.1.tar.gz
$ cd protobuf-3.20.1
$ sh autogen.sh
$./configure --prefix=/usr/local/protobuf-3.20.1
$ make && make install
$ export PROTOBUF_HOME=/usr/local/protobuf-3.20.1
$ export PATH=$PROTOBUF_HOME/bin:$PATH
$ export PKG_CONFIG_PATH=$PROTOBUF_HOME/lib/pkgconfig:$PKG_CONFIG_PATH
```

- 安装 Protobuf-c

```
$ wget --no-check-certificate https://sources.buildroot.net/protobuf-c/protobuf-c-1.4.1.tar.gz
$ tar -xvf protobuf-c-1.4.1.tar.gz
$ cd protobuf-c-1.4.1
$./configure --prefix=/usr/local/protobuf-c-1.4.1
$ make && make install
$ export PROTOBUFC_HOME=/usr/local/protobuf-c-1.4.1
$ export PATH=$PROTOBUFC_HOME/bin:$PATH
$ export PKG_CONFIG_PATH=$PROTOBUFC_HOME/lib:$PKG_CONFIG_PATH
```

- 安装 PostGIS

```
$ wget https://download.osgeo.org/postgis/source/postgis-3.4.0.tar.gz
$ tar -xvf postgis-3.4.0.tar.gz
```

```
$ cd postgis-3.4.0
$ sh autogen.sh
$./configure --with-geosconfig=/usr/local/geos-3.9.2/bin/geos-config --with
-projdir=/usr/local/proj-8.2.1 --with-gdalconfig=/usr/local/gdal-3.4.3/bin/gdal-config
--with-protobufdir=/usr/local/protobuf-c-1.4.1 --with
-pgconfig=/usr/local/ivorysql/ivorysql-4/bin/pg_config
$ make && make install
```



如出现PGXS报错, 请根据环境中IvorySQL安装路径, 修改—with-pgconfig的参数值。

## 创建Extension并确认PostGIS版本

psql 连接到数据库, 执行如下命令:

```
ivorysql=# CREATE extension postgis;
CREATE EXTENSION

ivorysql=# SELECT * FROM pg_available_extensions WHERE name = 'postgis';
 name | default_version | installed_version | comment
-----+-----+-----+-----
 postgis | 3.4.0 | 3.4.0 | PostGIS geometry and geography
spatial types and functions
(1 row)
```

## 使用

关于PostGIS的使用, 请参阅 [PostGIS3.4官方文档](#)

## pgvector

### 概述

向量数据库是生成式人工智能(GenAI)的关键组成部分。pgvector作为PostgreSQL的重要扩展, 不仅能够支持高达16000维度的向量计算, 还提供了强大的向量操作和索引功能, 使得PostgreSQL能够直接转化为高效的向量数据库。由于IvorySQL基于PostgreSQL研发, 这使得它具备了与pgvector扩展无缝集成的能力, 从而为用户提供了更广泛的数据处理和分析选项。在Oracle兼容模式下, pgvector扩展同样可用, 这为Oracle用户使用向量数据库提供了极大的便利, 使其能够轻松地迁移和管理数据, 实现更高效的业务操作。

### 原理介绍

IVFFLAT和HNSW是PGVector的两个索引算法

#### IVFFLAT

IVFFLAT的工作原理是将相似的向量聚类为区域, 并建立一个倒排索引, 将每个区域映射到其向量。这使得查询可以集中在数据的一个子集上, 从而实现快速搜索。通过调整列表和探针参数, ivfflat可以平衡数据集的速度和准确性, 使 PostgreSQL 有能力对复杂数据进行快速的语义相似性搜索。通过简单的查询, 应用程序可以在数百万个高维向量中找到与查询向量最近的邻居。对于自然语言处理、信息检索等, ivfflat 是一个比较好的解决方案 在建立 ivfflat 索引时, 你需要决定索引中包含多少个 list。每个 list 代表一个 "中心"; 这些中心通过 k-means

算法计算而来。一旦确定了所有中心，ivfflat 就会确定每个向量最靠近哪个中心，并将其添加到索引中。当需要查询向量数据时，你可以决定要检查多少个中心，这由 ivfflat.probes 参数决定。这就是 ANN 性能/召回率的结果：访问的中心越多，结果就越精确，但这是以牺牲性能为代价的。

## HNSW

HNSW (Hierarchical Navigating Small World)

是一种基于图的索引算法，它由多层的邻近图组成，因此称为分层的 NSW

方法。它会为一张图按规则建成多层导航图，并让越上层的图越稀疏，结点间的距离越远；越下层的图越稠密，结点间的距离越近。HNSW

算法是一种经典的空间换时间的算法，它的搜索质量和搜索速度都比较高，但是它的内存开销也比较大，因为不仅需要把所有的向量都存储在内存中。还需要维护一个图的结构，也同样需要存储。

## 安装



环境中已经安装了IvorySQL4.2及以上版本，安装路径为/usr/local/ivorysql/ivorysql-4

### 源码安装

- 设置PG\_CONFIG环境变量

```
export PG_CONFIG=/usr/local/ivorysql/ivorysql-4/bin/pg_config
```

- 拉取pg\_vector源码

```
git clone --branch v0.6.2 https://github.com/pgvector/pgvector.git
```

- 安装 pgvector

```
cd pgvector
```

```
sudo --preserve-env=PG_CONFIG make
sudo --preserve-env=PG_CONFIG make install
```

- 创建pgvector扩展

```
[ivorysql@localhost ivorysql-4]$ psql
psql (17.2)
Type "help" for help.
```

```
ivorysql=# create extension vector;
CREATE EXTENSION
```

至此，pgvector扩展安装已完成。更多用例，请参考 [pgvector文档](#)

## Oracle兼容性

在IvorySQL Oracle兼容模式下，pgvector扩展同样可以正确运行



建议用户使用1521端口进行测试， psql -p 1521

## 数据类型

```
ivorysql=# CREATE TABLE items5 (id bigserial PRIMARY KEY, name varchar2(20), num
number(20), embedding bit(3));
CREATE TABLE
ivorysql=# INSERT INTO items5 (name, num, embedding) VALUES ('1st oracle data',0,
'000'), ('2nd oracle data', 111, '111');
INSERT 0 2
ivorysql=# SELECT * FROM items5 ORDER BY bit_count(embedding # '101') LIMIT 5;
```

id	name	num	embedding
2	2nd oracle data	111	111
1	1st oracle data	0	000

## 匿名块

```
ivorysql=# declare
i vector(3) := '[1,2,3]';
begin
raise notice '%', i;
end;
ivorysql-# /
NOTICE: [1,2,3]
DO
```

## 存储过程 (PROCEDURE)

```
ivorysql=# CREATE OR REPLACE PROCEDURE ora_procedure()
AS
p vector(3) := '[4,5,6]';
begin
raise notice '%', p;
end;
/
CREATE PROCEDURE
ivorysql=# call ora_procedure();
NOTICE: [4,5,6]
CALL
```

```
ivorysql=# CREATE OR REPLACE FUNCTION AddVector(a vector(3), b vector(3))
RETURN vector(3)
IS
BEGIN
RETURN a + b;
END;
/
CREATE FUNCTION
ivorysql=# SELECT AddVector('[1,2,3]','[4,5,6]') FROM DUAL;
 addvector

 [5,7,9]
(1 row)
```

# IvorySQL架构设计

## 查询处理

### 双parser

双parser框架主要分为两部分，包括SQL端和服务器端编程语言。

#### SQL端词法语法分离

基本做法是新增一套兼容Oracle风格的语法和词法，在开启Oracle兼容的情况下，走Oracle风格的语法分析，生成相应的语法树。

具体方法：

在src/backend/下面，新建一个oracle\_parser目录，将src/backend/parser/下的scan.l和gram.y复制到该目录下，改名成ora\_gram.y和ora\_scan.l，添加Oracle风格的语法和此法分析代码，同时复制keywords.c到该目录下，用来存放自己的关键字。该oracle\_parser目录编译成一个动态库libparser\_oracle.so。当开启Oracle兼容的时候，配置文件ivorysql.conf被嵌入到postgresql.conf文件的末尾。配置文件ivorysql.conf中的shared\_preload\_libraries参数中添加“liboracle\_parser”，这样当数据库启动时能够自动载入liboracle\_parser动态库。

新增ora\_raw\_parser函数指针，当libparser\_oracle.so动态库被加载时，该动态库中的\_PG\_init()函数将oracle\_raw\_parser()函数的地址赋值给ora\_raw\_parser，\_PG\_fini()则在兼容模式切换时负责重置ora\_raw\_parser为空。

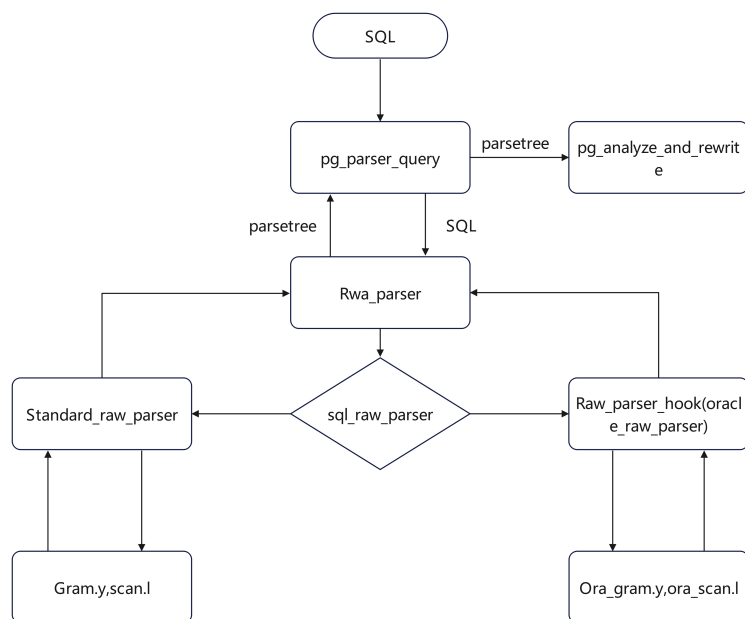
每一个后端进程调用BackendInitialize()函数，通过后端进程所连接的端口号来设置port→connmode。如果端口是Oracle兼容端口，则设置connmode为'o'，否则设置为'p'。

这之后PostgresMain()调用InitIvorysql()，如果port→connmode是'o'，则调用函数SetConfigOption("ivorysql.compatible\_mode", "oracle", PGC\_USERSET, PGC\_S\_OVERRIDE);由于这个参数设置了assign\_hook，因此当SetConfigOption()函数里执行assign\_hook()时，相当于调用assign\_compatible\_mode()，从而设置sql\_raw\_parser = ora\_raw\_parser;

在对SQL语句进行分析时，函数pg\_parse\_query()→raw\_parser()通过函数指针sql\_raw\_parser调用standard\_parser()或者ora\_raw\_parser()。

下面的图演示了SQL语句分析时发生的事情。





服务器端编程语言词法语法分离

在系统表pg\_language中新增pliSQL语言。

具体实现方法：

将PG源码中的plpgsql目录，复制一份，改名为plisql，里面的文件名称，改成plisql开头，因为plpgsql是一个language，改造的plisql也是一个language，因此，plpgsql language的注册函数如plpgsql\_validator, plpgsql\_call\_handler, plpgsql\_inline\_handler都需要改成plisql开头，里面其他函数的改名都改成以plisql开头。

plisql目录构建为一个插件，initdb时如果数据库模式是Oracle，则创建这个插件。这个插件会将pliSQL语言注册到数据库的系统表中。

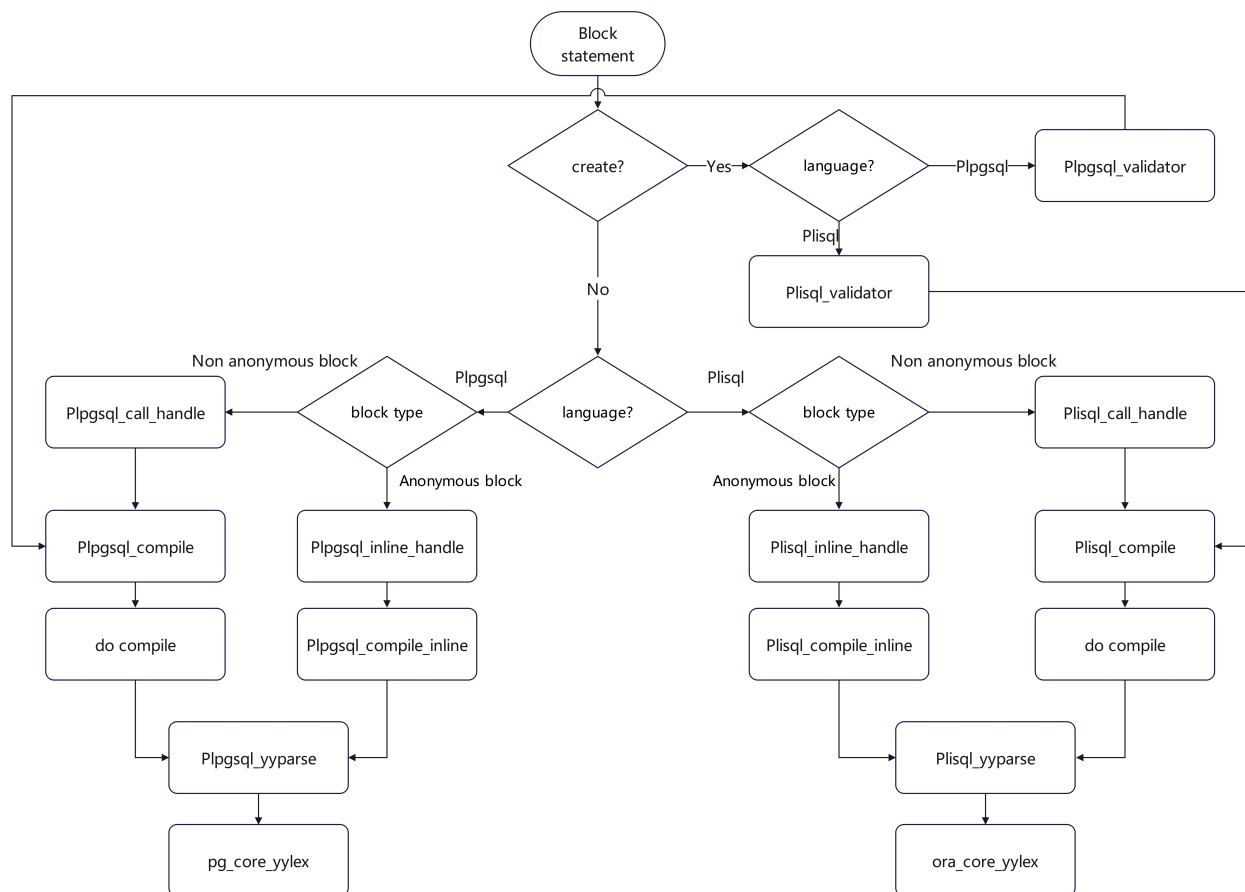
pliSQL端没有自己的词法规约，它依赖SQL端的词法规约，因此，针对pliSQL，强制其走兼容Oracle的词法规约，主要改造点是在 plisql\_scanner\_init 函数中，里面应该调用ora\_scanner\_init()函数，plisql目录中的 internal\_yylex() 函数调用 ora\_core\_yylex。

pliSQL语法规则在plisql/src/pl\_gram.y中，兼容Oracle pliSQL块的语法规则都在这个模块中实现。

SQL端创建函数，没有指定language的时候，如果当前是兼容Oracle模式，则默认language是plisql，如果是兼容pg模式，则默认language是plpgsql。oracle\_parser中的ora\_gram.y默认是plisql，pg parser中的gram.y默认是plpgsql。

匿名块没有指定language的时候，如果是兼容Oracle模式，默认plisql，如果是兼容pg模式，则默认plpgsql。

处理过程的ExecuteDoStmt函数同样根据兼容模式来决定默认language。



## 兼容框架

### initdb过程

IvorySQL 在初始化过程中支持两种数据库模式：

- PG 模式：保持与原生 PostgreSQL 的兼容性
- Oracle 模式（默认）：提供 Oracle 语法兼容及增强功能

用户可通过 initdb 命令参数指定初始化模式，实现不同场景下的兼容性需求。

### 参数解析处理

initdb在开始阶段会解析输入的命令行参数。

参数	功能描述	可选值	默认值
-m	指定数据库模式	oracle/pg	oracle
-C	设置 Oracle 兼容的大小写转换模式	interchange/normal/lowcase	interchange

参数解析流程：

- 1.继承 PostgreSQL 原有参数处理机制
- 2.新增模式选择参数 -m 的解析逻辑
- 3.增加大小写转换参数 -C 的处理模块

文件路径初始化

执行 `setup_data_file_paths()` 函数完成关键文件路径配置：

```
if (DB_PG == database_mode)
 set_input(&bki_file, "postgres.bki");
else
 set_input(&bki_file, "postgres_oracle.bki");
```

路径检查机制：

- 1.验证 BKI 文件的存在性（`postgres_oracle.bki` / `postgres.bki`）
- 2.确认配置文件模板的可用性
- 3.建立与数据库模式对应的系统目录结构

数据目录初始化流程

通过 `initialize_data_directory()` 函数完成核心初始化操作：

目录结构创建

调用 `create_data_directory()` 创建主数据目录（PGDATA）。

通过 `create_xlog_or_symlink()` 建立 WAL 日志目录。

循环创建 `base`, `global` 等标准子目录。

配置文件初始化

调用 `set_null_conf()` 创建空的 `postgresql.conf`。

Oracle 模式下额外创建 `ivorysql.conf` 配置文件。

调用 `setup_config()` 将配置信息写入 `postgresql.conf`。在写入配置时，如果是 `oracle` 模式，则会额外向 `ivorysql.conf` 中写入配置信息。

模板数据库引导

执行 `bootstrap_template1()` 加载对应模式的 BKI 文件初始化 `template1` 模板数据库，

IvorySQL 会额外设置 `template1` 模板数据库的数据库模式 (`oracle/pg`) 和大小写转换模式以。

`load_plsql()`：安装兼容 Oracle PL/SQL 的 PL/iSQL 过程语言

`load_ivorysql_ora()`：加载核心 Oracle 兼容层扩展

`make_ivorysql()`：创建默认的 `ivorysql` 数据库

Oracle 兼容用户名处理

当数据库模式是 `oracle` 时，会对用户名进行强制小写转换

### 提示

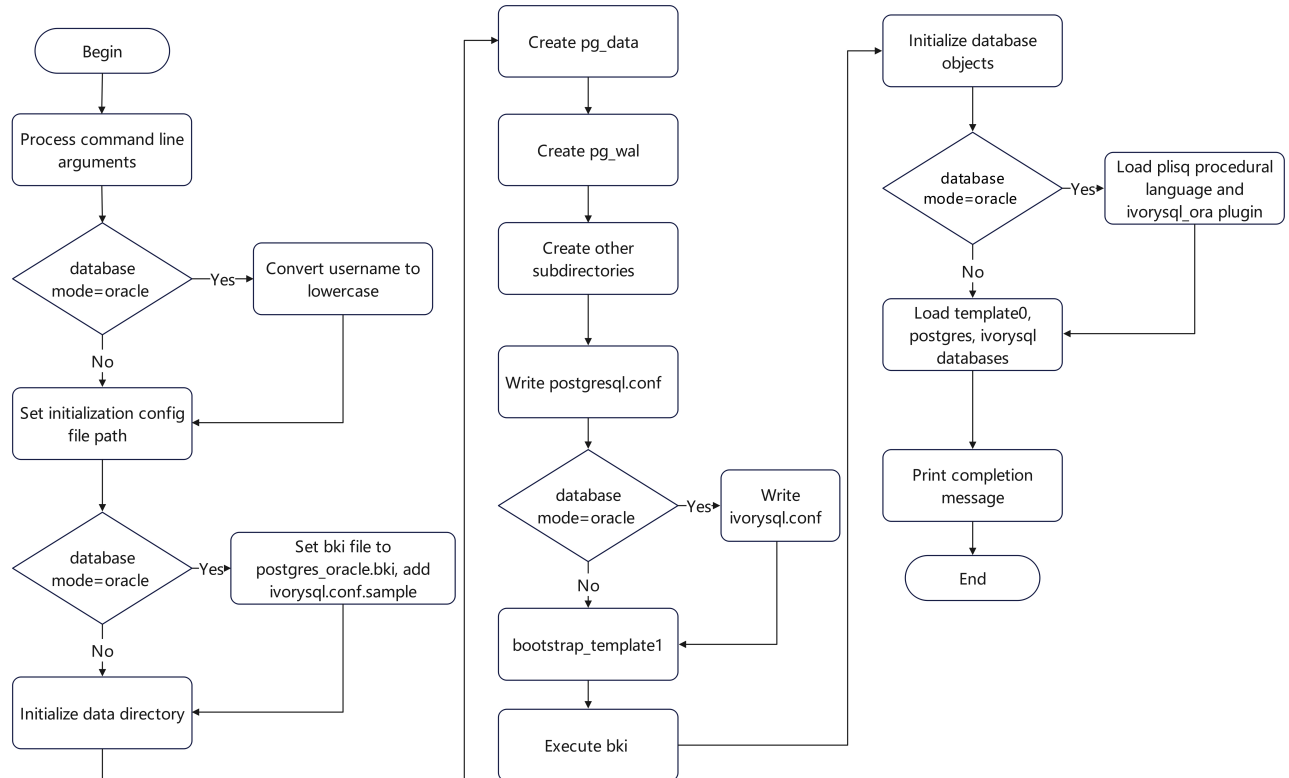
在 PostgreSQL 中，"bki" 是 "Bootstrap Kit" 的简写。Bootstrap Kit 是 PostgreSQL 内部用于引导（bootstrap）数据库系统的一组工具和文件集合。

Bootstrap Kit 主要用于初始化 PostgreSQL 数据库系统的基础结构，包括系统目录（system catalog）、系统表（system table）以及其他必要的系统对象。这些系统目录和表存储了 PostgreSQL 数据库系统的元数据信息，如数据库对象、表结构、索引信息等。

genbki.pl

处理系统表文件如pg\_class.h,pg\_namespace.h,pg\_proc.h等，并生成postgres.bki供initdb加载，主要是加载初始化数据，如创建系统表，系统函数和预定义类型等。

在initdb前的编译阶段，会通过genbki.pl生成postgres.bki 和 postgres\_ora.bki，postgres.bki保持PG原样，postgres\_ora.bki在pg的基础上增加oracle兼容的系统对象。

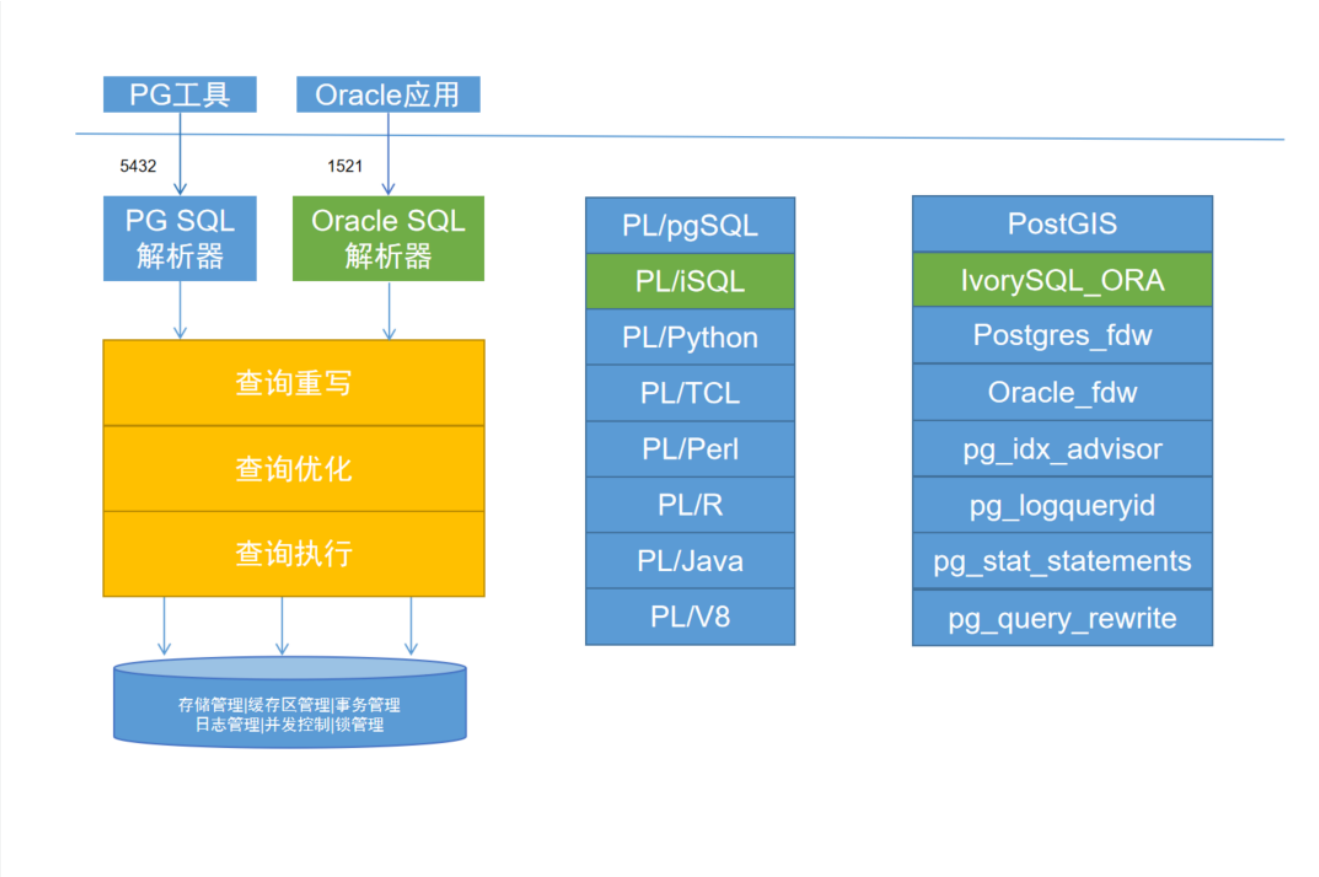


# Oracle兼容功能列表

## 1、 框架设计

### 目的

- 在对原有的PostgreSQL改动最小的前提下，实现对Oracle兼容。我们需要实现双parser、双端口、模式PL/PGSQL实现PL/iSQL的框架。实现流程图如下：



### 功能

#### 双端口设计

- 保持了Ivorysql 5432端口兼容原有postgres情况，因此Ivorysql采用另一个独立的端口登录，默认为1521。从该端口登录，默认采用Oracle兼容模式。如果需要从5432端口登录且还要进入兼容模式，则需要通过SET ivorysql.compatible\_mode=oracle进行设置。

#### parser模块设计

- 为了将Oracle语法与PostgreSQL语法间的相互干扰降到最低，因此新增parser模块，用于处理Oracle相关的语法。

#### 新增PL/iSQL过程语言

- 新增PL/iSQL过程语言兼容Oracle的PL/SQL过程语言。

## 2、 GUC框架

新增GUC变量

为了兼容Oracle，需要在原有的GUC变量基础之上增加一些用于控制数据库执行结果的变量，以达到和Oracle行为一致的目的。

为了以后更好的添加兼容的guc参数，以及为了更少的改动pg内核源码，我们需要设计一个框架将guc添加到一个统一的地方。

实现

添加兼容版本的guc参数时，我们需统一在 ivy\_guc.c 文件中添加。其中 **Ivy\_ConfigureNamesBool**、**Ivy\_ConfigureNamesInt**、**Ivy\_ConfigureNamesString**、**Ivy\_ConfigureNamesReal** 和 **Ivy\_ConfigureNamesEnum** 分别表示5种不同类型的guc参数，添加guc参数时，将guc的值添加到对应数组中即可。

新增变量（目前）

变量名	描述
ivorysql.compatible_mode	表示当前兼容哪种数据库（pg/oracle），可以通过show命令查看，set命令更改该变量，reset命令重置为连接时的数据库模式，reset all会影响该变量
ivorysql.database_mode	表示当前数据库的模式（pg/oracle），可以通过show命令查看，set/reset/reset all命令不影响该变量
ivorysql.datetime_ignore_nls_mask	表示日期格式是否会受NLS参数影响，默认为0，可以通过set命令设置，reset命令重置，reset all命令会重置该变量
ivorysql.enable_emptystring_to_NULL	取值为（on/off）,该变量为on时，会将插入的空字符串转成NULL值存储
ivorysql.identifier_case_switch	设置字符大小写转换模式
ivorysql.listen_address	表示兼容模式监听的地址，在初始化数据库时，从ivorysql.conf文件中读取该配置，在配置文件中修改该值，需要重启数据库生效，可以通过show命令查看
ivorysql.port	表示兼容模式下连接的端口号，在初始化数据库时，从ivorysql.conf文件中读取该配置，在配置文件中修改该值，需要重启数据库生效，可以通过show命令查看
nls_date_format	表示默认的日期格式，可以通过show命令查看，默认为‘YYYY-MM-DD’，可以通过set命令设置，可以通过reset命令重置回默认值，reset all命令会重置该变量
nls_length_semantics	兼容Oracle的同名参数，控制一个字符所占内存的大小
nls_timestamp_format	兼容Oracle的同名参数，控制带时间的日期格式
nls_timestamp_tz_format	兼容Oracle的同名参数，控制带时区的日期格式
shared_preload_libraries	在初始话数据库时，从ivorysql.conf文件中读取，可以通过show命令查看，在配置文件中修改该值，重启数据库生效。

部分变量使用示例

ivorysql.datetime\_ignore\_nls\_mask

该GUC变量的可选值为0-15

可选值	不经过nls格式化的类型
0	不屏蔽任何类型，所有时间格式均由nls格式化
1	date
2	timestamp

3	date、timestamp
4	timestamptz
5	date、timestamptz
6	timestamp、timestamptz
7	date、timestamp、timestamptz
8	timestamptz
9	date、timestamptz
10	timestamp、timestamptz
11	date、timestamp、timestamptz
12	timestamptz、timestamptz
13	date、timestamptz、timestamptz
14	timestamp、timestamptz、timestamptz
15	date、timestamp、timestamptz、timestamptz

- 使用方法(以date为例)

查看nls date 的格式及datetime\_ignore\_nls\_mask值

```

ivorysql=# set ivorysql.compatible_mode to oracle;
SET
ivorysql=# show nls_date_format;
nls_date_format

YYYY-MM-DD
(1 row)
ivorysql=# show ivorysql.datetime_ignore_nls_mask;
ivorysql.datetime_ignore_nls_mask

0
(1 row)

```

创建测试表

```

ivorysql=# create table test_nls_date(a int, created_at date);
CREATE TABLE

```

插入数据

```

ivorysql=# insert into test_nls_date values(1, '2024/04/05');
INSERT 0 1
ivorysql=# select * from test_nls_date;
a | created_at
---+-----

```

```
1 | 2024-04-05
(1 row)
```

修改nls\_date\_format

```
ivorysql=# set nls_date_format to 'yy-mm-dd';
SET
```

插入nls格式数据并查看，插入成功

```
ivorysql=# insert into test_nls_date values(2, '24/04/15');
INSERT 0 1
ivorysql=# select * from test_nls_date;
 a | created_at
---+-----
 1 | 24-04-05
 2 | 24-04-15
(2 rows)
```

将date类型改为不经过nls处理，插入相同格式数据，如改成 1 (3,5,7等都可以)，插入数据将报错，nls格式化将不对date的查询结果起效。

```
ivorysql=# set ivorysql.datetime_ignore_nls_mask to 1;
SET
ivorysql=# insert into test_nls_date values(3, '24/05/15');
ERROR: date/time field value out of range: "24/05/15"
LINE 1: insert into test_nls_date values(3, '24/05/15');
 ^
HINT: Perhaps you need a different "datestyle" setting.
ivorysql=# select * from test_nls_date;
 a | created_at
---+-----
 1 | 2024-04-05
 2 | 2024-04-15
(2 rows)
```

### 3、大小写转换

目的

- 为了满足PG和Oracle的引用标识符大小写兼容，ivorysql设计了三种引用标识符的大小写转换模式。通过guc参数“identifier\_case\_switch”选择转换模式；



## 功能

大小写转换的三种模式（默认为interchange）

- 如果 guc参数 “identifier\_case\_switch” 值为 “normal”

1). 保持双引号所引用的标识符中的字母大小写不变。

- 如果 guc参数 “identifier\_case\_switch” 值为 “interchange” :

1). 如果双引号所引用的标识符中的字母全部为大写，则将大写转换为小写。

2). 如果双引号所引用的标识符中的字母全部为小写，则将小写转换为大写。

3). 如果用双引号引起来的标识符中的字母是大小写混合的，则保持标识符不变。

- 如果 guc参数 “identifier\_case\_switch” 值为 “lowercase”

1). 如果双引号所引用的标识符中的字母全部为大写，则将大写转换为小写。

2). 如果用双引号引起来的标识符中的字母是大小写混合的，则保持标识符不变。

初始化数据库集簇时

- 在initdb程序中加入 -C选项设置大小写转换模式，-C对应的值为：

"normal" ----- "0"同义

"interchange" ----- "1"同义

"lowercase" ----- "2"同义

在初始化数据库集簇的过程中，将大小写转换模式保存到data目录的global/pg\_control文件中。

经典用例

normal

```
ivorysql=# SET ivorysql.compatible_mode to oracle;
```

SET

```
ivorysql=# SET ivorysql.enable_case_switch = true;
```

SET

```
ivorysql=# SET ivorysql.identifier_case_switch = normal;
```

SET

```
ivorysql=# CREATE TABLE "NORMAL_1"(c1 int, c2 int);
```

CREATE TABLE

```
ivorysql=# CREATE TABLE "Normal_2"(c1 int, c2 int);
```

CREATE TABLE

```
ivorysql=# CREATE TABLE "normal_3"(c1 int, c2 int);
```

CREATE TABLE

```
ivorysql=# select * from "NORMAL_1";
```

```
 c1 | c2
```

```
----+----
```

```
(0 rows)
```

```
ivorysql=# select * from "Normal_1";
```

```
ERROR: relation "Normal_1" does not exist
```

```
LINE 1: select * from "Normal_1";
```

```
ivorysql=# select * from "normal_1";
```

```
ERROR: relation "normal" does not exist
```

```
LINE 1: select * from "normal";
```

```
ivorysql=# select * from NORMAL_1;
```

```
ERROR: relation "normal_1" does not exist
```

```
LINE 1: select * from NORMAL_1;
```

```
ivorysql=# select * from "Normal_2";
```

```
 c1 | c2
```

```
----+----
```

```
(0 rows)
```

```
ivorysql=# select * from "NORMAL_2";
```

```
ERROR: relation "NORMAL_2" does not exist
```

```
LINE 1: select * from "NORMAL_2";
```

```
ivorysql=# select * from "normal_2";
ERROR: relation "normal_2" does not exist
LINE 1: select * from "normal_2";
```

```
ivorysql=# select * from Normal_2;
ERROR: relation "normal_2" does not exist
LINE 1: select * from Normal_2;
```

```
ivorysql=# select * from "normal_3";
 c1 | c2
----+----
(0 rows)
```

```
ivorysql=# select * from "NORMAL_3";
ERROR: relation "NORMAL_3" does not exist
LINE 1: select * from "NORMAL_3";
```

```
ivorysql=# select * from "Normal_3";
ERROR: relation "Normal_3" does not exist
LINE 1: select * from "Normal_3";
```

```
ivorysql=# drop table "NORMAL_1";
DROP TABLE
ivorysql=# drop table "Normal_2";
DROP TABLE
ivorysql=# drop table "normal_3";
DROP TABLE
```

interchange

```
ivorysql=# SET ivorysql.compatible_mode to oracle;
SET
```

```
ivorysql=# SET ivorysql.enable_case_switch = true;
SET
```

```
ivorysql=# SET ivorysql.identifier_case_switch = interchange;
SET
```

```
ivorysql=# CREATE TABLE "INTER_CHANGE_1"(c1 int, c2 int);
CREATE TABLE
```

```
ivorysql=# CREATE TABLE "Inter_Change_2"(c1 int, c2 int);
CREATE TABLE
```

```
ivorysql=# CREATE TABLE "inter_change_3"(c1 int, c2 int);
CREATE TABLE
```

```
ivorysql=# select * from "INTER_CHANGE_1";
 c1 | c2
----+----
(0 rows)
```

```
ivorysql=# select * from "Inter_Change_1";
ERROR: relation "Inter_Change_1" does not exist
LINE 1: select * from "Inter_Change_1";
```

```
ivorysql=# select * from "inter_change_1";
ERROR: relation "INTER_CHANGE_1" does not exist
LINE 1: select * from "inter_change_1";
```

```
ivorysql=# select * from INTER_CHANGE_1;
 c1 | c2
----+----
(0 rows)
```

```
ivorysql=# select * from "Inter_Change_2";
 c1 | c2
----+----
(0 rows)
```

```
ivorysql=# select * from "INTER_CHANGE_2";
ERROR: relation "inter_change_2" does not exist
LINE 1: select * from "INTER_CHANGE_2";
```

```
ivorysql=# select * from "inter_change_2";
ERROR: relation "INTER_CHANGE_2" does not exist
LINE 1: select * from "inter_change_2";
```

```
ivorysql=# select * from Inter_Change_2;
ERROR: relation "inter_change_2" does not exist
LINE 1: select * from Inter_Change_2;
```

```

ivorysql=# select * from "inter_change_3";
 c1 | c2
----+----
(0 rows)

ivorysql=# select * from "INTER_CHANGE_3";
ERROR: relation "inter_change_3" does not exist
LINE 1: select * from "INTER_CHANGE_3";

ivorysql=# select * from "Inter_Change_3";
ERROR: relation "Inter_Change_3" does not exist
LINE 1: select * from "Inter_Change_3";

ivorysql=# select * from inter_change_3;
ERROR: relation "inter_change_3" does not exist
LINE 1: select * from "INTER_CHANGE_3";

ivorysql=# drop table "INTER_CHANGE_1";
DROP TABLE
ivorysql=# drop table "Inter_Change_2";
DROP TABLE
ivorysql=# drop table "inter_change_3";
DROP TABLE

```

lowercase

```

ivorysql=# SET ivorysql.compatible_mode to oracle;
SET

ivorysql=# SET ivorysql.enable_case_switch = true;
SET

ivorysql=# SET ivorysql.identifier_case_switch = lowercase;
SET

ivorysql=# CREATE TABLE "LOWER_CASE_1"(c1 int, c2 int);
CREATE TABLE

ivorysql=# CREATE TABLE "Lower_Case_2"(c1 int, c2 int);
CREATE TABLE

ivorysql=# CREATE TABLE "lower_case_3"(c1 int, c2 int);

```

## CREATE TABLE

```
ivorysql=# select * from "LOWER_CASE_1";
```

```
 c1 | c2
```

```
----+----
```

```
(0 rows)
```

```
ivorysql=# select * from "Lower_Case_1";
```

```
ERROR: relation "Lower_Case_1" does not exist
```

```
LINE 1: select * from "Lower_Case_1";
```

```
ivorysql=# select * from "lower_case_1";
```

```
 c1 | c2
```

```
----+----
```

```
(0 行记录)
```

```
ivorysql=# select * from LOWER_CASE_1;
```

```
 c1 | c2
```

```
----+----
```

```
(0 行记录)
```

```
ivorysql=# select * from "Lower_Case_2";
```

```
 c1 | c2
```

```
----+----
```

```
(0 rows)
```

```
ivorysql=# select * from "LOWER_CASE_2";
```

```
ERROR: relation "lower_case_2" does not exist
```

```
LINE 1: select * from "LOWER_CASE_2";
```

```
ivorysql=# select * from "lower_case_2";
```

```
ERROR: relation "lower_case_2" does not exist
```

```
LINE 1: select * from "lower_case_2";
```

```
ivorysql=# select * from Lower_Case_2;
```

```
ERROR: relation "lower_case_2" does not exist
```

```
LINE 1: select * from Lower_Case_2;
```

```
ivorysql=# select * from "lower_case_3";
```

```
 c1 | c2
```

```

----+----
(0 rows)

ivorysql=# select * from "LOWER_CASE_3";
 c1 | c2
----+----
(0 rows)

ivorysql=# select * from "Lower_Case_3";
ERROR: relation "Lower_Case_3" does not exist
LINE 1: select * from "Lower_Case_3";

ivorysql=# select * from LOWER_CASE_3;
 c1 | c2
----+----
(0 行记录)

ivorysql=# drop table "NORMAL_1";
DROP TABLE
ivorysql=# drop table "Normal_2";
DROP TABLE
ivorysql=# drop table "normal_3";
DROP TABLE

```

## 4、双模式设计

### 目的

- 为了满足PG模式和兼容Oracle模式，IvorySQL在initdb时，可以指定-m参数，来获取PG模式数据库或Oracle兼容模式数据库。



1. 不指定-m参数时，默认为Oracle兼容模式
2. 指定-m参数为pg时，数据库将不再兼容Oracle语法

### 功能

- Initdb -m 初始化，需要判断不同的模式，其中Oracle模式下，需要执行postgres\_oracle.bki的SQL语句；
- 启动时会根据初始化模式，判断是否为oracle兼容模式。

**database\_mode：**用于表示初始化模式；  
**database\_mode=DB\_PG，**PG模式，且不可切换；  
**database\_mode=DB\_ORACLE，**Oracle兼容模式；

```
初始化PG模式：
./initdb -D ../data -m pg

初始化oracle兼容模式：
./initdb -D ../data -m oracle
或
./initdb -D ../data
```

5、兼容Oracle like

目的

- 本文档意为为使用 like 模糊查询的人员提供一个深入了解兼容Oracle 的模糊查询like实现的过程，是like兼容的实现文档。

功能说明

数据库名称	like模糊查询
oracle	oracle的字符串类型是varchar2，支持对数字、日期、字符串字段类型的列用Like关键字配合通配符来实现模糊查询
IvorySQL	IvorySQL的字符串基本类型是text，所以like是以text为基础，其他IvorySQL的类型能隐式转换成text，这样不用创建opeartor就能自动转换

测试用例

```
create table t_ora_like (id int ,str1 varchar(8), date1 timestamp with time zone,
date2 time with time zone, num int, str2 varchar(8));
insert into t_ora_like (id ,str1 ,date1 ,date2) values (123456,'test1','2022-09-26
16:39:20','2022-09-26 16:39:20');
insert into t_ora_like (id ,str1 ,date1 ,date2) values (123457,'test2','2022-09-26
16:40:20','2022-09-26 16:40:20');
insert into t_ora_like (id ,str1 ,date1 ,date2) values (223456,'test3','2022-09-26
16:41:20','2022-09-26 16:41:20');
insert into t_ora_like (id ,str1 ,date1 ,date2) values (123458,'test4','2022-09-26
16:42:20','2022-09-26 16:42:20');

select * from t_ora_like where str1 like 'test%';
 id | str1 | date1 | date2 | num | str2
-----+-----+-----+-----+-----+-----+-----
123456 | test1 | 2022-09-26 16:39:20.000000 +08:00 | 16:39:20+08 | |
123457 | test2 | 2022-09-26 16:40:20.000000 +08:00 | 16:40:20+08 | |
223456 | test3 | 2022-09-26 16:41:20.000000 +08:00 | 16:41:20+08 | |
123458 | test4 | 2022-09-26 16:42:20.000000 +08:00 | 16:42:20+08 | |
```



(4 rows)

```
select * from t_ora_like where date1 like '2022%';
```

id	str1	date1	date2	num	str2
123456	test1	2022-09-26 16:39:20.000000 +08:00	16:39:20+08		
123457	test2	2022-09-26 16:40:20.000000 +08:00	16:40:20+08		
223456	test3	2022-09-26 16:41:20.000000 +08:00	16:41:20+08		
123458	test4	2022-09-26 16:42:20.000000 +08:00	16:42:20+08		

(4 rows)

```
select * from t_ora_like where date2 like '16%';
```

id	str1	date1	date2	num	str2
123456	test1	2022-09-26 16:39:20.000000 +08:00	16:39:20+08		
123457	test2	2022-09-26 16:40:20.000000 +08:00	16:40:20+08		
223456	test3	2022-09-26 16:41:20.000000 +08:00	16:41:20+08		
123458	test4	2022-09-26 16:42:20.000000 +08:00	16:42:20+08		

(4 rows)

```
select * from t_ora_like where id like '123%';
```

id	str1	date1	date2	num	str2
123456	test1	2022-09-26 16:39:20.000000 +08:00	16:39:20+08		
123457	test2	2022-09-26 16:40:20.000000 +08:00	16:40:20+08		
123458	test4	2022-09-26 16:42:20.000000 +08:00	16:42:20+08		

(3 rows)

```
select * from t_ora_like where id like null;
```

id	str1	date1	date2	num	str2
----	------	-------	-------	-----	------

(0 rows)

## 6、兼容Oracle匿名块

### 目的

- 本文档是PLSQL匿名块（anonymous block）兼容Oracle语法功能的设计文档，目的是可以在IvorySQL中兼容Oracle的匿名块语句。

### 功能说明

- 匿名块是能够动态地创建和执行过程代码的PLSQL结构，而不需要以持久化的方式将代码作为数据库对象储存在系统目录中。本次实现中IvorySQL主要兼容的是PLSQL匿名块的语法格式，我们主要处理的部分包括客户端工具psql、主服务器和PSQL端支持。

```
declare
i integer := 10;
begin
 raise notice '%', i;
end;
/
NOTICE: 10
```

```
DECLARE
 grade CHAR(1);
BEGIN
 grade := 'B';
 CASE grade
 WHEN 'A' THEN raise notice 'Excellent';
 WHEN 'B' THEN raise notice 'Very Good';
 END CASE;
EXCEPTION
 WHEN CASE_NOT_FOUND THEN
 raise notice 'No such grade';
END;
/
NOTICE: Very Good
```

## 7、兼容Oracle函数与存储过程

### 目的

- 本文档意在兼容Oracle PLSQL函数和存储过程的语法，在IvorySQL中我们称其为PLISQL语言。

### 功能说明

#### 函数（FUNCTION）

CREATE FUNCTION语法支持EDITIONABLE/NONEDITIONABLE
CREATE FUNCTION语法支持RETURN, IS关键字，不指定language
CREATE FUNCTION语法函数没有参数，函数名后面不带()
CREATE FUNCTION参数个数最多是32767
CREATE FUNCTION语法中END; 在psql中以/结束
CREATE FUNCTION语法变量声明前面没有DECLARE关键字
CREATE FUNCTION语法支持OUT 参数NOCOPY功能
CREATE FUNCTION语法支持sharing_clause
CREATE FUNCTION语法支持invoker_rights_clause，默认权限改成DR（DEFINER）

CREATE FUNCTION语法支持ACCESSIBLE BY
CREATE FUNCTION语法支持DEFAULT COLLATION
CREATE FUNCTION语法支持result_cache_clause
CREATE FUNCTION语法支持aggregate_clause
CREATE FUNCTION语法支持pipelined_clause
CREATE FUNCTION语法支持sql_macro_clause
ALTER FUNCTION语法
函数和存储过程相关的视图

## 存储过程（PROCEDURE）

CREATE PROCEDURE语法支持EDITIONABLE/NONEDITIONABLE
CREATE PROCEDURE语法函数没有参数，函数名后面不带()
CREATE PROCEDURE参数个数最多是32767
CREATE PROCEDURE语法中END; 在psql中以/结束
CREATE PROCEDURE语法支持sharing_clause
CREATE PROCEDURE语法支持DEFAULT COLLATION
CREATE PROCEDURE语法支持invoker_rights_clause
CREATE PROCEDURE语法支持ACCESSIBLE BY
ALTER PROCEDURE语法
存储过程没有参数，调用支持不带（）
存储过程调用支持EXEC
在PL/SQL 中调用存储过程，可以省略CALL，直接使用存储过程名字
支持—和/**/两种注释方法

## 测试用例

```
CREATE or replace FUNCTION ora_func RETURN integer AS
BEGIN
 RETURN 1;
END;
/

CREATE OR REPLACE FUNCTION test_nocopy(a IN int, b OUT NOCOPY int, c IN OUT NOCOPY
int)
RETURN record
IS
BEGIN
 b := a;
 c := a;
END;
/
```

```
CREATE OR REPLACE PROCEDURE ora_procedure()
AS
 p integer := 20;
begin
 raise notice '%', p;
end;
/
call ora_procedure();
```

```
CREATE OR REPLACE PROCEDURE ora_procedure
SHARING = METADATA
DEFAULT COLLATION USING_NLS_COMP
AUTHID CURRENT_USER
ACCESSIBLE BY (FUNCTION A.B)
IS
 p integer := 20;
begin
 raise notice '%', p;
end;
/
```

## 8、内置数据类型与内置函数

### 内置数据类型

char
varchar
varchar2
number
binary_float
binary_double
date
timestamp
timestamp with time zone
timestamp with local time zone
interval year to month
interval day to second
raw
long

### 内置函数类型

sysdate
---------

systimestamp
add_months
last_day
next_day
months_between
current_date
current_timestamp
new_time
tz_offset
trunc
instrb
substr
substrb
trim
ltrim
rtrim
length
lengthb
rawtohex
replace
regexp_replace
regexp_substr
regexp_instr
regexp_like
to_number
to_char
to_date
to_timestamp
to_timestamp_tz
to_ymininterval
to_dsinterval
numtodsinterval
numtoyminterval
localtimestamp
from_tz
sys_extract_utc
sessiontimezone
hextoraw
uid
USERENV
asciistr
to_multi_byte

to_single_byte
compose
decompose

## 内置函数说明

1、兼容sysdate函数，功能：查看对应的日期与时间，测试用例如下： 查询当前系统的日期：

```
select sysdate() from dual;
 sysdate

2023-07-06
(1 row)
```

查询往前推1天的日期：

```
select sysdate()-1 from dual;
 ?column?

2023-07-05
(1 row)
```

2、兼容systimestamp函数，功能：返回本机数据库上当前系统日期和时间（包括微秒和时区），测试用例如下： 查询当前日期的日期和时间：

```
select systimestamp() from dual;
 systimestamp

2023-07-06 10:18:31.674322 +08:00
(1 row)
```

3、兼容add\_months函数，功能：函数将一个月数(n)添加一个日期，并返回相隔n月的同一天，支持参数：date, number；测试用例如下： 查询当前日期（七月六日）的下个月的同一天：

```
select add_months(sysdate(),1) from dual;
 add_months

2023-08-06
(1 row)
```

查询当前日期的上个月的天一天：

```
select add_months(sysdate(),-1) from dual;
 add_months
```

```

2023-06-06
(1 row)
```

4、兼容last\_day函数，功能：返回指定日期所在月份的最后一天，支持参数：date,测试用例如下：  
查询当天所在月份的最后一天：

```
select last_day(sysdate())from dual;
last_day

2023-07-31
(1 row)
```

查询某一天所在月份的最后一天：

```
select last_day(to_date('2019-09-01'))from dual;
last_day

2019-09-30
(1 row)
```

5、兼容next\_day函数，功能：返回指定日期的下一个日期。支持参数:date, integer /date ,text,  
说明：当函数中第二个参数传的星期数比现有星期数小时，会返回下一个星期的日期；当函数中第二个参数  
所传的日期比现有星期数大，会返回本周相应星期日期。测试用例如下： 查询当前日期的下一天：

```
select next_day(sysdate(),1) from dual;
next_day

2023-07-07
(1 row)
```

查询当前日期的下个星期五：

```
select next_day(sysdate(),'FRIDAY') from dual;
next_day

2023-07-07
(1 row)
```

6、兼容months\_between函数，功能：返回日期类型的date1和date2之间相差的月份，支持参数：date,date,  
说明：如果date1晚于date2，返回正数；如果date1早于date2返回负数；如果date1和date2是某月里的  
同一天，返回结果为整数；如果不是同一天，会在每月31天的基础上返回带有小数部分的结果。测试用例如  
下： 查询不同月份同一天之间相差的月份：

```
select months_between(to_date('2023-07-06'),to_date('2023-08-06')) from dual;
```

```
months_between

-1
(1 row)
```

查询不同月份不同日期之间相差的月份：

```
select months_between(to_date('2023-07-06'),to_date('2023-08-05')) from dual;
months_between

-0.967741935483871
(1 row)
```

7、兼容current\_date函数，功能：返回当前时区的当前日期，测试用例如下： 查询当前时区的当前日期：

```
select current_date from dual;
current_date

2023-07-06
(1 row)
```

8、兼容current\_timestamp函数，功能：返回当前时区的当前日期与当前时间，包含当前时区信息。支持参数：integer, 说明：返回的时间可调整精度。测试用例如下： 查询当前时区的当前日期与时间：

```
select current_timestamp from dual;
current_timestamp

2023-07-06 10:27:01.440600 +08:00
(1 row)
```

查询当前时区的当前日期与时间(精度调整为前三位小数)：

```
select current_timestamp(3) from dual;
current_timestamp

2023-07-06 10:27:14.182000 +08:00
(1 row)
```

9、兼容new\_time函数，功能：返回某个时间在某时区所对应的在另一个时区的日期，支持参数：date, text, text,测试用例如下： 返回当前日期在另一个时区对应的日期：

```
select sysdate() bj_time,new_time(sysdate(),'PDT','GMT')los_angles from dual;
bj_time | los_angles
```



```
-----+-----
2023-07-06 | 2023-07-06
(1 row)
```

10、兼容tz\_offset函数，功能：返回给定时区与标准时区的偏移量，支持参数：text,测试用例如下：  
返回给定时区与标准时区偏移量：

```
select tz_offset('US/Eastern') from dual;
tz_offset

-04:00
(1 row)
```

11、兼容trunc函数，功能：可以截取日期，得到想要的数值，如年，月，日，时，分，支持参数：date/date,text,测试用例如下： 截取当前日期：

```
select trunc(sysdate()) from dual;
trunc

2023-07-06
(1 row)
```

截取年，返回值只有年是正确的，月和日不是准确值：

```
select trunc(sysdate(),'yyyy') from dual;
trunc

2023-01-01
(1 row)
```

截取月，返回值只有月是正确的，年和日不是准确值：

```
select trunc(sysdate(),'mm') from dual;
trunc

2023-07-01
(1 row)
```

12、兼容instrb函数，功能：字符串查找函数，返回字符串的位置，支持参数： varchar2, text, number  
DEFAULT 1,number DEFAULT 1,以下为测试用例： 返回CORPORATE  
FLOOR中默认第一次出现OR时字符串的位置：

```
SELECT INSTRB('CORPORATE FLOOR','OR') "Instring in bytes" FROM DUAL;
Instring in bytes
```

```

 2
(1 row)
```

返回corporate floor中从第五个字符开始查询，第二次出现or时字符串的位置：

```
SELECT INSTRB('CORPORATE FLOOR','OR',5,2) "Instring in bytes" FROM DUAL;
Instring in bytes

 14
(1 row)
```

13、兼容substr函数，功能：截取字符串函数，以字符为单位截取，支持参数：text, integer, 测试用例如下：截取‘今天天气很好’中从第五个字符开始，往后的字符串：

```
SELECT SUBSTR('今天天气很好',5) "Substring with bytes" FROM DUAL;

Substring with bytes

很好
(1 row)
```

14、兼容substrb函数，功能：截取字符串函数，以字节为单位截取，支持参数：varchar2, number/varchar2, number, number,测试用例如下：截取‘今天天气很好’中从第五个字节开始，往后的字符串：

```
SELECT SUBSTRB('今天天气很好',5) "Substring with bytes" FROM DUAL;
Substring with bytes

天气很好
(1 row)
```

截取‘今天天气很好’中从第五个字节开始，第八个字节结束的字符串：

```
SELECT SUBSTRB('今天天气很好',5, 8) "Substring with bytes" FROM DUAL;
Substring with bytes

天气
(1 row)
```

15、兼容trim函数，功能：去除指定字符串的左右空格或对应数据，支持参数：varchar2 /varchar2, varchar2, 测试用例如下：去除'aaa bbb ccc'的左右空格：

```
select trim('aaa bbb ccc')trim from dual;
```

```
trim

aaa bbb ccc
(1 row)
```

去除' aaa bbb ccc' 中的aaa:

```
select trim('aaa bbb ccc','aaa')trim from dual;
trim

bbb ccc
(1 row)
```

16、兼容ltrim函数，功能：去除指定字符串的左侧空格或对应数据，支持参数：varchar2 /varchar2,varchar2,测试用例如下：去除' abcdefg'的左侧空格：

```
select ltrim(' abcdefg ')ltrim from dual;
ltrim

abcdefg
(1 row)
```

从' abcdefg' 左侧开始遍历，一旦存在某字符出现在' fegab' 中就去除，不存在则返回结果：

```
select ltrim('abcdefg','fegab')ltrim from dual;
ltrim

cdefg
(1 row)
```

17、兼容rtrim函数，功能：去除指定字符串的右侧空格，测试用例如下：去除' abcdefg'的右侧空格：

```
select rtrim(' abcdefg ')rtrim from dual;
rtrim

abcdefg
(1 row)
```

从' abcdefg' 右侧开始遍历，一旦存在某字符出现在' fegab' 中就去除，不存在则返回结果：

```
select rtrim('abcdefg','fegab')rtrim from dual;
rtrim

```

```
abcd
(1 row)
```

18、兼容length函数，功能：求取指定字符串字符的长度，支持参数：char/integer/varchar2测试用例如下：  
查询223的字符长度：

```
select length(223) from dual;
length

 3
(1 row)
```

查询'223' 的字符长度：

```
select length('223') from dual;
length

 3
(1 row)
```

查询' ivorysql数据库' 的字符长度：

```
select length('ivorysql数据库') from dual;
length

 11
(1 row)
```

19、兼容lengthb功能：求取指定字符串字节的长度，支持参数：char/bytea/varchar2测试用例如下：  
查询' ivorysql' 的字节长度：

```
select lengthb('ivorysq'::char) from dual;
lengthb

 1
(1 row)
```

查询'0x2C' 的字节长度：

```
select lengthb('0x2C'::bytea) from dual;
lengthb

 4
```

(1 row)

查询' ivorysql数据库' 的字节长度:

```
select lengthb('ivorysql数据库'::varchar2) from dual;
lengthb

 17
(1 row)
```

20、兼容replace函数，功能：替换指定字符串中的字符或删除字符，支持参数：text, text, text/varchar2, varchar2, varchar2 DEFAULT NULL::varchar2, 测试用例如下：替换' jack and jue' 中的' j' 为' bl':

```
select replace('jack and jue','j','bl') from dual;
replace

black and blue
(1 row)
```

删除' jack and jue' 中的' j':

```
select replace('jack and jue','j') from dual;
replace

ack and ue
(1 row)
```

21、兼容regexp\_replace函数，此函数为replace函数的扩展。功能：用于通过正则表达式来进行匹配替换。支持参数：text, text, text /text, text, text, integer/varchar2, varchar2/varchar2, varchar2 varchar2, 测试用例如下：将匹配到的数字替换为\*#:

```
select regexp_replace('01234abcd56789','[0-9]','*#')from dual;
regexp_replace

##*#*#abcd*#*#*#*#
(1 row)
```

从第二个数开始将匹配到的数字替换为\*#:

```
select regexp_replace('01234abcd56789','[0-9]','*#',2)from dual;
regexp_replace

0*#*#*#*#abcd*#*#*#*#
```

删除'01234abcd56789' 中的'01':

```
select regexp_replace('01234abcd56789','01')from dual;
 regexp_replace

234abcd56789
(1 row)
```

用'xxx' 替换01234abcd56789' 中的012:

```
select regexp_replace('01234abcd56789','012','xxx')from dual;
 regexp_replace

xxx34abcd56789
(1 row)
```

22、兼容regexp\_substr函数，功能：拾取符合正则表达式描述的字符子串，支持参数：text, text, integer /text, text, integer, integer/ text, text, integer, integer, text /varchar2, varchar2,测试用例如下：  
查询'012ab34' 中从第一个数开始的012字串：

```
select regexp_substr('012ab34', '012',1) from dual;
 regexp_substr

012
(1 row)
```

查询'012ab34' 中从第一个数第一组开始的012字串：

```
select regexp_substr('012ab34', '012',1,1) from dual;
 regexp_substr

012
(1 row)
```

查询'012a012Ab34' 中从第一个数第一组开始不区分大小写的012字串：

```
select regexp_substr('012a012Ab34', '012A',1,1,'i') from dual;
 regexp_substr

012a
(1 row)
```

查询'012a012Ab34' 中从第一个数第一组开始区分大小写的012字串：

```
select regexp_substr('012a012Ab34', '012A',1,1,'c') from dual;
regexp_substr

012A
(1 row)
```

查询'数据库'中'数据'子串：

```
select regexp_substr('数据库', '数据') from dual;
regexp_substr

数据
(1 row)
```

23、兼容regexp\_instr函数，功能：用于标定符合正则表达式的字符子串的开始位置，支持参数：text, text, integer /text, text, integer, integer/ text, text, integer, integer, text/text, text, integer, integer, text, integer/ varchar2, varchar2,测试用例如下：查询'abcaBcabC'中从第一个字符开始，出现abc子串的位置：

```
SELECT regexp_instr('abcaBcabC', 'abc', 1);
regexp_instr

1
(1 row)
```

查询'abcaBcabC'中从第一个字符开始，第三次出现abc子串的位置：

```
SELECT regexp_instr('abcaBcabC', 'abc', 1, 3);
regexp_instr

7
(1 row)
```

查询'abcaBcabC'中从第一个字符开始，第二次出现abc子串后发生的位置：

```
SELECT regexp_instr('abcaBcabC', 'abc', 1, 2,1);
regexp_instr

7
(1 row)
```

查询'abcaBcabC'中从第一个字符开始，第一次出现abc子串后发生的位置（区分大小写）：

```
SELECT regexp_instr('abcaBcabC', 'abc',1,2,1,'c');
```

```
regexp_instr
```

```

```

```
7
(1 row)
```

查询'数据库'中'库'出现的位置:

```
SELECT regexp_instr('数据库', '库');
```

```
regexp_instr
```

```

```

```
3
(1 row)
```

24、兼容regexp\_like函数, 功能: 与like类似, 用于模糊查询。支持参数: varchar2, varchar2 /varchar2, varchar2 varchar2, 首先创建一个regexp\_like表用于测试用例查询:

```
create table t_regexp_like
```

```
(
```

```
id varchar(4),
```

```
value varchar(10)
```

```
);
```

```
insert into t_regexp_like values ('1','1234560');
```

```
insert into t_regexp_like values ('2','1234560');
```

```
insert into t_regexp_like values ('3','1b3b560');
```

```
insert into t_regexp_like values ('4','abc');
```

```
insert into t_regexp_like values ('5','abcde');
```

```
insert into t_regexp_like values ('6','ADREasx');
```

```
insert into t_regexp_like values ('7','123 45');
```

```
insert into t_regexp_like values ('8','adc de');
```

```
insert into t_regexp_like values ('9','adc,.de');
```

```
insert into t_regexp_like values ('10','abcbvbnb');
```

```
insert into t_regexp_like values ('11','11114560');
```

测试用例如下: 查询t\_regexp\_like表中带有abc的列:

```
select * from t_regexp_like where regexp_like(value,'abc');
```

```
id | value
```

```
---+-----
```

```
4 | abc
```

```
5 | abcde
```

```
10 | abcbvbnb
```

```
(3 rows)
```



查询t\_regexp\_like表中带有ABC的列(不区分大小写):

```
select * from t_regexp_like where regexp_like(value,'ABC','i');
id | value
----+-----
4 | abc
5 | abcde
10 | abcbvbnb
(3 rows)
```

25、兼容to\_number函数，功能：是将一些处理过的按一定格式编排过的字符串变回数值型的格式，支持参数：text/text,text测试用例如下：将字符串'-34,338,492'转换为数值型格式：

```
SELECT to_number('34,338,492', '99,999,999') from dual;
to_number

-34338492
(1 row)
```

将字符串'5.01-'转换为数值型格式：

```
SELECT to_number('5.01-', '9.99S');

to_number

-5.01
(1 row)
```

26、兼容to\_char函数，功能：将数字或日期转换为字符类型，支持参数：date/date,text/timestamp/timestamp,text测试用例如下：将当前系统日期转换为字符格式：

```
select to_char(sysdate()) from dual;
to_char

2023-07-10
(1 row)
```

将当前系统日期转换为月份/日期/年字符格式：

```
select to_char(sysdate(), 'mm/dd/yyyy') from dual;
to_char

07/10/2023
```

(1 row)

将当前日期的timestamp格式转换为字符格式：

```
SELECT to_char(sysdate()::timestamp);
 to_char

2023-07-10 09:46:44.000000
```

将当前日期的timestamp格式转换为月份/日期/年字符格式：

```
SELECT to_char(sysdate()::timestamp,'MM-YYYY-DD');
 to_char

07-2023-10
(1 row)
```

27、兼容to\_date函数，功能：将字符类型转换为日期类型，支持参数：text/text,text测试用例如下：  
将'2023/07/06' 转换为日期类型：

```
select to_date('20230706') from dual;
 to_date

2023-07-06
(1 row)
```

将'-44-02-01' 转换为日期类型：

```
SELECT to_date('-44,0201','YYYY-MM-DD');
 to_date

0044-02-01
(1 row)
```

28、兼容to\_timestamp函数，功能：可以存储年、月、日、小时、分钟、秒，同时还可以存储秒的小数部分。  
支持参数：text/text,text测试用例如下： 查询'2018-11-02 12:34:56.025' 以日期形式输出：

```
SELECT to_timestamp('20181102.12.34.56.025');
 to_timestamp

2018-11-02 12:34:56.025000
(1 row)
```

查询' 2011,12,18 11:38' 以日期形式输出:

```
SELECT to_timestamp('2011,12,18 11:38 ', 'YYYY-MM-DD HH24:MI:SS');
 to_timestamp

2011-12-18 11:38:00.000000
(1 row)
```

29、兼容to\_timestamp\_tz函数,功能: 根据时间查询, 时间字符串有T,Z并有毫秒, 时区。测试用例如下:  
查询'2016-10-9 14:10:10.123000' 以日期形式输出:

```
SELECT to_timestamp_tz('2016-10-9 14:10:10.123000') FROM DUAL;
 to_timestamp_tz

2016-10-09 14:10:10.123000 +08:00
(1 row)
```

查询'10-9-2016 14:10:10.123000 +8:30' 以日期形式输出:

```
SELECT to_timestamp_tz('10-9-2016 14:10:10.123000 +8:30', 'DD-MM-YYYY HH24:MI:SS.FF
TZH:TZM') FROM DUAL;
 to_timestamp_tz

2016-09-10 13:40:10.123000 +08:00
(1 row)
```

30、兼容to\_yinterval函数, 功能: 将一个字符串类型转化为年和月的时间差类型, 支持参数:text, 测试用例如下: 查询'20110101' 以后两个年零八个月后的日期:

```
select to_date('20110101','yyyymmdd')+to_yinterval('02-08') from dual;
 ?column?

2013-09-01
(1 row)
```

31、兼容to\_dsinterval函数, 功能: 将一个日期加上一定的小时或者天数变成另外一个日期,支持参数:text,测试用例如下: 查询当前系统时间加上9个半小时后的日期 (当前为2023-07-06, 18: 00) :

```
select sysdate()+to_dsinterval('0 09:30:00')as newdate from dual;
 newdate

2023-07-07
(1 row)
```

32、兼容numtodsinterval函数，功能：将数字转换成时间间隔类型的数据。支持参数:double precision, text测试用例如下： 转换100.00个小时为时间间隔类型数据：

```
SELECT NUMTODSINTERVAL(100.00, 'hour');
 numtodsinterval

+0000000004 04:00:00.000000000
(1 row)
```

转换100分钟为时间间隔类型数据：

```
SELECT NUMTODSINTERVAL(100, 'minute');
 numtodsinterval

+0000000000 01:40:00.000000000
(1 row)
```

33、兼容numtoyminterval函数，功能：将数字转换成日期间隔类型的数据。支持参数： double precision,text,测试用例如下： 转换1.00，year为日期间隔：

```
SELECT NUMTOYMINTERVAL(1.00, 'year');
 numtoyminterval

+0000000001-00
(1 row)
```

转换1，month为日期间隔：

```
SELECT NUMTOYMINTERVAL(1, 'month');
 numtoyminterval

+0000000000-01
(1 row)
```

34、兼容localtimestamp函数，功能：返回会话中的日期和时间，支持参数： integer, 函数中增加参数为精度，测试用例如下： 返回当前会话中的日期和时间：

```
select localtimestamp from dual;
 localtimestamp

2023-07-07 09:18:15.896472
(1 row)
```

返回当前会话中的日期和时间（精度为1）：

```
select localtimestamp(1) from dual;
 localtimestamp

2023-07-07 09:18:16.100000
(1 row)
```

35、兼容from\_tz函数，功能：将时间从一个时区转换为另一个时区，支持参数:timestamp, text ,测试用例如下： 将'2000-03-28 08:00:00', '3:00' 转换为当前时区：

```
SELECT FROM_TZ(TIMESTAMP '2000-03-28 08:00:00', '3:00') FROM DUAL;
 from_tz

2000-03-28 13:00:00.000000 +08:00
(1 row)
```

36、兼容sys\_extract\_utc函数，功能：将一个timestampz转换为UTC时区时间。支持参数:timestamp with time zone 测试用例如下： 查询转换timestamp '2000-03-28 11:30:00.00 -8:00' 为UTC时区后的时间：

```
select sys_extract_utc(timestamp '2000-03-28 11:30:00.00 -8:00') from dual;
 sys_extract_utc

2000-03-28 19:30:00.000000
(1 row)
```

37、兼容sessiontimezone函数，功能：查看时区详细信息，测试用例如下： 查看当前时区的详细信息：

```
select sessiontimezone() from dual;
 sessiontimezone

Asia/Shanghai
(1 row)
```

修改timezone后，查看时区相信信息：

```
set timezone = 'Asia/Hong_Kong';
SET
select sessiontimezone() from dual;
 sessiontimezone

Asia/Hong_Kong
(1 row)
```

38、兼容hextoraw函数，功能：将字符串表示的二进制数值转换为一个raw数值。支持参数： text,测试用例

如下：将字符串'abcdef'转换为raw数值：

```
select hextoraw('abcdef')from dual;
 hextoraw

 \xabcdef
(1 row)
```

39、兼容uid函数，功能：获取数据库的实例名。测试用例如下：获取当前数据库的实例名：

```
select uid() from dual;
 uid

 10
(1 row)
```

40、兼容USERENV函数，功能：返回当前用户环境的信息，测试用例如下：  
查看当前用户是否是dba，如果是返回ture：

```
select userenv('isdba')from dual;
 get_isdba

 TRUE
(1 row)
```

查看会话标志：

```
select userenv('sessionid')from dual;
 get_sessionid

 1
(1 row)
```

41、兼容ASCIISTR函数，功能：传入字符串，返回对应的ASCII字符，测试用例如下：只有ASCII字符：

```
select asciistr('Hello, World!') from dual;
 asciistr

 Hello, World!
(1 row)
```

非ASCII字符：

```
select asciistr('你好') from dual;
asciistr

\4F60\597D
```

同时包含ASCII字符和非ASCII字符：

```
select asciistr('ABÄCDE') from dual;
asciistr

AB\00C4CDE
(1 row)
```

42、兼容TO\_MULTI\_BYTE函数, 功能：将字符串中的半角字符转换为全角字符：  
输入半角字符，转换为全角字符：

```
select to_multi_byte('1.2'::text) ;
to_multi_byte

1. 2
```

43、兼容TO\_SINGLE\_BYTE函数, 功能：将字符串中的半角字符转换为全角字符  
输入全角字符，转换为半角字符：

```
select to_single_byte('1. 2');
to_single_byte

1.2
```

44、兼容COMPOSE函数,功能：将基本字符和组合标记组合一个复合Unicode字符：  
输入基本字符a和组合标记768, 返回法语à

```
select compose('a' || chr(768)) from dual;
compose

à
(1 row)
```

45、兼容DECOMPOSE函数,功能：将复合Unicode字符（如带有重音或特殊符号的字符）分解为其基本字符和组合标记 输入法语é,返回基本字符e和组合标记301：

```
select asciistr(decompose('é')) from dual;
asciistr
```

## 9、新增Oracle兼容模式的端口与IP

### 目的

- 为了将Oracle端口、IP与PG的端口IP进行区分。现需要增加对ORAPORT和ORAHOST的处理;

### 功能

- 新增ivoryhost: 需要在连接时新增参数ivoryhost即可指定, 其功能类似 host;
- 新增ivoryport: 相较于host, port的功能相对复杂一些。其中涉及到可以在configure阶段配置, 连接阶段指定端口;

### 测试用例:

```
./configure --with-oraport=5555
./initdb
./pg_ctl -D ../data start

./pg_ctl -o "-p 5433 -o 1522" -D ../data
```

## 10、XML函数

### 目的

在Oracle中, 常会出现带有XML函数的SQL代码, IvorySQL在PostgreSQL的基础上, 实现与Oracle XML函数的高度兼容, 确保了从Oracle迁移到IvorySQL后的数据格式和结构的一致性。这种兼容性意味着用户无需对现有的XML处理逻辑进行大规模修改, 从而保证了数据的完整性和准确性。此外, IvorySQL的这种跨平台的兼容特性, 也降低了因格式差异带来的额外用户维护和升级成本, 使得数据处理和管理更加高效、可靠和灵活。



XML (eXtended Markup Language可扩展标记语言) 是一种基于文本的, 用于结构化任何可标记文档的格式语言。它是一种轻便的, 可扩展的, 标准的且简学易懂的保存数据的语言。

### 实现原理

IvorySQL在实现与Oracle 12c中11个常用XML SQL函数的兼容性时, 与PostgreSQL保持了一致, 底层处理函数同样采用了libxml2库提供的接口。这些XML函数作为ivorysql\_ora插件的一个子插件提供, 确保了与PostgreSQL数据库在XML处理方面的兼容性和一致性。

由于Oracle的xml函数要求某些参数类型是 XMLType, 比如下面这个existsnode()函数:

原型: EXISTSNODE(XMLType\_instance, XPath\_string [, namespace\_string])

示例: SELECT existsnode(XMLType('<a><b>d</b></a>'), '/a') from dual;

因此为了兼容, 添加了一个 XMLType 数据类型, 作用是把用户提供的字符串转换成内部的xmltype类型, 让用户的SQL语句从Oracle迁移到IvorySQL不用做改动



另外，为了避免与PostgreSQL中已有的同名关键字“extract”产生混淆，IvorySQL将原有的同名关键字重命名为“PGEXTRACT”，以确保函数调用的清晰性和准确性。

在实现这11个Oracle兼容的XML函数时，IvorySQL采用了两种不同的实现方式。其中，除了UPDATEXML函数外，其他10个函数均通过SQL函数的方式进行实现。由于UPDATEXML函数的参数数量是不确定的，因此采用了表达式方式来实现，这要求为其编写专门的语法分析代码和执行器代码，以确保其功能的正确性和灵活性。

兼容函数如下

序号	函数名	功能简介
1	<code>extract(XML)</code>	该函数用于返回XML节点路径下的相应内容，其中参数XMLType_instance用于指定XMLType实例，Xpath_string用于指定XML节点路径
2	<code>extractvalue</code>	该函数提供对XML内容的检索功能，extractvalue只能返回一个节点的一个值。
3	<code>existsnode</code>	该函数用于检查XML内容是否符合指定的路径表达式。
4	<code>deletexml</code>	该函数用于删除指定路径的XML节点
5	<code>appendchildxml</code>	该函数用于在XML对象中插入子节点。它接受一个XML对象和一个XML片段作为参数，并将XML片段作为子节点插入到XML对象中
6	<code>updatexml</code>	该函数用于更新特定XML相应的节点路径的内容
7	<code>insertxmlbefore</code>	该函数用于在XML中的指定路径前插入子节点
8	<code>insertxmlafter</code>	该函数用于在XML中的指定路径后插入子节点
9	<code>insertchildxml</code>	该函数用于向指定的XML路径中插入子节点
10	<code>insertchildxmlbefore</code>	该函数用于向指定的XML路径之前插入子节点
11	<code>insertchildxmlafter</code>	该函数用于向指定的XML路径之后插入子节点
12	<code>xmlisvalid</code>	该函数用于检查XML格式是否正确

XML函数使用示例

准备表与数据

```
ivorysql=# set ivorysql.compatible_mode to oracle;
SET
ivorysql=# create table inaf(a int, b xmltype);
CREATE TABLE
ivorysql=# insert into inaf values(1,xmltype('<a>100'));
INSERT 0 1
```

```

ivorysql=# insert into inaf values(2, '');
INSERT 0 1
ivorysql=# select * from inaf;
 a | b
---+-----
 1 | <a>100
 2 |
(2 rows)
ivorysql=# create table xmltest(id int, data XMLType);
CREATE TABLE
ivorysql=# insert into xmltest values(1, '<value>one</value>');
INSERT 0 1
ivorysql=# insert into xmltest values(2, '<value>two</value>');
INSERT 0 1
ivorysql=# select * from xmltest;
 id | data
----+-----
 1 | <value>one</value>
 2 | <value>two</value>
(2 rows)
ivorysql=# create table xmlnstest(id int, data xmltype);
CREATE TABLE
ivorysql=# INSERT INTO xmlnstest VALUES(1, xmltype('<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:typ="http://www.def.com"
xmlns:web="http://www.abc.com"><soapenv:Body><web:BBB><typ:EEE>41</typ:EEE><typ:FFF>42
</typ:FFF></web:BBB></soapenv:Body></soapenv:Envelope>'));
INSERT 0 1

```

extract(XML)

```

ivorysql# SELECT extract(XMLType('<AA><ID>1</ID></AA>'), '/AA/ID') from dual;
 extract

<ID>1</ID>
(1 row)

```

extractvalue

```

ivorysql# SELECT extractvalue(XMLType('<a>100'), '/a/b') from dual;
 extractvalue

100

```

(1 row)

existsnode

```
ivorysql=# SELECT existsnode(XMLType('<a>d'), '/a/b') from dual;
existsnode
```

```

1
(1 row)
```

deletexml

```
ivorysql=# SELECT
deletexml(XMLType('<test><value>oldnode</value><value>oldnode</value></test>'),
'/test/value') from dual;
deletexml
```

```

<test/>
(1 row)
```

appendchildxml

```
ivorysql=# SELECT
appendchildxml(XMLType('<test><value></value><value></value></test>'), '/test/value',
XMLTYPE('<name>newnode</name>')) from dual;
appendchildxml
```

```

<test> +
 <value> +
 <name>newnode</name>+
 </value> +
 <value> +
 <name>newnode</name>+
 </value> +
</test>
(1 row)
```

updatexml

```
ivorysql=# SELECT updatexml(xmltype('<value>one</value>'), '/value',
xmltype('<newvalue>1111</newvalue>')) FROM dual;
updatexml
```

```

```

```
<newvalue>1111</newvalue>
(1 row)
```

insertxmlbefore

```
ivorysql=# SELECT insertxmlbefore(XMLType('<a>222100200'), '/a/b',
XMLTYPE('<c>88</c>')) from dual;
 insertxmlbefore

<a>222<c>88</c>100<c>88</c>200
(1 row)
```

insertxmlafter

```
ivorysql=# SELECT
insertxmlafter(XMLType('<a>100'), '/a/b',XMLType('<c>88</c>')) from dual;
 insertxmlafter

<a> +
 100 +
 <c>88</c> +

(1 row)
```

insertchildxml

```
ivorysql=# SELECT insertchildxml(XMLType('<a>onethree'), '//b',
'name', XMLTYPE('<name>newnode</name>')) from dual;
 insertchildxml

<a>one<name>newnode</name>three<name>newnode</name>
(1 row)
```

insertchildxmlbefore

```
ivorysql=# SELECT insertchildxmlbefore(XMLType('<a>100'), '/a', 'b',
XMLTYPE('<c>88</c>')) from dual;
 insertchildxmlbefore

<a> +
 <c>88</c> +
 100 +

```

(1 row)

insertchildxmlafter

```
ivorysql=# SELECT insertchildxmlafter(XMLType('<a>100'), '/a', 'b',
XMLType('<c>88</c>')) from dual;
```

insertchildxmlafter

```

<a> +
 100 +
 <c>88</c> +

(1 row)
```

xmlisvalid

```
ivorysql=# SELECT xmlisvalid(XMLTYPE('<a>'));
```

xmlisvalid

```

f
(1 row)
```

```
ivorysql=# SELECT xmlisvalid(XMLTYPE('<a/>'));
```

xmlisvalid

```

t
(1 row)
```

## 11、兼容Oracle sequence

目的

- 本文档是IvorySQL兼容Oracle sequence文档，目的是可以在IvorySQL中使用Oracle的sequence语句。

功能说明

- 序列是一个数据库对象，与表和视图类似，它表示可以由全局数据库命名空间中的任何表和视图使用的整数序列。可以使用NEXTVAL和CURRVAL访问序列值。序列可以是升序或降序

测试用例

```
ivorysql=# CREATE sequence seq;
```

CREATE SEQUENCE

```
ivorysql=# SELECT seq.NEXTVAL FROM DUAL;
```

nextval

```

1
(1 row)
ivorysql=# ALTER sequence seq restart start with 10;
ALTER SEQUENCE
ivorysql=# SELECT seq.NEXTVAL FROM DUAL;
nextval

10
(1 row)
ivorysql=# DROP SEQUENCE seq;
DROP SEQUENCE

```

## 12、包

### 目的

IvorySQL提供了兼容Oracle自定义包的功能。包（package）是一个封装的相关程序对象集合，这些对象存储在数据库中。程序对象包括过程、函数、变量、常量、游标和异常。

本文档旨在为使用人员提供一个深入了解自定义包实现的过程。

### 功能说明

IvorySQL提供了兼容Oracle自定义包的功能，包括包和包体的创建、修改和描述，在PostgreSQL交互终端（psql）中增加了与包相关的命令，新增了 \dk 命令。

#### 创建包规范

使用CREATE OR REPLACE

PACKAGE语句创建或替换一个包的规范。包是相关的存储过程、函数、和其他程序对象的集合，在数据库中作为一个单元存储。包规范声明这些对象，在包体中定义这些对象。

在自己的模式下创建或替换一个包规范，必须有CREATE

PROCEDURE系统权限，如果在其他用户的模式下创建或替换一个包，则需要有CREATE ANY PROCEDURE系统权限。

#### 创建包体

使用CREATE OR REPLACE PACKAGE

BODY语句创建或替换一个包体。创建包体需要有与创建包规范一样的权限，要求包体与包规范在同一个模式下，且包规范必须存在，该语句定义包规范中声明的对象。

当包规范中有cursor或子过程时，那么必须拥有一个包体去定义它，否则包体是可选的。

#### 更新包

使用ALTER PACKAGE语句来改变包的属性。执行ALTER

PACKAGE语句的权限要求：必须是包的所有者，或者具有ALTER ANY PROCEDURE权限才能修改其他用户的包。

#### 删除包和包体

DROP PACKAGE语句删除数据库中的一个存储包。这个语句会同时删除包体和包规范。DROP PACKAGE

BODY则只删除包体。

不能使用此语句删除包中的一个单独对象。权限要求：包必须在用户模式下，或用户有DROP ANY PROCEDURE系统权限。

丢弃包

DISCARD PACKAGE功能是为了兼容PostgreSQL的DISCARD功能而做的。

在psql中使用 \dk[+] 查看包与包体定义

psql中支持 \dk[+] 命令查看包与包体的定义信息。

命令	描述
\dk[+]	列出当前能看到的包信息
\dk[+] xxx	列出xxx包的包规范和包体内容

测试用例

创建包规范

```
ivorysql=# create or replace package pkg is
ivorysql-# var1 integer;
ivorysql-# var2 integer;
ivorysql-# function test_f(id integer) return integer;
ivorysql-# procedure test_p(id integer);
ivorysql-# end;
ivorysql-# /
CREATE PACKAGE
```

创建包体

```
ivorysql=# create or replace package body pkg is
ivorysql-# var3 integer;
ivorysql-# function test_f(id integer) return integer is
ivorysql-# begin
ivorysql-# dbms_output.put_line('pkg test_f');
ivorysql-# return id;
ivorysql-# end;
ivorysql-# procedure test_p(id integer) is
ivorysql-# begin
ivorysql-# dbms_output.put_line('pkg proc');
ivorysql-# end;
ivorysql-# --private function
ivorysql-# function test_piv1(id integer) return integer is
ivorysql-# begin
ivorysql-# return id;
```

```

ivorysql-# end;
ivorysql-# --private procedure
ivorysql-# procedure test_piv2(id integer) is
ivorysql-# begin
ivorysql-# dbms_output.put_line('private proc');
ivorysql-# end;
ivorysql-# begin
ivorysql-# var1 := 1;
ivorysql-# var2 := 2;
ivorysql-# var3 := 4;
ivorysql-# end;
ivorysql-# /
CREATE PACKAGE BODY

```

## 更新包

```

ivorysql=# alter package pkg noneditionable;
ALTER PACKAGE

```

## 删除包和包体

```

ivorysql=# Drop package pkg;
DROP PACKAGE

ivorysql=# Drop package body pkg;
DROP PACKAGE BODY

```

## 丢弃包

```

ivorysql=# discard package;
DISCARD PACKAGES

```

## 在psql中使用 \dk[+] 查看包与包体定义

```

ivorysql=# \dk
 List of packages
 Schema | Name | Owner
-----+-----+-----
 public | pkg | ivorysql
 public | test_pkg | ivorysql
(2 rows)

```



ivorysql=# \dk pkg

## List of packages

Schema	Name	Owner
--------	------	-------

-----+-----+-----

```
public | pkg | ivorysql
```

(1 row)

ivorysql=# \dk pkg1

```
Did not find any package named "pkg1".
```

ivorysql=# \dk+

## List of packages

Schema	Name	Owner	Security	Editionable	Use Collation
Specification					Package Body

-----+-----+-----+-----+-----+

-----+

-----

```
public | pkg | ivorysql | definer | Editionable | default | var1 integer;
```

```
+ |
 |
 |
 |
 |
 | var2 integer;
```

+ |                  |                  |                  |                  | function

```
test_f(id integer) return integer; +|
```

| | | | | procedure

[illegible]

```
| public | test pkg | ivorysql | definer | Editionable | default | var1 integer;
```

```
+| FUNCTION test_f(id integer) RETURN integer IS
 +
```

					FUNCTION
test	f(id integer)	RETURN integer+1	BEGIN		

```
test_1 (id: Integer) RETURN Integer; | BEGIN
+
| | | | | | end
```

```
| dbms_output.put_line('invoke function test_pkg.t
```

```
est_f');+
|
|
|
|
|
|
```

```

| RETURN 23;
+

```

```
| end;
```

```
| BEGIN
 +
 | | | | |
| var1 := 23;
 +
 | | | | |
| end
(2 rows)
```

## 13、不可见列

### 目的

本功能的引入是为了兼容Oracle的不可见列功能，将使数据库设计更加简洁，数据管理的灵活性得到提高，用户可以更好地控制列的可见性。

在应用迁移过程中，不可见列也很有用。使新列不可见意味着它们不会被任何现有应用程序看到，但仍然可以被任何新应用程序引用，从而使应用程序的在线迁移更加简单。

### 功能说明

不可见列功能使用户能够在执行诸如SELECT \*等操作时隐藏特定列。可以使用不可见列对表进行更改，而不会干扰使用该表的应用程序。对表的任何通用访问都不会显示不可见的列，必须通过列名显式引用不可见列，查询和其他操作才能访问不可见列。

在Oracle模式下，psql的扩展描述命令 (\d+) 增加了对不可见列的支持。

### 测试用例

创建包含不可见列的表

```
ivorysql=# CREATE TABLE mytable (a INT, b INT INVISIBLE, c INT);
CREATE TABLE
```

### 向不可见列插入值

只有在INSERT语句中显式指定不可见列时，才能向不可见列插入值。向包含不可见列的表插入值时不能省略列列表，否则会报错。

```
ivorysql=# INSERT INTO mytable (a, b, c) VALUES (1,2,3);
INSERT 0 1

ivorysql=# INSERT INTO mytable VALUES (4,5,6);
ERROR: INSERT has more expressions than target columns
LINE 1: INSERT INTO mytable VALUES (4,5,6);
```

### 显示不可见列的输出

使用SELECT语句查询时，如果要显示不可见列信息，必须显式指定该不可见列，否则将不显示该不可见列。

```

ivorysql=# select * from mytable;
 a | c
---+---
 1 | 3
(1 row)

ivorysql=# select a,b,c from mytable;
 a | b | c
---+---+---
 1 | 2 | 3
(1 row)

```

在Oracle模式下psql扩展描述命令 (\d+) 对不可见列的支持

```

ivorysql=# \d+ mytable

```

Table "public.mytable"						
Column	Type	Collation	Nullable	Default	Invisible	Storage
Compression	Stats target	Description				
a	pg_catalog.int4					plain
b	pg_catalog.int4				invisible	plain
c	pg_catalog.int4					plain
Access method: heap						

## 限制

- 目前不支持modify column，将在未来版本实现；
- 在Oracle中，不可见列和列顺序有特别的注意事项。当一个表包含一个或多个不可见列时，这些不可见列不会包含在表的列顺序中。目前我们尚未对这块做处理。

# 社区贡献指南

## 概述

### 说明

IvorySQL由一个核心开发团队维护，该团队拥有对GitHub上的IvorySQL主存储库的提交权限。同时，我们非常渴望从更广泛的IvorySQL社区中的成员那里获得贡献。如果您希望看到您的代码或文档更改被添加到IvorySQL并出现在将来的版本中，本节的内容介绍是您需要知道的。

IvorySQL社区欢迎并赞赏所有类型的贡献，期待您的加入！

### 行为准则

作为成员，贡献者和领导者，我们每个人都应阅读我们的行为准则。我们承诺让每个人都能参与社区，成为一种无骚扰的体验，无论年龄、体型、有形或无形的残疾、种族、性特征、性别认同和表达、经验水平、教育、社会经济地位、国籍、个人外表、种族、宗教或性身份和取向如何。

我们承诺以有助于建立一个开放，热情，多样化，包容和健康社区的方式采取行动和互动。

### 管理说明

团队是持续开放的团队，专注于IvorySQL项目的一部分。一个团队有其审查者、提交者和维护者，并拥有一个或多个存储库。团队级别的决策来自其维护者。IvorySQL开发人员的典型提升路径是从用户到审查者，然后是提交者和维护者。但获得更多的角色并不意味着你对其他社区成员拥有任何特权。IvorySQL社区中的每个人都是平等的，都有责任与其他贡献者进行建设性合作，建立一个友好的社区。这些角色是对您在IvorySQL开发中做出重大贡献的自然奖励，并为您在开发工作流中提供更多权利，以提高您的效率。同时，他们要求你承担一些额外的责任：

团队荣誉：现在您已经是团队评审员/提交者/维护者的成员，意味着您代表着项目和您的团队成员。所以，请做一个好人来捍卫球队的声誉。

责任：提交者/维护者有权合并拉取请求，因此承担额外的责任来处理接受代码库或文档更改的后果。这包括在它导致问题时恢复或修复它，以及帮助发布经理解决发布前测试周期中发现的任何问题。

## 贡献者指南

在贡献之前，我们需要了解下IvorySQL目前的版本以及文档的版本。目前，我们维护着4.2等版本，我们的版本紧跟PG的更新步伐，贡献之前请更新至最新版本。之后我们需要细心浏览一下贡献的样式风格，熟悉代码贡献风格、提Issue样式、拉取PR标题样式、代码注释样式、文档贡献样式、文章贡献样式，这可以帮助您尽快成为IvorySQL的贡献者奥~。

### 贡献前的准备

#### 开始

IvorySQL是在GitHub上开发的，任何希望对其作出贡献的人都必须拥有GitHub帐户，并熟悉Git工具和工作流。还建议您遵循开发人员的邮件列表，因为一些贡献可能会在那里产生更详细的讨论。

如果您有GitHub帐户，fork这个存储库至您的个人仓库中，这样您就可以拥有您的私人副本来开始hacking，并将其用作拉取请求的来源。

#### IvorySQL贡献的许可

如果您提交的贡献是原创作品，那么您可以假设IvorySQL将作为整个IvorySQL版本的一部分发布给下游用户，该版本将遵循Apache许可证2.0版本。

如果您提交的内容不是原创作品，同样鼓励代码共享和尊重原作者的著作权，同样允许代码修改，再发布。\\\*请注意需要满足如下条件\\\*：

- (1) 需要给代码的用户一份Apache许可证。
- (2) 如果您修改了代码，需要在被修改的文件中说明。
- (3) 在延伸的代码中（修改和有源代码衍生的代码中）需要带有原来代码中的协议、商标、专利声明和其他原来作者规定需要包含的说明。
- (4) 如果再发布的产品中包含一个Notice文件，则在Notice文件中需要带有Apache许可证。您可以在Notice中增加自己的许可，但不可以表现为对Apache许可证构成更改。

最后，请记住，从非原始的工作中删除许可标头从来都不是一个好主意。即使您使用的文件部分最初在顶部有许可标头，您也应该保留它。与往常一样，如果您不太确定您的贡献所涉及的许可问题，请随时在开发人员邮件列表中联系我们。

## 您可以做什么贡献

### 贡献代码

可以将自己修改的bug、新增的功能等代码上传至个人仓库，最后提交PR请求合并至官网：<https://github.com/IvorySQL/IvorySQL>。

### 文档贡献

IvorySQL社区提供的是中英文文档。英文文档保存在…文档存储库中，中文文档保存在i18n文档存储库中。您可以为任一文档存储库做出贡献。

### 测试IvorySQL和报告问题

GitHub: <https://github.com/IvorySQL/IvorySQL>

码云：<https://gitee.com/IvorySQL/>

### 参与IvorySQL网站建设

IvorySQL官网下载：<https://github.com/IvorySQL/Ivory-www>

### 回答邮件列表问题

邮件列表位于 官网下：<https://lists.ivorysql.org/>

### 贡献文章

您可以在IvorySQL-WWW代码仓库自己提交至blog贡献，也可以联系我们IvorySQL小助理发送至邮箱[ivorysql@highgo.com](mailto:ivorysql@highgo.com)。

## 如何做出贡献

### 编码指南

您获得反馈和看到代码合并到项目中的机会在很大程度上取决于更改的粒度。如果您的想法发生了更大的变化，我们强烈建议您在花大量时间编写代码之前，先加入开发人员的邮件列表，并与我们分享您的建议。即使您的建议得到社区的验证，我们仍然建议您将实际工作作为一系列小型的、独立的提交来完成。这使得评审员的工作更加容易，并提高了反馈的及时性。

当谈到IvorySQL的C和C++部分时，我们尝试遵循PostgreSQL编码约定。除此之外：

对于C和Perl代码，如果需要，请运行pgindent。我们建议在查看更改时使用git diff—color，这样您提交的代码中就不会出现任何虚假的空白问题。

所有贡献给IvorySQL的新功能都应该有与其一起贡献的回归测试覆盖。如果您不确定如何测试或记录您的工作，请在ivorysql-hackers邮件列表中提出问题，社区的开发人员将尽力帮助您。

至少，您应该始终运行make install check world，以确保您没有破坏任何东西。

## 适用于上游PostgreSQL的更改

如果您正在进行的更改涉及PostgreSQL和IvorySQL之间的通用功能，则可能会要求您将其转发到PostgreSQL。这不仅是为了我们不断减少两个项目之间的差异，而且是为了让与PostgreSQL相关的任何变化都能从对上游PostgreSQL社区更广泛的审查中受益。一般来说，将这两个代码库都放在手边是个好主意，这样您就可以确定您的更改是否需要前移。

## 补丁提交

一旦您准备好与IvorySQL核心团队和IvorySQL社区的其他成员共享您的工作，您应该将所有提交推送到从官方IvorySQL派生的您自己的存储库中，并向我们发送请求。

## 补丁审查

假定提交的拉取请求通过验证检查，可供同行审查。同行审查是确保对IvorySQL的贡献具有高质量并与路线图和社区期望保持一致的过程。我们鼓励IvorySQL社区的每个成员审查请求并提供反馈。由于您不必成为核心团队成员就可以做到这一点，因此我们建议您向有兴趣成为IvorySQL长期贡献者的任何人提供一系列拉动手式评论。

同行评审的一个结果可能是达成共识，即您需要以某些方式修改pull请求。GitHub允许您将其他提交推送到从中发送请求的分支中。这些额外的提交将对所有审阅者可见。

当同行评议收到参与者至少+1张+1和no-1张的选票时，同行评议会趋于一致。在这一点上，您应该期望核心团队成员之一将您的更改引入到项目中。

在补丁审查期间的任何时候，您都可能会因审查人员和核心团队成员的工作效率而遇到延迟。请耐心等待，也不要气馁。如果您在几天内没有收到预期的反馈，请添加一条评论，要求更新pull请求本身，或者向邮件列表发送一封电子邮件。

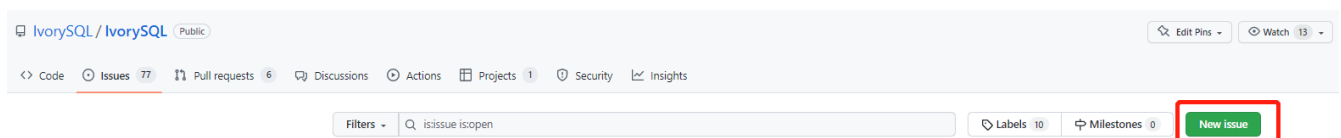
## 直接提交到存储库

有时，您会看到核心团队成员直接提交到存储库，而无需执行pull请求工作流。这仅适用于小的更改，我们使用的经验法则是：如果更改涉及任何可能导致测试失败的功能，那么它必须通过pull请求工作流。另一方面，如果更改发生在代码库的非功能部分（例如在注释块中修复打字错误），则核心团队成员可以决定直接提交到存储库。

# 提Issue

## 第1步 进入New issue 页面：

- 1、进入 IvorySql官网：<https://github.com/IvorySQL/IvorySQL>
- 2、点击New issue



## 第2步：选择需要填写的issue类型

### 1、 bug report

Title: 标题

## Bug Report

对bug进行描述

\### IvorySQL Version

在IvorySQL哪个版本发现的问题

\### OS Version (uname -a)

系统版本

\### Configuration options ( config.status --config )

配置参数

\### Current Behavior

当前的结果

\### Expected behavior/code

期望的结果

\### Step to reproduce

复现步骤

\### Additional context that can be helpful for identifying the problem

有助于识别问题的其它信息

### 2、 Enhancement

Title: 标题

## Enhancement

对于期望强化的功能作一个描述

### 3、 Feature Request

Title: 标题

## ## Feature Request

描述你期望实现的一个功能

### 第3步：提交

点击 submit new issue 按钮, 提交即可

## 贡献代码

### 第1步: Fork [ivorysql.org](https://github.com/ivorysql.org)仓库

- 1、打开ivorysql仓库 <https://github.com/IvorySQL/IvorySQL>
- 2、点击右上角fork按钮，等待fork完成

### 第2步:将fork的仓库克隆至本地

```
cd $working_dir # 将 $working_dir 替换为你想放置 repo 的目录。例如，`cd
~/Documents/GitHub`

git clone git@github.com:$user/IvorySQL.git # 将 `$user` 替换为你的 GitHub ID
```

### 第3步：创建一个新的Branch

```
cd $working_dir/IvorySQL

git checkout -b new-branch-name
```

### 第4步：编辑文档或修改代码

在新建的新-branch-name中修改代码。

### 第5步：生成commit

```
Git add <file>

Git commit -m "commit-message"
```

### 第6步：将修改推送至远端

```
Git push -u origin new-branch-name
```

### 第7步：创建一个Pull Request

- 1、打开你 Fork 的仓库：[https://github.com/\\$user/IvorySQL](https://github.com/$user/IvorySQL)（将 \$user 替换为你的 GitHub ID）。



2、点击 Compare & pull request 按钮即可创建 PR。

## 提交PR

对于提交一个PR应该保持一个功能，或者一个bug提交一次。禁止多个功能一次提交。

### 第1步：创建一个Pull Request

1、打开你 Fork 的仓库：[https://github.com/\\$user/IvorySQL](https://github.com/$user/IvorySQL)（将 \$user 替换为你的 GitHub ID）。

2、点击 Compare & pull request 按钮

### 第2步：填写PR信息

**Fix test**  
功能描述

**leave a comment**  
对该提交功能进行比较详细的描述

### 第3步：提交PR

点击Create pull request 按钮即可提交。

## 编写文档

### 准备工作

(1) 下载Markdown或者Typora文档编辑器。

(2) 检查源仓库是否有更新，如果有更新请先更新并同步到自己的仓库。如有更新请参阅以下步骤，更新至最新版本：

```
git remote
```

```
git fetch upstream
```

```
git merge upstream/main
```

```
git push
```

(3) 熟悉样式风格（规范说明）

### 贡献地方

IvorySQL社区提供双语文档。英文文档保存在IvorySQL/文档存储库（文档存储库）中，中文文档保存在IvorySQL/文档-i18n存储库（文档-i18n存储库）中。您可以为任何一方文档做出贡献，当然您也可以为两方同时做出贡献。

您可以从以下任何一项开始，以帮助改进IvorySQL网站（英文和-i18n）上的IvorySQL文档：

- (1) 编写完善文档
- (2) 修复拼写错误或格式（标点符号、空格、缩进、代码块等）
- (3) 修正或更新不当或过时的说明
- (4) 添加缺少的内容（句子、段落或新文档）
- (5) 将文档更改从英文翻译成中文，或从中文翻译成英文。
- (6) 提交、回复和解决文档问题或文档-i18n问题
- (7) （高级）查看其他人创建的拉取请求

## 规范说明

IvorySQL文档是用“markdown”编写的。为确保格式的质量和一致性，在修改更新文档时应遵循某些Markdown规则。

### Markdown规范

- 1、标题从一级开始递增使用，禁止跳级使用。例如：一级标题下面不能直接使用三级标题；二级标题下面不能直接使用四级标题。
- 2、标题必须统一使用 ATX 风格，即在标题前加 # 号来表示标题级别。
- 3、标题的引导符号 # 后必须空一格再接标题内容。
- 4、标题的引导符号“#”后只能空一格后再接标题内容，不能有多个空格。
- 5、标题必须出现在一行行首，即标题的 # 号前不能有任何空格。
- 6、标题末尾仅能出现中英文问号、反引号、中英文单双引号等符号。其余如冒号、逗号、句号、感叹号等符号均不能在标题末尾使用。
- 7、标题上面须空一行。
- 8、文档中不能连续出现内容重复的标题，如一级标题为 # IvorySQL 架构，紧接着的二级标题就不能是 ## IvorySQL 架构。如果不是连续的标题，则标题内容可重复。
- 9、文档中只能出现一个一级标题。
- 10、一般来说，除 TOC.md 文件可缩进 2 个空格外，其余所有 .md 文件每缩进一级，默认须缩进 4 个空格。
- 11、文档中（包括代码块内）禁止出现 Tab 制表符，如需缩进，必须统一用空格代替
- 12、禁止出现连续的空行。
- 13、块引用符号 > 后禁止出现多个空格，只能使用一个空格，后接引用内容。
- 14、使用有序列表时，必须从 1 开始，按顺序递增。
- 15、使用列表时，每一列表项的标识符（+、-、\* 或数字）后只能空一格，后接列表内容。

16、列表（包括有序和无序列表）前后必须各空一行。

17、代码块前后必须各空一行。

18、文档中禁止出现裸露的 URL。如果希望用户能直接点击并打开该 URL，则使用一对尖括号 (<URL>) 包裹该 URL。如果由于特殊情况必须使用裸露的 URL，不需要用户通过点击打开，则使用一对反引号 (`URL`) 包裹该 URL。

19、使用加粗、斜体等强调效果时，在强调标识符内禁止出现多余的空格。如不能出现 **加粗文本**。

20、单个反引号包裹的代码块内禁止出现多余的空格。如不能出现 ``示例文本``。

21、链接文本两边禁止出现多余的空格。如不能出现 [某链接](<https://www.example.com/>)。

22、链接必须有链接路径。如不能出现[空链接]()或[空链接](#)等情况。

## 示例

1、标题从一级开始递增使用，禁止跳级使用。

```
Heading 1
Heading 3

We skipped out a 2nd level heading in this document
```

2、标题必须统一使用 ATX 风格，即在标题前加 # 号来表示标题级别。

```
Heading 1
Heading 2
Heading 3
Heading 4
Another Heading 2
Another Heading 3
```

3、标题的引导符号 # 后必须空一格再接标题内容。禁止#后多个空格，禁止#前出现空格

错误示范：

```
Heading 1
Heading 2
```

正确示范：

```
Heading 1
Heading 2
```

4、标题末尾仅能出现中英文问号、反引号、中英文单双引号等符号。

错误示范

```
This is a heading.
```

正确示范

```
This is a heading
```

5、标题上面空一行

错误示范

```
Heading 1
Some text
Some more text## Heading 2
```

正确示范

```
Heading 1
Some text
Some more text

Heading 2
```

6、文档中不能连续出现内容重复的标题，如一级标题为 # IvorySQL 描述，紧接着的二级标题就不能是 ## IvorySQL 描述。如果不是连续的标题，则标题内容可重复。

错误示范

```
Some text

Some text
```

正确示范

```
Some text

Some more text
```

7、文档中只能出现一个一级标题。

错误示范

```
Top level heading
```

# Another top-level heading

正确释放

# Title

## Heading

## Another heading

8、一般来说，除 TOC.md 文件可缩进 2 个空格外，其余所有 .md 文件每缩进一级，默认须缩进 4 个空格。

错误示范

```
* List item
 * Nested list item indented by 3 spaces
```

正确示范:

```
* List item
 * Nested list item indented by 4 spaces
```

9、文档中（包括代码块内）禁止出现 Tab 制表符，如需缩进，必须统一用空格代替

错误示范：

```
Some text
 * hard tab character used to indent the list item
```

正确示范:

```
Some text
 * Spaces used to indent the list item instead
```

10、禁止出现连续的空行

错误示范

Some text here

Some more text here

正确释放:

Some text here

Some more text here

11、块引用符号 > 后禁止出现多个空格，只能使用一个空格，后接引用内容。

错误示范

```
> This is a blockquote with bad indentation> there should only be one.
```

正确示范

```
> This is a blockquote with correct> indentation.
```

12、使用有序列表时，必须从 1 开始，按顺序递增。

错误示范:

```
1. Do this.
1. Do that.
1. Done.
```

```
0. Do this.
1. Do that.
2. Done.
```

正确示范:

```
1. Do this.
2. Do that.
3. Done.
```

13、使用列表时，每一列表项的标识符（+、-、\* 或数字）后只能空一格，后接列表内容。

正确示范

```
* Foo
* Bar
* Baz

1. Foo
 * Bar
```

## 1. Baz

14、列表（包括有序和无序列表）前后必须各空一行。

错误示范

```
Some text* Some* List
```

```
1. Some2. List
```

```
Some text
```

正确示范

```
Some text
```

```
* Some
```

```
* List
```

```
1. Some
```

```
2. List
```

```
Some text
```

15、代码块前后必须各空一行。

错误示范

```
Some text
```

```
```
```

```
Code block
```

```
```
```

```
```
```

```
Another code block
```

```
```
```

```
Some more text
```

正确示范

```
Some text
```

```
```
```

```
Code block
```

```
```
```

...

Another code block

...

Some more text

16、文档中禁止出现裸露的 URL。如果希望用户能直接点击并打开该 URL，则使用一对尖括号 (<URL>) 包裹该 URL。如果由于特殊情况必须使用裸露的 URL，不需要用户通过点击打开，则使用一对反引号 (**URL**) 包裹该 URL。

错误示范

For more information, see <https://www.example.com/>.

正确示范

For more information, see <<https://www.example.com/>>.

17、使用加粗、斜体等强调效果时，在强调标识符内禁止出现多余的空格。如不能出现 **加粗文本**。

错误示范

Here is some **bold** text.

Here is some *italic* text.

Here is some more **bold** text.

Here is some more *italic* text.

正确示范:

Here is some **bold** text.

Here is some *italic* text.

Here is some more **bold** text.

Here is some more *italic* text.

18、单个反引号包裹的代码块内禁止出现多余的空格。如不能出现 `示例文本`。

错误示范:



```
some text
some text
```

正确示范:

```
some text
```

19、链接文本两边禁止出现多余的空格。如不能出现 [ 某链接 ](<https://www.example.com/>)。

错误示范

```
[a link](https://www.example.com/)
```

正确示范:

```
[a link](https://www.example.com/)
```

20、链接必须有链接路径。如不能出现[空链接]()或[空链接](#)等情况。

错误示范

```
[an empty link]()
```

```
[an empty fragment](#)
```

正确示范:

```
[a valid link](https://example.com/)
```

```
[a valid fragment](#fragment)
```

21文档中的代码块统一使用三个反引号进行包裹，禁止使用缩进四格风格的代码块。

错误示范：

```
Some text.
```

```
 # Indented code
```

```
More text.
```

正确示范

```
```ruby
```

Fenced code

...

More text.

环境准备

为了测试您所做的修改是否修改，您需要准备以下环境

- **Node.js** 安装
- **Antora** 安装

环境安装请参考 [Antora docs](#)。

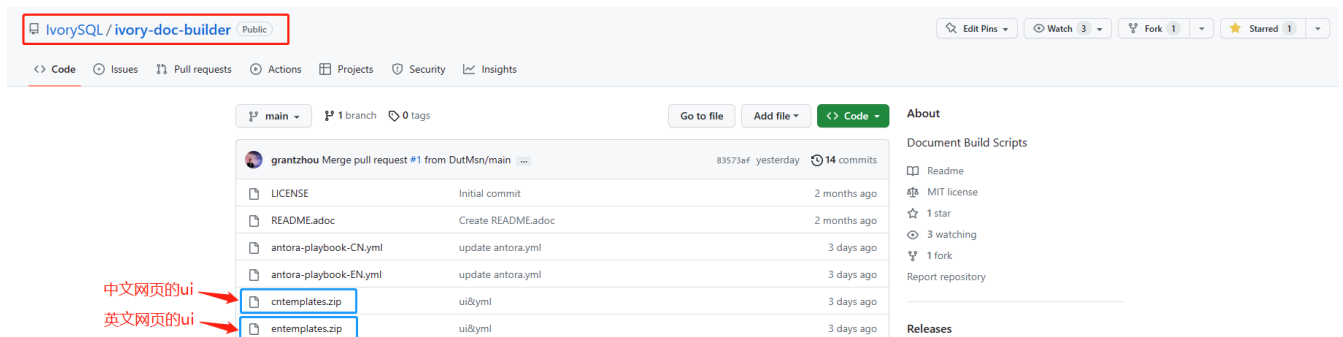
安装成功后，在终端上显示如下即为成功安装。

```
● [root@ivorysql ~]# node -v
v16.19.1
● [root@ivorysql ~]# antora -v
2.3.4
```

网页生成

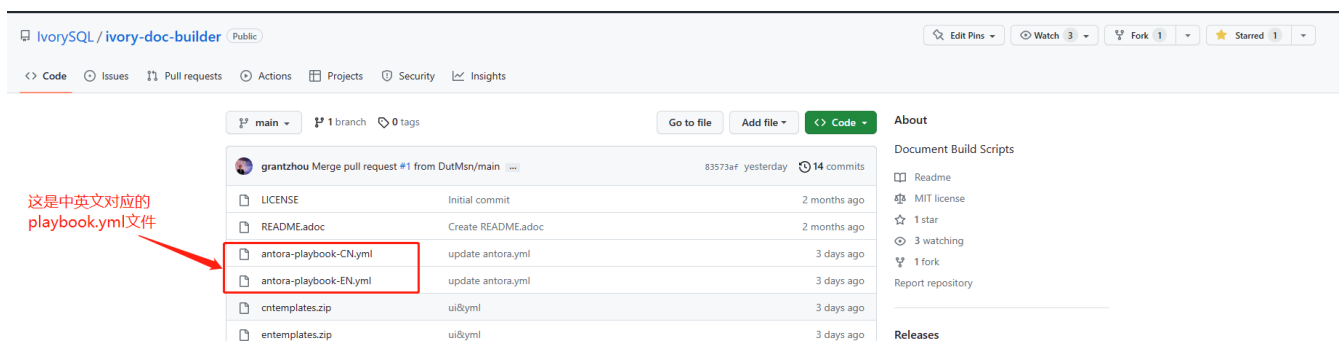
通过阅读之前内容，相信您已经有了充足的准备，包括 **fork** 我们文档中心相关的三个仓库到个人仓库中，**Antora** 工具的安装准备等环境的搭建。

- 首先，您要知道网页对应的ui的位置，如下图：



中英文的网页ui模板基本一致，因此修改时应该尽量保证同时修改两个模板，将修改过后的ui再上传至个人Github上，完成了这些，就可以考虑在本地生成您修改过后的网页了。

文档中心是由 **Antora** 进行搭建的，在运行 **Antora** 之前，记得修改对应 **playbook.yml** 文件



```

1 site:
2   title: IvorySQL Document Site
3   # the 404 page and sitemap files only get generated when the url property is set
4   url: https://docs.ivorysql.org
5   start_page: ivorysql-doc::v2.2/welcome.adoc
6 content:
7   sources:
8     - url: https://github.com/IvorySQL/ivorysql_docs.git
9       branches: [v2.*,v1.*]
10     start_paths:
11       - CN
12 asciidoc:
13   attributes:
14     experimental: ''
15     idprefix: ''
16     idseparator: '-'
17     page-pagination: ''
18 output:
19   dir: ../docs/cn
20 ui:
21   bundle:
22     url: https://github.com/DutMsn/ivory-doc-builder/raw/main/cntemplates.zip
23     snapshot: true
24 runtime:
25   fetch: true

```

这里是源文件的地址，如果有更改，记得修改这里的url

这里是ui包的地址，如果修改了，请把这里url修改为你想用的ui包的地址

完成上述流程之后，请在终端运行命令 **antora antora-playbook.yml --stacktrace**，然后耐心等待，当成功运行结束后，你就可以查看自己生成的网页了。

在检查之后，你就可以开始着手上传至我们的 ivorysql_web 仓库中，提交PR的流程和之前的流程相同，感谢您对社区的贡献-，我们会在审核过后，考虑是否更新网站。

提交blog

准备工作

- 1、下载 [Markdown](#)或者 [Typora](#)文档编辑器。
- 2、先下载博客源码到本地，检查源仓库(<https://github.com/IvorySQL/Ivory-www>)是否有更新，如果有更新请先更新并同步到自己的仓库。请参阅以下步骤，更新至最新版本：

```

# 拉取网站源码
git clone https://github.com/IvorySQL/Ivory-www.git
# 同步更新仓库
git pull

```

- 3、熟悉样式风格 ([\[v4-4::23:::_规范说明_2\]](#))

贡献地方

IvorySQL社区提供双语文档。英文博客保存在源码目录<u>Ivory-www/blog</u>中，中文博客保存在源码目录<u>Ivory-www/i18n/zh-CN/docusaurus-plugin-content-blog</u>中。您可以为任何一方博客做出贡献，当然您也可以为两方同时做出贡献。

如何贡献

在贡献之前，让我们快速浏览一下有关IvorySQL博客维护的信息。这可以帮助您尽快成功的提交blog成为贡献者。blog提交规范

(1) 将代码克隆到本地仓库

```
git clone https://github.com/IvorySQL/Ivory-www.git
```

(2) 创建一个分支

```
git checkout -b <branch-name>
```

(3) 在博客目录中创建自己文章的目录，目录名字规则参照 ([v4-4::23::_规范说明_2])。

```
# 生成英文博客目录及文件
cd Ivory-www/blog
mkdir <YEAR-MONTH-DAY-title>
cd <YEAR-MONTH-DAY-title>
touch index.md
# 生成中文博客目录及文件
cd Ivory-www/i18n/zh-CN/docusaurus-plugin-content-blog
mkdir <YEAR-MONTH-DAY-title>
cd <YEAR-MONTH-DAY-title>
touch index.md
```

(4) 在index.md编写要发布的博文，将博客中需要的图片放到和index.md同级目录。

(5) 提交发布博客

```
git add <file-path>
git commit -m "<message>"
git push origin <branch-name>:<branch-name>
```

规范说明

blog提交规范

(1) 文件夹命名格式：年-月-日-名称

示例：2022-1-28-ivorysql-arrived

(2) 文件属性统一为index.md

(3) 图片属性统一为.png形式，并将需要上传的图片提前放到要提交的文件夹中。

注意：图片名字唯一，不可重复奥~。

示例：po-one.png

博客是用markdown或者Typora来编写，您可以阅读 [博客 | Docusaurus](#)来了解博客的设计方式。

(1) 文章头部部署包括以下信息

```
---
slug: IvorySQL
title: 欢迎来到IvorySQL社区
authors: [official]
authorTwitter: IvorySql
tags: [IvorySQL, Welcome, Database, Join Us]
---
```

提示：您可以将以上模板复制到您的文件夹中并进行编辑。

注意：1) slug、title、authors、tags后添加文字均空一格。

2) slug每篇名字唯一且不可重复，相同文章中英文版可以相同。

(2) 文本格式

正文段落标题统一用h2/“二级标题”；

正文使用默认字体字号。

(3) 插入照片命名形式

[Hello](Hello-banner.png)

(4) 插入超链接命名形式

[名称] (链接)

[Github page](<https://github.com/IvorySQL/>) 下载源代码或发布的软件包。

网站贡献指南

[IvorySQL的文档中心](#) 采用开源工具 **Antora** 进行搭建，同样的，我们的文档中心也是开源的。文档中心由三部分组成，分别是 [文档文件](#), [静态网页文件](#) 以及生成网页所用到的 [网页模板文件](#)。

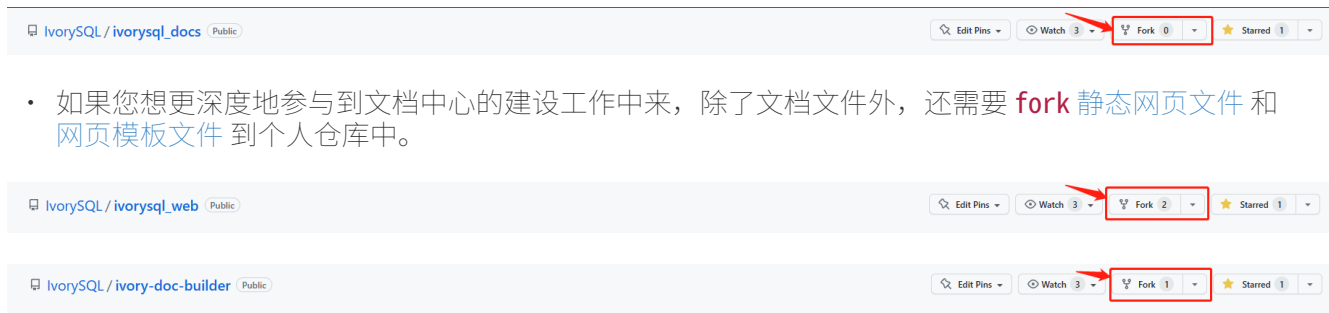
我们的文档中心同样欢迎每一位愿意参与到开源工作中的小伙伴的加入，记得遵守我们的行为准则-。

如何贡献

由于我们的文档中心全部托管于Github上，这使得任何用户都可以将我们的文档仓库 **fork** 到个人仓库中，然后对其进行修改，之后提交PR，由我们的开源团队的人员审核过后就可以将修改更新到我们的文档中心中。

为了更加便捷地使您达到纠正文档错误的目的，首先需要您按照想要更新的大小来建立个人仓库，如下：

- 如果您想修改原有的内容或者添加新的页面，仅需要 **fork 文档文件** 到个人仓库中。



- 如果您想更深度地参与到文档中心的建设工作中来，除了文档文件外，还需要 [fork 静态网页文件](#) 和 [网页模板文件](#) 到个人仓库中。

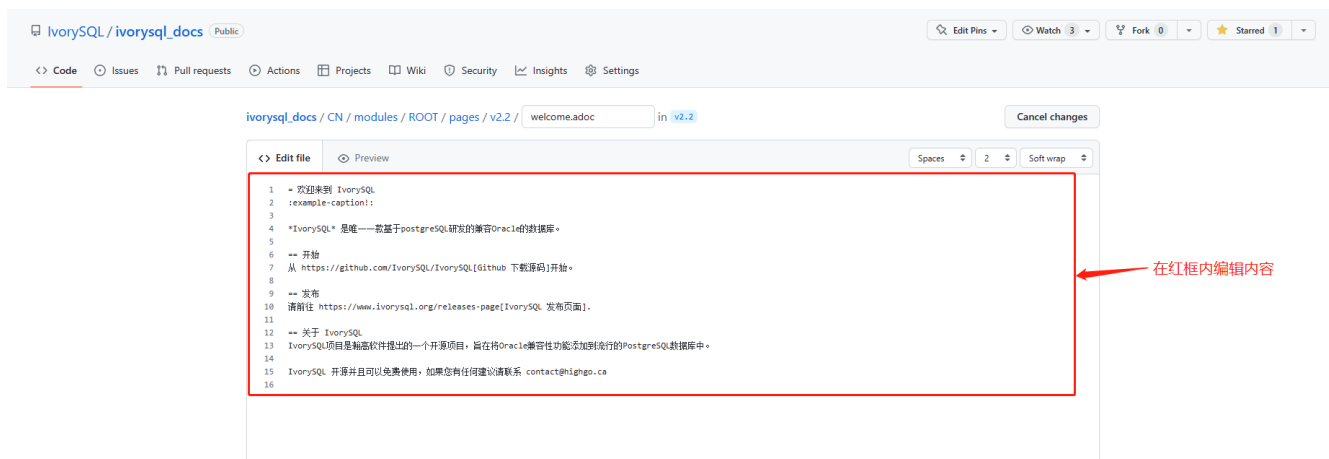
修改内容

本小节将会介绍发现网页内容不适宜之后，对网页内容进行修改的流程。

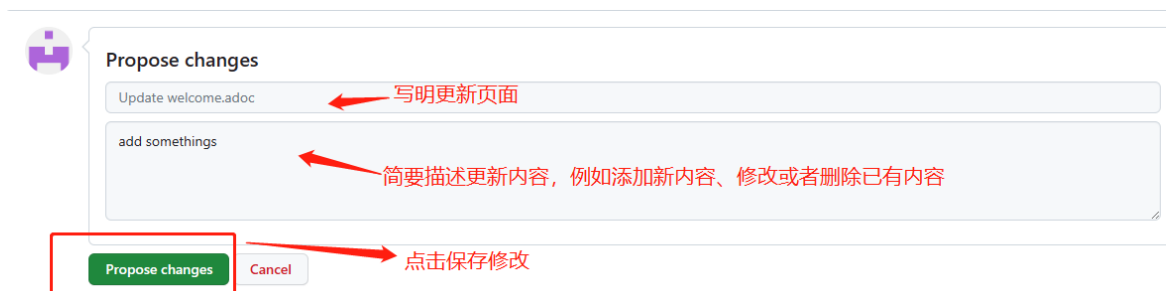
- 在有错误内容的网页的右上角，有一个 **edit this page** 的按钮，点击按钮。如图：



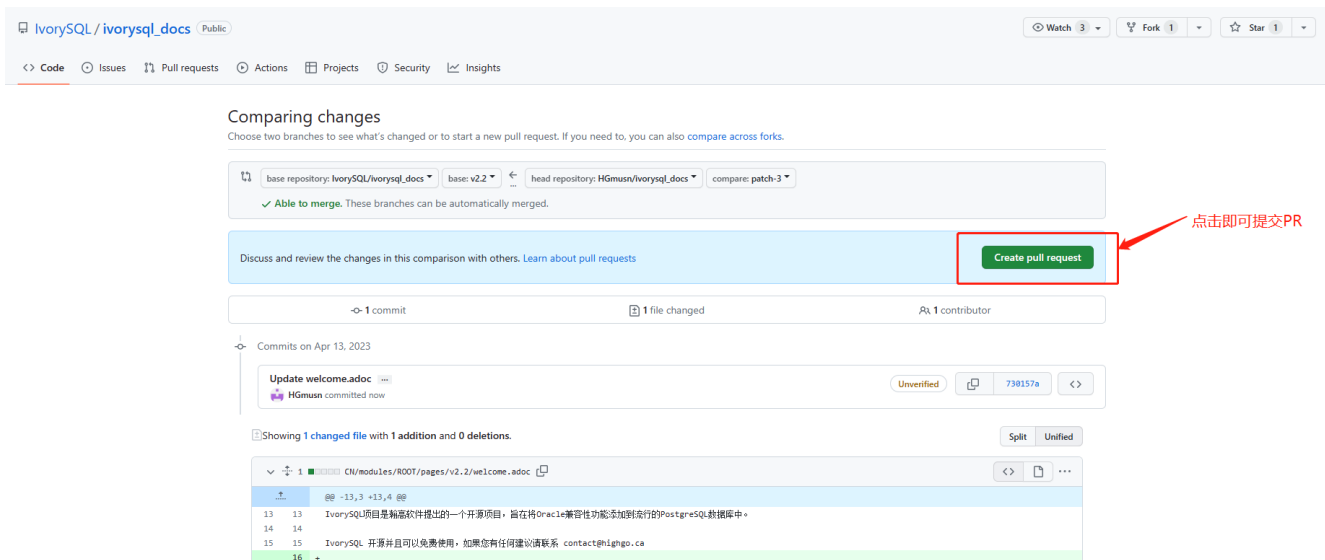
- 点击之后，就会跳转到我们存放当前页面源 **.adoc** 文件的编辑页面，请按照 **Ascii doc** 格式对内容进行修改。如图：



- 编辑完成后，如图：



- 确认更新后，如图：



- 接下来会由开源团队的相关人员负责审核您提交的内容，审核完成后您所提交的更新就会出现在对应页面上。

添加页面

本小节会介绍如何在网站上添加新的页面组件，添加新页面涉及的修改主要包括以下几种：

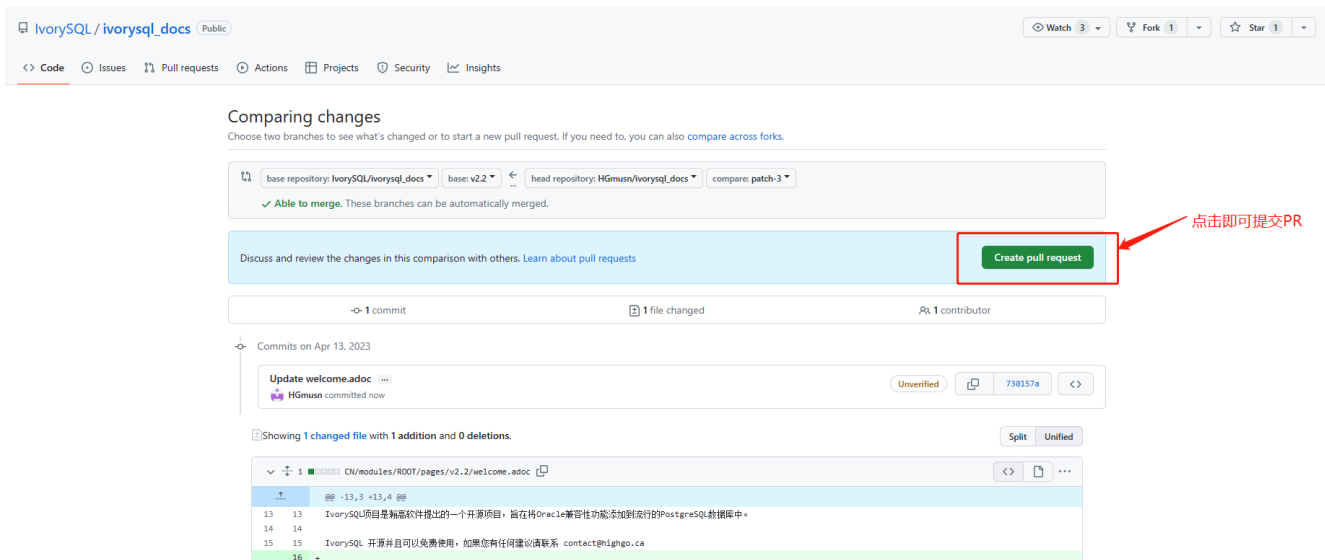
- **CN/modules/ROOT/pages/vX.X** 目录下的 **.adoc** 文件添加
 - **CN/modules/ROOT/nav.adoc** 的修改，如果修改涉及到图片的修改或者添加，请修改 **images** 中的图片
 - **EN/modules/ROOT/pages/vX.X** 目录下的 **.adoc** 文件添加
 - **EN/modules/ROOT/nav.adoc** 的修改，如果修改涉及到图片的修改或者添加，请修改 **images** 中的图片
1. 首先，您需要把您 **fork** 的仓库，clone到本地

```
git clone https://github.com/$username$/ivorysql_docs.git
```

2. 然后，将要添加的 **.adoc** 文件放至正确目录下，切记中英文的都应该准备（中英文文件应该同名），并且各自放至正确目录下，同时，修改对应的 **nav.adoc** 文件（修改方式可以参照文件已有内容进行修改）。
3. 上述修改完成后，先提交至个人仓库

```
git add .
git commit -m "$describe your change$"
git push
```

4. 之后按照如下提交PR即可



测试

如果您不满足于简单的在网页端进行提交或者仅修改网页内容，又或者您想要修改网页ui，本小节内容将会帮助您。

在阅读本小节内容之前，您需要确认您的Github个人仓库是否已经 **fork** 了网页模板文件仓库和静态网页文件仓库，如果没有请参考[\[v4-4::23:::如何贡献\]](#)。

部署网页

部署网页这部分工作，暂时还是由开源团队的人员负责，我们重视每一次提交和每一个issue，所以请不要担心自己对于社区的贡献被埋没。

Tip

本章节主要介绍了如何自己主动去更新我们的文档中心，但是我们明确地知道山高路远，时不我待。因此对于没有太多时间的小伙伴来说，可以发邮件到我们的 ivorysql-docs@ivorysql.org，我们会有专门的工作人员进行处理您的每一封来信，期待您的邮件。

工具参考

工具一览表

这部份包含IvorySQL客户端应用和工具的参考信息。不是所有这些命令都是通用工具，某些需要特殊权限。这些应用的共同特征是它们可以被运行在任何主机上，而不管数据库服务器在哪里。

当在命令行上指定用户和数据库名时，它们的大小写会被保留 — 空格或特殊字符的出现可能需要使用引号。表名和其他标识符的大小写不会被保留并且可能需要使用引号。

分类	工具名称	描述
客户端工具	clusterdb	clusterdb是一个工具，它用来对一个IvorySQL数据库中的表进行重新聚簇。它会寻找之前已经被聚簇过的表，并且再次在最后使用过的同一个索引上对它们重新聚簇。没有被聚簇过的表将不会被影响。clusterdb是SQL命令 CLUSTER 的一个包装器。在通过这个工具和其他方法访问服务器来聚簇数据库之间没有实质性的区别。
	createdb	createdb创建一个新的IvorySQL数据库。通常，执行这个命令的数据库用户将成为新数据库的所有者。但是，如果执行用户具有合适的权限，可以通过 -O 选项指定一个不同的所有者。createdb是SQL命令 CREATE DATABASE 的一个包装器。在通过这个工具和其他方法访问服务器来创建数据库之间没有实质性的区别。
	createuser	createuser创建一个新的IvorySQL用户（或者更准确些，是一个角色）。只有超级用户和具有 CREATEROLE 特权的用户才能创建新用户，因此createuser必须被以上两种用户调用。如果你希望创建一个新的超级用户，你必须作为一个超级用户连接，而不仅仅是具有 CREATEROLE 特权。作为一个超级用户意味着绕过数据库中所有访问权限检查的能力，因此超级用户访问权限不能轻易被授予。createuser是SQL命令 CREATE ROLE 的一个包装器。在通过这个工具和其他方法访问服务器来创建用户之间没有实质性的区别。
	dropdb	dropdb毁掉一个现有的IvorySQL数据库。执行这个命令的用户必须是一个数据库超级用户或该数据库的拥有者。dropdb是SQL命令 DROP DATABASE 的一个包装器。在通过这个工具和其他方法访问服务器来删除数据库之间没有实质性的区别。
	dropuser	dropuser移除一个已有的IvorySQL用户。只有超级用户以及具有 CREATEROLE 特权的用户能够移除IvorySQL用户（要移除一个超级用户，你必须自己是一个超级用户）。dropuser是SQL命令 DROP ROLE 的一个包装器。在通过这个工具和其他方法访问服务器来删除用户之间没有实质性的区别。
	ecpg	ecpg 是用于 C 程序的嵌入式 SQL 预处理器。它通过将 SQL 调用替换为特殊函数调用把带有嵌入式 SQL 语句的 C 程序转换为普通 C 代码。输出文件可以被任何 C 编译器工具链处理。 ecpg 将把命令行中给出的每一个输入文件转换为相应的 C 输出文件。如果输入文件名没有任何扩展名，则假定为 .pgc 。文件扩展名将由 .c 替换以构造输出文件名。但是输出文件名可以使用 -o 选项覆盖。如果输入文件名只是 - ， ecpg 从标准输入读取程序（并写入标准输出，除非用 -o 重写）。

pg_amcheck	pg_amcheck支持对一个或多个数据库运行 amcheck 的损坏检查函数，并提供选项来选择要检查的模式、表和索引、要执行的检查类型以及是否并行执行检查，如果是，按并行数建立连接并使用。当前仅支持表关系和btree索引。其他关系类型将自动跳过。如果指定了 dbname，则它应该是要检查的单个数据库的名称，并且不应该存在其他数据库选择选项。否则，如果存在任何数据库选择选项，将检查所有匹配的数据库。如果不存在此类选项，将选中默认数据库。数据库选择选项包括 --all，--database 和 --exclude-database。它们还包括 --relation，--exclude-relation，--table，--exclude-table，--index，和 --exclude-index，但仅当这些选项与三段式模式一起使用时（例如，mydb*.myschema*.myrel*）。最后，它们包括 --schema 和 --exclude-schema 当这些选项与两段式模式一起使用时（例如 mydb*.myschema*）。
pg_basebackup	pg_basebackup被用于获得一个正在运行的IvorySQL数据库集簇的基础备份。获得这些备份不会影响数据库的其他客户端，并且可以被用于时间点恢复，以及用作一个日志传送或流复制后备服务器的开始点。pg_basebackup对数据库群集的文件进行精确复制，同时确保服务器自动进入和退出备份模式。备份总是从整个数据库集簇获得，不可能备份单个数据库或数据库对象。关于选择性备份，必须使用一个像 pg_dump 的工具。备份通过一个使用复制协议常规IvorySQL连接制作。该连接必须由一个具有 REPLICATION 权限或者具有超级用户权限的用户ID建立，并且 pg_hba.conf 必须允许该复制连接。该服务器还必须被配置，使 max_wal_senders 设置得足够高以提供至少一个walsender用于备份以及一个用于WAL流（如果使用流）。在同一时间可以有多个 pg_basebackup 运行，但是从性能的角度来说，只进行一次备份并且复制结果通常更好。pg_basebackup不仅能从主控机也能从后备机创建一个基础备份。要从后备机获得一个备份，设置后备机让它能接受复制连接（也就是，设置 max_wal_senders 和 hot_standby，并且适当配置其 pg_hba.conf）。你也将也需要在主控机上启用 full_page_writes。注意在来自后备机的备份中有一些限制：不会在被备份的数据库集簇中创建备份历史文件。pg_basebackup不能强制备用服务器在备份结束时切换到新的WAL文件。当正在使用 -X none 时，如果服务器上的写活动比较低，pg_basebackup可能需要等待很长时间，以便切换和归档备份所需要的最后的WAL文件。在这种情况下，在主服务器上运行 pg_switch_wal 以触发立即的WAL文件切换可能是有用的。如果在备份期间后备机被提升为主控机，备份会失败。备份所需的所有 WAL 记录必须包含足够的全页写，这要求你在主控机上启用 full_page_writes 并且不使用一个类似pg_compresslog的工具以 archive_command 从 WAL 文件中移除全页写。每当pg_basebackup进行基本备份时，服务器的 pg_stat_progress_basebackup 视图将报告备份的进度。
pgbench	pgbench是一种在IvorySQL上运行基准测试的简单程序。它可能在并发的数据库会话中一遍一遍地运行相同序列的 SQL 命令，并且计算平均事务率（每秒的事务数）。默认情况下，pgbench会测试一种基于 TPC-B 但是要更宽松的场景，其中在每个事务中涉及五个 SELECT、UPDATE 以及 INSERT 命令。但是，通过编写自己的事务脚本文件很容易用来测试其他情况。
pg_config	pg_config工具打印当前安装版本的IvorySQL的配置参数。它的设计目的之一是便于想与IvorySQL交互的软件包能够找到所需的头文件和库。

pg_dump	pg_dump 是用于备份一种 IvorySQL 数据库的工具。即使数据库正在被并发使用，它也能创建一致的备份。pg_dump 不阻塞其他用户访问数据库（读取或写入）。pg_dump 只转储单个数据库。要备份一个集簇或者集簇中对于所有数据库公共的全局对象（例如角色和表空间），应使用 pg_dumpall。转储可以被输出到脚本或归档文件格式。脚本转储是包含 SQL 命令的纯文本文件，它们可以用来重构数据库到它被转储时的状态。要从这样一个脚本恢复，将它输入到 psql。脚本文件甚至可以被用来在其他机器和其他架构上重构数据库。在经过一些修改后，甚至可以在其他 SQL 数据库产品上重构数据库。另一种可选的归档文件格式必须与 pg_restore 配合使用来重建数据库。它们允许 pg_restore 能选择恢复什么，或者甚至在恢复之前对条目重排序。归档文件格式被设计为在架构之间可移植。当使用归档文件格式之一并与 pg_restore 组合时，pg_dump 提供了一种灵活的归档和传输机制。pg_dump 可以被用来备份整个数据库，然后 pg_restore 可以被用来检查归档并/或选择数据库的哪些部分要被恢复。最灵活的输出文件格式是“自定义”格式（-Fc）和“目录”格式（-Fd）。它们允许选择和重排序所有已归档项、支持并行恢复并且默认是压缩的。“目录”格式是唯一一种支持并行转储的格式。当运行 pg_dump 时，我们应该检查输出中有没有任何警告（打印在标准错误上）
pg_dumpall	pg_dumpall 工具可以一个集簇中所有的 IvorySQL 数据库写出到（“转储”）一个脚本文件。该脚本文件包含可以用作 psql 的输入 SQL 命令来恢复数据库。它会对集簇中的每个数据库调用 pg_dump 来完成该工作。pg_dumpall 还转储对所有数据库公用的全局对象（pg_dump 不保存这些对象），也就是说数据库角色和表空间都会被转储。目前这包括适数据库用户和组、表空间以及适合所有数据库的访问权限等属性。因为 pg_dumpall 从所有数据库中读取表，所以你很可能需要以一个数据库超级用户的身份连接以便生成完整的转储。同样，你也需要超级用户特权执行保存下来的脚本，这样才能增加角色和组以及创建数据库。SQL 脚本将被写出到标准输出。使用 -f / --file 选项或者 shell 操作符可以把它重定向到一个文件。pg_dumpall 需要多次连接到 IvorySQL 服务器（每个数据库一次）。如果你使用口令认证，可能每次都会要求口令。这种情况下使用一个 ~/.pgpass 会比较方便
pg_isready	pg_isready 是一个用来检查一个 IvorySQL 数据库服务器的连接状态的工具。其退出状态指定了连接检查的结果。
pg_receivewal	pg_receivewal 被用来从一个运行着的 IvorySQL 集簇以流的方式得到预写式日志。预写式日志会被使用流复制协议以流的方式传送，并且被写入到文件的一个本地目录。这个目录可以被用作归档位置来做一次使用时间点恢复的恢复。当预写式日志在服务器上被产生时，pg_receivewal 实时以流的方式传输预写式日志，并且不像 archive_command 那样等待段完成。由于这个原因，在使用 pg_receivewal 时不必设置 archive_timeout。与 IvorySQL 后备服务器上的 WAL 接收进程不同，pg_receivewal 默认只在一个 WAL 文件被关闭时才刷入 WAL 数据。要实时刷入 WAL 数据，必须指定选项 --synchronous。由于 pg_receivewal 不应用于 WAL，当 synchronous_commit 等于 remote_apply 时，你将不允许它成为同步备用。如果发生这样的情况，它将成为一个永远不能拉起的备用数据库，并且会导致事务提交阻塞。为了避免这种情况，你应该为 synchronous_standby_names 配置一个适当的值，或规定为 pg_receivewal 的 application_name 与它不匹配，或将 synchronous_commit 的值更改为 remote_apply 以外的内容。预写式日志在一个常规 IvorySQL 连接上被以流式传送，并且使用复制协议。连接必须由一个具有 REPLICATION 权限的用户或者一个超级用户建立，并且 pg_hba.conf 必须允许复制连接。服务器也必须被配置一个足够高的 max_wal_senders 来至少留出一个可用会话给流。首先，扫描 WAL 段文件所写入的目录，并发现最新完成的段文件，作为下一个段文件的开始的起始点。这是独立计算的，根据用于压缩每个段的压缩方法。如果用前面的方法无法计算出起点，最新的 WAL 刷写位置用作由服务器通过 IDENTIFY_SYSTEM 命令的报告。

pg_recvlogical	pg_recvlogical 控制逻辑解码复制槽以及来自这种复制槽的流数据。它会创建一个复制模式的连接，因此它受到和 pg_receivewal 相同的约束，还有逻辑复制的约束。pg_recvlogical 与逻辑解码SQL接口的peek和get模式没有等效性。它咋接收到数据以及干净地退出时，它会惰性地发送数据的确认。为了检查一个槽上还未消费的待处理数据，可以使用 pg_logical_slot_peek_changes。
pg_restore	pg_restore是一个用来从 pg_dump 创建的非文本格式归档恢复IvorySQL数据库的工具。它将发出必要的命令把该数据库重建成它被保存时的状态。这些归档文件还允许pg_restore选择恢复哪些内容或者在恢复前对恢复项重排序。这些归档文件被设计为可以在不同的架构之间迁移。pg_restore可以在两种模式下操作。如果指定了一个数据库名称，pg_restore会连接那个数据库并且把归档内容直接恢复到该数据库中。否则，会创建一个脚本，其中包含着重建该数据库所必要的 SQL 命令，它会被写入到一个文件或者标准输出。这个脚本输出等效于pg_dump的纯文本输出格式。因此，一些控制输出的选项与pg_dump的选项类似。显然，pg_restore无法恢复不在归档文件中的信息。例如，如果归档使用“以 INSERT 命令转储数据”选项创建，pg_restore将无法使用 COPY 语句装载数据。
pg_verifybackup	pg_verifybackup用于根据备份时服务器生成的 backup_manifest 检查使用 pg_basebackup 进行的数据库群集备份的完整性。备份必须以“普通”格式存储；“tar”格式的备份可以在解压缩后进行检查。需要注意的是，由pg_verifybackup执行的验证不包括也不可能包括运行中的服务器在尝试使用备份时执行的所有检查。即使使用此工具，也应执行测试还原，并验证生成的数据库是否按预期工作，以及它们是否包含正确的数据。但是，pg_verifybackup可以检测到由于存储问题或用户错误而经常出现的许多问题。备份验证分四个阶段进行。首先，pg_verifybackup 读取 backup_manifest 文件。如果该文件不存在、无法读取、格式不正确或无法根据其内部校验和进行验证，pg_verifybackup 将以致命错误终止。其次，pg_verifybackup 将尝试验证当前存储在磁盘上的数据文件是否与服务器打算发送的数据文件完全相同，下面将介绍一些例外情况。除了少数例外，额外和丢失的文件将被检测到。此步骤将忽略 postgresql.auto.conf、standby.signal 和 recovery.signal 的存在与否或对其的任何修改，因为预计这些文件可能是在备份过程中创建或修改的。它也不会抱怨目标目录中的 backup_manifest 文件或 pg_wal 中的任何内容，即使这些文件不会列在备份清单中。只检查文件；不验证目录的存在与否，除非间接验证：如果目录丢失，则它应该包含的任何文件也必然会丢失。
psql	psql是一个IvorySQL的基于终端的前端。它让你能交互式地键入查询，把它们发送给IvorySQL，并且查看查询结果。或者，输入可以来自于一个文件或者命令行参数。此外，psql还提供一些元命令和多种类似 shell 的特性来为编写脚本和自动化多种任务提供便利。
reindexdb	reindexdb是用于重建一个IvorySQL数据库中索引的工具。reindexdb是 SQL 命令 REINDEX的一个包装器。在通过这个工具和其他方法访问服务器来重索引数据库之间没有实质性的区别。
vacuumdb	vacuumdb是用于清理一个IvorySQL数据库的工具。vacuumdb也将产生由IvorySQL查询优化器所使用的内部统计信息。

服务器应用	initdb	initdb 创建一个新的IvorySQL数据库集簇。一个数据库集簇是由一个单一服务器实例管理的数据库的集合。一个数据库集簇的创建包括创建存放数据库数据的目录、生成共享目录表（属于整个集簇而不是任何特定数据库的表）并且创建 template1 和 postgres 数据库。当你后来创建一个新的数据库时，任何在 template1 数据库中的东西都会被复制（因此，任何已安装在 template1 中的东西都会被自动地复制到后来创建的每一个数据库中）。postgres 数据库是便于用户、工具和第三方应用使用的默认数据库。尽管 initdb 将尝试创建指定的数据目录，它可能没有权限（如果想要的数据目录的父目录被根用户拥有）。要在这样一种设置中初始化，作为 root 创建一个空数据目录，然后使用 chown 将该目录赋予给数据库用户账户，再然后 su 成为该数据库用户并运行 initdb。initdb 必须以将拥有该服务器进程的用户运行，因为该服务器需要访问 initdb 创建的文件和目录。因为该服务器不能作为 root 运行，你不能以 root 运行 initdb（事实上它会拒绝这样做）。由于安全原因，由 initdb 创建的新集簇默认将只能由集簇所有者访问。--allow-group-access 选项允许与集簇所有者同组的任何用户读取集簇中的文件。这对非特权用户执行备份很有用。initdb 初始化该数据库集簇的默认区域和字符集编码。当一个数据库被创建时，其字符集编码、排序顺序（LC_COLLATE）和字符集类（LC_CTYPE，例如大写、小写、数字）可以被独立设置。initdb 为 template1 数据库确定那些设置，它们将作为所有其他数据库的默认值。要修改默认排序顺序或字符集类，使用 --lc-collate 和 --lc-ctype 选项。除 C 或 POSIX 之外的排序顺序也有性能罚值。由于这些原因，在运行 initdb 时选择正确的区域很重要。余下的区域分类可以在服务器启动之后改变。你也可以使用 --locale 来为所有区域分类设置默认值，包括排序顺序和字符集类。所有服务器区域值（lc_*）可以通过 SHOW ALL 显示。要修改默认编码，使用 --encoding。
	pg_archivecleanup	pg_archivecleanup被设计用作 archive_cleanup_command 在作为后备服务器运行时来清理 WAL 文件归档。 pg_archivecleanup也可以被用作一个单独的程序来清理 WAL 文件归档。要配置一个后备服务器以使用pg_archivecleanup，把下面的内容放在 postgresql.conf 配置文件中：archive_cleanup_command = 'pg_archivecleanup archivelocation %r' 其中 archivelocation 是要从中移除 WAL 段文件的目录。当被用在 archive_cleanup_command中时，所有逻辑上在 %r 参数的值之前的 WAL 文件都将被从 archivelocation 移除。这能最小化需要被保留的文件数量，同时能保留崩溃后重启的能力。如果对于这台特定的后备服务器，archivelocation 是一个短暂需要的区域，使用这个参数就是合适的，但是当 archivelocation 要用作一个长期的 WAL 归档区域或者当多个后备服务器正在从这个归档位置恢复时，使用这个参数就不合适。当被用作一个单独的程序时，所有逻辑上在 oldestkeptwalfile 之前的 WAL 文件将被从 archivelocation 中移除。在这种模式中，如果指定了 .partial 或者 .backup 文件名，则 只有该文件前缀将被用作 oldestkeptwalfile。这种对 .backup 文件名的处理允许你移除所有在一个特定基础备份之前归档的 WAL 文件而不出错。
	pg_checksums	pg_checksums在IvorySQL集簇中检查、启用或禁用数据校验和。运行pg_checksums之前，必须彻底关闭服务器。验证校验和时，如果没有校验和错误，则退出状态为零，如果检测到至少一个校验和失败，则退出状态为非零。启用或禁用校验和时，如果操作失败，则退出状态为非零。验证校验和时，集簇中的每个文件都要被扫描。启用校验和时，集簇中的每个文件都会被原地重写。禁用校验和时，仅更新 pg_control 文件。
	pg_controldata	pg_controldata 打印在 initdb 期间初始化的信息，例如目录版本。它也显示关于预写式日志和检查点处理的信息。这种信息是集簇范围的，并且不针对任何一个数据库。这个工具只能由初始化集簇的用户运行，因为它要求对数据目录的读访问。你可以在命令行中指定数据目录，或者使用环境变量 PGDATA。这个工具支持选项 -V 和 --version，它们打印pg_controldata版本并退出。它也支持选项 -? 和 --help，它们输出支持的参数。

pg_ctl	pg_ctl是一个用于初始化IvorySQL数据库集簇，启动、停止或重启IvorySQL数据库服务器（ postgres ），或者显示一个正在运行服务器的状态的工具。尽管服务器可以被手工启动，pg_ctl包装了重定向日志输出以及正确地从终端和进程组脱离等任务。它也提供了方便的选项用来控制关闭。
pg_resetwal	pg_resetwal 会清除预写式日志（WAL）并且有选择地重置存储在 pg_control 文件中的一些其他控制信息。如果这些文件已经被损坏，某些时候就需要这个功能。当服务器由于这样的损坏而无法启动时，这应该被用作最后的手段。在运行这个命令之后，就可能可以启动服务器，但是记住数据库可能包含由于部分提交事务产生的不一致数据。你应当立刻转储你的数据、运行 initdb 并且重新载入。重新载入后，检查不一致并且根据需要修复之。这个工具只能被安装服务器的用户运行，因为它要求对数据目录的读写访问。出于安全原因，你必须在命令行中指定数据目录。 pg_resetwal 不使用环境变量 PGDATA 。如果 pg_resetwal 抱怨它无法为 pg_control 决定合法数据，你可以通过指定 -f （强制）选项强制它继续。在这种情况下，丢失的数据将被替换为看似合理的值。可以期望大部分域是匹配的，但是下一个 OID、下一个事务 ID 和纪元、下一个多事务 ID 和偏移以及 WAL 开始位置域可能还是需要人工协助。这些域可以使用下面讨论的选项设置。如果你不能为所有这些域决定正确的值， -f 还是可以被使用，但是恢复的数据库还是值得怀疑：一次立即的转储和重新载入是势在必行的。在你转储之前不要在该数据库中执行任何数据修改操作，因为任何这样的动作都可能使破坏更严重。
pg_rewind	pg_rewind是用于在集簇的时间线分叉以后，同步一个 IvorySQL 集簇和同一集簇的另一份拷贝的工具。一种典型的场景是在失效后让一个旧的主服务器重新上线，同时有一个后备机跟随着新的主机。成功回放后，目标数据目录的状态类似于源数据目录的基本备份。与进行新的基本备份或使用rsync等工具不同，pg_rewind不需要比较或复制集群中未更改的关系块。仅复制现有关系文件中更改的块；所有其他文件（包括新的关系文件、配置文件和WAL段）都将被完整复制。因此，当数据库很大并且集群之间只有一小部分块不同时，倒带操作比其他方法要快得多。pg_rewind检查源集簇和目标集簇的时间线历史来判断它们在哪一点分叉，并且期望在目标集簇的 pg_wal 目录中找到 WAL 来返回到分叉点。分叉点可能会在目标时间线、源时间线或者它们的共同祖先上找到。在典型的失效场景中，目标集簇在分叉后很快就被关闭，这不是问题，但是如果目标集簇在分叉后已经运行了很长时间，旧的 WAL 文件可能已经不存在了。在这样的情况下，您可以手动将它们从WAL存档复制到 pg_wal 目录，或使用 -c 选项运行pg_rewind以自动从WAL存档检索它们。pg_rewind的使用并不限于失效的场景，例如一个后备服务器可能被提升、运行一些写事务，然后被倒回再次成为一个后备。在运行pg_rewind之后，需要完成WAL重放以使数据目录处于一致状态。当目标服务器再次启动时，它将进入归档恢复，并从分歧点之前的最后一个检查点重放源服务器中生成的所有 WAL。当pg_rewind被运行时有某些 WAL 在源服务器上不可用，并且因此无法被pg_rewind会话所复制，则在目标服务器被启动时必须让这些 WAL 可用。这可以通过在目标数据目录中创建一个 recovery.signal 文件并且在 postgresql.conf 中配置适合的 restore_command 来实现。pg_rewind要求目标服务器在 postgresql.conf 中启用了 wal_log_hints 选项，或者在用initdb初始化集簇时启用了数据校验。目前默认情况下这两者都没有被打开。 http://www.postgresql.org/docs/17/runtime-config-wal.html#GUC-FULL-PAGE-WRITES[full_page_writes] 也必须被设置为 on ，这是默认的。
pg_test_fsync	pg_test_fsync是想告诉你在特定的系统上，哪一种 wal_sync_method 最快，还可以在发生认定的 I/O 问题时提供诊断信息。不过，pg_test_fsync 显示的区别可能不会在真实的数据库吞吐量上产生显著的区别，特别是由于很多数据库服务器被它们的预写日志限制了速度。pg_test_fsync为 wal_sync_method 报告以微秒计的平均文件同步操作时间，也能被用来提示用于优化 commit_delay 值的方法。
pg_test_timing	pg_test_timing是一种度量在你的系统上计时开销以及确认系统时间绝不会回退的工具。收集计时数据很慢的系统会给出不太准确的 EXPLAIN ANALYZE 结果。

pg_upgrade	pg_upgrade允许存储在IvorySQL数据文件中的数据被升级到一个较晚的IvorySQL主版本而无需进行主版本升级通常所需的数据转储/重载。对于次版本升级则不需要这个程序。IvorySQL主发行版本通常会加入新的特性，这些新特性常常会更改系统表的布局，但是内部数据存储格式很少会改变。pg_upgrade使用这一事实来通过创建新系统表并且重用旧的用户数据文件来执行快速升级。如果未来的主发行版本更改了数据存储格式，导致旧数据格式不可读，那么pg_upgrade将无法用于此类升级。（社区将努力避免这种情况）。pg_upgrade会尽力（例如通过检查兼容的编译时设置）确保新旧集簇在二进制上也是兼容的，包括 32/64 位二进制。保持外部模块也是二进制兼容的也很重要，不过pg_upgrade无法检查这一点。
pg_waldump	pg_waldump 显示预写式日志（WAL），它主要用于调试或者教育目的。这个工具只能由安装该服务器的用户运行，因为它要求对数据目录的只读访问。
postgres	postgres 是IvorySQL数据库服务器。一个客户端应用为了能访问一个数据库，它会（通过一个网络或者本地）连接到一个运行着的 postgres 实例。该 postgres 实例接着会开始一个独立的服务器进程来处理该连接。一个 postgres 实例总是管理正好一个数据库集簇的数据。一个数据库集簇是一个数据库的集合，它们被存储在一个共同的文件系统位置（“数据区”）上。一个系统上可以同时运行多个 postgres 实例，只要它们使用不同的数据区和不同的通信端口（见下文）。postgres 启动时需要知道数据区的位置，该位置必须通过 -D 选项或 PGDATA 环境变量指定，对此是没有默认值的。通常，-D 或 PGDATA 会直接指向由 initdb 创建的数据区目录。默认情况下，postgres 会在前台启动并将日志消息打印到标准错误流。但在实际应用中，postgres 应当作为一个后台进程启动，而且多数是在系统启动时自动启动。postgres 还能在单用户模式中被调用。这种模式的主要用途是在启动过程中由 initdb 使用。有时候它也被用于调试或者灾难性恢复。注意，运行一个单用户模式服务器并不真正地适合调试服务器，因为不会发生实际的进程间通信和锁定。当从 shell 中调用单用户模式时，用户可以输入查询并且结果会被以一种更适合开发者阅读（不适合普通用户）的形式打印在屏幕上。在单用户模式中，会话用户将被设置为 ID 为 1 的用户，并且这个用户会被隐式地赋予超级用户权限。该用户不必实际存在，因此单用户模式运行可以被用来对某些意外损坏的系统目录进行手工恢复。

客户端工具

clusterdb

大纲

```
clusterdb [connection-option...] [ --verbose | -v ] [ --table | -t table ] ... [dbname]
```

```
clusterdb` [*`connection-option`*...] [ `--verbose` | `-v` ] `--all` | `-a
```

选项

clusterdb接受下列命令行参数：

- **-a --all**

聚簇所有数据库。

- **[-d] dbname [--dbname=] dbname**

当不使用 **-a/ --all** 时，指定要被聚簇的数据库名称。如果数据库名称未指定，则从环境变量 **PGDATABASE** 中读取数据库名称。如果该环境变量也没有被设置，则使用为连接指定的用户名作数据库名。 **dbname** 可以是 **connection string**。如果是这样，连接时的字符串参数将覆盖所有冲突的命令行选项。

- **-e --echo**

回显clusterdb生成并发送给服务器的命令。

- **-q --quiet**

不显示进度消息。

- **-t table --table=table**

只聚簇 **table**。可以通过写多个 **-t** 开关来聚簇多个表。

- **-v --verbose**

在处理期间打印详细信息。

- **-V --version**

打印clusterdb版本并退出。

- **-? --help**

显示关于clusterdb命令行参数的帮助并退出。clusterdb也接受下列命令行参数用于连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制clusterdb在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，clusterdb将自动提示要求一个口令。但是，clusterdb将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

- **--maintenance-db= dbname**

当使用 **-a/ --all** 时，指定要连接到的数据库名称来发现哪些其他数据库应该被聚簇。如果没有指定，将使用 **postgres** 数据库。而如果它也不存在，将使用 **template1**。这可以是 **connection string**。如果是这样，连接时的字符串参数将覆盖所有冲突的命令行选项。另外，连接到其他数据库时，除了数据库名字本身其他的连接时字符串参数将被重新使用。

环境

- **PGDATABASE PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可选的值为 ``always`` , ``auto`` , ``never``

和大部分其他IvorySQL工具相似, 这个工具也使用libpq支持的环境变量。

诊断

在有困难时, 可以在 [CLUSTER](#) 和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样, 任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

例子

要聚簇数据库 **test**:

```
$ clusterdb test
```

要聚簇在数据库 **xyzyz** 中的一个表 **foo**:

```
$ clusterdb --table=foo xyzyz
```

createdb

大纲

createdb [**connection-option**...] [**option**...] [**dbname** [**description**]]

选项

createdb接受下列命令行参数:

- **dbname**

指定要被创建的数据库名。该名称必须在这个集簇中所有IvorySQL数据库中唯一。默认是创建一个与当前系统用户同名的数据库。

- **description**

指定与新创建的数据库相关联的一段注释。

- **-D tablespace --tablespace=tablespace**

指定该数据库的默认表空间 (这个名称被当做一个双引号引用的标识符处理) 。

- **-e --echo**

回显createdb生成并发送到服务器的命令。

- **-E encoding --encoding=encoding**

指定要在这个数据库中使用的字符编码模式。

- **-l locale --locale=locale**

指定要在这个数据库中使用的区域。这等效于同时指定 `--lc-collate` 和 `--lc-ctype`。

- **--lc-collate=locale**

指定要在这个数据库中使用的 LC_COLLATE 设置。

- **--lc-ctype=locale**

指定要在这个数据库中使用的 LC_CTYPE 设置。

- **-O owner --owner=owner**

指定拥有这个新数据库的数据库用户（这个名称被当做一个双引号引用的标识符处理）。

- **-T template --template=template**

指定用于创建这个数据库的模板数据库（这个名称被当做一个双引号引用的标识符处理）。

- **-V --version**

打印createdb版本并退出。

- **-? --help**

显示关于createdb命令行参数的帮助并退出。选项 **-D**、**-l**、**-E**、**-O**和 **-T** 对应于底层 SQL 命令 **CREATE DATABASE** 的选项，关于这些选项的信息可见该命令的内容。

createdb也接受下列命令行参数用于连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制createdb在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，createdb将自动提示要求一个口令。但是，createdb将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

- **--maintenance-db=dbname**

指定要连接到来发现哪些其他数据库应该被聚簇的数据库名。如果没有指定，将使用 **postgres** 数据库。而如果它也不存在（或者如果它就是要创建新数据库的名称），将使用 **template1**。这可以是 [connection string](#)。如果是这样，连接时字符串参数将覆盖所有冲突的命令行选项。

环境

- **PGDATABASE**

如果被设置，就是要创建的数据库名，除非在命令行中覆盖。

- **PGHOST PGPORT PGUSER**

默认连接参数。如果没有在命令行或 **PGDATABASE** 指定要创建的数据库名，**PGUSER** 也决定要创建的数据库名。

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**，**auto** 和 **never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [CREATE DATABASE](#)和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

例子

要使用默认数据库服务器创建数据库 **demo**：

```
$ createdb demo
```

要在主机 **eden**、端口 5000 上使用 **template0** 模板数据库创建数据库 **demo**，这里是命令行命令和底层SQL命令：

```
$ createdb -p 5000 -h eden -T template0 -e demo
CREATE DATABASE demo TEMPLATE template0;
```

createuser

createuser — 定义一个新的IvorySQL用户账户

大纲

createuser [**connection-option**...] [**option**...] [**username**]

选项

createuser接受下列命令行参数：

- **username**

指定要被创建的IvorySQL用户的名称。这个名称必须与这个IvorySQL安装中所有现存角色不同。

- **-c number --connection-limit=number**

为该新用户设置一个最大连接数。默认值为不设任何限制。

- **-d --createdb**

新用户将被允许创建数据库。

- **-D --no-createdb**

新用户将不被允许创建数据库。这是默认值。

- **-e --echo**

回显createuser生成并发送给服务器的命令。

- **-E --encrypted**

此选项已过时，但为了实现向后兼容仍然接受。

- **-g role --role=role**

指定一个角色，这个角色将立即加入其中成为其成员。如果要把这个角色加入到多个角色中作为成员，可以写多个 **-g** 开关。

- **-i --inherit**

新角色将自动继承把它作为成员的角色的特权。这是默认值。

- **-I --no-inherit**

新角色将不会自动继承把它作为成员的角色的特权。

- **--interactive**

如果在命令行没有指定用户名，提示要求用户名，并且在命令行没有指定选项 **-d / -D**、**-r / -R**、**-s / -S** 时也提示。

- **-l --login**

新用户将被允许登入（即，该用户名能被用作初始会话用户标识符）。这是默认值。

- **-L --no-login**

新用户将不被允许登入（一个没有登录特权的角色仍然可以作为管理数据库权限的方式而存在）。

- **-P --pwprompt**

如果给定，createuser将发出一个提示要求新用户的口令。如果你没有计划使用口令认证，这就不是必须的。

- **-r --createrole**

新用户将被允许创建新的角色（即，这个用户将具有 **CREATEROLE** 特权）。

- **-R --no-createrole**

新用户将不被允许创建新角色。这是默认值。

- **-s --superuser**

新用户将成为一个超级用户。

- **-S --no-superuser**

新用户将不会成为一个超级用户。这是默认值。

- **-V --version**

打印createuser版本并退出。

- **--replication**

新用户将具有 **REPLICATION** 特权，这在 [CREATE ROLE](#) 的文档中有更完整的描述。

- **--no-replication**

新用户将不具有 **REPLICATION** 特权，这在 [CREATE ROLE](#) 的文档中有更完整的描述。

- **-? --help**

显示有关createuser命令行参数的帮助并退出。

createuser也接受下列命令行参数作为连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接（不是要创建的用户名）。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制createuser在连接到一个数据库之前提示要求一个口令（用来连接到服务器，而不是新用户的口令）。这个选项不是必不可少的，因为如果服务器要求口令认证，createuser将自动提示要求一个口令。但是，createuser将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

环境

- **PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**，**auto** 和 **never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [CREATE ROLE](#) 和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

例子

要在默认数据库服务器上创建一个用户 **joe**：

```
$ createuser joe
```

要在默认数据库服务器上创建一个用户 **joe** 并提示要求一些额外属性：

```
$ createuser --interactive joe
Shall the new role be a superuser? (y/n) n
Shall the new role be allowed to create databases? (y/n) n
Shall the new role be allowed to create more new roles? (y/n) n
```

要使用在主机 **eden**、端口 5000 上的服务器创建同一个用户 **joe**，并带有显式指定的属性，看看下面的命令：

```
$ createuser -h eden -p 5000 -S -D -R -e joe
CREATE ROLE joe NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT LOGIN;
```

要创建用户 **joe** 为一个超级用户并且立刻分配一个口令：

```
$ createuser -P -s -e joe
Enter password for new role: xyzzy
Enter it again: xyzzy
CREATE ROLE joe PASSWORD 'md5b5f5ba1a423792b526f799ae4eb3d59e' SUPERUSER CREATEDB
CREATEROLE INHERIT LOGIN;
```

在上面的例子中，在录入新口令时新口令并没有真正地被回显，但是为了清晰，我们特意把它列了出来。如你所见，该口令在被发送给客户端之前会被加密。

dropdb

dropdb — 移除一个IvorySQL数据库

大纲

dropdb [**connection-option**...] [**option**...] **dbname**

选项

dropdb接受下列命令行参数：

- **dbname**

指定要被移除的数据库的名字。

- **-e --echo**

回显dropdb生成并发送给服务器的命令。

- **-f --force**

在删除目标数据库之前，尝试终止与该数据库的所有现有连接。有关此选项的详细信息，请参见 [DROP DATABASE](#)。

- **-i --interactive**

在做任何破坏性的工作之前发出一个验证提示。

- **-V --version**

打印dropdb版本并退出。

- **--if-exists**

如果数据库不存在也不抛出一个错误。在这种情况下会发出一个提示。

- **-? --help**

显示有关dropdb命令行参数的帮助并退出。

dropdb也接受下列命令行参数作为连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作Unix域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的TCP端口或本地Unix域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

不发出口令提示。如果服务器要求口令认证并且没有可用的口令（例如一个 **.pgpass** 文件），那么连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制dropdb在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，dropdb将自动提示要求一个口令。但是，dropdb将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

- **--maintenance-db=dbname**

指定一个数据库的名称，将连接到这个数据库以便删除目标数据库。如果没有指定，将使用 **postgres** 数据库。而如果它也不存在（或者它就是正在被删除的目标数据库），将使用 **template1**。这个值可以是一个 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何有冲突的命令行选项。

环境

- **PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto** 以及 **never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [DROP DATABASE](#) 和 [psql](#) 中找找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

示例

要在默认数据库服务器上毁掉数据库 **demo**：

```
$ dropdb demo
```

要使用在主机 **eden**、端口5000上的服务器中毁掉数据库 **demo**，并带有验证和回显，看看下面的命令：

```
$ dropdb -p 5000 -h eden -i -e demo
Database "demo" will be permanently deleted.
Are you sure? (y/n) y
DROP DATABASE demo;
```

dropuser

dropuser — 移除一个IvorySQL用户账户

大纲

dropuser [**connection-option**...] [**option**...] [**username**]

选项

dropuser接受下列命令行参数：

- **username**

指定要移除的IvorySQL用户的名字。如果没有在命令行指定并且使用了 **-i / --interactive** 选项，你将被提醒要求一个用户名。

- **-e --echo**

回显dropuser生成并发送给服务器的命令。

- **-i --interactive**

在实际移除该用户之前提示要求确认，并且在没有在命令行指定用户名提示要求一个用户名。

- **-V --version**

打印dropuser版本并退出。

- **--if-exists**

如果用户不存在时不要抛出一个错误。在这种情况下将发出一个提示。

- **-? --help**

显示有关dropuser命令行参数的帮助并退出。

dropuser也接受下列命令行参数作为连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作Unix域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的TCP端口或本地Unix域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

不发出口令提示。如果服务器要求口令认证并且没有可用的口令（例如一个 **.pgpass** 文件），那么连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制dropuser在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，dropuser将自动提示要求一个口令。但是，dropuser将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

环境

- **PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto** 以及 **never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [DROP ROLE](#) 和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

示例

要从默认数据库服务器移除用户 **joe**：

```
$ dropuser joe
```

要使用在主机 **eden**、端口5000上的服务器移除用户 **joe**，并带有验证和回显，可使用下面的命令：

```
$ dropuser -p 5000 -h eden -i -e joe
Role "joe" will be permanently removed.
Are you sure? (y/n) y
DROP ROLE joe;
```

ecpg

ecpg — 嵌入式 SQL C 预处理器

大纲

ecpg [**option**...] **file**...

- **-c**

自动从 SQL 代码生成确定的 C 代码。当前，这对 **EXEC SQL TYPE** 起效。

- **-C mode**

设置一个兼容性模式。**mode** 可以是 **INFORMIX**, **INFORMIX_SE** 或 **ORACLE**。

- **-D symbol**

定义一个 C 预处理器符号。

- **-h**

处理头文件。指定此选项后，输出文件扩展名变为 **.h** 而不是 **.c**，默认输入文件扩展名为 **.pgh** 而不是 **.pgc**。此外，将强制启用 **-c** 选项。

- **-i**

分析系统也包括文件。

- **-I directory**

指定一个额外的包括路径，用来寻找通过 **EXEC SQL INCLUDE** 包括的文件。默认值是 **.**（当前目录）、**/usr/local/include**、在编译时定义的 IvorySQL 包括目录（默认：**/usr/local/pgsql/include**）以及 **/usr/include**。

- **-o `filename`**

指定 **ecpg** 应该将它的所有输出写到给定的 **filename**。写 **-o-** 将所有输出发送到标准输出。

- **-r `option`**

选择运行时行为。**option** 可以是下列之一：**no_indicator** 不使用指示器而使用特殊值来表示空值。历史上曾有数据库使用这种方法。**prepare** 在使用所有语句之前准备它们。**libecpg** 将保持一个预备语句的缓冲并当语句再被执行时重用该语句。如果缓冲满了，**libecpg** 将释放最少使用的语句。**`questionmarks`** 为兼容性原因允许使用问号作为占位符。在很久以前这被用作默认值。

- **-t**

打开事务的自动提交。在这种模式下，每一个 SQL 命令会被自动提交，除非它位于一个显式事务块中。在默认模式中，命令只有当 **EXEC SQL COMMIT** 被发出时才被提交。

- **-v**

打印额外信息，包括版本和“包括”路径。

- **--version**

打印 **ecpg** 版本并退出。

- **-? --help**

显示关于ecpg命令行参数的帮助并退出。

注解

在编译预处理好的 C

代码文件时，编译器需要能够找到IvorySQL包括目录中的ECPG头文件。因此，在调用编译器时，你可能必须使用 **-I** 选项（例如， **-I/usr/local/pgsql/include**）。

使用带有嵌入式 SQL 的 C 代码的程序必须被链接到 **libecpg** 库，例如使用链接器选项 **-L/usr/local/pgsql/lib -lecp**g。

适合于安装的这些目录的值可以使用 `pg_config` 找到。

例子

如果你有一个名为 **prog1.pgc** 的嵌入式 SQL C 源文件，你可以使用下列命令序列创建一个可执行程序：

```
ecpg prog1.pgc
cc -I/usr/local/pgsql/include -c prog1.c
cc -o prog1 prog1.o -L/usr/local/pgsql/lib -lecp
```

pg_amcheck

pg_amcheck — 在一个或多个IvorySQL数据库中检查损坏

大纲

pg_amcheck [**option**...] [**dbname**]

选项

以下命令行选项控制要检查的内容：

- **-a --all**

检查所有数据库，但通过 `--exclude-database`` 排除的数据库除外。

- **-d pattern --database=pattern**

检查与指定的 **pattern**匹配的数据库，但 `--exclude-database`` 排除的数据库除外。可以多次指定此选项。

- **-D pattern --exclude-database=pattern**

排除与给定的 **pattern**匹配的数据库。可以多次指定此选项。

- **-i pattern --index=pattern**

检查与指定的 **pattern**匹配的索引，除非它们被排除在外。可以多次指定此选项。这类似于 **--relation** 选项，只是它只适用于索引，而不适用于表。

- **-I pattern --exclude-index=pattern**

排除与指定的 **pattern**匹配的索引。可以多次指定此选项。这类似于 **--exclude-relation** 选项，只是它只适用于索引，而不适用于表。

- **-r pattern --relation=pattern**

检查与指定的

pattern匹配的关系，除非它们被排除在外。可以多次指定此选项。模式可能是非限定的，例如 **myrel***，或者它们可能是模式限定的，如 **myschema*.myrel*** 或数据库限定和模式限定，例如 **mydb.myschema*.myrel***。数据库限定模式将向要检查的数据库列表中添加匹配的数据库。

- **-R pattern --exclude-relation=pattern**

排除与指定的 **pattern** 匹配的关系。可以多次指定此选项。与 `--relation`` 一样, **pattern** 可以是非限定的、模式限定的或数据库和模式限定的。

- **-s pattern --schema=pattern**

检查模式中指定的

pattern匹配的表和索引，除非另有排除。可以多次指定此选项。要仅选择与特定模式匹配的模式中的表，请考虑使用类似 **--table=SCHEMAPAT.* --no-dependent-indexes**。要仅选择索引，请考虑使用类似 **--index=SCHEMAPAT.** 的方法。模式模式可以是数据库限定的。例如，你可以编写 **--schema=mydb.myschema*** 在数据库中选择匹配 **mydb*** 的模式。

- **-S pattern --exclude-schema=pattern**

排除模式中指定的 **pattern** 匹配的表和索引。可以多次指定此选项。与 `--schema`` 一样，模式可以是数据库限定的。

- **-t pattern --table=pattern**

检查与指定的 **pattern** 匹配的表，除非它们被排除在外。可以多次指定此选项。这类似于 `--relation`` 选项，只是它只适用于表，而不适用于索引。

- **-T pattern --exclude-table=pattern**

排除与指定的 **pattern** 匹配的表。可以多次指定此选项。这类似于 **--exclude-relation** 选项，只是它只适用于表，而不适用于索引。

- **--no-dependent-indexes**

默认情况下，如果选中了一个表，则该表的任何btree索引也将被选中，即使它们没有被诸如 **--index** 或 **--relation** 之类的选项显式选择。此选项将抑制该行为。

- **--no-dependent-toast**

默认情况下，如果选中了一个表，则也将检查它的toast表（如果有），即使它没有通过诸如 **--table** 或 **--relation** 之类的选项显式选择。此选项将抑制该行为。

- **--no-strict-names**

默认情况下，如果 **--database**，**--table**，**--index**，或 **--relation** 的参数不匹配任何对象，则这是一个致命错误。此选项将该错误降级为警告。

以下命令行选项用于控制表的检查：

- **--exclude-toast-pointers**

默认情况下，每当在表中遇到toast指针时，都会执行查找以确保它引用toast表中明显有效的条目。这些检查可能非常慢，可以使用此选项跳过这些检查。

- **--on-error-stop**

在发现损坏的表的第一页上报告所有损坏后，停止处理该表关系，并转到下一个表或索引。请注意，索引检查总是在第一个损坏页之后停止。此选项仅与表关系相关。

- **--skip=option**

如果给定 **all-frozen**，表损坏检查将跳过所有表中标记为全部冻结的页面。如果给定了 **all-**

visible，则表损坏检查将跳过所有表中标记为所有可见的页面。默认情况下，不跳过任何页面。这可以指定为 **none**，但由于这是默认值，因此无需提及。

- **--startblock=block**

从指定的块号开始检查。如果正在检查的表关系的块数少于此数，则会发生错误。此选项不适用于索引，可能仅在检查单个表关系时有用。请参阅 **--endblock** 了解更多注意事项。

- **--endblock='block'**

在指定的块号结束检查。如果正在检查的表关系的块数少于此数，则会发生错误。此选项不适用于索引，可能仅在检查单个表关系时有用。如果同时选中常规表和toast表，则此选项将同时适用于这两个表，但在验证toast指针时，仍然可以访问编号更高的toast块，除非使用 **--exclude-toast-pointers** 抑制。

以下命令行选项用来控制B树索引的检查：

- **--heapallindexed**

对于每个选中的索引，使用 **amcheck's heapallindexed** 选项验证索引中是否存在作为索引元组的所有堆元组。

- **--parent-check**

对于检查的每个btree索引，使用 **amcheck** 的 **bt_index_parent_check** 函数，该函数在索引检查期间对父/子关系执行其他检查。默认情况是使用 **amcheck** 的 **bt_index_check** 函数，但请注意，使用 **--rootdescend** 选项会隐式选择 **bt_index_parent_check**。

- **--rootdescend**

对于每个选中的索引，通过使用 **amcheck's rootdescend** 选项从根页面对每个元组执行新搜索，在叶级重新查找元组。使用此选项也会隐式选择 **--parent-check** 选项。这种形式的验证最初是为了帮助开发btree索引功能而编写的。它在帮助检测实践中发生的损坏类型方面可能用处有限，甚至毫无用处。它还可能导致损坏检查花费相当长的时间，并消耗服务器上相当多的资源。

警告

当指定 **--parent-check** 选项或 **--rootdescend** 选项时，对B树索引执行的额外检查需要相对强的关系级锁。这些检查是唯一会阻止 **INSERT**，**UPDATE**，和 **DELETE** 命令并发数据修改的检查。

以下命令行选项用来控制与服务器的连接：

- **-h hostname --host=hostname**

指定运行服务器的计算机的主机名。如果该值以斜杠开头，它将用作Unix域套接字的目录。

- **-p port --port=port**

指定服务器正在侦听连接的TCP端口或本地Unix域套接字文件扩展名。

- **-U --username=username**

连接的用户名。

- **-w --no-password**

永远不要发出密码提示。如果服务器需要密码身份验证，并且密码无法通过其他方式（如 **.pgpass** 文件，连接尝试将失败。在没有用户输入密码的批处理作业和脚本中，此选项非常有用。

- **-W --password**

在连接到数据库之前，强制pg_amcheck提示输入密码。此选项从来都不是必需的，因为如果服务器要求密

码身份验证，pg_amcheck将自动提示输入密码。但是，pg_amcheck将浪费一次连接尝试，以发现服务器需要密码。在某些情况下，键入 **-W** 以避免额外的连接尝试是值得的。

- **--maintenance-db=dbname**

指定用于发现要检查的数据库列表的数据库或 [连接字符串](#)。如果既不使用 **--all** 也不使用任何包含数据库模式的选项，则不需要这种连接，并且该选项不起任何作用。否则，连接到正在检查的数据库时，也将使用此选项值中包含的数据库名称以外的任何连接字符串参数。如果省略此选项，则默认值为 **postgres**，或如果失败，则为 **template1**。

其它允许的选项：

- **-e --echo**

回显发送到服务器的所有SQL。

- **-j num --jobs=num**

使用 **num**

个到服务器的并发连接，或每个要检查的对象使用一个连接，以较小者为准。默认情况是使用单个连接。

- **-P --progress**

显示进度信息。进度信息包括已完成检查的关系数以及这些关系的总大小。它还包括最终要检查的关系的总数，以及这些关系的估计大小。

- **-v --verbose**

打印更多消息。特别是，这将为每个正在检查的关系打印一条消息，并将增加服务器错误的详细程度。

- **-V --version**

打印pg_amcheck版本并退出。

- **--install-missing --install-missing='schema'**

安装检查数据库所需的任何缺少的扩展。如果尚未安装，每个扩展的对象将安装到给定的 **schema** 中，或者如果未指定则安装到 **pg_catalog** 模式中。目前，唯一需要的扩展是 **amcheck**。

- **-? --help**

显示有关pg_amcheck命令行参数的帮助，然后退出。

pg_basebackup

pg_basebackup — 获得一个IvorySQL集簇的一个基础备份

大纲

pg_basebackup [option...]

选项

下列命令行选项控制输出的位置和格式：

- **-D directory --pgdata=directory**

设置目标目录以将输出写入。如果该目录不存在，pg_basebackup将创建该目录（以及所有丢失的父目录）。如果已经存在，则必须为空。当备份处于 tar 模式中，目标目录可以被指定为 **-**（破折号），从而将 tar 文件写入 **stdout**。这个选项是必需的。

- **-F format --format=format**

为输出选择格式。**format** 可以是下列之一：**p plain**

把输出写成平面文件，使用和源服务器数据目录和表空间相同的布局。

当集簇没有额外表空间时，整个数据库将被放在目标目录中。

如果集簇包含额外的表空间，主数据目录将被放置在目标目录中，但是所有其他表空间将被放在它们位于源服务器上的相同的绝对路径中。(参见 **--tablespace-mapping** 来更改。)这是默认格式。**t tar**

将输出作为tar文件写入目标目录中。主数据目录的内容将被写入名为 **base.tar**

的文件，而其他表空间将被写入以该表空间的OID命名的单独的tar文件。如果将目标目录指定为 - (破折号)，则tar内容将被写入标准输出，该标准输出适用于管道传输至gzip (例如)。

只有当集簇没有额外表空间并且没有使用WAL流时才允许这样做。

- **-R --write-recovery-conf**

创建一个 **standby.signal** 文件，并将连接设置附加到目标目录 (或使用tar格式的基本存档文件中) 的 **postgresql.auto.conf** 文件中。这样可以简化使用备份结果设置备用服务器的过程。

postgresql.auto.conf

文件将记录连接设置 (如果有) 以及pg_basebackup所使用的复制槽，这样流复制后面就会使用相同的设置。

- **-T olddir=newdir --tablespace-mapping=olddir=newdir**

在备份期间将目录 **olddir** 中的表空间重定位到 **newdir** 中。为使之有效，**olddir**

必须与源服务器上定义的表空间的路径规范完全匹配。(但如果备份中没有包含 **olddir**

中的表空间也不是错误)。同时，**newdir** 是接收主机文件系统中的目录。与主目标目录一样，**newdir**

不必已经存在，但是如果确实存在，则必须为空。**olddir** 和 **newdir**

必须是绝对路径。如果一个路径凑巧包含了一个 =

符号，可用反斜线对它转义。对于多个表空间可以多次使用这个选项。如果以这种方法重定位一个表空间，主数据目录中的符号链接会被更新或指向新位置。因此新数据目录已经可以被一个所有表空间位于更新后位置的新服务器实例使用。目前，这个选项仅以普通输出格式工作，如果选择了tar格式，请忽略它。

- **--waldir=waldir**

指定用于预写式日志目录的位置。**waldir**

必须是绝对路径。只有当备份是平面文件模式时才能指定事务日志目录。

设置要写入WAL (预写日志) 文件的目录。默认情况下，WAL文件将放置在目标目录的 **pg_wal**

子目录中，但是此选项可用于将它们放置在其他位置。**waldir** 必须是绝对路径。与主目标目录一样，**waldir** 不必已经存在，但是如果确实存在，则必须为空。只有当备份是平面文件模式时才可以指定此选项。

- **-X method --wal-method=method**

在备份中包括所需的WAL (预写式日志) 文件。这包括所有在备份期间产生的预写式日志。除非指定了方法 **none**，可以在目标目录中启动postmaster而无需参考日志归档，从而使输出成为完全独立的备份。支持下列

收集预写式日志的 **方法**：**n none** 不要在备份中包括预写式日志。**f fetch**

在备份末尾收集预写式日志文件。因此，有必要把源服务器的 **wal_keep_size**

参数设置得足够高，这样在备份末尾之前需要的日志数据不会被移除。如果所需的日志数据在传输之前已被回收，则备份将失败并且无法使用。如果使用tar格式，预写式日志文件将被包含在 **base.tar** 文件。**s stream**

在进行备份时，流式传输预写式日志数据。将开启一个到服务器的第二连接并且在运行备份时并行开始流传输预写式日志。因此，它将需要两个复制连接，而不仅仅是一个。

只要客户端可以跟上预写式日志数据，使用此方法就不需要在源服务器上保存额外的预写式日志。如果使用tar格式，预写式日志文件被写入到一个单独的名为 **pg_wal.tar**

的文件 (如果服务器的版本超过10，该文件将被命名为 **pg_xlog.tar**)。这个值是默认值。

- **-z --gzip**

启用对tar文件输出的gzip压缩，使用默认的压缩级别。只有使用tar格式时压缩才可用，并且会在所有tar文件名后面自动加上后缀 **.gz**。

- **-Z level --compress=level**

启用对tar文件输出的gzip压缩，并且制定压缩级别 (0 到 9，0 是不压缩，9 是最佳压缩)。只有使用tar格式时压缩才可用，并且会在所有tar文件名后面自动加上后缀 **.gz**。

下列命令行选项控制备份的生成和程序的调用：

- **-c fast|spread --checkpoint=fast|spread**

将检查点模式设置为 fast（立刻）或 spread（默认）。

- **-C --create-slot**

指定在启动备份之前应创建由 **--slot** 选项命名的复制插槽。如果插槽已存在，则会引发错误。

- **-l label --label=label**

为备份设置标签。如果没有指定，将使用一个默认值 “**pg_basebackup base backup**”。

- **-n --no-clean**

默认情况下，当 **pg_basebackup**

因为一个错误而中止时，它会把它意识到无法完成该工作之前已经创建的目录（例如目标目录和预写式日志目录）都移除。这个选项可以禁止这种清洗，因此可以用于调试。注意不管哪一种方式都不会清除表空间目录。

- **-N --no-sync**

默认情况下，**pg_basebackup** 将等待所有文件被安全地写到磁盘上。这个选项导致 **pg_basebackup**

不做这种等待就返回，这样会更快一些，但是也意味着后续发生的操作系统崩溃可能会使得这个基础备份损坏。通常这个选项对测试比较有用，在创建生产安装时不应该使用。

- **-P --progress**

启用进度报告。启用这个选项将在备份期间发表一个大致的进度报告。由于数据库可能在备份期间改变，这仅仅是一种近似并且可能不会刚好在 **100%** 结束。特别地，当 WAL 日志被包括在备份中时，总数据量无法预先估计，并且在这种情况下估计的目标尺寸会在它经过不带 WAL 的总估计后增加。

- **-r rate --max-rate=rate**

设置从源服务器收集数据的最大传输速率。这有助于限制 **pg_basebackup** 对服务器的影响。值以每秒千字节为单位。使用后缀 **M** 表示每秒兆字节数。后缀 **k** 也可以接受，没有任何影响。有效值介于每秒32千字节和每秒1024兆字节之间。此选项始终影响数据目录的传输。只有收集方法为 **fetch** 时，才会影响WAL文件的传输。

- **-S slotname --slot=slotname**

这个选项仅能与 **-X stream**

一起使用。它导致WAL流使用指定的复制槽。如果该基础备份的目的是被用作一台使用复制槽的流复制后备，则它应该使用与 **primary_slot_name** 中相同的复制槽名称。这可以确保主服务器不会移除位于该基础备份结束与新备用数据库上流复制开始之间产生的任何所需的WAL数据。指定的复制槽必须已经存在，除非同时使用了选项 **-C**。如果这个选项没有被指定并且服务器支持临时复制槽（版本10以后），则会自动使用一个临时复制槽来进行WAL流。

- **-v --verbose**

启用冗长模式。将在启动和关闭期间输出一些额外步骤，并且如果进度报告也被启用，还会显示当前正在被处理的确切文件名。

- **--manifest-checksums=algorithm**

指定应用于备份清单中包含的每个文件的校验和算法。目前，可用的算法有 **`NONE`**、**CRC32C**、**SHA224**、**SHA256**、**SHA384** 和 **SHA512**。默认值为 **`CRC32C`**。如果选择了 **`NONE`**，备份清单将不包含任何校验和。否则，它将包含备份中使用指定算法的每个文件的校验和。此外，清单将始终包含自身内容的 **`SHA256`** 校验和。**`SHA`** 算法比 **`CRC32C`** 算法占用大量CPU，因此选择其中一种算法可能会增加完成备份所需的时间。使用SHA散列函数可为希望验证备份是否遭到篡改的用户提供每个文件的加密安全摘要，而CRC32C算法提供

的校验和计算速度更快。它擅长捕获由于意外更改引起的错误，但不能抵抗恶意修改。请注意，为了对有权访问备份的对手有用，备份清单必须安全地存储在其他位置，否则必须经过验证以确保自从进行备份以来未进行任何修改。`pg_verifybackup` 可用于根据备份清单检查备份的完整性。

- **--manifest-force-encode**

强制备份清单中的所有文件名采用十六进制编码。如果未指定此选项，则仅对非UTF8文件名进行十六进制编码。此选项主要用于测试读取备份清单文件的工具是否正确处理此情况。

- **--no-estimate-size**

阻止服务器估计将要流式传输的备份数据总量，从而导致 `pg_stat_progress_basebackup` 视图中的 `backup_total` 列始终为 `NULL`

。如果没有此选项，则备份将从枚举整个数据库的大小开始，然后返回并发送实际内容。这可能会使备份花费的时间稍长一些，特别是在发送第一个数据之前，备份将花费更长的时间。

如果估计时间过长，此选项将有助于避免此类估计时间。使用 **--progress** 时不允许使用此选项。

- **--no-manifest**

禁用备份清单的生成。如果未指定此选项，则服务器将生成并发送备份清单，可以使用 `pg_verifybackup` 进行验证。清单是备份中存在的每个文件的列表，可能包含的所有WAL文件除外。它还存储大小、上次修改时间和每个文件的可选校验和。

- **--no-slot**

防止为备份创建临时复制插槽。默认情况下，如果选择了日志流，但没有用选项 **-S**

指定槽名称，则会创建一个临时复制插槽（如果源服务器支持）。这个选项的主要目的是允许在服务器没有空闲复制槽可用时制作基础备份。使用复制槽几乎总是最好的方式，因为它能防止备份期间所需的WAL被删除。

- **--no-verify-checksums**

如果在取基础备份的服务器上启用了校验码验证，则禁用校验码验证。默认情况下，校验码会被验证并且校验码失败将会导致一种非零的退出状态。不过，基础备份在这种情况下将不会被移除，就好像使用了 **--no-clean** 选项一样。校验和验证失败也将报告在 `pg_stat_database` 视图中。

以下命令行选项控制到源服务器的连接：

- **-d connstr --dbname=connstr**

指定用于连接到服务器的参数，比如

[连接字符串](#)；这些将覆盖所有冲突的命令行选项。为了和其他客户端应用一致，该选项被称为 **--dbname**。但是因为 `pg_basebackup` 并不连接到集群中的任何特定数据库，连接字符串中的任何数据库名将被忽略。

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。默认值取自 `PGHOST` 环境变量（如果设置），否则会尝试一个 Unix 域套接字连接。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。默认用 `PGPORT` 环境变量中的值（如果设置），或者一个编译在程序中的默认值。

- **-s interval --status-interval=interval**

指定发送回源服务器的状态包之间的秒数。较小的值可以更准确地监视服务器的备份进度。一个零值完全禁用这种周期性的状态更新，不过当服务器需要时还是会有一个更新会被发送来避免超时导致的断开连接。默认值是 10 秒。

- **-U username --username=username**

指定连接的用户名。

- **-w --no-password**

防止发出口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制pg_basebackup在连接到源服务器之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，pg_basebackup将自动提示要求一个口令。但是，pg_basebackup将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

其他选项也可用：

- **-V --version**

打印pg_basebackup版本并退出。

- **-? --help**

显示有关pg_basebackup命令行参数的帮助并退出。

环境

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

环境变量 **PG_COLOR** 规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

注解

在备份的开始时，需要在源服务器上执行检查点。这可能需要一点时间（尤其在未使用选项 **--checkpoint=fast** 时），在此期间pg_basebackup看起来处于闲置状态。

备份将包括数据目录和表空间中的所有文件，包括配置文件以及由第三方放在该目录中的任何额外文件，不过由IvorySQL管理的特定临时文件除外。但只有常规文件和目录会被拷贝，但用于表空间的符号链接会被保留。指向IvorySQL已知的特定目录的符号链接被拷贝为空目录。其他符号链接和特殊设备文件会被跳过。

在普通格式中，表空间将备份到源服务器上的相同路径，除非使用了 **--tablespace-mapping** 选项。如果没有这个选项并且表空间正在使用，在同一台服务器上进行普通格式的基础备份将无法工作，因为备份必须要写入到与原始表空间相同的目录位置。

在使用 tar 格式时，用户应负责在启动使用数据的 IvorySQL 服务器前解压每一个 tar 文件。如果有额外的表空间，用于它们的 tar 文件需要被解压到正确的位置。在这种情况下，服务器将根据包含在 **base.tar** 文件中的 **tablespace_map** 文件的内容为那些表空间创建符号链接。

pg_basebackup可以和具有相同或较低主版本的服务器一起工作。

如果在源集群上启用了组权限，pg_basebackup将保留数据文件的组权限。

例子

要创建服务器 **mydbserver** 的一个基础备份并将它存储在本地目录 **/usr/local/pgsql/data** 中：

```
$ pg_basebackup -h mydbserver -D /usr/local/pgsql/data
```

要创建本地服务器的一个备份，为其中每一个表空间产生一个压缩过的 tar 文件，并且将它存储在目录 **backup** 中，在运行期间显示一个进度报告：

```
$ pg_basebackup -D backup -Ft -z -P
```

要创建一个单一表空间本地数据库的备份并且使用bzip2压缩它：

```
$ pg_basebackup -D - -Ft -X fetch | bzip2 > backup.tar.bz2
```

（如果在该数据库中有多个表空间，这个命令将失败）。

要创建一个本地数据库的备份，其中 `/opt/ts` 中的表空间被重定位到 `./backup/ts`：

```
$ pg_basebackup -D backup/data -T /opt/ts=$(pwd)/backup/ts
```

pgbench

pgbench — 在IvorySQL上运行一个基准测试

大纲

pgbench -i [option...] [dbname]

pgbench [option...] [dbname]

小心

pgbench -i 会创建四个表 **pgbench_accounts**、**pgbench_branches**、**pgbench_history** 以及 **pgbench_tellers**，如果同名表已经存在会被先删除。如果你已经有同名表，一定要注意要使用另一个数据库！

在默认的情况下“比例因子”为 1，这些表初始包含的行数为：

table	# of rows
pgbench_branches	1
pgbench_tellers	10
pgbench_accounts	100000
pgbench_history	0

你可以使用 **-s**（比例因子）选项增加行的数量。**-F**（填充因子）选项也可以在这里使用。

一旦你完成了必要的设置，你就可以用不包括 **-i** 的命令运行基准，也就是：

```
pgbench [ options ] dbname
```

在近乎所有的情况中，你将需要一些选项来做一次有用的测试。最重要的选项是 **-c**（客户端数量）、**-t**（事务数量）、**-T**（时间限制）以及 **-f**（指定一个自定义脚本文件）。完整的列表见下文。

选项

下面分成三个部分。数据库初始化期间使用的选项和运行基准时会使用不同的选项，但也有一些选项在两种情况下都使用。

初始化选项

pgbench接受下列命令行初始化参数：

- **dbname**

指定要测试的数据库的名称。如果这个没有被指定，将使用环境变量 **PGDATABASE**。如果那个也没有设定，将使用指定要连接的用户名称。

- **-i --initialize**

要求调用初始化模式。

- **-I init_steps --init-steps=init_steps**

只执行选出的一组普通初始化步骤。**init_steps**

指定要被执行的初始化步骤，每一个步骤使用一个字符代表。每一个步骤都以指定的顺序被调用。默认是 **dtgvp**。可用的步骤是：**d**（删除）删除任何已有的pgbench表。**t**（创建表）创建标准pgbench场景使用的表，即 **pgbench_accounts**、**pgbench_branches**、**pgbench_history** 以及 **pgbench_tellers**。**g** 或 **G**（生成数据、客户端或服务器端）生成数据并且装入到标准的表中，替换掉已经存在的任何数据。使用 **g**（客户端数据生成），数据在 **pgbench** 客户端生成，然后发送到服务器。这通过 **COPY** 广泛使用客户端/服务器带宽。使用 **g** 会导致日志记录每 100,000 行打印一条消息，同时为 **pgbench_accounts** 表生成数据。使用 **G**（服务器端数据生成），仅从 **pgbench** 客户端发送少量查询，然后在服务器中实际生成数据。此变体不需要大量带宽，但服务器将完成更多工作。使用

G 会导致日志记录在生成数据时不打印任何进度消息。默认的初始化行为使用客户端数据生成（相当于 **g**）。**v**（清理）在标准的表上调用 **VACUUM**。**p**（创建主键）在标准的表上创建主键索引。**f**（创建外键）在标准的表之间创建外键约束（注意这一步默认不会被执行）。

- **-F fillfactor --fillfactor=fillfactor**

用给定的填充因子创建表 **pgbench_accounts**、**pgbench_tellers** 以及 **pgbench_branches**。默认是100。

- **-n --no-vacuum**

在初始化期间不执行清理（这个选项会抑制 **v** 初始化步骤，即便在 **-I** 中指定了该步骤）。

- **-q --quiet**

把记录切换到安静模式，只是每 5 秒产生一个进度消息。默认的记录会每 100,000 行打印一个消息，这经常会在每秒钟输出很多行（特别是在好的硬件上）。如果在 **-I** 中指定了 **G**，则此设置无效。

- **-s scale_factor --scale=scale_factor**

将生成的行数乘以比例因子。例如，**-s 100** 将在 **pgbench_accounts** 表中创建 10,000,000 行。默认为 1。当比例为 20,000 或更高时，用来保存账号标识符的列（**aid** 列）将切换到使用更大的整数（**bigint**），这样才能足以保存账号标识符。

- **--foreign-keys**

在标准的表之间创建外键约束（如果 **f** 在初始化步骤序列中不存在，这个选项会把它加入）。

- **--index-tablespace=index_tablespace**

在指定的表空间而不是默认表空间中创建索引。

- **--partition-method=NAME**

使用 **NAME** 方法创建一个分区的 **pgbench_accounts** 表。预期值为 **range** 或 **hash**。此选项要求 **--partitions** 设置为非零。如果未指定，默认值为 **range**。

- **--partitions=NUM**

创建一个分区 **pgbench_accounts** 表，其中 **NUM** 分区的大小与按比例缩放的帐户数几乎相等。默认为 **0**，表示没有分区。

- **--tablespace=tablespace**

在指定的表空间而不是默认表空间中创建表。

- **--unlogged-tables**

把所有的表创建为非日志记录表而不是永久表。

基准选项

pgbench接受下列命令行基准参数：

- **-b scriptname[@weight] --builtin= scriptname[@weight]**

把指定的内建脚本加入到要被执行的脚本列表中。可用的内建脚本有：**tpcb-like**、**simple-update** 和 **select-only**。这里也接受内建名称无歧义的前缀缩写。如果用上特殊的名字 **list**，将会显示内建脚本的列表并且立刻退出。可选的，在 **@** 后面写一个整数权重以调整选择这个脚本而不是其它的概率。默认的权重为1。详情如下。

- **-c clients --client= clients**

模拟的客户端数量，也就是并发数据库会话数量。默认为 1。

- **-C --connect**

为每一个事务建立一个新连接，而不是只为每个客户端会话建立一个连接。这对于度量连接开销有用。

- **-d --debug**

打印调试输出。

- **-D varname = value --define= varname = value**

定义一个由自定义脚本（见下文）使用的变量。允许多个 **-D** 选项。

- **-f filename[@weight] --file= filename[@weight]**

把一个从 **filename** 读到的事务脚本加入到被执行的脚本列表中。可选的，在 **@** 后写一个整数权重以调整选择这个脚本而非其他的概率。默认权重为1。(要使用包含 **@** 字符的脚本文件名，附加权重在后面以避免歧义，例如 **fi len@me@1**。) 详情如下。

- **-j threads --jobs= threads**

pgbench中的工作者线程数量。在多 CPU 机器上使用多于一个线程会有用。客户端会尽可能均匀地分布到可用的线程上。默认为 1。

- **-l --log**

把与每一个事务相关的信息写到一个日志文件中。详见下文。

- **-L limit --latency-limit= limit**

对持续超过 **limit** 毫秒的事务进行独立的计数和报告，这些事务被认为是 迟到 (late) 了的事务。在使用限流措施时 (**--rate=...**)，滞后于计划超过 **limit** 毫秒并且因此没有希望满足延迟限制的事务根本 不会被发送给服务器。这些事务被认为是 被跳过 (skipped) 的事务，它们会被单独计数并且报告。

- **-M querymode --protocol=querymode**

要用来提交查询到服务器的协议：**simple**：使用简单查询协议。**extended** 使用扩展查询协议。**prepared**：使用带预备语句的扩展查询语句。在 **prepared** 模式中，pgbench重用从第二次查询迭代开始的语法分析结果，因此pgbench运行速度比其他模式快。默认是简单查询协议。

- **-n --no-vacuum**

在运行测试前不进行清理。如果你在运行一个不包括标准的表 **pgbench_accounts**、**pgbench_branches**、**pgbench_history** 和 **pgbench_tellers** 的自定义测试场景时，这个选项是必需的。

- **-N --skip-some-updates**

运行内建的简单更新脚本。这是 **-b simple-update** 的简写。

- **-P sec --progress=sec**

每 **sec** 秒显示进度报告。该报告包括运行了多长时间、从上次报告以来的 tps 以及从上次报告以来事务延迟的平均值和标准偏差。如果低于限流值（**-R**），延迟会相对于事务预定的开始时间（而不是实际的事务开始时间）计算，因此其中也包括了平均调度延迟时间。

- **-r --report-latencies**

在基准结束后，报告平均的每个命令的每语句等待时间（从客户端的角度来说是执行时间）。详见下文。

- **-R rate --rate= rate**

按照指定的速率执行事务而不是尽可能快地执行（默认行为）。该速率以 tps（每秒事务数）形式给定。如果目标速率高于最大可能速率，则该速率限制不会影响结果。该速率的目标是按照一条泊松分布的调度时间线开始事务。期望的开始时间表会基于客户端第一次开始的时间（而不是上一个事务结束的时间）前移。这种方法意味着当事务超过它们的原定结束时间时，更迟的那些有机会再次追赶上来。当限流措施被激活时，运行结束时报告的事务延迟是从预订的开始时间计算而来的，因此它包括每一个事务不得不等待前一个事务结束所花的时间。该等待时间被称作调度延迟时间，并且它的平均值和最大值也会被单独报告。关于实际事务开始时间的事务延迟（即在数据库中执行事务所花的时间）可以用报告的延迟减去调度延迟时间计算得到。如果把 **--latency-limit** 和 **--rate** 一起使用，当一个事务在前一个事务结束时已经超过了延迟限制时，它可能会滞后非常多，因为延迟是从计划的开始时间计算得来。这类事务不会被发送给服务器，而是一起被跳过并且被单独计数。一个高的调度延迟时间表示系统无法用选定的客户端和线程数按照指定的速率处理事务。当平均的事务执行时间超过每个事务之间的调度间隔时，每一个后续事务将会落后更多，并且随着测试运行时间越长，调度延迟时间将持续增加。发生这种情况时，你将不得不降低指定的事务速率。

- **-s scale_factor --scale= scale_factor**

在pgbench的输出中报告指定的比例因子。对于内建测试，这并非必需；正确的比例因子将通过对 **pgbench_branches** 表中的行计数来检测。不过，当只测试自定义基准（**-f** 选项）时，比例因子将被报告为 1（除非使用了这个选项）。

- **-S --select-only**

执行内建的只有选择的脚本。是 **-b select-only** 简写形式。

- **-t transactions --transactions= transactions**

每个客户端运行的事务数量。默认为 10。

- **-T seconds --time= seconds**

运行测试这么多秒，而不是为每个客户端运行固定数量的事务。**-t** 和 **-T** 是互斥的。

- **-v --vacuum-all**

在运行测试前清理所有四个标准的表。在没有用 **-n** 以及 **-v** 时，`pgbench` 将清理 `pgbench_tellers` 和 `pgbench_branches` 表，并且截断 `pgbench_history`。

- **--aggregate-interval=seconds**

聚集区间的长度（单位是秒）。仅可以与 **-l** 选项一起使用。通过这个选项，日志会包含针对每个区间的概要数据，如下文所述。

- **--log-prefix=prefix**

设置 **--log** 创建的日志文件的文件名前缀。默认是 `pgbench_log`。

- **--progress-timestamp**

当显示进度（选项 **-P**）时，使用一个时间戳（Unix 时间）取代从运行开始的秒数。单位是秒，在小数点后是毫秒精度。这可以有助于比较多种工具生成的日志。

- **--random-seed=seed**

设置随机数生成器种子。为系统的随机数生成器提供种子，然后随机数生成器会产生一个初始生成器状态序列，每一个线程一个状态。**seed** 的值可以是：**time**（默认值，种子基于当前时间）、**rand**（使用一种强随机源，如果没有可用的源则失败）或者一个无符号十进制整数值。一个 `pgbench` 脚本中会显式（**random** 函数）地或者隐式地（如 **--rate** 使用随机数生成器调度事务）调用随机数生成器。在被明确设置时，用作种子的值会显示在终端上。还可以通过环境变量 `PGBENCH_RANDOM_SEED` 提供用于 **seed** 的值。为了确保所提供的种子影响所有可能的使用，把这个选项放在第一位或者使用环境变量。明确地设置种子允许准确地再生一个 `pgbench` 运行，对随机数而言。因为随机状态是针对每个线程管理，这意味着如果每一个线程有一个客户端并且没有外部或者数据依赖，则对于一个相同的调用就会有完全相同的 `pgbench` 运行。从一种统计的角度来看，再生运行不是什么好主意，因为它能隐藏性能可变性或者不正当地改进性能，即通过命中前一次运行的相同页面来改进性能。不过，它也可以对调试起到很大帮助作用，例如重新运行一种导致错误的棘手用例。请善用。

- **--sampling-rate=rate**

采样率，在写入数据到日志时被用来减少日志产生的数量。如果给出这个选项，只有指定比例的事务被记录。1.0 表示所有事务都将被记录，0.05 表示只有 5% 的事务会被记录。在处理日志文件时，记得要考虑这个采样率。例如，当计算 TPS 值时，你需要相应地乘以这个数字（例如，采样率是 0.01，你将只能得到实际 TPS 的 1/100）。

- **--show-script=scriptname**

在 `stderr` 上显示内置脚本 `scriptname` 的实际代码，并立即退出。

普通选项

`pgbench` 还接受以下用于连接参数的常见命令行参数：

- **-h hostname --host=hostname**

数据库服务器的主机名

- **-p port --port=port**

数据库服务器的端口号

- **-U login --username=login**

要作为哪个用户连接

- **-V --version**

打印pgbench版本并退出。

- **-? --help**

显示有关pgbench命令行参数的信息，并且退出。

退出状态

成功运行将以状态0退出。退出状态为1表示静态问题，如无效的命令行选项。运行过程中的错误，例如数据库错误或脚本中的问题将导致退出状态为2。在后一种情况下，pgbench将打印部分结果。

环境

- **PGDATABASE PGHOST PGPORT PGUSER**

默认连接参数。

此实用程序与大多数其他 IvorySQL 实用程序一样，使用 libpq 支持的环境变量。

环境变量 **PG_COLOR** 指定是否在诊断消息中使用颜色。可能的值是 **always**、**auto** 和 **never**。

pg_config

pg_config — 获取已安装的IvorySQL的信息

大纲

pg_config [option...]

选项

要使用pg_config，提供一个或多个下列选项：

- **--bindir**

打印用户可执行文件的位置。例如使用这个选项来寻找 **psql** 程序。这通常也是 **pg_config** 程序所在的位置。

- **--docdir**

打印文档文件的位置。

- **--htmldir**

打印 HTML 文档文件的位置。

- **--includedir**

打印客户端接口的 C 头文件的位置。

- **--pkgincludedir**

打印其它 C 头文件的位置。

- **--includedir-server**

打印用于服务器编程的 C 头文件的位置。

- **--libdir**

打印对象代码库的位置。

- **--pkglibdir**

打印动态可载入模块的位置，或者服务器可能搜索它们的位置（其它架构独立数据文件可能也被安装在这个目录）。

- **--localedir**

打印区域支持文件的位置（如果在IvorySQL被编译时没有配置区域支持，这将是一个空字符串）。

- **--mandir**

打印手册页的位置。

- **--sharedir**

打印架构独立支持文件的位置。

- **--sysconfdir**

打印系统范围配置文件的位置。

- **--pgxs**

打印扩展 makefile 的位置。

- **--configure**

打印当IvorySQL被配置编译时给予 **configure** 脚本的选项。这可以被用来重新得到相同的配置，或者找出是哪个选项编译了一个二进制包（不过注意二进制包通常包含厂商相关的自定补丁）。参见下面的例子。

- **--cc**

打印用来编译IvorySQL的 **CC** 变量值。这显示被使用的 C 编译器。

- **--cppflags**

打印用来编译IvorySQL的 **CPPFLAGS** 变量值。这显示在预处理时需要的 C 编译器开关（典型的是 **-I** 开关）。

- **--cflags**

打印用来编译IvorySQL的 **CFLAGS** 变量值。这显示被使用的 C 编译器开关。

- **--cflags_sl**

打印用来编译IvorySQL的 **CFLAGS_SL** 变量值。这显示被用来编译共享库的额外 C 编译器开关。

- **--ldflags**

打印用来编译IvorySQL的 **LDFLAGS** 变量值。这显示链接器开关。

- **--ldflags_ex**

打印用来编译IvorySQL的 **LDFLAGS_EX** 变量值。这只显示被用来编译可执行程序的链接器开关。

- **--ldflags_sl**

打印用来编译IvorySQL的 **LDFLAGS_SL** 变量值。这只显示被用来编译共享库的链接器开关。

- **--libs**

打印用来编译IvorySQL的 **LIBS** 变量值。这通常包含用于链接到IvorySQL中的外部库的 **-l** 开关。

- **--version**

打印IvorySQL的版本。

- **-? --help**

显示有关pg_config命令行参数的帮助并退出。

如果给定多于一个选项，将按照相同的顺序打印信息，每行一项。如果没有给定选项，将打印所有可用信息，并带有标签。

例子

要重建当前 IvorySQL 安装的编译配置，可运行下列命令：

```
eval `./configure `pg_config --configure`
```

pg_config --configure 的输出包含 shell 引号，这样带空格的参数可以被正确地表示。因此，为了得到正确的结果需要使用 **eval**。

pg_dump

pg_dump — 把IvorySQL数据库抽取为一个脚本文件或其他归档文件

大纲

pg_dump [**connection-option**...] [**option**...] [**dbname**]

选项

下列命令选项控制输出的内容和格式。

- **dbname**

指定要被转储的数据库名。如果没有指定，将使用环境变量 **PGDATABASE**。如果环境变量也没有设置，则使用指定给该连接的用户名。

- **-a --data-only**

只转储数据，而不转储模式（数据定义）。表数据、大对象和序列值都会被转储。这个选项类似于指定 **--section=data**，但是由于历史原因又不完全相同。

- **-b --blobs**

在转储中包括大对象。这是当 **--schema**、**--table** 或 **--schema-only** 被指定时的默认行为。因此，只有在请求转储一个特定方案或者表的情况下，**-b** 开关才对向转储中加入大对象有用。注意blobs是被考虑的数据，因此在使用 **--data-only** 时将被包括在内，但在使用 **--schema-only** 时则不会包括。

- **-B --no-blobs**

在转储中排除大对象。当同时给定 **-b** 和 **-B** 时，行为是在数据被转储时输出大对象，请参考 **-b** 文档。

- **-c --clean**

在输出创建数据库对象的命令之前输出清除（删除）它们的命令（除非也指定了 **--if-exists**，如果任何对象不存在于

目的数据库中，恢复可能会产生一些伤害性的错误消息）。当发出归档(非文本)输出文件时，这个选项被忽略。对于归档格式，你可以在调用 **pg_restore** 时指定该选项。

- **-C --create**

使得在输出的开始是一个创建数据库本身并且重新连接到被创建的数据库的命令（通过这种形式的一个脚本，在运行脚本之前你连接的是目标安装中的哪个数据库都没有关系）。如果也指定了 **--clean**，脚本会在重新连接到目标数据库之前先删除它然后再重建。通过 **--create**，输出还会包括数据库的注释（如果有）以及与这个数据库相关的任何配置变量设置，也就是任何提到了这个数据库的 **ALTER DATABASE ... SET ...** 命令和 **ALTER ROLE ... IN DATABASE ... SET ...** 命令。该数据库本身的访问特权也会被转储，除非指定有 **--no-acl**。当发出归档(非文本)输出文件时，这个选项被忽略。对于归档格式，你可以在调用 **pg_restore** 时指定该选项。

- **-e pattern --extension=pattern**

只转储匹配 **pattern** 的扩展。如果未指定此选项，目标数据库中的所有非系统扩展都将被转储。通过写入多个 **-e** 开关，可以选择多个扩展。**pattern** 参数可以理解为与 **psql's \d** 命令使用的规则相同的模式，所以还可以通过在模式中写入通配符来选择多个扩展。在使用通配符时，如果需要防止来自扩展通配符的shell，请小心的引用模式。任何由 **pg_extension_config_dump** 注册的配置关系被包括在转储中，如果他的扩展已经被 **--extension** 指定了。注意当指定了 **-e**，**pg_dump** 不尝试转储任何所选扩展可能依赖的其他数据库对象。也就是说，这里不保证指定扩展转储的结果能够由他们自己成功恢复到一个干净的数据库中。

- **-E encoding --encoding=encoding**

以指定的字符集编码创建转储。
在默认情况下，该转储会以该数据库的编码创建（另一种得到相同结果的方式是将 **PGCLIENTENCODING** 环境变量设置成想要的转储编码）。

- **-f file --file=file**

将输出发送到指定文件。对于基于输出格式的文件这个参数可以被忽略，在那种情况下将使用标准输出。不过对于目录输出格式必须给定这个参数，在目录输出格式中指定的是一个目录而不是一个文件。在这种情况下，该目录会由 **pg_dump** 创建并且不需要以前就存在。

- **-F format --format=format**

选择输出的格式。**format** 可以是下列之一：**pplain** 输出一个纯文本形式的SQL脚本文件（默认值）。**c custom**

输出一个适合于作为**pg_restore**输入的自定义格式归档。和目录输出格式一起，这是最灵活的输出格式，它允许在恢复时手动选择和排序已归档的项。这种格式在默认情况还会被压缩。**ddirectory**

输出一个适合作为**pg_restore**输入的目录格式归档。这将创建一个目录，其中每个被转储的表和大对象都有一个文件，外加一个所谓的目录文件，该文件以一种**pg_restore**能读取的机器可读格式描述被转储的对象。

一个目录格式归档能用标准 Unix

工具操纵，例如一个未压缩归档中的文件可以使用gzip工具压缩。这种格式默认情况下是被压缩的并且也支持并行转储。**ttar** 输出一个适合于输入到**pg_restore**中的 **tar** -格式归档。**tar**

格式可以兼容目录格式，抽取一个 **tar** 格式的归档会产生一个合法的目录格式归档。不过，**tar** 格式不支持压缩。还有，在使用 **tar** 格式时，表数据项的相对顺序不能在恢复过程中被更改。

- **-j njobs --jobs=njobs**

通过同时归档 **njobs**

个表来运行并行转储。这个选项可能会减少执行转储所需的时间，但也会增加数据库服务器上的负载。你只能和目录输出格式一起使用这个选项，因为这是唯一一种让多个进程能在同一时间写其数据的输出格式。**pg_dump**将打开 **njobs + 1** 个到该数据库的连接，因此确保你的 **max_connections** 设置足够高以容纳所有的连接。在运行一次并行转储时请求数据库对象上的排他锁可能导致转储失败。其原因是，**pg_dump**领导者进程会在工作者进程将要稍后转储的对象上请求共享锁，以便确保在转储运行时不会有人删除它们并让它们出错。

如果另一个客户端接着请求一个表上的排他锁，那个锁将不会被授予但是会被排入队列等待领导者进程的共享锁被释放。因此，任何其他对该表的访问将不会被授予或者将排在排他锁请求之后。

这包括尝试转储该表的工作者进程。如果没有任何防范措施，这可能会是一种经典的死锁情况。

要检测这种冲突，pg_dump工作者进程使用 **NOWAIT** 选项请求另一个共享锁。

如果该工作者进程没有被授予这个共享锁，其他某人必定已经在同时请求了一个排他锁并且没有办法继续转储，因此pg_dump除了中止转储之外别无选择。对于一个一致的备份，数据库服务器需要支持同步的快照，在IvorySQL的主服务器和10的后备服务器中引入了一种特性。

有了这种特性，即便数据库客户端使用不同的连接，也可以保证他们看到相同的数据集。 **pg_dump -j**

使用多个数据库连接，它用领导者进程连接到数据一次，并且为每一个工作者任务再一次连接数据库。

如果没有同步快照特征，在每一个连接中不同的工作者任务将不能被保证看到相同的数据，这可能导致一个不一致的备份。

- **-n pattern --schema=pattern**

只转储匹配 **pattern**

的模式，这会选择模式本身以及它所包含的所有对象。当没有指定这个选项时，目标数据库中所有非系统模式都将被转储。多个模式可以通过书写多个 **-n** 开关来选择。另外， **pattern** 参数可以被解释为一种根据psql的 `\d` 命令所用的相同规则（请参见下面的

Patterns）编写的模式，这样多个模式也可以通过在该模式中书写通配字符来选择。在使用通配符时，如果需要阻止 shell 展开通配符需要小心引用该模式，注意当 **-n**

被指定时，pg_dump不会尝试转储所选模式可能依赖的任何其他数据库对象。因此，无法保证一次指定模式转储的结果能够仅凭其本身被成功地恢复到一个干净的数据库中。注意当 **-n** 被指定时，非模式对象（如二进制大对象）不会被转储。你可以使用 **--blobs** 开关将二进制大对象加回到该转储中。

- **-N pattern --exclude-schema=pattern**

不转储匹配 **pattern** 模式的任何模式。该模式被根据 **-n** 所用的相同规则被解释。**-N**

可以被给定多次来排除匹配几个模式中任意一个的模式。当 **-n** 和 **-N**

都被给定时，该行为是只转储匹配至少一个 **-n** 开关但是不匹配 **-N** 开关的模式。如果只有 **-N** 而没有 **-n**，那么匹配 **-N** 的模式会被从一个正常转储中排除。

- **-O --no-owner**

不输出设置对象拥有关系来匹配原始数据库的命令。默认情况下，pg_dump会发出 `ALTER OWNER` 或 **SET SESSION AUTHORIZATION**

语句来设置被创建的数据库对象的拥有关系。除非该脚本被一个超级用户（或是拥有脚本中所有对象的同一个用户）启动，这些语句都将会失败。要使一个脚本能够被任意用户恢复，但把所有对象的拥有关系都给这个用户，可指定 **-O**。

当发出归档(非文本)输出文件时，这个选项被忽略。对于归档格式，你可以在调用 `pg_restore` 时指定该选项。

- **-R --no-reconnect**

这个选项已经废弃，但是为了向后兼容仍然能被接受。

- **-s --schema-only**

只转储对象定义（模式），而非数据。这个选项是 **--data-only** 的逆选项。它和指定 **--section=pre-data**

--section=post-data 相似，但是由于历史原因又不完全相同。（不要把这个选项和 **--schema**

选项混淆，后者在“schema”的使用上有不同的含义）。要为数据库中表的一个子集排除表数据，见 **--exclude-table-data**。

- **-S username --superuser=username**

指定要在禁用触发器时使用的超级用户的用户名。只有使用 **--disable-triggers**

时，这个选项才相关（通常，最好省去这个选项，而作为超级用户来启动结果脚本来取而代之）。

- **-t pattern --table=pattern**

只转储名字匹配 **pattern** 的表。通过写多个 **-t** 开关可以选择多个表。另外， **pattern**

参数可以被解释为一种根据psql的 `s \d`

命令所用的相同规则编写的模式，这样多个表也可以通过在该模式中书写通配字符来选择。在使用通配符时，如果需要阻止 shell 展开通配符需要小心引用该模式，当 **-t** 被使用时，**-n** 和 **-N** 开关不会有效果，因为被 **-t** 选择的表将被转储而忽视那些开关，并且非表对象将不会被转储。注意当 **-t**

被指定时，pg_dump不会尝试转储所选表可能依赖的任何其他数据库对象。因此，无法保证一次指定表转储

的结果能够仅凭其本身被成功地恢复到一个干净的数据库中。

- **-T pattern --exclude-table=pattern**

不转储匹配 **pattern** 模式的任何表。该模式被根据 **-t** 所用的相同规则被解释。**-T** 可以被给定多次来排除匹配几个模式中任意一个的模式。当 **-t** 和 **-T** 都被给定时，该行为是只转储匹配至少一个 **-t** 开关但是不匹配 **-T** 开关的表。如果只有 **-T** 而没有 **-t**，那么匹配 **-T** 的表会被从一个正常转储中排除。

- **-v --verbose**

指定冗长模式。这将导致 `pg_dump` 向标准错误输出详细的对象注释以及转储文件的开始/停止时间，还有进度消息。重复该选项会导致在标准错误上出现附加的调试级消息。

- **-V --version**

`pg_dump` 版本并退出。

- **-x --no-privileges --no-acl**

防止转储访问特权（授予/收回命令）。

- **-Z 0..9 --compress=0..9**

指定要使用的压缩级别。零意味着不压缩。对于自定义和目录归档格式，这会指定个体表数据段的压缩，并且默认是进行中等级别的压缩。对于纯文本输出，设置一个非零压缩级别会导致整个输出文件被压缩，就好像它被 `gzip` 处理过一样，但是默认是不压缩。`tar` 归档格式当前完全不支持压缩。

- **--binary-upgrade**

这个选项用于就地升级功能。我们不推荐也不支持把它用于其他目的。这个选项在未来的发行中可能被改变而不做通知。

- **--column-inserts --attribute-inserts**

将数据转储为带有显式列名的 **INSERT** 命令（**INSERT INTO table (column, ...) VALUES ...**）。这将使得恢复过程非常慢，这主要用于使转储能够被载入到非 `IvorySQL` 数据库中。重新加载期间的任何错误都将导致有问题的 **INSERT** 相关的行将丢失，而不是整个表内容。

- **--disable-dollar-quoting**

这个选项禁止在函数体中使用美元符号引用，并且强制它们使用 `SQL` 标准字符串语法被引用。

- **--disable-triggers**

只有在创建一个只转储数据的转储时，这个选项才相关。它指示 `pg_dump` 包括在数据被重新载入时能够临时禁用目标表上的触发器的命令。如果你在表上有引用完整性检查或其他触发器，并且你在数据重新载入期间不想调用它们，请使用这个选项。当前，为 **--disable-triggers** 发出的命令必须作为超级用户来执行。因此，你还应当使用 **-S** 指定一个超级用户名，或者宁可作为一个超级用户启动结果脚本。当发出归档(非文本)输出文件时，这个选项被忽略。对于归档格式，你可以在调用 `pg_restore` 时指定该选项。

- **--enable-row-security**

只有在转储具有行安全性的表的内容时，这个选项才相关。默认情况下，`pg_dump` 将把 `row_security` 设置为 `off` 来确保从该表中转储出所有的数据。如果用户不具有足够能绕过行安全性的特权，那么会抛出一个错误这个参数指示 `pg_dump` 将 `row_security` 设置为 `on`，允许用户只转储该表中它们能够访问到的部分内容。注意如果当前你使用了这个选项，你可能还想得到 **INSERT** 格式的转储，因为恢复期间的 **COPY FROM** 不支持行安全性。

- **--exclude-table-data=pattern**

不转储匹配 **pattern** 模式的任何表中的数据。该模式根据 **-t** 的相同规则被解释。**--exclude-table-data**

可以被给定多次来排除匹配多个模式的表。当你需要一个特定表的定义但不想要其中的数据时，这个选项就有用了。要排除数据库中所有表的数据，见 **--schema-only**。

- **--extra-float-digits=ndigits**

在转储浮点数据时使用规定的 **extra_float_digits** 值，而不是最大可用精度。以备份目的生成的常规转储不使用此选项。

- **--if-exists**

时间条件性命令（即增加一个 **IF EXISTS** 子句）来清除数据库和其他对象。只有同时指定了 **--clean** 时，这个选项才可用。

- **--include-foreign-data='foreignserver'**

使用与 **foreignserver** 模式匹配的外部服务器转储任何外部表的数据。可以通过编写多个 **--include-foreign-data** 开关来选择多个外部服务器。同样，根据 `psql's \d` 命令使用的相同规则，将 **foreignserver** 参数解释为模式。

因此也可以通过在模式中写入通配符来选择多个外部服务器。使用通配符时，如果需要，请小心引用该模式，以防止Shell扩展通配符。唯一的例外是不允许使用空模式。注意指定 **--include-foreign-data** 时，`pg_dump` 不会检查外部表是否可写。因此，不能保证可以成功还原外部表转储的结果。

- **--inserts**

将数据转储为 **INSERT** 命令（而不是 **COPY**）。这将使得恢复非常慢，这主要用于使转储能够被载入到非IvorySQL数据库中。重新加载期间的任何错误都将导致有问题的 **INSERT** 相关的行将丢失，而不是整个表内容。注意如果你已经重新安排了列序，该恢复可能会一起失败。**--column-inserts** 选项对于列序改变是安全的，但是会更慢。

- **--load-via-partition-root**

在为一个分区表转储数据时，让 **COPY** 语句或者 **INSERT** 语句把包含它的分区层次的根而不是分区自身作为目标。这导致在数据被装载时，会为每一个行重新确定合适的分区。如果在一台服务器上重新装载数据时会出现行并不是总是落入到和原始服务器上相同的分区中的情况，这个选项就很有用。例如，如果分区列是文本类型并且两个系统中用于排序分区列的排序规则有着不同的定义，就会发生这种情况。在从用这个选项制作的归档恢复时，最好不要使用并行，因为 `pg_restore` 将不能准确地知道一个给定的归档数据项将把数据装载到哪个分区中。这会导致效率不高，因为在并行任务见会有锁冲突，或者甚至可能由于在所有的相关数据被装载前建立了外键约束而导致重新装载失败。

- **--lock-wait-timeout='timeout'**

在转储的开始从不等待共享表锁的获得。而是在指定的 **timeout** 内不能锁定一个表时失败。超时时长可以用 **SET statement_timeout** 接受的任何格式指定。

- **--no-comments**

不转储注释。

- **--no-publications**

不转储publication。

- **--no-security-labels**

不转储安全标签。

- **--no-subscriptions**

不转储订阅。

- **--no-sync**

默认情况下，**pg_dump** 将等待所有文件被安全地写入磁盘。这个选项会让 **pg_dump** 不等待直接返回，这样会更快，但是也意味着后续的一次操作系统崩溃会让该转储损坏。通常这个选项对测试有用，但是不应该在从生产安装中转储数据时使用。

- **--no-tablespaces**

不要输出选择表空间的命令。通过这个选项，在恢复期间所有的对象都会被创建在任何作为默认的表空间中。当发出归档(非文本)输出文件时，这个选项被忽略。对于归档格式，你可以在调用 **pg_restore** 时指定该选项。

- **--no-toast-compression**

不要输出命令以设置 TOAST 压缩方法。采用这个选项，所有列将按默认压缩设置进行恢复。

- **--no-unlogged-table-data**

不转储非日志记录表的内容。这个选项对于表定义（模式）是否被转储没有影响，它只会限制转储表数据。当从一个后备服务器转储时，在非日志记录表中的数据总是会被排除。

- **--on-conflict-do-nothing**

增加 **ON CONFLICT DO NOTHING** 到 **INSERT** commands。除非规定了 **--inserts**, **--column-inserts** 或 **--rows-per-insert**，否则此选项是无效的。

- **--quote-all-identifiers**

强制引用所有标识符。当从IvorySQL主版本与**pg_dump**不同的服务器上转储一个数据库时或者当输出准备载入到一个具有不同主版本的服务器时，推荐使用这个选项。默认情况下，**pg_dump**只对在其主版本中是被保留词的标识符加上引号。在转储其他版本服务器时，这种默认行为有时会导致兼容性问题，因为那些版本可能具有些许不同的被保留词集合。使用 **--quote-all-identifiers** 能阻止这种问题，但代价是转储脚本更难阅读。

- **--rows-per-insert=nrows**

数据转储为 **INSERT** 命令（而不是 **COPY**）。控制每个 **INSERT** 命令的最大行数。指定的值必须大于零。重新加载期间的任何错误都将导致有问题的`INSERT`相关的行将丢失，而不是整个表内容。

- **--section=sectionname**

只转储命名节。节的名称可以是 **pre-data**、**data** 或 **post-data**。这个选项可以被指定多次来选择多个节。默认是转储所有节。数据节包含真正的表数据、大对象内容和序列值。数据后项包括索引、触发器、规则和除了已验证检查约束之外的约束的定义。数据前项包括所有其他数据定义项。

- **--serializable-deferrable**

为转储使用一个 **可序列化**

事务，以保证所使用的快照与后来的数据库状态是一致的。但是这样做是在事务流中等待一个点，在该点上不能存在异常，这样就不会有转储失败或者导致其他事务带着 **serialization_failure** 回滚的风险。对于一个只为灾难恢复存在的转储，这个选项没什么益处。如果一个转储被用来在原始数据库持续被更新期间载入一份用于报表或其他只读负载的数据库拷贝时，这个选项就有所帮助。如果没有这个选项，转储可能会反映一个与最终提交事务的任何执行序列都不一致的状态。例如，如果使用了批处理技术，一个批处理在转储中可以显示为关闭，而其中的所有项都不出现。如果 **pg_dump** 被启动时没有读写事务在活动，则这个选项没有什么不同。如果有读写事务在活动，该转储的启动可能会被延迟一段不确定的时间。一旦开始运行，有没有这个开关的表现是相同的。

- **--snapshot=snapshotname**

在做一个数据库的转储时指定一个同步的快照。在需要把转储和一个逻辑复制槽或者一个并发会话同步时可以用上这个选项。在并行转储的情况下，将使用这个选项指定的快照名而不是取

一个新快照。

- **--strict-names**

要求每一个扩展(**-e / --extension**)，模式 (**-n / --schema**) 和表 (**-t / --table**) 限定符匹配要转储的数据库中至少一个扩展/模式/表。

注意，如果没有找到有这样的扩展/模式/表限定符匹配，即便没有 **--strict-names**，**pg_dump**也将生成一个错误。这个选项对 **-N / --exclude-schema**、**-T / --exclude-table** 或者 **--exclude-table-data** 没有效果。无法匹配任何对象的排除模式不会被当作错误。

- **--use-set-session-authorization**

输出 SQL-标准的 **SET SESSION AUTHORIZATION** 命令取代 **ALTER OWNER**

命令来确定对象的所有关系。这让该转储更加兼容标准，但是取决于该转储中对象的历史，该转储可能无法正常恢复。而且，一个使用 **SET SESSION AUTHORIZATION** 的转储将一定会要求超级用户特权来正确地恢复，而 **ALTER OWNER** 要求更少的特权。

- **-? --help**

显示有关**pg_dump**命令行参数的帮助并退出。

下列命令行选项控制数据库连接参数。

- **-d dbname --dbname=dbname**

指定要连接的数据库的名称。这等效于在命令行上将 **dbname** 指定为第一个非选项参数。**dbname** 可以是连接字符串。如果是这样，连接字符串参数将覆盖所有冲突的命令行选项。

- **-h host --host=host**

指定服务器正在运行的机器的主机名。如果该值开始于一个斜线，它被用作一个 Unix 域套接字的目录。默认是从 **PGHOST** 环境变量中取得（如果被设置），否则将尝试一次 Unix 域套接字连接。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展名。默认是放在 **PGPORT** 环境变量中（如果被设置），否则使用编译在程序中的默认值。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那么连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制**pg_dump**在连接到一个数据库之前提示要求一个口令。这个选项从来不是必须的，因为如果服务器要求口令认证，**pg_dump**将自动提示要求一个口令。但是，**pg_dump**将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下，值得键入 **-W** 来避免额外的连接尝试。

- **--role=rolename**

指定一个用来创建该转储的角色名。这个选项导致**pg_dump**在连接到数据库后发出一个 **SET ROLE rolename** 命令。当已认证用户（由 **-U** 指定）缺少**pg_dump**所需的特权但是能够切换到一个具有所需权利的角色时，这个选项很有用。一些安装有针对直接作为超级用户登录的策略，使用这个选项可以让转储在不违反该策略的前提下完成。

环境

- **PGDATABASE PGHOST PGOPTIONS PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

pg_dump在内部执行 **SELECT**

语句。如果你运行pg_dump时出现问题，确定你能够从正在使用的数据库中选择信息，例如 **psql**。此外，libpq前端-后端库所使用的任何默认连接设置和环境变量都将适用。

pg_dump的数据库活动会被统计收集器正常地收集。如果不想这样，你可以通过 **PGOPTIONS** 或 **ALTER USER** 命令设置参数 **track_counts** 为假。

注解

如果你的数据库集簇对于 **template1**

数据库有任何本地添加，要注意将pg_dump的输出恢复到一个真正的空数据库。否则你很可能由于以增加对象的重复定义而得到错误。要创建一个不带任何本地添加的空数据库，从 **template0** 而不是 **template1** 复制它，例如：

```
CREATE DATABASE foo WITH TEMPLATE template0;
```

当一个只含数据的转储被选中并且使用了选项 **--disable-triggers**

时，pg_dump在开始插入数据之前会发出命令禁用用户表上的触发器，并且接着在数据被插入之后发出命令重新启用它们。如果恢复中途被停止，系统目录可能会停留在一种错误状态。

pg_dump产生的转储文件不包含优化器用来做出查询计划决定的统计信息。因此，在从一个转储文件恢复后运行 **ANALYZE** 来确保最优性能是明智的。

在转储逻辑复制订阅时，pg_dump将生成使用 **connect = false** 选项的 **CREATE SUBSCRIPTION**

命令，这样恢复订阅时不会建立远程连接来创建复制槽或者进行初始的表拷贝。通过这种方式，可以无需到远程服务器的网络访问就能恢复该转储。然后就需要用户以一种合适的方式重新激活订阅。如果涉及到的主机已经改变，连接信息可能也必须被改变。在开启一次新的全表拷贝之前，截断目标表也可能是合适的。

实例

要把一个数据库 **mydb** 转储到一个 SQL 脚本文件：

```
$ pg_dump mydb > db.sql
```

要把这样一个脚本重新载入到一个（新创建的）名为 **newdb** 的数据库中：

```
$ psql -d newdb -f db.sql
```

要转储一个数据库到一个自定义格式归档文件：

```
$ pg_dump -Fc mydb > db.dump
```

要转储一个数据库到一个目录格式的归档：

```
$ pg_dump -Fd mydb -f dumpdir
```

要用 5 个并行的工作者任务转储一个数据库到一个目录格式的归档：

```
$ pg_dump -Fd mydb -j 5 -f dumpdir
```

要把一个归档文件重新载入到一个（新创建的）名为 **newdb** 的数据库：

```
$ pg_restore -d newdb db.dump
```

把一个归档文件重新装载到同一个数据库（该归档正是从这个数据库中转储得来）中，丢掉那个数据库中的当前内容：

```
$ pg_restore -d postgres --clean --create db.dump
```

要转储一个名为 **mytab** 的表：

```
$ pg_dump -t mytab mydb > db.sql
```

要转储 **detroit** 模式中名称以 **emp** 开始的所有表，排除名为 **employee_log** 的表：

```
$ pg_dump -t 'detroit.emp*' -T detroit.employee_log mydb > db.sql
```

要转储名称以 **east** 或者 **west** 开始并且以 **gsm** 结束的所有模式，排除名称包含词 ``test`` 的任何模式：

```
$ pg_dump -n 'east*gsm' -n 'west*gsm' -N '*test*' mydb > db.sql
```

同样，用正则表达式记号法来合并开关：

```
$ pg_dump -n '(east|west)*gsm' -N '*test*' mydb > db.sql
```

要转储除了名称以 **ts_** 开头的表之外的所有数据库对象：

```
$ pg_dump -T 'ts_*' mydb > db.sql
```

要在 **-t**

和相关开关中指定一个大写形式或混合大小写形式的名称，你需要双引用该名称，否则它会被折叠到小写形式。但是双引号对于 shell 是特殊的，所以反过来它们必须被引用。因此，

要转储一个有混合大小写名称的表，你需要类似这样的东西：

```
$ pg_dump -t "\"MixedCaseName\"" mydb > mytab.sql
```

pg_dumpall

pg_dumpall — 将一个IvorySQL数据库集簇抽取到一个脚本文件中

大纲

pg_dumpall [connection-option...] [option...]

描述

pg_dumpall工具可以一个集簇中所有的IvorySQL数据库写出到（“转储”）一个脚本文件。该脚本文件包含可以用作 **psql** 的输入SQL命令来恢复数据库。它会对集簇中的每个数据库调用 **pg_dump** 来完成该工作。pg_dumpall还转储对所有数据库公用的全局对象（**pg_dump** 不保存这些对象），也就是说数据库角色和表空间都会被转储。目前这包括适数据库用户和组、表空间以及适合所有数据库的访问权限等属性。

因为pg_dumpall从所有数据库中读取表，所以你很可能需要以一个数据库超级用户的身份连接以便生成完整的转储。同样，你也需要超级用户特权执行保存下来的脚本，这样才能增加角色和组以及创建数据库。

SQL 脚本将被写出到标准输出。使用 **-f / --file** 选项或者 shell 操作符可以把它重定向到一个文件。

pg_dumpall需要多次连接到IvorySQL服务器（每个数据库一次）。如果你使用口令认证，可能每次都会要求口令。这种情况下使用一个 **~/.pgpass** 会比较方便。

选项

下列命令行选项用于控制输出的内容和格式。

- **-a --data-only**

只转储数据，不转储模式（数据定义）。

- **-c --clean**

包括在重建数据库之前清除（移除）它们的 SQL 命令。角色和表空间的 **DROP** 命令也会被加入进来。

- **-E encoding --encoding=encoding**

用指定的字符集编码创建转储。默认情况下，转储使用数据库的编码创建（另一种得到相同结果的方法是设置 **PGCLIENTENCODING** 环境变量为想要的转储编码）。

- **-f filename --file=filename**

将输出发送到指定的文件中。如果省略，将使用标准输出。

- **-g --globals-only**

只转储全局对象（角色和表空间），而不转储数据库。

- **-O --no-owner**

不输出用于设置对象所有权以符合原始数据库的命令。默认情况下，pg_dumpall发出 **ALTER OWNER** 或 **SET SESSION AUTHORIZATION**

语句来设置被创建的模式元素的所有权。除非脚本是由一个超级用户（或者是拥有脚本中所有对象的同一个用户）所运行，这些语句在脚本运行时失败。要使得一个脚本能被任意用户恢复，但又不想给予该用户所有对象的所有权，可以指定 **-O**。

- **-r --roles-only**

只转储角色，不转储数据库和表空间。

- **-s --schema-only**

只转储对象定义（模式），不转储数据。

- **-S username --superuser=username**

指定要在禁用触发器时使用的超级用户的用户名。只有使用 **--disable-triggers** 时，这个选项才相关（通常，最好省去这个选项，而作为超级用户来启动结果脚本来取而代之）。

- **-t --tablespaces-only**

只转储表空间，不转储数据库和角色。

- **-v --verbose**

指定细节模式。这将导致pg_dumpall向标准错误输出详细的对象注释以及转储文件的开始/停止时间，还有进度消息。重复该选项会导致在标准错误上出现附加的调试级信息。这个选项还会被传递到pg_dump。

- **-V --version**

打印pg_dumpall版本并退出。

- **-x --no-privileges --no-acl**

防止转储访问特权（授予/收回命令）。

- **--binary-upgrade**

这个选项用于就地升级功能。我们不推荐也不支持把它用于其他目的。这个选项在未来的发行中可能被改变而不做通知。

- **--column-inserts --attribute-inserts**

将数据转储为带有显式列名的 **INSERT** 命令（**INSERT INTO table(column, ...) VALUES ...**）。这将使得恢复过程非常慢，这主要用于使转储能够被载入到非IvorySQL数据库中。

- **--disable-dollar-quoting**

这个选项禁止在函数体中使用美元符号引用，并且强制它们使用 SQL 标准字符串语法被引用。

- **--disable-triggers**

只有在创建一个只转储数据的转储时，这个选项才相关。它指示pg_dumpall包括在数据被重新载入时能够临时禁用目标表上的触发器的命令。如果你在表上有引用完整性检查或其他触发器，并且你在数据重新载入期间不想调用它们，请使用这个选项。当前，为 **--disable-triggers** 发出的命令必须作为超级用户来执行。因此，你还应当使用 **-S** 指定一个超级用户名，或者宁可作为一个超级用户启动结果脚本。

- **--exclude-database='pattern'**

不要转储名字与 **pattern** 匹配的数据库。可以通过编写多个 **--exclude-database** 开关来排除多个模式。**pattern** 参数被解释为模式，根据psql的 **\d** 命令使用的相同规则，因此，通过在模式中编写通配符也可以排除多个数据库。使用通配符时，请谨慎的引用模式，如果需要防止shell通配符扩展。

- **--extra-float-digits=ndigits**

在转储浮点数据时使用extra_float_digits规定的值，而不是最大可用精度。备份目的进行的常规转储不使用此选项。

- **--if-exists**

时间条件性命令（即增加一个 **IF EXISTS** 子句）来清除数据库和其他对象。只有同时指定了 **--clean** 时，这个选项才可用。

- **--inserts**

将数据转储为 **INSERT** 命令（而不是 **COPY**）。这将使得恢复非常慢，这主要用于使转储能够被载入到非IvorySQL数据库中。注意如果你已经重新安排了列序，该恢复可能会一起失败。**--column-inserts** 选项对于列序改变是安全的，但是会更慢。

- **--load-via-partition-root**

在为一个分区表转储数据时，让 **COPY** 语句或者 **INSERT** 语句把包含它的分区层次的根而不是分区自身作为目标。这导致在数据被装载时，会为每一个行重新确定合适的分区。如果在一台服务器上重新装载数据时会出现行并不是总是落入到和原始服务器上相同的分区中的情况，这个选项就很有用。例如，如果分区列是文本类型并且两个系统中用于排序分区列的排序规则有着不同的定义，就会发生这种情况。

- **--lock-wait-timeout=timeout**

在转储的开始从不等待共享表锁的获得。而是在指定的 **timeout** 内不能锁定一个表时失败。超时时长可以用 **SET statement_timeout** 接受的任何格式指定。

- **--no-comments**

不转储注释。

- **--no-publications**

不转储publication。

- **--no-role-passwords**

不为角色转储口令。在恢复完后，角色的口令将是空口令，并且在设置口令之前口令认证都不会成功。由于指定这个选项时并不需要口令值，角色信息将从目录视图 **pg_roles** 而不是 **pg_authid** 中读出。因此，如果对 **pg_authid** 的访问被某条安全性策略所限制，那么这个选项也会有所帮助。

- **--no-security-labels**

不转储安全标签。

- **--no-subscriptions**

不转储subscription。

- **--no-sync**

默认情况下，**pg_dumpall** 将等待所有文件被安全地写入到磁盘。这个选项会让 **pg_dumpall** 不做这种等待而返回，这样会更快，但是意味着后续的操作系统崩溃可能留下被损坏的转储。通常来说，这个选项对测试有用，但不应该在从生产安装中转储数据时使用。

- **--no-tablespaces**

不要输出选择表空间的命令。通过这个选项，在恢复期间所有的对象都会被创建在任何作为默认的表空间中。

- **--no-toast-compression**

不要输出命令以设置 TOAST 压缩方法。采用这个选项，所有列将会以默认压缩设置来恢复。

- **--no-unlogged-table-data**

不转储非日志记录表的内容。这个选项对于表定义（模式）是否被转储没有影响，它只会限制转储表数据。

- **--on-conflict-do-nothing**

添加 **ON CONFLICT DO NOTHING** 到 **INSERT** 命令。除非 **--inserts** 或 **--column-inserts** 也被规定，否则此选项不生效。

- **--quote-all-identifiers**

强制引用所有标识符。在从一个与pg_dumpall主版本不同的IvorySQL服务器转储数据库时或者要将输出载入到一个不同主版本的服务器时，推荐使用这个选项。默认情况下，pg_dumpall只会对为其主版本中保留词的标识符加上引号。在与其他版本的具有不同保留词集合的服务器交互时，这有时会导致兼容性问题。使用 **--quote-all-identifiers** 可以阻止这类问题，但是代价是转储脚本会更加难读。

- **--rows-per-insert=nrows**

将数据转储为 **INSERT** 命令（而不是 **COPY**）。控制每个 **INSERT** 命令的最大行数。指定的值必须是大于零的数。重新加载期间的任何错误都将导致仅丢失有问题的 **INSERT** 的行，而不是整个表内容。

- **--use-set-session-authorization**

输出 SQL-标准的 **SET SESSION AUTHORIZATION** 命令取代 **ALTER OWNER** 命令来确定对象的所有关系。这让该转储更加兼容标准，但是取决于该转储中对象的历史，该转储可能无法正常恢复。

- **-? --help**

显示有关pg_dumpall命令行参数的帮助并退出。

下列命令行选项控制数据库连接参数。

- **-d connstr --dbname=connstr**

指定用于连接到服务器的参数，比如 [连接字符串](#)；这些将覆盖所有冲突的命令行选项。这个选项被称为 **--dbname** 是为了和其他客户端应用一致，但是因为pg_dumpall需要连接多个数据库，连接字符串中的数据库名将被忽略。使用 **-l** 选项指定一个数据库，该数据库被用于初始连接，这将转储全局对象并且发现需要转储哪些其他数据库。

- **-h host --host=host**

指定服务器正在运行的机器的主机名。如果该值开始于一个斜线，它被用作一个 Unix 域套接字的目录。默认是从 **PGHOST** 环境变量中取得（如果被设置），否则将尝试一次 Unix 域套接字连接。

- **-l dbname --database=dbname**

指定要连接到哪个数据库转储全局对象以及发现要转储哪些其他数据库。如果没有指定，将会使用 **postgres** 数据库，如果 **postgres** 不存在，就使用 **template1**。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展名。默认是放在 **PGPORT** 环境变量中（如果被设置），否则使用编译在程序中的默认值。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令

。

- **-W --password**

强制pg_dumpall在连接到一个数据库之前提示要求一个口令。这个选项从来不是必须的，因为如果服务器要求口令认证，pg_dumpall将自动提示要求一个口令。但是，pg_dumpall将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下，值得键入 **-W** 来避免额外的连接尝试。注意对每个要被转储的数据库，口令提示都会再次出现。通常，最好设置一个 **~/.pgpass** 文件来减少手工口令输入。

- **--role='rolename'**

指定一个用来创建该转储的角色名。这个选项导致pg_dump在连接到数据库后发出一个 **SET ROLE rolename** 命令。当已认证用户（由 **-U** 指定）缺少pg_dump所需的特权但是能够切换到一个具有所需权利的角色时，这个选项很有用。一些安装有针对直接作为超级用户登录的策略，使用这个选项可以让转储在不违反该策略的前提下完成。

环境

- **PGHOST PGOPTIONS PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

注解

因为pg_dumpall在内部调用pg_dump，所以，一些诊断消息可以参考pg_dump。

即使当用户的目的是把转储脚本恢复到一个空的集簇中，**--clean** 选项也有用武之地。**--clean** 的使用让该脚本删除并且重建内建的 **postgres** 和 **template1** 数据库，确保这两个数据库保持与源集簇中相同的属性（例如locale和编码）。如果不用这个选项，这两个数据库将保持它们现有的数据库级属性以及任何已有的内容。

一旦恢复，建议在每个数据库上运行 **ANALYZE**，这样优化器就可以得到有用的统计信息。你也可以运行 **vacuumdb -a -z** 来分析所有数据库。

不应该预期转储脚本运行到结束都不出错。特别是由于脚本将为源集簇中已有的每一个角色发出 **CREATE ROLE** 语句，对于bootstrap超级用户当然会得到一个“role already exists”错误，除非目标集簇用一个不同的bootstrap超级用户名完成的初始化。这种错误是无害的并且应该被忽略。**--clean** 选项的使用很可能会产生额外的有关于不存在对象的无害错误消息，不过可以通过加上 **--if-exists** 减少这类错误消息。

pg_dumpall要求所有需要的表空间目录在进行恢复之前就必须存在；否则，数据库创建就会由于在非默认位置创建数据库而失败。

例子

要转储所有数据库：

```
$ pg_dumpall > db.out
```

要从这个文件重新载入数据库，你可以使用：

```
$ psql -f db.out postgres
```

这里你连接哪一个数据库并不重要，因为由pg_dumpall创建的脚本将包含合适的命令来创建和连接到被保存的数据库。一个例外是，如果指定了 **--clean**，则开始时必须连接到 **postgres** 数据库，该脚本将立即尝试删除其他数据库，并且这种动作对于已连接上的这个数据库将会失败。

pg_isready

pg_isready — 检查一个IvorySQL服务器的连接状态

大纲

pg_isready [**connection-option**...] [**option**...]

选项

- **-d dbname --dbname=dbname**

指定要连接的数据库名。 **dbname**

可以是连接字符串。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。

- **-h hostname --host=hostname**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。默认值取自 **PGPORT** 环境变量。如果环境变量没有设置，则默认值使用编译时指定的端口（通常是 5432）。

- **-q --quiet**

不显示状态消息。当脚本编程时有用。

- **-t seconds --timeout=seconds**

尝试连接时，在返回服务器不响应之前等待的最大秒数。设置为 0 则禁用。默认值是 3 秒。

- **-U username --username=username**

作为用户 **username** 连接数据库，而不是用默认用户。

- **-V --version**

打印pg_isready版本并退出。

- **-? --help**

显示有关pg_isready命令行参数的帮助并退出。

退出状态

如果服务器正常接受连接，pg_isready返回 **0** 给 shell；如果服务器拒绝连接（例如处于启动阶段）则返回 **1**；如果连接尝试没有被相应则返回 **2**；如果没有尝试（例如由于非法参数）则返回 **3**。

环境

和大部分其他IvorySQL工具相似，**pg_isready** 也使用libpq支持的环境变量。

环境变量 **PG_COLOR** 规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

注解

要获得服务器状态，不一定需要提供正确的用户名、口令或数据库名。不过，如果提供了不正确的值，服务器将会记录一次失败的连接尝试。

例子

标准用法：

```
$ pg_isready
/tmp:5432 - accepting connections
$ echo $?
0
```

使用连接参数运行连接到处于启动中的IvorySQL集簇：

```
$ pg_isready -h localhost -p 5433
localhost:5433 - rejecting connections
$ echo $?
1
```

使用连接参数运行连接到无响应的IvorySQL集簇：

```
$ pg_isready -h someremotehost
someremotehost:5432 - no response
$ echo $?
2
```

pg_receivewal

pg_receivewal — 以流的方式从一个IvorySQL服务器得到预写式日志

大纲

pg_receivewal [option...]

选项

- **-D directory --directory=directory**

要把输出写到哪个目录。这个参数是必需的。

- **-E lsn --endpos=lsn**

当接收到达指定的LSN时，自动停止复制并且以正常退出状态0退出。如果有一个记录的LSN正好等于 **lsn**，则该记录将会被处理。

- **--if-not-exists**

当指定 **--create-slot** 并且具有指定名称的槽已经存在时不要抛出错误。

- **-n --no-loop**

不要在连接错误上循环。相反，碰到一个错误时立刻退出。

- **--no-sync**

这个选项导致 **pg_receivewal**

不强制WAL数据被刷回磁盘。这样会更快，但也意味着接下来的操作系统崩溃会让WAL段损坏。通常，这个选项对于测试有用，但不应该在对生产部署进行WAL归档时使用。这个选项与 **--synchronous** 不兼容。

- **-s interval --status-interval=interval**

指定发送回服务器的状态包之间的秒数。这允许我们更容易地监控服务器的进度。一个零值完全禁用这种周期性的状态更新，不过当服务器需要时还是会有一个更新会被发送来避免超时导致的断开连接。默认值是 10 秒。

- **-S slotname --slot=slotname**

要求pg_receivewal使用一个已有的复制槽。在使用这个选项时，pg_receivewal将会报告给服务器一个刷写位置，指示每一个段是何时被同步到磁盘的，这样服务器可以在不需要该段时移除它。当pg_receivewal的复制客户端在服务器上被配置为一个同步后备时，那么使用一个复制槽将会向服务器报告刷写位置，但只在一个WAL文件被关闭时报告。因此，该配置将导致主服务器上的事务等待很长的时间并且无法令人满意地工作。要让这种配置工作正确，还必须制定选项 **--synchronous**（见下文）。

- **--synchronous**

在WAL数据被收到后立即刷入到磁盘。还要在刷写后立即向服务器回送一个状态包（不考虑 **--status-interval**）。如果pg_receivewal的复制客户端在服务器上被配置为一个同步后备，应该指定这个选项来确保向服务器发送及时的反馈。

- **-v --verbose**

启用冗长模式。

- **-Z level --compress=level**

启用预写式日志上的gzip压缩，并且指定压缩级别（0到9，0是不压缩而9是最大压缩）。所有的文件名后都将被追加后缀 **.gz**。

下列命令行选项控制数据库连接参数。

- **-d connstr --dbname=connstr**

指定用于连接到服务器的参数，作为**连接字符串**；这些将覆盖所有冲突的命令行选项。为了和其他客户端应用一致，该选项被称为 **--dbname**。但是因为pg_receivewal并不连接到集簇中的任何特定数据库，连接字符串中的数据库名将被忽略。

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。默认值取自 **PGHOST** 环境变量（如果设置），否则会尝试一个 Unix 域套接字连接。

- **-p port --port=port**

指定服务器正在监听连接的TCP端口或本地 Unix 域套接字文件扩展。默认用 **PGPORT** 环境变量中的值（如果设置），或者一个编译在程序中的默认值。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制pg_receivewal在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，pg_receivewal将自动提示要求一个口令。但是，pg_receivewal将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

为了控制物理复制槽，pg_receivewal 可以执行下列两种动作之一：

- **--create-slot**

用 **--slot** 中指定的名称创建一个新的物理复制槽，然后退出。

- **--drop-slot**

删除 **--slot** 中指定的复制槽，然后退出。

其他选项也可用：

- **-V --version**

打印pg_receivewal版本并退出。

- **-? --help**

显示有关pg_receivewal命令行参数的帮助并退出。

退出状态

在被SIGINT信号终止（没有正常的方式结束它。因此这不是一种错误）时，pg_receivewal将以状态0退出。对于致命错误或者其他信号，退出状态将不是零。

环境

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

环境变量 **PG_COLOR** 规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

注解

在使用pg_receivewal替代 **archive_command** 作为主要的 WAL 备份方法时，强烈建议使用复制槽。否则，服务器可能会在预写式日志文件被备份好之前重用或者移除它们，因为没有任何信息（不管是来自 **archive_command** 或是复制槽）能够指示 WAL 流已经被归档到什么程度。不过要注意，如果接收者没有持续地取走 WAL 数据，一个复制槽将会填满服务器的磁盘空间。

如果在源集簇上启用了组权限，pg_receivewal将保留接收到的WAL文件上的组权限。

例子

要从位于 **mydbserver** 的服务器流式传送预写式日志并且将它存储在本地目录 `/usr/local/pgsql/archive``：

```
$ pg_receivewal -h mydbserver -D /usr/local/pgsql/archive
```

pg_recvlogical

pg_recvlogical — 控制 IvorySQL 逻辑解码流

大纲

pg_recvlogical [option...]

选项

必须至少要指定下列选项之一来选择动作：

- **--create-slot**

为 **--dbname** 指定的数据库用 **--slot** 指定的名称创建一个新的逻辑复制槽，使用 **--plugin** 指定的输出插件。

- **--drop-slot**

删除名称由 **--slot** 指定的复制槽，然后退出。

- **--start**

从 **--slot** 指定的逻辑复制槽开始进行流式传送更改，一直继续到被一个信号终止。如果服务器端关机或者断开连接导致更改流结束，会进入一个循环一直重试，通过指定 **--no-loop** 可以防止这种情况下进入循环重试。流格式由槽创建时指定的输出插件决定。连接必须是连接到用于创建该槽的同一个数据库上。

--create-slot 和 **--start** 可以被一起指定。 **--drop-slot** 不能和另一个动作组合在一起。

下面的命令行选项控制输出的位置和格式以及其他复制行为：

- **-E 'lsn' --endpos='lsn'**

在 **--start**

模式中，当接收过程到达指定的LSN时会自动地停止复制并且以正常的退出状态0退出。如果不处于 **--start** 模式时指定这个选项，则会发生错误。如果有一个记录的LSN正好等于 **lsn**，则该记录将被输出。**--endpos** 不会察觉到事务边界并且可能会在一个事务中间截断输出。任何部分输出的事务都将不会被消费，并且在下一次从该槽中读取时将会重放该事务。单个的消息不会被截断。

- **-f filename --file=filename**

把接收到并且解码好的事务数据写入到一个文件。使用 **-** 可以写到stdout。

- **-F interval_seconds --fsync-interval=interval_seconds**

指定pg_recvlogical发出 **fsync()**

调用确保输出文件被安全地刷到磁盘的频率。服务器将会偶尔要求客户端执行一次刷写并且把刷写位置报告给服务器。这个设置可以在此之外更加频繁地执行刷写。指定间隔为 **0** 会完全禁止发出 **fsync()** 调用，但是仍会报告进度给服务器。在这种情况下，发生崩溃会导致数据丢失。

- **-I lsn --startpos=lsn**

在 **--start** 模式中，从给定的 LSN 开始复制。

- **--if-not-exists**

当指定 **--create-slot** 并且具有指定名称的槽已经存在时不要抛出错误。

- **-n --no-loop**

当服务器连接丢失时，不要在循环中重试，直接退出。

- **-o name [=value] --option=name[=value]**

如果指定了输出插件，把选项值 **value** 传递给选项 **name**。存在哪些选项以及它们的效果取决于使用的输出插件。

- **-P plugin --plugin=plugin**

在创建一个槽时使用指定的逻辑解码输出插件。如果该槽已经存在，这个选项没有效果。

- **-s interval_seconds --status-interval=interval_seconds**

这个选项和 **pg_receivewal** 中的同名选项具有相同的效果。请参考那里的描述。

- **-S slot_name --slot=slot_name**

在 **--start** 模式中，使用名为 **slot_name** 的已有逻辑复制槽。在 **--create-slot** 模式中，使用这个名称创建该槽。在 **--drop-slot** 模式中，删除这个名称指定的槽。

- **-v --verbose**

开启详细输出模式。

下列命令行选项控制数据库连接参数。

- **-d database --dbname=database**

要连接的数据库。这个选项的详细含义请见动作的描述。**dbname** 可以是 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。默认为用户名。

- **-h hostname-or-ip --host=hostname-or-ip**

指定服务器正在运行的机器的主机名。如果该值开始于一个斜线，它被用作一个 Unix 域套接字的目录。默认是从 `PGHOST` 环境变量中取得（如果被设置），否则将尝试一次 Unix 域套接字连接。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展名。默认是放在 **PGPORT** 环境变量中（如果被设置），否则使用编译在程序中的默认值。

- **-U user --username=user**

要作为哪个用户连接。默认是用当前操作系统用户名。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那么连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制 **pg_dump** 在连接到一个数据库之前提示要求一个口令。这个选项不是必须的，因为如果服务器要求口令认证，**pg_dump** 将自动提示要求一个口令。但是，**pg_dump** 将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下，值得键入 **-W** 来避免额外的连接尝试。

还有下列附加选项可用：

- **-V --version**

打印pg_recvlogical的版本并且退出。

- **-? --help**

显示关于pg_recvlogical命令行参数的帮助，并且退出。

环境

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

环境变量`PG\_COLOR`规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

注解

如果在源服务器上启用了组权限，pg_recvlogical将会在接收到的WAL文件上保留组权限。

pg_restore

pg_restore — 从一个由pg_dump创建的归档文件恢复一个IvorySQL数据库

大纲

pg_restore [**connection-option**...] [**option**...] [**filename**]

选项

pg_restore接受下列命令行参数。

- **filename**

指定要被恢复的归档文件（对于一个目录格式的归档则是目录）的位置。如果没有指定，则使用标准输入。

- **-a --data-only**

只恢复数据，不恢复模式（数据定义）。如果在归档中存在，表数据、大对象和序列值会被恢复。这个选项类似于指定 **--section=data**，但是由于历史原因两者不完全相同。

- **-c --clean**

在重新创建数据库对象之前清除（丢弃）它们（除非使用了 **--if-exists**，如果有对象在目标数据库中不存在，这可能会生成一些无害的错误消息）。

- **-C --create**

在恢复一个数据库之前会创建它。如果还指定了 **--clean**，在连接到目标数据库之前丢弃并且重建它。如果使用 **--create**，pg_restore还会恢复数据库的注释（如果有）以及与其相关的配置变量设置，也就是任何提到过这个数据库的 **ALTER DATABASE ... SET ...** 和 **ALTER ROLE ... IN DATABASE ... SET ...** 命令。不管是否指定 **--no-acl**，数据库本身的访问特权都会被恢复。在使用这个选项时，**-d** 提到的数据库只被用于发出初始的 **DROP DATABASE** 和 **CREATE DATABASE** 命令。所有要恢复到该数据库名中的数据都出现在归档中。

- **-d dbname --dbname=dbname**

连接到数据库 **dbname** 并且直接恢复到该数据库中。**dbname** 可以是 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。

- **-e --exit-on-error**

在发送 SQL

命令到该数据库期间如果碰到一个错误就退出。默认行为是继续并且在恢复结束时显示一个错误计数。

- **-f filename --file=filename**

为生成的脚本指定输出文件，或在与 **-l** 选项一起使用时为列表指定输出文件。为 stdout 用 **-o**。

- **-F format --format=format**

指定归档的格式。并不一定要指定该格式，因为 `pg_restore` 将会自动决定格式。如果指定，可以是下列之一：
• **c custom** 归档是 `pg_dump` 的自定义格式。
• **d directory** 归档是一个目录归档。
• **t tar** 归档是一个 **tar** 归档。

- **-I index --index=index**

只恢复提及的索引的定义。可以通过写多个 **-I** 开关指定多个索引。

- **-j number-of-jobs --jobs=number-of-jobs**

使用并发任务运行 `pg_restore` 中最耗时的步骤 — 载入数据、创建索引或者创建约束；同时，最多使用 **number-of-jobs** 个并发会话。对于一个运行在多处理器机器上的服务器，这个选项能够大幅度减少恢复一个大型数据库的时间。当发出脚本而不是直接连接到数据库服务器时，将忽略此选项。每一个任务是一个进程或者一个线程，这取决于操作系统，它们都使用一个独立的服务器连接。这个选项的最佳值取决于服务器、客户端以及网络的硬件设置。因素包括 CPU 的核心数和磁盘设置。一个好的建议是服务器上 CPU 的核心数，但是更大的值在很多情况下也能导致更快的恢复时间。当然，过高的值会由于超负荷反而导致性能降低。这个选项只支持自定义和目录归档格式。输入必须是一个常规文件或目录（例如，不能是一个管道或者标准输出）。还有，多任务不能和选项 **--single-transaction** 一起用。

- **-l --list**

列出归档的内容的表格。这个操作的输出能被用作 **-L** 选项的输入。注意如果把 **-n** 或 **-t** 这样的过滤开关与 **-l** 一起使用，它们将会限制列出的项。

- **-L list-file --use-list=list-file**

只恢复在 **list-file** 中列出的归档元素，并且按照它们出现在该文件中的顺序进行恢复。注意如果把 **-n** 或 **-t** 这样的过滤开关与 **-L** 一起使用，它们将会进一步限制要恢复的项。**list-file** 通常是编辑一个 **-l** 操作的输出来创建。行可以被移动或者移除，并且也可以通过在行首放一个 **(;**) 将其注释掉。例子见下文。

- **-n schema --schema=schema**

只恢复在被提及的模式中的对象。可以用多个 **-n** 开关来指定多个模式。这可以与 **-t** 选项组合在一起只恢复一个指定的表。

- **-N schema --exclude-schema=schema**

不恢复所提及方案中的对象。可以用多个 **-N** 开关指定多个要被排除的方案。如果对同一个方案名称同时给出了 **-n** 和 **-N**，则 **-N** 会胜出并且该方案会被排除。

- **-O --no-owner**

不要输出将对象的所有权设置为与原始数据库匹配的命令。默认情况下，`pg_restore` 会发出 **ALTER OWNER** 或者 **SET SESSION AUTHORIZATION**

语句来设置已创建的模式对象的所有权。除非到该数据库的初始连接是一个超级用户（或者拥有脚本中所有对象的同一个用户）建立的，这些语句将会失败。通过 **-O**，任何用户名都可以被用于初始连接，并且这个用户将会拥有所有被创建的对象。

- **-P function-name(argtype [, ...]) --function=function-name(argtype [, ...])**

只恢复被提及的函数。要小心地拼写函数的名称和参数使它们正好就是出现在转储文件的内容表中的名称和

参数。可以使用多个 **-P** 开关指定多个函数。

- **-R --no-reconnect**

这个选项已被废弃，但是出于向后兼容性的目的，系统仍然还接受它。

- **-s --schema-only**

只恢复归档中的模式（数据定义）不恢复数据。这个选项是 **--data-only** 的逆选项。它与指定 **--section=pre-data --section=post-data** 相似，但是由于历史原因并不完全相同。（不要把这个选项和 **--schema** 选项弄混，后者把词“schema”用于一种不同的含义）。

- **-S username --superuser=username**

指定在禁用触发器时要用的超级用户名。只有使用 **--disable-triggers** 时这个选项才相关。

- **-t table --table=table**

只恢复所提及的表的定义和数据。出于这个目的，“table”包括视图、物化视图、序列和外部表。可以写上多个 **-t** 开关可以选择多个表。这个选项可以和 **-n** 选项结合在一起指定一个特定模式中的表。注意在指定 **-t** 时，`pg_restore` 不会尝试恢复所选表可能依赖的任何其他数据库对象。因此，无法确保能成功地把一个特定表恢复到一个干净的数据库中。注意这个标志的行为和 `pg_dump` 的 **-t** 标志不一样。在 `pg_restore` 中当前没有任何通配符匹配的规定，也不能在其 **-t** 选项中包括模式的名称。而且，虽然 `pg_dump` 的 **-t** 标志也会转储选中表的附属对象（例如索引），但是 `pg_restore` 的 **-t** 标志不包括这些附属对象。

- **-T trigger --trigger=trigger**

只恢复所提及的触发器。可以用多个 **-T** 开关指定多个触发器。

- **-v --verbose**

指定冗长模式。

这将导致 `pg_restore` 输出详细的对象注释和启动/停止时间到输出文件，并将推送消息输出到标准错误。重复该选项会造成附加的调试级消息出现在标准报错上。

- **-V --version**

打印该 `pg_restore` 的版本并退出。

- **-x --no-privileges --no-acl**

阻止恢复访问特权（授予/收回命令）。

- **-1 --single-transaction**

将恢复作为单一事务执行（即把发出的命令包裹在 **BEGIN / COMMIT** 中）。这可以确保要么所有命令完全成功，要么任何改变都不被应用。这个选项隐含了 **--exit-on-error**。

- **--disable-triggers**

只有在执行一个只恢复数据的恢复时，这个选项才相关。它指示 `pg_restore` 在装载数据时执行命令临时禁用目标表上的触发器。如果你在表上有参照完整性检查或者其他触发器并且你不希望在数据载入期间调用它们时，请使用这个选项。目前，为 **--disable-triggers** 发出的命令必须以超级用户身份完成。因此你还应该用 `\-` 指定一个超级用户名，或者更好的方法是以一个 `IvorySQL` 超级用户运行 `pg_restore`。

- **--enable-row-security**

只有在恢复具有行安全性的表的内容时，这个选项才相关。默认情况下，`pg_restore` 将把 `row_security` 设置为 `off`

来确保所有数据都被恢复到表中。如果用户不拥有足够绕过行安全性的特权，那么会抛出一个错误。这个参数指示 `pg_restore` 把 `row_security`

设置为on允许用户尝试恢复启用了行安全性的表的内容。如果用户没有从转储向表中插入行的权限，这仍将失败。注意当前这个选项还要求转储处于 **INSERT** 格式，因为 **COPY FROM** 不支持行安全性。

- **--if-exists**

使用条件命令（即增加一个 **IF EXISTS** 子句）删除数据库对象。只有指定了 **--clean** 时，这个选项才有效。

- **--no-comments**

即便归档中包含注释也不输出恢复注释的命令。

- **--no-data-for-failed-tables**

默认情况下，即便表的创建命令失败（例如因为表已经存在），表数据也会被恢复。通过这个选项，对这类表的数据会被跳过。如果目标数据库已经包含了想要的表内容，这种行为又很有用。例如，IvorySQL扩展（如PostGIS）的辅助表可能已经被载入到目标数据库中，指定这个选项就能阻止把重复的或者废弃的数据载入到这些表中。只有当直接恢复到一个数据库中时这个选项才有效，在产生SQL脚本输出时这个选项不会产生效果。

- **--no-publications**

即便归档中包含publication也不输出恢复publication的命令。

- **--no-security-labels**

不要输出恢复安全标签的命令，即使归档中包含安全标签。

- **--no-subscriptions**

即便归档中包含subscription也不输出恢复subscription的命令。

- **--no-tablespaces**

不输出命令选择表空间。通过这个选项，所有的对象都会被创建在恢复时的默认表空间中。

- **--section='sectionname'**

只恢复提及的小节。小节的名称可以是 **pre-data**、**data** 或者 **post-data**。可以把这个选项指定多次来选择多个小节。默认值是恢复所有小节。数据小节包含实际的表数据以及大对象定义。Post-data 项由索引定义、触发器、规则和除已验证的检查约束之外的约束构成。Pre-data 项由所有其他数据定义项构成。

- **--strict-names**

要求每一个模式（**-n / --schema**）以及表（**-t / --table**）限定词匹配备份文件中至少一个模式/表。

- **--use-set-session-authorization**

输出 SQL 标准的 **SET SESSION AUTHORIZATION** 命令取代 **ALTER OWNER** 命令来决定对象拥有权。这会让转储更加兼容标准，但是依赖于转储中对象的历史，可能无法正确恢复。

- **-? --help**

显示有关pg_restore命令行参数的帮助，并且退出。

pg_restore也接受下列用于连接参数的命令行参数：

- **-h host --host=host**

指定服务器正在运行的机器的主机名。如果该值开始于一个斜线，它被用作一个 Unix 域套接字的目录。默认是从 **PGHOST** 环境变量中取得（如果被设置），否则将尝试一次 Unix 域套接字连接。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展名。默认是放在 **PGPORT** 环境变量中（如果被设置），否则使用编译在程序中的默认值。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那么连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制 **pg_restore** 在连接到一个数据库之前提示要求一个口令。这个选项不是必须的，因为如果服务器要求口令认证，**pg_restore** 将自动提示要求一个口令。但是，**pg_restore** 将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下，值得键入 **-W** 来避免额外的连接尝试。

- **--role='rolename'**

指定一个用来创建该转储的角色名。这个选项导致 **pg_restore** 在连接到数据库后发出一个 **SET ROLE rolename** 命令。当已认证用户（由 **-U** 指定）缺少 **pg_restore** 所需的特权但是能够切换到一个具有所需权利的角色时，这个选项很有用。一些安装有针对直接作为超级用户登录的策略，使用这个选项可以让转储在不违反该策略的前提下完成。

环境

- **PGHOST PGOPTIONS PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

和大部分其他 **IvorySQL** 工具相似，这个工具也使用 **libpq** 支持的环境变量。

诊断

当使用 **-d** 选项指定一个直接数据库连接时，**pg_restore** 在内部执行 **SELECT** 语句。如果你运行 **pg_restore** 时出现问题，确定你能够从正在使用的数据库中选择信息，例如 **psql**。此外，**libpq** 前端-后端库所使用的任何默认连接设置和环境变量都将适用。

注解

如果你的数据库集簇对于 **template1** 数据库有任何本地添加，要注意将 **pg_restore** 的输出载入到一个真正的空数据库。否则你很可能由于以增加对象的重复定义而得到错误。要创建一个不带任何本地添加的空数据库，从 **template0** 而不是 **template1** 复制它，例如：

```
CREATE DATABASE foo WITH TEMPLATE template0;
```

下面将详细介绍 **pg_restore** 的局限性。

- 在恢复数据到一个已经存在的表中并且使用了选项 **--disable-triggers** 时，**pg_restore** 会在插入数据之前发出命令禁用用户表上的触发器，然后在完成数据插入后重新启用它们。如果恢复在中途停止，可能会让系统目录处于错误的状态。

- `pg_restore`不能有选择地恢复大对象，例如只恢复特定表的大对象。如果一个归档包含大对象，那么所有的大对象都会被恢复，如果通过 `-L`、`-t` 或者其他选项进行了排除，它们一个也不会被恢复。

一旦完成恢复，应该在每一个被恢复的表上运行 **ANALYZE**，这样优化器能得到有用的统计信息。

示例

假设我们已经以自定义格式转储了一个叫做 **mydb** 的数据库：

```
$ pg_dump -Fc mydb > db.dump
```

要删除该数据库并且从转储中重新创建它：

```
$ dropdb mydb
$ pg_restore -C -d postgres db.dump
```

`-d` 开关中提到的数据库可以是任何已经存在于集群中的数据库，`pg_restore`只会用它来为 **mydb** 发出 **CREATE DATABASE** 命令。通过 `-C`，数据总是会被恢复到出现在归档文件的数据库名中。

要把转储重新载入到一个名为 **newdb** 的新数据库中：

```
$ createdb -T template0 newdb
$ pg_restore -d newdb db.dump
```

注意我们不使用 `-C`，而是直接连接到要恢复到其中的数据库。还要注意我们是从 **template0** 而不是 **template1** 创建了该数据库，以保证它最初是空的。

要对数据库项重排序，首先需要转储归档的表内容：

```
$ pg_restore -l db.dump > db.list
```

列表文件由一个头部和一些行组成，这些行每一个都用于一个项，例如：

```

;
; Archive created at Mon Sep 14 13:55:39 2009
;   dbname: DBDEMOS
;   TOC Entries: 81
;   Compression: 9
;   Dump Version: 1.10-0
;   Format: CUSTOM
;   Integer: 4 bytes
;   Offset: 8 bytes
;   Dumped from database version: 8.3.5
;   Dumped by pg_dump version: 8.3.8
;
;
```

```
; Selected TOC Entries:
;
3; 2615 2200 SCHEMA - public pasha
1861; 0 0 COMMENT - SCHEMA public pasha
1862; 0 0 ACL - public pasha
317; 1247 17715 TYPE public composite pasha
319; 1247 25899 DOMAIN public domain0 pasha
```

分号表示开始一段注释，行首的数字表明了分配给每个项的内部归档 ID。

文件中的行可以被注释掉、删除以及重排序。例如：

```
10; 145433 TABLE map_resolutions postgres
;2; 145344 TABLE species postgres
;4; 145359 TABLE nt_header postgres
6; 145402 TABLE species_records postgres
;8; 145416 TABLE ss_old postgres
```

把这样一个文件作为pg_restore的输入将会只恢复项 10 和 6，并且先恢复 10 再恢复 6。

```
$ pg_restore -L db.list db.dump
```

pg_verifybackup

pg_verifybackup — 验证IvorySQL集群的基础备份的完整性

大纲

pg_verifybackup [option...]

选项

pg_verifybackup 接受以下命令行参数：

- **-e --exit-on-error**

检测到备份问题后立即退出。 如果没有指定这个选项，**pg_verifybackup** 将在检测到问题后继续检查备份，并将检测到的所有问题报告为错误。

- **-i path --ignore=path**

在将备份中实际存在的数据文件列表与 **backup_manifest**

文件中列出的数据文件列表进行比较时，忽略指定的文件或目录，该文件或目录应表示为相对路径名。如果指定了目录，则此选项会影响以该位置为根的整个子树。

如果相对路径名与指定的路径名匹配，有关额外文件、丢失文件、文件大小差异或校验和不匹配的投诉将被抑制。可以多次指定此选项。

- **-m path --manifest-path=path**

使用指定路径的清单文件，而不是位于备份目录根目录中的清单文件。

- **-n --no-parse-wal**

不要试图解析从该备份恢复所需的预写式日志数据。

- **-q --quiet**

成功验证备份后不要打印任何内容。

- **-s --skip-checksums**

不要验证数据文件校验和。但仍检查是否存在文件以及这些文件的大小。这样将会快得多，因为文件本身不需要读取。

- **-w path --wal-directory=path**

尝试解析存储在指定目录中的 WAL 文件，而不是 **pg_wal**。如果备份存储在与 WAL 存档不同的位置，则这可能很有用。

其他选项也可用：

- **-V --version**

打印 **pg_verifybackup** 版本并退出。

- **-? --help**

显示有关 **pg_verifybackup** 命令行参数的帮助，然后退出。

示例

要在 **mydbserver** 上创建服务器的基本备份并验证备份的完整性：

```
$ pg_basebackup -h mydbserver -D /usr/local/pgsql/data
$ pg_verifybackup /usr/local/pgsql/data
```

要在 **mydbserver** 上创建服务器的基本备份，请将清单移动到备份目录之外的某个位置，并验证备份：

```
$ pg_basebackup -h mydbserver -D /usr/local/pgsql/backup1234
$ mv /usr/local/pgsql/backup1234/backup_manifest
/my/secure/location/backup_manifest.1234
$ pg_verifybackup -m /my/secure/location/backup_manifest.1234
/usr/local/pgsql/backup1234
```

要在忽略手动添加到备份目录的文件的同时验证备份，并跳过校验和验证：

```
$ pg_basebackup -h mydbserver -D /usr/local/pgsql/data
$ edit /usr/local/pgsql/data/note.to.self
$ pg_verifybackup --ignore=note.to.self --skip-checksums /usr/local/pgsql/data
```

psql

psql — IvorySQL 的交互式终端

大纲

`psql [option...] [dbname [username]]`

选项

- `-a --echo-all`

把所有非空输入行按照它们被读入的形式打印到标准输出（不适用于交互式行读取）。这等效于把变量 `ECHO` 设置为 `all`。

- `-A --no-align`

切换到非对齐输出模式（默认输出模式是 `对齐` 的）。这等效于 `\pset format unaligned`。

- `-b --echo-errors`

把失败的 SQL 命令打印到标准错误输出。这等效于把变量 `ECHO` 设置为 `errors`。

- `-c command --command=command`

指定psql执行一个给定的命令字符串 `command`。这个选项可以重复多次并且以任何顺序与 `-f` 选项组合在一起。当 `-c` 或者 `-f` 被指定时，psql不会从标准输入读取命令，直到它处理完序列中所有的 `-c` 和 `-f` 选项之后终止。`command`

必须是一个服务器完全可解析的命令字符串（即不包含psql相关的特性）或者单个反斜线命令。因此不能在一个 `-c` 选项中混合SQL和psql元命令。要那样做，可以使用多个 `-c`

选项或者把字符串用管道输送到psql中，例如：`psql -c '\x' -c 'SELECT * FROM foo;'` 或者 `echo '\x \ SELECT * FROM foo;' | psql`（`\` 是分隔符元命令）。每一个被传递给 -c`

的SQL命令字符串会被当做一个单独的请求发送给服务器。因此，即便该字符串包括多个SQL命令，服务器也会把它当做一个事务来执行，除非在该字符串中有显式的 `BEGIN / COMMIT`

命令把它划分成多个事务。此外，psql只会打印出该字符串中最后一个SQL命令的结果。这和从文件中读取同一字符串或者把同一字符串传给psql的标准输出时的行为不同，因为那两种情况下psql会独立地发送每一个SQL命令。由于这种行为，把多于一个SQL命令放在 `-c`

字符串中通常会得到意料之外的结果。最好使用多个 `-c`

命令或者把多个命令输送给psql的标准输入，按照上文所说的使用echo或者通过一个shell，例如：``psql <<EOF \x SELECT * FROM foo; EOF``

- `--CSV`

切换到CSV（逗号分隔值）输出模式。这相当于 `\pset format csv`。

- `-d dbname --dbname=dbname`

指定要连接的数据库的名称。这等效于指定 `dbname` 为命令行上的第一个非选项参数。`dbname` 可以是 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。

- `-e --echo-queries`

也把发送到服务器的所有 SQL 命令复制到标准输出。这等效于把变量 `ECHO` 设置为 `queries`。

- `-E --echo-hidden`

回显 `\d` 以及其他反斜线命令生成的实际查询。可以用它来学习psql的内部操作。这等效于把变量 `ECHO_HIDDEN` 设置为 `on`。

- `-f filename --file=filename`

从文件 `filename` 而不是标准输入中读取命令。这个选项可以被重复多次，也可以以任意顺序与 `-c` 选项组合。当 `-c` 或者 `-f` 被指定时，psql不会从标准输入读取命令，直到它处理完序列中所有的 `-c` 和 `-f` 选项之后终止。除此以外，这个选项很大程度上等价于元命令 `\i`。如果 `filename` 是 `-`

（连字符），那么会读取标准输入直到遇见一个 EOF 指示或者 `\q` 元命令。这种方式可以用把自多个文件的输入组合成一种交互式输入。不过注意在这种情况下不会使用

Readline（很像指定了 `-n` 的情况）。使用这个选项与 `psql < filename` 有细微的不同。通常，两种形式都可以做到我们所期望的，但是使用 `-f` 启用了一些好的特性，例如带有行号的错误消息。使用这个选项还有一丝机会可以降低启动开销。在另一方面，使用shell输入重定向的变体（理论上）保证会得到与手工输入时相同的输出。

- `-F separator --field-separator=separator`

使用 `separator` 作为非对齐输出的域分隔符。这等效于 `\pset fieldsep` 或者 `\f`。

- `-h hostname --host=hostname`

指定运行服务器的机器的主机名。如果这个值由一个斜线开始，它会被用作 Unix 域套接字的目录。

- `-H --html`

切换到HTML输出模式。这等效于 `\pset format html` 或者 `\H` 命令。

- `-l --list`

列出所有可用的数据库，然后退出。其他非连接选项会被忽略。这与元命令 `\list` 类似。在使用这个选项时，psql将连接到数据库 `postgres`，除非在命令行上提及一个不同的数据（选项 `-d` 或非选项参数，可能是通过一个服务项，但不能通过一个环境变量）。

- `-L filename --log-file=filename`

除了把所有查询输出写到普通输出目标之外，还写到文件 `filename` 中。

- `-n --no-readline`

不使用Readline做行编辑并且不使用命令历史。在剪切和粘贴时，关掉 Tab 展开会有所帮助。

- `-o `filename` --output=`filename``

把所有查询输出放到文件 `filename` 中。这等效于命令 `\o`。

- `-p port --port=port`

指定服务器用于监听连接的 TCP 端口或者本地 Unix 域套接字文件扩展。默认是 `PGPORT` 环境变量的值，如果没有设置，则默认为编译时指定的端口号（通常是5432）。

- `-P assignment --pset=assignment`

以 `\pset` 的形式指定打印选项。注意，这里你必须用一个等号而不是空格来分隔名称和值。例如，要设置输出格式为 LaTeX，应该写成 `-P format=latex`。

- `-q --quiet`

指定psql应该安静地工作。默认情况下，它会打印出欢迎消息以及多种输出。如果使用了这个选项，以上那些就都不会输出。在使用 `-c` 选项时，配合这个选项很有用。这等效于设置变量 `QUIET` 为 `on`。

- `-R separator --record-separator=separator`

把 `separator` 用作非对齐输出的记录分隔符。这等效于 `\pset recordsep` 命令。

- `-s --single-step`

运行在单步模式中。这意味着在每个命令被发送给服务器之前都会提示用户一个可以取消执行的选项。使用这个选项可以调试脚本。

- `-S --single-line`

运行在单行模式中，其中新行会终止一个 SQL

命令，就像分号的作用一样。注意这种模式被提供给那些坚持使用它的用户，但是并不一定要使用它。特别地，如果在一行中混合了SQL和元命令，那对于没有经的用户来说，它们的执行顺序可能不总是那么清晰。

- **-t --tuples-only**

关闭打印列名和结果行计数页脚等。这等效于 `\t` 或者 `\pset tuples_only` 命令。

- **-T table_options --table-attr=table_options**

指定要替换HTML **table** 标签的选项。详见 `\pset tableattr`。

- **-U username --username=username**

作为用户 **username** 而不是默认用户连接到数据库（当然，你必须具有这样做的权限）。

- **-v assignment --set=assignment --variable=assignment**

执行一次变量赋值，和 `\set`

元命令相似。注意你必须在命令行上用等号分隔名字和值（如果有）。要重置一个变量，去掉等号就行。要把一个变量置为空值，使用等号但是去掉值。这些赋值在命令行处理期间被完成，因此反映连接状态的变量将在稍后被覆盖。

- **-V --version**

打印psql版本并且退出。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且口令不能从其他来源（例如一个 **.pgpass** 文件）获得，那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。注意这个选项将对整个会话保持设置，并且因此它会影响元命令 `\connect` 的使用，就像初始的连接尝试那样。

- **-W --password**

强制psql在连接到一个数据库之前提示要求一个口令，即使口令不会被使用。如果服务器需要口令认证并且口令不能从其他来源获得，例如 **.pgpass** 文件，psql 在任何情况下都会提示输入口令。然而，psql 将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。注意这个选项将对整个会话保持设置，并且因此它会影响元命令 `\connect` 的使用，就像初始的连接尝试那样。

- **-x --expanded**

打开扩展表格模式。这等效于 `\x` 或者 `\pset expanded` 命令。

- **-X, --no-psqlrc**

不读取启动文件（要么是系统范围的 **psqlrc** 文件，要么是用户的 `~/.psqlrc` 文件）。

- **-z --field-separator-zero**

设置非对齐输出的域分隔符为零字节。这等效于 `\pset fieldsep_zero`。

- **-0 --record-separator-zero**

设置非对齐输出的记录分隔符为零字节。例如，这对与 **xargs -0** 配合有关。这等效于 `\pset recordsep_zero`。

- **-1 --single-transaction**

这个选项只能被用于与一个或者多个 **-c** 以及/或者 **-f** 选项组合。它会让psql在第一个上述选项之前发出一条 **BEGIN** 命令并且在最后一个上述选项之后发出一条 **COMMIT**

命令，这样就把所有的命令都包裹在一个事务中。这个选项可以保证要么所有的命令都成功地完成，要么不

应用任何更改。如果命令本身包含 **BEGIN**、**COMMIT** 或者 **ROLLBACK**，这个选项将不会得到想要的效果。还有，如果当个命令不能在一个事务块中执行，指定这个选项将导致整个事务失败。

- **-? --help[='topic']`**

显示有关psql的帮助并且退出。可选的 **topic** 参数（默认为 **options**）选择要解释哪一部分的psql：**commands** 描述psql的反斜线命令；**options** 描述可以被传递给psql的命令行选项；而 **variables** 则显示有关psql配置变量的帮助。

退出状态

如果psql正常完成，它会向 shell 返回

0。如果它自身发生一个致命错误（例如内存用完、找不到文件），它会返回

1。如果到服务器的连接出问题并且事务不是交互式的，它会返回 2。如果在脚本中发生错误，它会返回 3 并且变量 **ON_ERROR_STOP** 会被设置。

环境

- **COLUMNS**

如果 **\pset columns** 为零，这个环境变量控制用于 **wrapped** 格式的宽度以及用来确定是否输出需要用到分页器或者切换到扩展自动模式中的垂直格式的宽度。

- **PGDATABASE PGHOST PGPORT PGUSER**

默认连接参数。

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

- **PSQL_EDITOR EDITOR VISUAL**

\e、**\ef** 以及 **\ev**

命令所使用的编辑器。会按照列出的顺序检查这些变量，第一个被设置的将被使用。如果都没有被设置，默认是使用Unix系统上的 **vi** 或者Windows系统上的 **notepad.exe**。

- **PSQL_EDITOR_LINENUMBER_ARG**

当 **\e**、**\ef** 或者 **\ev**

带有一个行号参数时，这个变量指定用于传递起始行号给用户编辑器的命令行参数。对于Emacs或者vi之类的编辑器，这个变量是一个加号。如果需要在选项名称和行号之间有空格，可以在该变量的值中包括一个结尾的空格。例如：**PSQL_EDITOR_LINENUMBER_ARG=' ' PSQL_EDITOR_LINENUMBER_ARG='--line ' `** 在 Unix 系统上默认是 **`**（对应于默认编辑器 **vi**，且对很多其他常见编辑器可用）。在 Windows 系统上没有默认值。

- **PSQL_HISTORY**

命令历史文件的替代位置。波浪线（~）扩展会被执行。

- **PSQL_PAGER PAGER**

如果一个查询的结果在屏幕上放不下，它们会通过这个命令分页显示。典型的值是 **more** 或 **less**。通过把

PSQL_PAGER 或 **PAGER** 设置为空字符串可以禁用分页器的使用，调整 **\pset**

命令与分页器相关的选项也能达到同样的效果。会按照列出的顺序检查这些变量，第一个被设置的将被使用。如果都没有被设置，则大部分平台上默认使用 **more**，但在Cygwin上使用 **less**。

- **PSQLRC**

用户的 **.psqlrc** 文件的替代位置。波浪线（~）扩展会被执行。

- **SHELL**

被 **\!** 命令执行的命令。

- **TMPDIR**

存储临时文件的目录。默认是 **/tmp**。

和大部分其他IvorySQL工具一样，这个工具也使用libpq所支持的环境变量。

文件

- **psqlrc** and **~/.psqlrc**

如果没有 **-X** 选项，在连接到数据库后但在接收正常的命令之前，psql会尝试依次从系统级的启动文件（**psqlrc**）和用户的个人启动文件（**~/.psqlrc**）中读取并且执行命令。这些文件可以被用来设置客户端或者服务器，通常是一些 **\set** 和 **SET** 命令。系统级的启动文件是 **psqlrc**，它应该在安装好的IvorySQL的“系统配置”目录中，最可靠的定位方法是运行 **pg_config --sysconfdir**。默认情况下，这个目录将是 **../etc/**（相对于包含IvorySQL可执行文件的目录）。可以通过 **PGSYSCONFDIR** 环境变量显式地设置这个目录的名称。用户个人的启动文件是 **.psqlrc**，它应该在调用用户的主目录中。在 Windows 上，由于没有用户主目录的概念，个人的启动文件是 **%APPDATA%\postgresql\psqlrc.conf**。用户启动文件的位置可以通过 **PSQLRC** 环境变量设置。系统级和用户个人的启动文件都可以弄成是针对特定psql版本的，方法是在文件名后面加上一个横线以及IvorySQL的主、次版本号，版本最为匹配的文件会优先于不那么匹配的文件读入。

- **.psql_history**

命令行历史被存储在文件 **~/.psql_history** 中，或者是 Windows 的文件 **%APPDATA%\postgresql\psql_history** 中。历史文件的位置可以通过 **HISTFILE** psql变量或者 **PSQL_HISTORY** 环境变量明确的设置。

注解

- psql和具有相同主版本或者更老的主版本服务器最为匹配。如果服务器的版本比psql本身要高，则反斜线命令尤其容易失败。运行 SQL 命令并且显示查询结果的一般功能应该也能和具有更新主版本的服务器一起使用，但是并非在所有的情况下都能保证如此。

如果你想用psql连接到多个具有不同主版本的服务器，推荐使用最新版本的psql。或者，你可以为每一个主版本保留一份psql拷贝，并且针对相应的服务器使用匹配的版本。但实际上，这种额外的麻烦是不必要的。

示例

第一个例子展示了如何如何跨越多行输入一个命令。注意提示符的改变：

```
testdb=> CREATE TABLE my_table (
testdb(> first integer not null default 0,
testdb(> second text)
testdb-> ;
CREATE TABLE
```

现在再看看表定义：

```
testdb=> \d my_table
Table "public.my_table"
```

Column	Type	Collation	Nullable	Default
first	integer		not null	0
second	text			

现在我们把提示符改一改：

```
testdb=> \set PROMPT1 '%n@m %~%R%# '
peter@localhost testdb=>
```

假定已经用数据填充了这个表并且想看看其中的数据：

```
peter@localhost testdb=> SELECT * FROM my_table;
 first | second
-----+-----
      1 | one
      2 | two
      3 | three
      4 | four
(4 rows)
```

你可以用 `\pset` 命令以不同的方式显示表：

```
peter@localhost testdb=> \pset border 2
Border style is 2.
peter@localhost testdb=> SELECT * FROM my_table;
+-----+-----+
| first | second |
+-----+-----+
|      1 | one    |
|      2 | two    |
|      3 | three  |
|      4 | four   |
+-----+-----+
(4 rows)

peter@localhost testdb=> \pset border 0
Border style is 0.
peter@localhost testdb=> SELECT * FROM my_table;
first second
-----
1 one
2 two
```

```

3 three
4 four
(4 rows)

peter@localhost testdb=> \pset border 1
Border style is 1.
peter@localhost testdb=> \pset format csv
Output format is csv.
peter@localhost testdb=> \pset tuples_only
Tuples only is on.
peter@localhost testdb=> SELECT second, first FROM my_table;
one,1
two,2
three,3
four,4
peter@localhost testdb=> \pset format unaligned
Output format is unaligned.
peter@localhost testdb=> \pset fieldsep '\t'
Field separator is "    ".
peter@localhost testdb=> SELECT second, first FROM my_table;
one      1
two      2
three    3
four     4

```

或者使用短命令：

```

peter@localhost testdb=> \a \t \x
Output format is aligned.
Tuples only is off.
Expanded display is on.
peter@localhost testdb=> SELECT * FROM my_table;
-[ RECORD 1 ]-
first | 1
second | one
-[ RECORD 2 ]-
first | 2
second | two
-[ RECORD 3 ]-
first | 3
second | three
-[ RECORD 4 ]-

```

```
first | 4
second | four
```

此外，可以使用 `\g` 为一个查询设置这些输出格式选项：

```
peter@localhost testdb=> SELECT * FROM my_table
peter@localhost testdb-> \g (format=aligned tuples_only=off expanded=on)
-[ RECORD 1 ]-
first | 1
second | one
-[ RECORD 2 ]-
first | 2
second | two
-[ RECORD 3 ]-
first | 3
second | three
-[ RECORD 4 ]-
first | 4
second | four
```

这是一个示例，用 `\df` 命令以发现名称匹配 `int*pl` 并且第二参数是 `bigint` 类型的函数：

```
testdb=> \df int*pl * bigint
                        List of functions
 Schema | Name | Result data type | Argument data types | Type
-----+-----+-----+-----+-----
pg_catalog | int28pl | bigint | smallint, bigint | func
pg_catalog | int48pl | bigint | integer, bigint | func
pg_catalog | int8pl | bigint | bigint, bigint | func
(3 rows)
```

如果需要，可以用 `\crosstabview` 命令以交叉表的形式显示查询结果：

```
testdb=> SELECT first, second, first > 2 AS gt2 FROM my_table;
first | second | gt2
-----+-----+-----
1 | one | f
2 | two | f
3 | three | t
4 | four | t
(4 rows)

testdb=> \crosstabview first second
```

first	one	two	three	four
1	f			
2		f		
3			t	
4				t

(4 rows)

这第二个例子展示了表的“乘法”（连接），行按照序号降序排序且列按照独立的、升序的方式排序。

```
testdb=> SELECT t1.first as "A", t2.first+100 AS "B", t1.first*(t2.first+100) as
"AxB",
testdb(> row_number() over(order by t2.first) AS ord
testdb(> FROM my_table t1 CROSS JOIN my_table t2 ORDER BY 1 DESC
testdb(> \crosstabview "A" "B" "AxB" ord
A | 101 | 102 | 103 | 104
---+-----+-----+-----+-----
4 | 404 | 408 | 412 | 416
3 | 303 | 306 | 309 | 312
2 | 202 | 204 | 206 | 208
1 | 101 | 102 | 103 | 104
(4 rows)
```

reindexdb

reindexdb — 重索引一个IvorySQL数据库

大纲

reindexdb [connection-option...] [option...] [-S | --schema schema] ... [-t | --table table] ... [-i | --index index] ... [dbname]

```
reindexdb `[*`connection-option`*...] [*`option`*...] `-a` | `--all`
```

reindexdb [connection-option...] [option...] -s | --system [dbname]

选项

reindexdb接受下列命令行参数：

- **-a --all**

重索引所有数据库。

- **--concurrently**

使用 `CONCURRENTLY` 选项。请参阅 <http://www.postgresql.org/docs/17/sql-reindex.html>[REINDEX]，其中详细解释了此选项的所有注意事项。

- **[-d] dbname dbname**

当 **-a / --all** 未使用时，指定要重新索引的数据库的名称。如果未指定，则从环境变量 **PGDATABASE** 中读取数据库名称。如果未设置，则使用为连接指定的用户名。**dbname** 可以是 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。

- **-e --echo**

回显reindexdb生成并发送到服务器的命令。

- **-i index --index=index**

只是重建 `*`index`*`。可以通过写多个 `-i`` 开关来重建多个索引。

- **-j njobs --jobs=njobs**

通过同时运行 **njobs** 命令并行执行 reindex 命令。
此选项可能会减少处理时间，但也会增加数据库服务器上的负载。reindexdb将打开到数据库的 **njobs** 连接，因此请确保 [max_connections](#) 设置足够高，可以容纳所有连接。请注意，此选项与 **--index** 和 **--system** 选项不兼容。

- **-q --quiet**

不显示进度消息。

- **-s --system**

仅索引数据库的系统目录。

- **-S schema --schema=schema**

只对 **schema** 重建索引。通过写多个 **-S** 开关可以指定多个要重建索引的模式。

- **-t table --table=table**

只索引 **table**。可以通过写多个 **-t** 开关来重索引多个表。

- **--tablespace=tablespace**

指定要重建索引的表空间。（这个名字以双引号标识符来处理。）

- **-v --verbose**

在处理时打印详细信息。

- **-V --version**

打印reindexdb版本并退出。

- **-? --help**

显示有关reindexdb命令行参数的帮助并退出。

reindexdb也接受下列命令行参数用于连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制reindexdb在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，reindexdb将自动提示要求一个口令。但是，reindexdb将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

- **--maintenance-db=dbname**

当使用 **-a / --all** 时，指定要连接到的数据库名称以发现应重新索引哪些数据库。如果未指定，将使用 **postgres** 数据库，如果不存在，将使用 **template1**。这可以是 [连接字符串](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。此外，在连接到其他数据库时，将重新使用除数据库名称本身之外的连接字符串参数。

环境

- **PGDATABASE PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [REINDEX](#)和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

注解

reindexdb可能需要多次连接到IvorySQL服务器，每一次都会询问一个口令。在这种情况下使用一个 **~/.pgpass** 文件会更方便。

例子

要重索引数据库 **test**：

```
$ reindexdb test
```


要重索引名为 **abcd** 的数据库中的表 **foo** 和索引 **bar**：

```
$ reindexdb --table foo --index bar abcd
```

vacuumdb

vacuumdb — 对一个IvorySQL数据库进行垃圾收集和分析

大纲

vacuumdb [**connection-option**...] [**option**...] [**-t** | **--table table** [(**column** [,...])]] ... [**dbname**]

```
vacuumdb` [*`connection-option`*...] [*`option`*...] `-a` | `--all
```

选项

vacuumdb接受下列命令行参数：

- **-a --all**

清理所有数据库。

- **[-d] dbname dbname**

指定要清理或分析的数据库的名称，当不使用 **-a / --all** 时。如果未指定，则从环境变量 **PGDATABASE** 中读取数据库名称。如果环境变量也没有设置，指定给该连接的用户名将用作数据库名。**dbname** 可以是 **connection string**。如果是这样，连接时的字符串参数将覆盖任何冲突的命令行选项。

- **--disable-page-skipping**

根据可见性地图的内容禁用跳过页面。

- **-e --echo**

回显**vacuumdb**生成并发送给服务器的命令。

- **-f --full**

执行“完全”清理。

- **-F --freeze**

强有力地“冻结”元组。

- **--force-index-cleanup**

总是移除指向死元组的索引条目。

- **-j njobs --jobs=njobs**

通过同时运行 **njobs** 个命令来并行执行清理或者分析命令。这个选项可能会减少处理的时间，但是它也会增加数据库服务器的负载。**vacuumdb**将开启 **njobs** 个到数据库的连接，因此请确认你的 **max_connections** 设置足够高以容纳所有的连接。注意如果某些系统目录被并行处理，使用这种模式加上 **-f** (**FULL**) 选项可能会导致死锁失败。

- **--min-mxid-age mxid_age**

仅在multixact ID 年龄至少为 **mxid_age** 的表上执行清空或分析命令。
此设置对于确定要处理的表的优先级比较有用，以防止multixact ID 回绕。对于此选项的用途，关系的multixact ID年龄是主关系及其关联的TOAST表的年龄中最大的，如果存在的话。由于vacuumdb发出的命令在需要时还将处理TOAST表的关系，它无需单独考虑。

- **--min-xid-age xid_age**

仅在事务ID 年龄至少为 **xid_age** 的表上执行清空或分析命令。
此设置对于确定要处理的表的优先级比较有用，以防止事务ID 回绕。对于此选项的用途，关系的事务ID年龄是主关系及其关联的TOAST表的年龄中最大的，如果存在的话。由于vacuumdb发出的命令在需要时还将处理TOAST表的关系，它无需单独考虑。

- **--no-index-cleanup**

不移除指向死元组的索引条目。

- **--no-process-toast**

略过与要清理的表有关联的任何 TOAST 表。

- **--no-truncate**

不要在表的结尾截断空页面。

- **-P parallel_workers --parallel=parallel_workers**

指定 parallel vacuum 的并发数量。这允许清理利用多个 CPU 来处理索引。请参见 [VACUUM](#)。

- **-q --quiet**

不显示进度消息。

- **--skip-locked**

跳过无法立即锁定以进行处理的关系。

- **-t 'table [(column [...])]' --table='table [(column [...])]'**

只清理或分析 **table**。列名只能和 **--analyze** 或 **--analyze-only** 选项一起被指定。通过写多个 **-t** 开关可以清理多个表。提示如果你指定列，你可能必须转义来自 shell 的括号（见下面的例子）。

- **-v --verbose**

在处理期间打印详细信息。

- **-V --version**

打印vacuumdb版本并退出。

- **-z --analyze**

也计算优化器使用的统计信息。

- **-Z --analyze-only**

只计算优化器使用的统计信息（不清理）。

- **--analyze-in-stages**

与 **--analyze-only** 相似，只计算优化器使用的统计信息（不做清理）。
使用不同的配置设置运行分析的几个（目前是

3个) 阶段以更快地产生可用的统计信息。这个选项对分析一个刚从转储恢复或者通过 **pg_upgrade** 得到的数据库有用。这个选项将尝试尽可能快地创建一些统计信息来让该数据库可用，然后在后续的阶段中产生完整的统计信息。

- **-? --help**

显示有关vacuumdb命令行参数的帮助并退出。

vacuumdb也接受下列命令行参数用于连接参数：

- **-h host --host=host**

指定运行服务器的机器的主机名。如果该值以一个斜线开始，它被用作 Unix 域套接字的目录。

- **-p port --port=port**

指定服务器正在监听连接的 TCP 端口或本地 Unix 域套接字文件扩展。

- **-U username --username=username**

要作为哪个用户连接。

- **-w --no-password**

从不发出一个口令提示。如果服务器要求口令认证并且没有其他方式提供口令（例如一个 **.pgpass** 文件），那儿连接尝试将失败。这个选项对于批处理任务和脚本有用，因为在其中没有一个用户来输入口令。

- **-W --password**

强制vacuumdb在连接到一个数据库之前提示要求一个口令。这个选项不是必不可少的，因为如果服务器要求口令认证，vacuumdb将自动提示要求一个口令。但是，vacuumdb将浪费一次连接尝试来发现服务器想要一个口令。在某些情况下值得用 **-W** 来避免额外的连接尝试。

- **--maintenance-db=dbname**

当使用 **-a / --all** 时，指定要连接到的数据库名称以发现应该清理的数据库。如果未指定，将使用 **postgres** 数据库，如果不存在，将使用 **template1**。这可以是 [connection string](#)。如果是这样，连接字符串参数将覆盖任何冲突的命令行选项。此外，在连接到其他数据库时，将重新使用除数据库名称本身之外的连接字符串参数。

环境

- **PGDATABASE PGHOST PGPORT PGUSER**

默认连接参数

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto** 和 **never**。

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

诊断

在有困难时，可以在 [VACUUM](#) 和 [psql](#) 中找潜在问题和错误消息的讨论。数据库服务器必须运行在目标主机上。同样，任何libpq前端库使用的默认连接设置和环境变量都将适用于此。

注解

`vacuumdb`可能需要多次连接到IvorySQL服务器，每次都询问一个口令。在这种情况下有一个`~/.pgpass`文件会很方便。

例子

要清理数据库 `test`:

```
$ vacuumdb test
```

要清理和为优化器分析一个名为 `bigdb` 的数据库:

```
$ vacuumdb --analyze bigdb
```

要清理在名为 `xyzyz` 的数据库中的一个表 `foo`，并且为优化器分析该表的 `bar` 列:

```
$ vacuumdb --analyze --verbose --table='foo(bar)' xyzyz
```

服务器应用

initdb

initdb — 创建一个新的IvorySQL数据库集簇

大纲

```
initdb [option...][ --pgdata | -D ] `directory`
```

选项

- **-A authmethod** **--auth=authmethod**

这个选项为本地用户指定在 `pg_hba.conf` 中使用的默认认证方法（`host` 和 `local` 行）。`initdb` 将使用指定的认证方法为非复制连接以及复制连接填充 `pg_hba.conf` 项。除非你信任你系统上的所有本地用户，不要使用 `trust`。为了安装方便，`trust` 是默认值。

- **--auth-host=authmethod**

这个选项为通过 TCP/IP 连接的本地用户指定在 `pg_hba.conf` 中使用的认证方法（`host` 行）。

- **--auth-local=authmethod**

这个选项为通过 Unix 域套接字连接的本地用户指定在 `pg_hba.conf` 中使用的认证方法（`local` 行）。

- **-D directory** **--pgdata=directory**

这个选项指定数据库集簇应该存放的目录。这是 `initdb` 要求的唯一信息，但是你可以通过设定 `PGDATA` 环境变量来避免书写它，这很方便因为之后数据库服务器（`postgres`）可以使用同一个变量来找到数据库目录。

- **-E encoding** **--encoding=encoding**

选择模板数据库的编码。这也将是后来创建的任何数据库的默认编码，除非你覆盖它。默认值来自于区域，

或者如果该值不起作用则为 `SQL_ASCII`。

- `-g --allow-group-access`

允许与集簇拥有者同组的用户读取 `initdb` 创建的所有集簇文件。Windows会忽略此选项，因为它不支持POSIX样式的组权限。

- `-k --data-checksums`

在数据页面上使用校验码来帮助检测 I/O 系统造成的损坏。启用校验码将会引起显著的性能惩罚。如果设置，则为所有对象计算校验和，在整个数据库中。所有校验和失败都将报告在 `pg_stat_database` 视图。

- `--locale=locale`

为数据库集簇设置默认区域。如果这个选项没有被指定，该区域将从 `initdb` 所运行的环境中继承。

- `--lc-collate=locale --lc-ctype=locale --lc-messages=locale --lc-monetary=locale --lc-numeric=locale --lc-time=locale`

和 `--locale` 相似，但是只在指定的分类中设置区域。

- `--no-locale`

等效于 `--locale=C`。

- `-N --no-sync`

默认情况下，`initdb` 将等待所有文件被安全地写到磁盘。这个选项会导致 `initdb` 不等待就返回，这当然更快，但是也意味着一次后续的操作系统崩溃可能让数据目录损坏。通常，这个选项对测试有用，但是不应该在创建生产安装时使用。

- `--no-instructions`

默认情况下，`initdb` 将在输出的尾部写入如何启动集群的指令。这个选项会导致那些指令被抛弃。这主要由平台特定行为中封装 `initdb` 的工具来使用，那些指令有可能是错误的。

- `--pwfile=filename`

让 `initdb` 从一个文件读取数据库超级用户的口令。该文件的第一行被当作口令。

- `-S --sync-only`

安全地把所有数据库文件写入到磁盘并退出。这不会执行任何正常的`initdb`操作。

- `-T config --text-search-config=config`

设置默认的文本搜索配置。详见 [default_text_search_config](#)。

- `-U username --username=username`

选择数据库超级用户的用户名。这个的默认值是实际运行 `initdb` 的用户的名称。超级用户的名字是什么真的不重要，但是你可以选择保留常用的名字`postgres`，即使操作系统的用户名不同。

- `-W --pwprompt`

让 `initdb`

提示要求为数据库超级用户给予一个口令。如果你没有计划使用口令认证，这就不重要。否则在你设置一个口令之前你就无法使用口令认证。

- `-X directory --waldir=directory`

这个选项指定预写式日志会被存储在哪个目录中。

- **--wal-segsize='size'**

设置

WAL段尺寸，以兆字节为单位。这是WAL日志中每个文件的尺寸。默认的尺寸为16兆字节。该值必须位于2的1次幂和1024次幂（兆字节）之间。这个选项只能在初始化期间设置，并且之后不能更改。调整这个值来控制WAL日志传送或者归档可能会有用。此外，在有大量WAL的数据库中，每个目录中数量巨大的WAL文件可能会成为性能和管理问题。增加WAL文件尺寸将会降低WAL文件的数量。

其他较少使用的选项：

- **-d --debug**

打印来自引导后端的调试输出以及普通大众不那么感兴趣的一些消息。引导后端被程序 **initdb** 用来创建目录表。这个选项会生成大量极端无聊的输出。

- **--discard-caches**

使用 **debug_discard_caches=1** 选项运行引导程序后端。这需要很长时间并且只适用于深度调试。

- **-L directory**

指定 **initdb**

应从哪里寻找它的输入文件来初始化数据库集簇。这通常没有必要。如果你需要显式指定它们的位置，你应该被告知。

- **-n --no-clean**

默认情况下，当 **initdb**

确定有一个错误阻止它完整地创建数据库集簇，它会移除在它发现无法完成任务之前创建的任何文件。这个选项会抑制这种整理并且对调试有用。

其他选项：

- **-V --version**

打印initdb版本并退出。

- **-? --help**

显示有关initdb命令行参数的帮助并退出。

环境

- **PGDATA**

指定数据库集簇应该被存放的目录，可以使用 **-D** 选项覆盖。

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

- **TZ**

指定创建的数据集簇的默认时区。值应该是一个完整的时区名称

和大部分其他IvorySQL工具相似，这个工具也使用libpq支持的环境变量。

注解

`initdb` 可以通过 `pg_ctl initdb` 被调用。

pg_archivecleanup

pg_archivecleanup — 清理IvorySQL WAL 归档文件

大纲

`pg_archivecleanup [option...] archive location oldestkeptwal file`

选项

pg_archivecleanup接受下列命令行参数：

- **-d**

在 **stderr** 上打印很多调试日志输出。

- **-n**

在 **stdout** 上打印将被移除的文件的名字（执行一次演习）。

- **-V --version**

打印pg_archivecleanup版本并退出。

- **-x extension**

提供一个扩展名，在决定所有的文件是否应该被删除之前，将从文件名中剥离这个扩展名。这通常有助于清理已经存储期间被压缩过并且被压缩程序增加了一个扩展名的归档。例如：**-x .gz**。

- **-? --help**

显示pg_archivecleanup命令行参数的帮助并退出。

环境

环境变量 **PG_COLOR** 指定是否在诊断消息中使用颜色。可能的值是 **always**, **auto** 和 **never**。

示例

在 Linux 或者 Unix 系统上，你可能会用：

```
archive_cleanup_command = 'pg_archivecleanup -d /mnt/standby/archive %r
2>>cleanup.log'
```

其中归档目录位于后备服务器上，这样 **archive_command** 通过 NFS 来访问它，但是文件对于后备服务器来说是本地的。这将会

- 在 **cleanup.log** 中产生调试输出
- 从归档目录中移除不再需要的文件

pg_checksums

pg_checksums — 在IvorySQL数据库集簇中启用、禁用或检查数据校验和

大纲

pg_checksums [**option**...] [[**-D** | **--pgdata**] **datadir**]

选项

下列命令选项可用：

- **-D directory --pgdata=directory**

指定存储数据库集簇的目录。

- **-c --check**

检查校验和。如果未指定其它任何内容，这是缺省模式。

- **-d --disable**

禁用校验和。

- **-e --enable**

启用校验和。

- **-f filename --filename=filename**

仅验证文件节点为 **filename** 的关系中的校验和。

- **-N --no-sync**

缺省地，**pg_checksums** 会等待所有文件安全地写到磁盘上。该选项使得 **pg_checksums** 不等待就返回，这样更快，但意味着后续如果操作系统崩溃会让更新的数据目录损坏。一般地，该选项对测试有用，但不应用在生产安装上。当使用 **--check** 时，该选项无效。

- **-P --progress**

启用进度报告。在检查或启用校验和时，打开该选项，会提供进度报告。

- **-v --verbose**

启用详细输出。列出所有检查的文件。

- **-V --version**

打印pg_checksums版本并退出。

- **-? --help**

显示关于pg_checksums命令行参数的帮助并退出。

环境

- **PGDATA**

指定数据库集簇存储的目录；可以用 **-D** 选项覆盖。

- **PG_COLOR**

指定是否在诊断消息中使用颜色。可能的值为 **always**, **auto**, **never**。

注意

在大型集群中启用校验和的时间可能很长。在此操作期间，写到数据目录的集群或其它程序必须是未启动的，否则可能出现数据丢失。

当复制设置与执行关系文件块的直接拷贝的工具（例如 [pg_rewind](#)）一起使用时，启用和禁用校验和会导致以不正确校验和形式出现的页面损坏，如果未在所有节点上执行一致的操作的话。故在复制设置中启用或禁用校验和时，推荐一致地切换所有集群之前停止所有集群。此外销毁所有备用数据库，在主数据库上执行操作，最后从头开始重建备用服务器，也是安全的。

如果在启用或禁用校验和时异常终止或杀掉pg_checksums，那么集群的数据校验和配置保持不变，pg_checksums可以重新运行以执行相同操作。

pg_controldata

pg_controldata — 显示一个IvorySQL数据库集群的控制信息

大纲

pg_controldata [**option**] [**-D** | **--pgdata**] **datadir**

环境

- **PGDATA**

默认的数据目录位置。

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

pg_ctl

pg_ctl — 初始化、启动、停止或控制一个IvorySQL服务器

大纲

pg_ctl init [**db**] [**-D datadir**] [**-s**] [**-o initdb-options**]

pg_ctl start [**-D datadir**] [**-l filename**] [**-W**] [**-t seconds**] [**-s**] [**-o options**] [**-p path**] [**-c**]

pg_ctl stop [**-D datadir**] [**-m s**[**mart**] | **f**[**ast**] | **i**[**mmediate**]] [**-W**] [**-t seconds**] [**-s**]

pg_ctl restart [**-D datadir**] [**-m s**[**mart**] | **f**[**ast**] | **i**[**mmediate**]] [**-W**] [**-t seconds**] [**-s**] [**-o options**] [**-c**]

pg_ctl reload [**-D datadir**] [**-s**]

pg_ctl status [**-D datadir**]

pg_ctl promote [**-D datadir**] [**-W**] [**-t seconds**] [**-s**]

pg_ctl logrotate [**-D datadir**] [**-s**]

pg_ctl kill signal_name process_id

在Microsoft Windows上，还有：

pg_ctl register [**-D datadir**] [**-N servicename**] [**-U username**] [**-P password**] [**-S a**[**uto**] | **d**[**emand**]] [**-e source**]

`[-W] [-t seconds] [-s] [-o options]`

`pg_ctl unregister [-N servicename]`

选项

- `-c --core-files`

在可行的平台上尝试允许服务器崩溃产生核心文件，方法是提升在核心文件上的任何软性资源限制。这通过允许从一个失败的服务器进程中获得一个栈跟踪而有助于调试或诊断问题。

- `-D `datadir` --pgdata=`datadir``

指定数据库配置文件的文件系统位置。如果这个选项被忽略，将使用环境变量 `PGDATA`。

- `-l `filename` --log=`filename``

追加服务器日志输出到 `*`filename`*`。如果该文件不存在，它会被创建。`umask`被设置成 077，这样默认情况下不允许其他用户访问该日志文件。

- `-m mode --mode=mode`

指定关闭模式。`mode` 可以是 `smart`、`fast` 或

`immediate`，或者这三者之一的第一个字母。如果这个选项被忽略，则 `fast` 是默认值。

- `-o options --options=options`

指定被直接传递给 `postgres` 命令的选项。`-o`

可以被指定多次，所有给定的选项都会被传过去。这些选项应该通常被单引号或双引号包围来确保它们被作为一个组传递。

- `-o initdb-options --options=initdb-options`

指定要直接传递给 `initdb` 命令的选项。`-o`

可以被指定多次，所有给定的选项都会被传过去。这些选项应该通常被单引号或双引号包围来确保它们被作为一个组传递。

- `-p path`

指定 `postgres` 可执行程序的位置。默认情况下，`postgres` 可执行程序可以从 `pg_ctl`

相同的目录得到，或者如果没有在那里找到，则在硬写的安装目录中获得。除非你正在做一些不同寻常的事并且得到错误说没有找到 `postgres` 可执行程序，这个选项不是必需的。在 `init` 模式中，这个选项类似于指定了 `initdb` 可执行程序的位置。

- `-s --silent`

只打印错误，不打印信息性的消息。

- `-t seconds --timeout=seconds`

指定等待一个操作完成时要等待的最大秒数（见选项 `-w`）。默认为 `PGCTLTIMEOUT` 环境变量的值，如果该环境变量没有设置则默认为60秒。

- `-V --version`

打印 `pg_ctl` 版本并退出。

- **-w --wait**

等待操作完成。模式 **start**、**stop**、**restart**、**promote** 以及 **register** 支持这个选项，并且对那些模式是默认的。在等待时，**pg_ctl** 会一遍又一遍地检查服务器的PID文件，在两次检查之间会休眠一小段时间。当PID文件指示该服务器已经做好准备接受连接时，启动操作被认为完成。当服务器移除PID文件时，关闭操作被认为完成。**pg_ctl** 会基于启动或关闭的成功与否返回一个退出代码。如果操作在超时时间（见选项 **-t**）内未能完成，则 **pg_ctl** 会以一个非零退出状态退出。但是注意该操作可能会在后台继续进行并且最终取得成功。

- **-W --no-wait**

不等待操作完成。这是选项 **-w** 的对立面。如果禁用等待，所请求的动作会被触发，但是不会有关于其成功与否的反馈。在这种情况下，可能必须用服务器日志文件或外部监控系统来检查该操作的进度以及成功与否。在以前版本的IvorySQL中，这是除 **stop** 模式之外的模式的默认选项。

- **-? --help**

显示有关**pg_ctl**命令行参数的帮助并退出。

如果一个指定的选项有效，但与选中的操作模式无关，则**pg_ctl**会忽略它。

用于 Windows 的选项

- **-e source**

作为一个 Windows 服务运行时，**pg_ctl**用来 在事件日志中记录日志的事件源的名称。默认是 **IvorySQL**。注意这只控制由**pg_ctl**本身发送的消息，一旦开始，服务器将使用 **event_source** 参数中指定的事件源。如果服务器在启动时很早（在该参数被设置前）就失败，它可能也会使用默认的事件源名称 **IvorySQL** 来记录。

- **-N servicename**

要注册的系统服务的名称。这个名称将被用于服务名和显示名。默认是 **IvorySQL**。

- **-P password**

用于运行该服务的用户的口令。

- **-S start-type**

要注册的系统服务的启动类型。启动类型可以是 **auto**、**demand** 或者两者之一的第一个字母。如果这个选项被忽略，则 **auto** 是默认值。

- **-U username**

用于运行该服务的用户的用户名。对于域用户，使用格式 **DOMAIN\username**。

环境

- **PGCTLTIMEOUT**

等待启动或者关闭完成时要等待的默认秒数限制。如果没有设置，默认值是 60 秒。

- **PGDATA**

默认的数据目录位置。

大部分的 **pg_ctl** 模式都要求知道数据目录的位置，因此 **-D** 选项是必需的，除非 **PGDATA** 被设置。

和大部分其他IvorySQL工具相似，**pg_ctl** 也使用libpq支持的环境变量。

更多影响服务器的变量请见 [postgres](#)。

文件

- `postmaster.pid`

`pg_ctl`在数据目录中检查这个文件来判断服务器当前是否正在运行。

- `postmaster.opts`

如果这个文件存在于数据目录中，`pg_ctl`（处于 `restart` 模式中）将把该文件的内容作为选项传递给 `postgres`，除非通过 `-o` 选项进行了覆盖。这个文件的内容也会被显示在 `status` 模式中。

例子

启动服务器

要启动服务器并且等到服务器接受连接：

```
$ pg_ctl start
```

要使用端口 5433 启动服务器并且运行时不使用 `fsync`：

```
$ pg_ctl -o "-F -p 5433" start
```

停止服务器

要停止服务器，使用：

```
$ pg_ctl stop
```

`-m` 选项允许控制服务器如何关闭：

```
$ pg_ctl stop -m smart
```

重启服务器

重启服务器几乎等价于停止服务器并且再次启动它，不过 `pg_ctl` 默认会保存并重用被传递给之前的运行实例的命令行选项。要以和之前相同的选项重启服务器，使用：

```
$ pg_ctl restart
```

但是如果指定了 `-o`，则会替换任何之前的选项。要使用端口 5433 重启并在重启时禁用 `fsync`：

```
$ pg_ctl -o "-F -p 5433" restart
```

显示服务器状态

这里是pg_ctl状态输出的例子：

```
$ pg_ctl status
```

```
pg_ctl: server is running (PID: 13718)  
/usr/local/pgsql/bin/postgres "-D" "/usr/local/pgsql/data" "-p" "5433" "-B" "128"
```

第二行是在重启模式可能被调用的命令行。

pg_resetwal

pg_resetwal — 重置一个IvorySQL数据库集簇的预写式日志以及其他控制信息

大纲

```
pg_resetwal [-f|--force] [-n|--dry-run] [option... ] [-D|--pgdata] datadir
```

选项

- **-f --force**

即使 **pg_resetwal** 无法从 **pg_control** 中确定有效的数据（如前面所解释的），也强迫 **pg_resetwal** 继续运行。

- **-n --dry-run**

-n / --dry-run 选项指示 **pg_resetwal** 打印从 **pg_control** 重构出来的值以及要被改变的值，然后不修改任何东西退出。这主要是一个调试工具，但是可以用来在允许 **pg_resetwal** 真正执行下去之前进行完整性检查。

- **-V --version**

显示版本信息然后退出。

- **-? --help**

显示帮助然后退出。

只有当 **pg_resetwal** 无法通过读取 **pg_control** 确定合适的值时，才需要下列选项。安全值可以按下文所述来确定。对于接收数字参数的值，可以使用前缀 **0x** 指定 16 进制值。

- **-c xid, xid --commit-timestamp-ids=xid, xid**

手工设置提交时间可以检索到的最老的和最新的事务 ID。能检索到提交时间的最老事务 ID 的安全值（第一部分）可以通过在数据目录下 **pg_commit_ts** 目录中数字上最小的文件名来决定。反过来，能检索到提交时间的最新事务 ID 的安全值（第二部分）可以通过同一个目录中数字上最大的文件名来决定。文件名都是十六进制的。

- **-e xid_epoch --epoch=xid_epoch**

手工设置下一个事务 ID 的 epoch。事务 ID 的 epoch 实际上并没有存储在数据库中的任何地方，除了被 **pg_resetwal** 设置在这个域中，所以只要关心的是数据库本身，任何值都可以用。你可能需要调整这个值来确保诸如 Slony-I 和 Skytools 之类的复制系统正确地工作 — 如果确实需要调整，应该可以从下游的复制数据库的状态中获得一个合适的值。

- `-l 'walfile' --next-wal-file='walfile'`

通过指定下一个WAL段文件名称来手工设置WAL开始位置。下一个WAL段文件的名称应该比当前存在于数据目录下 `pg_wal` 目录中的任意 WAL 段文件名更大。这些名称也是十六进制的并且有三个部分。第一部分是“时间线 ID”并且通常应该被保持相同。例如，如果 `00000001000000320000004A` 是 `pg_wal` 中最大的项，则使用 `-l 00000001000000320000004B` 或更高的值。注意在使用非默认WAL段尺寸时，WAL文件名中的数字与系统函数和系统视图报告的LSN不同。这个选项要的是WAL文件名而不是LSN。注意 `pg_resetwal` 本身查看 `pg_wal` 中的文件并选择一个超出最新现存文件名的默认 `-l` 设置。因此，只有当你知道 WAL 段文件当前不在 `pg_wal` 中时，或者当 `pg_wal` 的内容完全丢失时，才需要对 `-l` 的手工调整，例如一个离线归档中的项。

- `-m mxid,mxid --multixact-ids=mxid,mxid`

手工设置下一个和最老的多事务 ID。确定下一个多事务 ID（第一部分）的安全值的方法：在数据目录下的 `pg_multixact/offsets` 目录中查找最大的数字文件名，然后在它的基础上加一并且乘以 65536 (0x10000)。反过来，确定最老的多事务 ID（`-m` 的第二部分）的方法：在同一个目录中查找最小的数字文件名并且乘以 65536。文件名是十六进制的数字，因此实现上述方法最简单的方式是以十六进制指定选项值并且追加四个零。

- `-o oid --next-oid=oid`

手工设置下一个 OID。没有相对容易的方法来决定超过数据库中最大 OID 的下一个 OID。但幸运的是正确地得到下一个 OID 设置并不是决定性的。

- `-O mxoff --multixact-offset=mxoff`

手工设置下一个多事务偏移量。确定安全值的方法：查找数据目录下 `pg_multixact/members` 目录中最大的数字文件名，然后在它的基础上加一并且乘以 52352 (0xCC80)。文件名是十六进制数字。没有像其他选项那样追加零的简单方法。

- `--wal-segsize=wal_segment_size`

设置新的WAL段尺寸，以兆字节为单位。这个值必须被设为2的1次幂和1024次幂（兆字节）之间。更多信息请参考 `initdb` 的相同选项。注意虽然 `pg_resetwal` 将把WAL起始地址设置成超过最新的现有WAL段文件，但一些段尺寸的改变可能导致之前的WAL文件名被重用。如果WAL文件名重叠会导致归档策略出现问题，推荐把 `-l` 和这个选项一起使用来手动设置WAL起始地址。

- `-u xid --oldest-transaction-id=xid`

手工设置最老的未冻结事务ID。一个安全值，可以通过在数据目录下的 `pg_xact` 目录中查找数字最小的文件名然后乘以1048576 (0x100000)的方式来确定。注意该文件名是十六进制的。以十六进制来指定选项值通常也是最简单的。例如，如果 `0007` 是 `pg_xact` 中最小的记录，`-u 0x700000` 将有效(五个后补零提供适当的乘数)。

- `-x xid --next-transaction-id=xid`

手工设置下一个事务 ID。确定安全值的方法：在数据目录下的 `pg_xact` 目录中查找最大的数字文件名，然后在它的基础上加一并且乘以 1048576 (0x100000)。注意文件名是十六进制的数字。通常以十六进制的形式指定该选项值也是最容易的。例如，如果 `0011` 是 `pg_xact` 中的最大项，`-x 0x1200000` 就可以（五个尾部的零就表示了前面说的乘数）。

环境

- `PG_COLOR`

规定在诊断消息中是否使用颜色。可能的值为 `always`、`auto`、`never`。

注解

这个命令不能在服务器正在运行时被使用。如果在数据目录中发现一个服务器锁文件，**pg_resetwal** 将拒绝启动。如果服务器崩溃那么一个锁文件可能会被留下，在那种情况下你能移除该锁文件来让 **pg_resetwal** 运行。但是在你那样做之前，再次确认没有服务器进程仍然存活。

pg_resetwal 仅能在具有相同主版本的服务器上使用。

pg_rewind

pg_rewind — 把一个IvorySQL数据目录与另一个从该目录中复制出来的数据目录同步

大纲

```
pg_rewind [option...]{ -D | --target-pgdata }*` directory`*{ --source-pgdata=`directory` | --source-server=`connstr` }
```

警告

如果在处理时**pg_rewind**失败，则目标的数据目录很可能不在可恢复的状态。在这种情况下，推荐创建一个新的备份。

由于 **pg_rewind**

完全从源复制配置文件，因此可能需要在重新启动目标服务器之前更正用于恢复的配置，特别是当目标服务器作为源的备用服务器重新引入时。

如果在倒带操作完成后重新启动服务器但未配置恢复，则目标可能会再次与主服务器分离。

如果**pg_rewind**发现它无法直接写入的文件，它将立刻失败。例如当源服务器和目标服务器为只读的SSL密钥及证书使用相同的文件映射，就会发生这种情况。如果在目标服务器上存在这样的文件，推荐在运行**pg_rewind**之前移除它们。在做了rewind之后，一些那样的文件可能已经被从源服务器拷贝，这样就有必要移除已经拷贝的数据并且恢复到rewind之前使用的链接集合。

选项

pg_rewind接受下列命令行参数：

- **-D directory --target-pgdata=directory**

这个选项指定要与源数据目录同步的目标数据目录。在运行**pg_rewind**之前目标服务器必须被干净地关闭。

- **--source-pgdata=directory**

指定要和目标服务器同步的源服务器的数据目录的文件系统路径。这个选项要求源服务器必须被干净地关闭。

- **--source-server=`connstr`**

指定一个 libpq 连接串用于连接要与目标服务器同步的源IvorySQL服务器。

连接必须是常规（非复制）连接，角色具有足够权限执行源服务器上**pg_rewind**使用的函数（请参阅备注部分）或超级用户角色。这个选项要求源服务器正在运行并且接受连接。

- **-R --write-recovery-conf**

创建 **standby.signal** 并将连接设置附加到输出目录中的 **postgresql.auto.conf** 中。 **--source-server** 对于此选项是必需的。

- **-n --dry-run**

做除了实际修改目标目录之外的其他所有事情。

- **-N --no-sync**

默认情况下，`pg_rewind` 将等待所有文件安全地写入磁盘。此选项会导致 `pg_rewind` 不等待即可返回，这更快，但意味着后续操作系统崩溃会使同步数据目录损坏。通常情况，此选项可用于测试，但不应使用于生产安装。

- `-P --progress`

启用进度报告。在从源集簇拷贝数据时，打开这个选项将会发送一个近似的进度报告。

- `-c --restore-target-wal`

如果在 `pg_wal` 目录中不再可用这些文件，请使用在目标群集配置中定义的 `restore_command` 从WAL存档中检索WAL文件。

- `--debug`

打印冗长的调试输出，这主要对于调试`pg_rewind`的开发者有用。

- `--no-ensure-shutdown`

`pg_rewind` 要求目标服务器在倒带之前彻底关闭。默认情况下，如果目标服务器没有完全关闭，`pg_rewind` 会以单用户模式启动目标服务器，先完成崩溃恢复，然后将其停止。

通过传递这个选项，如果服务器没有完全关闭，`pg_rewind` 会跳过这个并立即出错。在这种情况下，用户应该自己处理这种情况。

- `-V --version`

显示版本信息然后退出。

- `-? --help`

显示帮助然后退出。

环境

在使用 `--source-server` 选项时，`pg_rewind` 也使用 `libpq` 支持的环境变量。

环境变量 `PG_COLOR` 规定在诊断消息中是否使用颜色。可能的值为 `always`、`auto`、`never`。

注解

当使用在线群集作为源执行`pg_rewind`时，具有充足权限来执行`pg_rewind`在源群集上使用的函数的角色可以用来代替超级用户。这里介绍如何创建这样的角色，在这里命名 `rewind_user`：

```
CREATE USER rewind_user LOGIN;
GRANT EXECUTE ON function pg_catalog.pg_ls_dir(text, boolean, boolean) TO rewind_user;
GRANT EXECUTE ON function pg_catalog.pg_stat_file(text, boolean) TO rewind_user;
GRANT EXECUTE ON function pg_catalog.pg_read_binary_file(text) TO rewind_user;
GRANT EXECUTE ON function pg_catalog.pg_read_binary_file(text, bigint, bigint, boolean) TO rewind_user;
```

当使用近期升级的在线群集作为源执行`pg_rewind`时，必须在升级后执行 `CHECKPOINT` 以使其控制文件反映最新的时间线信息，`pg_rewind` 使用这些信息检查目标群集是否可以使用指定的源群集倒回。

如何工作

其基本思想是从源集簇拷贝所有文件系统级别的改变到目标集簇：

1.以源集簇的时间线历史从目标集簇分叉出来的点之前的最后一个检查点为起点，扫描目标集簇的 WAL 日志。对于每一个 WAL 记录，读取每一个被动过的数据块。这会得到在目标集簇中从源集簇被分支出去以后所有被更改过的数据块列表。如果某些 WAL 文件 不再可用，请尝试使用 **-c** 选项重新运行 `pg_rewind` 以在 WAL 存档中搜索丢失的文件。2.使用直接的文件系统访问 (**--source-pgdata**) 或者 SQL (**--source-server**)，把所有那些更改过的块从源集簇拷贝到目标集簇。关系文件现在的状态相当于源和目标的WAL时间线偏离点之前最后一个完成的检查点的时刻加上偏离点之后目标上更改的任何块的源上的当前状态。3.将所有其他文件，包括新的关系文件、WAL段、**pg_xact** 和配置文件，从源集群复制到目标集群。与基本备份类似，从源集群复制的数据中省略了目录 **pg_dynshmem/**、**pg_notify/**、**pg_replslot/**、**pg_serial/**、**pg_snapshots/**、**pg_stat_tmp/** 以及 **pg_subtrans/** 的内容。文件 **backup_label**、**tablespace_map**、**pg_internal.init**、**postmaster.opts** 以及 **postmaster.pid**，以及任何以 **pgsql_tmp** 开始的文件或目录都会被忽略。4.创建一个 **backup_label** 文件，在故障转移时创建的检查点处开始WAL重放，并将 **pg_control** 文件配置为最小一致性LSN，该LSN定义为从活动源回放时的 **pg_current_wal_insert_lsn()** 结果，或从停止的源回放时的最后一个检查点LSN。5.启动目标时，IvorySQL将重放所有必需的WAL，从而使数据目录处于一致状态。

pg_test_fsync

`pg_test_fsync` — 为IvorySQL判断最快的 **wal_sync_method**

大纲

pg_test_fsync [**option**...]

选项

`pg_test_fsync`接受下列命令行选项：

- **-f --file name**

指定要写入测试数据到其中的文件名。这个文件必须位于和 **pg_wal** 目录所在或者将被放置的同一个文件系统中（**pg_wal** 包含WAL文件）。默认是当前 目录中的 **pg_test_fsync.out**。

- **-s --secs-per-test**

指定每次测试的秒数。每个测试的时间越长，测试的精度就越高，但是 它需要更多时间来运行。默认是 5 秒，这允许程序在 2 分钟以内完成。

- **-V --version**

打印`pg_test_fsync`版本并且退出。

- **-? --help**

显示有关`pg_test_fsync`命令行参数的帮助并且退出。

环境

环境变量 **PG_COLOR** 指定是否在诊断消息中使用颜色。可能的值为 **always**、**auto** 和 **never**。

pg_test_timing

`pg_test_timing` — 度量计时开销

大纲

pg_test_timing [**option**...]

选项

pg_test_timing接受下列命令行选项：

- **-d duration --duration=duration**

指定测试的持续时间，以秒计。更长的持续时间会给出更好一些的精确度，并且更可能发现系统时钟回退的问题。默认的测试持续时间是 3 秒。

- **-V --version**

打印pg_test_timing版本并退出。

- **-? --help**

显示有关pg_test_timing的命令行参数，然后退出。

用法

结果解读

好的结果将显示大部分 (>90%) 的单个计时调用用时都小于 1 微秒。每次循环的平均开销将会更低，低于 100 纳秒。下面的例子来自于使用了一份 TSC 时钟源码的 Intel i7-860 系统，它展示了非常好的性能：

Testing timing overhead for 3 seconds.

Per loop time including overhead: 35.96 ns

Histogram of timing durations:

< us	% of total	count
1	96.40465	80435604
2	3.59518	2999652
4	0.00015	126
8	0.00002	13
16	0.00000	2

注意每次循环时间和柱状图用的单位是不同的。循环的解析度可以在几个纳秒（ns），而单个计时调用只能解析到一个微秒（us）。

度量执行器计时开销

当查询执行器使用 **EXPLAIN ANALYZE**

运行一个语句时，单个操作会被计时，总结也会被显示。你的系统的负荷可以通过使用psql程序计数行来检查：

```
CREATE TABLE t AS SELECT * FROM generate_series(1,100000);
\timing
SELECT COUNT(*) FROM t;
EXPLAIN ANALYZE SELECT COUNT(*) FROM t;
```

i7-860 系统测到运行该计数查询用了 9.8 ms 而 **EXPLAIN ANALYZE** 版本则需要 16.6 ms，每次处理都在 100,000 行上进行。6.8 ms 的差别意味着在每行上的计时负荷是 68 ns，大概是 pg_test_timing 估计的两倍。即使这样相对少量的负荷也造成了带有计时的计数语句耗时多出了 70%。在更大量的查询上，计时开销带来的问题不会有这么明显。

改变时间来源Changing time sources

在一些较新的 Linux

系统上，可以在任何时候更改用来收集计时数据的时钟来源。第二个例子显示了在上述快速结果的相同系统上切换到较慢的 acpi_pm 时间源可能带来的降速：

```
# cat /sys/devices/system/clocksource/clocksource0/available_clocksource
tsc hpet acpi_pm
# echo acpi_pm > /sys/devices/system/clocksource/clocksource0/current_clocksource
# pg_test_timing
Per loop time including overhead: 722.92 ns
Histogram of timing durations:
```

< us	% of total	count
1	27.84870	1155682
2	72.05956	2990371
4	0.07810	3241
8	0.01357	563
16	0.00007	3

在这种配置中，上面的例子 **EXPLAIN ANALYZE** 用了 115.9 ms。其中有 1061 ns 的计时开销，还是用这个工具直接度量结果的一个小倍数。这么多的计时开销意味着实际的查询本身只占了时间的一个很小的分数，大部分的时间都耗在了计时所需的管理开销上。在这种配置中，任何涉及到很多计时操作的 `EXPLAIN ANALYZE` 都会受到计时开销的显著影响。

FreeBSD 也允许即时更改时间源，并且它会记录在启动期间有关计时器选择的信息：

```
# dmesg | grep "Timecounter"
Timecounter "ACPI-fast" frequency 3579545 Hz quality 900
Timecounter "i8254" frequency 1193182 Hz quality 0
Timecounters tick every 10.000 msec
Timecounter "TSC" frequency 2531787134 Hz quality 800
# sysctl kern.timecounter.hardware=TSC
kern.timecounter.hardware: ACPI-fast -> TSC
```

其他系统可能只允许在启动时设定时间源。在旧的 Linux

系统上，“clock”内核设置是做这类更改的唯一方法。并且即使在一些更近的系统上，对于一个时钟源你将只能看到唯一的选项“jiffies”。Jiffies 是老的 Linux

软件时钟实现，当有足够快的计时硬件支持时，它能够具有很好的解析度，就像在这个例子中：

```
$ cat /sys/devices/system/clocksource/clocksource0/available_clocksource
jiffies
$ dmesg | grep time.c
time.c: Using 3.579545 MHz WALL PM GTOD PIT/TSC timer.
time.c: Detected 2400.153 MHz processor.
$ pg_test_timing
Testing timing overhead for 3 seconds.
```

Per timing duration including loop overhead: 97.75 ns

Histogram of timing durations:

< us	% of total	count
1	90.23734	27694571
2	9.75277	2993204
4	0.00981	3010
8	0.00007	22
16	0.00000	1
32	0.00000	1

时钟硬件和计时准确性

收集准确的计时信息在计算机上通常是使用具有不同精度的时钟硬件完成的。使用一些硬件，操作系统能几乎直接把系统时钟时间传递给程序。一个系统时钟也可以得自于一块简单地提供计时中断、在某个已知时间区间内的周期性滴答的芯片。在两种情况中，操作系统内核提供一个隐藏这些细节的时钟源。但是时钟源的精确度以及能多快返回结果会根据底层硬件而变化。

不精确的计时能够导致系统不稳定性。对任何时钟源的更改都要仔细地测试。操作系统默认是有时会更倾向于可靠性而不是最好的精确性。并且如果你在使用一个虚拟机器，应查看与之兼容的推荐时间源。在模拟计时器时虚拟硬件面临着额外的困难，并且提供商常常会建议每个操作系统的设置。

时间戳计数器（TSC）时钟源是当前一代 CPU 上最精确的一种。当操作系统支持 TSC 并且 TSC 可靠时，它是跟踪系统时间更好的方式。有多种方式会使 TSC 无法提供准确的计时源，这会让它不可靠。旧的系统能有一种基于 CPU 温度变化的 TSC 时钟，这让它不能用于计时。尝试在一些就的多核 CPU 上使用 TSC 可能在多个核心之间给出不一致的时间报告。这可能导致时间倒退，这个程序会检查这种问题。并且即使最新的系统，在非常激进的节能配置下也可能无法提供准确的 TSC 计时。

更新的操作系统可能检查已知的 TSC 问题并且当它们被发现时切换到一种更慢、更稳定的时钟源。如果你的系统支持 TSC 时间但是并不默认使用它，很可能是由于某种充分的理由才禁用它。某些操作系统可能无法正确地检测所有可能的问题，或者即便在知道 TSC 不精确的情况下也允许使用 TSC。

如果系统上有高精度事件计时器（HPET）并且 TSC 不准确，该系统将会更喜欢 HPET 计时器。计时器芯片本身是可编程的，最高允许 100 纳米的解析度，但是在你的系统时钟中可能见不到那么高的准确度。

高级配置和电源接口（ACPI）提供了一种电源管理（PM）计时器，Linux 把它称之为 `acpi_pm`。得自于 `acpi_pm` 的时钟最好时将能提供 300 纳秒的解析度。

在旧的 PC 硬件上使用的计时器包括 8254 可编程区间计时器（PIT）、实时时钟（RTC）、高级可编程中断控制器（APIC）计时器以及 Cyclone 计时器。这些计时器是以毫秒解析度为目标的。

pg_upgrade

pg_upgrade — 升级IvorySQL服务器实例

大纲

pg_upgrade -b oldbindir -B newbindir -d oldconfigdir -D newconfigdir [option...]

选项

pg_upgrade接受下列命令行参数：

- **-b bindir --old-bindir=bindir**

旧的 IvorySQL 可执行文件目录；环境变量 **PGBINOLD**

- **-B bindir --new-bindir=bindir**

新的 IvorySQL 可执行文件目录；默认为pg_upgrade所在的目录；环境变量 **PGBINNEW**

- **-c --check**

只检查集簇，不更改任何数据

- **-d configdir --old-datadir=configdir**

旧的集簇数据目录；环境变量 **PGDATAOLD**

- **-D configdir --new-datadir=configdir**

新的集簇数据目录；环境变量 **PGDATANEW**

- **-j --jobs=`njobs`**

要同时使用的进程或线程数

- **-k --link**

使用硬链接来代替将文件拷贝到新集簇

- **-o options --old-options options**

直接发送给旧 **postgres** 命令的选项，多个选项可以追加在后面

- **-O options --new-options options**

直接发送给新 **postgres** 命令的选项，多个选项可以追加在后面

- **-p port --old-port=port**

旧的集簇端口号；环境变量 **PGPORTOLD**

- **-P port --new-port=port**

新的集簇端口号；环境变量 **PGPORTNEW**

- **-r --retain**

即使在成功完成后也保留 SQL 和日志文件

- **-s dir --socketdir=dir**

用于升级期间postmaster套接字的目录;默认是当前目录; 环境变量 **PGSOCKETDIR**

- **-U username --username=username**

集簇的安装用户名；环境变量 **PGUSER**

- **-v --verbose**

启用详细的内部日志

- **-V --version**

显示版本信息，然后退出

- **--clone**

使用有效的文件克隆（在一些系统上也被称为“reflinks”），而不是将文件拷贝到新群集。

这可以导致数据文件接近瞬时的复制，从而获得 **-k/--link**

的速度优势，同时保留旧群集不受影响。文件克隆仅在某些操作系统和文件系统上得到支持。如果选中但不被支持，则 `pg_upgrade` 运行将会出错。

目前，它支持在Linux（内核4.5或更高版本）上的Btrfs和XFS（在文件系统创建reflink支持），以及macOS上的APFS。

- **-? --help**

显示帮助，然后退出

使用

下面是用 `pg_upgrade` 执行一次升级的步骤：

1. 移动旧集簇（可选）

如果你的安装目录不是版本相关的（例如 `/usr/local/pgsql`），就有必要移动当前的 IvorySQL 安装目录，以免它干扰新的 IvorySQL 安装。一旦当前的 IvorySQL 服务器被关闭，就可以安全地重命名 IvorySQL 安装目录。假设旧目录是 `/usr/local/pgsql`，你可以这样：

```
mv /usr/local/pgsql /usr/local/pgsql.old
```

来重命名该目录。

2. 对于源码安装，编译新版本

用兼容旧集簇的 **configure** 标记编译新的 IvorySQL 源码。在开始升级之前，`pg_upgrade` 将检查 **pg_controldata** 来确保所有设置都是兼容的。

3. 安装新的 IvorySQL 二进制文件

安装新服务器的二进制文件和支持文件。`pg_upgrade` 会被包含在默认的安装中。

对于源码安装，如果你希望把新服务器安装在一个自定义的位置，可以使用 **prefix** 变量：

```
make prefix=/usr/local/pgsql.new install
```

4. 初始化新的 IvorySQL 集簇

使用 **initdb** 初始化新集簇。这里也要使用与旧集簇相兼容的 **initdb** 标志。许多预编译的安装程序会自动做这个步骤。这里没有必要启动新集簇。

5. 安装扩展共享对象文件

许多扩展和自定义模块，无论是来自 **contrib** 或其他源，使用共享对象文件(或DLLs)，例如，**pgcrypto.so**。

如果旧集群使用过这些，匹配新服务器二进制的共享对象文件，必须安装在新集群中。通常是通过操作系统命令。不要加载模式定义，例如 **CREATE EXTENSION pgcrypto**，因为这些将从旧集群复制。如果扩展更新是可用的，pg_upgrade将报告这一点，并创建一个脚本，可以稍后运行来更新它们。

6.拷贝定制的全文本检索文件

从旧集群向新集群拷贝任何定制化全文本检索文件(词典、同义词、辞典、停用词)

7.调整认证

pg_upgrade 将会多次连接到旧服务器和新服务器，因此 你可能想要在 **pg_hba.conf** 中把认证设置成 **peer** 或者使用一个 **~/.pgpass** 文件。

8.停止两个服务器

确认两个数据库服务器都被停止使用，例如在 Unix 上可以：

```
pg_ctl -D /opt/IvorySQL/1.5 stop
pg_ctl -D /opt/IvorySQL/2.1 stop
```

或者在 Windows 上使用正确的服务名：

```
NET STOP postgresql-1.5
NET STOP postgresql-2.1
```

直到后面的步骤之前，流复制和日志传送后备服务器可以保持运行。

9.为后备服务器升级做准备

升级后备服务器时，请对旧的主集簇和后备集簇运行pg_controldata以验证旧的后备服务器已经完全追上。验证“Latest checkpoint location”值在所有集簇中都匹配（如果旧后备服务器在旧的主服务器之前被关闭或者如果旧的后备服务器仍在运行，则将会出现失配）。此外，请确保在新的主集簇上的 **postgresql.conf** 文件中，**wal_level** 未设置为 **minimal**。

10.运行 pg_upgrade

总是应该运行新服务器而不是旧服务器的pg_upgrade二进制文件。pg_upgrade要求制定新旧集簇的数据和可执行文件（**bin**）目录。你也可以指定用户和端口值，以及你是否想要用链接或克隆来取代默认的复制行为对数据文件进行处理。

如果你使用链接模式，升级将会快很多（不需要文件拷贝）并且将使用更少的磁盘空间，但是在升级后一旦启动新集簇，旧集簇就无法被访问。链接模式也要求新旧集簇数据目录位于同一个文件系统中（表空间和 **pg_wal** 可以在不同的文件系统中）。克隆模式提供了相同的速度以及磁盘空间优势，但不会导致新群集启动后旧群集不可用。

克隆模式还需要新旧数据目录位于同一文件系统中。此模式仅在某些操作系统和文件系统上可用。

--jobs 选项允许多个 CPU

核心被用来复制/链接文件以及并行地转储和重载数据库模式。这个选项一个比较好的值是 CPU 核心数 和表空间数的最大值。这个选项可以显著地减少升级运行在一台多处理器机器上的多数据库服务器的时间。

对于 Windows 用户，你必须以一个超级账号登录，并且以 `postgres` 用户启动一个 shell 并且设置正确的路径：

```
RUNAS /USER:postgres "CMD.EXE"  
SET PATH=%PATH%;C:\Program Files\IvorySQL\14\bin;
```

并且用带引号的目录运行 `pg_upgrade`，例如：

```
pg_upgrade.exe  
--old-datadir "C:/Program Files/IvorySQL/1.5/data"  
--new-datadir "C:/Program Files/IvorySQL/2.1/data"  
--old-bindir "C:/Program Files/IvorySQL/1.5/bin"  
--new-bindir "C:/Program Files/IvorySQL/2.1/bin"
```

一旦启动，**pg_upgrade** 将验证两个集簇是否兼容并且 执行升级。你可以使用 **pg_upgrade --check** 来只执行检查，这种模式即使在旧服务器还在运行时也能使用。**pg_upgrade --check** 也将列出任何在更新后需要做的手工调整。如果你将要使用链接或克隆模式，你应该使用 **--link** 或 **--clone** 选项和 **--check** 一起来启用链接模式相关的检查。**pg_upgrade** 要求在当前目录中的写权限。

显然，没有人可以在升级期间访问这些集簇。`pg_upgrade` 默认会在端口 50432 上运行服务器来避免意外的客户端连接。在做升级时，可以对两个集簇使用相同的端口号，因为新旧集簇不会在同时被运行。不过，在检查一个旧的运行中服务器时，新旧端口号必须不同。

如果在恢复数据库模式时发生错误，**pg_upgrade** 将会退出 并且你必须按照恢复旧集簇。要再次尝试 **pg_upgrade**，你将需要修改旧集簇，这样 `pg_upgrade` 模式会成功恢复。如果问题是一个 **contrib** 模块，你可能需要从旧集簇中卸载该模块并且在升级后重新把它安装在新集簇中，不过这样做的前提是该模块没有被用来存储用户数据。

11. 升级流复制和日志传送后备服务器

如果使用链接模式并且有流复制或者日志 传送后备服务器，你可以遵照下面的步骤对它们进行快速的升级。你将不用在这些后备服务器上运行 `pg_upgrade`，而是在主服务器上运行 `rsync`。到这里还不要启动任何服务器。

如果你没有使用链接模式、没有或不想使用 `rsync` 或者想用一种更容易的解决方案，请跳过这一节中的过程并且在 `pg_upgrade` 完成并且新的主集簇开始运行后重建后备服务器。

1. 在后备服务器上安装新的 IvorySQL 二进制文件

确保新的二进制和支持文件被安装在所有后备服务器上。

2. 确保不存在新的后备机数据目录

确保新的后备机数据目录不存在或者为空。如果

运行过initdb，请删除后备服务器的新数据目录。

3. 安装扩展共享对象文件

在新的后备机上安装和新的主集群中相同的扩展共享对象文件。

4. 停止后备服务器

如果后备服务器仍在运行，现在使用上述的指令停止它们。

5. 保存配置文件

从旧后备机的配置目录保存任何需要保留的配置文件，例如`postgresql.conf`（以及它包含的任何文件）、`postgresql.auto.conf`、`pg\_hba.conf`，因为这些文件在下一步中会被重写或者移除。

6. 运行rsync

在使用链接模式时，后备服务器可以使用rsync快速升级。为了实现这一点，在主服务器上一个高于新旧数据库集群目录的目录中为每个后备服务器运行这个命令：

```
...  
rsync --archive --delete --hard-links --size-only --no-inc-recursive old_cluster  
new_cluster remote_dir  
...
```

其中`old\_cluster`和`new\_cluster`是相对于主服务器上的当前目录的，而`remote\_dir`是后备服务器上高于新旧集群目录的一个目录。在主服务器和后备服务器上指定目录之下的目录结构必须匹配。指定远程目录的详细情况请参考rsync的手册，例如：

```
...  
rsync --archive --delete --hard-links --size-only --no-inc-recursive  
/opt/IvorySQL/1.5 \  
    /opt/IvorySQL/2.1 standby.example.com:/opt/IvorySQL  
...
```

可以使用rsync的`--dry-run`选项验证该命令将做的事情。虽然在主服务器上必须为至少一台后备运行rsync，可以在一台已经升级过的后备服务器上运行rsync来升级其他的后备服务器，只要已升级的后备服务器还没有被启动。

这个命令所做的事情是记录由pg_upgrade的链接模式创建的链接，它们连接主服务器上新旧集簇中的文件。该命令接下来在后备服务器的旧集簇中寻找匹配的文件并且为它们在该后备的新集簇中创建链接。主服务器上没有被链接的文件会被从主服务器拷贝到后备服务器（通常都很小）。这提供了快速的后备服务器升级。不幸地是，rsync会不必要地拷贝与临时表和不做日志表相关的文件，因为通常在后备服务器上不存在这些文件。

如果你已经把`pg\_wal`放在数据目录外面，也必须在那些目录上运行rsync。

7. 配置流复制和日志传送后备服务器

为日志传送配置服务器（不需要运行`pg\_start\_backup()`以及`pg\_stop\_backup()`或者做文件系统备份，因为从属机 仍在与主机同步）。

12. 恢复 pg_hba.conf

如果你修改了 **pg_hba.conf**，则要将其恢复到原始的设置。也可能需要调整新集簇中的其他配置文件来匹配旧集簇，例如 **postgresql.conf**（以及它包含的任何文件）和 **postgresql.auto.conf**。

13. 启动新服务器

现在可以安全地启动新的服务器，并且可以接着启动任何rsync过的后备服务器。

14. 升级后处理

如果需要做任何升级后处理，pg_upgrade 将在完成后发出警告。它也将生成必须由管理员运行的脚本文件。这些脚本文件将连接到每一个需要做升级后处理的数据库。每一个脚本应该这样运行：

```
...  
psql --username=postgres --file=script.sql postgres  
...
```

这些脚本可以以任何顺序运行并且在运行之后立即删除。

小心

通常在重建脚本运行完成之前访问重建脚本中引用的表是不安全的，这样做可能会得到不正确的结果或者很差的性能。没有在重建脚本中引用的表可以随时被访问。

15. 统计信息

由于 **pg_upgrade** 并未传输优化器统计信息，在升级的尾声你将被指示运行一个命令来生成这些信息。你可能需要设置连接参数来匹配你的新集簇。

16. 删除旧集簇

一旦你对升级表示满意，你就可以通过运行 **pg_upgrade** 完成时提到的脚本来删除旧集群的数据目录（如果在旧数据目录中有用户定义的表空间就不可能实现自动删除）。你也可以删除旧安装目录（例如 **bin**、**share**）。

17. 恢复到旧集群

在运行 **pg_upgrade** 之后，如果你希望恢复到旧集群，有几个选项：

- 如果使用了 **--check** 选项, 则旧集群没有被修改；它可以被重新启动。
- 如果 **--link** 选项没有被使用, 旧集群没有被修改；它可以被重新启动。
- 如果使用了 **--link** 选项, 数据文件可能在新旧群集之间共享:
 - 如果 **pg_upgrade** 在链接启动之前中止, 旧集群没有被修改, 它可以重新启动。
 - 如果你没有启动新集群, 旧集群没有被修改, 当链接启动时, 一个 **.old** 后缀会附加到 **\$PGDATA/global/pg_control**。如果要重用旧集群, 从 **\$PGDATA/global/pg_control** 移除 **.old** 后缀；你就可以重启旧集群。
 - 如果你已经启动新群集, 它已经写入了共享文件, 并且使用旧群集会不安全。这种情况下, 需要从备份中还原旧群集。

注解

pg_upgrade 创建不同的工作文件, 如模式转储, 在当前工作目录中。为了安全, 请确保该目录不可被任何其他用户读取或者写入。

pg_upgrade 在新旧数据目录中启动短期的 **postmasters**。临时 Unix 套接字文件用于与这些 **postmasters** 通信, 默认情况下, 在当前工作目录中进行。在某些情况下, 当前目录的路径名称可能太长, 无法成为有效的套接字名称。这种情况下你可以使用 **-s** 选项将套接字文件放在某些具有较短路径名称的目录中。为了安全原因, 请确保该目录不可被任何其他用户读取或者写入。（这在 Windows 上不受支持。）

如果失败、重建和重索引会影响你的安装, **pg_upgrade** 将会报告这些情况。用来重建表和索引的升级后脚本将会自动被建立。如果你正在尝试自动升级很多集群, 你应该发现具有相同数据库模式的集群对所有集群升级都要求同样的升级后步骤, 这是因为升级后步骤是基于数据库模式而不是用户数据。

对于部署测试, 创建一个只有模式的旧集群副本, 在其中插入假数据并且升级。

pg_upgrade 不支持包含使用这些 **reg*** OID-引用系统数据类型的表的数据库的升级：

regcollation
regconfig
regdictionary
regnamespace
regoper
regoperator
regproc
regprocedure

(**regclass**, **regrole**, and **regtype** can be upgraded.)

如果你想要使用链接模式并且你不想让你的旧集群在新集群启动时被修改, 考虑使用克隆模式。如果(克隆模式)不可用, 可以复制一份旧集群并且在副本上以链接模式进行升级。要创建旧集群的一份合法拷贝, 可以在服务器运行时使用 **rsync** 创建旧集群的一份脏拷贝, 然后关闭旧服务器并且再次运行 **rsync --checksum** 把更改更新到该拷贝以让其一致（**--checksum** 是必要的, 因为 **rsync** 在判断文件修改时间的更改时的精度只能到秒级）。你可能想要排除一些文件, 例如 **postmaster.pid**。如果你的文件系统支持文件系统快照或者 copy-on-write 文件副本, 你可以使用它们来创建旧集群和

表空间的一个备份，不过快照和副本必须被同时创建或者在数据库服务器关闭 期间被创建。

pg_waldump

pg_waldump — 以人类可读的形式显示一个IvorySQL 数据库集簇的预写式日志

大纲

pg_waldump [**option**...] [**startseg** [**endseg**]]

选项

下列命令行选项控制输出的位置和格式：

- **startseg**

从指定的日志段文件开始读取。这也隐含地决定了要搜索文件的路径以及 要使用的时间线。

- **endseg**

在读取指定的日志段文件后停止。

- **-b --bkp-details**

输出有关备份块的细节。

- **-e end --end=end**

在指定的WAL位置停止读取，而不是一直读取到日志流的末尾。

- **-f --follow**

在到达可用 WAL 的末尾之后，保持每秒轮询一次是否有新的 WAL 出现。

- **-n limit --limit=limit**

显示指定数量的记录，然后停止。

- **-p path --path=path**

指定搜索日志段文件的目录或包含这些文件的包含 **pg_wal** 子目录的目录。

缺省值是在当前目录中搜索，当前目录的 **pg_wal** 子目录和 **PGDATA** 的 **pg_wal** 子目录。

- **-q --quiet**

除错误外，不要打印任何输出。当您想知道一系列WAL记录是否可以成功解析但不关心记录内容时，此选项非常有用。

- **-r rmgr --rmgr=rmgr**

只显示由指定资源管理器生成的记录。如果把 **list** 作为资源管理器名称传递给这个选项，则打印出可用资源管理器名称的列表然后退出。

- **-s start --start=start**

要从哪个WAL位置开始读取。默认是从找到的最早的文件的第一个可用日志记录开始。

- **-t `timeline` --timeline=`timeline`**

要从哪个时间线读取日志记录。默认是使用 **startseg**（如果指定） 中的值，否则默认为 1。

- **-V --version**

打印pg_waldump版本并且退出。

- **-x xid --xid=xid**

只显示用给定事务 ID 标记的记录。

- **-z --stats[=record]**

显示概括统计信息（记录的数量和尺寸以及全页镜像）而不是显示每个记录。可以选择针对每个记录生成统计信息，而不是针对每个资源管理器生成。

- **-? --help**

显示有关pg_waldump命令行参数的帮助并且退出。

环境

- **PGDATA**

数据目录；另请参阅 **-p** 选项。

- **PG_COLOR**

规定在诊断消息中是否使用颜色。可能的值为 **always**、**auto**、**never**。

注解

当服务器正在运行时可能会给出错误的结果。

只有指定的时间线 会被显示（如果没有指定，则显示默认时间线）。其他时间线上的记录会被忽略。

pg_waldump不能读取具有后缀 **.partial** 的 WAL 文件。如果需要读取那些文件，需要从文件名中移除 **.partial** 后缀。

postgres

postgres — IvorySQL数据库服务器

大纲

postgres [选项...]

选项

postgres

接受下列命令行参数。你也可以通过设置一个配置文件来减少键入大部分这些选项。有些（安全）选项还可以从连接的客户端以一种与应用相关只应用于会话的方法设置。例如，如果设置了 **PGOPTIONS** 环境变量，那么基于libpq的客户端将都把那个字符串传递给服务器，它将被服务器解释成 **postgres** 命令行选项。

通用选项

- **-B nbuffers**

设置被服务器进程使用的共享内存缓冲区数量。这个参数的默认值是initdb自动选择的。指定这个选项等效于设置 **shared_buffers** 配置参数。

- **-c name=value**

设置一个命名的运行时参数。大多数其它命令行选项实际上都是这种参数赋值的短形式。 **-c** 可以出现多次用于设置多个参数。

- **-C name**

打印命名运行时参数的值，并且退出（详见上面的 **-c** 选项）。这可以被用在一个运行服务器上，并且从 **postgresql.conf** 中返回值，这些值可能被在这次调用中的任何参数修改过。它并不反映集群启动时提供的参数。这个选项用于与一个服务器实例交互的其他程序来查询配置参数值，例如 **pg_ctl**。面向用户的应用应该使用 **SHOW** or the **pg_settings** 视图。

- **-d debug-level**

设置调试级别。数值设置得越高，写到服务器日志的调试输出就越多。取值范围是从 1 到 5。还可以针对某个特定会话使用 **-d 0** 来阻止父 **postgres** 进程的服务器日志级别被传播到这个会话。

- **-D datadir**

指定数据库配置文件的文件系统位置。

- **-e**

把默认日期风格设置为“European”，也就是输入日期域的顺序是 **DMY**。这也导致在一些日期输出格式中把日打印在月之前。

- **-F**

禁用 **fsync** 调用以提高性能，但是要冒系统崩溃时数据损坏的风险。指定这个选项等效于禁用 **fsync** 配置参数。

- **-h hostname**

指定 **postgres** 监听来自客户端应用 TCP/IP 连接的 IP 主机名或地址。该值也可以是一个用逗号分隔的地址列表，或者 ***** 表示监听所有可用的地址。一个空值表示不监听任何 IP 地址，在这种情况下可以使用 Unix 域套接字连接到服务器。缺省只监听localhost。声明这个选项等效于设置listen_addresses配置参数。默认只监听localhost。指定这个选项等效于设置 **listen_addresses** 配置参数。

- **-i**

允许远程客户端使用 TCP/IP（互联网域）连接。没有这个选项，将只接受本地连接。这个选项等效于在 **postgresql.conf** 中或者通过 **-h** 选项将 **listen_addresses** 设为 *****。这个选项已经被废弃，因为它不允许访问 **listen_addresses** 的完整功能。所以最好直接设置 **listen_addresses**。

- **-k `directory`**

指定 **postgres** 用来监听来自客户端应用连接的 Unix 域套接字的目录。这个值也可以是一个逗号分隔的目录列表。一个空值指定不监听任何 Unix 域套接字，在这种情况下只能用 TCP/IP 套接字来连接到服务器。默认值通常是 **/tmp**，但是可以在编译的时候修改。指定这个选项等效于设置 **unix_socket_directories** 配置参数。

- **-l**

启用使用SSL的安全连接。要使这个选项可用，编译IvorySQL时必须打开SSL支持。有关使用SSL的更多信息。

- **-N max-connections**

设置该服务器将接受的最大客户端连接数。该参数的默认值由initdb自动选择。指定这个选项等效于设置 **max_connections** 配置参数。

- **-p `port`**

指定 **postgres** 用于监听客户端应用连接的 TCP/IP 端口或本地 Unix 域套接字文件扩展。默认为 **PGPORT** 环境变量的值。如果 **PGPORT** 没有设置，那么默认值是编译期间设立的值（通常是 5432）。如果你指定了一个非默认端口，那么所有客户端应用都必须用命令行选项或者 **PGPORT** 指定同一个端口。

- **-s**

在每条命令结束时打印时间信息和其它统计信息。这个选项对测试基准和调节缓冲区数量有用处。

- **-S work-mem**

指定在使用临时磁盘文件之前排序和散列表使用的基本内存量。

- **-V --version**

打印postgres版本并退出。

- **--name=value**

设置一个命名的运行时参数；其缩写形式是 **-c**。

- **--describe-config**

这个选项会用制表符分隔的 **COPY** 格式导出服务器的内部配置变量、描述以及默认值。设计它的目的是用于管理工具。

- **-? --help**

显示有关postgres的命令行参数，并且退出。

半内部选项

这里描述的选项主要被用于调试目的，并且在某些情况下可以协助恢复严重受损的数据库。在生产数据库环境中应该不会去使用它们。在这里列举它们只是为了让IvorySQL系统开发者使用。此外，这些选项可能在将来的版本中更改或删除而不另行通知。

- **-f{s | i | o | b | t | n | m | h }**

禁止某种扫描和连接方法的使用：**s** 和 **i** 分别禁用顺序和索引扫描，**o**、**b** 和 **t** 分别禁用只用索引扫描、位图索引扫描以及 TID 扫描，而 **n**、**m** 和 **h** 则分别禁用嵌套循环、归并和哈希连接。顺序扫描和嵌套循环连接都不可能完全被禁用。**-fs** 和 **-fn** 选项仅仅是在有其他选择时不鼓励优化器使用这些计划类型。

- **-n**

该选项主要用于调试导致服务器进程异常崩溃的问题。对付这种情况的一般策略是通知所有其它服务器进程，让它们终止并且接着重新初始化共享内存和信号量。这是因为一个错误的服务器进程可能在终止之前就已经对共享状态造成了破坏。该选项指定 **postgres** 将不会重新初始化共享数据结构。一个有经验的系统程序员这时就可以使用调试器检查共享内存和信号量状态。

- **-O**

允许修改系统表的结构。这个选项用于 **initdb**。

- **-P**

读取系统表时忽略系统索引（但在更改系统表时仍然更新索引）。这在从损坏的系统索引中恢复时有用。

- **-tpa[rser] | pl[anner] | e[xecutor]**

打印与每个主要系统模块相关的查询的时间统计。这个选项不能和 **-s** 选项一起使用。

- **-T**

该选项主要用于调试导致服务器进程异常崩溃的问题。对付这种情况的一般策略是通知所有其它服务器进程，让它们终止并且接着重新初始化共享内存和信号量。这是因为一个错误的服务器进程可能在终止之前就已经对共享状态造成了破坏。该选项指定 **postgres** 将通过发送 **SIGSTOP** 信号停止其他所有服务器进程，但是并不让它们终止。这样就允许系统程序员手动从所有服务器进程收集内核转储。

- **-v protocol**

声明这次会话使用的前/后服务器协议的版本数。该选项仅在内部使用。

- **-W seconds**

在一个新服务器进程被启动时，它实施认证过程之后会延迟这个选项所设置的秒数。这就留出了机会来用一个调试器附着在服务器进程上。

用于单用户模式的选项

下面的选项仅适用于单用户模式（见 [Single-User Mode](#)）。

- **--single**

选择单用户模式。这必须是命令行中的第一个选项。

- **database**

指定要访问的数据库的名称。这必须是命令行中的最后一个参数。如果省略它，则默认为用户名。

- **-E**

在执行命令之前回显所有命令到标准输出。

- **-j**

使用跟着两个新行的分号而不是仅用新行作为命令终止符。

- **-r filename**

将所有服务器日志输出发送到 **filename** 中。只有在作为一个命令行选项提供时，这个选项才会兑现。

环境

- **PGCLIENTENCODING**

客户端使用的默认字符编码（客户端可以独立地覆盖它）。这个值也可以在配置文件中设置。

- **PGDATA**

默认的数据目录位置。

- **PGDATESTYLE**

[DateStyle](#) 运行时参数的默认值（这个环境变量的使用已被废弃）。

- **PGPORT**

默认端口号（在配置文件中设置更好）

诊断

一个提到了 **semget** 或 **shmget**

的错误消息可能意味着你需要配置内核来提供足够的共享内存和信号量。你也可以通过降低 **shared_buffers** 值减少IvorySQL的共享内存消耗，或者降低 **max_connections** 值减少信号量消耗，这样可以推迟对内核的重新配置。

如果一个消息说另外一个服务器已经在运行，应该仔细地检查，例如根据你的系统可以用命令

```
$ ps ax | grep postgres
```

或

```
$ ps -ef | grep postgres
```

如果你确信没有冲突的服务器正在运行，那么你可以删除消息中提到的锁文件然后再次尝试。

如果一个失败消息指示它无法绑定到一个端口，可能意味着该端口已经被某些非IvorySQL进程所使用。如果你终止 **postgres**

并且立即使用相同的端口重启它，你也可能会得到这种错误。在这种情况下，你必须等待几秒直到操作系统关闭该端口，然后再重试。最后，如果你指定了一个操作系统认为需要保留的端口号，你可能也会得到这个错误。例如，很多版本的 Unix 认为低于 1024 的端口号是“可信的”并且只允许 Unix 超级用户访问它们。

注解

实用命令 **pg_ctl** 可以用来安全方便地启动和关闭 **postgres** 服务器。

只要有可能，就不要使用 **SIGKILL** 杀死主 **postgres** 服务器。这样会阻止 **postgres** 在终止前释放它持有的系统资源（例如共享内存和信号量）。这样可能会导致启动新的 **postgres** 进程时出现问题。

要正常地终止 **postgres** 服务器，可以使用 **SIGTERM**、**SIGINT** 或者 **SIGQUIT** 信号。第一个在退出前将等待所有客户端终止，第二个将强行断开所有客户端的连接，第三个会不做正确的关闭立即退出并且会导致重启时的恢复。

SIGHUP 信号会重新加载服务器配置文件。也可以向一个单独的服务器进程发送 **SIGHUP** 信号，但是这样做通常没什么意义。

要取消一个正在运行的查询，可以向运行该查询的进程发送 **SIGINT** 信号。要干净地终止一个后端进程，可向它发送 `SIGTERM`。在 SQL 中可调用的与这两种动作等效的命令可参考 **pg_cancel_backend** 和 **pg_terminate_backend**。

postgres 服务器使用 **SIGQUIT**

来告诉子服务器进程终止但不做正常的清理。该信号不应该被用户使用。向一个服务器进程发送 **SIGKILL** 也是不明智的 — 主 **postgres** 进程将把这解释为一次崩溃，并且作为其标准崩溃恢复过程的一部分，它将强制所有的后代进程退出。

缺陷

-- 选项在FreeBSD或OpenBSD上无法运行，应该使用 **-c**。这在受影响的系统里是个缺陷；如果这个缺陷没有被修复，将来的IvorySQL版本将提供一种解决方案。

单用户模式

要启动一个单用户模式的服务器，使用这样的命令

```
postgres --single -D /usr/local/pgsql/data other-options my_database
```

用 **-D** 给服务器提供正确的数据库目录的路径，或者确保环境变量 **PGDATA** 被设置。同时还要指定你想在其中工作的特定数据库的名字。

通常，单用户模式的服务器会把换行符当做命令输入的终止符。它不明白分号的作用，因为那属于psql。要想把一个命令分成多行，必须在最后一个换行符以外的每个换行符前面敲一个反斜线。这个反斜线和旁边的新行都会被从输入命令中去掉。注意即使在字符串或者注释中也会这样做。

但是如果使用了 **-j** 命令行选项，那么单个新行将不会终止命令输入。相反，分号-新行-新行的序列才会终止命令输入。也就是说，输入一个紧跟着空行的分号。在这种模式下，反斜线-新行不会被特殊对待。此外，在字符串或者注释内的这类序列也不会被特殊对待。

不管在哪一种输入模式中，如果输入的一个分号不是正好在命令终止符之前或者不是命令终止符的一部分，它会被认为是一个命令分隔符。当真正输入一个命令终止符时，已经输入的多个语句将被作为一个单个事务执行。

要退出会话，输入EOF（通常是 Control + D）。如果从上一个命令终止符以来已经输入了任何文本，那么EOF将被当作命令终止符，并且如果要退出则需要另一个EOF。

请注意单用户模式的服务器不会提供复杂的行编辑特性（例如没有命令历史）。但用户模式也不会做任何后台处理，例如自动检查点或者复制。

例子

要用默认值在后台启动 **postgres**：

```
$ nohup postgres >logfile 2>&1 </dev/null &
```

要用指定端口启动 **postgres**，例如 1234：

```
$ postgres -p 1234
```

要使用psql连接到这个服务器，用 **-p** 选项指定这个端口：

```
$ psql -p 1234
```

或者设置环境变量 **PGPORT**：

```
$ export PGPORT=1234
$ psql
```

命名运行时参数可以用这些形式之一设置：

```
$ postgres -c work_mem=1234
$ postgres --work-mem=1234
```

两种形式都覆盖 **postgresql.conf** 中可能存在的 **work_mem** 设置。请注意在参数名中的下划线在命令行可以写成下划线或连字符。除了用于短期的实验外，更好的习惯

是编辑 `postgresql.conf` 中的设置，而不是倚赖命令行开关来设置参数。

FAQ

IvorySQL贡献的许可

如果您提交的贡献是原创作品，那么您可以假设IvorySQL将作为整个IvorySQL版本的一部分发布给下游用户，该版本将遵循Apache许可证2.0版本。

如果您提交的内容不是原创作品，同样鼓励代码共享和尊重原作者的著作权，同样允许代码修改，再发布。请注意需要满足如下条件：

- 1、需要给代码的用户一份Apache许可证。
- 2、如果您修改了代码，需要在被修改的文件中说明。
- 3、在延伸的代码中（修改和有源代码衍生的代码中）需要带有原来代码中的协议、商标、专利声明和其他原来作者规定需要包含的说明。
- 4、如果再发布的产品中包含一个Notice文件，则在Notice文件中需要带有Apache许可证。您可以在Notice中增加自己的许可，但不可以表现为对Apache许可证构成更改。

最后，请记住，从非原始的工作中删除许可标头从来都不是一个好主意。即使您使用的文件部分最初在顶部有许可标头，您也应该保留它。与往常一样，如果您不太确定您的贡献所涉及的许可问题，请随时在开发人员邮件列表中联系我们。

编码指南

您获得反馈和看到代码合并到项目中的机会在很大程度上取决于更改的粒度。如果您的想法发生了更大的变化，我们强烈建议您在花大量时间编写代码之前，先加入开发人员的邮件列表，并与我们分享您的建议。即使您的建议得到社区的验证，我们仍然建议您将实际工作作为一系列小型的、独立的提交来完成。这使得评审员的工作更加容易，并提高了反馈的及时性。

当谈到IvorySQL的C和C++部分时，我们尝试遵循PostgreSQL编码约定。除此之外：

对于C和Perl代码，如果需要，请运行pgindent。我们建议在查看更改时使用git diff—color，这样您提交的代码中就不会出现任何虚假的空白问题。

所有贡献给IvorySQL的新功能都应该由与其一起贡献的回归测试覆盖。如果您不确定如何测试或记录您的工作，请在ivorysql-hackers邮件列表中提出问题，社区的开发人员将尽力帮助您。

至少，您应该始终运行make installcheck world，以确保您没有破坏任何东西。

适用于上游PostgreSQL的更改

如果您正在进行的更改涉及PostgreSQL和IvorySQL之间的通用功能，则可能会要求您将其转发到PostgreSQL。这不仅是为了我们不断减少两个项目之间的差异，而且是为了让与PostgreSQL相关的任何变化都能从对上游PostgreSQL社区更广泛的审查中受益。一般来说，将这两个代码库都放在手边是个好主意，这样您就可以确定您的更改是否需要前移。

补丁提交

一旦您准备好与IvorySQL核心团队和IvorySQL社区的其他成员共享您的工作，您应该将所有提交推送到从官方IvorySQL派生的分支的您自己的存储库中，并向我们发送请求。

补丁审查

假定提交的拉取请求通过验证检查，可供同行审查。同行审查是确保对IvorySQL的贡献具有高质量并与路线图和社区期望保持一致的过程。我们鼓励IvorySQL社区的每个成员审查请求并提供反馈。由于您不必成为核心团队成员就可以做到这一点，因此我们建议您向有兴趣成为IvorySQL长期贡献者的任何人提供一系列拉动

式评论。

同行评审的一个结果可能是达成共识，即您需要以某些方式修改pull请求。GitHub允许您将其他提交推送到从中发送请求的分支中。这些额外的提交将对所有审阅者可见。

当同行评议收到参与者至少+1张+1和no-1张的选票时，同行评议会趋于一致。在这一点上，您应该期望核心团队成员之一将您的更改引入到项目中。

在补丁审查期间的任何时候，您都可能会因审查人员和核心团队成员的工作效率而遇到延迟。请耐心等待，也不要气馁。如果您在几天内没有收到预期的反馈，请添加一条评论，要求更新pull请求本身，或者向邮件列表发送一封电子邮件。

直接提交到存储库

有时，您会看到核心团队成员直接提交到存储库，而无需执行pull请求工作流。这仅适用于小的更改，我们使用的经验法则是：如果更改涉及任何可能导致测试失败的功能，那么它必须通过pull请求工作流。另一方面，如果更改发生在代码库的非功能部分（例如在注释块中修复打字错误），则核心团队成员可以决定直接提交到存储库。